Citrus

Authors: The Citrus Community

Version 5.1.0, 2026-10-19

citrus

1. Preface . . . 2
2. Introduction . . . 3
   2.1. Overview . . . 3
   2.2. Usage scenarios . . . 3
3. Setup . . . 5
   3.1. Maven . . . 5
   3.2. Gradle . . . 8
4. Runtimes . . . 10
   4.1. TestNG . . . 12
   4.2. JUnit Jupiter . . . 17
   4.3. JUnit4 . . . 22
   4.4. QuarkusTest . . . 25
   4.5. Cucumber . . . 35
   4.6. Main CLI . . . 46
   4.7. JBang . . . 47
   4.8. Sharding Test Cases . . . 54
5. Run tests . . . 56
   5.1. Java . . . 56
   5.2. Test source loader . . . 96
   5.3. XML . . . 99
   5.4. YAML . . . 108
   5.5. Groovy . . . 117
   5.6. Spring Bean XML . . . 128
6. Test variables . . . 139
   6.1. Global variables . . . 139
   6.2. Extract variables . . . 142
   6.3. Object graphs and the scenario container . . . 145
   6.4. Path expressions . . . 149
   6.5. Escape variables . . . 153
7. Message validation . . . 154
   7.1. Validation registry . . . 154
   7.2. Validation modules . . . 156
   7.3. Json validation . . . 157
   7.4. XML validation . . . 170
   7.5. Schema validation . . . 200
   7.6. Plain text validation . . . 205
   7.7. Binary validation . . . 211
   7.8. Hamcrest validation . . . 214

7.9. AssertJ validation . . . 215
7.10. Custom validation . . . 226
8. Test actions . . . 231
8.1. Send . . . 231
8.2. Receive . . . 248
8.3. SQL . . . 277
8.4. Sleep . . . 292
8.5. Delay . . . 294
8.6. Java . . . 295
8.7. Expect timeout . . . 298
8.8. Echo . . . 301
8.9. Print . . . 302
8.10. Stop time . . . 304
8.11. Create variables . . . 309
8.12. Create endpoints . . . 311
8.13. Trace variables . . . 316
8.14. Transform . . . 318
8.15. Groovy script execution . . . 324
8.16. Failing the test . . . 332
8.17. Input . . . 333
8.18. Load Properties . . . 336
8.19. Purging JMS destinations . . . 337
8.20. Purging Spring Message Channels . . . 342
8.21. Purging endpoints . . . 347
8.22. Assert failure . . . 352
8.23. Catch exceptions . . . 353
8.24. Apache Ant build . . . 355
8.25. Start/Stop server . . . 360
8.26. Stop Timer . . . 365
8.27. Custom action . . . 368
9. Containers . . . 372
9.1. Sequential . . . 372
9.2. Conditional . . . 373
9.3. Parallel . . . 377
9.4. Iterate . . . 383
9.5. Repeat . . . 389
9.6. Repeat on error . . . 395
9.7. Timer . . . 402
9.8. Async . . . 407
9.9. Wait . . . 414
9.10. Custom containers . . . 419

10. Endpoints . . . 423
10.1. Send messages . . . 425
10.2. Receive messages . . . 431
10.3. Local message store . . . 435
11. Direct endpoint . . . 440
11.1. Channel endpoint . . . 440
11.2. Synchronous direct endpoints . . . 442
11.3. Message selectors . . . 445
11.4. Payload matching selector . . . 446
11.5. Root QName selector . . . 447
11.6. Xpath selector . . . 449
11.7. JsonPath selector . . . 450
12. JMS support . . . 452
12.1. JMS endpoints . . . 452
12.2. JMS synchronous endpoints . . . 456
12.3. JMS topics . . . 460
12.4. JMS topic durable subscription . . . 462
12.5. JMS topic purging . . . 465
12.6. JMS message headers . . . 465
12.7. Dynamic destination names . . . 466
12.8. SOAP over JMS . . . 467
13. Apache Kafka Support . . . 468
13.1. Kafka Endpoint . . . 469
13.2. Kafka Synchronous Endpoints . . . 474
13.3. Kafka Message Headers . . . 474
13.4. Kafka Message . . . 476
13.5. Kafka Message Selector . . . 477
13.6. Dynamic Kafka Endpoints . . . 483
13.7. Embedded Kafka Server . . . 484
13.8. Kafka Settings . . . 485
14. Http REST support . . . 486
14.1. Http REST client . . . 486
14.2. Http client interceptors . . . 495
14.3. Http REST server . . . 496
14.4. Http headers . . . 500
14.5. Http query parameter . . . 504
14.6. Http server interceptors . . . 506
14.7. Http form urlencoded data . . . 507
14.8. Http error handling . . . 510
14.9. Http client basic authentication . . . 512
14.10. Http server basic authentication . . . 515

14.11. Http client SSL . . . 517
14.12. Http server SSL . . . 519
14.13. Http cookies . . . 520
14.14. Http Gzip compression . . . 525
14.15. Http servlet filters . . . 527
14.16. Http servlet context customization . . . 529
15. GraphQL support . . . 533
15.1. GraphQL client and server . . . 533
15.2. Calling a GraphQL API . . . 535
15.3. GraphQL errors . . . 537
15.4. Simulating a GraphQL API . . . 538
15.5. Schema validation . . . 540
15.6. Strict checks . . . 540
15.7. GraphQL message headers . . . 541
16. SOAP WebServices . . . 542
16.1. SOAP client . . . 542
16.2. SOAP client interceptors . . . 548
16.3. SOAP server . . . 550
16.4. SOAP send and receive . . . 555
16.5. SOAP headers . . . 558
16.6. SOAP HTTP mime headers . . . 569
16.7. SOAP Envelope handling . . . 574
16.8. SOAP server interceptors . . . 578
16.9. SOAP 1.2 . . . 582
16.10. SOAP faults . . . 585
16.11. Send SOAP faults . . . 585
16.12. Receive SOAP faults . . . 591
16.13. Multiple SOAP fault details . . . 610
16.14. Send HTTP error codes with SOAP . . . 619
16.15. SOAP attachment support . . . 621
16.16. Send SOAP attachments . . . 621
16.17. Receive SOAP attachments . . . 624
16.18. SOAP MTOM support . . . 627
16.19. SOAP client basic authentication . . . 638
16.20. SOAP server basic authentication . . . 641
16.21. WS-Addressing support . . . 644
16.22. SOAP client fork mode . . . 649
16.23. SOAP servlet context customization . . . 653
17. Apache Camel support . . . 657
17.1. Camel endpoint . . . 658
17.2. Synchronous Camel endpoint . . . 661

17.3. Camel exchange headers . . . 663
17.4. Camel exception handling . . . 664
17.5. Camel context handling . . . 667
17.6. Camel route actions . . . 669
17.7. Camel route statistics verification . . . 674
17.8. Camel route statistics reset . . . 676
17.9. Camel controlbus actions . . . 678
17.10. Camel CLI support . . . 681
17.11. Camel CLI Kubernetes support . . . 700
17.12. Camel infrastructure services . . . 705
17.13. Camel endpoint DSL . . . 711
17.14. Camel processor support . . . 712
17.15. Camel data format support . . . 715
18. Message channel support . . . 717
18.1. Channel endpoint . . . 717
18.2. Synchronous channel endpoints . . . 719
18.3. Message selectors . . . 722
18.4. Payload matching selector . . . 723
18.5. Root QName selector . . . 724
18.6. Xpath selector . . . 726
18.7. JsonPath selector . . . 728
19. WebSocket support . . . 729
19.1. WebSocket client . . . 729
19.2. WebSocket server endpoints . . . 732
19.3. WebSocket headers . . . 737
20. Mail support . . . 743
20.1. Mail client . . . 743
20.2. Mail server . . . 752
21. FTP support . . . 767
21.1. FTP client . . . 767
21.2. FTP server . . . 784
22. SFTP/SCP support . . . 800
22.1. SFTP client . . . 800
22.2. SFTP server . . . 818
22.3. SCP client . . . 832
23. File support . . . 842
23.1. Write files . . . 842
23.2. Read files . . . 844
24. Selenium support . . . 848
24.1. Selenium browser . . . 848
24.2. Selenium actions . . . 853

24.3. Start/stop browser . . . 859
24.4. Find . . . 861
24.5. Verify . . . 865
24.6. Extract . . . 869
24.7. Click . . . 872
24.8. Hover . . . 874
24.9. Double click . . . 875
24.10. Right click . . . 876
24.11. Drag and drop . . . 876
24.12. Press keys . . . 877
24.13. Form input actions . . . 879
24.14. Fill form . . . 883
24.15. Page actions . . . 887
24.16. Page validation . . . 890
24.17. Wait . . . 893
24.18. Navigate . . . 896
24.19. Window actions . . . 898
24.20. Frame . . . 900
24.21. Alert . . . 902
24.22. Make screenshot . . . 903
24.23. Temporary storage (Firefox) . . . 905
24.24. Javascript . . . 906
24.25. Cookies . . . 908
24.26. Clear browser cache . . . 911
25. Playwright support . . . 913
25.1. Playwright browser . . . 913
25.2. Browser annotation config . . . 914
25.3. Playwright actions . . . 914
25.4. Browser engines . . . 915
25.5. Interaction actions . . . 915
25.6. Frames . . . 916
25.7. Verify and extract . . . 916
25.8. JavaScript and screenshots . . . 917
25.9. Drag and drop . . . 917
25.10. Tracing, HAR and screencast . . . 918
25.11. Storage state . . . 919
25.12. Dialogs . . . 919
25.13. Console messages, page errors and response details . . . 919
25.14. HTTP credentials and passkeys . . . 920
25.15. API requests with the browser session . . . 921
25.16. XML and YAML DSL . . . 929

25.17. Failure evidence . . . 934
25.18. Browser runtimes . . . 934
26. Vert.x event bus support . . . 936
26.1. Vert.x endpoint . . . 936
26.2. Synchronous Vert.x endpoint . . . 941
26.3. Vert.x instance factory . . . 946
27. JDBC support . . . 949
27.1. The Citrus-JDBC-Driver . . . 949
27.2. The Citrus-JDBC-Server . . . 950
27.3. JdbcMessage . . . 956
28. Docker support . . . 961
28.1. Docker client . . . 961
28.2. Docker commands . . . 964
29. SSH support . . . 970
29.1. SSH Client . . . 970
29.2. SSH Server . . . 974
30. RMI support . . . 978
30.1. RMI client . . . 978
30.2. RMI server . . . 987
31. JMX support . . . 995
31.1. JMX client . . . 995
31.2. JMX server . . . 1005
32. Zookeeper support . . . 1023
32.1. Zookeeper client . . . 1023
32.2. Zookeeper commands . . . 1024
33. Dynamic endpoint components . . . 1031
34. Endpoint adapter . . . 1037
34.1. Empty response endpoint adapter . . . 1038
34.2. Static response endpoint adapter . . . 1038
34.3. Request dispatching endpoint adapter . . . 1042
34.4. Channel endpoint adapter . . . 1045
34.5. JMS endpoint adapter . . . 1046
35. Connectors . . . 1049
36. OpenAPI support . . . 1050
36.1. OpenAPI settings . . . 1050
36.2. OpenAPI specification . . . 1051
36.3. OpenAPI client . . . 1051
36.4. OpenAPI server . . . 1056
37. JBang support . . . 1059
37.1. JBang action . . . 1059
38. Kubernetes support . . . 1061

38.1. Kubernetes client . . . 1061
38.2. Kubernetes actions . . . 1063
38.3. Create Labels . . . 1063
38.4. Create Annotations . . . 1064
38.5. Create Service . . . 1066
38.6. Connect with Service . . . 1067
38.7. Disconnect from Service . . . 1071
38.8. Create ConfigMap . . . 1072
38.9. Create Secret . . . 1074
38.10. Create Resources . . . 1076
38.11. Delete Resources . . . 1077
38.12. Create Custom Resource . . . 1079
38.13. Delete Custom Resource . . . 1081
38.14. Verify Custom Resource . . . 1082
38.15. Verify Pod Status . . . 1084
38.16. Verify Pod Logs . . . 1085
38.17. Kubernetes Citrus agent . . . 1086
38.18. Spring bean config . . . 1090
38.19. Kubernetes commands . . . 1093
38.20. Kubernetes command bus . . . 1104
39. Knative support . . . 1108
39.1. Knative client . . . 1108
39.2. Knative broker . . . 1109
39.3. Verify broker status . . . 1110
39.4. Send events . . . 1112
39.5. Receive events . . . 1115
39.6. Knative channels . . . 1121
39.7. Knative subscription . . . 1121
40. Testcontainers support . . . 1122
40.1. Start and stop Testcontainers . . . 1122
40.2. PostgreSQL . . . 1126
40.3. MongoDB . . . 1128
40.4. LocalStack . . . 1129
40.5. Kafka . . . 1136
40.6. Redpanda . . . 1141
40.7. Docker compose . . . 1142
41. Functions . . . 1146
41.1. concat() . . . 1147
41.2. substring() . . . 1150
41.3. stringLength() . . . 1151
41.4. translate() . . . 1151

41.5. substringBefore() 1152
41.6. substringAfter() 1152
41.7. round() 1153
41.8. floor() 1153
41.9. ceiling() 1153
41.10. randomNumber() 1153
41.11. randomNumberGenerator() 1154
41.12. randomString() 1155
41.13. randomPattern() 1156
41.14. randomEnumValue() 1157
41.15. currentDate() 1159
41.16. upperCase() 1159
41.17. lowerCase() 1160
41.18. average() 1160
41.19. minimum() 1160
41.20. maximum() 1160
41.21. sum() 1160
41.22. absolute() 1161
41.23. mapValue() 1161
41.24. randomUUID() 1163
41.25. encodeBase64() 1163
41.26. decodeBase64() 1163
41.27. escapeXml() 1164
41.28. escapeJson() 1164
41.29. cdataSection() 1164
41.30. digestAuthHeader() 1165
41.31. localHostAddress() 1165
41.32. changeDate() 1165
41.33. readFile() 1166
41.34. message() 1167
41.35. xpath() 1169
41.36. jsonPath() 1174
41.37. jsonPatch() 1174
41.38. urlEncode()/urlDecode() 1176
41.39. systemProperty() 1176
41.40. env() 1177
41.41. unixTimestamp() 1177
42. Validation Matchers 1178
42.1. ignore() 1180
42.2. matchesXml() 1181
42.3. equalsIgnoreCase() 1182

42.4. contains() . . . 1182
42.5. startsWith() . . . 1183
42.6. endsWith() . . . 1183
42.7. matches() . . . 1183
42.8. matchesDatePattern() . . . 1183
42.9. isNumber() . . . 1183
42.10. lowerThan() . . . 1184
42.11. greaterThan() . . . 1184
42.12. greaterThanOrEqualTo() . . . 1184
42.13. lowerThanOrEqualTo() . . . 1184
42.14. isWeekday() . . . 1184
42.15. variable() . . . 1185
42.16. dateRange() . . . 1185
42.17. assertThat() . . . 1185
42.18. ignoreNewLine() . . . 1186
42.19. trim() . . . 1187
42.20. trimAllWhitespaces() . . . 1187
42.21. isUUIDv4() . . . 1188
42.22. allOf() . . . 1188
42.23. anyOf() . . . 1189
42.24. not() . . . 1190
42.25. Nesting combinators . . . 1191
43. Citrus agent . . . 1193
43.1. Agent REST service API . . . 1193
43.2. Configuration . . . 1195
43.3. Web UI . . . 1197
43.4. Docker container image . . . 1197
43.5. Agent JBang commands . . . 1201
43.6. Maven plugin . . . 1201
43.7. Message processors . . . 1203
44. Data dictionaries . . . 1205
44.1. XML data dictionaries . . . 1205
44.2. JSON data dictionaries . . . 1209
44.3. Dictionary scopes . . . 1212
44.4. Path mapping strategies . . . 1214
45. Test actors . . . 1217
45.1. Create test actors . . . 1217
45.2. Reference test actors . . . 1219
45.3. Disable test actors . . . 1221
45.4. Test actor environment settings . . . 1223
46. Test suite actions . . . 1224

46.1. Before suite . . . 1224
46.2. After suite . . . 1230
46.3. Before test . . . 1232
46.4. After test . . . 1238
47. Customize meta information . . . 1240
48. Reporting and Test Results . . . 1242
48.1. General report settings . . . 1242
48.2. Console logging . . . 1243
48.3. JUnit reports . . . 1244
48.4. HTML reports . . . 1244
49. Configuration options . . . 1248
49.1. Environment settings . . . 1248
49.2. Log settings . . . 1252
49.3. Property binding support . . . 1254
49.4. Spring configuration settings . . . 1256
49.5. Property file settings . . . 1256
50. Spring support . . . 1258
50.1. Spring XML application context . . . 1258
50.2. Spring Java config . . . 1259
51. Service Provider Interface (SPI) . . . 1261
51.1. Resource path lookup . . . 1261
51.2. Java ServiceLoader fallback . . . 1262
52. Tools . . . 1265
53. Cucumber Steps . . . 1266
53.1. Standard steps . . . 1266
53.2. Apache Camel steps . . . 1269
53.3. Groovy steps . . . 1275
53.4. Http steps . . . 1281
53.5. JDBC steps . . . 1298
53.6. JMS steps . . . 1301
53.7. Kafka steps . . . 1307
53.8. Kubernetes steps . . . 1313
53.9. Knative steps . . . 1325
53.10. OpenAPI steps . . . 1335
53.11. Playwright steps . . . 1343
53.12. Selenium steps . . . 1345
53.13. Testcontainers steps . . . 1356
54. Kaoto Visual Designer . . . 1358
54.1. Overview . . . 1358
54.2. Getting Started . . . 1358
54.3. Creating Tests . . . 1359

54.4. Component Catalog . . . 1359
54.5. Schema-Based Property Forms . . . 1362
54.6. Running Tests . . . 1362
54.7. Kaoto Example . . . 1363
54.8. Resources . . . 1365
54.9. Overview . . . 1366
54.10. What is MCP? . . . 1366
54.11. Installation . . . 1367
54.12. Running the MCP Server . . . 1369
54.13. Tools . . . 1370
54.14. Resources . . . 1373
54.15. Server Configuration . . . 1376
54.16. Use Cases . . . 1376
54.17. Spring Restdocs using Http . . . 1378
54.18. Spring Restdocs using SOAP . . . 1384
55. Tracing messages . . . 1388
56. Samples . . . 1391
56.1. The FlightBooking sample . . . 1391
57. Appendix . . . 1401
Maven archetype . . . 1401

Version: 5.1.0



Chapter 1. Preface

Integration tests are a critical part of software testing. In contrast to unit tests where the primary goal is to verify a single class or method in isolation to other parts of the software the integration tests look at a wider scope with several components and software parts interacting with each other.

Integration tests often rely on infrastructure components such as I/O, databases, 3rd party services and so on. In combination with messaging and multiple message transports as client and/or server the automated tests become a tough challenge. Testers need sufficient tool support to master this challenge for automated integration test. Citrus as an Open Source framework is here to help you master this challenge.

In a typical enterprise application and middleware scenario automated integration testing of message-based interfaces is exhausting and sometimes barely possible. Usually the tester is forced to simulate several interface partners in an end-to-end integration test.

The first thing that comes to one's mind is manual testing. No doubt manual testing is fast. In a long term perspective manual testing is time-consuming and causes severe problems regarding maintainability as they are error prone and not repeatable.

Citrus provides a complete test automation tool for integration testing of message-based enterprise applications. You can test your message interfaces (Http REST, SOAP, JMS, Kafka, TCP/IP, FTP, ...) to other applications as client and server. Every time a code change is made all automated Citrus tests ensure the stability of software interfaces and its message communication.

Regression testing and continuous integration is very easy as Citrus fits into your build lifecycle (Maven or Gradle) as usual Java unit test (JUnit, TestNG, Cucumber).

With powerful validation capabilities for various message formats like XML, CSV or JSON Citrus is ready to provide fully automated integration tests for end-to-end use cases. Citrus effectively composes complex messaging use cases with response generation, error simulation, database interaction and more.

This documentation provides a reference guide to all features of the Citrus test framework. It gives a detailed picture of effective integration testing with automated integration test environments. Since this document is open, please do not hesitate to give feedback in form of comments, change requests, fixes and pull requests. We are more than happy to continuously improve this documentation with your help!

Chapter 2. Introduction

Citrus provides automated integration tests for message-based enterprise applications. The framework is Open Source and supports various messaging transports (Http REST, SOAP, JMS, Kafka, TCP/IP, FTP, …) and data formats (XML, Json, plaintext, binary).

The Citrus tests use well-known unit test frameworks (JUnit, TestNG, Cucumber) for execution and integrates with build tools like Maven or Gradle. In addition, Citrus leverages standard libraries like Quarkus, the Spring Framework or Apache Camel.

It is also good to know that Citrus is written purely in Java. Thus, it is also fully compatible with Kotlin.

2.1. Overview

Citrus supports simulating interface partners across different messaging transports. You can easily produce and consume messages with a wide range of protocols like HTTP, JMS, TCP/IP, FTP, SMTP and more. The framework is able to act both as a client and server. In each communication step Citrus is able to validate message contents towards syntax and semantics.

In addition to that the Citrus framework offers a wide range of test actions to take control of the process flow during a test (e.g. iterations, system availability checks, database connectivity, parallelism, delays, error simulation, scripting and many more).

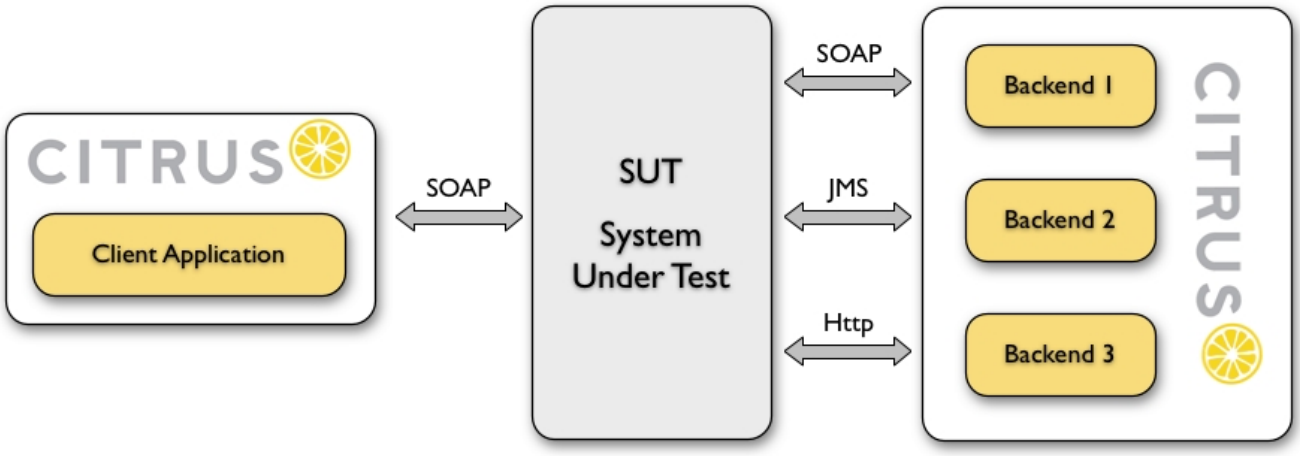
The test is able to describe a whole use case scenario including several interface partners that exchange many messages with each other. The composition of complex message flows in a single test case with several test steps is one of the major features in Citrus.

You can choose how to describe the test case definition either with pure XML or a Java domain specific language. The tests can be executed multiple times as automated integration test.

With JUnit and TestNG integration Citrus can easily be integrated into your build lifecycle process (Maven or Gradle). During a test Citrus simulates all surrounding interface partners (client or server) without any coding effort. With easy definition of expected message content (header and body) for XML, CSV, SOAP, JSON or plaintext messages Citrus is able to validate the incoming data towards syntax and semantics.

2.2. Usage scenarios

Citrus should help you whenever it comes to verify a message-based software with its interfaces to other components and partners using automated integration tests. Every project that interacts with other components over messaging transports needs to simulate these interface partners on the client or server side in a test scenario. Citrus is here to help you master these test automation tasks.



This test set up is typical for a Citrus use case. In such a test scenario we have a system under test (SUT) with several messaging interfaces to other applications. A client application invokes services on the SUT and triggers business logic. The SUT is linked to several backend applications over various messaging transports (here SOAP, JMS, and Http). As part of the business logic one or more of these backend services is called and interim message notifications and responses are sent back to the client application.

This generates a bunch of messages that are exchanged throughout the components involved.

In the automated integration test Citrus needs to send and receive those messages over different transports. Citrus takes care of all interface partners (ClientApplication, Backend1, Backend2, Backend3) and simulates their behavior by sending proper response messages in order to keep the message flow alive.

Each communication step comes with message validation and comparison against an expected message template (e.g. XML or JSON data). In addition to messaging steps Citrus is also able to perform arbitrary other test actions (e.g. perform a database query between requests).

In fact a Citrus test case is nothing but a normal JUnit or TestNG test case. This makes it very straight forward to run the tests from your favorite Java IDE (Eclipse, IntelliJ, VSCode, ...) and as part of your software build process (Maven or Gradle). The Citrus tests become repeatable and give you fully automated integration tests to ensure software quality and interface stability.

The following reference guide walks through all Citrus capabilities and shows how to have a great integration test experience.

Chapter 3. Setup

This chapter discusses how to get started with Citrus. It deals with the installation and set up of the framework, so you are ready to start writing test cases after reading this chapter.

Usually you add Citrus as a test-scoped dependency library in your project. Build tools like Maven or Gradle provide standard integration for test libraries such as Citrus. As Citrus tests are nothing but normal unit tests (JUnit, TestNG, Cucumber) you can run the tests with the standard unit test build integration (e.g. via maven-failsafe plugin).

This chapter describes the Citrus project setup possibilities, choose one of them that fits best to include Citrus into your project.

3.1. Maven

Citrus uses Maven internally as a project build tool and provides extended support for Maven projects. Maven will ease up your life as it manages project dependencies and provides extended build life cycles and conventions for compiling, testing, packaging and installing your Java project.

In case you already use Maven in your project you can just add Citrus as a test-scoped dependency.

As Maven handles all project dependencies automatically you do not need to download any Citrus project artifacts in advance. If you are new to Maven please refer to the official Maven documentation and find out how to set up a Maven project.

Assuming you have a proper Maven project setup you can integrate Citrus with it. Just add the Citrus project dependencies in your Maven pom.xml as a dependency like follows.

- We add Citrus as test-scoped project dependency to the project POM (pom.xml)

Add Citrus base dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-base</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

- The dependency above adds the base functionality of Citrus. You need to add modules as you require them.

Add modules as required in your project. For instance Http support

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-http</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

Choose test runtime (JUnit, TestNG, Cucumber) that is used to run the tests.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-testng</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

- Citrus integrates nicely with the Spring framework. In case you want to use the Spring dependency injection and bean configuration capabilities just add the Spring support in Citrus.

Add Spring support

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-spring</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

- Also, Citrus provides a Maven plugin that you can add. The plugin provides some convenience functionalities such as creating new tests from command line.

Add Citrus Maven plugin

```
<plugin>
  <groupId>org.citrusframework.mvn</groupId>
  <artifactId>citrus-maven-plugin</artifactId>
  <version>${citrus.version}</version>
  <configuration>
    <author>Donald Duck</author>
    <targetPackage>org.citrusframework</targetPackage>
  </configuration>
</plugin>
```

The Maven project is now ready to use Citrus. You can start writing new test cases with the Citrus Maven plugin:

Create new test

```
mvn citrus:create-test
```

The command above starts an interactive command line interface that helps you to create a test.

Once you have written the Citrus test cases you can execute them automatically in your Maven software build lifecycle. The tests will be included into your project's integration-test phase using the Maven failsafe plugin. Here is a sample failsafe configuration for Citrus.

Maven failsafe plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>${maven.failsafe.version}</version>
  <executions>
    <execution>
      <id>integration-tests</id>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

The Citrus test sources go to the default Maven test source directory `src/test/java` and `src/test/resources`.

You are now ready to call the usual Maven **verify** goal (`mvn verify`) in order to build your project and run the tests. The Citrus integration tests are executed automatically during the build process.

Run all tests with Maven

```
mvn verify
```

Run single test by its name

```
mvn verify -Dit.test=MyFirstCitrusIT
```



The Maven failsafe plugin by default executes tests with specific name pattern. This is because integration tests should not execute in Maven unit test phase, too. Your integration tests should follow the failsafe name pattern with each test name beginning or ending with **'IT'**.



If you need additional assistance in setting up a Citrus Maven project please visit our Maven setup tutorial on https://citrusframework.org/tutorials.html.

3.2. Gradle

As Citrus tests are nothing but normal JUnit or TestNG tests the integration to Gradle as build tool is as easy as adding the source files to a folder in your project. With the Gradle task execution for integration tests you are able to execute the Citrus tests like you would do with normal unit tests.

The Gradle build configuration goes to the **build.gradle** and **settings.gradle** files. The files define the project name and the project version.

Gradle project configuration

```
rootProject.name = 'citrus-sample-gradle'
group 'org.citrusframework.samples'
version '${citrus.version}'
```

The Citrus libraries are available on Maven central repository. This means you should add this repository so Gradle knows how to download the required Citrus artifacts.

Add Maven central repository

```
repositories {
    mavenCentral()
}
```

Citrus stable release versions are available on Maven central. Now let's move on with adding the Citrus libraries to the project.

Add Citrus test scoped dependencies

```
dependencies {
    testCompile group: 'org.citrusframework', name: 'citrus-base', version:
'${citrus.version}'
    testCompile group: 'org.citrusframework', name: 'citrus-http', version:
'${citrus.version}'
    testCompile group: 'org.testng', name: 'testng', version: '${testng.version}'
    [...]
}
```

Citrus provides various modules that encapsulate different functionalities. The `citrus-base` module is the basis and holds core functionality. In addition, you may add further modules that match your project needs (e.g. add Http support with `citrus-http`).

As a runtime the project chose to use TestNG. You can also use JUnit or Cucumber as a test runtime. Each of those frameworks integrates seamlessly with the Gradle build.

Choose test runtime provider

```
test {
    useTestNG()
}
```

Of course JUnit is also supported. This completes the Gradle build configuration settings. You can move on to writing some Citrus integration tests and add those to **src/test/java** directory.

You can use the Gradle wrapper for compile, package and test the sample with Gradle build command line.

Run the build with Gradle

```
gradlew clean build
```

This executes all Citrus test cases during the build. You will be able to see Citrus performing some integration test logging output.

If you just want to execute all tests you can call:

Run all tests

```
gradlew clean check
```

Of course, you can also run the Citrus tests from your favorite Java IDE. Just start the Citrus test as a normal unit test using the Gradle integration in IntelliJ, Eclipse or VSCode.

Chapter 4. Runtimes

The Citrus test case is nothing but a normal Java unit test. Citrus integrates with standard unit test frameworks such as JUnit, TestNG or Cucumber as a runtime.

In addition to that Citrus also works with `@QuarkusTest` and `@SpringBootTest` annotated classes.

Most Java developers are familiar with at least one of the standard tools. Everything you can do with JUnit or TestNG you can do with Citrus tests, too. For instance, you can integrate the Citrus tests in a Maven build, run and debug Citrus from your favorite Java IDE or include Citrus tests into a continuous build pipeline.



Why is Citrus integrating with unit test frameworks such as JUnit, although Citrus represents a framework for integration testing? The answer to this question is quite simple: Citrus benefits a lot from standard libraries such as JUnit and TestNG for Java test execution. The standard unit testing frameworks provide great tooling support and are widely supported by other components (e.g. continuous build tools, Maven build lifecycle, development IDEs).

Each runtime in Citrus supports polyglot test writing. You can choose from a set of supported programming languages and domain specific languages (DSL) to write your tests. Currently supported languages in Citrus are:

- Java
- XML
- YAML
- Cucumber (Gherkin)
- Groovy
- Spring XML

The Citrus Java DSL represents the most powerful way to write tests. Citrus provides a set of classes and methods to leverage the test code in form of a fluent API. Users can simply configure a test action with the different options using a fluent builder pattern style DSL.

In addition to that you can also write Citrus tests with XML, Groovy, YAML, Cucumber (Gherkin), Spring XML declaration files that get loaded as part of the test runtime. These test sources represent a more declarative approach of writing tests in Citrus. You do not necessarily need to know how to code Java when using one of these languages, but you may be limited in terms of customization opportunities.

See the following example to explore the different test source languages in Citrus.

Java

```
@CitrusTest
public void helloWorldTest() {
    variable("user", "Citrus");

    $(echo().message("Hello from ${user}!"));
}
```

XML

```
<test name="HelloWorldTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
      <variable name="user" value="Citrus"/>
    </variables>
    <actions>
      <echo message="Hello from ${user}!"/>
    </actions>
</test>
```

YAML

```
name: HelloWorldTest
variables:
  - name: user
    value: Citrus
actions:
  - echo:
      message: "Hello from ${user}!"
```

Groovy

```
name "HelloWorldTest"

variables {
   user="Citrus"
}

actions {
    $(echo().message('Hello from ${user}!'))
}
```

Cucumber

```
Feature: HelloWorldTest

  Background:
    Given variables
      | user | Citrus |

  Scenario: Should Say Hello
    Then print "Hello from ${user}!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="HelloWorldTest">
    <variables>
      <variable name="user" value="Citrus"/>
    </variables>
    <actions>
      <echo>
        <message>Hello from ${user}!</message>
      </echo>
    </actions>
  </testcase>
</spring:beans>
```

The sample above is a very simple Citrus test that creates a test variable and prints a message to the console. You can see the different programming language and data formats that you can choose to write the test. Citrus provides lots of different test actions that are ready to solve also complex test scenarios and challenges.

Each runtime supports the polyglot testing languages Java, XML, YAML, Groovy, Cucumber. This documentation tries to provide proper sample code for all supported test languages.

The following sections have a closer look at the different runtimes for Citrus.

4.1. TestNG

TestNG stands for next generation testing and has had a great influence in adding Java annotations to the unit test community. Citrus is able to define tests as executable TestNG Java classes.



The TestNG support is shipped in a separate Maven module. You need to include the module as a dependency in your project.

TestNG module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-testng</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

4.1.1. TestNG tests

See the following sample showing how to write a Citrus test on top of TestNG:

TestNG Citrus test

```
package org.citrusframework.samples;

import org.testng.annotations.Test;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

@Test
public class Simple_IT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusTest(name = "Simple_IT")
    public void simpleTest() {
        description("First example showing the basic Java DSL!");

        given(
            variable("user", "Citrus")
        );

        then(
            echo().message("Hello ${user}!"
        ));
    }
}
```

If you are familiar with TestNG you will see that the Java class is a normal TestNG test class using the usual `@Test` annotation. For convenience reasons you can extend a basic Citrus TestNG base class `TestNGCitrusSupport` which enables the Citrus test execution as well as the Java DSL features for us.



You can also combine Citrus with the Spring framework and its dependency injection and IoC capabilities. In order to enable Spring support in Citrus add the `citrus-spring` module to your project and extend `TestNGCitrusSpringSupport` as a base class. With the Spring support in Citrus the test is able to use `@Autowired` annotations for injecting Spring beans into the test class and you can define the Spring application context with `@Configuration` annotations for instance.

In addition, the test methods use the **@CitrusTest** annotation which allows setting properties such as test names and packages.

The Citrus test logic goes directly as the method body with using the Citrus Java domain specific language features. As you can see the Java DSL is able to follow BDD (Behavior Drive Design) principles with Given-When-Then syntax. As an alternative to that you can just use `run()` for all test actions.

Pure test action DSL

```
@CitrusTest(name = "Simple_IT")
public void simpleTest() {
    description("First example showing the basic Java DSL!");

    run(variable("user", "Citrus"));

    run(
        echo().message("Hello ${user}!"
    ));
}
```

The great news is that you can still use the awesome TestNG features in with the Citrus test class (e.g. parallel test execution, test groups, setup and tear down operations and so on). Just to give an example we can simply add a test group to our test like this:

Set test groups

```
@Test(groups = {"long-running"})
public void longRunningTest() {
    ...
}
```

For more information on TestNG please visit the official TestNG website, where you find a complete reference documentation. The following sections deal with a subset of these TestNG features in particular.

4.1.2. Use TestNG data providers

TestNG as a framework comes with lots of great features such as data providers. Data providers execute a test case several times with different test data. Each test execution works with a specific parameter value. You can use data provider parameter values as test variables in Citrus. See the next listing on how to use TestNG data providers in Citrus:

TestNG Citrus data provider test

```
public class DataProviderIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusTest
    @CitrusParameters({"message", "delay"})
    @Test(dataProvider = "messageDataProvider")
    public void dataProvider(String message, Long sleep) {
        run(echo(message));
        run(sleep().milliseconds(sleep));

        run(echo("${message}"));
        run(echo("${delay}"));
    }

    @DataProvider
    public Object[][] messageDataProvider() {
        return new Object[][] {
                { "Hello World!", 300L },
                { "Citrus rocks!", 1000L },
                { "Hi from Citrus!", 500L },
        };
    }
}
```

Above test case method is annotated with TestNG data provider called **messageDataProvider** . In the same class you can write the data provider that returns a list of parameter values. TestNG will execute the test case several times according to the provided parameter list. Each execution is shipped with the respective parameter value.

According to the **@CitrusParameter** annotation the test will have test variables called `message` and `delay`.

With Sharding

TestNG data provider based tests can additionally be run in a distributed manner, facilitating test sharding. The following snipped showcases you how to get started within a Spring test case environment.

Sharded TestNG Citrus data provider test

```
import static org.citrusframework.common.TestLoader.SPRING;
import static org.citrusframework.sharding.Shard.createShard;

import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.CitrusParameters;
import org.citrusframework.testng.spring.TestNGCitrusSpringSupport;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class DataProviderIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusParameters({"message", "delay"})
    @Test(dataProvider = "messageDataProvider")
    @CitrusTestSource(type = SPRING, name = "DataProviderIT")
    public void dataProvider(String message, Long sleep) {
        run(echo(message));
        run(sleep().milliseconds(sleep));
    }

    @DataProvider("messageDataProvider")
    public Object[][] messageDataProvider() {
        return createShard(
            new Object[][] {
                { "Hello World!", 300L },
                { "Citrus rocks!", 1000L },
                { "Hi from Citrus!", 500L },
            }
        );
    }
}
```

All the available configuration options are documented withing the chapter Sharding Test Cases

4.1.3. Run tests in parallel

Integration tests tend to be more time-consuming compared to pure unit tests when it comes to execute tests. This is because integration tests often need to initialize test infrastructure (e.g. test servers, database connections). Running tests in parallel can reduce the overall test suite time a lot.

When running tests in parallel you need to make sure each test operates on its own set of resources. Tests must not share components such as the Citrus Java DSL test action runner or the test context.

You should be using the resource injection to make sure each test operates on its own resources.

Resource injection

```
public class ResourceInjection_IT extends TestNGCitrusSupport implements
TestActionSupport {

    @Test
    @CitrusTest
    public void injectResources(@Optional @CitrusResource TestCaseRunner runner,
                                @Optional @CitrusResource TestContext context) {

        runner.given(
            createVariable("random", "citrus:randomNumber(10)")
        );

        runner.run(
            echo("The random number is: ${random}")
        );
    }
}
```

First of all the method parameters must be annotated with `@Optional` because the values are not injected by TestNG itself but by the Citrus base test class. Finally, the parameter requires the `@CitrusResource` annotation in order to mark the parameter for Citrus resource injection.

Now each method uses its own resource instances which makes sure that parallel test execution can take place without having the risk of side effects on other tests running at the same time. Of course, you also need to make sure that the message exchange in your tests is ready to be performed in parallel (e.g. use message selectors).

4.2. JUnit Jupiter

With JUnit version 5 the famous unit test framework offers a new major version. The JUnit platform provides awesome extension points for other frameworks like Citrus to integrate with the unit testing execution.

Citrus provides extensions in order to enable Citrus related dependency injection and parameter resolving in your JUnit test.



The JUnit support is shipped in a separate Maven module. You need to include the module as a dependency in your project.

JUnit module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-junit-jupiter</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

4.2.1. Citrus extension

You can use the Citrus JUnit extension on your test as follows:

JUnit Citrus test

```
package org.citrusframework.samples;

import org.citrusframework.GherkinTestActionRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class Simple_IT implements TestActionSupport {

    @Test
    @CitrusTest(name = "Simple_IT")
    public void simpleTest(@CitrusResource GherkinTestActionRunner runner) {
        runner.description("First example showing the basic Java DSL!");

        runner.given(
            variable("user", "Citrus")
        );

        runner.then(
            echo().message("Hello ${user}!"
        ));
    }
}
```

The class above is using the JUnit `@Test` annotation as a normal unit test would do. The `@CitrusSupport` annotation marks the test to use the Citrus JUnit extension. This enables us to use the `@CitrusTest` annotation on the test and adds support for the parameter injection for the `TestActionRunner`.



You can use the `@CitrusSupport` annotation, or you can use the classic `@ExtendWith(CitrusExtension.class)` annotation to enable the Citrus support for JUnit.

The Citrus Java DSL runner is the entrance to the Java fluent API provided by Citrus. The sample above uses the Gherkin test runner variation for leveraging the BDD (Behavior Driven Development) style Given-When-Then syntax.

The test class implements the interface `TestActionSupport` which holds all available test actions in Citrus. You can choose from a wide range of actions. The provided test action API uses fluent style build pattern to define the test action and its properties.



The `TestActionSupport` interface brings quite a lot of methods. In case you are facing issues with one of these methods colliding with your own test methods you can choose from a modular interface hierarchy that builds the Citrus Java test actions API. Instead of implementing the whole `TestActionSupport` API you may just implement the `BaseTestActionSupport` API or the very specific `HttpTestActionSupport` interface in your test. Just look for the `*TestActionSupport` interfaces provided by Citrus and include what you need into your test class.



For those of you who are not able to use the `TestActionSupport` or other API interfaces the **DefaultTestActions** class also provides access to the test actions Java DSL API.

JUnit Citrus test

```
package org.citrusframework.samples;

import org.citrusframework.dsl.DefaultTestActions;
import org.citrusframework.GherkinTestActionRunner;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class Simple_IT {

    private final DefaultTestActions actions = new DefaultTestActions();

    @Test
    @CitrusTest(name = "Simple_IT")
    public void simpleTest(@CitrusResource GherkinTestActionRunner runner) {
        runner.description("First example showing the basic Java DSL!");

        runner.given(
            variable("user", "Citrus")
        );

        runner.then(
            actions.echo().message("Hello ${user}!"
        ));
    }
}
```

You can also inject the current `TestContext` in order to get access to the current test variables used by Citrus.



You can also combine Citrus with the Spring framework and its dependency injection and IoC capabilities. In order to enable Spring support in Citrus add the `citrus-spring` module to your project and use the `@ExtendWith(CitrusSpringExtension.class)` annotation. With the Spring support in Citrus the test is able to load components via the Spring application context.

4.2.2. Endpoint injection

In addition to injecting test resources you can also inject endpoints via `@CitrusEndpoint` annotated field injection in your test class. This enables you to inject endpoint components that are defined in the Citrus context configuration.

JUnit Citrus endpoint injection

```
package org.citrusframework.samples;

import org.citrusframework.annotations.*;
import org.citrusframework.GherkinTestActionRunner;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.citrusframework.http.client.HttpClient;
import org.junit.jupiter.api.Test;
import org.springframework.http.HttpStatus;

@CitrusSupport
public class Simple_IT implements TestActionSupport {

    @CitrusEndpoint
    private HttpClient httpClient;

    @Test
    @CitrusTest
    public void test(@CitrusResource GherkinTestActionRunner runner) {
        runner.http().client(httpClient)
                    .send()
                    .get("/hello");

        runner.http().client(httpClient)
                    .receive()
                    .response(HttpStatus.OK.value());
    }
}
```

4.2.3. Citrus Spring extension

Spring is a famous dependency injection framework that also provides support for JUnit. Citrus is able to load its components as Spring beans in an application context. The Citrus JUnit extension works great with the Spring extension.

The Spring extension loads the application context and Citrus adds all components to the Spring

bean configuration.

JUnit Citrus Spring test

```
@CitrusSpringSupport
@ContextConfiguration(classes = CitrusSpringConfig.class)
public class SpringBean_IT {

    @Autowired
    private DirectEndpoint direct;

    @Test
    @CitrusTest
    void springBeanTest(@CitrusResource TestActionRunner actions) {
        actions.$(send().endpoint(direct)
                    .message()
                    .body("Hello from Citrus!"));

        actions.$(receive().endpoint(direct)
                    .message()
                    .body("Hello from Citrus!"));
    }
}
```

The test now uses the `@CitrusSpringSupport` annotation which combines the `@ExtendWith(CitrusSpringExtension.class)` and `@ExtendWith(SpringExtension.class)` annotation. This way the test combines the Spring application context management with the Citrus Java DSL functionality.

You can load Spring beans with `@Autowired` into your test. Also, you can use the `@CitrusResource` annotations to inject the test action runner fluent Java API.



The Spring application context should use the basic `CitrusSpringConfig` configuration class to load all Citrus components as Spring beans. You can customize the Spring application context by adding more configuration classes.

4.2.4. Test Sharding

You can run your JUnit test cases in order in a distributed manner, facilitating test sharding. The following snipped showcases you how to get started within a Spring/XML test case environment. Note that sharding is available with all kind of `Stream`-sources.

Sharded JUnit Spring/XML test

```
import static org.citrusframework.junit.jupiter.CitrusTestFactorySupport.springXml;
import static org.citrusframework.sharding.Shard.createShard;

import org.citrusframework.spring.config.CitrusSpringConfig;
import org.citrusframework.junit.jupiter.spring.CitrusSpringSupport;
import org.citrusframework.junit.jupiter.spring.CitrusSpringXmlTestFactory;
import org.springframework.test.context.ContextConfiguration;

@CitrusSpringSupport
@ContextConfiguration(classes = {CitrusSpringConfig.class})
class SpringBeanXml_IT {

    @CitrusSpringXmlTestFactory
    Stream<DynamicTest> shardedSpringBeanXml_IT() {
        return createShard(
            springXml().packageScan("org.citrusframework.junit.jupiter.simple")
        );
    }
}
```

All the available configuration options are documented withing the chapter Sharding Test Cases

4.3. JUnit4

JUnit4 is still very popular and widely supported by many tools even though there is a new major version with JUnit Jupiter already available. In general Citrus supports both JUnit4 and JUnit Jupiter as test execution framework.



The JUnit4 support is shipped in a separate Maven module. You need to include the module as a dependency in your project.

JUnit4 module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-junit</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

4.3.1. JUnit4 tests

See the following sample test class that uses JUnit4.

JUnit4 Citrus test

```
package org.citrusframework.samples;

import org.junit.Test;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.JUnit4CitrusSupport;

public class Simple_IT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest(name = "Simple_IT")
    public void simpleTest() {
        description("First example showing the basic Java DSL!");

        given(
            variable("user", "Citrus")
        );

        then(
            echo().message("Hello ${user}!"
        ));
    }
}
```

The simple test class above uses the normal `@Test` annotation and extends the base class `JUnit4CitrusSupport`. This is the most convenient way to access the Citrus Java DSL capabilities. As an alternative you may switch to using the `CitrusJUnit4Runner` in your test class.

The fine thing here is that we are still able to use all JUnit features such as before/after hooks or ignoring tests.

After the test run the result is reported exactly like a usual JUnit unit test would do. This also means that you can execute this Citrus JUnit class like every other JUnit test, especially out of any Java IDE, with Maven, with Gradle and so on.



You can also combine Citrus with the Spring framework and its dependency injection and IoC capabilities. In order to enable Spring support in Citrus add the `citrus-spring` module to your project and extend `JUnit4CitrusSpringSupport` as a base class. With the Spring support in Citrus the test is able to use `@Autowired` annotations for injecting Spring beans into the test class. You can define the Spring application context with `@Configuration` annotations as part of the Spring support.

The test class implements the interface `TestActionSupport` which holds all available test actions in Citrus. You can choose from a wide range of actions. The provided test action API uses fluent style build pattern to define the test action and its properties.



The `TestActionSupport` interface brings quite a lot of methods. In case you are facing issues with one of these methods colliding with your own test methods you can choose from a modular interface hierarchy that builds the Citrus Java test actions API. Instead of implementing the whole `TestActionSupport` API you may just implement the `BaseTestActionSupport` API or the very specific `HttpTestActionSupport` interface in your test. Just look for the `*TestActionSupport` interfaces provided by Citrus and include what you need into your test class.



For those of you who are not able to use the `TestActionSupport` or other API interfaces the **TestNGCitrusSupport** and **JUnit4CitrusSupport** base classes also provide a method called `actions()` that provides access to the test actions Java DSL API.

4.3.2. Run tests in parallel

Integration tests tend to be more time-consuming compared to pure unit tests when it comes to execute tests. This is because integration tests often need to initialize test infrastructure (e.g. test servers, database connections). Running tests in parallel can reduce the overall test suite time a lot.

When running tests in parallel you need to make sure each test operates on its own set of resources. Tests must not share components such as the Citrus Java DSL test action runner or the test context.

You should be using the resource injection to make sure each test operates on its own resources.

Resource injection

```
public class ResourceInjection_IT extends JUnit4CitrusSupport implements
TestActionSupport {

    @Test
    @CitrusTest
    public void injectResources(@CitrusResource TestCaseRunner runner,
                                @CitrusResource TestContext context) {

        runner.given(
            createVariable("random", "citrus:randomNumber(10)")
        );

        runner.run(
            echo("The random number is: ${random}")
        );
    }
}
```

The method parameters require the `@CitrusResource` annotations in order to mark the parameter for Citrus resource injection.

Now each method uses its own resource instances which makes sure that parallel test execution can take place without having the risk of side effects on other tests running at the same time. Of

course, you also need to make sure that the message exchange in your tests is ready to be performed in parallel (e.g. use message selectors).

4.4. QuarkusTest

Quarkus has emerged into a popular enterprise Java framework. For unit and integration testing the Quarkus framework provides integrations with JUnit Jupiter. Citrus adds a Quarkus test resource implementation that allows developers to combine Citrus with Quarkus during testing. You can use the Citrus test resource annotations on your Quarkus tests and include Citrus capabilities into arbitrary Quarkus tests.



The Citrus QuarkusTest extension is shipped in a separate Maven module. You need to include the module as a dependency in your project accordingly.

Citrus Quarkus module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-quarkus</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Usually a Quarkus test is annotated with the `@QuarkusTest` or `QuarkusIntegrationTest` annotation. Users just add an annotation named `@CitrusSupport` to also enable Citrus capabilities on the test.

The Citrus support will automatically hook into the QuarkusTest lifecycle management to inject Citrus resources with `@CitrusResource` annotation. Also, the Citrus extension makes sure to start a proper Citrus instance and call before/after suite and before/after test handlers.

This way you are able to combine Citrus with `@QuarkusTest` annotated classes very easily.

Enable Citrus support on QuarkusTest

```
import io.quarkus.test.junit.QuarkusTest;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.quarkus.CitrusSupport;
import org.junit.jupiter.api.Test;

@QuarkusTest
@CitrusSupport
public class DemoApplicationTest implements TestActionSupport {

    @CitrusFramework
    private Citrus citrus;

    @CitrusResource
    private TestCaseRunner t;

    @CitrusResource
    private TestContext context;

    @Test
    void shouldVerifyDemoApp() {
        t.when(
            send()
                .endpoint("messageEndpoint")
                .message()
                .body("How about Citrus!?")
        );

        t.when(
            receive()
                .endpoint("messageEndpoint")
                .message()
                .body("Citrus rocks!")
        );
    }
}
```

The `@CitrusSupport` annotation enables the Citrus features on the test. First of all users may inject Citrus related resources such as `TestCaseRunner` or the `TestContext`.

As usual the `TestCaseRunner` is the entrance to the Citrus domain specific language for running arbitrary Citrus actions as part of the test.

The test class implements the interface `TestActionSupport` which holds all available test actions in Citrus. You can choose from a wide range of actions. The provided test action API uses fluent style build pattern to define the test action and its properties.



The `TestActionSupport` interface brings quite a lot of methods. In case you are facing issues with one of these methods colliding with your own test methods you can choose from a modular interface hierarchy that builds the Citrus Java test actions API. Instead of implementing the whole `TestActionSupport` API you may just implement the `BaseTestActionSupport` API or the very specific `HttpTestActionSupport` interface in your test. Just look for the `*TestActionSupport` interfaces provided by Citrus and include what you need into your test class.

4.4.1. Endpoint configuration

The test is able to configure Message endpoints to connect to different messaging transports as part of the test.

Configure message endpoints

```
@QuarkusTest
@CitrusSupport
public class DemoApplicationTest implements TestActionSupport {

    @BindToRegistry
    private final KafkaEndpoint bookings = new KafkaEndpointBuilder()
            .topic("bookings")
            .build();

    @CitrusResource
    private TestCaseRunner t;

    @Test
    void shouldVerifyDemoApp() {
        t.when(
            send()
                .endpoint(bookings)
                .message()
                .body("How about Citrus!?")
        );

        t.when(
            receive()
                .endpoint(bookings)
                .message()
                .body("Citrus rocks!")
        );
    }
}
```

Creating new message endpoints is very easy. Just use the proper endpoint builder and optionally bind the new endpoint to the Citrus bean registry via `@BindToRegistry` annotation. You may then use the message endpoint in all `send` and `receive` test actions in order to exchange messages.

You may move the endpoint configuration into a separate class and load the endpoints with the configuration class as follows:

EndpointConfig.class

```
public class EndpointConfig {

    @BindToRegistry
    public KafkaEndpoint bookings() {
        return new KafkaEndpointBuilder()
            .topic("bookings")
            .build();
    }
}
```

The endpoint configuration class uses `@BindToRegistry` members or methods to add beans to the Citrus registry. The configuration class may be referenced by many tests then using the `@CitrusConfiguration` annotation.

Load endpoint config classes

```
@QuarkusTest
@CitrusSupport
@CitrusConfiguration(classes = EndpointConfig.class)
public class DemoApplicationTest implements TestActionSupport {

    @CitrusResource
    private KafkaEndpoint bookings;

    @CitrusResource
    private TestCaseRunner t;

    @Test
    void shouldVerifyDemoApp() {
        t.when(
            send()
                .endpoint(bookings)
                .message()
                .body("How about Citrus!?")
        );

        t.when(
            receive()
                .endpoint(bookings)
                .message()
                .body("Citrus rocks!")
        );
    }
}
```

Citrus loads the configuration class and injects the `KafkaEndpoint` instance to the test with `@CitrusResource` annotation.

4.4.2. Load dynamic tests

Citrus supports many test languages besides writing tests in pure Java. Users can load tests written in XML, YAML, Groovy and many more via dynamic tests.

Load YAML tests

```
@QuarkusTest
@CitrusSupport
@CitrusConfiguration(classes = EndpointConfig.class)
public class DemoApplicationTest implements TestActionSupport {

    @CitrusTestFactory
    public Stream<DynamicTest> loadYamlTests() {
        return
CitrusTestFactorySupport.factory(TestLoader.YAML).packageScan("some.package.name");
    }
}
```

The example above loads YAML test case definitions and runs those as dynamic tests with JUnit Jupiter. The package scan loads all files in the given folder and runs the tests via Citrus. All YAML tests are able to reference the message endpoints configured in the configuration class `EndpointConfig.class`.

A sample YAML test may look like this:

my.citrus.it.yaml

```
name: my-test
actions:
  - send:
      endpoint: bookings
      message:
        body:
          data: How about Citrus!?
  - receive:
      endpoint: bookings
      timeout: 5000
      message:
        body:
          data: Citrus rocks!
```

4.4.3. Set application properties

The `@QuarkusTest` annotation will automatically start the application under test. Citrus provides the ability to programmatically set application properties before the Quarkus application is started.

This is important when you need to overwrite configuration based on test message endpoints configured in the test.

The next example shows a Citrus enabled Quarkus test that supplies a set of application properties to configure the application under test.

Supply application properties

```
@QuarkusTest
@CitrusSupport(applicationPropertiesSupplier = DemoAppConfigurationSupplier.class)
@CitrusConfiguration(classes = EndpointConfig.class)
public class DemoApplicationTest implements TestActionSupport {

    // ...
}
```

The `DemoAppConfiguration` class implements the `Supplier` interface and set a config property. This property will be set on the application under test.

DemoAppConfigurationSupplier.class

```
public class DemoAppConfigurationSupplier implements ApplicationPropertiesSupplier {

    @Override
    public Map<String, String> get() {
        Map<String, String> conf = new Hasmap<>();
        conf.put("quarkus.log.level", "INFO");
        conf.put("greeting.message", "Hello, Citrus rocks!");
        return conf;
    }
}
```

The application properties supplier is able to set Quarkus properties as well as application domain properties. The example above sets `greeting.message` property which can be referenced in the Quarkus application:

DemoApplication

```
@ApplicationScoped
public class DemoApplication {

    private static final Logger logger = Logger.getLogger(DemoApplication.class);

    @ConfigProperty(name = "greeting.message")
    String message;

    void onStart(@Observes StartupEvent ev) {
        logger.info(message);
    }
}
```

4.4.4. Testcontainers support

Citrus integrates with Testcontainers to easily start/stop Testcontainers instances as part of the test. You can leverage the Citrus Testcontainers features within a Quarkus test very easily. Citrus implements Quarkus test resources for each of the supported containers (AWS LocalStack, Kafka, Redpanda, ...).

The following example starts an AWS LocalStack Testcontainers instance and uses the S3 service on that container to upload a file to the S3 bucket. The Quarkus application under test should handle this S3 file then.

AwsS3SourceTest

```
@QuarkusTest
@CitrusSupport
@LocalStackContainerSupport(services = LocalStackContainer.Service.S3,
containerLifecycleListener = AwsS3SourceTest.class)
public class AwsS3SourceTest implements TestActionSupport,
ContainerLifecycleListener<LocalStackContainer> {

    @CitrusResource
    private TestCaseRunner tc;

    @CitrusResource
    private LocalStackContainer localStackContainer;

    @Test
    public void shouldHandleUploadedS3File() {
        tc.given(this::uploadS3File);

        // verify that the Quarkus application has handled the S3 file
    }

    private void uploadS3File(TestContext context) {
        S3Client s3Client = createS3Client(localStackContainer);
```

```
        CreateMultipartUploadResponse initResponse = s3Client.createMultipartUpload(b
-> b.bucket(s3BucketName).key(s3Key));
        String etag = s3Client.uploadPart(b -> b.bucket(s3BucketName)
                        .key(s3Key)
                        .uploadId(initResponse.uploadId())
                        .partNumber(1),
                RequestBody.fromString(s3Data)).eTag();
        s3Client.completeMultipartUpload(b -> b.bucket(s3BucketName)
                .multipartUpload(CompletedMultipartUpload.builder()
                        .parts(Collections.singletonList(CompletedPart.builder()
                                .partNumber(1)
                                .eTag(etag).build())).build())
                .key(s3Key)
                .uploadId(initResponse.uploadId()));
    }

    @Override
    public Map<String, String> started(LocalStackContainer container) {
        S3Client s3Client = createS3Client(container);

        s3Client.createBucket(b -> b.bucket(s3BucketName));

        Map<String, String> conf = new HashMap<>();
        conf.put("my.app.aws-s3-source.accessKey", container.getAccessKey());
        conf.put("my.app.aws-s3-source.secretKey", container.getSecretKey());
        conf.put("my.app.aws-s3-source.region", container.getRegion());
        conf.put("my.app.aws-s3-source.bucketNameOrArn", s3BucketName);
        conf.put("my.app.aws-s3-source.uriEndpointOverride",
container.getServiceEndpoint().toString());
        conf.put("my.app.aws-s3-source.overrideEndpoint", "true");
        conf.put("my.app.aws-s3-source.forcePathStyle", "true");

        return conf;
    }

    private static S3Client createS3Client(LocalStackContainer container) {
        return S3Client.builder()
                .endpointOverride(container.getServiceEndpoint())
                .credentialsProvider(
                        StaticCredentialsProvider.create(
                                AwsBasicCredentials.create(container.getAccessKey(),
container.getSecretKey())
                        )
                )
                .forcePathStyle(true)
                .region(Region.of(container.getRegion()))
                .build();
    }
}
```

A few things happened in this example and let's explain those features one after another. First thing to notice is the `@LocalStackContainerSupport` annotation that makes Citrus run the AWS LocalStack Testcontainers instance. Also, the annotation provides the enabled services on that container (`services = LocalStackContainer.Service.S3`). This starts the Testcontainers instance as part of the Quarkus test.

The test also implements the `ContainerLifecycleListener` interface. This enables the test to handle the container instance after it has been started. This is a good place to create an S3 client and the bucket for the test.

Create S3 client

```
@Override
public Map<String, String> started(LocalStackContainer container) {
    S3Client s3Client = createS3Client(container);

    s3Client.createBucket(b -> b.bucket(s3BucketName));

    Map<String, String> conf = new HashMap<>();
    conf.put("my.app.aws-s3-source.accessKey", container.getAccessKey());
    conf.put("my.app.aws-s3-source.secretKey", container.getSecretKey());
    conf.put("my.app.aws-s3-source.region", container.getRegion());
    conf.put("my.app.aws-s3-source.bucketNameOrArn", s3BucketName);
    conf.put("my.app.aws-s3-source.uriEndpointOverride",
container.getServiceEndpoint().toString());
    conf.put("my.app.aws-s3-source.overrideEndpoint", "true");
    conf.put("my.app.aws-s3-source.forcePathStyle", "true");

    return conf;
}
```

Also, the started listener may return some application properties that get set for the Quarkus application under test. This is the opportunity to set the Testcontainers connection settings for the Quarkus application.

Obviously the Quarkus application uses some property based configuration with the `my.app.*` properties. The test is able to reference the Testcontainers exposed settings as values for these properties (e.g. `my.app.aws-s3-source.accessKey=container.getAccessKey()`).

With this configuration in place the test is able to upload and S3 file to the test bucket on the Testcontainers instance with the `uploadS3File()` method. This should trigger the Quarkus application under test to handle the new file accordingly. We can add some verification and assertion steps to verify that the Quarkus application has handled the S3 file.

This is how Citrus is able to start Testcontainers instances as part of a Quarkus test. The application properties supplier as well as the container lifecycle listener interfaces allow us to connect the Quarkus application with the Testcontainers instance. The test is able to use the services on the Testcontainers instance to trigger some test data that is consumed by the application under test.

4.4.5. Docker Compose support

Citrus provides a `@ComposeContainerSupport` annotation that allows you to start a Docker Compose environment as part of your Quarkus test. This is useful when your application under test depends on multiple services that are defined in a Docker Compose file.

ComposeContainerTest

```
@QuarkusTest
@CitrusSupport
@ComposeContainerSupport(
    composeFile = "compose.yaml",
    exposedServices = {
        @ComposeContainerSupport.ExposedService(name = "db", port = 5432),
        @ComposeContainerSupport.ExposedService(name = "broker", port = 9092)
    },
    containerLifecycleListener = ComposeContainerTest.class
)
public class ComposeContainerTest implements TestActionSupport,
ContainerLifecycleListener<ComposeContainer> {

    @CitrusResource
    private TestCaseRunner tc;

    @CitrusResource
    private ComposeContainer composeContainer;

    @Test
    public void shouldTestWithComposeServices() {
        // test logic using the compose services
    }

    @Override
    public Map<String, String> started(ComposeContainer container) {
        Map<String, String> conf = new HashMap<>();
        conf.put("my.app.db.port",
            String.valueOf(container.getServicePort("db", 5432)));
        conf.put("my.app.db.host",
            container.getServiceHost("db", 5432));
        return conf;
    }
}
```

The `@ComposeContainerSupport` annotation accepts the following attributes:

- `composeFile` - Path to the Docker Compose file. Defaults to `compose.yaml` on the classpath when not set.
- `exposedServices` - Services and ports to expose from the compose environment. Each exposed service is defined with a `name` (as defined in the compose file) and a `port`.

- `startupTimeout` - Startup timeout in seconds. Uses the default from `ComposeContainerSettings` when set to `-1`.
- `containerLifecycleListener` - Container lifecycle listener classes to handle post-start and pre-stop events.

The test can inject the `ComposeContainer` instance using the `@CitrusResource` annotation to interact with the compose environment directly. As with other Testcontainers annotations in Citrus the `ContainerLifecycleListener` interface allows you to react to container lifecycle events and return application properties that get set for the Quarkus application under test.

Please also have a look into the other provided Testcontainers annotations in Citrus:

- @LocalStackContainerSupport
- @KafkaContainerSupport
- @RedpandaContainerSupport
- @ComposeContainerSupport
- @TestcontainersSupport

All of these annotations allow you to start Testcontainers instances as part of your Quarkus test and provides the opportunity to participate in the container lifecycle to access managed ports and connectivity settings for instance.

4.5. Cucumber

Behavior driven development (BDD) is a very popular concept when it comes to find a common understanding of test scopes test logic. The idea of defining and describing the software behavior as basis for all tests in prior to translating those feature descriptions into executable tests is a very interesting approach because it includes the technical experts as well as the domain experts.

With BDD the domain experts should be able to read and verify tests and the technical experts get a detailed description of what should happen in the test.

The test scenario descriptions follow the Gherkin syntax with a **"Given-When-Then"** structure. The Gherkin language is business readable and helps to explain business logic with help of concrete examples.

There are several frameworks in the Java community supporting BDD concepts. Citrus has dedicated support for the Cucumber framework because Cucumber is well suited for extensions and plugins. So with the Citrus and Cucumber integration you can write Gherkin syntax scenarios in order to run those as Citrus integration tests.



The Cucumber components in Citrus are located in a separate Maven module. You need to include the module as a Maven dependency to your project.

Cucumber module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-cucumber</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Cucumber works with both JUnit and TestNG as unit testing framework. You can choose which framework to use with Cucumber. So following from that we need a Maven dependency for the unit testing framework support:

Cucumber JUnit support

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>${cucumber.version}</version>
</dependency>
```

In order to enable Citrus Cucumber support we need to specify a special object factory in the environment. The most comfortable way to specify a custom object factory is to add this property to the `cucumber.properties` in classpath.

cucumber.properties

```
cucumber.object-factory=org.citrusframework.cucumber.backend.CitrusObjectFactory
```

This special object factory takes care of creating all step definition instances. The object factory is able to inject `@CitrusResource` annotated fields in step classes. We will see this later on in the examples. The usage of this special object factory is mandatory in order to combine Citrus and Cucumber capabilities.

The **CitrusObjectFactory** will automatically initialize the Citrus world for us. This includes the default Citrus context configuration that is automatically loaded within the object factory. So you can define and use Citrus components as usual within your test.

After these preparation steps you are able to combine Citrus and Cucumber in your project.



In case you want to use Spring support in Citrus with a Spring application context you should use the following factory implementation.

cucumber.properties

```
cucumber.object-
factory=org.citrusframework.cucumber.backend.spring.CitrusSpringObjectFactory
```

4.5.1. Cucumber options

Cucumber is able to run tests with JUnit. The basic test case is an empty test which uses the respective JUnit runner implementation from cucumber.

MyFeature.java

```
@RunWith(Cucumber.class)
@CucumberOptions(
  plugin = { "pretty", "org.citrusframework.cucumber.CitrusReporter" } )
public class MyFeatureIT {
}
```

The test case above uses the **Cucumber** JUnit test runner. In addition to that we give some options to the Cucumber execution. In case you want to have the usual Citrus test results reported you can add the special Citrus reporter implementation `org.citrusframework.cucumber.CitrusReporter`. This class is responsible for printing the Citrus test summary. This reporter extends the default Cucumber reporter so the default Cucumber report summary is also printed to the console.

That completes the JUnit class configuration. Now we are able to add feature stories and step definitions to the package of our test **MyFeatureIT** . Cucumber and Citrus will automatically pick up step definitions and glue code in that test package. So let's write a feature story **echo.feature** right next to the **MyFeatureIT** test class.

echo.feature

```
Feature: Echo service

  Scenario: Say hello
    Given My name is Citrus
    When I say hello to the service
    Then the service should return: "Hello, my name is Citrus!"

  Scenario: Say goodbye
    Given My name is Citrus
    When I say goodbye to the service
    Then the service should return: "Goodbye from Citrus!"
```

As you can see this story defines two scenarios with the Gherkin **Given-When-Then** syntax. Now we need to add step definitions that glue the story description to Citrus test actions. Let's do this in a new class **EchoSteps** .

EchoSteps.java

```
public class EchoSteps {

    @CitrusResource
    private TestCaseRunner runner;

    @Given("^My name is (.*)$")
    public void my_name_is(String name) {
        runner.variable("username", name);
    }

    @When("^I say hello.*$")
    public void say_hello() {
        runner.when(
                send("echoEndpoint")
                    .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello, my name is ${username}!"));
    }

    @When("^I say goodbye.*$")
    public void say_goodbye() {
        runner.when(
                send("echoEndpoint")
                    .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Goodbye from ${username}!"));
    }

    @Then("^the service should return: \"([^\"]*)\"$")
    public void verify_return(final String body) {
        runner.then(
                receive("echoEndpoint")
                    .message()
                    .type(MessageType.PLAINTEXT)
                    .body(body));
    }
}
```

The step definition class is a normal POJO that uses some annotations such as `@CitrusResource` annotated `TestCaseRunner`. The Citrus backend injects the test runner instance at runtime.

The step definition contains normal @Given, @When or @Then annotated methods that match the scenario descriptions in our feature file. Cucumber will automatically find matching methods and execute them. The methods add test actions to the test runner as we used to do in normal Java DSL tests.

That is a first combination of Citrus and Cucumber BDD. The feature file gets translated into step implementations that use Citrus test action runner Java API to run integration tests with behavior

driven development.

4.5.2. Cucumber XML steps

The previous section handled glue code in Java in form of step definitions accessing the Java test runner fluent API. This chapter deals with the same concept with just XML configuration.

Citrus provides a separate configuration namespace and schema definition for Cucumber related step definitions. Include this namespace into your Spring configuration in order to use the Citrus Cucumber configuration elements.

Spring bean configuration schema

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xmlns="http://www.citrusframework.org/schema/cucumber/testcase"
     xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.citrusframework.org/schema/cucumber/testcase
         http://www.citrusframework.org/schema/cucumber/testcase/citrus-cucumber-
testcase.xsd">

    [...]

</spring:beans>
```

The JUnit Cucumber feature class itself does not change. We still use the Cucumber JUnit runner implementation with some options specific to Citrus:

MyFeatureIT.java

```
@RunWith(Cucumber.class)
@CucumberOptions(
    plugin = { "pretty", "org.citrusframework.cucumber.CitrusReporter" } )
public class MyFeatureIT {
}
```

The feature file with its Gherkin scenarios does also not change:

echo.feature

```
Feature: Echo service

  Scenario: Say hello
    Given My name is Citrus
    When I say hello to the service
    Then the service should return: "Hello, my name is Citrus!"

  Scenario: Say goodbye
    Given My name is Citrus
    When I say goodbye to the service
    Then the service should return: "Goodbye from Citrus!"
```

In the feature package **my.company.features** we add a new XML file `EchoSteps.xml` that holds the new XML step definitions:

EchoSteps.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<spring:beans xmlns:citrus="http://www.citrusframework.org/schema/testcase"
      xmlns:spring="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://www.citrusframework.org/schema/cucumber/testcase"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/cucumber/testcase
              http://www.citrusframework.org/schema/cucumber/testcase/citrus-cucumber-
testcase.xsd">

  <step given="^My name is (.*)$" parameter-names="username">
    <citrus:create-variables>
      <citrus:variable name="username" value="${username}"/>
    </citrus:create-variables>
  </step>

  <step when="^I say hello.*$">
    <citrus:send endpoint="echoEndpoint">
      <citrus:message type="plaintext">
        <citrus:data>Hello, my name is ${username}!</citrus:data>
      </citrus:message>
    </citrus:send>
  </step>

  <step when="^I say goodbye.*$">
    <citrus:send endpoint="echoEndpoint">
      <citrus:message type="plaintext">
        <citrus:data>Goodbye from ${username}!</citrus:data>
      </citrus:message>
    </citrus:send>
  </step>

  <step then="^the service should return: &quot;([^&quot;]*)&quot;$" parameter-
names="body">
    <citrus:receive endpoint="echoEndpoint">
      <citrus:message type="plaintext">
        <citrus:data>You just said: ${body}</citrus:data>
      </citrus:message>
    </citrus:receive>
  </step>

</spring:beans>
```

The above step definition uses pure XML actions. Citrus will automatically read the step definition and add those to the Cucumber runtime. Following from that the step definitions are executed when matching a statement in the feature story.

The XML step files follow a naming convention. Citrus will look for all files located in the feature package with name pattern `**/**.Steps.xml` and load those definitions when Cucumber starts up.

The XML steps are able to receive parameters from the Gherkin regexp matcher. The parameters are passed to the step as test variable. The parameter names get declared in the optional attribute `parameter-names`. In the step definitions you can use the parameter names as test variables.



The test variables are visible in all upcoming steps, too. This is because the test variables are global by default. If you need to set local state for a step definition you can use another attribute `global-context` and set it to `false` in the step definition. This way all test variables and parameters are only visible in the step definition. Other steps will not see the test variables.



Another notable thing is the XML escaping of reserved characters in the pattern definition. You can see that in the last step where the **then** attribute is escaping quotation characters.

Escape reserved characters

```
<step then="^the service should return: &quot;([^&quot;]*)&quot;$" parameter-
names="body">
...
</step>
```

We have to do this because otherwise the quotation characters will interfere with the XML syntax in the attribute.

This completes the description of how to add XML step definitions to the cucumber BDD tests.

4.5.3. Cucumber Spring support

Cucumber provides support for Spring dependency injection in step definition classes. The Cucumber Spring capabilities are included in a separate module. So first of all we have to add this dependency to our project:

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-spring</artifactId>
  <version>${cucumber.version}</version>
</dependency>
```

The Citrus Cucumber extension has to handle things different when Cucumber Spring support is enabled. Therefore we use another object factory implementation that also support Cucumber Spring features. Change the object factory property in **cucumber.properties** to the following:

cucumber.properties

```
cucumber.object-
factory=org.citrusframework.cucumber.backend.spring.CitrusSpringObjectFactory
```

Now we are ready to add **@Autowired** Spring bean dependency injection to step definition classes:

EchoSteps.java

```
@ContextConfiguration(classes = CitrusSpringConfig.class)
public class EchoSteps {
    @Autowired
    private Endpoint echoEndpoint;

    @CitrusResource
    protected TestDesigner designer;

    @Given("^My name is (.*)$")
    public void my_name_is(String name) {
        designer.variable("username", name);
    }

    @When("^I say hello.*$")
    public void say_hello() {
        designer.send(echoEndpoint)
            .messageType(MessageType.PLAINTEXT)
            .payload("Hello, my name is ${username}!");
    }

    @When("^I say goodbye.*$")
    public void say_goodbye() {
        designer.send(echoEndpoint)
            .messageType(MessageType.PLAINTEXT)
            .payload("Goodbye from ${username}!");
    }

    @Then("^the service should return: \"([^\"]*)\"$")
    public void verify_return(final String body) {
        designer.receive(echoEndpoint)
            .messageType(MessageType.PLAINTEXT)
            .payload("You just said: " + body);
    }
}
```

As you can see we used Spring autowiring mechanism for the **echoEndpoint** field in the step definition. Also be sure to define the **@ContextConfiguration** annotation on the step definition. The Cucumber Spring support loads the Spring application context and takes care of dependency injection. We use the Citrus **CitrusSpringConfig** Java configuration because this is the main entrance for Citrus test cases. You can add custom beans and further Spring related configuration to

this Spring application context. If you want to add more beans for autowiring do so in the Citrus Spring configuration. Usually this is the default **citrus-context.xml** which is automatically loaded.

Of course, you can also use a custom Java Spring configuration class here. Please be sure to always import the Citrus Spring Java configuration classes, too.

As usual, we are able to use **@CitrusResource** annotated **TestCaseRunner** fields for building the Citrus integration test logic. With this extension you can use the full Spring testing power in your tests in particular dependency injection and also transaction management for data persistence tests.

4.5.4. Predefined step definitions

Citrus provides a set of predefined step implementations for typical integration test scenarios that you can use out-of-the-box in a Cucumber feature.

You can basically define send/receive operations and many other predefined steps to leverage Citrus functionality such as endpoints and test actions. As these steps are predefined in Citrus you just need to add the step as a line in your feature stories. The step definitions with the glue to test actions is handled automatically when running Citrus with Cucumber.

If you want to enable predefined steps support in your test you need to include the Citrus Cucumber steps module as a Maven dependency.

Citrus Cucumber step module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-cucumber-core</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

After that you need to include the glue code package in your test class like this:

Include Citrus steps

```
@RunWith(Cucumber.class)
@CucumberOptions(
    extraGlue = { "org.citrusframework.cucumber" },
    plugin = { "pretty", "org.citrusframework.cucumber.CitrusReporter" } )
public class MyFeatureIT {

}
```

Instead of writing the glue code on our own in step definition classes we include the glue package `org.citrusframework.cucumber` as extra glue. This automatically loads all predefined Citrus step definitions. Once you have done this you can use predefined steps without having to write any glue code in Java.

```
Feature: Hello World

  Scenario: Print Hello World
    Given variable text="Hello World!"
    Then print 'You just said: ${text}'
```

The feature uses the default predefined steps `Given variable <name>="<value>"` and `Then print '<output>'` provided by Citrus.

The Citrus framework provides many different modules serving many different aspects of integration testing. You can add those steps with modular dependencies in Maven. See the following modules with predefined steps provided by Citrus:

Table 1. Citrus Cucumber step modules

Module	Description
citrus-cucumber-core	Standard steps such as test variables, sleep/delay, log/print, ...
citrus-cucumber-http	Http steps for client and server side communication
citrus-cucumber-openapi	Load Open API specifications and invoke/verify operations with generated test data
citrus-cucumber-kubernetes	Manage Kubernetes resources (e.g. pods, deployments, custom resources)
citrus-cucumber-knative	Steps to connect with Knative eventing and messaging
citrus-cucumber-jms	Send/receive steps via JMS queues/topics
citrus-cucumber-kafka	Steps to publish/subscribe on Kafka messaging
citrus-cucumber-jdbc	Steps to connect to relational databases
citrus-cucumber-camel	Steps to access Apache Camel components and Camel routes
citrus-cucumber-selenium	Run UI tests with Selenium using Selenium grid or standalone containers
citrus-cucumber-groovy	Leverage Groovy scripts as Citrus endpoint and component configuration
citrus-cucumber-testcontainers	Manage Testcontainers instances with starting/stopping databases, message brokers and many more

Once again it is important to understand that the predefined step definitions included in these modules can be used out-of-the-box. You can just start to write BDD features in Gherkin syntax that trigger these predefined steps.



If you want to include all available step definitions into your project at once you can use the `citrus-cucumber-all` module dependency.

Include all Citrus Cucumber steps

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-cucumber-all</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

Read more about the individual step implementations and their functionality in Tools - Citrus Cucumber steps.

4.6. Main CLI

Citrus provides a main Java class that you can run from command line. The Citrus Main CLI class supports some command line arguments that you can use to specify which tests to run.



Please also have a look into Citrus JBang CLI support as it makes running Citrus tests from a command line even more comfortable as it also takes care of setting up Java and the Citrus Maven project for free. With JBang running Citrus tests becomes as easy as running a single file (`.java`, `.xml`, `.groovy`, `.yaml`) from the command line.



The Main CLI support is shipped in a separate Maven module. You need to include the module as a dependency in your project.

Main CLI module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-main</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

You can call the Citrus Main CLI and provide some arguments.

Run Citrus main

```
CitrusApp.main(new String[] { "--help" });
```

This calls the Citrus Main CLI and prints the help message, so you can see which command line arguments you can choose from.

```
Citrus application option usage:
  -h or --help = Displays cli option usage
  -d or --duration = Maximum time in milliseconds the server should be up and running
- server will terminate automatically when time exceeds
  -c or --config = Custom configuration class
  -s or --skipTests = Skip test execution
  -p or --package = Test package to execute
  -D or --properties = Default system properties to set
  --exit = Force system exit when finished
  -e or --engine = Set test engine name used to run the tests
  -t or --test = Test class/method to execute
  -j or --jar = External test jar to load tests from
```

The `--engine` argument specifies which test framework to use when running the tests. Citrus supports these test engines as an argument value:

- JUnit Jupiter (`junit` or `junit-jupiter`)
- Cucumber (`cucumber`)
- TestNG (`testng`)
- JUnit 4 (`junit4`)

You may now use `--package`, `--test` and `--jar` to give a Java package name or test class name to run. With `--config` you can give a fully qualified class name that points to a Citrus endpoint configuration class. The configuration class is automatically loaded when the Main CLI is running.

4.7. JBang

You can easily create and run Citrus tests with JBang. The JBang tool support in Citrus is described in more detail in reference guide

Running Citrus via JBang does not require any project setup which is a fantastic match for fast prototyping of integration tests. The JBang command will automatically set up everything you need to run the Citrus test. This means you can run your test case sources directly from your command line.

To initialize a test you can run the script `citrus@citrusframework/citrus` with the `init` command as follows:

Initialize my.citrus.it.yaml

```
jbang citrus@citrusframework/citrus init my.citrus.it.yaml
```

The command above uses the JBang catalog `citrus@citrusframework/citrus` located on the Citrus GitHub repository. JBang will automatically resolve all dependencies and execute the command line script tool. This initializes the Citrus test file. You will find the created test source file in the current directory.

my.citrus.it.yaml

```
name: my-test
author: Citrus
status: FINAL
description: Sample test in YAML
variables:
  - name: message
    value: Citrus rocks!
actions:
  - echo:
      message: "${message}"
```

The JBang script is able to initialize any supported Citrus test domain specific language `.java`, `.xml`, `.yaml`, `.groovy` or `.feature`.

You can now run this test source file without any prior project setup using Citrus JBang:

Run my.citrus.it.yaml

```
jbang citrus@citrusframework/citrus run my.citrus.it.yaml
```

The command output will be like this:

Output

```
INFO 53887 --- [           main] citrusframework.testng.TestNGEngine : Running test
source my.citrus.it.yaml
INFO 53887 --- [           main] org.testng.internal.Utils           : [TestNG]
Running:
INFO 53887 --- [           main] rusframework.report.LoggingReporter :        .__  __
INFO 53887 --- [           main] rusframework.report.LoggingReporter :   ____ |__|/
|________ __ __  ______
INFO 53887 --- [           main] rusframework.report.LoggingReporter : _/ ___\|  \
__\_  __ \  |  \/  ___/
INFO 53887 --- [           main] rusframework.report.LoggingReporter : \  \___|  ||  |
|  | \/  |  /\___ \
INFO 53887 --- [           main] rusframework.report.LoggingReporter :  \___  >__||__|
|__|  |____//____  >
INFO 53887 --- [           main] rusframework.report.LoggingReporter :      \/
\/
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter : C I T R U S  T
E S T S  ${citrus.version}
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
-------------------------------------------------------------------------
INFO 53887 --- [           main] .citrusframework.actions.EchoAction : Citrus rocks!
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter : TEST SUCCESS
```

```
my-test (org.citrusframework)
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
------------------------------------------------------------------------
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter : CITRUS TEST
RESULTS
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter : SUCCESS (     3
ms) my-test
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter : TOTAL:       1
INFO 53887 --- [           main] rusframework.report.LoggingReporter : SUCCESS: 1
(100.0%)
INFO 53887 --- [           main] rusframework.report.LoggingReporter : FAILED:      0
(0.0%)
INFO 53887 --- [           main] rusframework.report.LoggingReporter : PERFORMANCE: 0
ms
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
INFO 53887 --- [           main] rusframework.report.LoggingReporter :
------------------------------------------------------------------------

================================================
Default Suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
================================================
```

4.7.1. Install Citrus JBang app

For a more convenient command line usage you can install Citrus as a JBang app.

Install Citrus as JBang app

```
jbang trust add https://github.com/citrusframework/citrus/
jbang app install citrus@citrusframework/citrus
```

Now you can just call `citrus` and create and run tests with Citrus JBang.

Run my.citrus.it.yaml

```
citrus run my.citrus.it.yaml
```

4.7.2. Run tests

You can directly run test sources with Citrus JBang. This includes test sources written in Java (`.java`), XML (`.xml`), YAML (`.yaml`), Groovy (`.groovy`) or as a Cucumber Gherkin feature file (`.feature`).

Java test sources

Initialize MyTest.java

```
citrus init MyTest.java
```

MyTest.java

```
import org.citrusframework.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;

public class MyTest implements Runnable, TestActionSupport {

    @CitrusResource
    TestCaseRunner t;

    @Override
    public void run() {
        t.given(
            createVariables().variable("message", "Citrus rocks!")
        );

        t.then(
            echo().message("${message}")
        );
    }
}
```

Run MyTest.java

```
citrus run MyTest.java
```

XML test sources

Initialize my.citrus.it.xml

```
citrus init my.citrus.it.xml
```

my.citrus.it.xml

```
<test name="EchoTest" author="Christoph" status="FINAL"
xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>
  <variables>
    <variable name="message" value="Citrus rocks!"/>
  </variables>
  <actions>
    <echo message="${message}"/>
  </actions>
</test>
```

Run my.citrus.it.xml

```
citrus run my.citrus.it.xml
```

YAML test sources

Initialize my.citrus.it.yaml

```
citrus init my.citrus.it.yaml
```

my.citrus.it.yaml

```
name: EchoTest
description: "Sample test in YAML"
variables:
  - name: "message"
    value: "Citrus rocks!"
actions:
  - echo:
      message: "${message}"
```

Run my.citrus.it.yaml

```
citrus run my.citrus.it.yaml
```

Groovy test sources

Initialize my.citrus.it.groovy

```
citrus init my.citrus.it.groovy
```

my.citrus.it.groovy

```
name "EchoTest"
description "Sample test in Groovy"

variables {
    message="Citrus rocks!"
}

actions {
    $(echo().message('${message}'))
}
```

Run my.citrus.it.groovy

```
citrus run my.citrus.it.groovy
```

Cucumber feature sources

Initialize my.citrus.it.feature

```
citrus init my.citrus.it.feature
```

my.citrus.it.feature

```
Feature: EchoTest

  Background:
    Given variables
    | message | Citrus rocks! |

  Scenario: Print message
    Then print '${message}'
```

Run my.citrus.it.feature

```
jbang --deps org.citrusframework:citrus-cucumber-all:5.2.0-SNAPSHOT citrus run
my.citrus.it.feature
```



Many of the predefined Cucumber steps (e.g. `Then print '<message>'`) in Citrus are provided in separate Citrus modules. You need to add additional project dependencies for that steps to be loaded as part of the JBang script. The `--deps` option adds dependencies using Maven artifact coordinates. You may also add the additional modules to the `jbang.properties` as described in the next section.

4.7.3. Additional JBang dependencies

Citrus JBang comes with a set of default dependencies that makes the scripts run as tests.

The default modules that you can use in Citrus JBang are:

- org.citrusframework:citrus-base
- org.citrusframework:citrus-jbang-connector
- org.citrusframework:citrus-groovy
- org.citrusframework:citrus-xml
- org.citrusframework:citrus-yaml
- org.citrusframework:citrus-http
- org.citrusframework:citrus-validation-json
- org.citrusframework:citrus-validation-yaml
- org.citrusframework:citrus-validation-xml

This enables you to run Java, YAML, XML, Groovy tests out of the box. In case your tests uses an additional feature from the Citrus project you may need to add the module so JBang can load the dependency at startup.

The easiest way to do this is to create a `jbang.properties` file that defines the additional dependencies:

jbang.properties

```
# Declare required additional dependencies
run.deps=org.citrusframework:citrus-camel:${citrus.version},\
org.citrusframework:citrus-testcontainers:${citrus.version},\
org.citrusframework:citrus-kafka:${citrus.version}
```

The file above adds the modules `citrus-camel`, `citrus-testcontainers` and `citrus-kafka` so you can use them in your JBang Citrus test source.

The `jbang.properties` file may be located right next to the test source file or in your user home directory for global settings.



In case you want to run Cucumber BDD Gherkin feature files and use the predefined Citrus steps, you need to add this dependency accordingly: `org.citrusframework:citrus-cucumber-all:5.2.0-SNAPSHOT`

4.7.4. Run from clipboard

You can run tests from your current clipboard. Just use the file name `clipboard.xxx` where the file extension defines the type of the test source (`.java`, `.yaml`, `.xml`, `.groovy`, `.feature`).

Run YAML test from Clipboard

```
citrus run clipboard.yaml
```

4.7.5. List tests

The `ls` command lists all running Citrus tests. These tests may be started

List running tests

```
citrus ls
```

Command output

```
PID   NAME         STATUS  AGE
19201 my.citrus.it.yaml Running 20s
```

4.8. Sharding Test Cases

Sharding is a powerful option within the Citrus framework designed to facilitate the loading and execution of test cases in a distributed manner. Sharding test execution is expected to save roughly `[TIME] * 1 / 2` of execution time per runner.

This chapter provides an overview of the available functionality and guides you through the process of configuring and using test loaders with sharding.

4.8.1. Configuration

First of all, you need to wrap your test case loading with the `org.citrusframework.sharding.Shard#createShard(Stream<T> items)` method. Have a look at the previous runtime chapters to get to know how to do this with the test framework of your choice.

Once you're done doing so, the sharding can be configured using environment variables or system properties. The following two key configuration parameters are required for sharded test loading:

- **Total number of shards**: Defines how many shards the test cases should be divided into.
- **Shard number**: Specifies the specific shard number that this loader instance will handle.

The following table summarizes the full configuration options:

Configuration	Environment Variable	System Property
Total number of shards	`CITRUS_SHARDING_TOTAL`	`citrus.sharding.total`
Shard number	`CITRUS_SHARDING_NUMBER`	`citrus.sharding.number`
Shard seed	`CITRUS_SHARDING_SEED`	`citrus.sharding.seed`

When writing test cases in Java, additional constructors exist, such that the total number of shards,

as well as the current shard number can be programmatically configured.

Chapter 5. Run tests

You can write Citrus tests in various programming languages. The following chapters describe the individual domain specific languages that support you in writing tests in Citrus.



We do our best to make all features equal in all supported languages. Some feature may only make sense in a specific test project setup and are not applicable to all languages (e.g. test behaviors only run in Java classes).

5.1. Java

You can code the Citrus test as you would do with any other unit test framework (e.g. JUnit Jupiter or TestNG). In fact Citrus builds on top of those famous unit testing frameworks because the tooling and adoption throughout the industry is very high for these libraries.

Citrus has a set of test engines that you can use to run Citrus tests with your favorite unit test framework. For instance Citrus provides a JUnit Jupiter extension that you can use to code Citrus tests in Java. Or you can use the Citrus Cucumber integration to run your tests as Gherkin feature files.

No matter what test engine you choose the Citrus test will assist you in writing tests through a Java domain specific language. This is a set of predefined methods, builders and classes that you can use to code the test in Java. In conclusion the Citrus test holds a sequence of test actions and each action performs a very special step in your test such as sending or receiving a message.

Despite the fact that message exchange is one of the main actions in an integration test framework for message-based applications Citrus is more than just that. Each test case in Citrus is able to perform various actions such as connecting to the database, transforming data, adding iterations and conditional steps.

With the default Citrus actions provided out of the box users can accomplish very complex use cases in integration testing. As a tester you can configure test actions in Java using a test action runner API that leverages a fluent builder pattern.

5.1.1. Test action DSL

The test action domain specific language is the entry to the fluent Citrus Java API. You can configure test actions with a builder pattern style API and the test action runner immediately executes the test actions.

See the following example to see the Java domain specific language in action.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class HelloServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSayHello() {
        variable("user", "Citrus");

        $(echo("Hello from ${user}!"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class HelloServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    @Test
    @CitrusTest
    public void shouldSayHello() {
        t.createVariable("user", "Citrus");

        t.run(echo("Hello from ${user}!"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class HelloServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSayHello() {
        variable("user", "Citrus");

        $(echo("Hello from ${user}!"));
    }
}
```

The test action runner executes each test action immediately as you use the provided `run()` method. All actions provided in Citrus represent a certain functionality (e.g. send/receive messages, delay the test, access a database). Citrus ships with a wide range of test actions, but you are also able to write your own test actions and execute them during a test.



The `$()` method is a shortcut of the `run()` method. Please choose whatever fits best in terms of readability of the test code.

The test class implements the interface `TestActionSupport` which holds all available test actions in Citrus. You can choose from a wide range of actions. The provided test action API uses fluent style build pattern to define the test action and its properties.



The `TestActionSupport` interface brings quite a lot of methods. In case you are facing issues with one of these methods colliding with your own test methods you can choose from a modular interface hierarchy that builds the Citrus Java test actions API. Instead of implementing the whole `TestActionSupport` API you may just implement the `BaseTestActionSupport` API or the very specific `HttpTestActionSupport` interface in your test. Just look for the `*TestActionSupport` interfaces provided by Citrus and include what you need into your test class.



For those of you who are not able to use the `TestActionSupport` or other API interfaces the **TestNGCitrusSupport** and **JUnit4CitrusSupport** base classes also provide a method called `actions()` that provides access to the test actions Java DSL API.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class HelloServiceIT extends TestNGCitrusSupport {

    @Test
    @CitrusTest
    public void shouldSayHello() {
        variable("user", "Citrus");

        $(actions().echo("Hello from ${user}!"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class HelloServiceIT {

    @CitrusResource
    private TestCaseRunner t;

    @Test
    @CitrusTest
    public void shouldSayHello() {
        variable("user", "Citrus");

        t.run(
            t.actions().echo("Hello from ${user}!")
        );
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class HelloServiceIT extends JUnit4CitrusSupport {

    @Test
    @CitrusTest
    public void shouldSayHello() {
        variable("user", "Citrus");

        $(actions().echo("Hello from ${user}!"));
    }
}
```

By default, all actions run sequentially in the same order as they are defined in the test case. In case one single action fails the whole test case is failing. Of course, you can leverage parallel action execution with the usage of test containers.



The **TestNGCitrusSupport** and **JUnit4CitrusSupport** base classes are not thread safe by default. This is simply because the base class is holding state to the current test action runner instance in order to delegate method calls to this instance. Parallel test execution is not available with this approach. Fortunately there is a way to support parallel test execution through resource injection. Read more about this in JUnit4 or TestNG support.

5.1.2. Gherkin test action runner

The test action runner is also available as Gherkin style runner with `given()`, `when()`, `then()` methods. The Gherkin test action runner follows the Behavior Driven Development concepts of structuring the test into the three parts: **Given** a certain context, **when** an event occurs, **then** an outcome should be verified.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest() {
        given(
            echo("Setup the context")
        );

        when(
            echo("Trigger the event")
        );

        then(
            echo("Verify the outcome")
        );
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    @Test
    @CitrusTest
    public void serviceTest() {
        t.given(
            echo("Setup the context")
        );

        t.when(
            echo("Trigger the event")
        );

        t.then(
            echo("Verify the outcome")
        );
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest() {
        given(
            echo("Setup the context")
        );

        when(
            echo("Trigger the event")
        );

        then(
            echo("Verify the outcome")
        );
    }
}
```

5.1.3. Test meta information

The user is able to provide some additional information about the test case. The meta-info section at the very beginning of the test case holds information like author, status or creation date.

Test meta information

```
@CitrusTest
public void sampleTest() {
    description("This is a Test");
    author("Christoph");
    status(Status.FINAL);

    $(echo("Hello Citrus!"));
}
```

The status allows the following values:

- DRAFT
- READY_FOR_REVIEW
- DISABLED
- FINAL

This information gives the reader first impression about the test and is also used to generate test documentation. By default, Citrus is able to generate test reports in HTML and Excel in order to list all tests with their metadata information and description.



Tests with the status DISABLED will not be executed during a test suite run. So someone can just start adding planned test cases that are not finished yet in status DRAFT. In case a test is not runnable yet because it is not finished, someone may disable a test temporarily to avoid causing failures during a test run.

The test description should give a short introduction to the intended use case scenario that will be tested. The user should get a short summary of what the test case is trying to verify.

5.1.4. Finally block

Java developers might be familiar with the concept of try-catch-finally blocks. The ***finally*** section contains a list of test actions that will be executed guaranteed at the very end of the test case even if errors did occur during the execution before.

This is the right place to tidy up things that were previously created by the test like cleaning up the database for instance.

Finally block

```
@CitrusTest
public void sampleTest() {
    given(
        doFinally()
            .actions(echo("Do finally - regardless of any error before"))
    );

    $(echo("Hello Test Framework"));
}
```

As an example imagine that you have prepared some data inside the database at the beginning of the test and you need to make sure the data is cleaned up at the end of the test case.

Finally block example

```
@CitrusTest
public void finallyBlockTest() {
    variable("orderId", "citrus:randomNumber(5)");
    variable("date", "citrus:currentDate('dd.MM.yyyy')");

    given(
        doFinally()
            .actions(sql(dataSource).statement("DELETE FROM ORDERS WHERE
ORDER_ID='${orderId}'"))
    );

    when(
        sql(dataSource).statement("INSERT INTO ORDERS VALUES (${orderId}, 1, 1,
'${date}')")
    );

    then(
        echo("ORDER creation time: citrus:currentDate('dd.MM.yyyy')")
    );
}
```

In the example the first action creates an entry in the database using an ***INSERT*** statement. To be sure that the entry in the database is deleted after the test, the `finally` section contains the respective ***DELETE*** statement that is always executed regardless the test case state (successful or failed).



The `finally` section must be placed at the very beginning of the test. This is because the test action runner is immediately executing each test action as it is called within the Java DSL methods. This is the only way the test case can perform the final actions also in case of previous error.

A `finally` block placed at the very end of the test will not take action unless put in a traditional Java try-finally-block:

Traditional try-finally block

```
@CitrusTest
public void finallyBlockTest() {
    variable("orderId", "citrus:randomNumber(5)");
    variable("date", "citrus:currentDate('dd.MM.yyyy')");

    try {
        when(
            sql(dataSource).statement("INSERT INTO ORDERS VALUES (${orderId}, 1, 1,
'${date}')")
        );

        then(
            echo("ORDER creation time: citrus:currentDate('dd.MM.yyyy')")
        );
    } finally {
        then(
            sql(dataSource).statement("DELETE FROM ORDERS WHERE
ORDER_ID='${orderId}'")
        );
    }
}
```

Using the traditional Java `try-finally` feels more natural no doubt. Please notice that the Citrus report and logging will not account the traditional finally block actions then. Good news is whatever layout you choose the outcome is always the same.

The `finally` block is executed safely even in case some previous test action raises an error for some reason.

5.1.5. Test behaviors

The concept of test behaviors is a good way to reuse test action blocks in the Java DSL. Test behaviors combine action sequences to a logical unit. The behavior defines a set of test actions that can be applied multiple times to different test cases.

The behavior is a separate Java DSL class with a single *apply* method that configures the test actions. Test behaviors follow this basic interface:

Test behaviors

```
@FunctionalInterface
public interface TestBehavior {

    /**
     * Behavior building method.
     */
    void apply(TestActionRunner runner);

    /**
     * Name of this behavior in test reports.
     */
    default String getName() { ... }

    /**
     * Sequential composition, reported as "<this name> then <next name>".
     */
    default TestBehavior andThen(TestBehavior next) { ... }

    /**
     * Gives a behavior, usually a lambda, an explicit report name.
     */
    static TestBehavior named(String name, TestBehavior delegate) { ... }
}
```

The behavior is provided with a test action runner and all actions in the behavior should run on that runner. Every time the behavior is applied to a test the actions get executed accordingly.

Test behaviors

```
public class FooBehavior implements TestBehavior, TestActionSupport {
    public void apply(TestActionRunner runner) {
        runner.run(createVariable("foo", "test"));

        runner.run(echo("fooBehavior"));
    }
}

public class BarBehavior implements TestBehavior, TestActionSupport {
    public void apply(TestActionRunner runner) {
        runner.run(createVariable("bar", "test"));

        runner.run(echo("barBehavior"));
    }
}
```

The listing above shows two test behaviors that add very specific test actions and test variables to the test case. As you can see the test behavior is able to use the same Java DSL action methods and defines test variables and actions as a normal test case would do. You can apply the behaviors

multiple times in different tests:

```
@CitrusTest
public void behaviorTest() {
    $(apply(new FooBehavior()));

    $(echo("Successfully applied foo behavior"));

    $(apply(new BarBehavior()));

    $(echo("Successfully applied bar behavior"));
}
```

The behavior is applied to the test case by calling the **apply()** method. As a result the behavior is executed adding its logic at this point of the test execution. The same behavior can now be called in multiple test cases so we have a reusable set of test actions.

Behavior names in reports

An applied behavior is a test action container, reported under the name of the behavior.

The test flow report written to `target/citrus-reports/<TestClass>.<method>-flow-<timestamp>.json` lists the behavior as one entry, with the actions it runs nested underneath:

```
{
  "name": "foo behavior",
  "path": "actions.0.foo behavior",
  "actions": [
    { "name": "create-variables", "path": "actions.0.foo behavior.actions.0.create-
variables" },
    { "name": "echo", "path": "actions.0.foo behavior.actions.1.echo" }
  ]
}
```

The console log shows a behavior applied at the top level of the test as a single step. Its nested actions get no step lines of their own; only what they log themselves appears in between:

```
 foo behavior - container
Setting variable: foo to value: test
fooBehavior
 foo behavior
```

When a behavior runs inside another container, the console shows that container's step lines only, as for any nested action. The flow report still lists the behavior under its name.

By default the name is the simple class name of the behavior split into lower case words, so `FooBehavior` is reported as `foo behavior`. Acronyms are kept as they are: `SendSOAPRequest` is reported

as `send SOAP request`. Nested classes and records use their simple name only.

Lambdas and anonymous classes have no usable class name, so they are reported as `behavior`. Use `TestBehavior.named(..)` to give them a name, or set a name for one particular application on the action:

```
@CitrusTest
public void namedBehaviorTest() {
    $(apply(TestBehavior.named("sign in", runner -> runner.run(echo("Signing in")))));

    $(apply(new FooBehavior()).name("create foo variable"));
}
```

A behavior class may also override `getName()` to describe itself.



A `doFinally()` block run inside a behavior still belongs to the test. Its actions run when the test finishes, not when the behavior is done, exactly like a `doFinally()` block in the test itself.

Behaviors in containers

A behavior is applied with the same spelling everywhere: at the top level of the test, inside test action containers such as `sequential()`, `parallel()` or `iterate()`, and inside another behavior.

```
@CitrusTest
public void behaviorInContainerTest() {
    $(sequential().actions(
        echo("Before the behavior"),
        apply(new FooBehavior()),
        echo("After the behavior")
    ));
}
```

The test action runner is passed to the behavior automatically. The explicit form `apply().behavior(new FooBehavior()).on(runner)` keeps working, and a runner set that way is kept, but the `on(runner)` call is not needed anymore.

Composing behaviors

Behaviors compose into larger behaviors without writing a wrapper class for each combination. The composite applies its parts in order and is reported as one behavior whose name joins the names of its parts:

```
@CitrusTest
public void composedBehaviorTest() {
    TestBehavior signIn = TestBehavior.named("sign in", runner ->
runner.run(echo("Signing in")));

    // reported as "sign in then foo behavior"
    $(apply(signIn.andThen(new FooBehavior())));

    // reported as "sign in then foo behavior then bar behavior"
    $(apply(List.of(signIn, new FooBehavior(), new BarBehavior())));
}
```

Fluent behaviors

A behavior does not have to be created with `new`. The `apply(..)` method accepts any `TestBehavior` instance, so a behavior can offer a static factory method and fluent methods for its inputs:

```
public record PlaceAnOrder(String item, int quantity) implements TestBehavior,
TestActionSupport {

    public static PlaceAnOrder placeAnOrder() {
        return new PlaceAnOrder(null, 1);
    }

    public PlaceAnOrder forItem(String item) {
        return new PlaceAnOrder(item, quantity);
    }

    public PlaceAnOrder inQuantity(int quantity) {
        return new PlaceAnOrder(item, quantity);
    }

    @Override
    public void apply(TestActionRunner runner) {
        if (item == null) {
            throw new CitrusRuntimeException("Missing item to order - use
forItem(..)");
        }

        runner.run(echo("Ordering %d x %s".formatted(quantity, item)));
    }
}
```

```
@CitrusTest
public void fluentBehaviorTest() {
    variable("sku", "SKU-1");

    $(apply(placeAnOrder().forItem("${sku}").inQuantity(5)));
}
```

The behavior is reported as `place an order`. When writing fluent behaviors, keep in mind:

- Return a new instance from every fluent method, as the record above does. A mutable builder shares its state when the same instance is applied more than once or inside `parallel()`.
- Check required inputs at the start of `apply(..)`. The check runs before any of the behavior's actions, so a missing input fails the test with a clear message. To require test variables instead, use `requires(..)` as described in Behavior variables.
- Inputs may contain test variable expressions such as `${sku}`. The behavior's actions resolve them when they run.
- The report name comes from the class, not from the inputs. Override `getName()` to include inputs in the name. The name is fixed when the action is built, so variable expressions appear in it unresolved.

Behavior variables

A behavior may use different variable names than the test and vice versa. No doubt the behavior will fail as soon as special variables with respective values are not present. Unknown variables cause the behavior and the whole test to fail with errors.

So a good approach would be to harmonize variable usage across behaviors and test cases, so that behaviors and test cases do use the same variable naming. By default the behavior automatically knows all variables in the test case and all test variables created inside the behavior are visible to the test case after applying. When a behavior changes a variable that the test case defines, the change affects the variable in the global test context.

A behavior that declares the variables it needs fails fast when they are missing. The check runs before any of the behavior's actions, and the error names the missing variable:

```
@CitrusTest
public void requiredVariablesTest() {
    variable("foo", "test");

    $(apply(new BarBehavior()).requires("foo"));
}
```

When a behavior is applied several times in a test, especially in combination with parallel containers (see containers-parallel), sharing the global test context is often not what you want. Apply the behavior with an isolated variable scope instead. An isolated behavior reads all variables of the test, but works on a copy of the test context, so its changes do not leak into the test. Use

`publish(..)` to hand selected variables back:

```
@CitrusTest
public void isolatedBehaviorTest() {
    // "foo" is not visible to the test after the behavior
    $(apply(new FooBehavior()).isolated());

    // only "bar" is copied back into the test
    $(apply(new BarBehavior()).isolated().publish("bar"));

    $(echo("${bar}"));
}
```

A published variable that the behavior does not set fails the test with an error naming the variable. Exceptions raised by forked actions and `doFinally()` actions inside an isolated behavior still reach the test.

Asking questions with typed answers

Today the only way a value leaves an action is a test variable:

```
$(http().client("orders").receive().response(HttpStatus.OK)
        .message().extract(fromBody().expression("$.id", "orderId")));
// from here on the value is "${orderId}" - a String, usable only inside Citrus
expressions
```

Everything is a `String`, the value cannot leave the DSL into plain Java, a value derived from several sources has no single named home, and the assignment is invisible in the report. A `Question<T>` is the named, reusable unit that runs as an ordinary reported test action and returns a typed Java value. This is the Screenplay question concept fitted into core: the injected runner is the capability a question acts through — with no Actor, Ability, or new module:

```
public static class TheOrderId implements Question<String> {
    @Override
    public String answeredBy(TestActionRunner runner, TestContext context) {
        return context.getVariable("orderId", String.class);
    }
}
```

The report name follows the same rule as behaviors, so `TheOrderId` is reported as `the order id`. Lambdas and anonymous classes are reported as `behavior`; use the pure-read form to name them:

```
Question<Integer> lineCount = Question.about("line count",
        ctx -> ctx.getVariable("lines", String.class).split("\n").length);
```

Two ready-made questions cover the common reads without touching `TestContext` internals:

```
Question<String> orderId = Question.variable("orderId", String.class);  // reported as
"variable orderId"
Question<Message> orders = Question.message("orders");                  // reported as
"message orders"
```

Ask the question with `ask(..)`, holding the value in an `Answer<T>`:

```
Answer<String> orderId = new Answer<>();

$(ask(theOrderId()).into(orderId).saveAs("orderId"));

assertThat(orderId.get(), startsWith("ORD-"));

$(http().client("warehouse").send().get("/orders/" + orderId.get()));
```

The action is reported as `ask: the order id` at the position it ran, and a throwing question fails the test with the question's name in the message. `Answer<T>` is a single-assignment holder: reading it before the action has run throws an error naming the question instead of returning `null`, a second assignment fails the same way, and a `null` answer is a valid answer distinct from "not answered yet". It implements `Supplier<T>` for free interop with `Optional` and stream code.

The holder exists because `run(..)` returns the runner for chaining, never the built action, so a value cannot come back by return value without changing a core interface. `saveAs(..)` additionally publishes the value as a test variable, so a question feeds both plain Java and later `${…}` expressions. A `null` answer is not published: test contexts forbid `null` values, so the variable stays undefined. Holders are single-use: asking into an already-answered holder fails instead of overwriting it, so loops need a fresh holder per iteration. The `ask(..)` builder receives the active runner automatically, so asking works inside `sequential()` and inside an applied behavior with no extra wiring.

A question is answered synchronously at the point of the call: it reads what the test already has, or performs a synchronous exchange. Correlated async replies stay the job of receive-actions and message selectors.

Upgrading from earlier versions

Since Citrus 5.1 an applied behavior is a test action container. This changes the structure of a test case, not the way it runs: execution order, variable visibility, exception propagation and the placement of finally actions are the same as before.

What you can observe:

- `TestCase.getActions()` and `TestCase.getActionCount()` list an applied behavior as a single `ApplyTestBehaviorAction`. The actions the behavior runs are no longer added to the test case. Read them from `ApplyTestBehaviorAction.getExecutedActions()` instead, like the nested actions of any other container.
- Test action listeners and reporters receive the applied behavior instead of each nested action,

the same way they treat actions nested in `sequential()` and other containers.

- The applied behavior is named after the behavior, as described above, instead of `apply-behavior`.
- A failure that the behavior catches itself no longer marks the test result as failed. Before, the test passed but was reported as a failure.

`ApplyTestBehaviorAction.Builder` now extends `AbstractTestContainerBuilder`, which in turn extends the former superclass `AbstractTestActionBuilder`. Code using the builder keeps compiling. Custom implementations of the `ApplyTestBehaviorActionBuilder` interface need to implement its new methods.

`TestBehavior` gained a default `getName()` method. A behavior class that already declares a `getName()` method now overrides it, so that method becomes the report name of the behavior. It must be public and return a `String`.

5.1.6. Run custom code

In general, you are able to mix Citrus Java DSL actions with custom Java code as you like.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        $(echo("Before service call"));

        myService.doSomething("Now calling custom service");

        $(echo("After service call"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        t.run(echo("Before service call"));

        myService.doSomething("Now calling custom service");

        t.run(echo("After service call"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        $(echo("Before service call"));

        myService.doSomething("Now calling custom service");

        $(echo("After service call"));
    }
}
```

The test above uses a mix of Citrus test actions and custom service calls. The test logic will execute as expected. It is recommended though to wrap custom code in a test action in order to have a consistent test reporting and failure management in Citrus.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        $(echo("Before service call"));

        $(
            action(context -> myService.doSomething("Now calling custom service"))
        );

        $(echo("After service call"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        t.run(echo("Before service call"));

        t.run(
            action(context -> myService.doSomething("Now calling custom service"))
        );

        t.run(echo("After service call"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    private MyService myService = new MyService();

    @Test
    @CitrusTest
    public void serviceTest() {
        $(echo("Before service call"));

        $(
            action(context -> myService.doSomething("Now calling custom service"))
        );

        $(echo("After service call"));
    }
}
```

The sample above wraps the call to the custom service `myService` in an abstract test action represented as Java lambda expression. This way the service call becomes part of the Citrus test execution and failures are reported properly. Also you have access to the current test context which holds the list of test variables as well as many other Citrus related test objects (e.g. message store).

This is why you should wrap custom code in a test action and run that code via the test action runner methods. You can also put your custom code in a test action implementation and reference the logic from multiple tests.

5.1.7. Bind objects to registry

The Citrus context is a place where objects can register themselves in order to enable dependency injection and instance sharing in multiple tests. Once you register the object in the context others can resolve the reference with its given name.

In a simple example the context can register a new endpoint that is injected in several tests.

You can access the Citrus context within the provided before/after methods on the test.

Register endpoint in Citrus context

```
public class CitrusRegisterEndpoint_IT extends TestNGCitrusSupport implements
TestActionSupport {

    @Override
    public void beforeSuite(CitrusContext context) {
        context.bind("foo", new FooEndpoint());
    }
}
```

With the CitrusContext you can bind objects to the registry. Each binding receives a name so others can resolve the instance reference for injection.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    private FooEndpoint foo;

    @Test
    @CitrusTest
    public void serviceTest() {
        $(send(foo)
                .message()
                .body("Hello foo!"));

        $(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusEndpoint
    private FooEndpoint foo;

    @CitrusResource
    private TestCaseRunner t;

    @Test
    @CitrusTest
    public void serviceTest() {
        t.run(send(foo)
                .message()
                .body("Hello foo!"));

        t.run(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    private FooEndpoint foo;

    @Test
    @CitrusTest
    public void serviceTest() {
        $(send(foo)
                .message()
                .body("Hello foo!"));

        $(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

The `@CitrusEndpoint` annotation injects the endpoint resolving the instance with the given name `foo`. Test methods can use this endpoint in the following send and receive actions.

@BindToRegistry

An alternative to using the `bind()` method on the CitrusContext is to use the `@BindToRegistry` annotation. Methods and fields annotated will automatically register in the CitrusContext registry.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusFramework
    private Citrus citrus;

    @BindToRegistry(name = "fooQueue")
    private MessageQueue queue = new DefaultMessageQueue("fooQueue");

    @BindToRegistry
    public FooEndpoint foo() {
        return new FooEndpoint();
    }

    @Test
    @CitrusTest
    public void serviceTest() {
        // ...
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    @CitrusFramework
    private Citrus citrus;

    @BindToRegistry(name = "fooQueue")
    private MessageQueue queue = new DefaultMessageQueue("fooQueue");

    @BindToRegistry
    public FooEndpoint foo() {
        return new FooEndpoint();
    }

    @Test
    @CitrusTest
    public void serviceTest() {
        // ...
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @CitrusFramework
    private Citrus citrus;

    @BindToRegistry(name = "fooQueue")
    private MessageQueue queue = new DefaultMessageQueue("fooQueue");

    @BindToRegistry
    public FooEndpoint foo() {
        return new FooEndpoint();
    }

    @Test
    @CitrusTest
    public void serviceTest() {
        // ...
    }
}
```

The annotation is able to specify an explicit binding name. The annotation works with public methods and fields in tests.

Configuration classes

As an alternative to adding the registry binding configuration directly to the test you can load configuration classes.

Configuration classes are automatically loaded before a test suite run and all methods and fields are parsed for potential bindings. You can use the environment settings `citrus.java.config` and/or `CITRUS_JAVA_CONFIG` to set a default configuration class.

citrus-application.properties

```
citrus.java.config=MyConfig.class
```

MyConfig.class

```
public class MyConfig {

    @BindToRegistry(name = "fooQueue")
    private MessageQueue queue = new DefaultMessageQueue("fooQueue");

    @BindToRegistry
    public FooEndpoint foo() {
        return new FooEndpoint();
    }
}
```

@CitrusConfiguration

Each test is able to use the `@CitrusConfiguration` annotation to add registry bindings, too.

TestNG

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusConfiguration;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

@CitrusConfiguration(classes = MyConfig.class)
public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    private FooEndpoint foo;

    @Test
    @CitrusTest
    public void serviceTest() {
        $(send(foo)
                .message()
                .body("Hello foo!"));

        $(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusConfiguration;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
@CitrusConfiguration(classes = MyConfig.class)
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    @CitrusEndpoint
    private FooEndpoint foo;

    @Test
    @CitrusTest
    public void serviceTest() {
        t.run(send(foo)
                .message()
                .body("Hello foo!"));

        t.run(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusConfiguration;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

@CitrusConfiguration(classes = MyConfig.class)
public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    private FooEndpoint foo;

    @Test
    @CitrusTest
    public void serviceTest() {
        $(send(foo)
                .message()
                .body("Hello foo!"));

        $(receive(foo)
                .message()
                .body("Hello Citrus!"));
    }
}
```

The `@CitrusConfiguration` annotation is able to load configuration classes and bind all components to the registry for later usage. The test can inject endpoints and other components using the `@CitrusEndpoint` and `@CitrusResource` annotation on fields.

5.1.8. Resource injection

Resource injection is a convenient mechanism to access Citrus internal objects such as TestRunner or TestContext instances. The following sections deal with resource injection of different objects.

Inject Citrus framework

You can access the Citrus framework instance in order to access all components and functionalities. Just use the `@CitrusFramework` annotation in your test class.

TestNG

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusFramework
    private Citrus citrus;

    @Test
    @CitrusTest
    public void serviceTest() {
        citrus.getCitrusContext().getMessageListeners().addMessageListener(new
MyListener());
    }
}
```

JUnit

```
import org.citrusframework.TestCaseRunner;
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusResource
    private TestCaseRunner t;

    @CitrusFramework
    private Citrus citrus;

    @Test
    @CitrusTest
    public void serviceTest() {
        citrus.getCitrusContext().getMessageListeners().addMessageListener(new
MyListener());
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @CitrusFramework
    private Citrus citrus;

    @Test
    @CitrusTest
    public void serviceTest() {
        citrus.getCitrusContext().getMessageListeners().addMessageListener(new
MyListener());
    }
}
```

The framework instance provides access to the Citrus context which is a central registry for all components. The example above adds a new message listener.



The Citrus context is a shared component. Components added will perform with all further tests and changes made affect all tests.

Test action runner injection

The test action runner is the entry to the fluent Java API. You can inject the runner as a method parameter.

TestNG

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Optional;
import org.testng.annotations.Test;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@Optional @CitrusResource TestCaseRunner runner) {
        t.run(
            createVariable("random", "citrus:randomNumber(10)")
        );

        t.run(
            echo("The random number is: ${random}")
        );
    }
}
```

JUnit

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@CitrusResource TestCaseRunner runner) {
        t.run(
            createVariable("random", "citrus:randomNumber(10)")
        );

        t.run(
            echo("The random number is: ${random}")
        );
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@CitrusResource TestCaseRunner runner) {
        t.run(
            createVariable("random", "citrus:randomNumber(10)")
        );

        t.run(
            echo("The random number is: ${random}")
        );
    }
}
```

The parameter requires the `@CitrusResource` annotations in order to mark the parameter for Citrus resource injection.

Now each method uses its own runner instance which makes sure that parallel test execution can take place without having the risk of side effects on other tests running at the same time.

Test context injection

The Citrus test context combines a set of central objects and functionalities that a test is able to make use of. The test context holds all variables and is able to resolve functions and validation matchers.

In general a tester will not have to explicitly access the test context because the framework is working with it behind the scenes. In terms of advanced operations and customizations accessing the test context may be a good idea though.

Each test action implementation has access to the test context as it is provided to the execution method in the interface:

Test action interface

```
@FunctionalInterface
public interface TestAction {
    /**
     * Main execution method doing all work
     * @param context
     */
    void execute(TestContext context);
}
```

In addition to that Citrus provides a resource injection mechanism that allows to access the current test context in a test class or test method.

TestNG

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Optional;
import org.testng.annotations.Test;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@Optional @CitrusResource TestCaseRunner runner,
                            @Optional @CitrusResource TestContext context) {
        context.setVariable("myVariable", "some value");

        t.run(echo("${myVariable}"));
    }
}
```

JUnit

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@CitrusResource TestCaseRunner runner,
                            @CitrusResource TestContext context) {
        context.setVariable("myVariable", "some value");

        t.run(echo("${myVariable}"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void serviceTest(@CitrusResource TestCaseRunner runner,
                            @CitrusResource TestContext context) {
        context.setVariable("myVariable", "some value");

        t.run(echo("${myVariable}"));
    }
}
```

As you can see the test method defines a parameter of type **org.citrusframework.context.TestContext**. The annotation **@CitrusResource** tells Citrus to inject this parameter with the according instance of the context for this test.

Now you have access to the context and all its capabilities such as variable management. As an alternative you can inject the test context as a class member variable.

Endpoint injection

Endpoints play a significant role when sending/receiving messages over various transports. An endpoint defines how to connect to a message transport (e.g. Http endpoint URL, JMS message broker connection, Kafka connection and topic selection).

Endpoints can live inside the Citrus context (e.g. in Spring application context) or you can inject the endpoint into the test class with given configuration.

TestNG

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Optional;
import org.testng.annotations.Test;

public class ServiceIT extends TestNGCitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    @DirectEndpointConfig(queueName = "FOO.test.queue")
    private Endpoint directEndpoint;

    @Test
    @CitrusTest
    public void serviceTest(@Optional @CitrusResource TestCaseRunner runner) {
        t.run(send(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));

        t.run(receive(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));
    }
}
```

JUnit

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ServiceIT implements TestActionSupport {

    @CitrusEndpoint
    @DirectEndpointConfig(queueName = "FOO.test.queue")
    private Endpoint directEndpoint;

    @Test
    @CitrusTest
    public void serviceTest(@CitrusResource TestCaseRunner runner) {
        t.run(send(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));

        t.run(receive(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));
    }
}
```

JUnit4

```
import org.citrusframework.dsl.TestActionSupport;
import org.citrusframework.TestCaseRunner;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ServiceIT extends JUnit4CitrusSupport implements TestActionSupport {

    @CitrusEndpoint
    @DirectEndpointConfig(queueName = "FOO.test.queue")
    private Endpoint directEndpoint;

    @Test
    @CitrusTest
    public void serviceTest() {
        $(send(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));

        $(receive(directEndpoint)
                .message()
                    .type(MessageType.PLAINTEXT)
                    .body("Hello!"));
    }
}
```

The sample above creates a new endpoint as a direct in-memory channel endpoint. Citrus reads the `@CitrusEndpoint` annotation and adds the configuration as given in the `@DirectEndpointConfig` annotation. This way you can create and inject endpoints directly to your test.



Citrus also supports the Spring framework as a central bean registry. You can add endpoints as Spring beans and use the `@Autowired` annotation to inject the endpoint in your test.

5.2. Test source loader

Besides writing Citrus tests directly in Java you can code tests in domain specific languages using one of the supported tests sources:

- Java runnable
- XML
- YAML
- Cucumber

- Groovy
- Spring XML

The test case definition file is loaded by Citrus and run as a normal test. In Citrus the `@CitrusTestSource` annotation in the Java class defines the test source location to be loaded. This annotation makes Citrus search for the XML, YAML, Groovy, Cucumber, Spring XML files that represent the Citrus test.

The next example shows how the annotation is used in a normal Java test to load XML test case definitions.

MyFirstCitrus_IT.java

```
package org.citrusframework;

import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

public class MyFirstCitrus_IT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", name="MyFirstCitrus_IT")
    public void myFirstCitrus_IT() {
    }
}
```

In the basic example above this means that Citrus searches for an XML test file in **org/citrusframework/MyFirstCitrus_IT.xml** .

You can customize this path and tell Citrus to search for another XML file by using the `@CitrusTestSource` annotation properties.

Following annotation properties are available:

type	Type of the test source to load (spring, groovy, xml, ...)
name	List of test case names to execute. Names also define XML file names to look for (**.xml** file extension is not needed here).
packageName	Custom package location for the XML files to load
packageScan	List of packages that are automatically scanned for XML test files to execute. For each XML file found separate test is executed. Note that this performs a Java Classpath package scan so all XML files in package are assumed to be valid Citrus XML test cases. In order to minimize the amount of accidentally loaded XML files the scan will only load XML files with `**/*Test.xml` and `**/*IT.xml` file name pattern.

Customize @CitrusTestSource

```
public class Sample_IT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", name = "CustomName_IT", packageName =
"com.other.test.package")
    public void customXmlTest() {}
}
```

The annotation above loads a different XML test file named `CustomName_IT` in package `com.other.test.package`.

You can also load multiple XML files and run each of them.

Load multiple XML files

```
public class Sample_IT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", name = { "Test_1", "Test_2" })
    public void multipleTests() {}
}
```

This tells Citrus to search for the files `Test_1.xml` and `Test_2.xml`. Citrus loads the files and runs each of them as a separate test. You can also load all test in a package with a `packageScan`.

Load multiple XML files

```
public class Sample_IT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", packageScan =  { "com.some.test.package",
"com.other.test.package" })
    public void packageScanTest() {}
}
```

This loads all XML files in the given packages and executes each of them as a separate test.

You can also mix the various `@CitrusTestSource` annotations in a single Java class. The class can have several methods with different annotations. Each annotated method represents one or more Citrus XML test cases.

@CitrusTestSource annotations

```
public class SampleIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", name = "SampleIT")
    public void sampleTest() {}

    @Test
    @CitrusTestSource(type = "xml", name = { "Test_1", "Test_2" })
    public void multipleTests() {}

    @Test
    @CitrusTestSource(type = "xml", name = "CustomName_IT", packageName =
"com.other.test.package")
    public void customXmlTest() {}

    @Test
    @CitrusTestSource(type = "xml", packageScan =  { "com.some.test.package",
"com.other.test.package" })
    public void packageScanTest() {}
}
```

You are free to combine these test annotations as you like in your class. Each XML test loaded as part of the class will be reported separately as a unit test. So the test reports will have the exact number of tests executed with proper success and failed stats. You can use the reports as normal unit test reports, for instance in a continuous build.



When test execution takes place each test method annotation is evaluated in sequence. XML test cases that match several times, for instance by explicit name reference and a package scan will be executed several times respectively.



The best thing about using the **@CitrusTestSource** annotation is that you can continue to use the test framework capabilities (e.g. test groups, invocation count, thread pools, data providers, and so on).

The following chapters describe the different supported test sources that you can load as a Citrus test.

5.3. XML

As an alternative to coding tests in a programming language you can specify the test logic in XML only, too. The XML language gives you some XSD schema and a set of elements that you can choose from to specify test actions.

As usual, you need to choose one of the supported test engines in Citrus to actually run the test (e.g. JUnit Jupiter or TestNG). You can then use the `@CitrusTestSource` annotation to load the XML file that represents the test case.

MyFirstXmlIT.java

```
package org.citrusframework;

import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

public class MyFirstXmlIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "xml", name="my.citrus.it.xml")
    public void myFirstXmlIT() {
    }
}
```

The test logic itself is defined in the XML file like this:

my.citrus.it.xml

```
<test name="EchoTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <variables>
    <variable name="text" value="Hello from Citrus!"/>
  </variables>

  <actions>
    <echo message="${text}"/>
  </actions>
</test>
```

5.3.1. Bean definitions

The XML test is able to define custom beans as part of the test configuration. Beans are registered in the Citrus bean registry and can be referenced by test actions and other components.

This does not require the `citrus-groovy` dependency.

my.citrus.it.xml

```
<test name="BeanTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <configuration>
    <beans>
      <bean name="myValidator"
type="org.citrusframework.validation.DefaultTextEqualsMessageValidator"/>
      <bean name="connectionFactory" type="org.example.jms.ActiveMQConnectionFactory">
        <property name="brokerUrl" value="tcp://localhost:61616"/>
        <property name="userName" value="admin"/>
      </bean>
    </beans>
  </configuration>

  <actions>
    <echo message="Bean test"/>
  </actions>
</test>
```

The `<configuration>` element holds a `<beans>` section where each `<bean>` specifies:

- `name` - the bean identifier used to register the instance in the Citrus bean registry
- `type` - the fully qualified Java class name (must have a default constructor)
- `<property>` - optional child elements with `name` and `value` attributes set via setter methods on the bean instance

The bean type must have a default constructor. Property values are automatically converted to the target setter parameter type (e.g. String to int).

Beans that implement `InitializingPhase` will have their `initialize()` method called after all properties are set. Beans that implement `ReferenceResolverAware` will receive the current reference resolver.

5.3.2. Configuration scripts

The XML test is able to create endpoints and other configuration entities as part of the test. The test leverages the Groovy programming language that provides a comfortable way to define beans in the Citrus registry. You can set any of the supported properties on the configuration components.

my.citrus.it.xml

```
<test name="EchoTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <variables>
    <variable name="text" value="Hello from Citrus!"/>
  </variables>

  <actions>
    <groovy>
      <endpoints>
        <script>
        configuration {
            queues {
                queue('say-hello')
            }

            endpoints {
                direct('hello')
                    .asynchronous()
                    .queue('say-hello')
            }
        }
        </script>
      </endpoints>
    </groovy>

    <send endpoint="hello">
      <message>
        <body>
          <data>Hello Citrus!</data>
        </body>
      </message>
    </send>
  </actions>
</test>
```

The test above creates a new `direct` endpoint called `hello` that uses an im-memory queue `say-hello`. The send action is able to reference the endpoint by its name to send a message body to the queue.

5.3.3. Templates

Templates group action sequences to a logical unit. You can think of templates as reusable components that are used in several tests. The maintenance is much more efficient because you need to apply changes only on the templates and all referenced use cases are updated

automatically.

The template gets identified by a unique name. Inside a test case we apply the template referencing this unique name. Have a look at a first example:

echo-template.xml

```
<template name="echo">
  <description>Sample template in XML</description>
  <parameters>
    <parameter name="message" value="Citrus rocks!"/>
  </parameters>

  <actions>
    <echo message="${message}"/>
  </actions>
</template>
```

The code example above describes a template called `echo`. Templates hold a sequence of test actions or calls other templates themselves. The template may define a set of parameters that represent customizable user input when applying the template.

The template is available for multiple tests that may apply the template as part of their test action sequence.

Apply templates

```
<test name="ApplyTemplateTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <actions>
    <apply-template name="echo"/>

    <apply-template name="echo">
      <parameters>
        <parameter name="message" value="Hello from Citrus!"/>
      </parameters>
    </apply-template>
  </actions>
</test>
```

The example references the `echo` template and call it as part of the test with `<apply-template/>`. You can customize the template parameter to overwrite the default variable `text` with a custom value.

The example above assumes that you have added the template to the Citrus bean registry with the name `echo`. If this is not the case you may also reference the file resource path when applying the

template:

Apply templates

```
<test name="ApplyTemplateTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <actions>
    <apply-template file="classpath:templates/xml/echo-template.xml">
      <parameters>
        <parameter name="message" value="Hello from Citrus!"/>
      </parameters>
    </apply-template>
  </actions>
</test>
```

Template parameters

How to handle test variables when dealing with templates, in particular when templates are defined in separate source files? A template may use different variable names compared to those names used a test and vice versa. The template fails immediately when respective variables are not present. Therefore, templates introduce the concept of parameters to declare its variable names.

Assume you have the following template:

sayHello template

```
<template name="sayHello">
  <description>Sample template in XML</description>
  <parameters>
    <parameter name="user" value="Citrus"/>
  </parameters>

  <actions>
    <echo message="Hello ${user}!"/>
  </actions>
</template>
```

The template `sayHello` in the example uses the variable `${user}`.

In case the test case already uses the template parameter as a value you do not need to set this variable explicitly on the apply template action.

Implicit template parameter

```
<test name="SayHelloTemplateTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <variables>
    <variable name="user" value="Christoph"/>
  </variables>

  <actions>
    <apply-template name="sayHello"/>
  </actions>
</test>
```

Output

```
Hello Christoph!
```



The implicit template parameter may break as soon as the template or the test changes its parameter or variable name.

The test case may also explicitly set the template parameter when applying the template:

Explicit template parameter

```
<test name="SayHelloTemplateTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <variables>
    <variable name="username" value="Mickey"/>
  </variables>

  <actions>
    <apply-template name="sayHello">
      <parameters>
        <parameter name="user" value="${username}"/>
      </parameters>
    </apply-template>
  </actions>
</test>
```

Output

```
Hello Mickey!
```

The variable **username** is already present in the test case and gets translated into the **user** parameter. Following from that the template works fine although test and template do work on different variable names.

With template parameters you are able to solve the calibration effort when working with templates and variables. It is always a good idea to check the used variables/parameters inside a template when calling it. There might be a variable that is not declared yet inside your test. So you need to define this value as a parameter.

Template parameter scope

A template is able to change a test variable value for the rest of the test case. It is important to know about this when changing test variables in a template as it affects the descendant test actions. Especially when running test actions or templates in parallel to each other this might lead to unexpected behavior. You may choose the template parameter scope for this scenario.



When a template works on variable values and parameters changes to these variables will automatically affect the variables in the whole test. So if you change a variable's value inside a template and the variable is defined inside the test case the changes will affect the variable in a global context. We have to be careful with this when executing a template several times in a test, especially in combination with parallel containers (see containers-parallel).

print template

```
<template name="print">
  <description>Sample template in XML</description>
  <parameters>
    <parameter name="index" value="0"/>
    <parameter name="area" value="Atlantis"/>
  </parameters>

  <actions>
    <echo message="#${index} Hello ${area}!"/>
  </actions>
</template>
```

Global scope parameter (default)

```
<test name="SayHelloTemplateTest" author="Christoph" status="FINAL"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
            http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <description>Sample test in XML</description>

  <actions>
    <parallel>
      <actions>
        <apply-template name="print">
          <parameters>
            <parameter name="index" value="1"/>
            <parameter name="area" value="Europe"/>
          </parameters>
        </apply-template>
        <apply-template name="print">
          <parameters>
            <parameter name="index" value="2"/>
            <parameter name="area" value="Asia"/>
          </parameters>
        </apply-template>
        <apply-template name="print">
          <parameters>
            <parameter name="index" value="3"/>
            <parameter name="area" value="Africa"/>
          </parameters>
        </apply-template>
      </actions>
    </parallel>
  </actions>
</test>
```

In the listing above a template **print** is called several times in a parallel container. The parameter values will be handled in a global context, so it is quite likely to happen that the template instances influence each other during execution. We might get such print messages:

Output

```
#2 Hello Europe!
#2 Hello Africa!
#3 Hello Africa!
```

Index parameters do not fit and the message **'Hello Asia'** is completely gone. This is because templates overwrite parameters to each other as they are executed in parallel at the same time. To avoid this behavior we need to tell the template that it should handle parameters as well as variables in a local context. This will enforce that each template instance is working on a dedicated

local context. See the **global-context** attribute that is set to **false** in this example:

print template

```
<template name="print" global-context="false">
  <description>Sample template in XML</description>
  <parameters>
    <parameter name="index" value="0"/>
    <parameter name="area" value="Atlantis"/>
  </parameters>

  <actions>
    <echo message="#${index} Hello ${area}!"/>
  </actions>
</template>
```

After that template instances will not influence each other anymore. But notice that variable changes inside the template then do not affect the test case neither.

5.4. YAML

As an alternative to coding tests in a programming language you can specify the test logic in YAML only, too. The YAML language gives you some XSD schema and a set of elements that you can choose from to specify test actions.

As usual, you need to choose one of the supported test engines in Citrus to actually run the test (e.g. JUnit Jupiter or TestNG). You can then use the `@CitrusTestSource` annotation to load the YAML file that represents the test case.

MyFirstYamlIT.java

```
package org.citrusframework;

import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

public class MyFirstYamlIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "yaml", name="my.citrus.it.yaml")
    public void myFirstXmlIT() {
    }
}
```

The test logic itself is defined in the YAML file like this:

my.citrus.it.yaml

```
name: EchoTest
author: Christoph
status: FINAL
description: "Sample test in YAML"
variables:
  - name: "message"
    value: "Hello from Citrus!"
actions:
  - echo:
      message: "${message}"
```

5.4.1. Bean definitions

The YAML test is able to define custom beans as part of the test configuration. Beans are registered in the Citrus bean registry and can be referenced by test actions and other components.

This does not require the `citrus-groovy` dependency.

my.citrus.it.yaml

```
name: BeanTest
author: Christoph
status: FINAL
description: "Sample test in YAML"
configuration:
  beans:
    - name: myValidator
      type: org.citrusframework.validation.DefaultTextEqualsMessageValidator
    - name: connectionFactory
      type: org.example.jms.ActiveMQConnectionFactory
      properties:
        brokerUrl: "tcp://localhost:61616"
        userName: "admin"
actions:
  - echo:
      message: "Bean test"
```

The `configuration` section holds a `beans` list where each entry specifies:

- `name` - the bean identifier used to register the instance in the Citrus bean registry
- `type` - the fully qualified Java class name (must have a default constructor)
- `properties` - optional map of property name-value pairs set via setter methods on the bean instance

The bean type must have a default constructor. Property values are automatically converted to the target setter parameter type (e.g. String to int).

Beans that implement `InitializingPhase` will have their `initialize()` method called after all properties are set. Beans that implement `ReferenceResolverAware` will receive the current reference resolver.

5.4.2. Configuration scripts

The YAML test is able to create endpoints and other configuration entities as part of the test. The test leverages the Groovy programming language that provides a comfortable way to define beans in the Citrus registry.

The script creates the instances as beans in the Citrus registry.

A configuration script may use methods and closures to define the bean instances. The configuration script supports following methods:

- queues()
- endpoints()
- beans()

my.citrus.it.yaml

```
name: EchoTest
author: Christoph
status: FINAL
description: "Sample test in YAML"
variables:
  - name: "message"
    value: "Hello from Citrus!"
actions:
  - groovy:
      endpoints:
        script: |
          configuration {
              queues {
                  queue('say-hello')
              }

              endpoints {
                  direct('hello')
                      .asynchronous()
                      .queue('say-hello')
              }
          }
  - send:
      endpoint: "hello"
      message>:
        body:
          data: Hello Citrus!
```

The test above creates a new `direct` endpoint called `hello` that uses an in-memory queue named `say-hello`.

The `send` test action is able to reference the new endpoint by its name to send a message body to the queue.

You can also create bean instances with a Groovy script:

my.citrus.it.yaml

```
actions:
  - groovy:
      beans:
        script: |
          beans {
            bean(DefaultTextEqualsMessageValidator.class)
          }
```

The configuration script initializes a new bean instance of type `org.citrusframework.validation.DefaultTextEqualsMessageValidator.class`. A `receive` test action will automatically resolve the new message validator instance from the Citrus bean registry to perform its message validation.

The bean type must have a default constructor in this case.

Create beans in registry

```
beans {
    bean('foo', org.example.foo.FooBean.class)
    fooClient = new org.example.foo.FooClientBuilder().build()
}
```

The example creates a new instance of `FooBean` with the id `foo` in the Citrus bean registry. The `FooBean` class must have a default constructor in this case, too.

You can also create the instances on your own (e.g. using a builder factory) as seen with the `fooClient` bean in this example.

All beans added to the Citrus registry are subject to being injected into or resolved by test actions and other components in Citrus.

5.4.3. Templates

Templates group action sequences to a logical unit. You can think of templates as reusable components that are used in several tests. The maintenance is much more efficient because you need to apply changes only on the templates and all referenced use cases are updated automatically.

The template gets identified by a unique name. Inside a test case we apply the template referencing this unique name. Have a look at a first example:

echo-template.yaml

```
name: echo
description: Sample template in YAML
parameters:
  - name: message
    value: "Citrus rocks!"
actions:
  - echo:
      message: "${message}"
```

The code example above describes a template called `echo`. Templates hold a sequence of test actions or calls other templates themselves. The template may define a set of parameters that represent customizable user input when applying the template.

The template is available for multiple tests that may apply the template as part of their test action sequence.

Apply templates

```
name: ApplyTemplateTest
author: Christoph
status: FINAL
description: Sample test in YAML
actions:
  - applyTemplate:
      name: echo
  - applyTemplate:
      name: echo
      parameters:
        - name: message
          value: "Hello from Citrus!"
```

The example references the `echo` template and call it as part of the test with `<apply-template/>`. You can customize the template parameter to overwrite the default variable `text` with a custom value.

The example above assumes that you have added the template to the Citrus bean registry with the name `echo`. If this is not the case you may also reference the file resource path when applying the template:

Apply templates

```
name: ApplyTemplateTest
author: Christoph
status: FINAL
description: Sample test in YAML
actions:
  - applyTemplate:
      file: "classpath:templates/yaml/echo-template.yaml"
      parameters:
        - name: message
          value: "Citrus rocks!"
```

Template parameters

How to handle test variables when dealing with templates, in particular when templates are defined in separate source files? A template may use different variable names compared to those names used a test and vice versa. The template fails immediately when respective variables are not present. Therefore, templates introduce the concept of parameters to declare its variable names.

Assume you have the following template:

sayHello template

```
name: sayHello
description: Sample template in YAML
parameters:
  - name: user
    value: "Citrus"
actions:
  - echo:
      message: "Hello ${user}!"
```

The template `sayHello` in the example uses the variable `${user}`.

In case the test case already uses the template parameter as a value you do not need to set this variable explicitly on the apply template action.

Implicit template parameter

```
name: SayHelloTemplateTest
author: Christoph
status: FINAL
description: Sample test in YAML
variables:
  - name: "user"
    value: "Christoph"
actions:
  - applyTemplate:
      name: sayHello
```

Output

```
Hello Christoph!
```



The implicit template parameter may break as soon as the template or the test changes its parameter or variable name.

The test case may also explicitly set the template parameter when applying the template:

Explicit template parameter

```
name: SayHelloTemplateTest
author: Christoph
status: FINAL
description: Sample test in YAML
variables:
  - name: "username"
    value: "Mickey"
actions:
  - applyTemplate:
      name: sayHello
      parameters:
        - name: user
          value: "${username}"
```

Output

```
Hello Mickey!
```

The variable **username** is already present in the test case and gets translated into the **user** parameter. Following from that the template works fine although test and template do work on different variable names.

With template parameters you are able to solve the calibration effort when working with templates and variables. It is always a good idea to check the used variables/parameters inside a template

when calling it. There might be a variable that is not declared yet inside your test. So you need to define this value as a parameter.

Template parameter scope

A template is able to change a test variable value for the rest of the test case. It is important to know about this when changing test variables in a template as it affects the descendant test actions. Especially when running test actions or templates in parallel to each other this might lead to unexpected behavior. You may choose the template parameter scope for this scenario.



When a template works on variable values and parameters changes to these variables will automatically affect the variables in the whole test. So if you change a variable's value inside a template and the variable is defined inside the test case the changes will affect the variable in a global context. We have to be careful with this when executing a template several times in a test, especially in combination with parallel containers (see containers-parallel).

print template

```
name: print
description: Sample template in YAML
parameters:
  - name: index
    value: "0"
  - name: area
    value: "Atlantis"
actions:
  - echo:
      message: "#${index} Hello ${area}!"
```

Global scope parameter (default)

```
name: SayHelloTemplateTest
author: Christoph
status: FINAL
description: Sample test in YAML
variables:
  - name: "username"
    value: "Mickey"
actions:
  - parallel:
      actions:
      - applyTemplate:
          name: print
          parameters:
            - name: index
              value: "1"
            - name: area
              value: "Europe"
      - applyTemplate:
          name: print
          parameters:
            - name: index
              value: "2"
            - name: area
              value: "Asia"
      - applyTemplate:
          name: print
          parameters:
            - name: index
              value: "3"
            - name: area
              value: "Africa"
```

In the listing above a template **print** is called several times in a parallel container. The parameter values will be handled in a global context, so it is quite likely to happen that the template instances influence each other during execution. We might get such print messages:

Output

```
#2 Hello Europe!
#2 Hello Africa!
#3 Hello Africa!
```

Index parameters do not fit and the message **'Hello Asia'** is completely gone. This is because templates overwrite parameters to each other as they are executed in parallel at the same time. To avoid this behavior we need to tell the template that it should handle parameters as well as variables in a local context. This will enforce that each template instance is working on a dedicated local context. See the **globalContext** key that is set to **false** in this example:

print template

```
name: print
description: Sample template in YAML
globalContext: false
parameters:
  - name: index
    value: "0"
  - name: area
    value: "Atlantis"
actions:
  - echo:
      message: "#${index} Hello ${area}!"
```

After that template instances will not influence each other anymore. But notice that variable changes inside the template then do not affect the test case neither.

5.5. Groovy

Running Citrus tests in Groovy is a specialization of running Citrus tests in Java. The Groovy language gives you some syntax sugar and optimized statements that you can use to write your tests.

As usual, you need to choose one of the supported test engines in Citrus to actually run the test (e.g. JUnit Jupiter or TestNG). You can then use the `@CitrusTestSource` annotation to load the Groovy code that represents the test case.

MyFirstGroovyIT.java

```
package org.citrusframework;

import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Test;

public class MyFirstGroovyIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "groovy", name="my-test.groovy")
    public void myFirstGroovyIT() {
    }
}
```

The test logic itself is coded in a Groovy file like this:

my-test.groovy

```
name "MyFirstGroovyTest"
author "Christoph"
status "FINAL"
description "Sample test in Groovy"

variables {
    text="Hello from Citrus!"
}

actions {
    $(echo().message("${text}"))
}
```

5.5.1. Configuration scripts

A Groovy test is able to create endpoints and other configuration entities as part of the test. The script creates these instances as beans in the Citrus registry.

A configuration script may use methods and closures to define the bean instances. The configuration script supports following methods:

- queues()
- endpoints()
- beans()

my-test.groovy

```
configuration {
    queues {
        queue('say-hello')
    }

    endpoints {
        direct('hello')
            .asynchronous()
            .queue('say-hello')
    }

    beans {
        bean(DefaultTextEqualsMessageValidator.class)
    }
}

actions {
    $(send().endpoint(hello)
            .message()
            .body("Hello Citrus!")
    )
}
```

The test above creates a new `direct` endpoint called `hello` that uses an in-memory queue named `say-hello`. Also, the configuration script initializes a new bean instance of type `org.citrusframework.validation.DefaultTextEqualsMessageValidator.class`.

The `send` test action is able to reference the new endpoint by its name to send a message body to the queue. A `receive` test action will automatically resolve the new message validator instance from the Citrus bean registry to perform its message validation.

The Groovy language provides a comfortable way to define common beans in the Citrus registry.

Create beans in registry

```
configuration {
    beans {
        bean('foo', org.example.foo.FooBean.class)
        fooClient = new org.example.foo.FooClientBuilder().build()
    }
}
```

The example creates a new instance of `FooBean` with the id `foo` in the Citrus bean registry. The `FooBean` class must have a default constructor in this case.

You can also create the instances on your own (e.g. using a builder factory) as seen with the `fooClient` bean in this example.

All beans added to the Citrus registry are subject to being injected into or resolved by test actions and other components in Citrus. For instance a JMS test action requires a client connection factory bean instance in the Citrus registry.

5.5.2. Gherkin style syntax

The Gherkin `Given-When-Then` syntax helps you to categorize and structure the individual steps in your test. The Groovy test language in Citrus can use this concept, too.

my-test.groovy

```
configuration {
    queues {
        queue('say-hello')
    }

    endpoints {
        direct('hello') {
            asynchronous()
                .queue('say-hello')
        }
    }
}

given:
    $(createVariables()
        .variable("text", "Citrus rocks!")
    )

when:
    $(send().endpoint(hello)
            .message()
            .body('${text}')
    )

then:
    $(receive().endpoint(hello)
            .message()
            .body('${text}')
    )
```

5.5.3. Message builders

The Groovy language is good choice when it comes to creating domain specific languages. You can provide syntactical sugar to make it as comfortable as possible to use your domain.

As an example you can use message builders to create message body payloads in XML or Json format.

my-test.groovy

```
configuration {
    queues {
        queue('say-hello')
    }

    endpoints {
        direct('hello') {
            asynchronous()
                .queue('say-hello')
        }
    }
}

given:
    $(createVariables()
        .variable("text", "Citrus rocks!"))

when:
    $(send().endpoint(hello)
        .message {
            body {
                json()
                    .greeting {
                        text '${text}'
                        language 'eng'
                    }
            }
            headers {
                operation = "sayHello"
            }
        })

then:
    $(receive().endpoint(hello)
        .message {
            body().json {
                greeting {
                    text '${text}'
                    language 'eng'
                }
            }
            headers {
                operation = "sayHello"
            }
        })
```

The example above deals with Json message payloads and leverages the message builders to create a Json body like this:

body.json

```
{
  "greeting": {
    "text": "${text}",
    "language": "eng"
  }
}
```

The same approach works fine with XML payloads:

my-test.groovy

```
configuration {
    queues {
        queue('say-hello')
    }

    endpoints {
        direct('hello') {
            asynchronous()
                .queue('say-hello')
        }
    }
}

given:
    $(createVariables()
        .variable("text", "Citrus rocks!"))

when:
    $(send().endpoint(hello)
        .message {
            body {
                xml()
                    .greeting(language: 'eng') {
                        text '${text}'
                    }
            }
            headers {
                operation = "sayHello"
            }
        })

then:
    $(receive().endpoint(hello)
        .message {
            body().xml {
                greeting(language: 'eng') {
                    text '${text}'
                }
            }
            headers {
                operation = "sayHello"
            }
        })
```

The test actions use the XML message builder and create XML body payloads such as:

body.xml

```
<greeting language='eng'>
  <text>Citrus rocks!</text>
</greeting>
```

5.5.4. Templates

Templates group action sequences to a logical unit. You can think of templates as reusable components that are used in several tests. The maintenance is much more efficient because you need to apply changes only on the templates and all referenced use cases are updated automatically.

The template gets identified by a unique name. Inside a test case we apply the template referencing this unique name. Have a look at a first example:

echo-template.groovy

```
name "echo"
description "Sample template in Groovy"

parameters {
    message="Citrus rocks!"
}

actions {
    $(echo().message('${message}'))
}
```

The code example above describes a template called `echo`. Templates hold a sequence of test actions or calls other templates themselves. The template may define a set of parameters that represent customizable user input when applying the template.

The template is available for multiple tests that may apply the template as part of their test action sequence.

Apply templates

```
name "ApplyTemplateTest"
author "Christoph"
status "FINAL"
description "Sample test in Groovy"

actions {
    $(applyTemplate().templateName("echo"))

    $(applyTemplate().templateName("echo")
        .parameter("text", "Hello from Citrus!"))
}
```

The example references the `echo` template and call it as part of the test with `applyTemplate()`. You can customize the template parameter to overwrite the default variable `text` with a custom value.

The example above assumes that you have added the template to the Citrus bean registry with the name `echo`. If this is not the case you may also reference the file resource path when applying the template:

Apply templates

```
actions {
    $(applyTemplate().file("classpath:templates/groovy/echo-template.groovy"))
}
```

Template parameters

How to handle test variables when dealing with templates, in particular when templates are defined in separate source files? A template may use different variable names compared to those names used a test and vice versa. The template fails immediately when respective variables are not present. Therefore, templates introduce the concept of parameters to declare its variable names.

Assume you have the following template:

sayHello template

```
name "sayHello"
description "Sample template in Groovy"

parameters {
    user="Citrus"
}

actions {
    $(echo().message('Hello ${user}!'))
}
```

The template `sayHello` in the example uses the variable `${user}`.

In case the test case already uses the template parameter as a value you do not need to set this variable explicitly on the apply template action.

Implicit template parameter

```
name "SayHelloTemplateTest"
description "Sample test in Groovy"

variables {
    user = "Christoph"
}

actions {
    $(applyTemplate().templateName("sayHello"))
}
```

Output

```
Hello Christoph!
```



The implicit template parameter may break as soon as the template or the test changes its parameter or variable name.

The test case may also explicitly set the template parameter when applying the template:

Explicit template parameter

```
name "SayHelloTemplateTest"
description "Sample test in Groovy"

variables {
    username = "Mickey"
}

actions {
    $(applyTemplate().templateName("sayHello")
        .parameter("user", "${username}"))
}
```

Output

```
Hello Mickey!
```

The variable **username** is already present in the test case and gets translated into the **user** parameter. Following from that the template works fine although test and template do work on different variable names.

With template parameters you are able to solve the calibration effort when working with templates and variables. It is always a good idea to check the used variables/parameters inside a template when calling it. There might be a variable that is not declared yet inside your test. So you need to define this value as a parameter.

Template parameter scope

A template is able to change a test variable value for the rest of the test case. It is important to know about this when changing test variables in a template as it affects the descendant test actions. Especially when running test actions or templates in parallel to each other this might lead to unexpected behavior. You may choose the template parameter scope for this scenario.



When a template works on variable values and parameters changes to these variables will automatically affect the variables in the whole test. So if you change a variable's value inside a template and the variable is defined inside the test case the changes will affect the variable in a global context. We have to be careful with this when executing a template several times in a test, especially in combination with parallel containers (see containers-parallel).

print template

```
name "print"
description "Sample template in Groovy"

parameters {
    index = "0"
    area = "Atlantis"
}

actions {
    $(echo().message('#${index} Hello ${area}!'))
}
```

Global scope parameter (default)

```
actions {
    $(parallel().actions(
        applyTemplate().templateName("print")
            .parameter("index", "1")
            .parameter("area", "Europe"),
        applyTemplate().templateName("print")
            .parameter("index", "2")
            .parameter("area", "Asia"),
        applyTemplate().templateName("print")
            .parameter("index", "3")
            .parameter("area", "Africa")
        )
    )
}
```

In the listing above a template **print** is called several times in a parallel container. The parameter values will be handled in a global context, so it is quite likely to happen that the template instances influence each other during execution. We might get such print messages:

Output

```
#2 Hello Europe!
#2 Hello Africa!
#3 Hello Africa!
```

Index parameters do not fit and the message **'Hello Asia'** is completely gone. This is because templates overwrite parameters to each other as they are executed in parallel at the same time. To avoid this behavior we need to tell the template that it should handle parameters as well as variables in a local context. This will enforce that each template instance is working on a dedicated local context. See the **globalContext()** method that is set to **false** in this example:

print template

```
name "print"
description "Sample template in Groovy"

globalContext(false)

parameters {
    index = "0"
    area = "Atlantis"
}

actions {
    $(echo().message('#${index} Hello ${area}!'))
}
```

After that template instances will not influence each other anymore. But notice that variable changes inside the template then do not affect the test case neither.

5.6. Spring Bean XML



The Spring bean XML domain specific language for writing tests is deprecated. Please consider using the arbitrary XML test language syntax instead (see chapter Run XML tests])

The Spring bean XML domain specific language is a variation of the arbitrary XML domain specific language in Citrus. The Spring XML file holds all test actions in the form of Spring XML beans.



You need to have the runtime module `citrus-spring` enabled in your project to use the Spring bean XML test case language.

Citrus uses Spring bean definition parsers to load the XML test actions. The Spring application context loads the Spring XML test case file as a set of Spring bean configurations. Citrus runs the resulting test Spring bean as a normal unit test using one of the provided runtimes like JUnit Jupiter or TestNG.

You define the complete test logic in the Spring bean XML file:

MyFirstCitrus_IT.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<spring:beans
    xmlns="http://www.citrusframework.org/schema/testcase"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:spring="http://www.springframework.org/schema/beans"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/testcase
            http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <testcase name="MyFirstCitrus_IT">
      <description>
          First example showing the basic test case definition elements!
      </description>
      <variables>
        <variable name="text" value="Hello Test Framework"/>
      </variables>
      <actions>
        <echo>
          <message>${text}</message>
        </echo>
      </actions>
    </testcase>
</spring:beans>
```

All Spring XML test definitions use a custom Spring XML schema that aims to reach the convenience of a domain specific language (DSL).

The definition uses the `<spring:beans>` root element that declares all XML namespaces used in the file. This is because the XML file will be loaded as a Spring bean definition. The root element defines a `testcase` element which represents the actual Citrus test.

The test case itself gets a mandatory name that must be unique throughout all test cases in a project. You will receive errors when using duplicate test names. The test name has to follow the common Java naming conventions and rules for Java classes. This means names must not contain whitespace characters except `'-'`, `'.'` and `'_'`.

For example, ***TestFeature_1*** is valid but ***Test Feature 1*** is not because of the space characters.

5.6.1. Java test runner

Despite the fact that the complete test logic is defined in XML we need a Java class to actually load the XML file and run the test as part of JUnit, Cucumber or TestNG depending on the test engine you choose. The Java class acts as a wrapper to load the XML file and execute the test.

The Spring XML test case definition in Citrus uses two files that are connected via naming

conventions. In a Maven project these two files would be:

XML test case files

```
src/test/java/org/citrusframework/MyFirstCitrus_IT.java
src/test/resources/org/citrusframework/MyFirstCitrus_IT.xml
```

The files above represent a test called `MyFirstCitrus_IT`. The `.java` file defines the runtime that should be used to execute the test. This Java file does not have any test logic and is not likely to be changed. In fact, you can generate the Java file from Citrus (e.g. Maven plugin).

Generate test files via Maven plugin

```
mvn citrus:create-test
```

Interactive test creation

```
Enter test name:: MyFirstCitrus_IT
Enter test author: Unknown::
Enter test description:: Sample XML test
Enter test package: org.citrusframework::
Choose unit test framework: (testng/junit/junit-jupiter/junit4) testng::
Choose target code base type: (java/xml) java:: xml
Create test with XML schema? (y/n) n::
Create test with WSDL? (y/n) n::
Create test with Swagger API? (y/n) n::

Confirm test creation:
type: xml
framework: testng
name: MyFirstCitrus_IT
author: Unknown
description:
package: org.citrusframework
 (y/n) y:: y

[INFO] Successfully created new test case org.citrusframework.MyFirstCitrus_IT
```

The command above creates the two test files (`.java` and `.xml`) using an interactive mode. The user provides the test case information such as test name, package, runtime and so on.

MyFirstCitrus_IT.java

```
package org.citrusframework;

import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTestSource;
import org.citrusframework.testng.TestNGCitrusSupport;
import org.testng.annotations.Optional;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class MyFirstCitrus_IT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTestSource(type = "spring", name="MyFirstCitrus_IT")
    public void myFirstCitrus_IT() {
    }
}
```

The purpose of the generated Java class is to load the XML file as a Spring bean test and execute the test case.

The default naming convention requires the XML file with the test name in the same package as the generated Java class. This makes sure that the Java class is able to find the XML file when loading and running the test.

5.6.2. Test meta information

The user is able to provide some additional information about the test case. The meta-info section at the very beginning of the test case holds information like author, status or creation date.

Test meta information

```
<testcase name="metaInfoTest">
    <meta-info>
        <author>Christoph Deppisch</author>
        <creationdate>2008-01-11</creationdate>
        <status>FINAL</status>
        <last-updated-by>Christoph Deppisch</last-updated-by>
        <last-updated-on>2008-01-11T10:00:00</last-updated-on>
    </meta-info>
    <description>
        ...
    </description>
    <actions>
        ...
    </actions>
</testcase>
```

The status allows the following values:

- DRAFT
- READY_FOR_REVIEW
- DISABLED
- FINAL

This information gives the reader first impression about the test and is also used to generate test documentation. By default, Citrus is able to generate test reports in HTML and Excel in order to list all tests with their metadata information and description.



Tests with the status DISABLED will not be executed during a test suite run. So someone can just start adding planned test cases that are not finished yet in status DRAFT. In case a test is not runnable yet because it is not finished, someone may disable a test temporarily to avoid causing failures during a test run.

The test description should give a short introduction to the intended use case scenario that will be tested. The user should get a short summary of what the test case is trying to verify. You can use free text in your test description no limit to the number of characters. Please be aware of the XML validation rules of well-formed XML (e.g. special character escaping). The usage of CDATA sections for large descriptions may be a good idea, too.

5.6.3. Finally block

Java developers might be familiar with the concept of try-catch-finally blocks. The ***finally*** section contains a list of test actions that will be executed guaranteed at the very end of the test case even if errors did occur during the execution before.

This is the right place to tidy up things that were previously created by the test like cleaning up the database for instance.

Finally block

```
<finally>
    <echo>
        <message>Do finally - regardless of what has happened before</message>
    </echo>
</finally>
```

As an example imagine that you have prepared some data inside the database at the beginning of the test and you need to make sure the data is cleaned up at the end of the test case.

Finally block example

```
<testcase name="finallyTest">
    <variables>
        <variable name="orderId" value="citrus:randomNumber(5)"/>
        <variable name="date" value="citrus:currentDate('dd.MM.yyyy')"/>
    </variables>
    <actions>
        <sql datasource="testDataSource">
            <statement>
                INSERT INTO ORDERS VALUES (${orderId}, 1, 1, '${date}')
            </statement>
        </sql>

        <echo>
            <message>
                ORDER creation time: ${date}
            </message>
        </echo>
    </actions>
    <finally>
        <sql datasource="testDataSource">
            <statement>
              DELETE FROM ORDERS WHERE ORDER_ID='${orderId}'
            </statement>
        </sql>
    </finally>
</testcase>
```

In the example the first action creates an entry in the database using an ***INSERT*** statement. To be sure that the entry in the database is deleted after the test, the finally section contains the respective ***DELETE*** statement that is always executed regardless the test case state (successful or failed).

5.6.4. Variables with CDATA sections

When using the XML test definition you must obey the XML rules for variable values. First of all you need to escape XML reserved characters such as `<`, `&` or `"` with `<`, `&` or `"`. Other values such as XML snippets would also interfere with the XML well-formed paradigm. You can use CDATA sections within the variable value element as a solution.

Variable CDATA sections

```
<variables>
  <variable name="persons">
    <value>
      <data>
        <![CDATA[
          <persons>
            <person>
              <name>Theodor</name>
              <age>10</age>
            </person>
            <person>
              <name>Alvin</name>
              <age>9</age>
            </person>
          </persons>
        ]]>
      </data>
    </value>
  </variable>
</variables>
```

That is how you can use structured variable values in the XML DSL.

5.6.5. Variables with Groovy

You can also use a script to create variable values. This is extremely handy when you have very complex variable values. Just code a small Groovy script for instance in order to define the variable value. A small sample should give you the idea how that works:

Groovy variable script

```
<variables>
  <variable name="avg">
    <value>
      <script type="groovy">
        <![CDATA[
          a = 4
          b = 6
          return (a + b) / 2
        ]]>
      </script>
    </value>
  </variable>
  <variable name="sum">
    <value>
      <script type="groovy">
        <![CDATA[
          5 + 5
        ]]>
      </script>
    </value>
  </variable>
</variables>
```

Just use the script code right inside the variable value definition. The value of the variable is the result of the last operation performed within the script. For longer script code the use of `<![CDATA[ ]]>` sections is recommended.

Citrus uses the JavaScript engine mechanism to evaluate the script code. By default, Groovy is supported as a script engine implementation. You can add additional engine implementations to your project and support other script types, too.

5.6.6. Templates

Templates group action sequences to a logical unit. You can think of templates as reusable components that are used in several XML tests. The maintenance is much more efficient because you need to apply changes only on the templates and all referenced use cases are updated automatically.

The template always has a unique name. Inside a test case we call the template by this unique name. Have a look at a first example:

XML templates

```
<template name="doCreateVariables">
    <create-variables>
        <variable name="var" value="123456789"/>
    </create-variables>

    <call-template name="doTraceVariables"/>
</template>

<template name="doTraceVariables">
    <echo>
        <message>Current time is: ${time}</message>
    </echo>

    <trace-variables/>
</template>
```

The code example above describes two template definitions. Templates hold a sequence of test actions or call other templates themselves as seen in the example above.



The `<call-template>` action calls other templates by their name. The called template not necessarily has to be located in the same test case XML file. The template might be defined in a separate XML file other than the test case itself:

Call XML templates

```
<testcase name="templateTest">
    <variables>
        <variable name="myTime" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <call-template name="doCreateVariables"/>

        <call-template name="doTraceVariables">
            <parameter name="time" value="${myTime}">
        </call-template>
    </actions>
</testcase>
```

There is an open question when dealing with templates that are defined somewhere else outside the test case. How to handle variables? A templates may use different variable names then the test and vice versa. No doubt the template will fail as soon as special variables with respective values are not present. Unknown variables cause the template and the whole test to fail with errors.

So a first approach would be to harmonize variable usage across templates and test cases, so that templates and test cases do use the same variable naming. But this approach might lead to high calibration effort. Therefore, templates support parameters to solve this problem. When a template is called the calling actor is able to set some parameters. Let us discuss an example for this issue.

The template "doDateConversion" in the next sample uses the variable ${date}. The calling test case can set this variable as a parameter without actually declaring the variable in the test itself:

Template parameter

```
<call-template name="doDateConversion">
    <parameter name="date" value="${sampleDate}"/>
</call-template>
```

The variable **sampleDate** is already present in the test case and gets translated into the **date** parameter. Following from that the template works fine although test and template do work on different variable namings.

With template parameters you are able to solve the calibration effort when working with templates and variables. It is always a good idea to check the used variables/parameters inside a template when calling it. There might be a variable that is not declared yet inside your test. So you need to define this value as a parameter.

Template parameters may contain more complex values like XML fragments. The call-template action offers following CDATA variation for defining complex parameter values:

Complex parameter values

```
<call-template name="printXMLPayload">
    <parameter name="payload">
      <value>
        <![CDATA[
          <HelloRequest
xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
            <Text>Hello South ${var}</Text>
          </HelloRequest>
        ]]>
      </value>
    </parameter>
</call-template>
```



When a template works on variable values and parameters changes to these variables will automatically affect the variables in the whole test. So if you change a variable's value inside a template and the variable is defined inside the test case the changes will affect the variable in a global context. We have to be careful with this when executing a template several times in a test, especially in combination with parallel containers (see containers-parallel).

Global scope parameter

```
<parallel>
    <call-template name="print">
        <parameter name="param1" value="1"/>
        <parameter name="param2" value="Hello Europe"/>
    </call-template>
    <call-template name="print">
        <parameter name="param1" value="2"/>
        <parameter name="param2" value="Hello Asia"/>
    </call-template>
    <call-template name="print">
        <parameter name="param1" value="3"/>
        <parameter name="param2" value="Hello Africa"/>
    </call-template>
</parallel>
```

In the listing above a template **print** is called several times in a parallel container. The parameter values will be handled in a global context, so it is quite likely to happen that the template instances influence each other during execution. We might get such print messages:

Output

```
2. Hello Europe
2. Hello Africa
3. Hello Africa
```

Index parameters do not fit and the message **'Hello Asia'** is completely gone. This is because templates overwrite parameters to each other as they are executed in parallel at the same time. To avoid this behavior we need to tell the template that it should handle parameters as well as variables in a local context. This will enforce that each template instance is working on a dedicated local context. See the **global-context** attribute that is set to **false** in this example:

Local scope parameter

```
<template name="print" global-context="false">
    <echo>
        <message>${param1}.${param2}</message>
    </echo>
</template>
```

After that template instances won't influence each other anymore. But notice that variable changes inside the template then do not affect the test case neither.

Chapter 6. Test variables

The usage of test variables is a core concept when writing maintainable tests. The key identifiers of a test case should be exposed as test variables at the very beginning of a test. This avoids hard coded identifiers and multiple redundant values inside the test.

Java

```
public void fooService_IT() {
    variable("text", "Hello Citrus!");
    variable("customerId", "123456789");

    run(echo("Text: ${text} Id: ${id}"));
}
```

XML

```
<testcase name="FooService_IT">
  <variables>
    <variable name="text" value="Hello Citrus!"/>
    <variable name="customerId" value="123456789"/>
  </variables>

  <actions>
    <echo>
      <message>Text: ${text} Id: ${id}</message>
    </echo>
  </actions>
</testcase>
```

Test variables help significantly when writing complex tests with lots of identifiers and semantic data. The variables are valid for the whole test case. You can reference a variable multiple times using a common variable expression `${variable-name}`.

The usage of variables should make the test easier to maintain and more flexible. All essential entities and identifiers are present right at the beginning of the test, which may also give the opportunity to easily create test variants by simply changing the variable values for other test scenarios (e.g. different error codes, identifiers).

The name of the variable is arbitrary. Of course, you need to be careful with special characters and reserved XML entities like '&', '<', '>'. In general, you can apply to the Java naming convention, and you will be fine.

6.1. Global variables

You can work with different variable scopes (local or global). Local variables are accessible throughout a single test. Global variables are visible for all tests yet global variables are immutable, so tests cannot change their values.

This is a good opportunity to declare constant values for all tests. As these variables are global we need to add those to the basic Citrus context. The following example demonstrates how to add global variables in Citrus:

Citrus Bean

```
@BindToRegistry
public GlobalVariables globalVariables() {
    return new GlobalVariables.Builder()
        .variable("projectName", "Citrus Integration Testing")
        .variable("userName", "TestUser")
        .build();
}
```

Spring Bean

```
@Bean
public GlobalVariables globalVariables() {
    return new GlobalVariables.Builder()
        .variable("projectName", "Citrus Integration Testing")
        .variable("userName", "TestUser")
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <citrus:global-variables>
      <citrus:variable name="projectName" value="Citrus Integration Testing"/>
      <citrus:variable name="userName" value="TestUser"/>
    </citrus:global-variables>
</beans>
```

We add the Spring bean component to the application context file. The component receives a list of name-value variable elements. You can reference the global variables in your test cases as usual.

Another possibility to set global variables is to load those from external property files. This may give you more powerful global variables with user specific properties for instance. See how to load property files as global variables in this example:

Citrus Bean

```
@BindToRegistry
public GlobalVariablesPropertyLoader propertyLoader() {
    GlobalVariablesPropertyLoader propertyLoader = new
GlobalVariablesPropertyLoader();

    propertyLoader.getPropertyFiles().add("classpath:global-variable.properties");

    return propertyLoader;
}
```

Spring Bean

```
@Bean
public GlobalVariablesPropertyLoader propertyLoader() {
    GlobalVariablesPropertyLoader propertyLoader = new
GlobalVariablesPropertyLoader();

    propertyLoader.getPropertyFiles().add("classpath:global-variable.properties");

    return propertyLoader;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <citrus:global-variables>
      <citrus:file path="classpath:global-variable.properties"/>
    </citrus:global-variables>
</beans>
```

You can use the `GlobalVariablesPropertyLoader` component and add it to the context as a Spring bean. Citrus loads the given property file content as global test variables. You can mix property file and name-value pair variable definitions in the global variables component.



The global variables can have variable expressions and Citrus functions. It is possible to use previously defined global variables as values of new variables, like in this example:

global-variable.properties

```
user=Citrus
greeting=Hello ${user}!
date=citrus:currentDate('yyyy-MM-dd')
```

6.2. Extract variables

Imagine you receive a message in your test with some generated message identifier values. You have no chance to predict the identifier value because it was generated at runtime by a foreign application. You can ignore the value in order to protect your validation. In many cases you might want to save this identifier in order to use this value in the respective response message or somewhat later on in the test.

The solution is to extract dynamic values from received messages and save those to test variables at runtime.

6.2.1. JsonPath expressions

When an incoming message is passing the message validation the user can extract some values of that received message to new test variables for later use in the test.

```
<message type="json">
  <data>
    { "user":
      {
        "name": "Admin",
        "password": "secret",
        "admin": "true",
        "aliases": ["penny","chef","master"]
      }
    }
  </data>
  <extract>
    <message path="$.user.name" variable="userName"/>
    <message path="$.user.aliases" variable="userAliases"/>
    <message path="$.user[?(@.admin)].password" variable="adminPassword"/>
  </extract>
</message>
```

With this example we have extracted three new test variables via JSONPath expression evaluation. The three test variables will be available to all upcoming test actions. The variable values are:

```
userName=Admin
userAliases=["penny","chef","master"]
adminPassword=secret
```

As you can see we can also extract complex JSONObject items or JSONArray items. The test variable value is a String representation of the complex object.

6.2.2. XPath expressions

Add this code to your message receiving action.

Java

```
@CitrusTest
public void receiveMessageTest() {
    when(
        receive("helloService")
            .extract(extract().fromBody()
                    .expression("//TestRequest/VersionId", "versionId"))
            .extract(extract().fromHeaders()
                    .header("Operation", "operation"))
    );

    then(
        echo("Operation from header is: ${operation}")
    );

    then(
        echo("Version from body is: ${versionId}")
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="helloService">
        <message>
          <body>
              <data>[...]</data>
          </body>
        </message>
        <extract>
          <header name="Operation"
                  variable="operation"/>
          <body path="/TestRequest/VersionId"
                variable="versionId"/>
        </extract>
      </receive>

      <echo>
        <message>Operation from header is: ${operation}</message>
      </echo>

      <echo>
        <message>Version from body is: ${versionId}</message>
      </echo>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: helloService
      message:
        body:
          data: {}
        extract:
          header:
            - name: "Operation"
              variable: "operation"
          body:
            - path: "/TestRequest/VersionId"
              variable: "versionId"
  - echo:
      message: "Operation from header is: ${operation}"
  - echo:
      message: "Version from body is: ${versionId}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="ReceiveMessageTest">
    <actions>
        <receive endpoint="helloService">
            <message>
              <data>[...]</data>
            </message>
            <extract>
              <header name="Operation"
                      variable="operation"/>
              <message path="/TestRequest/VersionId"
                       variable="versionId"/>
            </extract>
        </receive>

        <echo>
          <message>Operation from header is: ${operation}</message>
        </echo>

        <echo>
          <message>Version from body is: ${versionId}</message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

As you can see Citrus is able to extract both header and message body content into test variables. The extraction will automatically create a new variable in case it does not exist. The time the variable was created all following test actions can access the test variables as usual. So you can reference the variable values in response messages or other test steps ahead.



We can also use expression result types in order to manipulate the test variable outcome. In case we use a **boolean** result type the existence of elements can be saved to variable values. The result type **node-set** translates a node list result to a comma separated string of all values in this node list. Simply use the expression result type attributes as shown in previous sections.

6.3. Object graphs and the scenario container

A test variable can hold any Java object, not just strings. Store a rich object with `setVariable` (or `variable(…)` in the Java DSL) and traverse its fields, list/array elements, and map entries directly in `${…}` expressions. A common pattern is to keep all scenario-lifetime data under a single `container`

variable — sometimes called the scenario container or data container — so every test step reads from one predictable root instead of scattered variables:

```
 Scattered: separate variables per domain, static helpers, ad-hoc context entries
 Container:  container
              ├──── customer   → Customer entity (Java object)
              ├──── order      → Order entity (Java object)
              ├──── currencies → reference data (Java List)
              └──── campaign   → runtime-created data (Java Map or object)
```

```
context.setVariable("container", scenarioContainer);
echo("Customer id is ${container.customer.id}, first currency is
${container.currencies[0].code}");
```



Never store credentials, tokens, or secrets in the container. It lives in plain memory in the test context and is designed for business fixtures, not sensitive material. Container data should be treated as read-only once published: write it during setup, then only read it during test execution.

6.3.1. How Citrus resolves path expressions

When Citrus encounters `${container.customer.id}`, it walks the path segment by segment:

1. **Root resolution**: find the `container` variable in the `TestContext`.
2. **Segment parsing**: split the remainder into segments: `customer` → `id`.
3. **Reflection traversal**: for `customer`, read the `customer` field of the container object; for `id`, read the `id` field of the customer object. Each step uses the declared or inherited Java field (including `static` fields).
4. **Type conversion**: convert the final value to a string for expression substitution.

Resolution rules by segment type:

| Segment | Syntax | What Citrus does | Example | |---|---|---|---| | Object field | `.fieldName` | Reads the declared or inherited Java field via reflection | `${container.customer}` | | Nested object | `.field.subfield` | Traverses into the referenced object | `${container.customer.address.city}` | | Static field | `.CONSTANT` | Reads a `static` field | `${container.data.CONSTANT}` | | Array or `List` index | `[N]` | Reads element `N` (`Array.get` for arrays, `List.get` for lists); `N` may have multiple digits | `${container.currencies[12].code}` | | Map key | `.keyName` | Reads `map.get(keyName)` | `${container.campaign.id}` |

The index grammar is strict: `[N]` holds one or more ASCII digits only — no signs, no spaces, one index per segment (`matrix[0][1]` is not supported). Indexes beyond the `int` range fail with a Citrus error. Resolution uses fields, not property getters: a value exposed only through `getX()` with no backing field `x` cannot be resolved.

What Citrus CAN resolve:

```
 Direct field access:      ${container.customer}
 Nested field access:      ${container.customer.id}
 Array indexing:           ${container.intVals[1]}
 List indexing:            ${container.currencies[0].code}
 Multi-digit indexes:      ${container.currencies[12].code}
 Nested array/list+field:  ${containerArray[3].data[1].data}
 Map key access:           ${container.campaign.id}
 Static final fields:      ${container.data.CONSTANT}
 Inherited fields:         (fields from parent classes)
```

What Citrus CANNOT resolve:

```
 Method invocation:        ${container.getCustomer().getId()}
 Conditional expressions:  ${container.customer != null ? ...}
 Lambda/functional:        ${container.list.stream().filter(...)}
 Constructor calls:        ${new String("test")}
 Sets and other collections (only arrays and java.util.List support [N])
```

6.3.2. Worked example

Given a holder class such as the `DataContainer` used in the Citrus test suite:

```
private static class DataContainer {
    private final int number = 99;
    private final Object data;
    private final int[] intVals = new int[] {0, 1, 2, 3, 4};
    private final List<String> strVals = List.of("A", "B", "C");
    private static final String CONSTANT = "FOO";

    public DataContainer(Object data) {
        this.data = data;
    }
}
```

Field	Type	Notes
`number`	`int`	Primitive, always accessible
`data`	`Object`	Can hold any type, including nested holders for deep traversal
`intVals`	`int[]`	Primitive array, supports `[N]`
`strVals`	`List<String>`	List, supports `[N]` exactly like an array
`CONSTANT`	`String`	`static final`, accessible by name

Resolution walkthrough with `context.setVariable("container", new DataContainer(new DataContainer("nested")))`:

```
${container.number}           → 99                                  (direct field)
${container.data.data}        → "nested"                            (nested traversal)
${container.data.number}      → 99                                  (field of nested
object)
${container.data.CONSTANT}    → "FOO"                               (static field)
${container.intVals[1]}       → 1                                   (array index)
${container.strVals[1]}       → "B"                                 (list index)
${containerArray[3].data[1].data} → "B"                             (index composes
with nesting)
```

6.3.3. Type conversion

The resolved value is converted to a string (`String.valueOf`, or `toString()` for objects):

| Java type | Result | Example | |---|---|---| | `String` | As-is | `"hello"` → `"hello"` | | `int`, `long`, `double`, wrappers | `String.valueOf()` | `99` → `"99"` | | `boolean` | `String.valueOf()` | `true` → `"true"` | | `LocalDate` and other objects | `toString()` | `2024-01-15` → `"2024-01-15"` |

When you need the raw object instead of its string form, use `TestContext.getVariableObject("container.customer")`.

6.3.4. Errors

Unknown segments fail fast with a `CitrusRuntimeException`:

- Unknown variable root (`${nope.id}`), unknown field (`${container.bogus}`), or an indexed segment on a value that is neither an array nor a `List` (`Expected an instance of Array type…`).
- A field that exists but currently holds `null` also fails: guard optional data in your test (e.g. assert presence first) rather than relying on the exception. The same applies to missing map keys.
- Out-of-bounds indexes propagate the JDK exception (`IndexOutOfBoundsException` for lists, `ArrayIndexOutOfBoundsException` for arrays).

6.3.5. Best practices

- Use direct expressions for values that must exist (`${container.customer.id}`) — a failure then means a real bug.
- Keep container data read-only after publication: publish once during setup, read many times.
- Use descriptive names matching your domain (`${container.customer.name}`, not `${container.c.name}`).
- Verify the container is populated before access; a clear setup assertion beats a cryptic resolution error later.

6.3.6. Troubleshooting

- `Unknown variable 'container'` — the container has not been published to the `TestContext` yet. Check that setup ran before the failing step.

- `Unknown segment-value for segment 'X'` — the field name is misspelled, does not exist on the entity, or currently holds `null`. Check the entity class for the exact camelCase field name.
- `Expected an instance of Array type…` — `[N]` was applied to a value that is neither an array nor a `List` (e.g. a `Set` or a plain object). Only arrays and `java.util.List` support indexed access.
- Stale data from a previous test — variables live for the whole test case, so republish the container per test instead of reusing it across tests.

6.4. Path expressions

Some elements in message body might be of dynamic nature. Just think of generated identifiers or timestamps. This is the right time to use test variables and dynamic message element overwrite. You can overwrite specific elements in the message body with path expressions (XPath or JsonPath).

6.4.1. JsonPath expressions

First thing we want to do with JsonPath is to manipulate a message content before it is actually processed. This is very useful when working with message file resources that are reused across multiple test cases. Each test case can manipulate the message content individually with JsonPath before processing the message content.

Let's have a look at this simple sample Json message body:

Json message body user.json

```
{ "user":
  {
    "id": citrus:randomNumber(10),
    "name": "Unknown",
    "admin": "?",
    "projects":
      [{
        "name": "Project1",
        "status": "open"
      },
      {
        "name": "Project2",
        "status": "open"
      },
      {
        "name": "Project3",
        "status": "closed"
      }]
  }
}
```

Citrus can load the file content and uses it as message body when sending or receiving messages in a test case. You can apply JsonPath expressions in order to manipulate the message content.

```
<message type="json">
  <resource file="file:path/to/user.json" />
  <element path="$.user.name" value="Admin" />
  <element path="$.user.admin" value="true" />
  <element path="$..status" value="closed" />
</message>
```

When all path expressions are evaluated the resulting message looks like follows:

```
{ "user":
  {
    "id": citrus:randomNumber(10),
    "name": "Admin",
    "admin": "true",
    "projects":
      [{
        "name": "Project1",
        "status": "closed"
      },
      {
        "name": "Project2",
        "status": "closed"
      },
      {
        "name": "Project3",
        "status": "closed"
      }]
  }
}
```

The JsonPath expressions set the username to **Admin** . The **admin** boolean property was set to **true** and all project status values were set to **closed**. In case a JsonPath expression should fail to find a matching element within the message structure the test case will fail.

With this JsonPath mechanism you are able to manipulate message content before it is sent or received within Citrus. This makes life very easy when using message resource files that are reused across multiple test cases.

6.4.2. XPath expressions

In case of XML message bodies you can use XPath expressions to manipulate the body content before any message processing takes place.

Java

```
@CitrusTest
public void receiveMessageTest() {
    receive("helloService")
        .message()
            .header("Operation", "sayHello")
            .body("""
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
              <MessageId>${messageId}</MessageId>
              <CreatedAt>?</CreatedAt>
              <VersionId>${version}</VersionId>
            </TestMessage>
            """)
            .process(processor().path()
                .expression("/TestMessage/CreatedAt", "${date}"));
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="helloService">
        <message>
          <headers>
            <header name="Operation" value="sayHello"/>
          </headers>
          <body>
            <payload>
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                  <MessageId>${messageId}</MessageId>
                  <CreatedAt>?</CreatedAt>
                  <VersionId>${version}</VersionId>
                </TestMessage>
            </payload>
          </body>
          <expression path="/TestMessage/CreatedAt" value="${date}"/>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: helloService
      message:
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
              <MessageId>${messageId}</MessageId>
              <CreatedAt>?</CreatedAt>
              <VersionId>${version}</VersionId>
            </TestMessage>
        expressions:
          - path: "/TestMessage/CreatedAt"
            value: "${date}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="ReceiveMessageTest">
    <actions>
        <receive endpoint="helloService">
            <message>
              <payload>
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                  <MessageId>${messageId}</MessageId>
                  <CreatedAt>?</CreatedAt>
                  <VersionId>${version}</VersionId>
                </TestMessage>
              </payload>
              <element path="/TestMessage/CreatedAt" value="${date}"/>
            </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

The program listing above shows ways of setting variable values inside a message template. First you can simply place variable expressions inside the message (see how `${messageId}` is used in the sample). In addition to that you can also use path expressions to explicitly overwrite message elements before message processing takes place.

The sample above uses an XPath expression that evaluates and searches for the right element in the

message body in order to set the given value. The previously defined variable **${date}** replaces the respective element value. Of course this works with XML attributes too (e.g. path expression `/TestMessage/Person/@age`).

Both ways via XPath or JsonPath or inline variable expressions are equal to each other. With respect to the complexity of XML namespaces and XPath you may find the inline variable expression more comfortable to use. Anyway feel free to choose the way that fits best for you.

This is how you can overwrite values in message templates in order to increase maintainability and robustness of your test.



Validation matchers put validation mechanisms to a new level offering dynamic assertion statements for validation. Have a look at the possibilities with assertion statements in validation-matcher.

6.5. Escape variables

The test variable expression syntax `${variable-name}` is preserved to evaluate to a test variable within the current test context. In case the same syntax is used in one of your message content values you need to escape the syntax from being interpreted as test variable expression. You can do this by using the variable expression escaping character sequence `//` wrapping the actual variable name like this:

Plain text message content with escapes

```
This is an escaped variable expression ${//escaped//} and should not lead to unknown
variable exceptions within Citrus.
```

The escaped expression `${//escaped//}` above will result in the string **${escaped}** where *escaped* is not treated as a test variable name but as a normal string in the message body.

This way you are able to have the same variable syntax in a message content without interfering with the Citrus variable expression syntax. As a result Citrus will not complain about not finding the test variable **escaped** in the current context.

The variable syntax escaping characters `//` are automatically removed when the expression is processed by Citrus. So we will get the following result after processing.

Parsed plain text mesage content

```
This is an escaped variable expression ${escaped} and should not lead to unknown
variable exceptions within Citrus.
```

Chapter 7. Message validation

When Citrus receives a message from external applications it is time to verify the message content. This message validation includes syntax rules with schema validation and message content comparison to expected templates. Citrus provides powerful message validation capabilities for different data formats. The tester is able to define expected message headers and body content. The Citrus message validator finds values not matching the expectations and reports the difference as test failure.

7.1. Validation registry

Citrus provides default message validator implementations for different data formats. The Citrus project context automatically loads these default message validators. In case one of these message validators matches the incoming message the message validator performs its validation steps with the message.

All default message validators can be overwritten by binding a component with the same id to the project context (e.g. as Spring bean in the application context).

The default message validator implementations of Citrus are:

defaultXmlMessageValidator	org.citrusframework.validation.xml.DomXmlMessageValidator
defaultXpathMessageValidator	org.citrusframework.validation.xml.XpathMessageValidator
defaultJsonMessageValidator	org.citrusframework.validation.json.JsonTextMessageValidator
defaultJsonPathMessageValidator	org.citrusframework.validation.json.JsonPathMessageValidator
defaultPlaintextMessageValidator	org.citrusframework.validation.text.PlainTextMessageValidator
defaultYamlMessageValidator	org.citrusframework.validation.yaml.YamlMessageValidator
defaultMessageHeaderValidator	org.citrusframework.validation.DefaultMessageHeaderValidator
defaultBinaryBase64MessageValidator	org.citrusframework.validation.text.BinaryBase64MessageValidator

defaultGzipBinaryBase64MessageValidator	org.citrusframework.validation.text.GzipBinaryBase64MessageValidator
defaultXhtmlMessageValidator	org.citrusframework.validation.xhtml.XhtmlMessageValidator
defaultGroovyXmlMessageValidator	org.citrusframework.validation.script.GroovyXmlMessageValidator
defaultGroovyTextMessageValidator	org.citrusframework.validation.script.GroovyScriptMessageValidator
defaultGroovyJsonMessageValidator	org.citrusframework.validation.script.GroovyJsonMessageValidator



You can overwrite a default message validator with a custom implementation. Just add your customized validator implementation as a bean to the Citrus context and use one of the default bean identifiers.

You can add a custom message validator as a component in the context (e.g. as Spring bean in the application context).

Citrus Bean

```
@BindToRegistry
public CustomMessageValidator customMessageValidator() {
    return new CustomMessageValidator();
}
```

Spring Bean

```
@Bean
public CustomMessageValidator customMessageValidator() {
    return new CustomMessageValidator();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="customMessageValidator"
          class="org.citrusframework.validation.custom.CustomMessageValidator"/>
</beans>
```

The listing above adds a custom message validator implementation. The message validator registry will automatically add this validator to the list of available validators in the project.

The custom implementation class has to implement the basic interface **org.citrusframework.validation.MessageValidator**<>. Now Citrus will try to match the custom implementation to incoming message types and occasionally execute the message validator logic when applicable.

7.2. Validation modules

The list of available message validators in your project is controlled by the available message validator implementations on the project classpath.

You need to add validator modules to the project accordingly. For instance if you want to use the default Json message validation capabilities in Citrus you need to add the following dependency:

Json validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-json</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

This adds the Citrus message validator component for dealing with Json message format. This includes message validators and JsonPath support. Now your Citrus project is able to validate Json messages.

Citrus provides the following validation modules:

- citrus-validation-json
- citrus-validation-xml
- citrus-validation-yaml
- citrus-validation-text
- citrus-validation-binary

- citrus-validation-groovy
- citrus-validation-hamcrest
- citrus-validation-assertj (see its assertion reference for every assertion usable in `@assertj()@`)

Read more about the individual validation modules in the next sections.

7.3. Json validation

Message formats such as Json have become very popular, in particular when dealing with RESTful services. Citrus is able to expect and validate Json messages with a powerful comparison of Json structures.



By default, Citrus will use XML message formats when sending and receiving messages. This also reflects to the message validation logic Citrus uses for incoming messages. So by default Citrus will try to parse the incoming message as XML DOM element tree. In case we would like to enable Json message validation we have to tell Citrus that we expect a Json message right now.

Json message validation is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

Json validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-json</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Citrus provides several default message validator implementations for Json messages:

JsonTextMessageValidator — Basic Json message validator implementation compares Json objects/arrays (expected and received).

- The order of Json object properties can differ, whereas array order is always validated.
- Test variables and ignored-placeholders (`@ignore@`) can be used.
- JsonArray as well as nested JsonObjects are supported.

GroovyJsonMessageValidator — Extended groovy message validator provides specific Json slurper support. With Json slurper the tester can validate the Json message body with closures for instance.



The Json validator offers two different modes to operate. By default, **strict** mode is enabled and the validator will also check the exact amount of entries (object properties / array items) to match in received and control message. No additional fields in received Json data structure will be accepted. In **soft** mode the validator allows additional entries (object properties / array items) in received Json, so the control Json can be a subset of the received. Additional entries in the received Json data structure are ignored.



The Json validation mode (strict or soft) is settable via environment variable `CITRUS_JSON_MESSAGE_VALIDATION_STRICT` or system property `citrus.json.message.validation.strict=false`. This will set soft mode to all Json text message validators.

The Json parser mode is also configurable via system property `citrus.json.permissive.mode` or environment variable `CITRUS_JSON_PERMISSIVE_MODE`. Accepted values are: `SIMPLE` (default), `RFC4627`, `STRICTEST`, `PERMISSIVE`, or a numeric parser mode value. This controls how strictly the underlying JSON parser handles the input format.

You can also overwrite these default message validators for Json by placing a bean into the Spring Application context. The bean uses a default name as identifier. Then your custom bean will overwrite the default validator:

Java

```
@Bean
public JsonTextMessageValidator defaultJsonMessageValidator() {
    return new JsonTextMessageValidator();
}
```

XML

```
<bean id="defaultJsonMessageValidator"
class="org.citrusframework.validation.json.JsonTextMessageValidator"/>
```

The same approach applies to the Groovy message validator implementation.

Java

```
@Bean
public GroovyJsonMessageValidator defaultGroovyJsonMessageValidator() {
    return new GroovyJsonMessageValidator();
}
```

XML

```
<bean id="defaultGroovyJsonMessageValidator"
class="org.citrusframework.validation.groovy.GroovyJsonMessageValidator"/>
```

This is how you can customize the message validators used for Json message data.

When a message has been received in Citrus the message validation will try to find a matching message validator according to the message content. You can also specify the Json message format on a receive action in order to force Json message validation.

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .body("{" +
            "\"type\" : \"read\"," +
            "\"mbean\" : \"java.lang:type=Memory\"," +
            "\"attribute\" : \"HeapMemoryUsage\"," +
            "\"path\" : \"@equalsIgnoreCase('USED')@\"," +
            "\"value\" : \"${heapUsage}\"," +
            "\"timestamp\" : \"@ignore@\"" +
          "}");
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json">
        <data>
          {
            "type" : "read",
            "mbean" : "java.lang:type=Memory",
            "attribute" : "HeapMemoryUsage",
            "path" : "@equalsIgnoreCase('USED')@",
            "value" : "${heapUsage}",
            "timestamp" : "@ignore@"
          }
        </data>
    </message>
</receive>
```

The message receiving action in our test case specifies a message format type `type="json"` . This tells Citrus to look for some message validator implementation capable of validating Json messages. As we have added the proper message validator to the Spring application context Citrus will pick the right validator and Json message validation is performed on this message.

Instead of defining an expected message body template we can also use Groovy validation scripts. Let's have a look at the Groovy Json message validator example. As usual the default Groovy Json message validator is active by default. But the special Groovy message validator implementation will only jump in when we used a validation script in our receive message definition. Let's have an example for that.

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(groovy()
        .script("assert json.type == 'read'\n" +
                "assert json.mbean == 'java.lang:type=Memory'\n" +
                "assert json.attribute == 'HeapMemoryUsage'\n" +
                "assert json.value == '${heapUsage}'"));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json">
        <validate>
            <script type="groovy">
                <![CDATA[
                  assert json.type == 'read'
                  assert json.mbean == 'java.lang:type=Memory'
                  assert json.attribute == 'HeapMemoryUsage'
                  assert json.value == '${heapUsage}'
                ]]>
            </script>
        </validate>
    </message>
</receive>
```

Again the message type tells Citrus that we expect a message of type **json**. The action uses a validation script written in Groovy to verify the incoming message. Citrus will automatically activate the special message validator that executes our Groovy script.

The script validation is very powerful as we can use the full power of the Groovy language. The validation script automatically has access to the incoming Json message object **json**. We can use the Groovy Json dot notated syntax in order to navigate through the Json structure. The Groovy Json slurper object **json** is automatically injected in the validation script. This way you can access the Json object elements in your code doing some assertions.

There is even more object injection for the validation script. With the automatically added object ***receivedMessage*** you have access to the Citrus message object for this receive action. This enables you to do whatever you want with the message body or header.

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(groovy()
        .script("assert receivedMessage.getPayload(String.class).contains(\"Hello
Citrus!\")\n" +
                "assert receivedMessage.getHeader(\"Operation\") == 'sayHello'\n" +
                "context.setVariable(\"request_body\",
receivedMessage.getPayload(String.class))"));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json">
        <validate>
            <script type="groovy">
                assert receivedMessage.getPayload(String.class).contains("Hello
Citrus!")
                assert receivedMessage.getHeader("Operation") == 'sayHello'

                context.setVariable("request_body",
receivedMessage.getPayload(String.class))
            </script>
        </validate>
    </message>
</receive>
```

The listing above shows some power of the validation script. We can access the message body, we can access the message header. With test context access we can also save the whole message body as a new test variable for later usage in the test.

In general Groovy code inside the XML test case definition or as part of the Java DSL code is not very comfortable to maintain. Neither you do have code syntax assist nor code completion when using inline Groovy scripts.

Also, in case the validation script gets more complex you might want to load the script from an external file resource.

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(groovy()
        .script(new ClassPathResource("path/to/validationScript.groovy")));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json">
        <validate>
            <script type="groovy" file="path/to/validationScript.groovy"/>
        </validate>
    </message>
</receive>
```

We referenced some external file resource ***validationScript.groovy*** . This file content is loaded at runtime and is used as script body. Now that we have a normal groovy file we can use the code completion and syntax highlighting of our favorite Groovy editor.



Using several message validator implementations at the same time in the Spring application context is also no problem. Citrus automatically searches for all available message validators applicable for the given message format and executes these validators in sequence. This means that multiple message validators can coexist in a Citrus project.

Multiple message validators that all apply to the message content format will run in sequence. In case you need to explicitly choose a message validator implementation you can do so in the receive action:

Java

```
receive(someEndpoint)
    .validator(groovyJsonMessageValidator)
    .message()
    .type(MessageType.JSON)
    .validate(groovy()
        .script(new ClassPathResource("path/to/validationScript.groovy")));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json" validator="groovyJsonMessageValidator">
        <validate>
            <script type="groovy" file="path/to/validationScript.groovy"/>
        </validate>
    </message>
</receive>
```

In this example we use the **groovyJsonMessageValidator** explicitly in the receive test action. The message validator implementation was added as Spring bean with id **groovyJsonMessageValidator** to the Spring application context before. Now Citrus will only execute the explicit message validator. Other implementations that might also apply are skipped.



By default, Citrus consolidates all available message validators. You can explicitly pick a special message validator in the receive message action as shown in the example above. In this case all other validators will not take part in this special message validation. But be careful: When picking a message validator explicitly you are of course limited to this message validator capabilities. Validation features of other validators are not valid in this case (e.g. message header validation, XPath validation, etc.)

7.3.1. Ignore with JsonPath

The next usage scenario for JsonPath expressions in Citrus is the ignoring of elements during message validation. As you already know Citrus provides powerful validation mechanisms for XML and Json message format. The framework is able to compare received and expected message contents with powerful validator implementations. You can use a JsonPath expression for ignoring a very specific entry in the Json object structure.

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .body("{\"users\":" +
            "[{" +
                "\"name\": \"Jane\"," +
                "\"token\": \"?\"," +
                "\"lastLogin\": 0" +
            "}," +
            "{" +
                "\"name\": \"Penny\"," +
                "\"token\": \"?\"," +
                "\"lastLogin\": 0" +
            "}," +
            "{" +
                "\"name\": \"Mary\"," +
                "\"token\": \"?\"," +
                "\"lastLogin\": 0" +
            "}]" +
        "}")
        .validate(validation().json()
                .ignore("$.users[*].token")
                .ignore("$..lastLogin"));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json">
      <data>
        {
          "users":
          [{
            "name": "Jane",
            "token": "?",
            "lastLogin": 0
          },
          {
            "name": "Penny",
            "token": "?",
            "lastLogin": 0
          },
          {
            "name": "Mary",
            "token": "?",
            "lastLogin": 0
          }]
        }
      </data>
      <ignore expression="$.users[*].token" />
      <ignore expression="$..lastLogin" />
    </message>
</receive>
```

This time we add JsonPath expressions as ignore statements. This means that we explicitly leave out the evaluated elements from validation. Obviously this mechanism is a good thing to do when dynamic message data simply is not deterministic such as timestamps and dynamic identifiers. In the example above we explicitly skip the **token** entry and all **lastLogin** values that are obviously timestamp values in milliseconds.

The JsonPath evaluation is very powerful when it comes to select a set of Json objects and elements. This is how we can ignore several elements with one single JsonPath expression which is very powerful.

7.3.2. JsonPath validation

Let's continue to use JsonPath expressions when validating a received message in Citrus:

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(validation().jsonPath()
        .expression("$.user.name", "Penny")
        .expression("$['user']['name']", "${userName}")
        .expression("$.user.aliases", "[\"penny\",\"jenny\",\"nanny\"]")
        .expression("$.user[?(@.admin)].password", "@startsWith('$%00')@")
        .expression("$.user.address[?(@.type='office')]",
"{\"city\":\"Munich\",\"street\":\"Company Street\",\"type\":\"office\"}"));
```

XML

```
<receive endpoint="someEndpoint">
  <message type="json">
    <validate>
      <json-path expression="$.user.name" value="Penny"/>
      <json-path expression="$['user']['name']" value="${userName}"/>
      <json-path expression="$.user.aliases" value="['penny','jenny','nanny']"/>
      <json-path expression="$.user[?(@.admin)].password"
value="@startsWith('$%00')@"/>
      <json-path expression="$.user.address[?(@.type='office')]"
          value="{&quot;city&quot;:&quot;Munich&quot;,&quot;street&quot;:&quot;Company
Street&quot;,&quot;type&quot;:&quot;office&quot;}"/>
    </validate>
  </message>
</receive>
```

Use path expression map

```
final Map<String, Object> validationMap = new HashMap<>();
validationMap.put("$.user.name", "Penny");
validationMap.put("$['user']['name']", "${userName}");
validationMap.put("$.user.aliases", "[\"penny\",\"jenny\",\"nanny\"]");
validationMap.put("$.user[?(@.admin)].password", "@startsWith('$%00')@");
validationMap.put("$.user.address[?(@.type='office')]",
"{\"city\":\"Munich\",\"street\":\"Company Street\",\"type\":\"office\"}");

receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(validation().jsonPath().expressions(validationMap));
```

The above JsonPath expressions will be evaluated when Citrus validates the received message. The expression result is compared to the expected value where expectations can be static values as well as test variables and validation matcher expressions. In case a JsonPath expression should not be able to find any elements the test case will also fail.

Json is a pretty simple yet powerful message format. Simply put, a Json message just knows JsonObject, JsonArray and JsonValue items. The handling of JsonObject and JsonValue items in JsonPath expressions is straight forward. We just use a dot notated syntax for walking through the JsonObject hierarchy. The handling of JsonArray items is also not very difficult either. Citrus will try the best to convert JsonArray items to String representation values for comparison.



JsonPath expressions will only work on Json message formats. This is why we have to tell Citrus the correct message format. By default, Citrus is working with XML message data and therefore the XML validation mechanisms do apply by default. With the message type attribute set to **json** we make sure that Citrus enables Json specific features on the message validation such as JsonPath support.

Now let's get a bit more complex with validation matchers and Json object functions. Citrus tries to give you the most comfortable validation capabilities when comparing Json object values and Json arrays. One first thing you can use is object functions like **keySet()** or **size()** . This functionality is not covered by JsonPath out of the box but added by Citrus. See the following example on how to use it:

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(validation().jsonPath()
        .expression("$.user.keySet()", "[id,name,admin,projects]")
        .expression("$.user.aliases.size()", "3"));
```

XML

```
<receive endpoint="someEndpoint">
  <message type="json">
    <validate>
      <json-path expression="$.user.keySet()" value="[id,name,admin,projects]"/>
      <json-path expression="$.user.aliases.size()" value="3"/>
    </validate>
  </message>
</receive>
```

The object functions do return special Json object related properties such as the set of **keys** for an object or the size of a Json array.

Now let's get even more comfortable validation capabilities with matchers. Citrus supports Hamcrest matchers which gives us a very powerful way of validating Json object elements and arrays. See the following examples that demonstrate how this works:

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .validate(validation().jsonPath()
                .expression("$.user.keySet()",
contains("id","name","admin","projects"))
                .expression("$.user.aliases.size()", allOf(greaterThan(0),
lessThan(5))));
```

XML

```
<receive endpoint="someEndpoint">
  <message type="json">
    <validate>
      <json-path expression="$.user.keySet()"
value="@assertThat(contains(id,name,admin,projects))@"/>
      <json-path expression="$.user.aliases.size()"
value="@assertThat(allOf(greaterThan(0), lessThan(5)))@"/>
    </validate>
  </message>
</receive>
```

When using the XML DSL we have to use the **assertThat** validation matcher syntax for defining the Hamcrest matchers. You can combine matcher implementation as seen in the **allOf(greaterThan(0), lessThan(5))** expression. When using the Java DSL you can just add the matcher as expected result object. Citrus evaluates the matchers and makes sure everything is as expected. This is a very powerful validation mechanism as it combines the Hamcrest matcher capabilities with Json message validation.

7.3.3. Json schema validation

The Json schema validation in Citrus is based on the drafts provided by json-schema.org. Because Json schema is a fast evolving project, only Json schema V3 and V4 are currently supported.



In contrast to the XML validation, the Json validation is an optional feature. You have to activate it within every receive-message action by setting `schema-validation="true"`

Java

```
receive(someEndpoint)
    .message()
    .type(MessageType.JSON)
    .body()
    .validate(validation().json()
        .schemaValidation(true)
        .schema("bookStore"));
```

XML

```
<receive endpoint="echoHttpServer">
  <message type="json" schema="bookStore" schema-validation="true">
    <data>
      {
        "isbn" : "0345391802",
        "title": "The Hitchhiker's Guide to the Galaxy",
        "author": "Douglas Adams"
      }
    </data>
  </message>
</receive>
```

Json schema validation in Citrus is optional and disabled by default. This is why the action required to explicitly enable the schema validation with `schemaValidation(true)`. The schema references a bean in the Citrus context (e.g. a Spring bean in the application context). Read more about how to declare schemas in schema validation.

We encourage you to add Json schema validation to your test cases as soon as possible, because we think that message validation is an important part of integration testing.

7.3.4. Json schema repositories

Because Citrus supports different types of schema repositories, it is necessary to declare a Json schema repository as `type="json"`. This allows Citrus to collect all Json schema files for the message validation.

Java

```
@Bean
public JsonSchemaRepository schemaRepository() {
    JsonSchemaRepository repository = new JsonSchemaRepository();
    repository.getSchemas().add(productSchema());
    return repository;
}
```

XML

```
<citrus:schema-repository type="json" id="jsonSchemaRepository">
    <citrus:schemas>
        <citrus:schema ref="productSchema"
location="classpath:org/citrusframework/validation/ProductsSchema.json"/>
    </citrus:schemas>
</citrus:schema-repository>
```

The referenced schema is another bean in the configuration that represents the schema definition.

Java

```
@Bean
public SimpleJsonSchema productSchema() {
    return new SimpleJsonSchema(
             new
ClassPathResource("classpath:org/citrusframework/validation/ProductsSchema.json"));
}
```

XML

```
<citrus:schema id="productSchema"
location="classpath:org/citrusframework/validation/ProductsSchema.json"/>
```

7.3.5. Json schema filtering and validation strategy

In reference to the current Json schema definition, it is not possible to create a direct reference between a Json message and a set of schemas, as it would be possible with XML namespaces. Because of that, Citrus follows a rule set for choosing the relevant schemas based on the configuration within the test case in relation to the given context. The following table assumes that the Json schema validation is activated for the test action.

Scenario	**Validation rules**
No Json schema repositories are defined in the context.	No Json schema validation applies.
There is at least one Json schema repository defined in the context.	The message of the test action must be valid regarding at least one of the available schemas within the context.
A schema overruling is configured in the test case.	The configured schema must exist and the message must be valid regarding the specified schema.
A schema repository overruling is configured in the test case.	The configured schema repository must exist and the message must be valid regarding at least one of the schemas within the specified schema repository.

7.4. XML validation

XML is a very common message format especially in the SOAP WebServices and JMS messaging world. Citrus provides XML message validator implementations that are able to compare XML message structures. The validator will notice differences in the XML message and supports XML namespaces, attributes and XML schema validation.

XML message validation is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

XML validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-xml</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The default XML message validator implementation is active by default and can be overwritten with a custom implementation using the bean id **defaultXmlMessageValidator** .

Java

```
@Bean
public DomXmlMessageValidator defaultXmlMessageValidator() {
    return new DomXmlMessageValidator();
}
```

XML

```
<bean id="defaultXmlMessageValidator"
class="org.citrusframework.validation.xml.DomXmlMessageValidator"/>
```

The default XML message validator is very powerful when it comes to compare XML structures. The validator supports namespaces with different prefixes and attributes as well as namespace qualified attributes. See the following sections for a detailed description of all capabilities.

7.4.1. XML payload validation

Citrus is able to verify XML message content on a received message. You as a tester can compare the whole XML message body to a predefined control message template. The Citrus message validator will walk through the XML document and compare the elements, attributes and values.

You can define the expected XML message template in multiple ways:

- Defines the message body as nested XML message template in the test code. The whole message body is defined inside the test case.
- Defines an expected XML message template via external file resources. This time the body

content is loaded at runtime from the external file.

In Java tests that use the Citrus domain specific language you must use verbose String concatenation when constructing XML message contents inline. You need to escape reserved characters like quotes and line breaks. This is why you should consider to using external file resources in Java when dealing with large complex message data.

Java

```
@CitrusTest
public void externalBodyResourceTest() {
    receive("someEndpoint")
        .message()
        .body(new
ClassPathResource("org/citrusframework/message/data/TestRequest.xml"))
        .header("Operation", "sayHello")
        .header("MessageId", "${messageId}");
}
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <payload file="classpath:org/citrusframework/message/data/TestRequest.xml"/>
  </message>
</receive>
```

Inline message body definition or external file resource give us a control message template that the test case expects to validate. Citrus uses this control template for extended message comparison. All elements, namespaces, attributes and node values are validated in this comparison. When using XML message bodies Citrus will navigate through the whole XML structure validating each element and its content.

Only in case received message and control message are equal to each other as expected the message validation will pass. In case differences occur the test case fails with detailed error message.

Citrus supports various ways to add dynamic message content in the message template. Secondly, Citrus can ignore some elements that should not be part of message comparison (e.g. when generated content or timestamps are part of the message content). The tester can enrich the expected message template with test variables or ignore-expressions so we get a more robust validation mechanism. We will talk about this in the next sections to come.

7.4.2. XML header validation

A message can have multiple headers in addition to the body content. You can validate headers with expected name and the control value. Just add the following header validation to your receiving action.

Java

```
receive("someEndpoint")
    .message()
    .header("Operation", "sayHello")
    .header("MessageId", "${messageId}");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <header>
        <element name="Operation" value="GetCustomer"/>
        <element name="RequestTag" value="${requestTag}"/>
    </header>
  </message>
</receive>
```

Message headers are represented as name-value pairs. Each expected header element identified by its name has to be present in the received message. In addition to that the header value is compared to the given control value. If a header entry is not available, or the value does not match the expectations the test raises validation errors.

7.4.3. XML header fragment validation

Sometimes message headers may not apply to the name-value pair pattern. For example SOAP headers can also contain XML fragments as header values. You can add complex header data as expected value in the validation.

Java

```
receive("someEndpoint")
    .message()
    .header("Operation", "SayHello")
    .header("<ns0:HelloHeader
xmlns:ns0=\"http://citrusframework.org/schemas/samples/HelloService.xsd\">" +
                "<ns0:MessageId>${messageId}</ns0:MessageId>" +
                "<ns0:CorrelationId>${correlationId}</ns0:CorrelationId>" +
                "<ns0:User>${user}</ns0:User>" +
                "<ns0:Text>Hello from Citrus!</ns0:Text>" +
            "</ns0:HelloHeader>");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <header>
      <data>
        <![CDATA[
          <ns0:HelloHeader
xmlns:ns0="http://citrusframework.org/schemas/samples/HelloService.xsd">
              <ns0:MessageId>${messageId}</ns0:MessageId>
              <ns0:CorrelationId>${correlationId}</ns0:CorrelationId>
              <ns0:User>${user}</ns0:User>
              <ns0:Text>Hello from Citrus!</ns0:Text>
          </ns0:HelloHeader>
        ]]>
      </data>
      <element name="Operation" value="SayHello"/>
    </header>
  </message>
</receive>
```

The header data has no name but uses a complex XML fragment as a value. In SOAP this header fragment will be added as a `SOAP-ENV:Header` then. Please read more about this in SOAP support.

7.4.4. Ignore XML elements

Some elements in the message payload might not apply for validation at all. Just think of communication timestamps or dynamic values that have been generated from a foreign service.

You as a tester may not be able to predict such a timestamp or dynamic value for expected validation. This is why you can safely ignore elements and attributes in the XML message validation.

Java

```
receive("someEndpoint")
    .message()
    .header("<TestMessage>" +
                "<VersionId>${versionId}</VersionId>" +
                "<Timestamp>?</Timestamp>" +
                "<MessageId>?</MessageId>" +
            "</TestMessage>")
    .validate(validation().xpath()
            .ignore("/TestMessage/Timestamp")
            .ignore("/TestMessage/MessageId"));
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <payload>
      <TestMessage>
        <VersionId>${versionId}</VersionId>
        <Timestamp>?</Timestamp>
        <MessageId>?</MessageId>
      </TestMessage>
    </payload>
    <ignore path="/TestMessage/Timestamp"/>
    <ignore path="/TestMessage/MessageId"/>
  </message>
</receive>
```

The receive action above is not able to verify the elements `Timestamp` and `MessageId`. This is because the timestamp uses milliseconds and the message id has been generated by the server application. Both values must be excluded from XML validation.

You can use ignore XPath expressions that match elements in the message content that should be excluded. XPath expressions can be cumbersome and error prone though.

You can also use inline **@ignore@** expressions as expected template values in order to exclude elements from valdidation. This is for those of you that do not like to write XPath expressions. As a result the ignored message elements are automatically skipped when Citrus compares and validates message contents and do not break the test case.

Java

```
receive("someEndpoint")
    .message()
    .header("<TestMessage>" +
                "<VersionId>${versionId}</VersionId>" +
                "<Timestamp>@ignore@</Timestamp>" +
                "<MessageId>@ignore@</MessageId>" +
            "</TestMessage>");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <payload>
      <TestMessage>
        <VersionId>${versionId}</VersionId>
        <Timestamp>@ignore@</Timestamp>
        <MessageId>@ignore@</MessageId>
      </TestMessage>
    </payload>
  </message>
</receive>
```

Feel free to mix both mechanisms to ignore message elements. Ignore expressions are valid as elements, sub-tree nodes and attributes. You can use the **@ignore@** placeholder in external file resources, too.

7.4.5. XPath validation

The section XML payload validation showed how to validate the complete XML message structure with control message template. All elements are validated and compared one after another.

In some cases this approach might be too extensive. Imagine the tester only needs to validate a small subset of message elements. You would rather want to use explicit element validation with XPath.

Java

```
receive("someEndpoint")
    .message()
    .validate(validation().xpath()
        .expression("/TestRequest/MessageId", "${messageId}")
        .expression("//VersionId", "2"));
}
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="/TestRequest/MessageId" value="${messageId}"/>
      <xpath expression="/TestRequest/VersionId" value="2"/>
    </validate>
  </message>
</receive>
```

In Java the use of a map may be the easiest way to declare multiple expressions for XPath validation.

Java DSL

```
final Map<String, Object> expressions = new HashMap<>();
expressions.put("/TestRequest/MessageId", "${messageId}");
expressions.put("//VersionId", "2");

receive("someEndpoint")
    .message()
    .validate(validation().xpath()
        .expressions(expressions));
}
```

Instead of comparing the whole message some message elements are validated explicitly via XPath. Citrus evaluates the XPath expression on the received message and compares the result value to the control value. The basic message structure as well as all other message elements are not included into this explicit validation.



If this type of element validation is chosen neither <payload> nor <data> nor <resource> template definitions are allowed in Citrus XML test cases.



Citrus offers an alternative dot-notated syntax in order to walk through XML trees. In case you are not familiar with XPath or simply need a very easy way to find your element inside the XML tree you might use this way. Every element hierarchy in the XML tree is represented with a simple dot - for example:

`TestRequest.VersionId`

The expression will search the XML tree for the respective `<TestRequest><VersionId>` element. Attributes are supported too. In case the last element in the dot-notated expression is an XML attribute the framework will automatically find it.

Of course this dot-notated syntax is very simple and might not be applicable for more complex tree navigation. XPath is much more powerful - no doubt. The dot-notated syntax might help those of you that are not familiar with XPath. So the dot-notation is supported wherever XPath expressions might apply.

The Xpath expressions can evaluate to different result types. By default, Citrus is operating on **NODE** and **STRING** result types so that you can validate some element value. But you can also use different result types such as **NODESET** and **BOOLEAN** .

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("boolean:/TestRequest/Error", false)
        .expression("number:/TestRequest/Status[.='success']", 3)
        .expression("node-set:/TestRequest/OrderType", "[single, multi, multi]");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="/TestRequest/Error" value="false" result-type="boolean"/>
      <xpath expression="/TestRequest/Status[.='success']" value="3" result-
type="number"/>
      <xpath expression="/TestRequest/OrderType" value="[single, multi, multi]"
result-type="node-set"/>
    </validate>
  </message>
</receive>
```

In the example above we use different expression result types. First we want to make sure no **/TestRequest/Error** element is present. This can be done with a boolean result type and **false** value. Second we want to validate the number of found elements for the expression **/TestRequest/Status[.='success']** . The XPath expression evaluates to a node list that results in its list size to be checked. And last but not least we evaluate to a **node-set** result type where all values in the node list will be translated to a comma delimited string value.

You can use even more powerful validation expressions with matcher implementations. With validation matchers you are able to use validations such as **greaterThan**, **lessThan**, **hasSize** and much more.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("/TestRequest/Error", anyOf(empty(), nullValue()))
        .expression("number:/TestRequest/Status[.='success']", greaterThan(0.0))
        .expression("integer:/TestRequest/Status[.='failed']", lowerThan(1))
        .expression("node-set:/TestRequest/OrderType", hasSize(3));
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="/TestRequest/Error" value="@assertThat(anyOf(empty(),
nullValue()))@"/>
      <xpath expression="/TestRequest/Status[.='success']"
value="@assertThat(greaterThan(0.0))@" result-type="number"/>
      <xpath expression="/TestRequest/Status[.='failed']"
value="@assertThat(lowerThan(1))@" result-type="integer"/>
      <xpath expression="/TestRequest/OrderType" value="@assertThat(hasSize(3))@"
result-type="node-set"/>
    </validate>
  </message>
</receive>
```



The validation matchers used in the example above use the citrus-hamcrest-validation module.



XPath uses decimal number type **Double** by default when evaluating expressions with **number** result type. This means we have to use Double typed expected values, too. Citrus also provides the result type **integer** that automatically converts the XPath expression result to an **Integer** type.

When using the XML DSL we have to use the **assertThat** validation matcher syntax for defining the Hamcrest matcher. You can combine matcher implementation as seen in the **anyOf(empty(), nullValue())** expression. When using the Java DSL you can just add the matcher as expected result object. Citrus evaluates the matchers and makes sure everything is as expected. This is a very powerful validation mechanism as it also works with node-sets containing multiple values as list.

This is how you can add very powerful message element validation in XML using XPath expressions.

7.4.6. XML namespaces

Namespaces represent an essential concept in XML. A namespace declares an element to be part of a very specific ruleset. You have to specify namespaces also when using XPath expressions. Let's have a look at an example message that uses XML namespaces:

Sample XML body with namepaces

```
<ns1:TestMessage xmlns:ns1="http://citrus.com/namespace">
    <ns1:TestHeader>
        <ns1:CorrelationId>_</ns1:CorrelationId>
        <ns1:Timestamp>2001-12-17T09:30:47.0Z</ns1:Timestamp>
        <ns1:VersionId>2</ns1:VersionId>
    </ns1:TestHeader>
    <ns1:TestBody>
        <ns1:Customer>
            <ns1:Id>1</ns1:Id>
        </ns1:Customer>
    </ns1:TestBody>
</ns1:TestMessage>
```

Now we would like to validate some elements in this message using XPath

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("//TestMessage/TestHeader/VersionId", 2L)
        .expression("//TestMessage/TestHeader/CorrelationId", "${correlationId}");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="//TestMessage/TestHeader/VersionId" value="2"/>
      <xpath expression="//TestMessage/TestHeader/CorrelationId"
value="${correlationId}"/>
    </validate>
  </message>
</receive>
```

The validation will fail although the XPath expression looks correct regarding the XML tree. This is because the message uses the namespace `xmlns:ns1="http://citrus.com/namespace"`. The XPath expression is not able to find the elements because of the missing namespace declaration in the expression. The correct XPath expression uses the namespace prefix as defined in the message.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("//ns1:TestMessage/ns1:TestHeader/ns1:VersionId", 2L)
        .expression("//ns1:TestMessage/ns1:TestHeader/ns1:CorrelationId",
"${correlationId}");
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="//ns1:TestMessage/ns1:TestHeader/ns1:VersionId" value="2"/>
      <xpath expression="//ns1:TestMessage/ns1:TestHeader/ns1:CorrelationId"
value="${correlationId}"/>
    </validate>
  </message>
</receive>
```

Now the expressions works fine, and the validation is successful. Relying on the namespace prefix `ns1` is quite error-prone though. This is because the test depends on the very specific namespace prefix. As soon as the message is sent with a different namespace prefix (e.g. ns2) the validation will fail again.

You can avoid this effect when specifying your own namespace context and your own namespace prefix during validation.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("//pfx:TestMessage/pfx:TestHeader/pfx:VersionId", 2L)
        .expression("//pfx:TestMessage/pfx:TestHeader/pfx:CorrelationId",
"${correlationId}")
        .namespaceContext("pfx", "http://citrus.com/namespace"));
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="//pfx:TestMessage/pfx:TestHeader/pfx:VersionId" value="2"/>
      <xpath expression="//pfx:TestMessage/pfx:TestHeader/pfx:CorrelationId"
value="${correlationId}"/>
      <namespace prefix="pfx" value="http://citrus.com/namespace"/>
    </validate>
  </message>
</receive>
```

Now the test is independent of any namespace prefix in the received message. The namespace context will resolve the namespaces and find the elements although the message might use different prefixes. The only thing that matters is that the namespace value (http://citrus.com/namespace) matches.



Instead of this namespace context on validation level you can also have a global namespace context which is valid in all test cases. We just add a bean in the basic Spring application context configuration which defines global namespace mappings.

Java

```
@Bean
public NamespaceContextBuilder namespaceContext() {
    NamespaceContextBuilder builder = new NamespaceContextBuilder();
    builder.getNamepspaceMappings().put("pfx",
"http://citrusframework.org/samples/sayHello");
    return builder;
}
```

XML

```
<namespace-context>
    <namespace prefix="def" uri="http://citrusframework.org/samples/sayHello"/>
</namespace-context>
```

Once defined the **def** namespace prefix is valid in all test cases and all XPath expressions. This enables you to free your test cases from namespace prefix bindings that might be broken with time. You can use these global namespace mappings wherever XPath expressions are valid inside a test case (validation, ignore, extract).

In the previous section we have seen that XML namespaces can get tricky with XPath validation. Default namespaces can do even more! So let's look at the example with default namespaces:

Sample XML body with default namespaces

```
<TestMessage xmlns="http://citrus.com/namespace">
    <TestHeader>
        <CorrelationId>_</CorrelationId>
        <Timestamp>2001-12-17T09:30:47.0Z</Timestamp>
        <VersionId>2</VersionId>
    </TestHeader>
    <TestBody>
        <Customer>
            <Id>1</Id>
        </Customer>
    </TestBody>
</TestMessage>
```

The message uses default namespaces. The following approach in XPath will fail due to namespace problems.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("//TestMessage/TestHeader/VersionId", 2L)
        .expression("//TestMessage/TestHeader/CorrelationId", "${correlationId}"));
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="//TestMessage/TestHeader/VersionId" value="2"/>
      <xpath expression="//TestMessage/TestHeader/CorrelationId"
value="${correlationId}"/>
    </validate>
  </message>
</receive>
```

Even default namespaces need to be specified in the XPath expressions. Look at the following code listing that works fine with default namespaces:

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("//:TestMessage/:TestHeader/:VersionId", 2L)
        .expression("//:TestMessage/:TestHeader/:CorrelationId", "${correlationId}"));
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="//:TestMessage/:TestHeader/:VersionId" value="2"/>
      <xpath expression="//:TestMessage/:TestHeader/:CorrelationId"
value="${correlationId}"/>
    </validate>
  </message>
</receive>
```



It is recommended to use the namespace context as described in the previous chapter when validating. Only this approach ensures flexibility and stable test cases regarding namespace changes.

7.4.7. Customize XML parser and serializer

When working with XML data format parsing and serializing is a common task. XML structures are

parsed to a DOM (Document Object Model) representation in order to process elements, attributes and text nodes. DOM node objects get serialized to a String message payload representation. The XML parser and serializer is customizable to a certain level. By default, Citrus uses the DOM Level 3 Load and Save implementation with following settings:

Parser settings

cdata-sections	**true**
split-cdata-sections	**false**
validate-if-schema	**true**
element-content-whitespace	**false**

Serializer settings

format-pretty-print	**true**
split-cdata-sections	**false**
element-content-whitespace	**true**

The parameters are also described in W3C DOM configuration documentation. We can customize the default settings by adding an *XmlConfigurer* Spring bean to the Citrus application context.

Java

```
@Bean
public XmlConfigurer xmlConfigurer() {
    XmlConfigurer configurer = new XmlConfigurer();
    configurer.getParseSettings().put("validate-if-schema", false);

    configurer.getSerializeSettings().put("comments", false);
    configurer.getSerializeSettings().put("format-pretty-print", false);
    return configurer;
}
```

XML

```
<bean id="xmlConfigurer" class="org.citrusframework.xml.support.XmlConfigurer">
    <property name="parseSettings">
        <map>
            <entry key="validate-if-schema" value="false" value-
type="java.lang.Boolean"/>
        </map>
    </property>
    <property name="serializeSettings">
        <map>
            <entry key="comments" value="false" value-type="java.lang.Boolean"/>
            <entry key="format-pretty-print" value="false" value-
type="java.lang.Boolean"/>
        </map>
    </property>
</bean>
```



This configuration is of global nature. All XML processing operations will be affected with this configuration.

7.4.8. Groovy XML validation

With the Groovy XmlSlurper you can easily validate XML message payloads without having to deal directly with XML. People who do not want to deal with XPath may also like this validation alternative.

The tester directly navigates through the message elements and uses simple code assertions in order to control the message content. Here is an example how to validate messages with Groovy script:

Java

```
receive("someEndpoint")
    .validate(groovy().script("assert root.children().size() == 4\n" +
                              "assert root.MessageId.text() == '${messageId}'\n" +
                              "assert root.CorrelationId.text() ==
'${correlationId}'\n")
                              "assert root.Text.text() == 'Hello ' +
context.getVariable(\"user\")"))
    .header("Operation", "sayHello")
    .header("CorrelationId", "${correlationId}")
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint" timeout="5000">
    <message>
        <validate>
            <script type="groovy">
                assert root.children().size() == 4
                assert root.MessageId.text() == '${messageId}'
                assert root.CorrelationId.text() == '${correlationId}'
                assert root.Text.text() == 'Hello ' + context.getVariable("user")
            </script>
        </validate>
    </message>
    <header>
        <element name="Operation" value="sayHello"/>
        <element name="CorrelationId" value="${correlationId}"/>
    </header>
</receive>
```

The Groovy XmlSlurper validation script goes right into the message-tag instead of an XML control template or XPath validation. The Groovy script supports Java ***assert*** statements for message element validation. Citrus automatically injects the root element ***root*** to the validation script. This is the Groovy XmlSlurper object and the start of element navigation. Based on this root element you can access child elements and attributes with a dot notated syntax. Just use the element names separated by a simple dot. Very easy! If you need the list of child elements use the ***children()*** function on any element. With the ***text()*** function you get access to the element's text-value. The ***size()*** is very useful for validating the number of child elements which completes the basic validation statements.

As you can see from the example, we may use test variables within the validation script, too. Citrus has also injected the actual test context to the validation script. The test context object holds all test variables. So you can also access variables with ***context.getVariable("user")*** for instance. On the test context you can also set new variable values with ***context.setVariable("user", "newUserName")*** .

There is even more object injection for the validation script. With the automatically added object ***receivedMessage*** You have access to the Citrus message object for this receive action. This enables you to do whatever you want with the message payload or header.

Java

```
receive("someEndpoint")
    .validate(groovy().script("assert
receivedMessage.getPayload(String.class).contains(\"Hello Citrus!\")\n" +
                              "assert receivedMessage.getHeader(\"Operation\") ==
'sayHello'\n" +
                              "context.setVariable(\"request_payload\",
receivedMessage.getPayload(String.class))"))
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint" timeout="5000">
    <message>
        <validate>
            <script type="groovy">
                assert receivedMessage.getPayload(String.class).contains("Hello
Citrus!")
                assert receivedMessage.getHeader("Operation") == 'sayHello'

                context.setVariable("request_payload",
receivedMessage.getPayload(String.class))
            </script>
        </validate>
    </message>
</receive>
```

The listing above shows some power of the validation script. We can access the message payload, we can access the message header. With test context access we can also save the whole message payload as a new test variable for later usage in the test.

In general Groovy code inside the XML test case definition or as part of the Java DSL code is not very comfortable to maintain. You do not have code syntax assist or code completion. This is why we can also use external file resources for the validation scripts. The syntax looks like follows:

Java

```
receive("someEndpoint")
    .validate(groovy()
            .script(new ClassPathResource("validationScript.groovy"))
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint" timeout="5000">
    <message>
        <validate>
            <script type="groovy" file="classpath:validationScript.groovy"/>
        </validate>
    </message>
</receive>
```

We referenced some external file resource ***validationScript.groovy*** . This file content is loaded at runtime and is used as script body. Now that we have a normal groovy file we can use the code completion and syntax highlighting of our favorite Groovy editor.



You can use the Groovy validation script in combination with other validation types like XML tree comparison and XPath validation.



For further information on the Groovy XmlSlurper please see the official Groovy website and documentation

7.4.9. XML schema validation

There are several possibilities to describe the structure of XML documents. The two most popular ways are DTD (Document type definition) and XSD (XML Schema definition). In case the XML document is classified using a schema definition the structure of the document has to fit the predefined rules inside the schema definition. XML document instances are valid only in case they meet all these structure rules defined in the schema definition. Citrus can validate XML documents using the schema languages DTD and XSD.

XSD schema repositories

Citrus tries to validate all incoming XML messages against a schema definition in order to ensure that all rules are fulfilled. This means that the message receiving actions in Citrus have to know the XML schema definition file resources that belong to our test context.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getSchemas().add(travelAgencySchema());
    repository.getSchemas().add(royalAirlineSchema());
    repository.getSchemas().add(smartAirlineSchema());
    return repository;
}

@Bean
public SimpleXsdSchema travelAgencySchema() {
    return new SimpleXsdSchema(
            new
ClassPathResource("classpath:citrus/flightbooking/TravelAgencySchema.xsd"));
}

@Bean
public SimpleXsdSchema royalAirlineSchema() {
    return new SimpleXsdSchema(
            new
ClassPathResource("classpath:citrus/flightbooking/RoyalAirlineSchema.xsd"));
}

@Bean
public SimpleXsdSchema smartAirlineSchema() {
    return new SimpleXsdSchema(
            new
ClassPathResource("classpath:citrus/flightbooking/SmartAirlineSchema.xsd"));
}
```

XML

```
<citrus:schema-repository id="schemaRepository">
    <citrus:schemas>
        <citrus:schema id="travelAgencySchema"
            location="classpath:citrus/flightbooking/TravelAgencySchema.xsd"/>
        <citrus:schema id="royalAirlineSchema"
            location="classpath:citrus/flightbooking/RoyalAirlineSchema.xsd"/>
        <citrus:reference schema="smartAirlineSchema"/>
    </citrus:schemas>
</citrus:schema-repository>

<citrus:schema id="smartAirlineSchema"
      location="classpath:citrus/flightbooking/SmartAirlineSchema.xsd"/>
```

By convention there is a default schema repository component defined in the Citrus Spring application context with the id **schemaRepository**. Spring application context is then able to inject the schema repository into all message receiving test actions at runtime. The receiving test action consolidates the repository for a matching schema definition file in order to validate the incoming XML document structure.

The connection between incoming XML messages and xsd schema files in the repository is done by a mapping strategy which we will discuss later in this chapter. By default, Citrus picks the right schema based on the target namespace that is defined inside the schema definition. The target namespace of the schema definition has to match the namespace of the root element in the received XML message. With this mapping strategy you will not have to wire XML messages and schema files manually, all is done automatically by the Citrus schema repository at runtime. All you need to do is to register all available schema definition files regardless of which target namespace or nature inside the Citrus schema repository.



XML schema validation is mandatory in Citrus. This means that Citrus always tries to find a matching schema definition inside the schema repository in order to perform syntax validation on incoming schema qualified XML messages. A classified XML message is defined by its namespace definitions. Consequently you will get validation errors in case no matching schema definition file is found inside the schema repository. So if you explicitly do not want to validate the XML schema for some reason you have to disable the validation explicitly in your test with **schema-validation="false"**.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
            .expression("//ns1:TestMessage/ns1:MessageHeader/ns1:MessageId",
"${messageId}")
            .expression("//ns1:TestMessage/ns1:MessageHeader/ns1:CorrelationId",
"${correlationId}")
            .schemaValidation(false)
            .namespaceContext("ns1", "http://citrus.com/namespace"))
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint">
    <message schema-validation="false">
      <validate>
        <xpath expression="//ns1:TestMessage/ns1:MessageHeader/ns1:MessageId"
             value="${messageId}"/>
        <xpath expression="//ns1:TestMessage/ns1:MessageHeader/ns1:CorrelationId"
             value="${correlationId}"/>
        <namespace prefix="ns1" value="http://citrus.com/namespace"/>
      </validate>
    </message>
</receive>
```

This mandatory schema validation might sound annoying to you but in our opinion it is very important to validate the structure of the received XML messages, so disabling the schema validation should not be the standard for all tests. Disabling automatic schema validation should only apply to very special situations. So please try to put all available schema definitions to the schema repository and you will be fine.

WSDL schemas

In SOAP WebServices world the WSDL (WebService Schema Definition Language) defines the structure and nature of the XML messages exchanged across the interface. Often the WSDL files do hold the XML schema definitions as nested elements. In Citrus you can directly set the WSDL file as location of a schema definition like this:

Java

```
@Bean
public WsdlXsdSchema airlineWsdl() {
    return new WsdlXsdSchema(
            new
ClassPathResource("classpath:citrus/flightbooking/AirlineSchema.wsdl"));
}
```

XML

```
<citrus:schema id="airlineWsdl"
    location="classpath:citrus/flightbooking/AirlineSchema.wsdl"/>
```

Citrus is able to find the nested schema definitions inside the WSDL file in order to build a valid schema file for the schema repository. So incoming XML messages that refer to the WSDL file can be validated for syntax rules.

Schema collections

Sometimes an XML schema definition is separated into multiple files. This is a problem for the Citrus schema repository as the schema mapping strategy then is not able to pick the right file for validation, in particular when working with target namespace values as key for the schema mapping strategy. As a solution for this problem you have to put all schemas with the same target namespace value into a schema collection.

Java

```
@Bean
public XsdSchemaCollection flightbookingSchemaCollection() {
    XsdSchemaCollection collection = new XsdSchemaCollection();
    collection.getSchemas().add("classpath:citrus/flightbooking/BaseTypes.xsd");
    collection.getSchemas().add("classpath:citrus/flightbooking/AirlineSchema.xsd");
    return collection;
}
```

XML

```
<citrus:schema-collection id="flightbookingSchemaCollection">
  <citrus:schemas>
    <citrus:schema location="classpath:citrus/flightbooking/BaseTypes.xsd"/>
    <citrus:schema location="classpath:citrus/flightbooking/AirlineSchema.xsd"/>
  </citrus:schemas>
</citrus:schema-collection>
```

Both schema definitions **BaseTypes.xsd** and **AirlineSchema.xsd** share the same target namespace and therefore need to be combined in schema collection component. The schema collection can be referenced in any schema repository as normal schema definition.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getSchemas().add(flightbookingSchemaCollection());
    return repository;
}
```

XML

```
<citrus:schema-repository id="schemaRepository">
  <citrus:schemas>
    <citrus:reference schema="flightbookingSchemaCollection"/>
  </citrus:schemas>
</citrus:schema-repository>
```

Schema mapping strategy

The schema repository in Citrus holds one to many schema definition files and dynamically picks up the right one according to the validated message payload. The repository needs to have some strategy for deciding which schema definition to choose. See the following schema mapping strategies and decide which of them is suitable for you.

Target Namespace Mapping Strategy

This is the default schema mapping strategy. Schema definitions usually define some target namespace which is valid for all elements and types inside the schema file. The target namespace is also used as root namespace in XML message payloads. According to this information Citrus can pick up the right schema definition file in the schema repository. You can set the schema mapping strategy as property in the configuration files:

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.setSchemaMappingStrategy(schemaMappingStrategy());
    repository.getSchemas().add(helloSchema());
    return repository;
}

@Bean
public TargetNamespaceSchemaMappingStrategy schemaMappingStrategy() {
    return new TargetNamespaceSchemaMappingStrategy();
}

@Bean
public SimpleXsdSchema helloSchema() {
    return new SimpleXsdSchema(
            new ClassPathResource("classpath:citrus/samples/sayHello.xsd"));
}
```

XML

```
<citrus:schema-repository id="schemaRepository"
    schema-mapping-strategy="schemaMappingStrategy">
  <citrus:schemas>
    <citrus:schema id="helloSchema"
        location="classpath:citrus/samples/sayHello.xsd"/>
  </citrus:schemas>
</citrus:schema-repository>

<bean id="schemaMappingStrategy"
    class="org.citrusframework.xml.schema.TargetNamespaceSchemaMappingStrategy"/>
```

The **sayHello.xsd** schema file defines a target namespace (http://citrusframework.org/schemas/sayHello.xsd):

Schema target namespace

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns="http://citrusframework.org/schemas/sayHello.xsd"
    targetNamespace="http://citrusframework.org/schemas/sayHello.xsd"
    elementFormDefault="qualified"
    attributeFormDefault="unqualified">

</xs:schema>
```

Incoming request messages should also have the target namespace set in the root element and this is how Citrus matches the right schema file in the repository.

HelloRequest.xml

```
<HelloRequest xmlns="http://citrusframework.org/schemas/sayHello.xsd">
   <MessageId>123456789</MessageId>
   <CorrelationId>1000</CorrelationId>
   <User>Christoph</User>
   <Text>Hello Citrus</Text>
</HelloRequest>
```

Root QName Mapping Strategy

The next possibility for mapping incoming request messages to a schema definition is via the XML root element QName. Each XML message payload starts with a root element that usually declares the type of an XML message. According to this root element you can set up mappings in the schema repository.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.setSchemaMappingStrategy(schemaMappingStrategy());
    repository.getSchemas().add(helloSchema());
    repository.getSchemas().add(goodbyeSchema());
    return repository;
}

@Bean
public RootQNameSchemaMappingStrategy schemaMappingStrategy() {
    RootQNameSchemaMappingStrategy rootQnameStrategy = new
RootQNameSchemaMappingStrategy();
    rootQnameStrategy.getMappings().put("HelloRequest", helloSchema());
    rootQnameStrategy.getMappings().put("GoodbyeRequest", goodbyeSchema());

    return rootQnameStrategy;
}

@Bean
public SimpleXsdSchema helloSchema() {
    return new SimpleXsdSchema(
            new ClassPathResource("classpath:citrus/samples/sayHello.xsd"));
}

@Bean
public SimpleXsdSchema goodbyeSchema() {
    return new SimpleXsdSchema(
            new ClassPathResource("classpath:citrus/samples/sayGoodbye.xsd"));
}
```

XML

```
<citrus:schema-repository id="schemaRepository"
    schema-mapping-strategy="schemaMappingStrategy">
  <citrus:schemas>
    <citrus:reference schema="helloSchema"/>
    <citrus:reference schema="goodbyeSchema"/>
  </citrus:schemas>
</citrus:schema-repository>

<bean id="schemaMappingStrategy"
    class="org.citrusframework.xml.schema.RootQNameSchemaMappingStrategy">
  <property name="mappings">
    <map>
      <entry key="HelloRequest" value="helloSchema"/>
      <entry key="GoodbyeRequest" value="goodbyeSchema"/>
    </map>
  </property>
</bean>

<citrus:schema id="helloSchema"
    location="classpath:citrus/samples/sayHello.xsd"/>

<citrus:schema id="goodbyeSchema"
     location="classpath:citrus/samples/sayGoodbye.xsd"/>
```

The listing above defines two root qname mappings - one for **HelloRequest** and one for **GoodbyeRequest** message types. An incoming message of type <HelloRequest> is then mapped to the respective schema and so on. With these dedicated mappings you are able to control which schema is used on an XML request, regardless of target namespace definitions.

Schema mapping strategy chain

Let's discuss the possibility to combine several schema mapping strategies in a logical chain. You can define more than one mapping strategy that are evaluated in sequence. The first strategy to find a proper schema definition file in the repository wins.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.setSchemaMappingStrategy(schemaMappingStrategy());
    repository.getSchemas().add(helloSchema());
    repository.getSchemas().add(goodbyeSchema());
    return repository;
}

@Bean
public SchemaMappingStrategyChain schemaMappingStrategy() {
    SchemaMappingStrategyChain chain = new SchemaMappingStrategyChain();

    RootQNameSchemaMappingStrategy rootQnameStrategy = new
RootQNameSchemaMappingStrategy();
    rootQnameStrategy.getMappings().put("HelloRequest", helloSchema());

    chain.setStrategies(Arrays.asList(
        rootQnameStrategy,
        new TargetNamespaceSchemaMappingStrategy()
    ));

    return chain;
}
```

XML

```
<citrus:schema-repository id="schemaRepository"
    schema-mapping-strategy="schemaMappingStrategy">
  <citrus:schemas>
    <citrus:reference schema="helloSchema"/>
    <citrus:reference schema="goodbyeSchema"/>
  </citrus:schemas>
</citrus:schema-repository>

<bean id="schemaMappingStrategy"
    class="org.citrusframework.xml.schema.SchemaMappingStrategyChain">
  <property name="strategies">
    <list>
      <bean class="org.citrusframework.xml.schema.RootQNameSchemaMappingStrategy">
        <property name="mappings">
          <map>
            <entry key="HelloRequest" value="helloSchema"/>
          </map>
        </property>
      </bean>
      <bean
class="org.citrusframework.xml.schema.TargetNamespaceSchemaMappingStrategy"/>
    </list>
  </property>
</bean>
```

So the schema mapping chain uses both **RootQNameSchemaMappingStrategy** and **TargetNamespaceSchemaMappingStrategy** in combination. In case the first root qname strategy fails to find a proper mapping the next target namespace strategy comes in and tries to find a proper schema.

DTD validation

XML DTD (document type definition) is another way to validate the structure of an XML document. Many people say that DTD is outdated and XML schema is the much more efficient way to describe the rules of an XML structure. We do not disagree with that, but we also know that legacy systems might still use DTD. So in order to avoid validation errors we have to deal with DTD validation as well.

First thing you can do about DTD validation is to specify an inline DTD in your expected message template.

Java

```
receive("someEndpoint")
    .message()
    .body("<!DOCTYPE root [\n" +
                "<!ELEMENT root (message)>\n" +
                "<!ELEMENT message (text)>\n" +
                "<!ELEMENT text (#PCDATA)>\n" +
            "]>\n" +
            "<root>\n" +
                "<message>\n" +
                    "<text>Hello from Citrus!</text>\n" +
                "</message>\n" +
            "</root>")
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint">
    <message schema-validation="false">
        <data>
        <![CDATA[
            <!DOCTYPE root [
                <!ELEMENT root (message)>
                <!ELEMENT message (text)>
                <!ELEMENT text (#PCDATA)>
                ]>
            <root>
                <message>
                    <text>Hello from Citrus!</text>
                </message>
            </root>
        ]]>
        </data>
    </message>
</receive>
```

The system under test may also send the message with an inline DTD definition. So validation will succeed.

In most cases the DTD is referenced as external .dtd file resource. You can do this in your expected message template as well.

Java

```
receive("someEndpoint")
    .message()
    .body("<!DOCTYPE root SYSTEM \"org/citrusframework/validation/example.dtd\">\n" +
            "<root>\n" +
                "<message>\n" +
                    "<text>Hello from Citrus!</text>\n" +
                "</message>\n" +
            "</root>")
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint">
    <message schema-validation="false">
        <data>
        <![CDATA[
            <!DOCTYPE root SYSTEM "org/citrusframework/validation/example.dtd">
            <root>
                <message>
                    <text>Hello from Citrus!</text>
                </message>
            </root>
        ]]>
        <data/>
    </message>
</receive>
```

7.4.10. XHTML validation

Html message content is hard to verify with XML validation capabilities such as XML tree comparison or XPath support. Usually Html messages do not follow the XML well-formed rules very strictly. This implies that XML message validation will fail because of non-well-formed Html code.

XHTML closes this gap by automatically fixing the most common Html XML incompatible rule violations such as missing end tags (e.g.
).

Please add a new library dependency to your project. Citrus is using the **jtidy** library in order to prepare the HTML and XHTML messages for validation. As this 3rd party dependency is optional in Citrus we have to add it now to our project dependency list. Just add the **jtidy** dependency to your Maven project POM.

Jtidy library

```
<dependency>
    <groupId>net.sf.jtidy</groupId>
    <artifactId>jtidy</artifactId>
    <version>r938</version>
</dependency>
```

Please refer to the **jtidy** project documentation for the latest versions. Now everything is ready. As usual the Citrus message validator for XHTML is active in background by default. You can overwrite this default implementation by placing a Spring bean with id **defaultXhtmlMessageValidator** to the Citrus application context.

Java

```
@Bean
public XhtmlMessageValidator defaultXhtmlMessageValidator() {
    return new XhtmlMessageValidator();
}
```

XML

```
<bean id="defaultXhtmlMessageValidator"
class="org.citrusframework.validation.xhtml.XhtmlMessageValidator"/>
```

Now you can use the XHTML message validation in your test case.

Java

```
receive("someEndpoint")
    .message()
    .type(MessageType.XHTML)
    .body("<!DOCTYPE html PUBLIC \"-//W3C//DTD XHTML 1.1//EN\" \"org/w3c/xhtml/xhtml1-
strict.dtd\">" +
            "<html xmlns=\"http://www.w3.org/1999/xhtml\">" +
                "<head>" +
                    "<title>Citrus Hello World</title>" +
                "</head>" +
                "<body>" +
                    "<h1>Hello World!</h1>" +
                    "<br/>" +
                    "<p>This is a test!</p>" +
                "</body>" +
            "</html>")
    .timeout(5000L);
```

XML

```
<receive endpoint="someEndpoint">
    <message type="xhtml">
        <data>
          <![CDATA[
            <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "org/w3c/xhtml/xhtml1-
strict.dtd">
            <html xmlns="http://www.w3.org/1999/xhtml">
              <head>
                <title>Citrus Hello World</title>
              </head>
              <body>
                <h1>Hello World!</h1>
                <br/>
                <p>This is a test!</p>
              </body>
            </html>
          ]]>
        </data>
    </message>
</receive>
```

The message receiving action in our test case has to specify a message format type **type="xhtml"** . As you can see the Html message payload get XHTML specific DOCTYPE processing instruction. The **xhtml1-strict.dtd** is mandatory in the XHTML message validation. For better convenience all XHTML dtd files are packaged within Citrus so you can use this as a relative path.

The incoming Html message is automatically converted into proper XHTML code with well formed XML. So now the XHTML message validator can use the XML message validation mechanism of Citrus for comparing received and expected data. You can also use test variables, ignore element expressions and XPath expressions.

7.5. Schema validation

When structured data is transmitted from one system to another, it is important that both sender and receiver agree on an interface contract. The contract introduces rules of communication for both parties.

Schemas represent the most popular way to define contracts. Citrus is able to manage multiple schemas in the project context. You can define mapping rules to pick the right schema for a message validation.

Let's start with this chapter by introducing some basic concepts of the schema validation.

7.5.1. Schema definition

Complex applications require multiple schemas that are relevant for different use cases. You should organize these schemas in your test project. First you need to add a schema definition that points to

the schema location.

Java

```
@Bean
public SimpleXsdSchema bookstoreSchema() {
    return new SimpleXsdSchema(new
ClassPathResource("classpath:org/citrusframework/xml/BookStore.wsdl"));
}
```

XML

```
<citrus:schema id="bookstoreSchema"
location="classpath:org/citrusframework/xml/BookStore.wsdl"/>
```

Please keep in mind, that the id of the schema has to be unique within the context.



The samples above are using XML XSD/WSDL schema definitions. The same approach applies to Json schemas, too. You just need to use the `SimpleJsonSchema` class in the Java configuration. The XML configuration components derive the schema type automatically based on the file extension (`.xsd`, `.wsdl`, `.json`).

7.5.2. Schema repository

You can now reference the schema definition in a repository component. The repository is able to hold multiple schema definitions.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getSchemas().add(bookstoreSchema());
    return repository;
}
```

XML

```
<citrus:schema-repository id="schemaRepository">
  <citrus:schemas>
    <citrus:reference schema="bookstoreSchema" />
  </citrus:schemas>
</citrus:schema-repository>
```

Citrus allows you to reuse your schema definition within your context by referencing them. For a valid reference, the id of the schema and the value of the schema attribute within the reference element have to match.

As an alternative to a schema reference you can also provide the schema location in a repository.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getLocations().add("classpath:schemas/flightbooking.xsd");
    return repository;
}
```

XML

```
<citrus:schema-repository id="schemaRepository">
  <citrus:locations>
    <citrus:location path="classpath:schemas/flightbooking.xsd"/>
  </citrus:locations>
</citrus:schema-repository>
```

The given location points to the schema definition file. Setting all schemas one by one can be verbose and cumbersome, especially when dealing with lots of schema files. Therefore, Citrus provides schema location patterns which will import all matching schema files within the given location.

Java

```
@Bean
public XsdSchemaRepository schemaRepository() {
    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getLocations().add("classpath:schemas/*.xsd");
    return repository;
}
```

XML

```
<citrus:schema-repository id="schemaRepository">
  <citrus:locations>
    <citrus:location path="classpath:schemas/*.xsd"/>
  </citrus:locations>
</citrus:schema-repository>
```

The schema repository is able to receive many schemas with different locations and schema sources.

Java

```
@Bean
public SimpleXsdSchema testSchema() {
    return new SimpleXsdSchema(new
ClassPathResource("classpath:org/citrusframework/xml/test.xsd"));
}

@Bean
public XsdSchemaRepository schemaRepository() {
    SimpleXsdSchema bookstoreSchema = new SimpleXsdSchema(
            new
ClassPathResource("classpath:org/citrusframework/xml/BookStore.wsdl"));

    XsdSchemaRepository repository = new XsdSchemaRepository();
    repository.getSchemas().add(bookstoreSchema);
    repository.getSchemas().add(testSchema());
    repository.getLocations().add("classpath:schemas/*.xsd");
    return repository;
}
```

XML

```
<citrus:schema id="testSchema" location="classpath:org/citrusframework/xml/test.xsd"/>

<citrus:schema-repository id="xmlSchemaRepository">
  <citrus:schemas>
    <citrus:schema id="bookstoreSchema"
location="classpath:org/citrusframework/xml/BookStore.wsdl"/>
    <citrus:reference schema="testSchema"/>
    <citrus:location path="classpath:schemas/*.xsd"/>
  </citrus:schemas>
</citrus:schema-repository>
```



The examples in this chapter have been using XML XSD schema repository components. Of course, the same components are available for Json schema repositories, too. By default, the type of the schema repository is `type=xml`. You can use `type=json` to mark the schema repository as Json nature. In Java configuration please use the `JsonSchemaRepository` class.

The schema repository component holds a set of schema files for a project disjoint by their type (xml, json, etc.) and identified by its unique id.

As you can see the schema repository is a simple bean defined inside the Spring application context. The repository can hold nested schema definitions, references and location definitions for all types of schema repositories.



In case you have several schema repositories in your project do always define a default repository (name="schemaRepository"). This helps Citrus to always find at least one repository to interact with.

7.5.3. Schema definition mapping

Depending on the type of message you want to validate, there are different attempts to find the correct schema for the given message. The XML schema repository will apply a mapping strategy that decides which schema should verify the current message. Citrus knows multiple mapping strategies that you can review in chapter XML schema validation.

As a user you always have the chance to explicitly pick the right schema definition for a `receive` operation. You can overrule all schema mapping strategies in Citrus by directly setting the desired schema in your receiving message action.

Java

```
receive(httpMessageEndpoint)
    .message()
    .validate(
        xml().schema("helloSchema")
    );
```

XML

```
<receive endpoint="httpMessageEndpoint">
    <message schema="helloSchema">
      <payload>
        ...
      </payload>
    </message>
</receive>
```

In the example above the tester explicitly sets a schema definition in the `receive` action (schema="helloSchema"). The schema value refers to named schema bean defined in the project context (e.g. Spring application context).



This overrules all schema mapping strategies used in the central schema repository as the given schema is directly used for validation. This feature is helpful when dealing with different schema versions at the same time.

Another possibility would be to set a custom schema repository at this point. This means you can have more than one schema repository in your Citrus project and you pick the right one by yourself in the `receive` action.

Java

```
receive(httpMessageEndpoint)
    .message()
    .validate(
        xml().schemaRepository("helloSchemaRepository")
    );
```

XML

```
<receive endpoint="httpMessageEndpoint">
    <message schema-repository="helloSchemaRepository">
      <payload>
        ...
      </payload>
    </message>
</receive>
```

The **schema-repository** attribute refers to a Citrus schema repository component which is defined as bean in the project context.

7.6. Plain text validation

Plain text message validation performs an exact Java String match of received and expected message payloads.

Plaintext message validation is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

Plaintext validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-text</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

This adds the default message validator for plaintext messages. Citrus will pick this message validator for all messages of **type="plaintext"**. The default message validator implementation can be overwritten by placing a Spring bean with id **defaultPlaintextMessageValidator**.

Java

```
@Bean
public PlainTextMessageValidator defaultPlaintextMessageValidator() {
    return new PlainTextMessageValidator();
}
```

XML

```
<bean id="defaultPlaintextMessageValidator"
class="org.citrusframework.validation.text.PlainTextMessageValidator"/>
```

Citrus will try to auto-guess the appropiate message validator for the incoming message. You can explicitly set the message type on the receive action so Citrus knows how to apply plaintext message validation.

Java

```
receive("someEndpoint")
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Hello World!");
```

XML

```
<receive endpoint="someEndpoint">
    <message type="plaintext">
        <data>Hello World!</data>
    </message>
</receive>
```

With the message format type **type="plaintext"** set Citrus performs String equals on the message payloads (received and expected). Only exact match will pass the test.

Of course test variables are supported in the plaintext payloads. The variables are replaced before processing the message template.

Java

```
receive("someEndpoint")
    .message()
    .type(MessageType.PLAINTEXT)
    .body("${hello} ${world}!");
```

XML

```
<receive endpoint="someEndpoint">
    <message type="plaintext">
        <data>${hello} ${world}!</data>
    </message>
</receive>
```

7.6.1. Whitespace characters

Plaintext message payloads may only differ in system-dependent line separator characters (**CR**, *LF*,

CRLF). By default, the plaintext message validation fails because of that differences even if only whitespace characters are different.

You can disable this default validation behavior and ignore new line types with following system property or environment variable:

Plaintext validation settings

```
citrus.plaintext.validation.ignore.newline.type=true
CITRUS_PLAINTEXT_VALIDATION_IGNORE_NEWLINE_TYPE=true
```

In case you need to ignore all whitespaces during plaintext validation such as multiple new line characters or tabs you need to set this system property or environment variable:

Plaintext validation settings

```
citrus.plaintext.validation.ignore.whitespace=true
CITRUS_PLAINTEXT_VALIDATION_IGNORE_WHITESPACE=true
```

This property will not only ignore new line types but also normalize the whitespaces. As a result all empty lines, tabs and double whitespace characters are filtered before comparison.

Of course, you can also set the properties directly on the plaintext message validator bean:

Java

```
@Bean
public PlainTextMessageValidator defaultPlaintextMessageValidator() {
    PlainTextMessageValidator validatoe = new PlainTextMessageValidator();
    validator.setIgnoreNewLineType(true);
    validator.setIgnoreWhitespace(true);
    return validator;
}
```

XML

```
<bean id="defaultPlaintextMessageValidator"
class="org.citrusframework.validation.text.PlainTextMessageValidator">
  <property name="ignoreNewLineType" value="true"/>
  <property name="ignoreWhitespace" value="true"/>
</bean>
```

7.6.2. Ignoring text parts

The plaintext validator performs a String equals operation. Test variables are automatically replaced before that comparison takes place but what about ignore statements? The plaintext message validator is able to ignore words and character sequences based on their position in the text value. Given a source plaintext value:

Received text

```
Your current id is "1234567890"
```

In the plaintext validation you need to ignore the actual id value due to some reason. Maybe the id is generated on a foreign application and you simply do not know the actual value at runtime.

In this case we can use the common `@ignore@` statement in the control message payload as follows:

Control text

```
Your current id is "@ignore@"
```

Citrus and the plaintext message validator will ignore the marked part of the text during validation. This mechanism is based on the fact that the `@ignore@` statement is placed at the exact same position as the actual id value. So this mechanism requires you to know the exact structure of the plaintext value including all whitespace characters. In case Citrus finds the `@ignore@` keyword in the control value the placeholder is replaced with the actual character sequence that is located at the exact same position in the source message payload that is validated.

The character sequence is defined as sequence of Java word characters. This word sequence is ending with a non-word character defined in Java (`\\W` which is a character that is not in `[a-zA-Z_0-9]`).

Instead of ignoring a single word you can also specify the amount of characters that should be ignored. This is when you have Java non-word characters that you need to ignore. Let's have an example for that, too:

Received text

```
Your current id is "#12345-67890"
```

Given that text the simple `@ignore@` statement will fail because of the non-word characters '**#**' and '**-**' that are located in the id value. This time we ignore the whole id sequence with:

Control text

```
Your current id is "@ignore(12)@"
```

This will ignore exactly **12** characters starting from the exact position of the `@ignore@` keyword. So knowing that the id is exactly **12** characters long we can ignore that part.

7.6.3. Creating variables

Instead of just ignoring certain text parts we can also extract those parts into test variables. The actual character sequence is ignored during validation and in addition to that the actual value is stored to a new test variable. Given the following text payload:

Received text

```
Your current id is "1234567890"
```

And the expected control text:

Control text

```
Your current id is "@variable('id')@"
```

The validation will automatically ignore the id part in the text and create a new test variable with name `id` that holds the actual value. The name of the variable to create is given in the `@variable()@` statement. This enables us to extract dynamic text parts that we are not able to validate. After that we can access the dynamic text part using the normal test variable syntax `${id}`.

Also notice that the `@variable()@` keyword expression has to be placed at the exact same position in the text as the actual value. The variable extractor will read the variable value from the source message payload starting from that position. The ending of the variable value is defined by a non-word Java character. Dashes '**-**' and dots '**.**' are automatically included in these values, too. So this will also work for you:

Received text

```
Today is "2017-12-24"
```

And the expected control text:

Control text

```
Today is "@variable('date')@"
```

This results in a new variable called `date` with value `2017-12-24`. In addition, the European date representation works fine here, too because dots and dashes are automatically included:

Received text

```
Today is "24.12.2017"
```

7.6.4. Gzip validation

Gzip is a message compression library to optimize the message transport of large content. Citrus is able to handle compressed message payloads on send and receive operations. Sending compressed data sets the message type to **gzip**.

Java

```
send("someEndpoint")
    .message()
    .type(MessageType.GZIP)
    .body("Hello World!")
```

XML

```
<send endpoint="someEndpoint">
    <message type="gzip">
        <data>Hello World!</data>
    </message>
</send>
```

Just use the **type="gzip"** message type in the send operation. Citrus now converts the message payload to a gzip binary stream as payload.

When validating gzip binary message content the messages are compared with a given control message in binary base64 String representation. The gzip binary data is automatically unzipped and encoded as base64 character sequence in order to compare with an expected content.

The received message content is using gzip format but the actual message content does not have to be base64 encoded. Citrus is doing this conversion automatically before validation takes place. The binary data can be anything e.g. images, pdf or plaintext content.

The default message validator for gzip messages is active by default. Citrus will pick this message validator for all messages of **type="gzip_base64"** . The default message validator implementation can be overwritten by placing a Spring bean with id **defaultGzipBinaryBase64MessageValidator** to the Spring application context.

Java

```
@Bean
public GzipBinaryBase64MessageValidator defaultGzipBinaryBase64MessageValidator() {
    return new GzipBinaryBase64MessageValidator();
}
```

XML

```
<bean id="defaultGzipBinaryBase64MessageValidator"
      class="org.citrusframework.validation.text.GzipBinaryBase64MessageValidator"/>
```

In the test case receiving action we tell Citrus to use gzip message validation.

Java

```
receive("someEndpoint")
    .message()
    .type(MessageType.GZIP_BASE64)
    .body("citrus:encodeBase64('Hello World!')")
```

XML

```
<receive endpoint="someEndpoint">
    <message type="gzip_base64">
        <data>citrus:encodeBase64('Hello World!')</data>
    </message>
</receive>
```

With the message format type **type="gzip_base64"** Citrus performs the gzip base64 character sequence validation. Incoming message content is automatically unzipped and encoded as base64 String and compared to the expected data. This way we can make sure that the binary content is as expected.



If you are using http client and server components the gzip compression support is built in with the underlying Spring and http commons libraries. So in http communication you just have to set the header **Accept-Encoding=gzip** or **Content-Encoding=gzip**. The message data is then automatically zipped/unzipped before Citrus gets the message data for validation. Read more about this http specific gzip compression in chapter http.

7.7. Binary validation

Binary message validation is not an easy task in particular when it comes to compare data with a given control message.

Binary message validation is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

Binary validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-binary</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

There are basically two ways in Citrus how to compare binary message content for validation purpose.

7.7.1. Stream message validation

A first approach to validate incoming binary message content is to compare the binary stream data with an expected stream. This comparison is straight forward as each byte in the binary stream is compared to an expected stream.

The default message validator for binary messages is active by default. Citrus will pick this message validator for all messages of **type="binary_base64"** . The default message validator implementation can be overwritten by placing a Spring bean with id **defaultBinaryBase64MessageValidator** to the Spring application context.

Java

```
@Bean
public BinaryMessageValidator defaultBinaryMessageValidator() {
    return new BinaryMessageValidator();
}
```

XML

```
<bean id="defaultBinaryMessageValidator"
class="org.citrusframework.validation.text.BinaryMessageValidator"/>
```

You can use the binary message type in a receive action in order to enable this stream comparison during validation.

Java

```
.receive("someEndpoint")
    .message(new DefaultMessage(FileCopyUtils.copyToByteArray(new
ClassPathResource("templates/foo.png").getFile())))
    .type(MessageType.BINARY);
}
```

XML

```
<receive endpoint="someEndpoint">
    <message type="binary">
        <resource file="classpath:templates/foo.png"/>
    </message>
</receive>
```

It is very important to set the message type to `MessageType.BINARY` as this is the message type that is automatically handled by the binary stream message validator.

By the way sending binary messages in Citrus is also very easy. Just use the **type="binary"** message type in the send operation. Citrus now converts the message payload to a binary stream as payload.

Java

```
send("someEndpoint")
    .message()
    .type(MessageType.BINARY)
    .body("Hello World")
```

XML

```
<send endpoint="someEndpoint">
    <message type="binary">
        <data>Hello World!</data>
    </message>
</send>
```

7.7.2. Base64 message validation

Another way to validate binary message content is to use base64 String encoding. The binary data is encoded as base64 character sequence and therefore is comparable with an expected content.

The received message content does not have to be base64 encoded. Citrus is doing this conversion automatically before validation takes place. The binary data can be anything e.g. images, pdf or gzip content.

The default message validator for binary messages is active by default. Citrus will pick this message validator for all messages of **type="binary_base64"** . The default message validator implementation can be overwritten by placing a Spring bean with id **defaultBinaryBase64MessageValidator** to the Spring application context.

Java

```
@Bean
public BinaryBase64MessageValidator defaultBinaryBase64MessageValidator() {
    return new BinaryBase64MessageValidator();
}
```

XML

```
<bean id="defaultBinaryBase64MessageValidator"
class="org.citrusframework.validation.text.BinaryBase64MessageValidator"/>
```

In the test case receiving action we tell Citrus to use binary base64 message validation.

Java

```
receive("someEndpoint")
    .message()
    .body("citrus:encodeBase64('Hello World!')")
```

XML

```
<receive endpoint="someEndpoint">
    <message type="binary_base64">
        <data>citrus:encodeBase64('Hello World!')</data>
    </message>
</receive>
```

With the message format type **type="binary_base64"** Citrus performs the base64 character sequence validation. Incoming message content is automatically encoded as base64 String and compared to the expected data. This way we can make sure that the binary content is as expected.

Base64 encoding is also supported in outbound messages. Just use the **encodeBase64** function in Citrus. The result is a base64 encoded String as message payload.

Java

```
send("someEndpoint")
    .message()
    .body("citrus:encodeBase64('Hello World!')")
```

XML

```
<send endpoint="someEndpoint">
    <message>
        <data>citrus:encodeBase64('Hello World!')</data>
    </message>
</send>
```

7.8. Hamcrest validation

Hamcrest validation empowers the validation capabilities by adding special assertions.

Hamcrest message validation capability is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

Binary validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-hamcrest</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

You can use the Hamcrest assertions when receiving a message in order to validate elements, for instance with XPath.

Java

```
receive("someEndpoint")
    .validate(validation().xpath()
        .expression("/TestRequest/Error", anyOf(empty(), nullValue()))
        .expression("number:/TestRequest/Status[.='success']", greaterThan(0.0))
        .expression("integer:/TestRequest/Status[.='failed']", lowerThan(1))
        .expression("node-set:/TestRequest/OrderType", hasSize(3));
```



If you want to match text containing any of the following characters: ' , ( ) You need to enclose the respective string in quotation marks when defining your matcher. If you intend to match an actual single quote, it should be escaped with a backslash (\').

For example:

```
anyOf(equalTo('text containing a \\' (quote) and a , (comma)  '),
anyOf(isEmptyOrNullString()))
anyOf(equalTo('text containing a backslash and quote \\\\' and a , (comma)  '),
anyOf(isEmptyOrNullString()))
```

XML

```
<receive endpoint="someEndpoint">
  <message>
    <validate>
      <xpath expression="/TestRequest/Error" value="@assertThat(anyOf(empty(),
nullValue()))@"/>
      <xpath expression="/TestRequest/Status[.='success']"
value="@assertThat(greaterThan(0.0))@" result-type="number"/>
      <xpath expression="/TestRequest/Status[.='failed']"
value="@assertThat(lowerThan(1))@" result-type="integer"/>
      <xpath expression="/TestRequest/OrderType" value="@assertThat(hasSize(3))@"
result-type="node-set"/>
    </validate>
  </message>
</receive>
```

7.9. AssertJ validation

AssertJ validation lets you check received values, message headers and container conditions with AssertJ assertions. It covers the same places as the Hamcrest validation, and both modules can be used side by side.

AssertJ validation is not enabled by default in your project. You need to add the validation module to your project as a Maven dependency.

AssertJ validation module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-validation-assertj</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The module brings `assertj-core` with it, so your tests can use AssertJ conditions and assertions right away.

7.9.1. Java DSL

In the Java DSL you pass an AssertJ check as the expected value of a JsonPath or XPath expression, or of a message header. There are two kinds of checks:

- an AssertJ `Condition`, passed as it is
- an assertion lambda, wrapped with `satisfies(…)` from `org.citrusframework.validation.assertj.AssertJ`

Java

```
import static org.assertj.core.api.Assertions.assertThat;
import static org.citrusframework.validation.assertj.AssertJ.satisfies;

$(receive(direct)
    .message()
    .type(MessageType.JSON)
    .header("count", satisfies(Integer.class, c -> assertThat(c).isBetween(40, 50)))
    .validate(validation().jsonPath()
        .expression("$.qty", satisfies(Integer.class, q ->
assertThat(q).isGreaterThan(5)))
        .expression("$.status", new Condition<String>(s -> s.startsWith("OK"), "starts
with OK"))
        .expression("$.note", satisfies(n -> assertThat(n).isNull())));
```

`satisfies(lambda)` hands the received value to the lambda as it is. `satisfies(Type.class, lambda)` converts the value to the given type first, using the Citrus type converter. That is how the header `count`, which arrives as the String `"42"`, reaches the lambda as the Integer `42`. A value that cannot be converted fails the validation with a message that names the target type.



Mind the type of the received value when you use a `Condition` or an untyped `satisfies(…)`. XPath values and message headers are usually Strings, and JsonPath reads numbers as `Long`. A `Condition<Integer>` on a JsonPath number therefore fails. Use `Condition<Number>` or `Condition<Long>`, or let `satisfies(Integer.class, …)` convert the value.

The validation fails as soon as an assertion throws an `AssertionError`. The validation error carries AssertJ's own failure description, for instance `Expecting actual: 3 to be greater than: 5`.

JUnit assertions

The lambda in `satisfies(…)` fails by throwing any `AssertionError`, not only AssertJ's. You can use the JUnit Jupiter `Assertions` in the very same way, so there is no need for a separate JUnit validation module.

Java

```
import static org.junit.jupiter.api.Assertions.assertEquals;

$(receive(direct)
    .message()
    .header("region", satisfies(r -> assertEquals("eu", r))));
```

7.9.2. Container conditions

`conditionOf(…)` turns an AssertJ check into a condition for the `iterate`, `repeat`, `repeatOnError`, `conditional` and `waitFor` containers. The condition is `true` when the assertion holds. A failed assertion makes the condition `false`, which keeps the container iterating or waiting, instead of failing the test.

Without an explicit value the condition checks the iteration index. With an explicit value, test variables in the value are resolved on every evaluation.

Java

```
import static org.citrusframework.validation.assertj.AssertJ.conditionOf;

$(iterate()
    .condition(conditionOf(i -> assertThat(i).isLessThan(3)))
    .index("i")
    .actions(echo("iteration ${i}")));

$(repeat()
    .until(conditionOf(Integer.class, "${count}", c -> assertThat(c).isEqualTo(3)))
    .actions(...));

$(waitFor()
    .condition(conditionOf(Integer.class, "${count}", c ->
assertThat(c).isEqualTo(3)))
    .milliseconds(1000L)
    .interval(50L));
```

When `waitFor` times out, its error message contains the last AssertJ failure description. A value that cannot be converted to the given type is logged at debug level and counts as `false`.

7.9.3. The @assertj()@ validation matcher

In YAML and XML tests you use the `@assertj()@` validation matcher. It takes a chain of AssertJ

assertion calls and applies it to `assertThat(receivedValue)`. It works wherever validation matchers are allowed: message headers, XPath and JsonPath expressions, and container conditions.

YAML

```
actions:
  - receive:
      endpoint: orders
      message:
        type: "json"
        headers:
          - name: status
            value: "@assertj(startsWith('${prefix}').endsWith('1'))@"
          - name: count
            value: "@assertj(asInt().isBetween(40, 50))@"
      validate:
        - jsonPath:
            - expression: "$.qty"
              value: "@assertj(asInt().isBetween(1, 10))@"
            - expression: "$.tags"
              value: "@assertj(asList().containsExactly('a', 'b'))@"
```

Spring XML

```
<receive endpoint="helloEndpoint">
  <message>
    <validate>
      <xpath expression="/Order/Status"
value="@assertj(isNotBlank().startsWith('OK').endsWith('123'))@"/>
      <xpath expression="/Order/Item"
value="@assertj(asList().hasSize(3).contains('b'))@" result-type="node-set"/>
      <xpath expression="count(/Order/Item)"
value="@assertj(asDouble().isEqualTo(3))@" result-type="number"/>
    </validate>
  </message>
  <header>
    <element name="Operation" value="@assertj(isIn('sayHello', 'sayHi'))@"/>
  </header>
</receive>
```

The received value is always a String, so the chain starts with the AssertJ String assertions. These calls switch to other assertion types:

`asInt()`, `asLong()`, `asShort()`, `asByte()`	integer assertions, e.g. `asInt().isBetween(1, 10)`
`asDouble()`, `asFloat()`	floating point assertions, e.g. `asDouble().isGreaterThan(3)`

`asBoolean()`	boolean assertions, e.g. `asBoolean().isTrue()`
`asList()`	list assertions on values such as `[a, b, c]`, e.g. a XPath node-set or a JsonPath array
`asMap()`	map assertions on values such as `{key=value}`

`asList()` and `asMap()` are provided by Citrus and must be the first call of the chain.

Arguments are literals only: quoted text (`'OK'`, with `\'` for a quote inside the text), integers, decimals, `true`, `false`, `null` and test variables such as `'${prefix}'`. When an assertion method is overloaded, the matcher picks the variant that fits the literals best. Assertions that need objects as arguments, for example `isCloseTo(3.4, offset)`, cannot be written as literals and are not available in the matcher. Use the Java DSL for those.

To assert on another value than the received one, give the value first and the chain second:

Spring XML

```
<iterate condition="@assertj('${i}', 'asInt().isLessThan(5)')@" index="i">
  <echo>
    <message>index is: ${i}</message>
  </echo>
</iterate>
```

In a container condition without an explicit value the matcher receives the iteration index as a String, so start the chain with `asInt()`: `@assertj(asInt().isLessThan(5))@`.

The assertion reference below lists every assertion you can call, per type.

A chain that cannot be resolved fails the validation with a message that names the problem, for instance `No AssertJ assertion 'isAwesome()' on StringAssert`.



Container conditions in XML and YAML tests treat every validation error as `false`. A typo in an `@assertj()@` container condition therefore ends an `iterate` loop right away rather than failing the test. Check new container conditions with an `echo` in the loop body.

7.9.4. Assertion reference

These are the AssertJ assertions you can call in an `@assertj()@` chain, grouped by the type the chain is on. The list follows the matcher's rules: methods declared by AssertJ that return an assertion, take only literal arguments and are not deprecated (AssertJ 3.27.7). Every example in the tables passes against the shown received value.

Placeholders: `'text'` is quoted text, `5` a whole number, `1.5` a decimal number, `value` any literal, and `, ⋯` means more arguments of the same kind may follow.

Text

The chain starts here: the received value is always a String.

Checks	Calls	Example
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, …)` `hasToString('text')` `hasToString('text', value, …)` `isEqualTo('text')` `isEqualTo('text', value, …)` `isEqualToIgnoringCase('text')` `isEqualToIgnoringNewLines('text')` `isEqualToIgnoringWhitespace('text')` `isEqualToNormalizingNewlines('text')` `isEqualToNormalizingPunctuationAndWhitespace('text')` `isEqualToNormalizingUnicode('text')` `isEqualToNormalizingWhitespace('text')` `isNotEqualTo(value)` `isNotEqualToIgnoringCase('text')` `isNotEqualToIgnoringWhitespace('text')` `isNotEqualToNormalizingWhitespace('text')`	`@assertj(isEqualToIgnoringCase('OK-123'))@` on `ok-123` `@assertj(isEqualToNormalizingWhitespace('a b'))@` on `a b`
Null, empty & blank	`isBlank()` `isEmpty()` `isNotBlank()` `isNotEmpty()` `isNotNull()` `isNull()` `isNullOrEmpty()`	`@assertj(isNotBlank())@` on `OK` `@assertj(isNullOrEmpty())@` on an empty value
Contains	`contains('text', …)` `containsAnyOf('text', …)` `containsIgnoringCase('text')` `containsIgnoringNewLines('text', …)` `containsIgnoringWhitespaces('text', …)` `containsOnlyDigits()` `containsOnlyOnce('text')` `containsOnlyWhitespaces()` `containsSequence('text', …)` `containsSubsequence('text', …)` `containsWhitespaces()` `doesNotContain('text', …)` `doesNotContainAnyWhitespaces()` `doesNotContainIgnoringCase('text', …)` `doesNotContainOnlyWhitespaces()` `isSubstringOf('text')`	`@assertj(contains('Hello', 'Citrus'))@` on `Hello Citrus` `@assertj(containsOnlyOnce('b'))@` on `a-b-a`

Checks	Calls	Example
Start & end	`doesNotEndWith('text')` `doesNotEndWithIgnoringCase('text')` `doesNotEndWithWhitespaces()` `doesNotStartWith('text')` `doesNotStartWithIgnoringCase('text')` `doesNotStartWithWhitespaces()` `endsWith('text')` `endsWithIgnoringCase('text')` `startsWith('text')` `startsWithIgnoringCase('text')`	`@assertj(startsWith('OK').endsWith('123'))@` on `OK-123` `@assertj(startsWithIgnoringCase('OK'))@` on `ok-123`
Regular expressions	`containsPattern('text')` `doesNotContainPattern('text')` `doesNotMatch('text')` `matches('text')`	`@assertj(matches('[A-Z]{3}'))@` on `ABC`
Size	`hasLineCount(5)` `hasSize(5)` `hasSizeBetween(5, 5)` `hasSizeGreaterThan(5)` `hasSizeGreaterThanOrEqualTo(5)` `hasSizeLessThan(5)` `hasSizeLessThanOrEqualTo(5)`	`@assertj(hasSizeBetween(1, 5))@` on `abc`
Character classes & case	`isAlphabetic()` `isAlphanumeric()` `isASCII()` `isLowerCase()` `isMixedCase()` `isPrintable()` `isUpperCase()` `isVisible()`	`@assertj(containsOnlyDigits())@` on `12345` `@assertj(isUpperCase())@` on `ABC`
Alphabetical order	`isBetween('text', 'text')` `isGreaterThan('text')` `isGreaterThanOrEqualTo('text')` `isLessThan('text')` `isLessThanOrEqualTo('text')` `isStrictlyBetween('text', 'text')`	`@assertj(isBetween('a', 'c'))@` on `b`
One of a set	`isIn(value, …)` `isNotIn(value, …)`	`@assertj(isIn('OK', 'PENDING'))@` on `OK`
Encoding	`isBase64()` `isHexadecimal()`	`@assertj(isBase64())@` on `SGVsbG8=` `@assertj(asBase64Decoded().hasSize(5))@` on `SGVsbG8=`
Switch type	`asBase64Decoded()` `asBoolean()` `asByte()` `asDouble()` `asFloat()` `asInt()` `asLong()` `asShort()` `asString()`	
Failure message & description	`as('text', value, …)` `describedAs('text', value, …)` `inBinary()` `inHexadecimal()` `inUnicode()` `overridingErrorMessage('text', value, …)` `withFailMessage('text', value, …)`	`@assertj(as('status').isEqualTo('OK'))@` on `OK`

Whole numbers

Reached with `asInt()` or `asLong()` or `asShort()` or `asByte()`. All four have the same assertions.

Checks	Calls	Example
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, ⋯)` `hasToString('text')` `hasToString('text', value, ⋯)` `isEqualTo(5)` `isNotEqualTo(5)`	`@assertj(asInt().isEqualTo(7))@` on `7`
Comparison & range	`isBetween(5, 5)` `isGreaterThan(5)` `isGreaterThanOrEqualTo(5)` `isLessThan(5)` `isLessThanOrEqualTo(5)` `isStrictlyBetween(5, 5)`	`@assertj(asInt().isBetween(1, 10))@` on `7` `@assertj(asLong().isGreaterThan(5))@` on `9999999999`
Sign, zero & one	`isNegative()` `isNotNegative()` `isNotPositive()` `isNotZero()` `isOne()` `isPositive()` `isZero()`	`@assertj(asInt().isPositive())@` on `7` `@assertj(asInt().isZero())@` on `0`
Parity	`isEven()` `isOdd()`	`@assertj(asInt().isOdd())@` on `7`
One of a set	`isIn(value, ⋯)` `isNotIn(value, ⋯)`	`@assertj(asInt().isIn(5, 6, 7))@` on `7`
Null, empty & blank	`isNotNull()` `isNull()`	
Switch type	`asString()`	
Failure message & description	`as('text', value, ⋯)` `describedAs('text', value, ⋯)` `inBinary()` `inHexadecimal()` `overridingErrorMessage('text', value, ⋯)` `withFailMessage('text', value, ⋯)`	

Decimals

Reached with `asDouble()` or `asFloat()`. Both have the same assertions.

Checks	Calls	Example
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, ⋯)` `hasToString('text')` `hasToString('text', value, ⋯)` `isEqualTo(1.5)` `isNotEqualTo(1.5)`	`@assertj(asFloat().isEqualTo(3.5))@` on `3.5`
Comparison & range	`isBetween(1.5, 1.5)` `isGreaterThan(1.5)` `isGreaterThanOrEqualTo(1.5)` `isLessThan(1.5)` `isLessThanOrEqualTo(1.5)` `isStrictlyBetween(1.5, 1.5)`	`@assertj(asDouble().isBetween(3, 4))@` on `3.5` `@assertj(asDouble().isGreaterThan(3.4))@` on `3.5`
Sign, zero & one	`isNegative()` `isNotNegative()` `isNotPositive()` `isNotZero()` `isOne()` `isPositive()` `isZero()`	
Special values	`isFinite()` `isInfinite()` `isNaN()` `isNotFinite()` `isNotInfinite()` `isNotNaN()`	`@assertj(asDouble().isNaN())@` on `NaN` `@assertj(asDouble().isFinite())@` on `3.5`

Checks	Calls	Example
One of a set	`isIn(value, …)` `isNotIn(value, …)`	
Null, empty & blank	`isNotNull()` `isNull()`	
Switch type	`asString()`	
Failure message & description	`as('text', value, …)` `describedAs('text', value, …)` `inBinary()` `inHexadecimal()` `overridingErrorMessage('text', value, …)` `withFailMessage('text', value, …)`	

Boolean

Reached with `asBoolean()`.

Checks	Calls	Example
True / false	`isFalse()` `isTrue()`	`@assertj(asBoolean().isTrue())@` on `true`
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, …)` `hasToString('text')` `hasToString('text', value, …)` `isEqualTo(true)` `isNotEqualTo(true)`	
One of a set	`isIn(value, …)` `isNotIn(value, …)`	
Null, empty & blank	`isNotNull()` `isNull()`	
Switch type	`asString()`	
Failure message & description	`as('text', value, …)` `describedAs('text', value, …)` `overridingErrorMessage('text', value, …)` `withFailMessage('text', value, …)`	

List

Reached with `asList()`. Parses values such as [a, b, c] (XPath node-sets, JsonPath arrays). Elements are Strings.

Checks	Calls	Example
Size	`hasSize(5)` `hasSizeBetween(5, 5)` `hasSizeGreaterThan(5)` `hasSizeGreaterThanOrEqualTo(5)` `hasSizeLessThan(5)` `hasSizeLessThanOrEqualTo(5)`	`@assertj(asList().hasSize(3))@` on `[a, b, c]`

Checks	Calls	Example
Contains	`contains(value, …)` `containsAnyOf(value, …)` `containsOnly(value, …)` `containsOnlyOnce(value, …)` `doesNotContain(value, …)`	`@assertj(asList().contains('b'))@` on `[a, b, c]` `@assertj(asList().containsOnly('a', 'b', 'c'))@` on `[a, b, c]`
Order & uniqueness	`containsExactly(value, …)` `containsExactlyInAnyOrder(value, …)` `containsSequence(value, …)` `containsSubsequence(value, …)` `doesNotContainSequence(value, …)` `doesNotContainSubsequence(value, …)` `doesNotHaveDuplicates()` `endsWith(value, value, …)` `isSorted()` `startsWith(value, …)`	`@assertj(asList().containsExactly('a', 'b', 'c'))@` on `[a, b, c]` `@assertj(asList().containsExactlyInAnyOrder('c', 'a', 'b'))@` on `[a, b, c]`
Navigate to an element or the size	`element(5)` `elements(5, …)` `first()` `last()` `singleElement()` `size()`	`@assertj(asList().first().isEqualTo('a'))@` on `[a, b, c]` `@assertj(asList().element(1).isEqualTo('b'))@` on `[a, b, c]`
Null, empty & blank	`containsNull()` `containsOnlyNulls()` `doesNotContainNull()` `isEmpty()` `isNotEmpty()` `isNotNull()` `isNull()` `isNullOrEmpty()`	`@assertj(asList().isEmpty())@` on `[]`
One of a set	`isIn(value, …)` `isNotIn(value, …)` `isSubsetOf(value, …)`	`@assertj(asList().isSubsetOf('a', 'b', 'c'))@` on `[a, b]`
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, …)` `hasToString('text')` `hasToString('text', value, …)`	
Switch type	`asString()`	
Failure message & description	`as('text', value, …)` `describedAs('text', value, …)` `inBinary()` `inHexadecimal()` `overridingErrorMessage('text', value, …)` `withFailMessage('text', value, …)`	

Map

Reached with `asMap()`. Parses values such as {k=v, x=y}. Keys and values are Strings.

Checks	Calls	Example
Keys	`containsKey(value)` `containsKeys(value, …)` `containsOnlyKeys(value, …)` `doesNotContainKey(value)` `doesNotContainKeys(value, …)`	`@assertj(asMap().containsKeys('k', 'x'))@` on `{k=v, x=y}` `@assertj(asMap().doesNotContainKey('z'))@` on `{k=v, x=y}`
Values	`containsValue(value)` `containsValues(value, …)` `doesNotContainValue(value)`	`@assertj(asMap().containsValue('v'))@` on `{k=v, x=y}`

Checks	Calls	Example
Entries	`containsEntry(value, value)` `doesNotContainEntry(value, value)`	`@assertj(asMap().containsEntry('k', 'v'))@` on `{k=v, x=y}`
Size	`hasSize(5)` `hasSizeBetween(5, 5)` `hasSizeGreaterThan(5)` `hasSizeGreaterThanOrEqualTo(5)` `hasSizeLessThan(5)` `hasSizeLessThanOrEqualTo(5)`	`@assertj(asMap().hasSize(2))@` on `{k=v, x=y}`
Navigate to a value or the size	`extractingByKey(value)` `extractingByKeys(value, …)` `size()` `values()`	`@assertj(asMap().extractingByKey('k').isEqualTo('v'))@` on `{k=v, x=y}` `@assertj(asMap().size().isEqualTo(2))@` on `{k=v, x=y}`
Null, empty & blank	`isEmpty()` `isNotEmpty()` `isNotNull()` `isNull()` `isNullOrEmpty()`	
One of a set	`isIn(value, …)` `isNotIn(value, …)`	
Equality	`doesNotHaveToString('text')` `doesNotHaveToString('text', value, …)` `hasToString('text')` `hasToString('text', value, …)`	
Switch type	`asString()`	
Failure message & description	`as('text', value, …)` `describedAs('text', value, …)` `overridingErrorMessage('text', value, …)` `withFailMessage('text', value, …)`	

Assertions that look at Java object identity, fields or hash codes (for example `isSameAs(value)`, `hasFieldOrProperty('text')`, `usingRecursiveComparison()`) also resolve, but they rarely make sense on values parsed from a message, so the tables leave them out.

Pitfalls

Avoid	Why	Use instead
`@assertj(asList().contains(1))@` on `[1, 2]`	List elements, map keys and map values are Strings, so the number `1` never equals `"1"`.	`@assertj(asList().contains('1'))@`
`@assertj(asDouble().isCloseTo(7.0, 0.1))@`	`isCloseTo` needs an `Offset` object, which a literal cannot express. The matcher reports an error.	`@assertj(asDouble().isBetween(6.9, 7.1))@`, or the Java DSL
`@assertj(isNotEmpty().asList())@`	`asList()` and `asMap()` are only allowed as the first call.	`@assertj(asList().isNotEmpty())@`
`@assertj(isEmpty().isNotNull())@`	`isEmpty()` returns nothing, so it ends the chain.	`@assertj(isNotNull().isEmpty())@`

Avoid	Why	Use instead
`@assertj(isLessThan(5))@` as an `iterate` condition	The index arrives as text, and text has no `isLessThan` for a number. A container condition treats the error as `false`, so the loop ends at once without failing the test.	`@assertj(asInt().isLessThan(5))@`

7.9.5. Migrating from Hamcrest

The Hamcrest example from the Hamcrest validation section reads like this with AssertJ:

Hamcrest	AssertJ
`.expression("/TestRequest/Error", anyOf(empty(), nullValue()))`	`.expression("/TestRequest/Error", satisfies(e → assertThat(e).satisfiesAnyOf(v → assertThat(v).isNull(), v → assertThat(v).asString().isEmpty())))`
`.expression("number:/TestRequest/Status[.='success']", greaterThan(0.0))`	`.expression("number:/TestRequest/Status[.='success']", satisfies(Double.class, s → assertThat(s).isGreaterThan(0.0)))`
`.expression("node-set:/TestRequest/OrderType", hasSize(3))`	`.expression("node-set:/TestRequest/OrderType", satisfies(List.class, t → assertThat(t).hasSize(3)))`
`@assertThat(greaterThan(0.0))@`	`@assertj(asDouble().isGreaterThan(0.0))@`
`@assertThat(hasSize(3))@`	`@assertj(asList().hasSize(3))@`

The matcher name `assertThat` stays with the Hamcrest module. With both modules on the classpath, `@assertThat()@` uses Hamcrest and `@assertj()@` uses AssertJ.

7.10. Custom validation

In a previous section you have seen how to customize the global set of available message validators in a Citrus project using the validator registry.

7.10.1. Custom message validator

You can also explicitly use a custom message validator in a receive test action. This approach skips the automatic message validator resolving mechanism, and the receive action uses the defined validator implementation.

Java

```
receive("someEndpoint")
    .validator(groovyJsonMessageValidator)
    .message()
    .type(MessageType.JSON)
    .validate(groovy()
                .script("assert json.type == 'read'\n" +
                  "assert json.mbean == 'java.lang:type=Memory'\n" +
                  "assert json.attribute == 'HeapMemoryUsage'\n" +
                  "assert json.value == '${heapUsage}'");
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json" validator="groovyJsonMessageValidator">
        <validate>
            <script type="groovy">
                <![CDATA[
                  assert json.type == 'read'
                  assert json.mbean == 'java.lang:type=Memory'
                  assert json.attribute == 'HeapMemoryUsage'
                  assert json.value == '${heapUsage}'
                ]]>
            </script>
        </validate>
    </message>
</receive>
```

The receive action defines the message validator to use in this specific use case. You can set a custom message validator implementation here.



Be careful when overwriting default message validation behavior. Setting a specific message validator has the effect that your receive action may not use the other default validators. The explicit validator definition in a receive action is exclusive so no further message validator resolving is performed on this receive action. If you wish otherwise, take a look at the custom validator strategy

You may want to set multiple validators here in order to meet your validation requirements. For instance when setting a `DomXmlMessageValidator` explicitly you may not be able to use the `XpathMessageValidator` capabilities on that specific receive action anymore. Fortunately you can set multiple validators on a receive action that will all perform validation tasks on the received message.

Java

```
receive("someEndpoint")
    .validators(myXmlMessageValidator, defaultXpathMessageValidator)
    .message()
    .body("...")
    .validate(validation().xpath().expression("//some/xpath/expression",
"someControlValue"));
```

XML

```
<receive endpoint="someEndpoint">
    <message type="json"
validators="myXmlMessageValidator,defaultXpathMessageValidator">
        <payload>...</data>
        <validate path="//some/xpath/expression" value="someControlValue"/>
    </message>
</receive>
```

7.10.2. Custom validator strategy (new)

Historically, Citrus executed **only** the custom validators defined on a receive action. This meant that **default validation steps were skipped entirely** when a custom validator was present - even if the test also defined body validation, JSON validation, or other built-in mechanisms.

This exclusive behavior was intentional: Users could provide highly specialized validation logic (e.g., custom JSON comparison), and the default validators would stay out of the way to avoid duplicate or conflicting validation.

However, this behavior can be surprising when users expect default validators to **also** run alongside custom logic. To address this, Citrus introduces a configurable validator strategy with two modes:

Available strategies

- `EXCLUSIVE` **(default)**
  - Only the custom validator(s) are executed.
  - Default validators and body assertions are not applied.
  - The same substitution applies when a validation processor (callback) is set on the receive action.
- `COMBINED`
  - Both custom validators and the default Citrus validators run.
  - Useful when custom validation should extend - not replace - the default behavior.

Configuration

The strategy can be globally configured using either a system property or an environment variable:

- System property: `citrus.custom.validator.strategy`
- Environment variable: `CITRUS_CUSTOM_VALIDATOR_STRATEGY`

Default value: `EXCLUSIVE`.

This allows you to switch the behavior project-wide without changing any test cases.

7.10.3. Validation processor

The Java DSL offers some additional validation tricks and possibilities when dealing with messages that are sent and received over Citrus. One of them is the validation processor functionality. With this feature you can marshal/unmarshal message payloads and code validation steps on Java objects.

Validation processor usage

```
receive(bookResponseEndpoint)
    .validate(new XmlMarshallingValidationProcessor<AddBookResponseMessage>() {
        @Override
        public void validate(AddBookResponseMessage response, MessageHeaders headers)
{
            Assert.isTrue(response.isSuccess());
        }
    });
```



With the default `EXCLUSIVE` custom validator strategy, a validation processor (callback) **replaces** the usual receive validation. Control-message comparison, HTTP status checks, and other `MessageValidator` steps on that receive action are not run. Only the callback runs. Set `citrus.custom.validator.strategy` / `CITRUS_CUSTOM_VALIDATOR_STRATEGY` to `COMBINED` if those default steps should still apply. See custom validator strategy.

By default, the validation processor needs some XML unmarshaller implementation for transforming the XML payload to a Java object. Citrus will automatically search for the unmarshaller bean in your Spring application context if nothing specific is set. Of course, you can also set the unmarshaller instance explicitly.

Use autowired unmarshaller

```
@Autowired
private Unmarshaller unmarshaller;

@CitrusTest
public void receiveMessageTest() {
    receive(bookResponseEndpoint)
        .validate(new
XmlMarshallingValidationProcessor<AddBookResponseMessage>(unmarshaller) {
            @Override
            public void validate(AddBookResponseMessage response, MessageHeaders
headers) {
                Assert.isTrue(response.isSuccess());
            }
        });
}
```

Obviously working on Java objects is much more comfortable than using the XML String concatenation. This is why you can also use this feature when sending messages.

Message body marshalling

```
@Autowired
private Marshaller marshaller;

@CitrusTest
public void sendMessageTest() {
    send(bookRequestEndpoint)
        .message()
        .body(createAddBookRequestMessage("978-citrus:randomNumber(10)"), marshaller)
        .header(SoapMessageHeaders.SOAP_ACTION, "addBook");
}

private AddBookRequestMessage createAddBookRequestMessage(String isbn) {
    AddBookRequestMessage requestMessage = new AddBookRequestMessage();
    Book book = new Book();
    book.setAuthor("Foo");
    book.setTitle("FooTitle");
    book.setIsbn(isbn);
    book.setYear(2008);
    book.setRegistrationDate(Calendar.getInstance());
    requestMessage.setBook(book);
    return requestMessage;
}
```

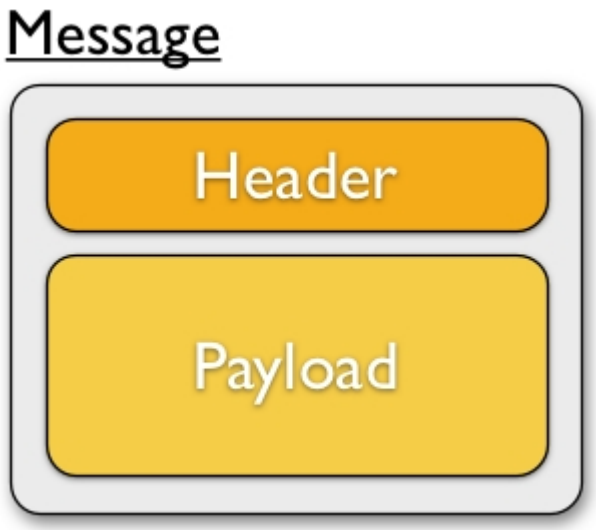
The example above creates an **AddBookRequestMessage** object and puts this as payload to a send action. In combination with a marshaller instance Citrus is able to create a proper XML message payload then.

Chapter 8. Test actions

This chapter gives a brief description to all test actions that a tester can incorporate into the test case. Besides sending and receiving messages the tester may access these actions in order to build a more complex test scenario that fits the desired use case.

8.1. Send

Integration test scenarios need to trigger business logic on foreign applications by calling service interfaces on the system under test. The Citrus test is able to send messages over various message transports (e.g. Http REST, JMS, Kafka, filte transfer).



A message consists of a message header (name-value pairs) and a message body. Later in this section we will see different ways of constructing a message with body and header values.

Java

```
@CitrusTest
public void sendMessageTest() {
    variable("text", "Hello Citrus!");
    variable("messageId", "Mx1x123456789");

    $(send("helloService")
        .message()
        .name("helloMessage")
        .header("Operation", "sayHello")
        .header("RequestTag", "${messageId}")
        .body("""
            <HelloMessage>
                <Text>Hello Citrus!</Text>
            </HelloMessage>
        """)
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="text" value="Hello Citrus!"/>
        <variable name="messageId" value="Mx1x123456789"/>
    </variables>

    <actions>
        <send endpoint="helloService">
            <message name="helloMessage">
                <headers>
                    <header name="Operation" value="sayHello"/>
                    <header name="MessageId" value="${messageId}"/>
                </headers>
                <body>
                    <![CDATA[
                    <HelloMessage>
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                    ]]>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
variables:
    - name: "text"
      value: "Hello Citrus!"
    - name: "messageId"
      value: "Mx1x123456789"
actions:
  - send:
      endpoint: "helloService"
      message:
        name: "helloMessage"
        headers:
          - name: "Operation"
            value: "sayHello"
          - name: "MessageId"
            value: '${messageId}'
        body: |
          <HelloMessage>
              <Text>Hello Citrus!</Text>
          </HelloMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SendMessageTest">
        <variables>
            <variable name="text" value="Hello Citrus!"/>
            <variable name="messageId" value="Mx1x123456789"/>
        </variables>

        <actions>
            <send endpoint="helloService">
                <message name="helloMessage">
                    <payload>
                        <![CDATA[
                        <HelloMessage>
                            <Text>${text}</Text>
                        </HelloMessage>
                        ]]>
                    </payload>
                </message>
                <header>
                    <element name="Operation" value="sayHello"/>
                    <element name="MessageId" value="${messageId}"/>
                </header>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The message name is optional and defines the message identifier in the local message store. This message name is very useful when accessing the message content later on during the test case. The local message store is handled per test case and contains all exchanged messages.

The sample uses both header and payload as message parts to send. In both parts you can use variable definitions (see **${text}** and **${messageId}**). So first of all let us recap what variables do. Test variables are defined at the very beginning of the test case and are valid throughout all actions that take place in the test. This means that actions can simply reference a variable by the expression ***${variable-name}*** .



Use variables wherever you can! At least the important entities of a test should be defined as variables at the beginning. The test case improves maintainability and flexibility when using variables.

Now let's have a closer look at the sending action. The **'endpoint'** attribute might catch your attention first. This attribute references a message endpoint in Citrus configuration by name. As previously mentioned the message endpoint definition lives in a separate configuration file and contains the actual message transport settings. In this example the **"helloService"** is referenced which is a message endpoint for sending out messages via JMS or HTTP for instance.

The test case is not aware of any transport details, because it does not have to. The advantages are obvious: On the one hand multiple test cases can reference the message endpoint definition for better reuse. Secondly test cases are independent of message transport details. So connection factories, user credentials, endpoint uri values and so on are not present in the test case.

In other words the **"endpoint"** attribute of the `<send>` element specifies which message endpoint definition to use and therefore where the message should go to. Once again all available message endpoints are configured in a separate Citrus configuration file. We will come to this later on. Be sure to always pick the right message endpoint type in order to publish your message to the right destination.

Now that the message sender pattern is clear we can concentrate on how to specify the message content to be sent. There are several possibilities for you to define message content in Citrus:

8.1.1. Send message body

The most important thing when dealing with sending actions is to prepare the message payload and header. You can specify the body as nested String value.

Java

```
@CitrusTest
public void sendMessageTest() {
    $(send("helloService")
        .message()
        .body("""
            <HelloMessage>
                <Text>Hello Citrus!</Text>
            </HelloMessage>
        """)
    );
}
```

XML

```
<test name="SendMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="helloService">
            <message>
                <body>
                    <![CDATA[
                    <HelloMessage>
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                    ]]>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: SendMessageTest
actions:
  - send:
      endpoint: "helloService"
      message:
        body: |
          <HelloMessage>
              <Text>Hello Citrus!</Text>
          </HelloMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SendMessageTest">
        <actions>
            <send endpoint="helloService">
                <message>
                    <payload>
                        <![CDATA[
                        <HelloMessage>
                            <Text>Hello Citrus!</Text>
                        </HelloMessage>
                        ]]>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

A simple way of defining the message body content is to provide the message body as a String. You can do this in the different supported languages by embedding the message content in the body section of the `send` action.

In XML you can embed the content as a `CDATA` section and in Java, or YAML you may want to use text blocks.

When the message body content is an XML payload you can also use nested XML elements in the XML domain specific languages as the next example shows:

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="helloService">
            <message name="helloRequest">
                <body>
                    <HelloMessage xmlns="http://sample.org/">
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                </body>
            </message>
        </send>
    </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ReceiveMessageTest">
        <actions>
            <send endpoint="helloService">
              <message>
                <payload>
                  <HelloMessage xmlns="http://sample.org/">
                      <Text>Hello Citrus!</Text>
                  </HelloMessage>
                </payload>
              </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```



In XML you can use nested XML elements or CDATA sections. Sometimes the nested XML message payload elements may cause XSD schema violations. This is because of variable values not fitting the XSD schema rules for example. In this scenario you could also use simple CDATA sections as payload data. In this case you need to use the `<***data***>` element in contrast to the `<***payload***>` element that we have used in our examples so far.

With this alternative you can skip the XML schema validation from your IDE at design time. Unfortunately you will lose the XSD auto-completion features many XML editors offer when constructing your payload.

Message body content may be quite huge, so you can also load the message content form an external file resource. The file path is given as either a classpath or file system resource.

When writing tests in Java you can use one of the classpath or file system resource implementations to resolve the file path. In XML and other languages you may use a resource path given as: `file="classpath:path/to/request.xml"`. The file path prefix indicates the file resource type (`file:` or `classpath:`), so the file location is resolved either as file system resource (`file:`) or classpath resource (`classpath:`).

Java

```
@CitrusTest
public void sendMessageTest() {
    $(send("helloService")
        .message()
        .body(new ClassPathResource("path/to/request.xml"))
    );
}
```

XML

```
<test name="SendMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="helloService">
            <message>
                <body>
                    <resource file="classpath:path/to/request.xml"/>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: SendMessageTest
actions:
  - send:
      endpoint: "helloService"
      message:
        body:
          resource:
            file: "classpath:path/to/request.xml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SendMessageTest">
        <actions>
            <send endpoint="helloService">
                <message>
                    <resource file="classpath:path/to/request.xml" />
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

In addition to defining message payloads as normal Strings and via external file resource (classpath and file system) you can also provide a POJO model object as a message payload. The model object will be serialized with a marshaller or object mapper implementation which gets loaded from the Citrus context.

You can use the marshalling message payload builders in a `send` action as follows.

Marshalling message payload builder

```
$(send("helloService")
    .message()
    .body(marshal(new TestRequest("Hello Citrus!")))
);
```

The `send` action uses the marshalling message builder provided with Citrus and just provides the model object `new TestRequest()`. The marshalling message builder automatically loads a proper XML message marshaller that should be available as a bean in the project context (e.g. the Spring application context). By default, Citrus is searching for a bean of type **org.citrusframework.api.xml.Marshaller**. You can add the marshaller to your project context as a bean.

Marshaller bean

```
@Bean
public Marshaller xmlMarshaller() {
    return new Jaxb2Marshaller();
}
```

Now Citrus is able to automatically resolve the marshaller from the project context as soon as the `receive` action uses the model object in combination with the `marshal` instruction.

When you have multiple message marshaller instances in your project context you have to tell Citrus which one to use in this particular receive message action.

Reference message marshaller

```
$(send("helloService")
    .message()
    .body(marshal(new TestRequest("Hello Citrus!"), "myMessageMarshallerBean"))
);
```

Now Citrus will marshal the message body with the message marshaller bean named **myMessageMarshallerBean**. This way you can have multiple message marshaller implementations in your project (XML, JSON, and so on).

You can implement your own message payload builder or use one of the provided Citrus message payload builders.

Custom message payload builder

```
@Autowired
private MessagePayloadBuilder mySpecialPayloadBuilder = new FooPayloadBuilder();

$(send("helloService")
    .message()
    .body(mySpecialPayloadBuilder)
);
```

The message payload builder must implement the `MessagePayloadBuilder` interface with the method `buildPayload(TestContext context)`.

MessagePayloadBuilder interface

```
public class FooPayloadBuilder implements MessagePayloadBuilder {

    @Override
    public Object buildPayload(TestContext context) {
        // return some special payload
        return new FooModel();
    }
}
```

You can also use a Citrus message instance directly. Citrus provides different message implementations with fluent APIs to have a convenient way of setting properties (e.g. HttpMessage, MailMessage, FtpMessage, SoapMessage, ...).

Citrus message object

```
$(send("helloService")
    .message(new DefaultMessage("Hello World!"))
);
```

You can explicitly overwrite some message values in the body before the validations is performed. This is for overwriting specific message elements with variable values for instance. Also, you can overwrite values using XPath (xpath) or JsonPath (json-path) expressions.

Java

```
@CitrusTest
public void jsonPathTest() {
    $(receive("someEndpoint")
        .message()
        .type(MessageType.JSON)
        .body(new ClassPathResource("path/to/request.xml"))
        .process(processor().jsonPath()
            .expression("$.user.name", "Penny")
            .expression("$['user']['name']", "${userName}"))
    );
}
```

XML

```
<test name="JsonPathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <message type="json">
                <body>
                    <resource file="classpath:path/to/request.xml" />
                </body>
                <expression path="$.user.name" value="Penny"/>
                <expression path="$['user']['name']" value="${userName}"/>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: JsonPathTest
actions:
  - receive:
      endpoint: "someEndpoint"
      message:
        type: json
        resource:
          file: "classpath:path/to/request.xml"
        expression:
          - path: '$.user.name'
            value: "Penny"
          - path: '$["user"]["name"]'
            value: '${userName}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JsonPathTest">
        <actions>
            <receive endpoint="someEndpoint">
                <message type="json">
                    <resource file="classpath:path/to/request.xml" />
                    <element path="$.user.name" value="Penny"/>
                    <element path="$['user']['name']" value="${userName}"/>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

8.1.2. Send message headers

Defining the message header is an essential part. Citrus uses name-value pairs like "Operation" and "MessageId" in the next example to set message header entries.

Depending on what message endpoint and which message transport underneath is used the header values will be shipped in different ways. In JMS for instance the headers go to the header section of the message, in Http we set mime headers accordingly, in SOAP we can access the SOAP header elements and so on. Citrus knows how to set headers on different message transports and aims to do the hard work for you .

Java

```
@CitrusTest
public void messageHeaderTest() {
    $(send("helloService")
        .message()
        .header("Operation", "sayHello")
        .header("MessageId", "${messageId}")
        .body("""
            <TestMessage xmlns="http://citrusframework.org/schema">
                <Text>Hello!</Text>
            </TestMessage>
        """)
    );
}
```

XML

```
<test name="MessageHeaderTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="someEndpoint">
            <message>
                <headers>
                    <header name="Operation" value="sayHello"/>
                </headers>
                <body>
                    <payload>
                        <TestMessage xmlns="http://citrusframework.org/schema">
                            <Text>Hello!</Text>
                        </TestMessage>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: MessageHeaderTest
actions:
  - send:
      endpoint: "helloService"
      message:
        headers:
          - name: "Operation"
            value: "sayHello"
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schema">
                <Text>Hello!</Text>
            </TestMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="MessageHeaderTest">
        <actions>
            <send endpoint="helloService">
                <message>
                    <payload>
                        <TestMessage xmlns="http://citrusframework.org/schema">
                            <Text>Hello!</Text>
                        </TestMessage>
                    </payload>
                </message>
                <header>
                    <element name="Operation" value="sayHello"/>
                </header>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The message headers to send are defined by a simple name and value pairs. Of course you can use test variables in header values as well.

This is basically how to send messages in Citrus. The test case is responsible for constructing the message content while the predefined message endpoint holds transport specific settings. Test cases reference endpoint components to publish messages to the outside world. The variable support in message payload and message header enables you to add dynamic values before sending out the message.

8.1.3. Groovy Markup builder

With the Groovy markup builder you can build XML message body content in a simple way, without having to write the typical XML overhead.



The Groovy test action support lives in a separate module. You need to add the module to your project to use the functionality.

citrus-groovy dependency module

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-groovy</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

For example, we use a Groovy script to construct the XML message to be sent out. Instead of a plain CDATA XML section or the nested body XML data we write a Groovy script snippet.

Java

```
@CitrusTest
public void scriptMessageBuilderTest() {
    $(send("helloService")
        .message()
        .body(new GroovyScriptPayloadBuilder("""
                    markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                        Message('Hello World!')
                    }
        """))
    );
}
```

XML

```
<test name="ScriptMessageBuilderTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="helloService">
            <message>
                <body>
                    <builder type="groovy">
                        markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                            Message('Hello World!')
                        }
                    </builder>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: ScriptMessageBuilderTest
actions:
  - send:
      endpoint: "helloService"
      message:
        builder:
          type: "groovy"
          value: |
            markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                Message('Hello World!')
            }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ScriptMessageBuilderTest">
        <actions>
            <send endpoint="helloService">
              <message>
                <builder type="groovy">
                    markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                        Message('Hello World!')
                    }
                </builder>
              </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The Groovy markup builder generates the XML message body with following content:

Genereted markup

```
<TestRequest xmlns="https://citrus.schemas/samples/sayHello.xsd">
  <Message>Hello World</Message>
</TestRequest>
```

We use the **builder** element with type **groovy** and the markup builder code is directly written to this element. As you can see from the example above, you can mix XPath and Groovy markup builder code. The markup builder syntax is very easy and follows the simple rule: **markupBuilder.ROOT-ELEMENT{ CHILD-ELEMENTS }** . However the tester has to follow some simple rules and naming conventions when using the Citrus markup builder extension:

- The markup builder is accessed within the script over an object named markupBuilder. The name of the custom root element follows with all its child elements.
- Child elements may be defined within curly brackets after the root-element (the same applies for further nested child elements)
- Attributes and element values are defined within round brackets, after the element name
- Attribute and element values have to stand within apostrophes (e.g. attribute-name: 'attribute-value')

The Groovy markup builder script may also be used as external file resource:

Java

```
@CitrusTest
public void scriptMessageBuilderTest() {
    $(send("helloService")
        .message()
        .body(new
GroovyFileResourcePayloadBuilder("classpath:path/to/helloRequest.groovy"))
    );
}
```

XML

```
<test name="ScriptMessageBuilderTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="helloService">
            <message>
                <body>
                    <builder type="groovy"
file="classpath:path/to/helloRequest.groovy"/>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: ScriptMessageBuilderTest
actions:
  - send:
      endpoint: "helloService"
      message:
        builder:
          type: "groovy"
          file: "classpath:path/to/helloRequest.groovy"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ScriptMessageBuilderTest">
        <actions>
            <send endpoint="helloService">
              <message>
                <builder type="groovy" file="classpath:path/to/helloRequest.groovy"/>
              </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The markup builder implementation in Groovy offers great possibilities in defining message body content. We do not need to write XML the tag overhead anymore. The approach also enables us to construct complex message body content with Groovy script logic like iterations and conditional elements. For detailed markup builder descriptions please see the official Groovy documentation.

8.2. Receive

Receiving and validating messages is an essential part of an integration test. You can perform assertions and checks on incoming messages in order to verify that everything works as expected.

A message consists of a message header (name-value pairs) and a message body. You can specify expected message content on a receive message test action. Citrus will perform validation steps and raise errors in case the incoming message does not match these expectations.

Java

```
@CitrusTest
public void receiveMessageTest() {
    $(receive("helloService")
        .message()
        .name("helloRequest")
        .header("Operation", "sayHello")
        .header("MessageId", "${messageId}")
        .body("""
            <HelloMessage>
                <Text>Hello Citrus!</Text>
            </HelloMessage>
        """)
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message name="helloRequest">
                <headers>
                    <header name="Operation" value="sayHello"/>
                    <header name="MessageId" value="${messageId}"/>
                </headers>
                <body>
                    <![CDATA[
                    <HelloMessage>
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                    ]]>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        name: "helloRequest"
        headers:
          - name: "Operation"
            value: "sayHello"
          - name: "MessageId"
            value: '${messageId}'
        body: |
          <HelloMessage>
              <Text>Hello Citrus!</Text>
          </HelloMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ReceiveMessageTest">
        <actions>
            <receive endpoint="helloService">
                <message name="helloRequest">
                    <payload>
                        <![CDATA[
                        <HelloMessage>
                            <Text>Hello Citrus!</Text>
                        </HelloMessage>
                        ]]>
                    </payload>
                </message>
                <header>
                    <element name="Operation" value="sayHello"/>
                    <element name="MessageId" value="${messageId}"/>
                </header>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The message name is optional and defines the message identifier in the local message store. This message name is very useful when accessing the message content later on during the test case. The local message store is handled per test case and contains all exchanged messages.

The test action waits for a message to arrive. The whole test execution is blocked while waiting for the expected message. This is important to ensure the step by step test workflow processing. Of course, you can specify a message timeout setting so the receiver will only wait a given amount of time before raising a timeout error. Following from that timeout exception the test case fails as the message did not arrive in time. Citrus defines default timeout settings for all message receiving tasks.

In case the message arrives in time, the test action moves on to validate the message content as a next step. The user is able to choose from different validation capabilities. On the one hand you can specify a whole message body content that you expect as control template. In this case the received message structure is compared to the expected message content element by element. On the other hand you can use explicit element validation where only a small subset of message elements is validated.

In addition to verifying the message body content, Citrus will also perform validation on the received message header values. Test variable usage is supported as usual during the whole validation process for body and header checks.

In general the validation component (validator) in Citrus works hand in hand with a message receiving component as the following figure shows:



The message receiving component passes the message to the validator where the individual validation steps are performed. Let us have a closer look at the validation options and features step by step.

8.2.1. Validate message body

The most detailed validation of incoming messages is to define some expected message body. The Citrus message validator will then perform a detailed message body comparison. The incoming message has to match exactly to the expected message body. The different message validator implementations in Citrus provide deep comparison of message structures such as XML, JSON and so on.

So by defining an expected message body we validate the incoming message in syntax and semantics. In case a difference is identified by the message validator the validation and the test case fails with respective exceptions. This is how you can define message body content in receive action:

Java

```
@CitrusTest
public void receiveMessageTest() {
    $(receive("helloService")
        .message()
        .body("""
            <HelloMessage>
                <Text>Hello Citrus!</Text>
            </HelloMessage>
        """)
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message>
                <body>
                    <![CDATA[
                    <HelloMessage>
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                    ]]>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        body: |
          <HelloMessage>
              <Text>Hello Citrus!</Text>
          </HelloMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ReceiveMessageTest">
        <actions>
            <receive endpoint="helloService">
                <message>
                    <payload>
                        <![CDATA[
                        <HelloMessage>
                            <Text>Hello Citrus!</Text>
                        </HelloMessage>
                        ]]>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

A simple way of defining the expected message body content is to provide the message body as a String. You can do this in the different supported languages by embedding the message content in the body section of the `receive` action.

In XML you can embed the content as a `CDATA` section and in Java, or YAML you may want to use text blocks.

When the message body content is an XML payload you can also use nested XML elements in the XML domain specific languages as the next example shows:

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message name="helloRequest">
                <body>
                    <HelloMessage xmlns="http://sample.org/">
                        <Text>Hello Citrus!</Text>
                    </HelloMessage>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ReceiveMessageTest">
        <actions>
            <receive endpoint="helloService">
              <message>
                <payload>
                  <HelloMessage xmlns="http://sample.org/">
                      <Text>Hello Citrus!</Text>
                  </HelloMessage>
                </payload>
              </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```



In XML you can use nested XML elements or CDATA sections. Sometimes the nested XML message payload elements may cause XSD schema violations. This is because of variable values not fitting the XSD schema rules for example. In this scenario you could also use simple CDATA sections as payload data. In this case you need to use the `<***data***>` element in contrast to the `<***payload***>` element that we have used in our examples so far.

With this alternative you can skip the XML schema validation from your IDE at design time. Unfortunately you will lose the XSD auto-completion features many XML editors offer when constructing your payload.

Message body content may be quite huge, so you can also load the message content form an external file resource. The file path is given as either a classpath or file system resource.

When writing tests in Java you can use one of the classpath or file system resource implementations to resolve the file path. In XML and other languages you may use a resource path given as: `file="classpath:path/to/request.xml"`. The file path prefix indicates the file resource type (`file:` or `classpath:`), so the file location is resolved either as file system resource (`file:`) or classpath resource (`classpath:`).

Java

```
@CitrusTest
public void receiveMessageTest() {
    $(receive("helloService")
        .message()
        .body(new ClassPathResource("path/to/request.xml"))
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message>
                <body>
                    <resource file="classpath:path/to/request.xml"/>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        body:
          resource:
            file: "classpath:path/to/request.xml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ReceiveMessageTest">
        <actions>
            <receive endpoint="helloService">
                <message>
                    <resource file="classpath:path/to/request.xml" />
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

In addition to defining message payloads as normal Strings and via external file resource (classpath and file system) you can also provide a POJO model object as a message payload. The model object will be serialized with a marshaller or object mapper implementation which gets loaded from the Citrus context.

You can use the marshalling message payload builders in a `receive` action as follows.

Marshalling message payload builder

```
$(receive("helloService")
    .message()
    .body(marshal(new TestRequest("Hello Citrus!")))
);
```

The `receive` action uses the marshalling message builder provided with Citrus and just provides the model object `new TestRequest()`. The marshalling message builder automatically loads a proper XML message marshaller that should be available as a bean in the project context (e.g. the Spring application context). By default, Citrus is searching for a bean of type **org.citrusframework.api.xml.Marshaller**. You can add the marshaller to your project context as a bean.

Marshaller bean

```
@Bean
public Marshaller xmlMarshaller() {
    return new Jaxb2Marshaller();
}
```

Now Citrus is able to automatically resolve the marshaller from the project context as soon as the `receive` action uses the model object in combination with the `marshal` instruction.

When you have multiple message marshaller instances in your project context you have to tell Citrus which one to use in this particular receive message action.

Reference message marshaller

```
$(receive("helloService")
    .message()
    .body(marshal(new TestRequest("Hello Citrus!"), "myMessageMarshallerBean"))
);
```

Now Citrus will marshal the message body with the message marshaller bean named **myMessageMarshallerBean**. This way you can have multiple message marshaller implementations in your project (XML, JSON, and so on).

You can implement your own message payload builder or use one of the provided Citrus message payload builders.

Custom message payload builder

```
@Autowired
private MessagePayloadBuilder mySpecialPayloadBuilder = new FooPayloadBuilder();

$(receive("helloService")
    .message()
    .body(mySpecialPayloadBuilder)
);
```

The message payload builder must implement the `MessagePayloadBuilder` interface with the method `buildPayload(TestContext context)`.

MessagePayloadBuilder interface

```
public class FooPayloadBuilder implements MessagePayloadBuilder {

    @Override
    public Object buildPayload(TestContext context) {
        // return some special payload
        return new FooModel();
    }
}
```

You can also use a Citrus message instance directly. Citrus provides different message implementations with fluent APIs to have a convenient way of setting properties (e.g. HttpMessage, MailMessage, FtpMessage, SoapMessage, ...).

Citrus message object

```
$(receive("helloService")
    .message(new DefaultMessage("Hello World!"))
);
```

You can explicitly overwrite some message values in the body before the validations is performed. This is for overwriting specific message elements with variable values for instance. Also, you can overwrite values using XPath (xpath) or JsonPath (json-path) expressions.

Java

```
@CitrusTest
public void jsonPathTest() {
    $(receive("someEndpoint")
        .message()
        .type(MessageType.JSON)
        .body(new ClassPathResource("path/to/request.xml"))
        .validate(validation().jsonPath()
            .expression("$.user.name", "Penny")
            .expression("$['user']['name']", "${userName}"))
    );
}
```

XML

```
<test name="JsonPathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <message type="json">
                <body>
                    <resource file="classpath:path/to/request.xml" />
                </body>
            </message>
            <validate path="$.user.name" value="Penny"/>
            <validate path="$['user']['name']" value="${userName}"/>
        </receive>
    </actions>
</test>
```

YAML

```
name: JsonPathTest
actions:
  - receive:
      endpoint: "someEndpoint"
      message:
        type: json
        resource:
          file: "classpath:path/to/request.xml"
        validate:
          jsonPath:
            - expression: '$.user.name'
              value: "Penny"
            - expression: '$["user"]["name"]'
              value: '${userName}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JsonPathTest">
        <actions>
            <receive endpoint="someEndpoint">
                <message type="json">
                    <resource file="classpath:path/to/request.xml" />
                    <validate path="$.user.name" value="Penny"/>
                    <validate path="$['user']['name']" value="${userName}"/>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

In addition to that you can ignore some elements that are skipped in comparison. We will describe this later on in this section. Now let's continue with message header validation.

8.2.2. Validate message headers

Message headers are used widely in enterprise messaging. The message headers often represent a critical part of the message semantics and need to be validated, too. Citrus is able to validate message headers by name and value.

Java

```
@CitrusTest
public void messageHeaderTest() {
    $(receive("helloService")
        .message()
        .header("Operation", "sayHello")
        .body("""
            <TestMessage xmlns="http://citrusframework.org/schema">
                <Text>Hello!</Text>
            </TestMessage>
        """)
    );
}
```

XML

```
<test name="MessageHeaderTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <message>
                <headers>
                    <header name="Operation" value="sayHello"/>
                </headers>
                <body>
                    <payload>
                        <TestMessage xmlns="http://citrusframework.org/schema">
                            <Text>Hello!</Text>
                        </TestMessage>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: MessageHeaderTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        headers:
          - name: "Operation"
            value: "sayHello"
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schema">
                <Text>Hello!</Text>
            </TestMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="MessageHeaderTest">
        <actions>
            <receive endpoint="helloService">
                <message>
                    <payload>
                        <TestMessage xmlns="http://citrusframework.org/schema">
                            <Text>Hello!</Text>
                        </TestMessage>
                    </payload>
                </message>
                <header>
                    <element name="Operation" value="sayHello"/>
                </header>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The expected message headers are defined by a name and value pair. Citrus will check that the expected message header is present and will verify the value accordingly. In case the message header is not found or the value does not match Citrus will raise an exception and the test fails. You can use validation matchers (validation-matcher) for a more powerful validation of header values, too.

Header definition in Java DSL is straight forward as we just define name and value as usual. This completes tha basic message validation in Citrus. The individual message validator implementations may add additional validation capabilities such as XML schema validation or XPath and JSONPath validation. Please refer to the respective chapters in this guide to learn more about that.

8.2.3. Ignore elements

Sometimes a tester can not verify all values because specifying expected values is not possible for non deterministic values (e.g. timestamps, dynamic and generated identifiers).

You can use a path expressions (e.g. Xpath, JsonPath) to ignoring a very specific entry in the message body.

Java

```
@CitrusTest
public void ignorePathTest() {
    $(receive(someEndpoint)
        .message()
        .type(MessageType.JSON)
        .body("""
            {
              "users":
              [{
                "name": "Jane",
                "token": "?",
                "lastLogin": 0
              },
              {
                "name": "Penny",
                "token": "?",
                "lastLogin": 0
              },
              {
                "name": "Mary",
                "token": "?",
                "lastLogin": 0
              }]
            }
        """)
        .validate(validation().json()
                    .ignore("$.users[*].token")
                    .ignore("$..lastLogin"))
    );
}
```

XML

```
<test name="IgnorePathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <message type="json">
                <body>
                    <data>
                    {
                      "users":
                      [{
                        "name": "Jane",
                        "token": "?",
                        "lastLogin": 0
                      },
                      {
                        "name": "Penny",
                        "token": "?",
                        "lastLogin": 0
                      },
                      {
                        "name": "Mary",
                        "token": "?",
                        "lastLogin": 0
                      }]
                    }
                    </data>
                </body>
            </message>
            <ignore path="$..lastLogin" />
            <ignore path="$.users[*].token" />
        </receive>
    </actions>
</test>
```

YAML

```
name: IgnorePathTest
actions:
  - receive:
      endpoint: "someEndpoint"
      message:
        type: "json"
        body:
          data: |
            {
              "users":
              [{
                "name": "Jane",
                "token": "?",
                "lastLogin": 0
              },
              {
                "name": "Penny",
                "token": "?",
                "lastLogin": 0
              },
              {
                "name": "Mary",
                "token": "?",
                "lastLogin": 0
              }]
            }
          ignore:
            - path: "$.users[*].token"
            - path: "$..lastLogin"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="IgnorePathTest">
        <actions>
            <receive endpoint="someEndpoint">
                <message type="json">
                    <data>
                    {
                      "users":
                      [{
                        "name": "Jane",
                        "token": "?",
                        "lastLogin": 0
                      },
                      {
                        "name": "Penny",
                        "token": "?",
                        "lastLogin": 0
                      },
                      {
                        "name": "Mary",
                        "token": "?",
                        "lastLogin": 0
                      }]
                    }
                    </data>
                    <ignore path="$.users[*].token" />
                    <ignore path="$..lastLogin" />
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The sample above adds JsonPath expressions as ignore statements. This means that we explicitly leave out the evaluated elements from validation. In the example above we explicitly skip the **token** entry and all **lastLogin** values that are obviously timestamp values in milliseconds.

The path evaluation is very powerful when it comes to select a set of objects and elements. This is how you can ignore several elements with path expressions.

As an alternative you can also use the `@ignore@` placeholder in the message content. The placeholder tells Citrus to skip the element in validation.

Java

```
@CitrusTest
public void ignorePathTest() {
    $(receive(someEndpoint)
        .message()
        .type(MessageType.JSON)
        .body("""
            {
              "users":
              [{
                "name": "Jane",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              },
              {
                "name": "Penny",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              },
              {
                "name": "Mary",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              }]
            }
        """)
    );
}
```

XML

```
<test name="IgnorePathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <message type="json">
                <body>
                    <data>
                    {
                      "users":
                      [{
                        "name": "Jane",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      },
                      {
                        "name": "Penny",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      },
                      {
                        "name": "Mary",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      }]
                    }
                    </data>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: IgnorePathTest
actions:
  - receive:
      endpoint: "someEndpoint"
      message:
        type: "json"
        body:
          data: |
            {
              "users":
              [{
                "name": "Jane",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              },
              {
                "name": "Penny",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              },
              {
                "name": "Mary",
                "token": "@ignore@",
                "lastLogin": "@ignore@"
              }]
            }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="IgnorePathTest">
        <actions>
            <receive endpoint="someEndpoint">
                <message type="json">
                  <data>
                    {
                      "users":
                      [{
                        "name": "Jane",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      },
                      {
                        "name": "Penny",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      },
                      {
                        "name": "Mary",
                        "token": "@ignore@",
                        "lastLogin": "@ignore@"
                      }]
                    }
                  </data>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```



You can also ignore a whole sub-tree in XML and whole objects and arrays in Json with the ignore expression/placeholder.



The `ignore` expression as well as the `@ignore@` placeholder will only skip the value matching validations for the selected element or object. The element still has to be present in the message structure. In case the element is missing for any reason the validation fails even for ignored values.

8.2.4. Message selectors

The `<selector>` element inside the receiving action defines key-value pairs in order to filter the messages being received. The filter applies to the message headers. This means that a receiver will only accept messages matching a header element value. In messaging applications the header information often holds message ids, correlation ids, operation names and so on. With this information given you can explicitly listen for messages that belong to your test case. This is very

helpful to avoid receiving messages that are still available on the message destination.

Let's say the tested software application keeps sending messages that belong to previous test cases. This could happen in retry situations where the application error handling automatically tries to solve a communication problem that occurred during previous test cases. As a result a message destination (e.g. a JMS message queue) contains messages that are not valid anymore for the currently running test case. The test case might fail because the received message does not apply to the actual use case. So we will definitely run into validation errors as the expected message control values do not match.

Now we have to find a way to avoid these problems. The test could filter the messages on a destination to only receive messages that apply for the use case that is being tested. The Java Messaging System (JMS) came up with a message header selector that will only accept messages that fit the expected header values.

Let us have a closer look at a message selector inside a receiving action:

Java

```
@CitrusTest
public void selectorTest() {
    Map<String, String> selectorMap = new HashMap<>();
    selectorMap.put("correlationId", "Cx1x123456789");
    selectorMap.put("operation", "getOrders");

    $(receive(someEndpoint)
        .selector(selectorMap)
        .message()
        .body("...")
    );
}
```

XML

```
<test name="SelectorTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <selector>
                <element name="correlationId" value="Cx1x123456789"/>
                <element name="operation" value="getOrders"/>
            </selector>
            <message>
                <!-- -->
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: SelectorTest
actions:
  - receive:
      endpoint: "someEndpoint"
      selector:
        element:
          - name: correlationId
            value: "Cx1x123456789"
          - name: operation
            value: "getOrders"
      message:
        # ...
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SelectorTest">
        <actions>
            <receive endpoint="someEndpoint">
              <selector>
                <element name="correlationId" value="Cx1x123456789"/>
                <element name="operation" value="getOrders"/>
              </selector>
              <!-- ... -->
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

This example shows how message selectors work. The selector will only accept messages that meet the correlation id and the operation in the header values. All other messages on the message destination are ignored. The selector elements are automatically associated to each other using the logical AND operator. This means that the message selector string would look like this:

```
correlationId = 'Cx1x123456789' AND operation = 'getOrders'
```

Instead of using several elements in the selector you can also define a selector string directly which gives you more power in constructing the selection logic yourself. This way you can use **AND** logical operators yourself.

Java

```
@CitrusTest
public void selectorTest() {
    $(receive(someEndpoint)
        .selector("correlationId = 'Cx1x123456789' AND operation = 'getOrders'")
        .message()
        .body("...")
    );
}
```

XML

```
<test name="SelectorTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="someEndpoint">
            <selector>
                <value>
                    correlationId = 'Cx1x123456789' AND operation = 'getOrders'
                </value>
            </selector>
            <message>
                <!-- -->
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: SelectorTest
actions:
  - receive:
      endpoint: "someEndpoint"
      selector:
        value: "correlationId = 'Cx1x123456789' AND operation = 'getOrders'"
      message:
        # ...
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SelectorTest">
        <actions>
            <receive endpoint="someEndpoint">
              <selector>
                <value>
                    correlationId = 'Cx1x123456789' AND operation = 'getOrders'
                </value>
              </selector>
              <!-- ... -->
            </receive>
        </actions>
    </testcase>
</spring:beans>
```



In case you want to run tests in parallel message selectors become essential in your test cases. The different tests running at the same time will steal messages from each other when you lack of message selection mechanisms.

8.2.5. Groovy Markup builder

With the Groovy markup builder you can build XML message body content in a simple way, without having to write the typical XML overhead.



The Groovy test action support lives in a separate module. You need to add the module to your project to use the functionality.

citrus-groovy dependency module

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-groovy</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

For example, we use a Groovy script to construct the XML message to be sent out. Instead of a plain CDATA XML section or the nested body XML data we write a Groovy script snippet.

Java

```
@CitrusTest
public void scriptMessageBuilderTest() {
    $(receive("helloService")
        .message()
        .body(new GroovyScriptPayloadBuilder("""
                    markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                        Message('Hello World!')
                    }
        """))
    );
}
```

XML

```
<test name="ScriptMessageBuilderTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message>
                <body>
                    <builder type="groovy">
                        markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                            Message('Hello World!')
                        }
                    </builder>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: ScriptMessageBuilderTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        builder:
          type: "groovy"
          value: |
            markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                Message('Hello World!')
            }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ScriptMessageBuilderTest">
        <actions>
            <receive endpoint="helloService">
              <message>
                <builder type="groovy">
                    markupBuilder.TestRequest(xmlns:
'https://citrus.schemas/samples/sayHello.xsd') {
                        Message('Hello World!')
                    }
                </builder>
              </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The Groovy markup builder generates the XML message body with following content:

Generated markup

```
<TestRequest xmlns="https://citrus.schemas/samples/sayHello.xsd">
  <Message>Hello World</Message>
</TestRequest>
```

We use the **builder** element with type **groovy** and the markup builder code is directly written to this element. As you can see from the example above, you can mix XPath and Groovy markup builder code. The markup builder syntax is very easy and follows the simple rule: **markupBuilder.ROOT-ELEMENT{ CHILD-ELEMENTS }** . However the tester has to follow some simple rules and naming conventions when using the Citrus markup builder extension:

- The markup builder is accessed within the script over an object named markupBuilder. The name of the custom root element follows with all its child elements.
- Child elements may be defined within curly brackets after the root-element (the same applies for further nested child elements)
- Attributes and element values are defined within round brackets, after the element name
- Attribute and element values have to stand within apostrophes (e.g. attribute-name: 'attribute-value')

The Groovy markup builder script may also be used as external file resource:

Java

```
@CitrusTest
public void scriptMessageBuilderTest() {
    $(receive("helloService")
        .message()
        .body(new
GroovyFileResourcePayloadBuilder("classpath:path/to/helloRequest.groovy"))
    );
}
```

XML

```
<test name="ScriptMessageBuilderTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="helloService">
            <message>
                <body>
                    <builder type="groovy"
file="classpath:path/to/helloRequest.groovy"/>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: ScriptMessageBuilderTest
actions:
  - receive:
      endpoint: "helloService"
      message:
        builder:
          type: "groovy"
          file: "classpath:path/to/helloRequest.groovy"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ScriptMessageBuilderTest">
        <actions>
            <receive endpoint="helloService">
              <message>
                <builder type="groovy" file="classpath:path/to/helloRequest.groovy"/>
              </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The markup builder implementation in Groovy offers great possibilities in defining message body content. We do not need to write XML the tag overhead anymore. The approach also enables us to construct complex message body content with Groovy script logic like iterations and conditional elements. For detailed markup builder descriptions please see the official Groovy documentation.

8.3. SQL

In many cases it is necessary to access the database during a test. This enables a tester to also validate the persistent data in a database. It might also be helpful to prepare the database with some test data before running a test. You can do this using the two database actions that are described in the following sections.



The SQL test actions live in a separate module. You need to add the module to your project to use the actions.

citrus-sql dependency module

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-sql</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

In general Citrus handles SELECT statements differently to other statements like INSERT, UPDATE and DELETE. When executing an SQL query with SELECT you are able to add validation steps on the result sets returned from the database. This is not allowed when executing update statements like INSERT, UPDATE, DELETE.



Do not mix statements of type ***SELECT*** with others in a single sql test action. This will lead to errors because validation steps are not valid for statements other than SELECT. Please use separate test actions for update statements.

8.3.1. SQL update, insert, delete

The <sql> action simply executes a group of SQL statements in order to change data in a database. Typically the action is used to prepare the database at the beginning of a test or to clean up the database at the end of a test. You can specify SQL statements like INSERT, UPDATE, DELETE, CREATE TABLE, ALTER TABLE and many more.

On the one hand you can specify the statements as inline SQL or stored in an external SQL resource file as shown in the next two examples.

Java

```
@Autowired
@Qualifier("myDataSource")
private DataSource dataSource;

@CitrusTest
public void sqlTest() {
    $(sql()
        .dataSource(dataSource)
        .statement("DELETE FROM CUSTOMERS")
        .statement("DELETE FROM ORDERS"))
    );

    $(sql()
        .dataSource(dataSource)
        .sqlResource("file:tests/unit/resources/script.sql"))
    );
}
```

XML

```
<test name="SQLTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <sql datasource="myDataSource">
            <statements>
                <statement>DELETE FROM CUSTOMERS</statement>
                <statement>DELETE FROM ORDERS</statement>
            </statements>
        </sql>

        <sql datasource="myDataSource">
            <statements file="file:tests/unit/resources/script.sql"/>
        </sql>
    </actions>
</test>
```

YAML

```
name: "SQLTest"
actions:
  - sql:
      dataSource: "myDataSource"
      statements:
        - statement: "DELETE FROM CUSTOMERS"
        - statement: "DELETE FROM ORDERS"

  - sql:
      dataSource: "myDataSource"
      statements:
        - file: "file:tests/unit/resources/script.sql"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SQLTest">
        <actions>
            <sql datasource="myDataSource">
                <statement>DELETE FROM CUSTOMERS</statement>
                <statement>DELETE FROM ORDERS</statement>
            </sql>

            <sql datasource="myDataSource">
                <resource file="file:tests/unit/resources/script.sql"/>
            </sql>
        </actions>
    </testcase>
</spring:beans>
```

The first action uses inline SQL statements defined directly inside the test case. The next action uses an external SQL resource file instead. The file resource can hold several SQL statements separated by new lines. All statements inside the file are executed sequentially by the framework.



You have to pay attention to some rules when dealing with external SQL resources.

- Each statement should begin in a new line
- It is not allowed to define statements with word wrapping
- Comments begin with two dashes "–"



The external file is referenced either as file system resource or class path resource, by using the "file:" or "classpath:" prefix.

Both examples use the "datasource" attribute. This value defines the database data source to be used. The connection to a data source is mandatory, because the test case does not know about user

credentials or database names. The 'datasource' attribute references predefined data sources that are located in a separate Spring configuration file.

8.3.2. SQL query

The <sql> query action is specially designed to execute SQL queries (SELECT * FROM). So the test is able to read data from a database. The query results are validated against expected data as shown in the next example.

Java

```
@Autowired
@Qualifier("myDataSource")
private DataSource dataSource;

@CitrusTest
public void sqlQueryTest() {
    $(sql()
        .dataSource(dataSource)
        .query()
            .statement("select NAME from CUSTOMERS where CUSTOMER_ID='${customerId}'")
            .statement("select COUNT(1) as overall_cnt from ERRORS")
            .statement("select ORDER_ID from ORDERS where DESCRIPTION LIKE
'Migrate%'")
            .statement("select DESCRIPTION from ORDERS where ORDER_ID = 2")
        .validate("ORDER_ID", "1")
        .validate("NAME", "Christoph")
        .validate("OVERALL_CNT", "${rowsCount}")
        .validate("DESCRIPTION", "NULL"));
}
```

XML

```
<test name="SqlQueryTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <sql datasource="myDataSource">
            <statements>
                <statement>select NAME from CUSTOMERS where
ID='${customerId}'</statement>
                <statement>select count(*) from ERRORS</statement>
                <statement>select ID from ORDERS where DESC LIKE 'Def%'</statement>
                <statement>select DESCRIPTION from ORDERS where ID='${id}'</statement>
            </statements>
            <validate column="ID" value="1"/>
            <validate column="NAME" value="Christoph"/>
            <validate column="COUNT(*)" value="${rowsCount}"/>
            <validate column="DESCRIPTION" value="null"/>
        </sql>
    </actions>
</test>
```

YAML

```
name: "SqlQueryTest"
actions:
  - sql:
      dataSource: "dataSource"
      statements:
        - statement: "select NAME from CUSTOMERS where CUSTOMER_ID='${customerId}'"
        - statement: "select COUNT(1) as overall_cnt from ERRORS"
        - statement: "select ORDER_ID from ORDERS where DESCRIPTION LIKE 'Migrate%'"
        - statement: "select DESCRIPTION from ORDERS where ORDER_ID = 2"
      validate:
        - column: "ID"
          value: "1"
        - column: "NAME"
          value: "Christoph"
        - column: "COUNT(*)"
          value: '${rowsCount}'
        - column: "DESCRIPTION"
          value: "null"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- ... -->
    <sql datasource="myDataSource">
        <statement>select NAME from CUSTOMERS where ID='${customerId}'</statement>
        <statement>select count(*) from ERRORS</statement>
        <statement>select ID from ORDERS where DESC LIKE 'Def%'</statement>
        <statement>select DESCRIPTION from ORDERS where ID='${id}'</statement>

        <validate column="ID" value="1"/>
        <validate column="NAME" value="Christoph"/>
        <validate column="COUNT(*)" value="${rowsCount}"/>
        <validate column="DESCRIPTION" value="null"/>
    </sql>
</spring:beans>
```

The action offers a wide range of validating functionality for database result sets. First of all you have to select the data via SQL statements. Here again you have the choice to use inline SQL statements or external file resource pattern.

The result sets are validated through <validate> elements. It is possible to do a detailed check on every selected column of the result set. Simply refer to the selected column name in order to validate its value. The usage of test variables is supported as well as database expressions like count(), avg(), min(), max().

You simply define the <validate> entry with the column name as the "column" attribute and any expected value expression as expected "value". The framework then will check the column to fit the expected value and raise validation errors in case of mismatch.

Looking at the first SELECT statement in the example you will see that test variables are supported in the SQL statements. The framework will replace the variable with its respective value before sending it to the database.

In the validation section variables can be used too. Look at the third validation entry, where the variable "${rowsCount}" is used. The last validation in this example shows, that ***NULL*** values are also supported as expected values.

If a single validation happens to fail, the whole action will fail with respective validation errors.



The validation with *"<**validate column**="..." **value**="..."/>"* meets single row result sets as you specify a single column control value. In case you have multiple rows in a result set you can pass a variable argument list to the validate method like this:

Java

```
$(sql()
    .query()
    .statement("...")
    .validate("SOME_COLUMN",
        "Value in 1st row",
        "Value in 2nd row",
        "Value in 3rd row",
        "Value in x row")
);
```

XML

```
<sql datasource="myDataSource">
    <statements>
        <statement>...</statement>
    </statement>
    <validate column="SOME_COLUMN">
      <values>
          <value>Value in 1st row</value>
          <value>Value in 2nd row</value>
          <value>Value in 3rd row</value>
          <value>Value in x row</value>
      </values>
    </validate>
</sql>
```

YAML

```
- sql:
  dataSource: myDataSource
  statements:
    - statement: {}
  validate:
    - column: "SOME_COLUMN"
      values:
        - "Value in 1st row"
        - "Value in 2nd row"
        - "Value in 3rd row"
        - "Value in x row"
```

Spring XML

```
<sql dataSource="myDataSource">
    <statement>...</statement>
    <validate column="SOME_COLUMN">
      <values>
          <value>Value in 1st row</value>
          <value>Value in 2nd row</value>
          <value>Value in 3rd row</value>
          <value>Value in x row</value>
      </values>
    </validate>
</sql>
```

Next example shows how to work with multiple row result sets and multiple values to expect within one column:

Java

```
$(sql()
    .dataSource(myDataSource)
    .query()
    .statement("select WEEKDAY as DAY, DESCRIPTION from WEEK")
    .validate("DAY",
        "Monday",
        "Tuesday",
        "Wednesday",
        "Thursday",
        "Friday",
        "@ignore@",
        "@ignore@")
    .validate("DESCRIPTION",
        "I hate Mondays!",
        "Tuesday is sports day",
        "The mid of the week",
        "Thursday we play chess",
        "Friday, the weekend is near!",
        "@ignore@",
        "@ignore@")
);
```

XML

```
<sql datasource="myDataSource">
    <statements>
        <statement>select WEEKDAY as DAY, DESCRIPTION from WEEK</statement>
    </statements>
    <validate column="DAY">
      <values>
          <value>Monday</value>
          <value>Tuesday</value>
          <value>Wednesday</value>
          <value>Thursday</value>
          <value>Friday</value>
          <value>@ignore@</value>
          <value>@ignore@</value>
      </values>
    </validate>
    <validate column="DESCRIPTION">
      <values>
          <value>I hate Mondays!</value>
          <value>Tuesday is sports day</value>
          <value>The mid of the week</value>
          <value>Thursday we play chess</value>
          <value>Friday, the weekend is near!</value>
          <value>@ignore@</value>
          <value>@ignore@</value>
      </values>
    </validate>
</sql>
```

YAML

```
- sql:
  dataSource: "myDataSource"
  statements:
    - statement: "select WEEKDAY as DAY, DESCRIPTION from WEEK"
  validate:
    - column: "SOME_COLUMN"
      values:
        - "Value in 1st row"
        - "Value in 2nd row"
        - "Value in 3rd row"
        - "Value in x row"
```

Spring XML

```
<sql datasource="myDataSource">
    <statement>select WEEKDAY as DAY, DESCRIPTION from WEEK</statement>
    <validate column="DAY">
      <values>
          <value>Monday</value>
          <value>Tuesday</value>
          <value>Wednesday</value>
          <value>Thursday</value>
          <value>Friday</value>
          <value>@ignore@</value>
          <value>@ignore@</value>
      </values>
    </validate>
    <validate column="DESCRIPTION">
      <values>
          <value>I hate Mondays!</value>
          <value>Tuesday is sports day</value>
          <value>The mid of the week</value>
          <value>Thursday we play chess</value>
          <value>Friday, the weekend is near!</value>
          <value>@ignore@</value>
          <value>@ignore@</value>
      </values>
    </validate>
</sql>
```

For the validation of multiple rows the `<*validate*>` element is able to host a list of control values for a column. As you can see from the example above, you have to add a control value for each row in the result set. This also means that we have to take care of the total number of rows. Fortunately we can use the ignore placeholder, in order to skip the validation of a specific row in the result set. Functions and variables are supported as usual.



It is important, that the control values are defined in the correct order, because they are compared one on one with the actual result set coming from database query. You may need to add "order by" SQL expressions to get the right order of rows returned. If any of the values fails in validation or the total number of rows is not equal, the whole action will fail with respective validation errors.

8.3.3. Transaction management

By default no transactions are used when Citrus executes SQL statements on a datasource. You can enable transaction management by selecting a transaction manager.

Java

```
$(sql()
    .dataSource(myDataSource)
    .transactionManager(transactionManager)
    .transactionTimeout(15000)
    .transactionIsolationLevel("ISOLATION_READ_COMMITTED")
    .statement("DELETE FROM CUSTOMERS")
    .statement("DELETE FROM ORDERS")
);
```

XML

```
<sql datasource="someDataSource">
    <transaction>
        <manager>"someTransactionManager"</manager>
        <timeout>"15000"</timeout>
        <isolation-level>"ISOLATION_READ_COMMITTED"</isolation-level>
    </transaction>
    <statements>
        <statement>DELETE FROM CUSTOMERS</statement>
        <statement>DELETE FROM ORDERS</statement>
    </statements>
</sql>
```

YAML

```
- sql:
    datasource: "myDataSource"
    transaction:
      manager: "someTransactionManager"
      timeout: 15000
      isolation-level: "ISOLATION_READ_COMMITTED"
    statements:
      - statement: "DELETE FROM CUSTOMERS"
      - statement: "DELETE FROM ORDERS"
```

Spring XML

```
<sql datasource="myDataSource"
     transaction-manager="someTransactionManager"
     transaction-timeout="15000"
     transaction-isolation-level="ISOLATION_READ_COMMITTED">
    <statement>DELETE FROM CUSTOMERS</statement>
    <statement>DELETE FROM ORDERS</statement>
</sql>
```

The *transaction-manager* attribute references a Spring bean of type

"*org.springframework.transaction.PlatformTransactionManager*". You can add this transaction manager to the Spring bean configuration:

Spring Bean

```
@Bean
public TransactionManager myTransactionManager(DataSource myDataSource) {
    return new DataSourceTransactionManager(myDataSource);
}
```

Spring XML

```
<bean id="myTransactionManager"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <constructor-arg ref="myDataSource"/>
</bean>
```

The transaction isolation level as well as the transaction timeout get set on the transaction definition used during SQL statement execution. The isolation level should evaluate to one of the constants given in "*org.springframework.transaction.TransactionDefinition*". Valid isolation level are:

- ISOLATION_DEFAULT
- ISOLATION_READ_UNCOMMITTED
- ISOLATION_READ_COMMITTED
- ISOLATION_REPEATABLE_READ
- ISOLATION_SERIALIZABLE

8.3.4. Groovy SQL result set validation

Groovy provides great support for accessing Java list objects and maps. As a Java SQL result set is nothing but a list of map representations, where each entry in the list defines a row in the result set and each map entry represents the columns and values. So with Groovy's list and map access we have great possibilities to validate a SQL result set - out of the box.

Java

```
$(sql()
    .dataSource(myDataSource)
    .query()
    .statement("select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'")
    .validateScript("assert rows.size() == 2;" +
            "assert rows[0].ID == '1';" +
            "assert rows[0].STATUS == 'in progress';", "groovy")
);
```

XML

```
<sql datasource="myDataSource">
    <statements>
        <statement>select ID from CUSTOMERS where NAME='${customerName}'</statement>
        <statement>select ORDERTYPE, STATUS from ORDERS where
ID='${orderId}'</statement>
    </statements>
    <validate>
        <script type="groovy">
          assert rows.size() == 2
          assert rows[0].ID == '1'
          assert rows[1].STATUS == 'in progress'
          assert rows[1] == [ORDERTYPE:'SampleOrder', STATUS:'in progress']
        </script>
    </validate>
</sql>
```

YAML

```
- sql:
  dataSource: "myDataSource"
  statements:
    - statement: "select ID from CUSTOMERS where NAME='${customerName}'"
    - statement: "select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'"
  validate:
    - script:
        type: "groovy"
        value: |
          assert rows.size() == 2
          assert rows[0].ID == '1'
          assert rows[1].STATUS == 'in progress'
          assert rows[1] == [ORDERTYPE:'SampleOrder', STATUS:'in progress']
```

Spring XML

```
<sql datasource="myDataSource">
    <statement>select ID from CUSTOMERS where NAME='${customerName}'</statement>
    <statement>select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'</statement>

    <validate-script type="groovy">
        assert rows.size() == 2
        assert rows[0].ID == '1'
        assert rows[1].STATUS == 'in progress'
        assert rows[1] == [ORDERTYPE:'SampleOrder', STATUS:'in progress']
    </validate-script>
</sql>
```

As you can see Groovy provides fantastic access methods to the SQL result set. We can browse the

result set with named column values and check the size of the result set. We are also able to search for an entry, iterate over the result set and have other helpful operations. For a detailed description of the list and map handling in Groovy my advice for you is to have a look at the official Groovy documentation.



In general other script languages do also support this kind of list and map access. For now we just have implemented the Groovy script support, but the framework is ready to work with all other great script languages out there, too (e.g. Scala, Clojure, Fantom, etc.). So if you prefer to work with another language join and help us implement those features.

8.3.5. Java SQL result set validation

When plain Java assertions are preferred over a script, bind a `SqlResultSetValidator` to the query action. In contrast to the script validator this validator runs with or without a validation script — no `validateScript(…)` is required to trigger it:

Java

```
$(sql()
    .dataSource(myDataSource)
    .query()
    .statement("select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'")
    .validator((resultSet, context) -> {
        Assert.assertEquals(resultSet.size(), 2);
        Assert.assertEquals(resultSet.get(0).get("STATUS"), "in progress");
    })
);
```

The validator receives every result row in execution order as a `List<Map<String, Object>>`; an empty result arrives as an empty list. A `ValidationException` (or an `AssertionError` from your assertion library) fails the action. The Java validator composes with the other paths: when a script validation and control result set are configured as well, validators run in order — plain-Java validator, script validation, control result set — and the first failure aborts the rest. For script-based validation use `SqlResultSetScriptValidator` as before — that remains the hook scripts execute through.

Rows can also be asserted as typed POJOs instead of raw column maps. The `SqlPojoValidationCallback` maps each row to your class with case-insensitive, underscore-folded column-to-property matching, then checks every row until one satisfies the assertion:

Java

```
$(sql()
    .dataSource(myDataSource)
    .query()
    .statement("select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'")
    .validator(new SqlPojoValidationCallback<Order>(Order.class) {
        @Override
        protected void validate(Order order, TestContext context) {
            Assert.assertEquals(order.getStatus(), "in progress");
        }
    })
);
```

The POJO needs a no-argument constructor and JavaBean setters for the asserted properties. With several rows the assertion holds when ANY row satisfies it ("the table contains a row like this"); when none does, the failure lists the first few rows seen. Reject a row with a `ValidationException` or an `AssertionError` — any other exception propagates as a programming error. A row that cannot be mapped fails loudly, naming the column, the row index and the columns present. To take over multi-row semantics, override `validate(List<T>, TestContext)` instead — the empty list still reaches that override.



The builder has two `validator(…)` overloads (plain-Java and script). A literal `validator(null)` is ambiguous between them and needs a cast to the intended type.

8.3.6. Save result set values

Now the validation of database entries is a very powerful feature but sometimes we simply do not know the persisted content values. The test may want to read database entries into test variables without validation. Citrus is able to do that with the following <extract> expressions:

Java

```
$(sql()
    .dataSource(myDataSource)
    .query()
        .statement("select ID from CUSTOMERS where NAME='${customerName}'")
        .statement("select STATUS from ORDERS where ID='${orderId}'")
    .extract("ID", "customerId")
    .extract("STATUS", "orderStatus")
);
```

XML

```
<sql datasource="myDataSource">
    <statements>
        <statement>select ID from CUSTOMERS where NAME='${customerName}'</statement>
        <statement>select STATUS from ORDERS where ID='${orderId}'</statement>
    </statements>
    <extract  column="ID" variable="customerId"/>
    <extract  column="STATUS" variable="orderStatus"/>
</sql>
```

YAML

```
- sql:
  dataSource: "myDataSource"
  statements:
    - statement: "select ID from CUSTOMERS where NAME='${customerName}'"
    - statement: "select ORDERTYPE, STATUS from ORDERS where ID='${orderId}'"
  extract:
    - column: "ID"
      variable: "customerId"
    - column: "STATUS"
      variable: "orderStatus"
```

Spring XML

```
<sql datasource="testDataSource">
    <statement>select ID from CUSTOMERS where NAME='${customerName}'</statement>
    <statement>select STATUS from ORDERS where ID='${orderId}'</statement>

    <extract column="ID" variable="${customerId}"/>
    <extract column="STATUS" variable="${orderStatus}"/>
</sql>
```

We can save the database column values directly to test variables. Of course you can combine the value extraction with the normal column validation described earlier in this chapter. Please keep in mind that we can not use these operations on result sets with multiple rows. Citrus will always use the first row in a result set.

8.4. Sleep

This action shows how to make the test framework execution stop for a given amount of time. The attribute 'time' defines the amount of time to wait in seconds. As shown in the next example decimal values are supported too. When no waiting time is specified the default time of 50000 milliseconds applies.

Java

```
@CitrusTest
public void sleepTest() {
    $(sleep().seconds(3.5));

    $(sleep().milliseconds(500));

    $(sleep());
}
```

XML

```
<test name="SleepTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <sleep seconds="3.5"/>

        <sleep milliseconds="500"/>

        <sleep/>
    </actions>
</test>
```

YAML

```
name: SleepTest
actions:
  - sleep:
      seconds: "3.5"
  - sleep:
      milliseconds: "500"
  - sleep: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SleepTest">
        <actions>
            <sleep seconds="3.5"/>

            <sleep milliseconds="500"/>

            <sleep/>
        </actions>
    </testcase>
</spring:beans>
```

When should somebody use this action? To us this action was always very useful in case the test

needed to wait until an application had done some work. For example in some cases the application took some time to write some data into the database. We waited then a small amount of time in order to avoid unnecessary test failures, because the test framework simply validated the database too early. Or as another example the test may wait a given time until retry mechanisms are triggered in the tested application and then proceed with the test actions.

8.5. Delay

This action shows how to make the test framework execution stop for a given amount of time. The attribute 'time' defines the amount of time to wait in seconds. As shown in the next example decimal values are supported too. When no waiting time is specified the default time of 50000 milliseconds applies.

Java

```
@CitrusTest
public void delayTest() {
    $(delay().seconds(3.5));

    $(delay().milliseconds(500));

    $(delay());
}
```

XML

```
<test name="DelayTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <delay seconds="3.5"/>

        <delay milliseconds="500"/>

        <delay/>
    </actions>
</test>
```

YAML

```
name: DelayTest
actions:
  - delay:
      seconds: "3.5"
  - delay:
      milliseconds: "500"
  - delay: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="DelayTest">
        <actions>
            <delay seconds="3.5"/>

            <delay milliseconds="500"/>

            <delay/>
        </actions>
    </testcase>
</spring:beans>
```

When should somebody use this action? To us this action was always very useful in case the test needed to wait until an application had done some work. For example in some cases the application took some time to write some data into the database. We waited then a small amount of time in order to avoid unnecessary test failures, because the test framework simply validated the database too early. Or as another example the test may wait a given time until retry mechanisms are triggered in the tested application and then proceed with the test actions.

8.6. Java

The test framework is written in Java and runs inside a Java virtual machine. The functionality of calling other Java objects and methods in this same Java VM through Java Reflection is self-evident. With this action you can call any Java API available at runtime through the specified Java classpath.

The action syntax looks like follows:

Java

```
@CitrusTest
public void javaActionTest() {
    $(java("org.citrusframework.test.util.InvocationDummy")
        .constructorArgs("Test Invocation")
        .method("invoke")
        .methodArgs(new String[] { "1","2" })
    );

    $(java(new InvocationDummy("Test Invocation"))
        .method("invoke")
        .methodArgs(new Object[] { 4, "Test Invocation", true })
    );

    $(java(InvocationDummy.class)
        .method("main")
        .methodArgs(new String[] { "4", "Test Invocation", "true" })
    );
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JavaTest">
        <actions>
            <java class="org.citrusframework.test.util.InvocationDummy">
                <constructor>
                    <argument type="">Test Invocation</argument>
                </constructor>
                <method name="invoke">
                    <argument type="String[]">1,2</argument>
                </method>
            </java>

            <java class="org.citrusframework.test.util.InvocationDummy">
                <constructor>
                    <argument type="">Test Invocation</argument>
                </constructor>
                <method name="invoke">
                    <argument type="int">4</argument>
                    <argument type="String">Test Invocation</argument>
                    <argument type="boolean">true</argument>
                </method>
            </java>

            <java class="org.citrusframework.test.util.InvocationDummy">
                <method name="main">
                    <argument type="String[]">4,Test,true </argument>
                </method>
            </java>
        </actions>
    </testcase>
</spring:beans>
```

The Java class is specified by fully qualified class name. Constructor arguments are added using the <constructor> element with a list of <argument> child elements. The type of the argument is defined within the respective attribute "type". By default the type would be String.

The invoked method on the Java object is simply referenced by its name. Method arguments do not bring anything new after knowing the constructor argument definition, do they?.

Method arguments support data type conversion too, even string arrays (useful when calling CLIs). In the third action in the example code you can see that colon separated strings are automatically converted to string arrays.

Simple data types are defined by their name (int, boolean, float etc.). Be sure that the invoked method and class constructor fit your arguments and vice versa, otherwise you will cause errors at runtime.

Besides instantiating a fully new object instance for a class how about reusing a bean instance available in Spring bean container. Simply use the **ref** attribute and refer to an existing bean in Spring application context.

Java

```
InvocationDummy invocationDummy = new InvocationDummy("Test Invocation");
$(java(invocationDummy)
    .method("invoke")
    .methodArgs(new Object[] { 4, "Test Invocation", true })
);
```

Spring XML

```
<java ref="invocationDummy">
    <method name="invoke">
        <argument type="int">4</argument>
        <argument type="String">Test Invocation</argument>
        <argument type="boolean">true</argument>
    </method>
</java>

<bean id="invocationDummy" class="org.citrusframework.test.util.InvocationDummy"/>
```

The method is invoked on the Spring bean instance. This is very useful as you can inject other objects (e.g. via Autowiring) to the Spring bean instance before method invocation in test takes place. This enables you to execute any Java logic inside a test case.

8.7. Expect timeout

In some cases it might be necessary to validate that a message is **not** present on a destination. This means that this action expects a timeout when receiving a message from an endpoint destination. For instance the tester intends to ensure that no message is sent to a certain destination in a time period. In that case the timeout would not be a test aborting error but the expected behavior. And in contrast to the normal behavior when a message is received in the time period the test will fail with error.

In order to validate such a timeout situation the action <expectTimout> shall help. The usage is very simple as the following example shows:

Java

```
@Autowired
@Qualifier("myEndpoint")
private Endpoint myEndpoint;

@CitrusTest
public void expectTimeoutTest() {
    $(expectTimeout()
            .endpoint(myEndpoint)
            .timeout(500)
    );
}
```

XML

```
<test name="ExpectTimeoutTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <expect-timeout endpoint="myEndpoint" wait="500"/>
    </actions>
</test>
```

YAML

```
name: ExpectTimeoutTest
actions:
  - expect-timeout:
      endpoint: "myEndpoint"
      wait: "500"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ExpectTimeoutTest">
        <actions>
            <expect-timeout endpoint="myEndpoint" wait="500"/>
        </actions>
    </testcase>
</spring:beans>
```

The action offers two attributes:

endpoint	Reference to a message endpoint that will try to receive messages.
wait/timeout	Time period to wait for messages to arrive

Sometimes you may want to add some selector on the timeout receiving action. This way you can

very selective check on a message to not be present on a message destination. This is possible with defining a message selector on the test action as follows.

Java

```
@Autowired
@Qualifier("myEndpoint")
private Endpoint myEndpoint;

@CitrusTest
public void expectTimeoutTest() {
    $(expectTimeout()
            .endpoint(myEndpoint)
            .timeout(500)
            .selector("MessageId = '123456789'")
    );
}
```

XML

```
<test name="ExpectTimeoutTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <expect-timeout endpoint="myEndpoint" wait="500">
            <selector>MessageId='123456789'</selector>
        </expect-timeout>
    </actions>
</test>
```

YAML

```
name: ExpectTimeoutTest
actions:
  - expect-timeout:
      endpoint: "myEndpoint"
      wait: "500"
      selector:
        value: "MessageId='123456789'"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ExpectTimeoutTest">
        <actions>
            <expect-timeout endpoint="myEndpoint" wait="500">
                <select>MessageId='123456789'<select/>
            </expect-timeout>
        </actions>
    </testcase>
</spring:beans>
```

8.8. Echo

The <echo> action prints messages to the console/logger. This functionality is useful when debugging test runs. The property "message" defines the text that is printed. Tester might use it to print out debug messages and variables as shown in the next code example:

Java

```
@CitrusTest
public void echoTest() {
    variable("today", "citrus:currentDate()");

    $(echo().message("Hello Test Framework"));
    $(echo().message("Current date is: ${today}"));
}
```

XML

```
<test name="EchoTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="today" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <echo>
            <message>Hello Test Framework</message>
        </echo>

        <echo>
            <message>Current date is: ${today}</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: EchoTest
variables:
  - name: "today"
    value: "citrus:currentDate()"
actions:
  - echo:
      message: "Hello Test Framework"
  - echo:
      message: "Current date is: ${today}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="EchoTest">
        <variables>
            <variable name="today" value="citrus:currentDate()"/>
        </variables>
        <actions>
            <echo>
                <message>Hello Test Framework</message>
            </echo>

            <echo>
                <message>Current date is: ${today}</message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

Result on the console:

```
Hello Test Framework
Current time is: 05.08.2008
```

8.9. Print

The <print> action prints messages to the console/logger. This functionality is useful when debugging test runs. The property "message" defines the text that is printed. Tester might use it to print out debug messages and variables as shown the next code example:

Java

```
@CitrusTest
public void printTest() {
    variable("today", "citrus:currentDate()");

    $(print().message("Hello Test Framework"));
    $(print().message("Current date is: ${today}"));
}
```

XML

```
<test name="PrintTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="today" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <print>
            <message>Hello Test Framework</message>
        </print>

        <print>
            <message>Current date is: ${today}</message>
        </print>
    </actions>
</test>
```

YAML

```
name: PrintTest
variables:
  - name: "today"
    value: "citrus:currentDate()"
actions:
  - print:
      message: "Hello Test Framework"
  - print:
      message: "Current date is: ${today}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="PrintTest">
        <variables>
            <variable name="today" value="citrus:currentDate()"/>
        </variables>
        <actions>
            <print>
                <message>Hello Test Framework</message>
            </print>

            <print>
                <message>Current date is: ${today}</message>
            </print>
        </actions>
    </testcase>
</spring:beans>
```

Result on the console:

```
Hello Test Framework
Current time is: 05.08.2008
```

8.10. Stop time

Time measurement during a test can be very helpful. The `<trace-time>` action creates and monitors multiple timelines. The action offers the attribute *id* to identify a timeline. The tester can of course use more than one timeline with different ids simultaneously.

Read the next example, and you will understand the mix of different timelines:

Java

```
@CitrusTest
public void sleepTest() {
    $(stopTime());

    $(stopTime().id("time_line_id"));

    $(sleep().seconds(3.5));

    $(stopTime().id(" time_line_id "));

    $(sleep().milliseconds(5000));

    $(stopTime());

    $(stopTime().id(" time_line_id "));

}
```

XML

```
<test name="SleepTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <stop-time/>

        <stop-time id="time_line_id"/>

        <sleep seconds="3.5"/>

        <stop-time id=" time_line_id "/>

        <sleep milliseconds="5000"/>

        <stop-time/>

        <stop-time id=" time_line_id "/>
    </actions>
</test>
```

YAML

```
name: SleepTest
actions:
  - stop-time: {}

  - stop-time:
      id: "time_line_id"

  - sleep:
      seconds: "3.5"

  - stop-time:
      id: "time_line_id"

  - sleep:
      milliseconds: "5000"

  - stop-time: {}

  - stop-time:
      id: "time_line_id"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SleepTest">
        <actions>
            <trace-time/>

            <trace-time id="time_line_id"/>

            <sleep seconds="3.5"/>

            <trace-time id=" time_line_id "/>

            <sleep milliseconds="5000"/>

            <trace-time/>

            <trace-time id=" time_line_id "/>
        </actions>
    </testcase>
</spring:beans>
```

The test output looks like follows:

```
Starting TimeWatcher:
Starting TimeWatcher: time_line_id
TimeWatcher time_line_id after 3500 milliseconds
TimeWatcher after 8500 seconds
TimeWatcher time_line_id after 8500 milliseconds
```



Timeline ids should not exist as test variables before the action is called for the first time. This would break the timeline initialization.



In case no timeline id is specified the framework will measure the time for a default timeline. To print out the current elapsed time for a timeline you simply have to place the `<trace-time> action into the action chain again and again, using the respective timeline identifier. The elapsed time will be printed out to the console every time.

Each timeline is stored as test variable in the test case. By default you will have the following test variables set for each timeline:

CITRUS_TIMELINE	first timestamp of timeline
CITRUS_TIMELINE_VALUE	latest time measurement value (time passed since first timestamp in milliseconds)

According to your timeline id you will get different test variable names. Also you can customize the time value suffix (default: **_VALUE**):

Java

```
@CitrusTest
public void sleepTest() {
    $(stopTime().id("custom_watcher").suffix("_1st"));

    $(sleep());

    $(stopTime().id(" custom_watcher ").suffix("_2nd"));
}
```

XML

```
<test name="SleepTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <stop-time id="custom_watcher" suffix="_1st"/>

        <sleep/>

        <stop-time id="custom_watcher" suffix="_2nd"/>
    </actions>
</test>
```

YAML

```
name: SleepTest
actions:
  - stop-time:
      id: "custom_watcher"
      suffix: "_1st"

  - sleep: {}

  - stop-time:
      id: "custom_watcher"
      suffix: "_2nd"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SleepTest">
        <actions>
            <trace-time id="custom_watcher" suffix="_1st"/>

            <sleep/>

            <trace-time id="custom_watcher" suffix="_2nd"/>
        </actions>
    </testcase>
</spring:beans>
```

You will get following test variables set:

custom_watcher	first timestamp of timeline
custom_watcher_1st	time passed since start

custom_watcher_2nd time passed since start

Of course using the same suffix multiple times will overwrite the timestamps in test variables.

8.11. Create variables

As you know variables usually are defined at the beginning of the test case (test-variables). It might also be helpful to reset existing variables as well as to define new variables during the test. The action `<create-variables>` is able to declare new variables or overwrite existing ones.

Java

```
@CitrusTest
public void createVariableTest() {
    variable("myVariable", "12345");
    variable("id", "54321");

    $(echo()
        .message("Current variable value: ${myVariable}"));

    $(createVariable("myVariable", "${id}"));
    $(createVariable("newVariable", "this is a test"));

    $(echo()
        .message("Current variable value: ${myVariable}"));

    $(echo()
        .message("New variable 'newVariable' has the value: ${newVariable}"));
}
```

XML

```
<test name="CreateVariablesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="myVariable" value="12345"/>
        <variable name="id" value="54321"/>
    </variables>
    <actions>
        <echo>
            <message>Current variable value: ${myVariable}</message>
        </echo>

        <create-variables>
            <variable name="myVariable" value="${id}"/>
            <variable name="newVariable" value="'this is a test'"/>
        </create-variables>

        <echo>
            <message>Current variable value: ${myVariable} </message>
        </echo>

        <echo>
            <message>New variable 'newVariable' has the value:
${newVariable}</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: "CreateVariablesTest"
variables:
  - name: "myVariable"
    value: "12345"
  - name: "id"
    value: "54321"
actions:
  - echo:
      message: "Current variable value: ${myVariable}"
  - create-variable:
      variables:
        - name: "myVariable"
          value: "${id}"
        - name: "newVariable"
          value: "this is a test"
  - echo:
      message: "Current variable value: ${myVariable}"
  - echo:
      message: "New variable 'newVariable' has the value: ${newVariable}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="createVariablesTest">
        <variables>
            <variable name="myVariable" value="12345"/>
            <variable name="id" value="54321"/>
        </variables>
        <actions>
            <echo>
                <message>Current variable value: ${myVariable}</message>
            </echo>

            <create-variables>
                <variable name="myVariable" value="${id}"/>
                <variable name="newVariable" value="'this is a test'"/>
            </create-variables>

            <echo>
                <message>Current variable value: ${myVariable} </message>
            </echo>

            <echo>
                <message>
                  New variable 'newVariable' has the value: ${newVariable}
                </message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```



Please note the difference between the **variable()** method and the **createVariable()** method. The first initializes the test case with the test variables. So all variables defined with this method are valid from the very beginning of the test. In contrary to that the **createVariable()** is executed within the test action chain. The newly created variables are then valid for the rest of the test. Trailing actions can reference the variables as usual with the variable expression.

8.12. Create endpoints

Endpoints represent the connectivity to other services in Citrus. An endpoint may create producers and consumers to send and receive messages as part of the test.

Users are able to declare endpoints in the form of a project wide configuration in Citrus. Also, users are able to use dynamic endpoint definitions with a `send` or `receive` action where the endpoint is defined as a URI.

Java

```
@CitrusTest
public void useDynamicEndpoints() {
    $(send()
        .endpoint("kafka:bookings?server=${kafka.bootstrapServer}")
        .message()
        // ...
    );

    $(receive()
        .endpoint("kafka:bookings?server=${kafka.bootstrapServer}")
        .message()
        // ...
    );
}
```

XML

```
<test name="UseDynamicEndpoints"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="kafka:bookings?server=${kafka.bootstrapServer}">
            <message>
              <!--
              ...
              -->
            </message>
        </send>
        <receive endpoint="kafka:bookings?server=${kafka.bootstrapServer}">
            <message>
              <!--
              ...
              -->
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: "UseDynamicEndpoints"
actions:
  - send:
      endpoint: "kafka:bookings?server=${kafka.bootstrapServer}"
      message:
        # ...
  - receive:
      endpoint: "kafka:bookings?server=${kafka.bootstrapServer}"
      message:
        # ...
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="useDynamicEndpoints">
        <actions>
            <send endpoint="kafka:bookings?server=${kafka.bootstrapServer}">
                <message>
                  <!--
                  ...
                  -->
                </message>
            </send>
            <receive endpoint="kafka:bookings?server=${kafka.bootstrapServer}">
                <message>
                  <!--
                  ...
                  -->
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The dynamic endpoint receives its properties in the form of endpoint URI parameters: `kafka:bookings?server=${kafka.bootstrapServer}`

The test needs to repeat the same configuration endpoint URI several times with multiple send/receive test actions.

This can be improved by using the `createEndpoint()` test action that declares the endpoint once with a given name. Following test actions are able to reference the endpoint as usual by its give name.

Java

```
@CitrusTest
public void useDynamicEndpoints() {
    $(createEndpoint()
        .endpointName("bookings")
        .uri("kafka:bookings?server=${kafka.bootstrapServer}")
    );

    $(send()
        .endpoint("bookings")
        .message()
        // ...
    );

    $(receive()
        .endpoint("bookings")
        .message()
        // ...
    );
}
```

XML

```
<test name="UseDynamicEndpoints"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <create-endpoint name="bookings"
                         uri="kafka:bookings?server=${kafka.bootstrapServer}"/>

        <send endpoint="bookings">
            <message>
              <!--
              ...
              -->
            </message>
        </send>
        <receive endpoint="bookings">
            <message>
              <!--
              ...
              -->
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: "UseDynamicEndpoints"
actions:
  - createEndpoint:
      name: bookings
      uri: "kafka:bookings?server=${kafka.bootstrapServer}"
  - send:
      endpoint: "bookings"
      message:
        # ...
  - receive:
      endpoint: "bookings"
      message:
        # ...
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

As an alternative syntax aou may also declare the properties with key-value pairs in the `createEndpoint()` action:

Java

```
@CitrusTest
public void useDynamicEndpoints() {
    $(createEndpoint()
        .endpointName("bookings")
        .type("kafka")
        .property("topic", "bookings")
        .property("server", "${kafka.bootstrapServer}")
    );
}
```

XML

```
<test name="UseDynamicEndpoints"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <create-endpoint name="bookings" type="kafka">
          <properties>
            <property name="topic" value="bookings"/>
            <property name="server" value="${kafka.bootstrapServer}"/>
          </properties>
        </create-endpoint>
    </actions>
</test>
```

YAML

```
name: "UseDynamicEndpoints"
actions:
  - createEndpoint:
      name: bookings
      type: "kafka"
      properties:
        topic: bookings
        server: "${kafka.bootstrapServer}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

8.13. Trace variables

You already know the <echo> action that prints messages to the console or logger. The <trace-variables> action is specially designed to trace all currently valid test variables to the console. This was mainly used by us for debug reasons. The usage is quite simple:

Java

```
@CitrusTest
public void traceVariablesTest() {
    variable("myVariable", "12345");
    variable("nextVariable", "54321");

    $(traceVariables("myVariable", "nextVariable"));
    $(traceVariables());
}
```

XML

```
<test name="TraceVariablesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="myVariable" value="12345"/>
        <variable name="nextVariable" value="54321"/>
    </variables>
    <actions>
        <trace>
            <variable name="myVariable"/>
            <variable name="nextVariable"/>
        </trace>

        <trace-variables/>
    </actions>
</test>
```

YAML

```
name: TraceVariablesTest
variables:
  - name: "myVariable"
    value: "12345"
  - name: "nextVariable"
    value: "54321"
actions:
  - trace:
      variables:
          - name: "myVariable"
          - name: "nextVariable"
  - trace: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="TraceVariablesTest">
        <variables>
            <variable name="myVariable" value="12345"/>
            <variable name="nextVariable" value="54321"/>
        </variables>
        <actions>
            <trace-variables>
                <variable name="myVariable"/>
                <variable name="nextVariable"/>
            </trace-variables>

            <trace-variables/>
        </actions>
    </testcase>
</spring:beans>
```

Simply add the <trace-variables> action to your action chain and all variables will be printed out to the console. You are able to define a special set of variables by using the <variable> child elements. See the output that was generated by the test example above:

```
Current value of variable myVariable = 12345
Current value of variable nextVariable = 54321
```

8.14. Transform

The `<***transform***>` action transforms XML fragments with XSLT in order to construct various XML representations. The transformation result is stored into a test variable for further usage. The property **xml-data** defines the XML source, that is going to be transformed, while **xslt-data** defines the XSLT transformation rules. The attribute **variable** specifies the target test variable which receives the transformation result. The tester might use the action to transform XML messages as shown in the next code example:

Java

```
@CitrusTest
public void transformTest() {
    $(transform()
        .variable("result")
        .source("""
            <TestRequest>
                <Message>Hello World!</Message>
            </TestRequest>
        """)
        .xslt("""
            <xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
              <xsl:template match="/">
                <html>
                    <body>
                        <h2>Test Request</h2>
                        <p>Message: <xsl:value-of select="TestRequest/Message"/></p>
                    </body>
                </html>
              </xsl:template>
            </xsl:stylesheet>
        """)
    );

    $(echo().message("${result}"));
}
```

XML

```
<test name="TransformTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <transform variable="result">
            <source>
              <![CDATA[
                  <TestRequest>
                      <Message>Hello World!</Message>
                  </TestRequest>
              ]]>
            </source>
            <xslt>
              <![CDATA[
                  <xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
                      <xsl:template match="/">
                          <html>
                              <body>
                                  <h2>Test Request</h2>
                                  <p>Message: <xsl:value-of
select="TestRequest/Message"/></p>
                              </body>
                          </html>
                      </xsl:template>
                  </xsl:stylesheet>
              ]]>
            </xslt>
        </transform>
        <echo>
          <message>${result}</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: TransformTest
actions:
  - transform:
      variable: result
      source:
        value: |
          <TestRequest>
              <Message>Hello World!</Message>
          </TestRequest>
      xslt:
        value: |
          <xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
              <xsl:template match="/">
              <html>
                  <body>
                      <h2>Test Request</h2>
                      <p>Message: <xsl:value-of select="TestRequest/Message"/></p>
                  </body>
              </html>
              </xsl:template>
          </xsl:stylesheet>
  - echo:
      message: '${result}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="TransformTest">
        <actions>
            <transform variable="result">
                <xml-data>
                  <![CDATA[
                      <TestRequest>
                          <Message>Hello World!</Message>
                      </TestRequest>
                  ]]>
                </xml-data>
                <xslt-data>
                    <![CDATA[
                    <xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
                        <xsl:template match="/">
                          <html>
                              <body>
                                  <h2>Test Request</h2>
                                  <p>Message: <xsl:value-of
select="TestRequest/Message"/></p>
                              </body>
                          </html>
                        </xsl:template>
                    </xsl:stylesheet>
                    ]]>
                </xslt-data>
            </transform>
            <echo>
              <message>${result}</message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

The transformation above results to:

```
  <html>
      <body>
          <h2>Test Request</h2>
          <p>Message: Hello World!</p>
      </body>
  </html>
```

In the example we used CDATA sections to define the transformation source as well as the XSL transformation rules. As usual you can also use external file resources here. The transform action

with external file resources looks like follows:

Java

```
@CitrusTest
public void transformTest() {
    $(transform()
        .variable("result")
        .source(new ClassPathResource("org/citrusframework/actions/transform-
source.xml"))
        .xslt(new ClassPathResource("org/citrusframework/actions/transform.xslt"))
    );

    $(echo().message("${result}"));
}
```

XML

```
<test name="TransformTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <transform variable="result">
            <source file="classpath:transform-source.xml"/>
            <xslt file="classpath:transform.xslt"/>
        </transform>
        <echo>
          <message>${result}</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: TransformTest
actions:
  - transform:
      variable: result
      source:
        file: "classpath:transform-source.xml"
      xslt:
        file: "classpath:transform.xslt"
  - echo:
      message: '${result}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="TransformTest">
        <actions>
            <transform variable="result">
                <xml-resource file="classpath:transform-source.xml"/>
                <xslt-resource file="classpath:transform.xslt"/>
            </transform>
            <echo>
              <message>${result}</message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

Defining multi-line Strings with nested quotes is no fun in Java. So you may want to use external file resources for your scripts as shown in the second part of the example. In fact, you could also use script languages like Groovy or Scala that have much better support for multi-line Strings.

8.15. Groovy script execution

Groovy is an agile dynamic language for the Java Platform. Groovy ships with a lot of very powerful features and fits perfectly with Java as it is based on Java and runs inside the JVM.



The Groovy test action support lives in a separate module. You need to add the module to your project to use the functionality.

citrus-groovy dependency module

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-groovy</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The Citrus Groovy support might be the entrance for you to write customized test actions. You can easily execute Groovy code inside a test case, just like a normal test action. The whole test context with all variables is available to the Groovy action. This means someone can change variable values or create new variables very easily.

Let's have a look at some examples in order to understand the possible Groovy code interactions in Citrus:

Java

```
@CitrusTest
public void GroovyTest() {
    $(groovy()
        .script("println 'Hello Citrus'")
    );

    $(groovy()
        .script("println 'The variable is: ${time}'")
    );

    $(groovy()
        .scriptResourcePath("classpath:org/citrusframework/script/example.groovy")
    );
}
```

XML

```
<test name="GroovyTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="time" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <groovy>
          <script>
            println 'Hello Citrus'
          </script>
        </groovy>
        <groovy>
          <script>
            println 'The variable is: ${time}'
          </script>
        </groovy>
        <groovy>
          <script file="classpath:org/citrusframework/script/example.groovy"/>
        </groovy>
    </actions>
</test>
```

YAML

```
name: GroovyTest
variables:
  - name: "time"
    value: "citrus:currentDate()"
actions:
  - groovy:
      script: "println 'Hello Citrus'"
  - groovy:
      script: |
        println 'The variable is: ${time}'
  - groovy:
      script:
        file: classpath:org/citrusframework/script/example.groovy
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="GroovyTest">
      <variables>
        <variable name="time" value="citrus:currentDate()"/>
      </variables>
      <actions>
        <groovy>
            println 'Hello Citrus'
        </groovy>
        <groovy>
            println 'The variable is: ${time}'
        </groovy>
        <groovy resource="classpath:org/citrusframework/script/example.groovy"/>
      </actions>
    </testcase>
</spring:beans>
```

As you can see it is possible to write Groovy code directly into the test case. Citrus will interpret and execute the Groovy code at runtime. As usual nested variable expressions are replaced with respective values. In general this is done in advance before the Groovy code is interpreted. For more complex Groovy code sections which grow in lines of code you can also reference external file resources.

After this basic Groovy code usage inside a test case we might be interested accessing the whole TestContext. The TestContext Java object holds all test variables and function definitions for the test case and can be referenced in Groovy code via simple naming convention. Just access the object reference 'context' and you are able to manipulate the TestContext (e.g. setting a new variable which is directly ready for use in following test actions).

Java

```
@CitrusTest
public void GroovyTest() {
    $(groovy()
        .script("""
            context.setVariable("greetingText","Hello Citrus")
            println context.getVariable("greetingText")
        """)
    );

    $(echo()
        .message("New variable: ${greetingText}")
    );
}
```

XML

```
<test name="GroovyTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <groovy>
            context.setVariable("greetingText","Hello Citrus")
            println context.getVariable("greetingText")
        </groovy>
        <echo>
            <message>New variable: ${greetingText}</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: GroovyTest
actions:
  - groovy:
      script: |
        context.setVariable("greetingText","Hello Citrus")
        println context.getVariable("greetingText")
  - echo:
      message: "New variable: ${greetingText}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="groovyTest">
      <actions>
        <groovy>
          context.setVariable("greetingText","Hello Citrus")
          println context.getVariable("greetingText")
        </groovy>

        <echo>
          <message>New variable: ${greetingText}</message>
        </echo>
      </actions>
    </testcase>
</spring:beans>
```



The implicit TestContext access that was shown in the previous sample works with a default Groovy script template provided by Citrus. The Groovy code you write in the test case is automatically surrounded with a Groovy script which takes care of handling the TestContext. The default template looks like follows:

Groovy script template

```
import org.citrusframework.*
import org.citrusframework.variable.*
import org.citrusframework.context.TestContext
import org.citrusframework.groovy.actions.GroovyAction.ScriptExecutor

public class GScript implements ScriptExecutor {
    public void execute(TestContext context) {
        @SCRIPTBODY@
    }
}
```

Your code is placed in substitution to the ***@SCRIPTBODY@*** placeholder. Now you might understand how Citrus handles the context automatically. You can also write your own script templates making more advanced usage of other Java APIs and Groovy code. Just add a script template path to the test action like this:

Java

```
$(groovy()
    .template("classpath:my-custom-template.groovy")
    .script("...")
);
```

XML

```
<groovy script-template="classpath:my-custom-template.groovy">
    <!-- ... -->
</groovy>
```

YAML

```
- groovy:
  script-template: "classpath:my-custom-template.groovy"
  script: "..."
```

Spring XML

```
<groovy script-template="classpath:my-custom-template.groovy">
  <!-- ... -->
</groovy>
```

On the other hand you can disable the automatic script template wrapping in your action at all:

Java

```
$(groovy()
    .useScriptTemplate(false)
    .script("println 'Just use some Groovy code'")
);
```

XML

```
<groovy use-script-template="false">
    println 'Just use some Groovy code'
</groovy>
```

YAML

```
- groovy:
  use-script-template: false
  script: "println 'Just use some Groovy code'"
```

Spring XML

```
<groovy use-script-template="false">
  println 'Just use some Groovy code'
</groovy>
```

The next example deals with advanced Groovy code and writing whole classes. We write a new Groovy class which implements the ScriptExecutor interface offered by Citrus. This interface

defines a special execute method and provides access to the whole TestContext for advanced test variables access.

Java

```
@CitrusTest
public void GroovyTest() {
    variable("time", "citrus:currentDate()");

    $(groovy()
        .script("""
            import org.citrusframework.*
            import org.citrusframework.variable.*
            import org.citrusframework.context.TestContext
            import org.citrusframework.groovy.actions.GroovyAction.ScriptExecutor

            public class GScript implements ScriptExecutor {
                public void execute(TestContext context) {
                    println context.getVariable("time")
                }
            }
        """)
    );
}
```

XML

```
<test name="GroovyTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="time" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <groovy>
        <![CDATA[
            import org.citrusframework.*
            import org.citrusframework.variable.*
            import org.citrusframework.context.TestContext
            import org.citrusframework.groovy.actions.GroovyAction.ScriptExecutor

            public class GScript implements ScriptExecutor {
                public void execute(TestContext context) {
                    println context.getVariable("time")
                }
            }
        ]]>
        </groovy>
    </actions>
</test>
```

YAML

```
name: GroovyTest
variables:
  - name: time
    value: citrus:currentDate()
actions:
  - groovy:
      script: |
        import org.citrusframework.*
        import org.citrusframework.variable.*
        import org.citrusframework.context.TestContext
        import org.citrusframework.groovy.actions.GroovyAction.ScriptExecutor

        public class GScript implements ScriptExecutor {
            public void execute(TestContext context) {
                println context.getVariable("time")
            }
        }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="groovyTest">
      <variables>
        <variable name="time" value="citrus:currentDate()"/>
      </variables>
      <actions>
        <groovy>
          <![CDATA[
            import org.citrusframework.*
            import org.citrusframework.variable.*
            import org.citrusframework.context.TestContext
            import org.citrusframework.groovy.actions.GroovyAction.ScriptExecutor

            public class GScript implements ScriptExecutor {
                public void execute(TestContext context) {
                    println context.getVariable("time")
                }
            }
          ]]>
        </groovy>
      </actions>
    </testcase>
</spring:beans>
```

Implementing the ScriptExecutor interface in a custom Groovy class is applicable for very special test context manipulations as you are able to import and use other Java API classes in this code.

8.16. Failing the test

The fail action will generate an exception in order to terminate the test case with error. The test case will therefore not be successful in the reports.

The user can specify a custom error message for the exception in order to describe the error cause. Here is a very simple example to clarify the syntax:

Java

```
@CitrusTest
public void failTest() {
    $(fail().message("Test will fail with custom message"));
}
```

XML

```
<test name="FailTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <fail message="Test will fail with custom message"/>
    </actions>
</test>
```

YAML

```
name: FailTest
actions:
  - fail:
      message: "Test will fail with custom message"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FailTest">
        <actions>
            <fail message="Test will fail with custom message"/>
        </actions>
    </testcase>
</spring:beans>
```

Test results:

```
Execution of test: failTest failed! Nested exception is:
org.citrusframework.exceptions.CitrusRuntimeException:
Test will fail with custom message

[...]

CITRUS TEST RESULTS

failTest          : failed - Exception is: Test will fail with custom message

Found 1 test cases to execute
Skipped 0 test cases (0.0%)
Executed 1 test cases, containing 3 actions
Tests failed:        1 (100.0%)
Tests successfully:  0 (0.0%)
```

While using the Java DSL you can also raise arbitrary Java exceptions and let the test fail.

Java

```
@CitrusTest
public void throwExceptionSample() {
    // some test actions

    throw new ValidationException("This test should fail now");
}
```

The validation exception above will cause the test to fail as expected. However you may consider using the fail action to raise errors because this ensures to properly report the error in the Citrus test reports.

8.17. Input

During the test case execution it is possible to read some user input from the command line. The test execution will stop and wait for keyboard inputs over the standard input stream. The user has to type the input and end it with the return key.

The user input is stored to the respective variable value.

Java

```
@CitrusTest
public void inputActionTest() {
    variable("userinput", "");
    variable("userinput1", "");
    variable("userinput2", "y");
    variable("userinput3", "yes");
    variable("userinput4", "");

    $(input());

    $(echo().message("user input was: ${userinput}"));

    $(input().message("Now press enter:").result("userinput1"));

    $(echo().message("user input was: ${userinput1}"));

    $(input().message("Do you want to continue?").answers("y",
"n").result("userinput2"));

    $(echo().message("user input was: ${userinput2}"));

    $(input().message("Do you want to continue?").answers("yes",
"no").result("userinput3"));

    $(echo().message("user input was: ${userinput3}"));

    $(input().result("userinput4"));

    $(echo().message("user input was: ${userinput4}"));
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="InputTest">
        <variables>
            <variable name="userinput" value=""></variable>
            <variable name="userinput1" value=""></variable>
            <variable name="userinput2" value="y"></variable>
            <variable name="userinput3" value="yes"></variable>
            <variable name="userinput4" value=""></variable>
        </variables>
        <actions>
            <input/>
            <echo><message>user input was: ${userinput}</message></echo>

            <input message="Now press enter:" variable="userinput1"/>
            <echo><message>user input was: ${userinput1}</message></echo>

            <input message="Do you want to continue?"
                      valid-answers="y/n" variable="userinput2"/>
            <echo><message>user input was: ${userinput2}</message></echo>

            <input message="Do you want to continue?"
                      valid-answers="yes/no" variable="userinput3"/>
            <echo><message>user input was: ${userinput3}</message></echo>

            <input variable="userinput4"/>
            <echo><message>user input was: ${userinput4}</message></echo>
        </actions>
    </testcase>
</spring:beans>
```

As you can see the input action is customizable with a prompt message that is displayed to the user and some valid answer possibilities. The user input is stored to a test variable for further use in the test case. In detail the input action offers following attributes:

message	message displayed to the user
valid-answers	possible valid answers separated with '/' character
variable	result variable name holding the user input (default = ${userinput})

When the user input is restricted to a set of valid answers the input validation of course can fail due to mismatch. This is the case when the user provides some input not matching the valid answers given. In this case the user is again asked to provide valid input. The test action will continue to ask for valid input until a valid answer is given.



User inputs may not fit to automatic testing in terms of continuous integration testing where no user is present to type in the correct answer over the keyboard. In this case you can always skip the user input in advance by specifying a variable that matches the user input variable name. As the user input variable is then already present the user input is missed out and the test proceeds automatically.

8.18. Load Properties

You are able to load properties from external property files and store them as test variables. The action will require a file resource either from class path or file system in order to read the property values.

Let us look at an example to get an idea about this action:

Content of load.properties

```
username=Mickey Mouse
greeting.text=Hello Test Framework
```

Java

```
@CitrusTest
public void loadPropertiesTest() {
    $(load().filePath("file:tests/resources/load.properties"));
}
```

XML

```
<test name="LoadPropertiesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <load>
            <properties file="file:tests/resources/load.properties"/>
        </load>
    </actions>
</test>
```

YAML

```
name: LoadPropertiesTest
actions:
  - load:
      properties:
        file: "file:tests/resources/load.properties"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="LoadPropertiesTest">
        <actions>
            <load>
                <properties file="file:tests/resources/load.properties"/>
            </load>
        </actions>
    </testcase>
</spring:beans>
```

Output

```
Current value of variable username = Mickey Mouse
Current value of variable greeting.text = Hello Test Framework
```

The action will load all available properties in the file load.properties and store them to the test case as local variables.



Please be aware of the fact that existing variables are overwritten!

8.19. Purging JMS destinations

Purging JMS destinations during the test run is quite essential. Different test cases can influence each other when sending messages to the same JMS destinations. A test case should only receive those messages that actually belong to it.

Therefore, it is a good idea to purge all JMS queue destinations between the test cases. Obsolete messages that are stuck in a JMS queue for some reason are then removed so that the following test case is not offended.

Java

```
@Autowired
@Qualifier("connectionFactory")
private ConnectionFactory connectionFactory;

@CitrusTest
public void purgeJmsQueuesTest() {
    $(purgeQueues()
        .connectionFactory(connectionFactory)
        .queue("JMS.Queue.1")
        .queue("JMS.Queue.2")
        .queue("JMS.Queue.3")
    );
}
```

XML

```
<test name="PurgeJmsQueuesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-jms-queues connection-factory="connectionFactory">
            <queue name="JMS.Queue.1"/>
            <queue name="JMS.Queue.2"/>
            <queue name="JMS.Queue.3"/>
        </purge-jms-queues>
    </actions>
</test>
```

YAML

```
name: PurgeJmsQueuesTest
actions:
  - purge-queues:
      connection-factory: "connectionFactory"
      queues:
        - "JMS.Queue.1"
        - "JMS.Queue.2"
        - "JMS.Queue.3"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="purgeJmsQueuesTest">
      <actions>
          <jms:purge-jms-queues connection-factory="connectionFactory">
              <jms:queue name="JMS.Queue.1"/>
              <jms:queue name="JMS.Queue.2"/>
              <jms:queue name="JMS.Queue.3"/>
          </jms:purge-jms-queues>
      </actions>
    </testcase>
</spring:beans>
```



Citrus provides special support for JMS related features when using the Spring XML bean integration. You have to activate those JMS features in our test case by adding a special "jms" namespace and schema definition location to the test case XML.

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
        xmlns:spring="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:jms="http://www.citrusframework.org/schema/jms/testcase"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/testcase
            http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
            http://www.citrusframework.org/schema/jms/testcase
            http://www.citrusframework.org/schema/jms/testcase/citrus-jms-
testcase.xsd">

    [...]

</beans>
```

Notice that we have referenced the **jms** namespace when using the **purge-jms-queues** test action.

Purging the JMS queues in every test case is quite exhausting because every test case needs to define a purging action at the very beginning of the test. Fortunately the test suite definition offers tasks to run before, between and after the test cases which should ease up these tasks a lot. The test suite offers a very simple way to purge the destinations between the tests. See testsuite-before-test for more information about this.

As you can see in the next example it is quite easy to specify a group of destinations in the Spring configuration that get purged before a test is executed.

Spring Bean

```
@Bean
public SequenceBeforeTest beforeTest() {
    return SequenceBeforeTest.Builder.beforeTest()
            .actions(
                purgeQueues()
                    .queue("fooChannel")
                    .queue("barChannel")
            );
}
```

Spring XML

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:citrus="http://www.citrusframework.org/schema/config">
    <citrus:before-test id="purgeBeforeTest">
        <citrus:actions>
            <jms:purge-jms-queues>
                <jms:queue name="fooChannel"/>
                <jms:queue name="barChannel"/>
            </jms:purge-jms-queues>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```



Please keep in mind that the JMS related configuration components in Citrus belong to a separate XML namespace **jms:** . We have to add this namespace declaration to each test case XML and Spring bean XML configuration file as described at the very beginning of this section.

The syntax for purging the destinations is the same as we used it inside the test case. So now we are able to purge JMS destinations with given destination names. But sometimes we do not want to rely on queue or topic names as we retrieve destinations over JNDI for instance. We can deal with destinations coming from JNDI lookup like follows:

Spring XML

```
<jee:jndi-lookup id="jmsQueueHelloRequestIn" jndi-name="jms/jmsQueueHelloRequestIn"/>
<jee:jndi-lookup id="jmsQueueHelloResponseOut" jndi-
name="jms/jmsQueueHelloResponseOut"/>

<citrus:before-test id="purgeBeforeTest">
    <citrus:actions>
        <jms:purge-jms-queues>
            <jms:queue ref="jmsQueueHelloRequestIn"/>
            <jms:queue ref="jmsQueueHelloResponseOut"/>
        </jms:purge-jms-queues>
    </citrus:actions>
</citrus:before-test>
```

We just use the attribute **'ref'** instead of **'name'** and Citrus is looking for a bean reference for that identifier that resolves to a JMS destination. You can use the JNDI bean references inside a test case, too.

Java

```
@Autowired
@Qualifier("jmsQueue1")
private Queue jmsQueue1;

@Autowired
@Qualifier("jmsQueue2")
private Queue jmsQueue2;

@CitrusTest
public void purgeJmsQueuesTest() {
    $(purgeQueues()
        .queue(jmsQueue1)
        .queue(jmsQueue1)
    );
}
```

XML

```
<test name="PurgeJmsQueuesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-jms-queues>
            <queue ref="jmsQueue1"/>
            <queue ref="jmsQueue2"/>
        </purge-jms-queues>
    </actions>
</test>
```

YAML

```
name: PurgeJmsQueuesTest
actions:
  - purge-queues:
      queue: jmsQueue1
  - purge-queues:
      queue: jmsQueue2
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="purgeJmsQueuesTest">
      <actions>
          <jms:purge-jms-queues>
              <jms:queue ref="jmsQueue1"/>
              <jms:queue ref="jmsQueue2"/>
          </jms:purge-jms-queues>
      </actions>
    </testcase>
</spring:beans>
```

Of course you can use queue object references also in Java DSL test cases. Here we easily can use Spring's dependency injection with autowiring to get the object references from the IoC container.



You can mix queue name and queue object references as you like within one single purge queue test action.

8.20. Purging Spring Message Channels

The Spring Integration project defines message channels as a central messaging destination. These are in-memory message queues holding messages for test cases. Messages that are queued on a channel may become obsolete during a test run, especially when test cases fail and stop in their message consumption. Purging these message channel destinations is essential in these scenarios in order to avoid bad influence on upcoming test cases. Each test case should only receive those messages that actually refer to the test model. Therefore, it is a good idea to purge all message channel destinations between the test cases. Obsolete messages that get stuck in a message channel destination for some reason are then removed so that upcoming test case are not broken.

Following action definition purges all messages from a list of message channels:

Java

```
@Autowired
@Qualifier("someChannelName")
private MessageChannel someChannel;

@Autowired
@Qualifier("anotherChannelName")
private MessageChannel anotherChannel;

@CitrusTest
public void purgeChannelTest() {
    $(purgeChannels()
        .channelResolver(channelResolver)
        .channelNames("someChannelName", "anotherChannelName")
    );

    $(purgeChannels()
        .channels(someChannel, anotherChannel)
    );
}
```

XML

```
<test name="PurgeChannelTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-channels channel-resolver="channelResolver">
            <channel name="someChannelName"/>
            <channel name="anotherChannelName"/>
        </purge-channels>

        <purge-channels>
            <channel ref="someChannel"/>
            <channel ref="anotherChannel"/>
        </purge-channels>
    </actions>
</test>
```

YAML

```
name: PurgeChannelTest
actions:
  - purge-channels:
      channel-resolver: "channelResolver"
      channels:
        - "someChannelName"
        - "anotherChannelName"
  - purge-channels:
      channel: someChannel
  - purge-channels:
      channel: anotherChannel
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="PurgeChannelTest">
      <actions>
          <purge-channel>
              <channel name="someChannelName"/>
              <channel name="anotherChannelName"/>
          </purge-channel>

          <purge-channel>
              <channel ref="someChannel"/>
              <channel ref="anotherChannel"/>
          </purge-channel>
      </actions>
    </testcase>
</spring:beans>
```

As you can see the test action supports channel names as well as channel references to Spring bean instances. When using channel references you refer to the bean id or name in your application context.

The channel resolver reference is optional and points to a channel resolver that is used to resolve names to actual channel instances. By default, Citrus will automatically use a Spring application context channel resolver, so you just have to use the respective Spring bean names that are configured in the Spring application context. However, setting a custom channel resolver may be adequate for you in some special cases.

Message selectors enable you to selectively remove messages from the destination. All messages that pass the message selection logic get deleted the other messages will remain unchanged inside the channel destination. The message selector is a Spring bean that implements a special message selector interface. A possible implementation could be a selector deleting all messages that are older than five seconds:

Message selector implementation

```
import org.springframework.messaging.Message;
import org.springframework.integration.core.MessageSelector;

public class TimeBasedMessageSelector implements MessageSelector {

    public boolean accept(Message<?> message) {
        if (System.currentTimeMillis() - message.getHeaders().getTimestamp() > 5000) {
            return false;
        } else {
            return true;
        }
    }
}
```



The message selector returns **false** for those messages that should be deleted from the channel!

You simply define the message selector as a new Spring bean in the Citrus application context and reference it in your test action property.

Spring Bean

```
@Bean
public MessageSelector specialMessageSelector() {
    return new TimeBasedMessageSelector();
}
```

Spring XML

```
<bean id="specialMessageSelector"
    class="org.citrusframework.special.TimeBasedMessageSelector"/>
```

Now let us have a look at how you reference the selector in your test case:

Java

```
@CitrusTest
public void purgeChannelTest() {
    $(purgeChannels()
        .selector(specialMessageSelector)
        .channels(someChannel, anotherChannel)
    );
}
```

XML

```
<test name="PurgeChannelTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-channels message-selector="specialMessageSelector">
            <channel ref="someChannel"/>
            <channel ref="anotherChannel"/>
        </purge-channels>
    </actions>
</test>
```

YAML

```
name: PurgeChannelTest
actions:
  - purge-channels:
      message-selector: "specialMessageSelector"
      channels:
        - "someChannel"
        - "anotherChannel"
```

Spring XML

```
<purge-channel message-selector="specialMessageSelector">
  <channel ref="someChannel"/>
  <channel red="anotherChannel"/>
</purge-channel>
```

In the examples above we use a message selector implementation that gets injected via Spring IoC container.

Purging channels in each test case every time is quite exhausting because every test case needs to define a purging action at the very beginning of the test. A more straight forward approach would be to introduce some purging action which is automatically executed before each test. Fortunately the Citrus test suite offers a very simple way to do this. It is described in testsuite-before-test.

When using the special action sequence before test cases we are able to purge channel destinations every time a test case executes. See the upcoming example to find out how the action is defined in the Spring configuration application context.

Spring Bean

```
@Bean
public SequenceBeforeTest beforeTest() {
    return SequenceBeforeTest.Builder.beforeTest()
            .actions(
                purgeChannels()
                    .channel("fooChannel")
                    .channel("barChannel")
            );
}
```

Spring XML

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:citrus="http://www.citrusframework.org/schema/config">
    <citrus:before-test id="purgeBeforeTest">
        <citrus:actions>
            <purge-channel>
                <channel name="fooChannel"/>
                <channel name="barChannel"/>
            </purge-channel>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```

Just use this before-test bean in the Spring bean application context and the purge channel action is active. Obsolete messages that are waiting on the message channels for consumption are purged before the next test in line is executed.



Purging message channels becomes also very interesting when working with server instances in Citrus. Each server component automatically has an inbound message channel where incoming messages are stored internally. So if you need to clean up a server that has already stored some incoming messages you can do this easily by purging the internal message channel. The message channel follows a naming convention **{serverName}.inbound** where **{serverName}** is the Spring bean name of the Citrus server endpoint component. If you purge this internal channel in a before test nature you are sure that obsolete messages on a server instance get purged before each test is executed.

8.21. Purging endpoints

Citrus works with message endpoints when sending and receiving messages. In general endpoints can also queue messages. This is especially the case when using JMS message endpoints or any server endpoint component in Citrus. These are in-memory message queues holding messages for test cases. The enqueued messages may become obsolete during a test run, especially when a test case that would consume the messages fails.

Deleting all messages from a message endpoint is therefore a useful task and is essential in such scenarios so that upcoming test cases are not influenced. Each test case should only receive those messages that actually refer to the test model. Therefore it is a good idea to purge all message endpoint destinations between the test cases. Obsolete messages that get stuck in a message endpoint destination for some reason are then removed so that upcoming test case are not broken.

Following action definition purges all messages from a list of message endpoints:

Java

```
@Autowired
@Qualifier("someEndpointName")
private DirectEndpoint someEndpoint;

@Autowired
@Qualifier("anotherEndpointName")
private DirectEndpoint anotherEndpoint;

@CitrusTest
public void purgeEndpointTest() {
    $(purgeEndpoints()
        .endpointNames("someEndpointName", "anotherEndpointName")
    );

    $(purgeEndpoints()
        .endpoints(someEndpoint, anotherEndpoint)
    );
}
```

XML

```
<test name="PurgeEndpointTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-endpoint>
            <endpoint name="someChannelName"/>
            <endpoint name="anotherChannelName"/>
        </purge-endpoint>

        <purge-endpoint>
            <endpoint ref="someChannel"/>
            <endpoint ref="anotherChannel"/>
        </purge-endpoint>
    </actions>
</test>
```

YAML

```
name: PurgeEndpointTest
actions:
  - purge:
      endpoints:
        - name: "someEndpointName"
        - name: "anotherEndpointName"
  - purge:
      endpoints:
        - ref: someEndpoint
        - ref: anotherEndpoint
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="purgeEndpointTest">
      <actions>
          <purge-endpoint>
              <endpoint name="someEndpointName"/>
              <endpoint name="anotherEndpointName"/>
          </purge-endpoint>

          <purge-endpoint>
              <endpoint ref="someEndpoint"/>
              <endpoint ref="anotherEndpoint"/>
          </purge-endpoint>
      </actions>
    </testcase>
</spring:beans>
```

As you can see the test action supports endpoint names as well as endpoint references to Spring bean instances. When using endpoint references you refer to the Spring bean name in your application context.

When using the Java DSL we can inject endpoint objects with Spring bean container IoC.

Message selectors enable you to selectively remove messages from an endpoint. All messages that meet the message selector condition get deleted and the other messages remain inside the endpoint destination. The message selector is either a normal String name-value representation or a map of key value pairs:

Java

```
@CitrusTest
public void purgeEndpointTest() {
    $(purgeEndpoints()
        .selector("operation = 'sayHello'")
        .endpoints(someEndpoint, anotherEndpoint)
    );
}
```

XML

```
<test name="PurgeEndpointTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <purge-endpoint>
            <selector>
                <value>operation = 'sayHello'</value>
            </selector>
            <endpoint ref="someEndpoint"/>
            <endpoint ref="anotherEndpoint"/>
        </purge-endpoint>
    </actions>
</test>
```

YAML

```
name: PurgeEndpointTest
actions:
  - purge-endpoints:
      selector:
        value: "operation = 'sayHello'"
      endpoints:
        - "someEndpoint"
        - "anotherEndpoint"
```

Spring XML

```
<purge-endpoints>
  <selector>
    <value>operation = 'sayHello'</value>
  </selector>
  <endpoint name="someEndpointName"/>
  <endpoint name="anotherEndpointName"/>
</purge-endpoints>
```

In the examples above we use a String to represent the message selector expression. In general the message selector operates on the message header. So following on from that we remove all messages selectively that have a message header **operation** with its value **sayHello** .

Purging endpoints in each test case every time is quite exhausting because every test case needs to define a purging action at the very beginning of the test. A more straight forward approach would be to introduce some purging action which is automatically executed before each test. Fortunately the Citrus test suite offers a very simple way to do this. It is described in testsuite-before-test.

When using the special action sequence before test cases we are able to purge endpoint destinations every time a test case executes. See the upcoming example to find out how the action is defined in the Spring configuration application context.

Spring Bean

```
@Bean
public SequenceBeforeTest beforeTest() {
    return SequenceBeforeTest.Builder.beforeTest()
            .actions(
                purgeEndpoints()
                    .endpoint("fooChannel")
                    .endpoint("barChannel")
            );
}
```

Spring XML

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:citrus="http://www.citrusframework.org/schema/config">
    <citrus:before-test id="purgeBeforeTest">
        <citrus:actions>
            <purge-endpoints>
                <channel name="fooChannel"/>
                <channel name="barChannel"/>
            </purge-endpoints>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```

Just use this before-test bean in the Spring bean application context and the purge endpoint action is active. Obsolete messages that are waiting on the message endpoints for consumption are purged before the next test in line is executed.



Purging message endpoints becomes also very interesting when working with server instances in Citrus. Each server component automatically has an inbound message endpoint where incoming messages are stored too internally. Citrus will automatically use this incoming message endpoint as target for the purge action so you can just use the server instance as you know it from your configuration in any purge action.

8.22. Assert failure

Citrus test actions fail with Java exceptions and error messages. This gives you the opportunity to expect an action to fail during test execution. You can simply assert a Java exception to be thrown during execution. See the example for an assert action definition in a test case:

Java

```
@CitrusTest
public void assertFailureTest() {
    $(assertException()
            .exception(org.citrusframework.exceptions.CitrusRuntimeException.class)
            .message("Unknown variable ${date}")
            .when(
                echo().message("Current date is: ${date}")
            )
    );
}
```

XML

```
<test name="AssertFailureTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <assert exception="org.citrusframework.exceptions.CitrusRuntimeException"
                message="Unknown variable ${date}">
            <when>
                <echo>
                    <message>Current date is: ${date}</message>
                </echo>
            </when>
        </assert>
    </actions>
</test>
```

YAML

```
name: "AssertFailureTest"
actions:
  - assert:
      exception: "org.citrusframework.exceptions.CitrusRuntimeException"
      message: 'Unknown variable ${date}'
      when:
        - echo:
            message: 'Current date is: ${date}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AssertFailureTest">
        <actions>
            <assert exception="org.citrusframework.exceptions.CitrusRuntimeException"
                    message="Unknown variable ${date}">
                <when>
                    <echo>
                        <message>Current date is: ${date}</message>
                    </echo>
                </when>
            </assert>
        </actions>
    </testcase>
</spring:beans>
```



Note that the assert action requires an exception. In case no exception is thrown by the embedded test action the assertion and the test case will fail!

The assert action always wraps a single test action, which is then monitored for failure. In case the nested test action fails with error you can validate the error in its type and error message (optional). The failure has to fit the expected one exactly otherwise the assertion fails itself.



Important to notice is the fact that asserted exceptions do not cause failure of the test case. As you expect the failure to happen the test continues with its work once the assertion is done successfully.

8.23. Catch exceptions

In the previous chapter we have seen how to expect failures in Citrus with assert action. Now the assert action is designed for single actions to be monitored and for failures to be expected in any case. The **'catch'** action in contrary can hold several nested test actions and exception failure is optional.

The nested actions are error proof for the chosen exception type. This means possible exceptions are caught and ignored - the test case will not fail for this exception type. But only for this particular exception type! Other exception types that occur during execution do cause the test to fail as usual.

Java

```
@CitrusTest
public void catchFailureTest() {
    $(catchException()
        .exception(CitrusRuntimeException.class)
        .when(
            echo().message("Current date is: ${date}")
        );
    );
}
```

XML

```
<test name="CatchFailureTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <catch exception="org.citrusframework.exceptions.CitrusRuntimeException">
            <when>
                <echo>
                    <message>Current date is: ${date}</message>
                </echo>
            </when>
        </assert>
    </actions>
</test>
```

YAML

```
name: "CatchFailureTest"
actions:
  - catch:
      exception: "org.citrusframework.exceptions.CitrusRuntimeException"
      when:
        - echo:
            message: 'Current date is: ${date}'
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="catchExceptionTest">
        <actions>
            <catch exception="org.citrusframework.exceptions.CitrusRuntimeException">
                <echo>
                    <message>Current date is: ${date}</message>
                </echo>
            </catch>
        </actions>
    </testcase>
</spring:beans>
```



Note that there is no validation available in a catch block. So catching exceptions is just to make a test more stable towards errors that can occur. The caught exception does not cause any failure in the test. The test case may continue with execution as if there was no failure. Also notice that the catch action is also happy when no exception at all is raised. In contrary to that the assert action requires the exception and an assert action is failing in positive processing.

Catching exceptions like this may only fit to very error-prone action blocks where failures do not harm the test case success. Otherwise, a failure in a test action should always reflect to the whole test case to fail with errors.



Java developers might ask why not use try-catch Java block instead? The answer is simple yet very important to understand. The test method is called by the Java DSL test case builder for building the Citrus test. This can be referred to as the design time of the test. After the building test method was processed the test gets executed, which can be called the runtime of the test. This means that a try-catch block within the design time method will never perform during the test run. The only reliable way to add the catch capability to the test as part of the test case runtime is to use the Citrus test action which gets executed during test runtime.

8.24. Apache Ant build

The <ant> action loads a build.xml Ant file and executes one or more targets in the Ant project. The target is executed with optional build properties passed to the Ant run. The Ant build output is logged with Citrus logger and the test case success is bound to the Ant build success. This means in case the Ant build fails for some reason the test case will also fail with build exception accordingly.

See this basic Ant run example to see how it works within your test case:

Java

```
@CitrusTest
public void antRunTest() {
    variable("today", "citrus:currentDate()");

    $(antrun()
        .buildFilePath("classpath:org/citrusframework/actions/build.xml")
        .target("sayHello")
        .property("date", "${today}")
        .property("welcomeText", "$Hello!"))
    );
}
```

XML

```
<test name="AntRunTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="today" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <ant build-file="classpath:org/citrusframework/actions/build.xml">
            <execute target="sayHello"/>
            <properties>
                <property name="date" value="${today}"/>
                <property name="welcomeText" value="Hello!"/>
            </properties>
        </ant>
    </actions>
</test>
```

YAML

```
name: AntRunTest
variables:
  - name: "today"
    value: "citrus:currentDate()"
actions:
  - ant:
      build-file: "classpath:org/citrusframework/actions/build.xml"
      execute:
        target: "sayHello"
      properties:
        - name: "date"
          value: "${today}"
        - name: "welcomeText"
          value: "Hello!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AntRunTest">
        <variables>
            <variable name="today" value="citrus:currentDate()"/>
        </variables>
        <actions>
            <ant build-file="classpath:org/citrusframework/actions/build.xml">
                <execute target="sayHello"/>
                <properties>
                    <property name="date" value="${today}"/>
                    <property name="welcomeText" value="Hello!"/>
                </properties>
            </ant>
        </actions>
    </testcase>
</spring:beans>
```

The respective build.xml Ant file must provide the target to call. For example:

build.xml

```
<project name="citrus-build" default="sayHello">
    <property name="welcomeText" value="Welcome to Citrus!"></property>

    <target name="sayHello">
        <echo message="${welcomeText} - Today is ${date}"></echo>
    </target>

    <target name="sayGoodbye">
        <echo message="Goodbye everybody!"></echo>
    </target>
</project>
```

As you can see you can pass custom build properties to the Ant build execution. Existing Ant build properties are replaced and you can use the properties in your build file as usual.

You can also call multiple targets within one single build run by using a comma separated list of target names:

Java

```
@CitrusTest
public void antRunTest() {
    variable("today", "citrus:currentDate()");

    $(antrun()
        .buildFilePath("classpath:org/citrusframework/actions/build.xml")
        .targets("sayHello", "sayGoodbye")
        .property("date", "${today}"))
    );
}
```

XML

```
<test name="AntRunTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="today" value="citrus:currentDate()"/>
    </variables>
    <actions>
        <ant build-file="classpath:org/citrusframework/actions/build.xml">
            <execute targets="sayHello,sayGoodbye"/>
            <properties>
                <property name="date" value="${today}"/>
            </properties>
        </ant>
    </actions>
</test>
```

YAML

```
name: AntRunTest
variables:
  - name: "today"
    value: "citrus:currentDate()"
actions:
  - ant:
      build-file: "classpath:org/citrusframework/actions/build.xml"
      execute:
        targets: "sayHello,sayGoodbye"
      properties:
        - name: "date"
          value: "${today}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AntRunTest">
        <variables>
            <variable name="today" value="citrus:currentDate()"/>
        </variables>
        <actions>
            <ant build-file="classpath:org/citrusframework/actions/build.xml">
                <execute targets="sayHello,sayGoodbye"/>
                <properties>
                    <property name="date" value="${today}"/>
                </properties>
            </ant>
        </actions>
    </testcase>
</spring:beans>
```

The build properties can live in external file resource as an alternative to the inline property definitions. You just have to use the respective file resource path and all nested properties get loaded as build properties.

In addition to that you can also define a custom build listener. The build listener must implement the Ant API interface **org.apache.tools.ant.BuildListener** . During the Ant build run the build listener is called with several callback methods (e.g. buildStarted(), buildFinished(), targetStarted(), targetFinished(), ...). This is how you can add additional logic to the Ant build run from Citrus. A custom build listener could manage the fail state of your test case, in particular by raising some exception forcing the test case to fail accordingly.

Java

```
@Autowired
private BuildListener customBuildListener;

@CitrusTest
public void antRunTest() {
    $(antrun()
        .buildFilePath("classpath:org/citrusframework/actions/build.xml")
        .target("sayHello")
        .propertyFile("classpath:org/citrusframework/actions/build.properties")
        .listener(customBuildListener))
    );
}
```

XML

```
<test name="AntRunTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <ant build-file="classpath:org/citrusframework/actions/build.xml"
             build-listener="customBuildListener">
            <execute target="sayHello"/>
            <properties  file="classpath:org/citrusframework/actions/build.properties"
/>
        </ant>
    </actions>
</test>
```

YAML

```
name: AntRunTest
actions:
  - ant:
      build-file: "classpath:org/citrusframework/actions/build.xml"
      build-listener: "customBuildListener"
      execute:
        target: "sayHello"
      properties-file: "classpath:org/citrusframework/actions/build.properties"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AntRunTest">
        <actions>
            <ant build-file="classpath:org/citrusframework/actions/build.xml"
                 build-listener="customBuildListener">
                <execute target="sayHello"/>
                <properties
file="classpath:org/citrusframework/actions/build.properties"/>
            </ant>
        </actions>
    </testcase>
</spring:beans>
```

The **customBuildListener** used in the example above should reference a Spring bean in the Citrus application context. The bean implements the interface **org.apache.tools.ant.BuildListener** and controls the Ant build run.

8.25. Start/Stop server

Citrus is working with server components that are started and stopped within a test run. This can be a Http server or some SMTP mail server for instance. Usually the Citrus server components are

automatically started when Citrus is starting and respectively stopped when Citrus is shutting down. Sometimes it might be helpful to explicitly start and stop a server instance within your test case. Here you can use special start and stop test actions inside your test. This is a good way to test downtime scenarios of interface partners with respective error handling when connections to servers are lost

Let me explain with a simple sample test case:

Java

```
@Autowired
@Qualifier("myMailServer")
private MailServer myMailServer;

@CitrusTest
public void manageServerTest() {
    $(start()
        .server(myMailServer));

    $(sequential()
        .actions(...)); // do something

    $(stop()
        .server(myMailServer));
}
```

XML

```
<test name="ManageServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <start server="myMailServer"/>

        <sequential>
            <!-- do something -->
        </sequential>

        <stop server="myMailServer"/>
    </actions>
</test>
```

YAML

```
name: ManageServerTest
actions:
  - start:
      server: "myMailServer"
  - sequential:
      actions: {}
  - stop:
      server: "myMailServer"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="manageServerTest">
        <actions>
            <start server="myMailServer"/>

            <sequential>
                <!-- do something -->
            </sequential>

            <stop server="myMailServer"/>
        </actions>
    </testcase>
</spring:beans>
```

The start and stop server test action receive a server name which references a Spring bean component of type **org.citrusframework.server.Server** in your basic Spring application context. The server instance is started or stopped within the test case. As you can see in the next listing we can also start and stop multiple server instances within a single test action.

Java

```
@Autowired
@Qualifier("myMailServer")
private MailServer myMailServer;

@Autowired
@Qualifier("myFtpServer")
private FtpServer myFtpServer;

@CitrusTest
public void manageServerTest() {
    $(start()
        .server(myMailServer, myFtpServer));

    $(sequential()
        .actions(...)); // do something

    $(stop()
        .server(myMailServer, myFtpServer));
}
```

XML

```
<test name="ManageServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <start>
            <servers>
                <server name="myMailServer"/>
                <server name="myFtpServer"/>
            </servers>
        </start>

        <sequential>
            <!-- do something -->
        </sequential>

        <stop>
            <servers>
                <server name="myMailServer"/>
                <server name="myFtpServer"/>
            </servers>
        </stop>
    </actions>
</test>
```

YAML

```
name: ManageServerTest
actions:
  - start:
      servers:
        - "myMailServer"
        - "myFtpServer"
  - sequential:
      actions: {}
  - stop:
      servers:
        - "myMailServer"
        - "myFtpServer"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="manageServerTest">
        <actions>
            <start>
                <servers>
                    <server name="myMailServer"/>
                    <server name="myFtpServer"/>
                </servers>
            </start>

            <sequential>
                <!-- do something -->
            </sequential>

            <stop>
                <servers>
                    <server name="myMailServer"/>
                    <server name="myFtpServer"/>
                </servers>
            </stop>
        </actions>
    </testcase>
</spring:beans>
```

When using the Java DSL the best way to reference a server instance is to autowire the Spring bean via dependency injection. The Spring framework takes care of injecting the proper Spring bean component defined in the Spring application context. This way you can easily start and stop server instances within Java DSL test cases.



Starting and stopping server instances is a synchronous test action. This means that your test case is waiting for the server to start before other test actions take place. Startup times and shut down of server instances may delay your test accordingly.

As you can see starting and stopping Citrus server instances is very easy. You can also write your own server implementations by implementing the interface **org.citrusframework.server.Server**. All custom server implementations can then be started and stopped during a test case.

8.26. Stop Timer

The <stop-timer> action can be used for stopping either a specific timer (containers-timer) or all timers running within a test. This action is useful when timers are started in the background (using parallel or fork=true) and you wish to stop these timers at the end of the test. Some examples of using this action are provided below:

Java

```
@CitrusTest
public void timerTest() {
    $(timer()
      .id("forkedTimer")
      .fork(true)
      .actions(sleep(50L))
    );

    $(timer()
      .fork(true)
      .actions(sleep(50L))
    );

    $(timer()
      .repeatCount(5)
      .actions(sleep(50L))
    );

    $(stopTimer("forkedTimer"));

    $(doFinally().actions(
      stopTimer()
    ));
}
```

XML

```
<test name="TimerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <timer id="forkedTimer" fork="true">
            <actions>
                <sleep milliseconds="50" />
            </actions>
        </timer>

        <timer fork="true">
            <actions>
                <sleep milliseconds="50" />
            </actions>
        </timer>

        <timer repeatCount="5">
            <actions>
                <sleep milliseconds="50" />
            </actions>
        </timer>

        <stop-timer id="forkedTimer" />
    </actions>

    <finally>
        <stop-timer/>
    </finally>
</test>
```

YAML

```
name: TimerTest
actions:
  - timer:
      id: "forkedTimer"
      fork: true
      actions:
        - sleep:
            milliseconds: "50"
  - timer:
      fork: true
      actions:
        - sleep:
            milliseconds: "50"
  - timer:
      repeatCount: 5
      actions:
        - sleep:
            milliseconds: "50"
  - stop-timer:
      id: "forkedTimer"
finally:
  - stop-timer: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="TimerTest">
        <actions>
            <timer id="forkedTimer" fork="true">
                <sleep milliseconds="50" />
            </timer>

            <timer fork="true">
                <sleep milliseconds="50" />
            </timer>

            <timer repeatCount="5">
                <sleep milliseconds="50" />
            </timer>

            <stop-timer timerId="forkedTimer" />
        </actions>

        <finally>
            <stop-timer/>
        </finally>
    </testcase>
</spring:beans>
```

In the above example 3 timers are started, the first 2 in the background and the third in the test execution thread. Timer #3 has a repeatCount set to 5 so it will terminate automatically after 5 runs. Timer #1 and #2 however have no repeatCount set so they will execute until they are told to stop.

Timer #1 is stopped explicitly using the first stopTimer action. Here the stopTimer action includes the name of the timer to stop. This is convenient when you wish to terminate a specific timer. However since no timerId was set for timer #2, you can terminate this (and all other timers) using the 'stopTimer' action with no explicit timerId set.

8.27. Custom action

Now we have a look at the opportunity to add custom test actions to the test case flow. Let us start this section with an example:

Java

```
@CitrusTest
public void actionReferenceTest() {
    $(action().reference("clearDatabase"));
    $(action().reference("mySpecialAction"));
}
```

XML

```
<test name="ActionReferenceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <action reference="clearDatabase"/>
        <action reference="mySpecialAction"/>
    </actions>
</test>
```

YAML

```
name: ActionReferenceTest
actions:
  - action:
      ref: "clearDatabase"
  - action:
      ref: "mySpecialAction"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ActionReferenceTest">
        <actions>
            <action reference="clearDatabase"/>
            <action reference="mySpecialAction"/>
        </actions>
    </testcase>
</spring:beans>
```

The generic <action> element references Spring beans that implement the Java interface ***org.citrusframework.TestAction*** . This is a very fast way to add your own action implementations to a Citrus test case. This way you can easily implement your own actions in Java and include them into the test case.

In the example above the called actions are special database cleanup implementations. The actions are defined as Spring beans in the Citrus configuration and get referenced by their bean name or id.

Spring Bean

```
@Bean
public SpecialDatabaseCleanupAction clearDatabase(DataSource ds) {
    return new SpecialDatabaseCleanupAction(ds);
}
```

Spring XML

```
<bean id="clearDatabase"
class="my.domain.citrus.actions.SpecialDatabaseCleanupAction">
    <constructor-arg ref="myDataSource"/>
</bean>
```

The Spring application context holds your custom bean implementations. You can set properties and use the full Spring power while implementing your custom test action in Java. Let us have a look on how such a Java class may look like.

SpecialDatabaseCleanupAction

```
import org.citrusframework.actions.AbstractTestAction;
import org.citrusframework.context.TestContext;

public class SpecialDatabaseCleanupAction extends AbstractTestAction {

    @Autowired
    private DataSource dataSource;

    @Override
    public void doExecute(TestContext context) {
        JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);

        jdbcTemplate.execute("...");
    }
}
```

All you need to do in your Java class is to implement the Citrus ***org.citrusframework.TestAction*** interface. The abstract class ***org.citrusframework.actions.AbstractTestAction*** may help you to start with your custom test action implementation as it provides basic method implementations so you just have to implement the ***doExecute()*** method.

When using the Java test case DSL you are also quite comfortable with including your custom test actions.

Java

```
@Autowired
private SpecialDatabaseCleanupAction clearDatabaseAction;

@CitrusTest
public void actionReferenceTest() {
    $(echo().message("Now let's include our special test action"));

    $(clearDatabaseAction);

    $(echo().message("That's it!"));
}
```

Using anonymous class uses as Lambda expression is also possible.

Java

```
@CitrusTest
public void genericActionTest() {
    $(echo().message("Now let's call our special test action anonymously"));

    $(context -> {
        // do something
    });

    $(echo().message("That's it!"));
}
```

The test action receives a single argument `context` which represent the `org.citrusframework.context.TestContext`. The context provides access to test variables and shared components resolvable via reference resolver API.

Chapter 9. Containers

Similar to templates a container element holds one to many test actions. In contrast to the template the container appears directly inside the test case action chain, meaning that the container is not referenced by more than one test case.

Containers execute the embedded test actions in specific logic. This can be an execution in iteration for instance. Combine different containers with each other and you will be able to generate very powerful hierarchical structures in order to create a complex execution logic. In the following sections some predefined containers are described.

9.1. Sequential

The sequential container executes the embedded test actions in strict sequence. Readers now might search for the difference to the normal action chain that is specified inside the test case. The actual power of sequential containers does show only in combination with other containers like iterations and parallels. We will see this later when handling these containers.

For now the sequential container seems not very sensational - one might say boring - because it simply groups a pair of test actions to sequential execution.

Java

```
@CitrusTest
public void sequentialTest() {
    $(sequential()
        .actions(
            sleep().seconds(1),
            echo("Hello Citrus!")
        )
    );
}
```

XML

```
<test name="SequentialTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <sequential>
            <actions>
                <sleep seconds="1"/>
                <echo>
                    <message>Hello Citrus!</message>
                </echo>
            </actions>
        </sequential>
    </actions>
</test>
```

YAML

```
name: SequentialTest
actions:
  - sequential:
      actions:
        - sleep:
            seconds: 1
        - echo:
            message: "Hello Citrus!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="SequentialTest">
        <actions>
            <sequential>
                <sleep seconds="1"/>
                <echo>
                    <message>Hello Citrus!</message>
                </echo>
            </sequential>
        </actions>
    </testcase>
</spring:beans>
```

9.2. Conditional

Now we deal with conditional executions of test actions. Nested actions inside a conditional container are executed only in case a boolean expression evaluates to true. Otherwise the container execution is not performed at all.

See some example to find out how it works with the conditional expression string.

Java

```
@CitrusTest
public void conditionalTest() {
    variable("index", 5);
    variable("shouldSleep", true);

    $(conditional().when("${index} = 5")
        .actions(
            sleep(10000L)
        )
    );

    $(conditional().when("${shouldSleep}")
        .actions(
            sleep(10000L)
        )
    );

    $(conditional().when("${shouldSleep}", anyOf(is("true"), isEmptyString()))
        .actions(
            sleep(10000L)
        )
    );
}
```

XML

```
<test name="ConditionalTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
      <variable name="index" value="5"/>
      <variable name="shouldSleep" value="true"/>
    </variables>

    <actions>
        <conditional when="${index} = 5">
            <actions>
                <sleep seconds="10"/>
            </actions>
        </conditional>

        <conditional when="${shouldSleep}">
            <actions>
                <sleep seconds="10"/>
            </actions>
        </conditional>

        <conditional when="@assertThat('${shouldSleep}', 'anyOf(is(true),
isEmptyString())')@">
            <actions>
                <sleep seconds="10"/>
            </actions>
        </conditional>
    </actions>
</test>
```

YAML

```
name: ConditionalTest
variables:
  - name: "index"
    value: 5
  - name: "shouldSleep"
    value: true

actions:
  - conditional:
      when: '${index} = 5'
      actions:
        - sleep:
            seconds: 10
  - conditional:
      when: '${shouldSleep}'
      actions:
        - sleep:
            seconds: 10
  - conditional:
      when: '@assertThat("${shouldSleep}", "anyOf(is(true), isEmptyString())")@'
      actions:
        - sleep:
            seconds: 10
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ConditionalTest">
        <variables>
          <variable name="index" value="5"/>
          <variable name="shouldSleep" value="true"/>
        </variables>

        <actions>
            <conditional expression="${index} = 5">
                <sleep seconds="10"/>
            </conditional>

            <conditional expression="${shouldSleep}">
                <sleep seconds="10"/>
            </conditional>

            <conditional expression="@assertThat('${shouldSleep}', 'anyOf(is(true),
isEmptyString())')@">
                <sleep seconds="10"/>
            </conditional>
        </actions>
    </testcase>
</spring:beans>
```

The nested sleep action is executed in case the variable ${index} is equal to the value '5'. This conditional execution of test actions is useful when dealing with different test environments such as different operating systems for instance. The conditional container also supports expressions that evaluate to the character sequence "true" or "false" as shown in the ${shouldSleep} example.

The last conditional container in the example above makes use of Hamcrest matchers. The matcher evaluates to **true** or **false** and based on that the container actions are executed or skipped. The Hamcrest matchers are very powerful when it comes to evaluation of multiple conditions at a time.

9.3. Parallel

Parallel containers execute embedded test actions concurrently to each other. Every action in this container will be executed in a separate Java thread. The following example should clarify the usage:

Java

```
@CitrusTest
public void parallelTest() {
    $(parallel()
        .actions(
            sleep(),
            sequential().actions(
                sleep(),
                echo("1")
            ),
            echo("2"),
            echo("3"),
            iterate().condition("i lt= 5").index("i"))
                .actions(
                    echo("10")
                )
        )
    );
}
```

XML

```
<test name="ParallelTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <parallel>
            <actions>
                <sleep/>

                <sequential>
                    <actions>
                        <sleep/>
                        <echo>
                            <message>1</message>
                        </echo>
                    </actions>
                </sequential>

                <echo>
                    <message>2</message>
                </echo>

                <echo>
                    <message>3</message>
                </echo>

                <iterate condition="i lt= 5"
                            index="i">
                    <actions>
                        <echo>
                            <message>10</message>
                        </echo>
                    </actions>
                </iterate>
            </actions>
        </parallel>
    </actions>
</test>
```

YAML

```
name: ParallelTest
actions:
  - parallel:
      actions:
        - sleep: {}
        - sequential:
            actions:
              - sleep: {}
              - echo:
                  message: "1"
        - echo:
            message: "2"
        - echo:
            message: "3"
        - iterate:
            index: "i"
            condition: "i lt 5"
            actions:
              - echo:
                  message: "10"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="ParallelTest">
        <actions>
            <parallel>
                <sleep/>

                <sequential>
                    <sleep/>
                    <echo>
                        <message>1</message>
                    </echo>
                </sequential>

                <echo>
                    <message>2</message>
                </echo>

                <echo>
                    <message>3</message>
                </echo>

                <iterate condition="i lt= 5"
                            index="i">
                    <echo>
                        <message>10</message>
                    </echo>
                </iterate>
            </parallel>
        </actions>
    </testcase>
</spring:beans>
```

By default, test actions are processed and executed one action after another. Since the first action is a sleep of five seconds, the whole test would stop and wait for 5 seconds. Things are different inside the parallel container. Here, the descending test actions will not wait, but execute at the same time.

If you are using this container to send or receive messages, you have to use the unique correlation ID of the message to link the actions concerning this message. Otherwise, the testcase might associate a send or receive action with the wrong message. Please note that this ID is **not** passed to your system under test. The management of correlation IDs as well as the assignment to messages is done internally. Only the mapping between the request and response has to be done by the author of the test. As you can see in the following example, the value of the header `MessageHeaders.ID` is stored in the variable `request#1` respectively `request#2`. This variable is reused in the `receive` action to identify the correct response from the server.

Java

```
@CitrusTest
public void paralletTest() {

    $(parallel()
        .actions(
            sequential()
                .actions(
                    http().client(httpClient)
                            .send()
                            .post("/foo")
                            .extract(extract().fromHeaders().header(MessageHeaders.ID,
"request#1")
                            .payload("{ \"info\": \"foo\"}"),

                    //SUT echoing the input

                    http().client(httpClient)
                            .receive()
                            .response(HttpStatus.OK.value())
                            .payload("{ \"info\": \"foo\"}")
                            .selector(
                                Collections.singletonMap(
                                    MessageHeaders.ID, "${request#1}"))
                ),

            sequential()
                .actions(
                    http().client(httpClient)
                         .send()
                         .post("/boo")
                         .extract(extract().fromHeaders().header(MessageHeaders.ID,
"request#2")
                         .payload("{ \"info\": \"boo\"}"),

                    //SUT echoing the input

                    http().client(httpClient)
                         .receive()
                         .response(HttpStatus.OK.value())
                         .payload("{ \"info\": \"boo\"}")
                         .selector(
                            Collections.singletonMap(
                                MessageHeaders.ID, "${request#2}"))
                )
        )
    );
}
```



Containers can easily wrap other containers. The example shows a simple combination of sequential and parallel containers that will achieve more complex execution logic. Actions inside the sequential container will execute one after another. But actions in parallel will be executed at the same time.

9.4. Iterate

Iterations are very powerful elements when describing complex logic. The container executes the embedded actions several times. The container will continue with looping as long as the defined breaking condition string evaluates to ***true*** . In case the condition evaluates to ***false*** the iteration will break and finish execution.

Java

```
@CitrusTest
public void iterateTest() {
    $(iterate().condition("i lt 5").index("i")
        .actions(
            echo().message("index is: ${i}")
        )
    );
}
```

XML

```
<test name="IterateTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <iterate index="i" condition="i lt 5">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </iterate>
    </actions>
</test>
```

YAML

```
name: IterateTest
actions:
  - iterate:
      index: "i"
      condition: "i lt 5"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="IterateTest">
        <actions>
            <iterate index="i" condition="i lt 5">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </iterate>
        </actions>
    </testcase>
</spring:beans>
```

The attribute "index" automatically defines a new variable that holds the actual loop index starting at "1". This index variable is available as a normal variable inside the iterate container. Therefore it is possible to print out the actual loop index in the echo action as shown in the above example.

9.4.1. Fixed number of iterations

When you simply want to execute the actions a fixed number of times you can use the `times` option as a shortcut instead of writing a condition expression.

Java

```
@CitrusTest
public void iterateTimesTest() {
    $(iterate().times(5)
        .actions(
            echo().message("index is: ${i}")
        )
    );
}
```

XML

```
<test name="IterateTimesTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <iterate times="5">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </iterate>
    </actions>
</test>
```

YAML

```
name: IterateTimesTest
actions:
  - iterate:
      times: 5
      actions:
        - echo:
            message: "index is: ${i}"
```

The `times` option automatically creates the proper condition expression to execute the loop exactly the given number of times. This is equivalent to setting `condition("i lt= 5")` but is more concise and readable.



The `times` option is an alternative to the `condition` expression. You should use one or the other, not both.

9.4.2. Condition expression

The condition string describes the actual end of the loop. In iterate containers the loop will break in case the condition evaluates to ***false***.

The condition string can be any Boolean expression and supports several operators:

lt lower than

lt= lower than equals

gt greater than

gt= greater than equals

= equals

and logical combining of two Boolean values

or logical combining of two Boolean values

() brackets



It is very important to notice that the condition is evaluated before the very first iteration takes place. The loop therefore can be executed 0-n times according to the condition value.

Now the boolean expression evaluation as described above is limited to very basic operation such as **lower than**, **greater than** and so on. We also can use Hamcrest matchers in conditions that are

way more powerful than that.

Java

```
@CitrusTest
public void iterateTest() {
    $(iterate().condition(assertThat(lessThan(5)).index("i"))
        .actions(
            echo().message("index is: ${i}")
        )
    );
}
```

XML

```
<test name="IterateTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <iterate index="i" condition="@assertThat(lessThan(5))@">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </iterate>
    </actions>
</test>
```

YAML

```
name: IterateTest
actions:
  - iterate:
      index: "i"
      condition: "@assertThat(lessThan(5))@"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="IterateTest">
        <actions>
            <iterate index="i" condition="@assertThat(lessThan(5))@">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </iterate>
        </actions>
    </testcase>
</spring:beans>
```

In the example above we use Hamcrest matchers as a condition expression. In Java you may use the `org.citrusframework.container.HamcrestConditionExpression.assertThat(org.hamcrest.Matcher<?>)` method. This way you can combine Hamcrest matchers and create very powerful condition evaluations here.

The Hamcrest expression evaluation in Citrus gets added with this module:

citrus-validation-hamcrest Module

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-validation-hamcrest</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

You need to add this module to your project accordingly when using the Hamcrest matcher support.

9.4.3. AssertJ conditions

Alternatively, you can use AssertJ assertions in conditions. This is way more powerful than the basic boolean expression evaluation described above.

Java

```
@CitrusTest
public void iterateTest() {
    $(iterate().condition(conditionOf(i -> assertThat(i).isLessThan(5)))
        .index("i")
        .actions(
            echo().message("index is: ${i}")
        )
    );
}
```

XML

```
<test name="IterateTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <iterate index="i" condition="@assertj(asInt().isLessThan(5))@">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </iterate>
    </actions>
</test>
```

YAML

```
name: IterateTest
actions:
  - iterate:
      index: "i"
      condition: "@assertj(asInt().isLessThan(5))@"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="IterateTest">
        <actions>
            <iterate index="i" condition="@assertj(asInt().isLessThan(5))@">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </iterate>
        </actions>
    </testcase>
</spring:beans>
```

In the example above we use the `@assertj()@` validation matcher as a condition expression. The matcher receives the iteration index as a String, so the chain starts with `asInt()`. In Java you may use the `org.citrusframework.validation.assertj.AssertJ.conditionOf(…)` method instead.

To assert on another value than the iteration index, give the value first and the chain second, for instance `@assertj('${count}', 'asInt().isLessThan(5)')@`.

The AssertJ expression evaluation in Citrus gets added with this module:

citrus-validation-assertj Module

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-validation-assertj</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

You need to add this module to your project accordingly when using the AssertJ support. See AssertJ container conditions for details.

9.5. Repeat

Quite similar to the previously described iterate container this repeating container will execute its actions in a loop according to an ending condition. The condition describes a Boolean expression using the operators as described in the previous chapter.



The loop continues its work until the provided condition evaluates to **true** . It is very important to notice that the repeat loop will execute the actions before evaluating the condition. This means the actions get executed `n-1` times.

Java

```
@CitrusTest
public void repeatTest() {
    $(repeat().until("(i gt 5) or (i = 3)").index("i")
        .actions(
            echo("index is: ${i}")
        )
    );
}
```

XML

```
<test name="RepeatTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat index="i" until="(i = 3) or (i = 5)">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </repeat>
    </actions>
</test>
```

YAML

```
name: RepeatTest
actions:
  - repeat:
      index: "i"
      until: "(i = 3) or (i = 5)"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatTest">
        <actions>
            <repeat-until-true index="i" condition="(i = 3) or (i = 5)">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </repeat-until-true>
        </actions>
    </testcase>
</spring:beans>
```

As you can see the repeat container is only executed when the iterating condition expression evaluates to **false** . By the time the condition is **true** execution is discontinued. You can use basic logical operators such as **and**, **or** and so on.

9.5.1. Fixed number of iterations

When you simply want to repeat the actions a fixed number of times you can use the `times` option as a shortcut instead of writing a condition expression.

Java

```
@CitrusTest
public void repeatTimesTest() {
    $(repeat().times(5)
        .actions(
            echo("index is: ${i}")
        )
    );
}
```

XML

```
<test name="RepeatTimesTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat times="5">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </repeat>
    </actions>
</test>
```

YAML

```
name: RepeatTimesTest
actions:
  - repeat:
      times: 5
      actions:
        - echo:
            message: "index is: ${i}"
```

The `times` option automatically creates the proper condition expression to execute the loop exactly the given number of times. This is equivalent to setting `until("i gt 5")` but is more concise and readable.



The `times` option is an alternative to the `until` condition expression. You should use one or the other, not both.

9.5.2. Hamcrest matcher conditions

A more powerful way is given by Hamcrest matchers that are directly supported in condition expressions.

Java

```
@CitrusTest
public void repeatTest() {
    $(repeat().until(assertThat(anyOf(is(3), is(5))).index("i")
        .actions(
            echo("index is: ${i}")
        )
    );
}
```

XML

```
<test name="RepeatTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat index="i" until="@assertThat(anyOf(is(3), is(5))@">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </repeat>
    </actions>
</test>
```

YAML

```
name: RepeatTest
actions:
  - repeat:
      index: "i"
      until: "@assertThat(anyOf(is(3), is(5))@"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatTest">
        <actions>
            <repeat-until-true index="i" condition="@assertThat(anyOf(is(3), is(5))@">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </repeat-until-true>
        </actions>
    </testcase>
</spring:beans>
```

The Hamcrest matcher usage simplifies the reading a lot. In addition to that it empowers you to combine more complex condition expressions. In Java you may use the `org.citrusframework.container.HamcrestConditionExpression.assertThat(org.hamcrest.Matcher<?>)` method.

The Hamcrest expression evaluation in Citrus gets added with this module:

citrus-validation-hamcrest Module

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-validation-hamcrest</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

You need to add this module to your project accordingly when using the Hamcrest matcher support.

9.5.3. AssertJ conditions

Alternatively, you can use AssertJ assertions in conditions. This is way more powerful than the basic boolean expression evaluation described above.

Java

```
@CitrusTest
public void repeatTest() {
    $(repeat().until(conditionOf(i -> assertThat(i).isGreaterThanOrEqualTo(5)))
        .index("i")
        .actions(
            echo("index is: ${i}")
        )
    );
}
```

XML

```
<test name="RepeatTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat index="i" until="@assertj(asInt().isGreaterThanOrEqualTo(5))@">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </actions>
        </repeat>
    </actions>
</test>
```

YAML

```
name: RepeatTest
actions:
  - repeat:
      index: "i"
      until: "@assertj(asInt().isGreaterThanOrEqualTo(5))@"
      actions:
        - echo:
            message: "index is: ${i}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatTest">
        <actions>
            <repeat-until-true index="i"
condition="@assertj(asInt().isGreaterThanOrEqualTo(5))@">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
            </repeat-until-true>
        </actions>
    </testcase>
</spring:beans>
```

In the example above we use the `@assertj()@` validation matcher as a condition expression. The matcher receives the iteration index as a String, so the chain starts with `asInt()`. In Java you may use the `org.citrusframework.validation.assertj.AssertJ.conditionOf(…)` method instead.

To assert on another value than the iteration index, give the value first and the chain second, for instance `@assertj('${count}', 'asInt().isGreaterThanOrEqualTo(5)')@`.

The AssertJ expression evaluation in Citrus gets added with this module:

citrus-validation-assertj Module

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-validation-assertj</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

You need to add this module to your project accordingly when using the AssertJ support. See AssertJ container conditions for details.

9.6. Repeat on error

The next looping container is called repeat-on-error-until-true. This container repeats a group of actions in case one embedded action failed with error. In case of an error inside the container the loop will try to execute ***all*** embedded actions again in order to seek for overall success. The execution continues until all embedded actions were processed successfully ***or*** the ending condition evaluates to true and the error-loop will lead to final failure.

Java

```
@CitrusTest
public void repeatOnErrorTest() {
    $(repeatOnError().until("i = 5").index("i")
        .actions(
            echo().message("index is: ${i}"),
            fail().message("Force loop to fail!")
        )
    );
}
```

XML

```
<test name="RepeatOnErrorTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat-on-error index="i" until="i = 5">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
                <fail message="Force loop to fail!"/>
            </actions>
        </repeat-on-error>
    </actions>
</test>
```

YAML

```
name: RepeatOnErrorTest
actions:
  - repeat-on-error:
      index: "i"
      until: "i = 5"
      actions:
        - echo:
            message: "index is: ${i}"
        - fail:
            message: "Force loop to fail!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatOnErrorTest">
        <actions>
            <repeat-onerror-until-true index="i" condition="i = 5">
                <echo>
                    <message>index is: ${i}</message>
                </echo>
                <fail message="Force loop to fail!"/>
            </repeat-onerror-until-true>
        </actions>
    </testcase>
</spring:beans>
```

In the code example the error-loop continues four times as the <fail> action definitely fails the test. During the fifth iteration the condition "i=5" evaluates to true and the loop breaks its processing leading to a final failure as the test actions were not successful.

9.6.1. Fixed number of retries

When you simply want to retry the actions a fixed number of times on error you can use the `times` option as a shortcut instead of writing a condition expression.

Java

```
@CitrusTest
public void repeatOnErrorTimesTest() {
    $(repeatOnError().times(5)
        .actions(
            echo().message("index is: ${i}"),
            fail().message("Force loop to fail!")
        )
    );
}
```

XML

```
<test name="RepeatOnErrorTimesTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat-on-error times="5">
            <actions>
                <echo>
                    <message>index is: ${i}</message>
                </echo>
                <fail message="Force loop to fail!"/>
            </actions>
        </repeat-on-error>
    </actions>
</test>
```

YAML

```
name: RepeatOnErrorTimesTest
actions:
  - repeat-on-error:
      times: 5
      actions:
        - echo:
            message: "index is: ${i}"
        - fail:
            message: "Force loop to fail!"
```

The `times` option automatically creates the proper condition expression to retry the loop at most the given number of times. This is equivalent to setting `until("i gt 5")` but is more concise and readable.



The `times` option is an alternative to the `until` condition expression. You should use one or the other, not both.



The overall success of the test case depends on the error situation inside the repeat-onerror-until-true container. In case the loop breaks because of failing actions and the loop will discontinue its work the whole test case is failing too. The error loop processing is successful in case all embedded actions were not raising any errors during an iteration.

9.6.2. AssertJ conditions

Alternatively, you can use AssertJ assertions in the ending condition. This is way more powerful than the basic boolean expression evaluation described above.

Java

```
@CitrusTest
public void repeatOnErrorTest() {
    $(repeatOnError().until(conditionOf(i -> assertThat(i).isEqualTo(3)))
        .index("i")
        .autoSleep(50)
        .actions(
            echo().message("${i}. attempt!"),
            fail().message("Force loop to fail!")
        )
    );
}
```

XML

```
<test name="RepeatOnErrorTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat-on-error auto-sleep="50" until="@assertj(asInt().isEqualTo(3))@"
index="i">
            <actions>
                <echo>
                    <message>${i}. attempt!</message>
                </echo>
                <fail message="Force loop to fail!"/>
            </actions>
        </repeat-on-error>
    </actions>
</test>
```

YAML

```
name: RepeatOnErrorTest
actions:
  - repeat-on-error:
      index: "i"
      until: "@assertj(asInt().isEqualTo(3))@"
      auto-sleep: 50
      actions:
        - echo:
            message: "${i}. attempt!"
        - fail:
            message: "Force loop to fail!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatOnErrorTest">
        <actions>
            <repeat-onerror-until-true auto-sleep="50"
condition="@assertj(asInt().isEqualTo(3))@" index="i">
                <echo>
                    <message>${i}. attempt!</message>
                </echo>
                <fail message="Force loop to fail!"/>
            </repeat-onerror-until-true>
        </actions>
    </testcase>
</spring:beans>
```

In the example above we use the `@assertj()@` validation matcher as a condition expression. The matcher receives the iteration index as a String, so the chain starts with `asInt()`. In Java you may use the `org.citrusframework.validation.assertj.AssertJ.conditionOf(…)` method instead.

The AssertJ expression evaluation in Citrus gets added with this module:

citrus-validation-assertj Module

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-validation-assertj</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

You need to add this module to your project accordingly when using the AssertJ support. See AssertJ container conditions for details.

The repeat-on-error container also offers an automatic sleep mechanism. This auto-sleep property will force the container to wait a given amount of time before executing the next iteration. We used this mechanism a lot when validating database entries. Let's say we want to check the existence of an order entry in the database. Unfortunately the system under test is not very well performing and may need some time to store the new order. This amount of time is not predictable, especially when dealing with different hardware on our test environments (local testing vs. server testing). Following from that our test case may fail unpredictably only because of runtime conditions.

We can avoid unstable test cases that are based on these runtime conditions with the auto-sleep functionality.

Java

```
@CitrusTest
public void repeatOnErrorTest() {
    $(repeatOnError().until("i = 5").index("i").autoSleep(1000)
        .actions(
            sql()
                .dataSource(testDataSource)
                .query()
                .statement("""
                    SELECT COUNT(1) AS CNT_ORDERS
                    FROM ORDERS
                    WHERE CUSTOMER_ID='${customerId}'
                """)
                .validate("CNT_ORDERS", 1)
        )
    );
}
```

XML

```
<test name="RepeatOnErrorTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <repeat-on-error auto-sleep="1000" until="i = 5" index="i">
            <actions>
                <sql datasource="testDataSource">
                    <statements>
                        <statement>
                          SELECT COUNT(1) AS CNT_ORDERS
                          FROM ORDERS
                          WHERE CUSTOMER_ID='${customerId}'
                        </statement>
                    </statements>
                    <validate column="CNT_ORDERS" value="1"/>
                </sql>
            </actions>
        </repeat-on-error>
    </actions>
</test>
```

YAML

```
name: RepeatOnErrorTest
actions:
  - repeat-on-error:
      index: "i"
      until: "i = 5"
      auto-sleep: 1000
      actions:
        - sql:
            dataSource: "testDataSource"
            statements:
              - statement: |
                  SELECT COUNT(1) AS CNT_ORDERS
                  FROM ORDERS
                  WHERE CUSTOMER_ID='${customerId}'
            validate:
              - column: "CNT_ORDERS"
                value: 1
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RepeatOnErrorTest">
        <actions>
            <repeat-onerror-until-true auto-sleep="1000" condition="i = 5" index="i">
                <sql datasource="testDataSource">
                    <statement>
                      SELECT COUNT(1) AS CNT_ORDERS
                      FROM ORDERS
                      WHERE CUSTOMER_ID='${customerId}'
                    </statement>
                    <validate column="CNT_ORDERS" value="1"/>
                </sql>
            </repeat-onerror-until-true>
        </actions>
    </testcase>
</spring:beans>
```

We surrounded the database check with a repeat-onerror container having the auto-sleep property set to 1000 milliseconds. The repeat container will try to check the database up to five times with an automatic sleep of 1 second before every iteration. This gives the system under test up to five seconds time to store the new entry to the database. The test case is very stable and just fits to the hardware environment. On slow test environments the test may need several iterations to successfully read the database entry. On very fast environments the test may succeed right on the first try.



We changed auto sleep time from seconds to milliseconds with Citrus 2.0 release. So if you are coming from previous Citrus versions be sure to now use proper millisecond values.

So fast environments are not slowed down by static sleep operations and slower environments are still able to execute this test case with high stability.

9.7. Timer

Timers are very useful containers when you wish to execute a collection of test actions several times at regular intervals. The timer component generates an event which in turn triggers the execution of the nested test actions associated with the timer. This can be useful in a number of test scenarios for example when Citrus needs to simulate a heart beat or if you are debugging a test and you wish to query the contents of the database, to mention just a few. The following code sample should demonstrate the power and flexibility of timers:

Java

```
@CitrusTest
public void timerTest() {
    $(timer()
        .id("forkedTimer")
        .interval(100L)
        .fork(true)
        .actions(
            echo()
                .message("I'm going to run in the background and let some other test
actions run (nested action run ${forkedTimer-index} times)"),
            sleep().milliseconds(50L)
        )
    );

    $(timer()
        .repeatCount(3)
        .interval(100L)
        .delay(50L)
        .actions(
            sleep().milliseconds(50L),
            echo().message("I'm going to repeat this message 3 times before the next
test actions are executed")
        )
    );

    $(echo()
        .message("Test almost complete. Make sure all timers running in the background
are stopped")
    );

    $(doFinally().actions(
        stopTimer("forkedTimer")
    ));
}
```

XML

```
<test name="TimerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <timer id="forkedTimer" interval="100" fork="true">
            <actions>
                <echo>
                    <message>I'm going to run in the background and let some other
test actions run (nested action run ${forkedTimer-index} times)</message>
                </echo>
                <sleep milliseconds="50" />
            </actions>
        </timer>

        <timer repeatCount="3" interval="100" delay="50">
            <actions>
                <sleep milliseconds="50" />
                <echo>
                    <message>I'm going to repeat this message 3 times before the next
test actions are executed</message>
                </echo>
            </actions>
        </timer>

        <echo>
          <message>Test almost complete. Make sure all timers running in the
background are stopped</message>
        </echo>
    </actions>

    <finally>
        <stop-timer id="forkedTimer" />
    </finally>
</test>
```

YAML

```
name: TimerTest
actions:
  - timer:
      id: forkedTimer
      fork: true
      delay: 5000
      interval: 100
      repeatCount: 1
      actions:
        - echo:
            message: |
              I'm going to run in the background and let some other test actions run
(nested action run ${forkedTimer-index} times)
        - sleep:
            milliseconds: 50
  - timer:
      delay: 50
      interval: 100
      repeatCount: 3
      actions:
        - sleep:
            milliseconds: 50
        - echo:
            message: |
              I'm going to repeat this message 3 times before the next test actions
are executed
  - echo:
      message: |
        Test almost complete. Make sure all timers running in the background are
stopped
finally:
  - stop-timer:
      id: "forkedTimer"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="TimerTest">
        <actions>
            <timer id="forkedTimer" interval="100" fork="true">
              <echo>
                <message>I'm going to run in the background and let some other test
actions run (nested action run ${forkedTimer-index} times)</message>
              </echo>
              <sleep milliseconds="50" />
            </timer>

            <timer repeatCount="3" interval="100" delay="50">
              <sleep milliseconds="50" />
              <echo>
                <message>I'm going to repeat this message 3 times before the next test
actions are executed</message>
              </echo>
            </timer>

            <echo>
              <message>Test almost complete. Make sure all timers running in the
background are stopped</message>
            </echo>
        </actions>

        <finally>
            <stop-timer timerId="forkedTimer" />
        </finally>
    </testcase>
</spring:beans>
```

In the above example the first timer (timerId = forkedTimer) is started in the background. By default timers are run in the current thread of execution but to start it in the background just use "fork=true". Every 100 milliseconds this timer emits an event which will result in the nested actions being executed. The nested 'echo' action outputs the number of times this timer has already been executed. It does this with the help of an 'index' variable, in this example ${forkedTimer-index}, which is named according to the timer **id** with the suffix '-index'. No limit is set on the number of times this timer should run so it will keep on running until either a nested test action fails or it is instructed to stop (more on this below).

The second timer is configured to run 3 times with a delay of 100 milliseconds between each iteration. Using the attribute 'delay' we can get the timer pause for 50 milliseconds before running the nested actions for the first time. The timer is configured to run in the current thread of execution so the last test action, the 'echo', has to wait for this timer to complete before it is executed.

So how do we tell the forked timer to stop running? If we forget to do this the timer will just execute

indefinitely. To help us out here we can use the 'stop-timer' action. By adding this to the finally block we ensure that the timer will be stopped, even if some nested test action fails. We could have easily added it as a nested test action, to the forkedTimer for example, but if some other test action failed before the stop-timer was called, the timer would never stop.



You can also configure timers to run in the background using the 'parallel' container, rather than setting the attribute 'fork' to true. Using parallel allows more fine-grained control of the test and has the added advantage that all errors generated from a nested timer action are visible to the test executer. If an error occurs within the timer then the test status is set to failed. Using fork=true an error causes the timer to stop executing, but the test status is not influenced by this error.

9.8. Async

Now we deal with parallel execution of test actions. Nested actions inside an async container are executed in a separate thread. This allows to continue test execution without having to wait for actions inside the async container to complete. The test immediately continues to execute the next test actions, which will be executed in parallel to those actions inside the async container.

This mechanism comes in handy when a test action should be forked to the rest of the test. In send operations we were already able to achieve this by setting `fork="true"`. With async containers, we're able to execute all kinds of test actions asynchronous.

See some example to find out how it works.

Java

```
@CitrusTest
public void asyncTest() {
    $(async()
        .actions(
            send(fooEndpoint)
                .message(fooRequest()),
            receive(fooEndpoint)
                .message(fooResponse())
        )
    );

    $(echo().message("Continue with test"));
}
```

XML

```
<test name="AsyncTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <async>
            <actions>
                <send endpoint="fooEndpoint">
                    <message><!-- ... --></message>
                </send>
                <receive endpoint="fooEndpoint">
                    <message><!-- ... --></message>
                </receive>
            </actions>
        </async>

        <echo>
          <message>Continue with test</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: AsyncTest
actions:
  - async:
      actions:
        - send:
            endpoint: "fooEndpoint"
            message: {}
        - receive:
            endpoint: "fooEndpoint"
            message: {}
  - echo:
      message: "Continue with test"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AsyncTest">
        <actions>
            <async>
                <actions>
                    <send endpoint="fooEndpoint">
                        <message><!-- ... --></message>
                    </send>
                    <receive endpoint="fooEndpoint">
                        <message><!-- ... --></message>
                    </receive>
                </actions>
            </async>

            <echo>
              <message>Continue with test</message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

The nested `send` and `receive` actions get executed in parallel to the other test actions in that test case. So the test will not wait for these actions to finish before executing next actions. Of course possible errors inside the async container will also cause the whole test case to fail. And the test will definitely wait for all async actions to be finished before finishing the whole test case. This safely lets us execute test actions in parallel to each other.

The async container also supports success and error callback actions.

Java

```
@CitrusTest
public void asyncTest() {
    $(async()
        .actions(
            send(fooEndpoint)
                .message(fooRequest()),
            receive(fooEndpoint)
                .message(fooResponse())
        )
        .successActions(
            echo().message("Success!")
        )
        .errorActions(
            echo().message("Error!")
        )
    );

    $(echo().message("Continue with test"));
}
```

XML

```
<test name="AsyncTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <async>
            <actions>
                <send endpoint="fooEndpoint">
                    <message><!-- ... --></message>
                </send>
                <receive endpoint="fooEndpoint">
                    <message><!-- ... --></message>
                </receive>
            </actions>
            <success>
                <echo><message>Success!</message></echo>
            </success>
            <error>
                <echo><message>Failed!</message></echo>
            </error>
        </async>

        <echo>
          <message>Continue with test</message>
        </echo>
    </actions>
</test>
```

YAML

```
name: AsyncTest
actions:
  - async:
      actions:
        - send:
            endpoint: "fooEndpoint"
            message: {}
        - receive:
            endpoint: "fooEndpoint"
            message: {}
      success:
        - echo:
            message: "Success!"
      error:
        - echo:
            message: "Error!"
  - echo:
      message: "Continue with test"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AsyncTest">
        <actions>
            <async>
                <actions>
                    <send endpoint="fooEndpoint">
                        <message><!-- ... --></message>
                    </send>
                    <receive endpoint="fooEndpoint">
                        <message><!-- ... --></message>
                    </receive>
                </actions>
                <success>
                    <echo><message>Success!</message></echo>
                </success>
                <error>
                    <echo><message>Failed!</message></echo>
                </error>
            </async>

            <echo>
              <message>Continue with test</message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

So you can add test actions which are executed based on the async test actions outcome `success` or `error`.

If you are using this container to send or receive messages, you have to use the unique correlation ID of the message to link the actions concerning this message. Otherwise the testcase might associate a `send` or `receive` action with the wrong message. Please note that this ID is **not** provided to your system under test. The management of correlation IDs as well as the assignment to messages is done internally. Only the mapping between the request and response has to be done by the author of the test. As you can see in the following example, the value of the header `MessageHeaders.ID` is stored in the variable `request#1` respectively `request#2`. This variable is reused in the `receive` action to identify the correct response from the server.

Java

```
@CitrusTest
public void testAsync() {

    $(async()
        .actions(
            http().client(httpClient)
                    .send()
                    .post("/foo")
                    .extract(extract().fromHeaders().header(MessageHeaders.ID,
"request#1")
                    .payload("{ \"info\": \"foo\"}"),

            //SUT echoing the input

            http().client(httpClient)
                    .receive()
                    .response(HttpStatus.OK.value())
                    .payload("{ \"info\": \"foo\"}")
                    .selector(
                        Collections.singletonMap(
                            MessageHeaders.ID, "${request#1}"))
        )
    );

    $(async()
        .actions(
            http().client(httpClient)
                    .send()
                    .post("/boo")
                    .extract(extract().fromHeaders().header(MessageHeaders.ID,
"request#2")
                    .payload("{ \"info\": \"boo\"}"),

            //SUT echoing the input

            http().client(httpClient)
                    .receive()
                    .response(HttpStatus.OK.value())
                    .payload("{ \"info\": \"boo\"}")
                    .selector(
                        Collections.singletonMap(
                            MessageHeaders.ID, "${request#2}"))
        )
    );
}
```

9.9. Wait

With this action you can make your test wait until a certain condition is satisfied. The attribute **seconds** defines the amount of time to wait in seconds. You can also use the milliseconds attribute for a more fine grained time value. The attribute **interval** defines the amount of time to wait **between** each check. The interval is always specified as millisecond time interval.

If the check does not exceed within the defined overall waiting time then the test execution fails with an appropriate error message. There are different types of conditions to check.

http This condition is based on an Http request call on a server endpoint. Citrus will wait until the Http response is as defined (e.g. Http 200 OK). This is useful when you want to wait for a server to start.

file This condition checks for the existence of a file on the local file system. Citrus will wait until the file is present.

message This condition checks for the existence of a message in the local message store of the current test case. Citrus will wait until the message with the given name is present.

action This condition executes another test action and checks for successful execution. Citrus will wait until the nested action is executed without any errors.

When should somebody use this action? This action is very useful when you want your test to wait for a certain event to occur before continuing with the test execution. For example if you wish that your test waits until a Docker container is started or for an application to create a log file before continuing, then use this action. You can also create your own condition statements and bind it to the test action.

9.9.1. Http condition

Next let us have a look at a simple example:

Java

```
@CitrusTest
public void waitTest() {
    $(waitFor()
        .time(Duration.ofMillis(10000))
        .interval(2000)
        .http()
            .url("http://sample.org/resource")
            .status(200)
            .timeout(2000);
    );
}
```

XML

```
<test name="WaitTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <wait-for timeout="10" interval="2000">
          <http url="http://sample.org/resource" status="200" timeout="2000" />
        </wait-for>
    </actions>
</test>
```

YAML

```
name: WaitTest
actions:
  - wait-for:
      timeout: "10"
      interval: "2000"
      http:
        url: path/to/resource/file.txt
        status: 200
        timeout: 2000
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="WaitTest">
        <actions>
            <wait seconds="10" interval="2000">
              <http url="http://sample.org/resource" status="200" timeout="2000" />
            </wait>
        </actions>
    </testcase>
</spring:beans>
```

The example waits for some Http server resource to be available with **Http 200 OK** response. Citrus will use **HEAD** request method by default. You can set the request method with the **method** attribute on the Http condition.

9.9.2. File condition

Next let us have a look at the file condition usage:

Java

```
@CitrusTest
public void waitTest() {
    $(waitFor()
        .time(Duration.ofMillis(10000))
        .interval(2000)
        .file().path("path/to/resource/file.txt")
    );
}
```

XML

```
<test name="WaitTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <wait-for timeout="10" interval="2000">
          <file path="path/to/resource/file.txt" />
        </wait-for>
    </actions>
</test>
```

YAML

```
name: WaitTest
actions:
  - wait-for:
      timeout: "10"
      interval: "2000"
      file:
        path: path/to/resource/file.txt
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="WaitTest">
        <actions>
            <wait seconds="10" interval="2000">
              <file path="path/to/resource/file.txt" />
            </wait>
        </actions>
    </testcase>
</spring:beans>
```

Citrus checks for the file to exist under the given path. Only if the file exists the test will continue with further test actions.

9.9.3. Message condition

Next let us have a look at the message condition usage:

Java

```
@CitrusTest
public void waitTest() {
    $(waitFor()
        .time(Duration.ofMillis(10000))
        .interval(2000)
        .message().name("helloRequest")
    );
}
```

XML

```
<test name="WaitTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <wait-for timeout="10" interval="2000">
          <message name="helloRequest" />
        </wait-for>
    </actions>
</test>
```

YAML

```
name: WaitTest
actions:
  - wait-for:
      timeout: "10"
      interval: "2000"
      message:
        name: "helloRequest"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="WaitTest">
        <actions>
            <wait seconds="10" interval="2000">
              <message name="helloRequest" />
            </wait>
        </actions>
    </testcase>
</spring:beans>
```

Citrus checks for the message with the name **helloRequest** in the local message store. Only if the

message with the given name is found the test will continue with further test actions. The local message store is automatically filled with all exchanged messages (send or receive) in a test case. The message names are defined in the respective send or receive operations in the test.

9.9.4. Action condition

Now we would like to wait for some other test action to execute.

Java

```
@CitrusTest
public void waitTest() {
    $(waitFor()
        .time(Duration.ofMillis(10000))
        .interval(2000)
        .execution()
        .action(
            receive(jmsEndpoint)
                .message()
                .body("Wait for me!")
        )
    );
}
```

XML

```
<test name="WaitTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <wait-for timeout="10" interval="2000">
          <action>
            <receive endpoint="jmsEndpoint">
                <message>
                    <body><data>Wait for me!</data></body>
                </message>
            </receive>
          </action>
        </wait-for>
    </actions>
</test>
```

YAML

```
name: WaitTest
actions:
  - wait-for:
      timeout: "10"
      interval: "2000"
      action:
        receive:
          endpoint: "jmsEndpoint"
          message:
            body: |
              Wait for me!
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="WaitTest">
        <actions>
            <wait seconds="10" interval="2000">
              <action>
                <receive endpoint="jmsEndpoint">
                    <message>
                        <payload>Wait for me!</payload>
                    </message>
                </receive>
              </action>
            </wait>
        </actions>
    </testcase>
</spring:beans>
```

You can add any test action to the wait condition so you can execute any other action and wait for its success. This enables us to also use the full send and receive operations on other message transports.

9.10. Custom containers

In case you have a custom action container implementation you might also want to use it in Java DSL. The action containers are handled with special care in the Java DSL because they have nested actions. So when you call a test action container in the Java DSL you always have something like this:

Java

```
@CitrusTest
public void containerTest() {
    $(echo("This echo is outside of the action container"));

    $(sequential()
        .actions(
            echo().message("Inside"),
            echo().message("Inside once more"),
            echo().message("And again: Inside!")
        )
    );

    $(echo().message("This echo is outside of the action container"));
}
```

Now the three nested actions are added to the action **sequential** container rather than to the test case itself although we are using the same action Java DSL methods outside the container. This mechanism is only working because Citrus is handling test action containers with special care.

A custom test action container implementation could look like this:

Custom container implementation

```
public class ReverseActionContainer extends AbstractActionContainer {
    @Override
    public void doExecute(TestContext context) {
        for (int i = getActions().size(); i > 0; i--) {
            getActions().get(i-1).execute(context);
        }
    }
}
```

The container logic is very simple: The container executes the nested actions in reverse order. As already mentioned Citrus needs to take special care on all action containers when executing a Java DSL test. This is why you should not execute a custom test container implementation on your own.

Run custom container

```
@CitrusTest
public void containerTest() {
    ReverseActionContainer reverseContainer = new ReverseActionContainer();
    reverseContainer.addTestActions(
        echo().message("Foo"),
        echo().message("Bar")
    );

    $(reverseContainer);
}
```

The above custom container execution is going to fail with internal error as the Citrus Java DSL was not able to recognise the action container as it should be. Also the **EchoAction** instance creation is not very comfortable. Instead you can use a special container Java DSL syntax also with your custom container implementation:

Generic container syntax

```
@CitrusTest
public void containerTest() {
    $(container(new ReverseActionContainer())
        .actions(
            echo("Foo"),
            echo("Bar")
        )
    );
}
```

The custom container implementation now works fine with the automatically nested echo actions. And we are able to use the usual Java DSL syntactic sugar for test actions like **echo** .

In a next step we add a custom superclass for all our test classes which provides a helper method for the custom container implementation in order to have an even more comfortable syntax.

Container creating helper method

```
public static AbstractTestContainerBuilder<ReverseActionContainer> reverse() {
    return container(new ReverseActionContainer());
}
```

Now all subclasses can use the new **reverse** method for calling the custom container implementation.

Java

```
@CitrusTest
public void containerTest() {
    $(reverse()
        .actions(
            echo().message("Foo"),
            echo().message("Bar")
        )
    );
}
```

Nice! This is how we should integrate customized test action containers to the Citrus Java DSL.

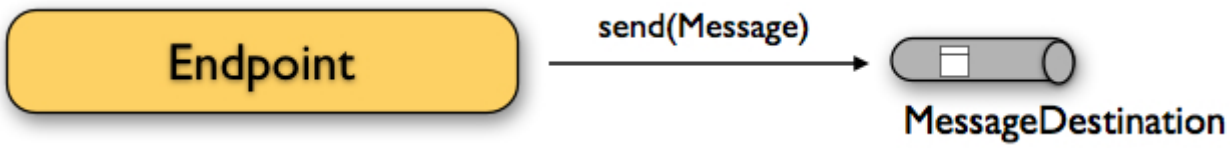
Chapter 10. Endpoints

In one of the previous chapters we have discussed the basic test case structure as we introduced **variables** and **test actions**. The <actions> section contains a list of test actions that take place during the test case. Each test action is executed in sequential order by default. Citrus offers several built-in test actions that the user can choose from to construct a complex testing workflow without having to code everything from scratch. In particular Citrus aims to provide all the test actions that you need as predefined components ready for you to use. The goal is to minimize the coding effort for you so you can concentrate on the test logic itself.

Exactly the same approach is used in Citrus to provide ready-to-use endpoint components for connecting to different message transports. There are several ways in an enterprise application to exchange messages with some other application. We have synchronous interfaces like Http and SOAP WebServices. We have asynchronous messaging with JMS or file transfer FTP interfaces.

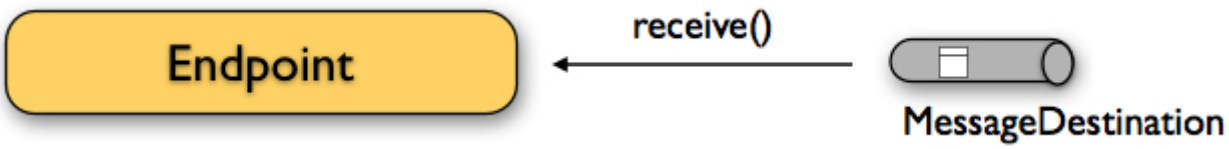
Citrus provides endpoint components as client and server to connect with these typical message transports. So you as a tester must not care about how to send a message to a JMS queue. The Citrus endpoints are configured in the Spring application context and receive endpoint specific properties like endpoint uri or ports or message timeouts as configuration.

The next figure shows a typical message sending endpoint component in Citrus:



The endpoint producer publishes messages to a destination. This destination can be a JMS queue/topic, a SOAP WebService endpoint, an Http URL, an FTP folder destination and so on. The producer just takes a previously defined message definition (header and body) and sends it to the message destination.

Similar to that Citrus defines several endpoint consumer components to consume messages from destinations. This can be a simple subscription on message channels and JMS queues/topics. In case of SOAP WebServices and Http GET/POST things are more complicated as we have to provide a server component that clients can connect to. We will handle server related communication in more detail later on. For now the endpoint consumer component in its most simple way is defined like this:



This is all you need to know about Citrus endpoints. We have mentioned that the endpoints are defined in the Spring application context. Let's have a simple example that shows the basic idea:

Citrus Bean

```
@BindToRegistry
public JmsEndpoint helloEndpoint() {
    return new JmsEndpointBuilder()
            .destinationName("Citrus.HelloService.Request.Queue")
            .connectionFactory(myConnectionFactory())
            .build();
}
```

Spring Bean

```
@Bean
public JmsEndpoint helloEndpoint() {
    return new JmsEndpointBuilder()
            .destinationName("Citrus.HelloService.Request.Queue")
            .connectionFactory(myConnectionFactory())
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jms="http://www.citrusframework.org/schema/jms/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jms/config
           http://www.citrusframework.org/schema/jms/config/citrus-jms-config.xsd">

    <citrus-jms:endpoint id="helloEndpoint"
        destination-name="Citrus.HelloService.Request.Queue"
        connection-factory="myConnectionFactory"/>
</beans>
```

This is a simple JMS endpoint component in Citrus. The endpoint XML bean definition follows a custom XML namespace and defines endpoint specific properties like the JMS destination name and the JMS connection factory. The endpoint id is a significant property as the test cases will reference this endpoint when sending and receiving messages by its identifier.

In the next sections you will learn how a test case uses those endpoint components for producing and consuming messages.

10.1. Send messages

The <send> action in a test case publishes messages to a destination. The actual message transport connection is defined with the endpoint component. The test case simply defines the message contents and references a predefined message endpoint component by its identifier. Endpoint specific configurations are centralized in the Spring bean application context while multiple test cases can reference the endpoint to actually publish the constructed message to a destination. There are several message endpoint implementations in Citrus available representing different transport protocols like JMS, SOAP, HTTP, TCP/IP and many more.

Again the type of transport to use is not specified inside the test case but in the message endpoint definition. The separation of concerns (test case/message sender transport) gives us a good flexibility of our test cases. The test case does not know anything about connection factories, queue names or endpoint uri, connection timeouts and so on. The transport internals underneath a sending test action can change easily without affecting the test case definition. We will see later in this document how to create different message endpoints for various transports in Citrus. For now we concentrate on constructing the message content to be sent.

We assume that the message's body will be plain XML format. Citrus uses XML as the default data format for message body data. But Citrus is not limited to XML message format though; you can always define other message data formats such as JSON, plain text, CSV. As XML is still a very popular message format in enterprise applications and message-based solution architectures we have this as a default format. Anyway Citrus works best on XML bodies and you will see a lot of example code in this document using XML. Finally let us have a look at a first example how a sending action is defined in the test.

Java

```
@CitrusTest
public void sendMessageTest() {
    run(send()
        .endpoint("helloEndpoint")
        .message()
            .header("Operation", "sayHello")
            .body("""
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            """)
    );
}
```

XML

```
<test name="SendMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="helloEndpoint">
        <message>
          <headers>
            <header name="Operation" value="sayHello"/>
          </headers>
          <body>
            <payload>
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            </payload>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SendMessageTest
actions:
  - send:
      endpoint: helloEndpoint
      message:
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
        headers:
          - name: Operation
            value: sayHello
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="SendMessageTest">
    <description>Basic send message example</description>
    <actions>
        <send endpoint="helloEndpoint">
            <message>
                <payload>
                    <TestMessage xmlns="http://citrusframework.org/schemas/test">
                        <Text>Hello!</Text>
                    </TestMessage>
                </payload>
            </message>
            <header>
                <element name="Operation" value="sayHello"/>
            </header>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

Now let's have a closer look at the sending action. The **'endpoint'** attribute might catch your attention first. This attribute references the message endpoint in Citrus configuration by its identifier. As previously mentioned the message endpoint definition lives in a separate configuration file and contains the actual message transport settings. In this example the **"helloEndpoint"** is referenced which is a JMS endpoint for sending out messages to a JMS queue for instance.

The test case is not aware of any transport details, because it does not have to. The advantages are obvious: On the one hand multiple test cases can reference the message endpoint definition for better reuse. Secondly test cases are independent of message transport details. So connection factories, user credentials, endpoint uri values and so on are not present in the test case.

In other words the **"endpoint"** attribute of the `<send>` element specifies which message endpoint definition to use and therefore where the message should go to. Once again all available message endpoints are configured in a separate Citrus configuration file. Be sure to always pick the right message endpoint type in order to publish your message to the right destination.

In a Java test you can choose from a set of supported runtimes. In fact, Citrus is able to run tests with popular Java testing frameworks such as JUnit Jupiter, JUnit4, TestNG or Cucumber.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class SendMessageIT extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void sendMessageTest() {
        run(send()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

JUnit

```
import org.citrusframework.TestActionRunner;
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class SendMessageIT implements TestActionSupport {

    @CitrusResource
    private TestActionRunner runner;

    @Test
    @CitrusTest
    public void sendMessageTest() {
        runner.run(send()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

JUnit4

```
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class SendMessageIT extends JUnit4CitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void sendMessageTest() {
        run(send()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

You can choose your favorite Java testing framework to run the Citrus tests. Each framework JUnit Jupiter, JUnit4, TestNG, Cucumber may require slightly different setup. Please also refer to the individual framework documentation and see how it works.



Please be aware of proper test case naming conventions when separating unit and integration tests. In Maven for example unit tests are run with the `surefire` plugin in the unit test phase. Maven surefire includes tests with a naming pattern (e.g. `*Test`). Integration tests are run with `failsafe` plugin in the Maven integration test phase and these tests follow a different naming pattern (e.g. `*IT`).



It is good practice to follow naming conventions when defining names for message endpoints. The intended purpose of the message endpoint as well as the sending/receiving actor should be clear when choosing the name. For instance messageEndpoint1, messageEndpoint2 will not give you much hints to the purpose of the message endpoint.

This is basically how to send messages in Citrus. The test case is responsible for constructing the message content while the predefined message endpoint holds transport specific settings. Test cases reference endpoint components to publish messages to the outside world. This is just the start of action. Citrus supports a whole package of other ways how to define and manipulate the message contents. Read more about message sending actions in actions-send.

10.2. Receive messages

Now we have a look at the message receiving part inside the test. A simple example shows how it works.

Java

```
@CitrusTest
public void receiveMessageTest() {
    run(receive()
        .endpoint("helloEndpoint")
        .message()
            .header("Operation", "sayHello")
            .body("""
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            """)
    );
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="helloEndpoint">
        <message>
          <headers>
            <header name="Operation" value="sayHello"/>
          </headers>
          <body>
            <payload>
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: helloEndpoint
      message:
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
        headers:
          - name: Operation
            value: sayHello
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="ReceiveMessageTest">
    <actions>
        <receive endpoint="helloEndpoint">
            <message>
                <payload>
                    <TestMessage xmlns="http://citrusframework.org/schemas/test">
                        <Text>Hello!</Text>
                    </TestMessage>
                </payload>
            </message>
            <header>
                <element name="Operation" value="sayHello"/>
            </header>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

If we recap the send action of the previous chapter we can identify some common mechanisms that apply for both sending and receiving actions. The test action also uses the **endpoint** attribute for referencing a predefined message endpoint. This time we want to receive a message from the endpoint. Again the test is not aware of the transport details such as JMS connections, endpoint uri, and so on. The message endpoint component encapsulates this information.

Before we go into detail on validating the received message we have a quick look at the different

options to run the Java test. Basically you can choose from a set of popular Java testing frameworks to actually run the Citrus test. Citrus integrates with the famous frameworks such as JUnit Jupiter, JUnit4, TestNG or Cucumber.

TestNG

```
import org.testng.annotations.Test;
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.testng.TestNGCitrusSupport;

public class ReceiveMessageIT extends TestNGCitrusSupport implements TestActionSupport
{

    @Test
    @CitrusTest
    public void receiveMessageTest() {
        run(receive()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

JUnit

```
import org.citrusframework.TestActionRunner;
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusResource;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.jupiter.CitrusSupport;
import org.junit.jupiter.api.Test;

@CitrusSupport
public class ReceiveMessageIT implements TestActionSupport {

    @CitrusResource
    private TestActionRunner runner;

    @Test
    @CitrusTest
    public void receiveMessageTest() {
        runner.run(receive()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

JUnit4

```
import org.citrusframework.TestActionSupport;
import org.citrusframework.annotations.CitrusTest;
import org.citrusframework.junit.spring.JUnit4CitrusSupport;
import org.junit.Test;

public class ReceiveMessageIT extends JUnit4CitrusSupport implements TestActionSupport
{

    @Test
    @CitrusTest
    public void receiveMessageTest() {
        run(receive()
            .endpoint("helloEndpoint")
            .message()
                .header("Operation", "sayHello")
                .body("""
                <TestMessage xmlns="http://citrusframework.org/schemas/test">
                    <Text>Hello!</Text>
                </TestMessage>
                """)
        );
    }
}
```

Whatever Java test framework you choose you may apply very specific configuration and setup details to run the test. Please also have a look into the individual Java test frameworks JUnit Jupiter, JUnit4, TestNG, Cucumber and so on to see how it works.

The `receive` action waits for a message to arrive. The whole test execution is stopped while waiting for the message. This is important to ensure the step by step test workflow processing. Of course you can specify message timeouts so the receiver will only wait a given amount of time before raising a timeout error. Following from that timeout exception the test case fails as the message did not arrive in time. Citrus defines default timeout settings for all message receiving tasks.

At this point you know the two most important test actions in Citrus. Sending and receiving actions will become the main components of your integration tests when dealing with loosely coupled message based components in an enterprise application environment. It is very easy to create complex message flows, meaning a sequence of sending and receiving actions in your test case. You can replicate use cases and test your message exchange with extended message validation capabilities. See actions-receive for a more detailed description on how to validate incoming messages and how to expect message contents in a test case.

10.3. Local message store

All messages that are sent and received during a test case are stored in a local memory storage. This is because we might want to access the message content later on in a test case. We can do so by using message store functions for loading messages that have been exchanged earlier in the test.

When storing a message in the local storage Citrus uses a message name as identifier key. This message name is later on used to access the message. You can define the message name in any send or receive action:

Java

```
@CitrusTest
public void receiveMessageTest() {
    receive("helloEndpoint")
        .message()
            .name("helloMessage")
            .header("Operation", "sayHello")
            .body("""
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            """);
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="helloEndpoint">
        <message name="helloMessage">
          <headers>
            <header name="Operation" value="sayHello"/>
          </headers>
          <body>
            <payload>
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - receive:
      endpoint: helloEndpoint
      message:
        name: helloMessage
        body:
          data: |
            <TestMessage xmlns="http://citrusframework.org/schemas/test">
                <Text>Hello!</Text>
            </TestMessage>
        headers:
          - name: Operation
            value: sayHello
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="ReceiveMessageTest">
    <actions>
        <receive endpoint="helloEndpoint">
            <message name="helloMessage">
                <payload>
                    <TestMessage xmlns="http://citrusframework.org/schemas/test">
                        <Text>Hello!</Text>
                    </TestMessage>
                </payload>
            </message>
            <header>
                <element name="Operation" value="sayHello"/>
            </header>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

The receive operation above set the message name to **helloMessage**. The message received is automatically stored in the local storage with that name. You can access the message content for instance by using a function:

Java

```
@CitrusTest
public void receiveMessageTest() {
    echo()
        .message("citrus:message(helloMessage.body())");
}
```

XML

```
<test name="ReceiveMessageTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <echo message="citrus:message(helloMessage.body())"/>
    </actions>
</test>
```

YAML

```
name: ReceiveMessageTest
actions:
  - echo:
      message: "citrus:message(helloMessage.body())"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="ReceiveMessageTest">
    <actions>
        <echo>
            <message>citrus:message(helloMessage.body())</message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

The function loads the **helloMessage** and prints the body information with the **echo** test action. In combination with Xpath or JsonPath functions this mechanism is a good way to access the exchanged message contents later in a test case.



The storage is for both sent and received messages in a test case. The storage is per test case and contains all sent and received messages.

When no explicit message name is given the local storage will construct a default message name. The default name is built from the action (send or receive) plus the endpoint used to exchange the message. For instance:

```
send(helloEndpoint)
receive(helloEndpoint)
```

The names above would be generated by a send and receive operation on the endpoint named **helloEndpoint**.



The message store is not able to handle multiple message of the same name in one test case. So messages with identical names will overwrite existing messages in the local storage.

Now we have seen the basic endpoint concept in Citrus. The endpoint components represent the connections to the test boundary systems. This is how we can connect to the system under test for message exchange. And this is our main goal with this integration test framework. We want to provide easy access to common message transports on client and server side so that we can test the communication interfaces on a real message transport exchange.

Chapter 11. Direct endpoint

Direct endpoints represent the default in memory messaging solution in Citrus. Producer and consumer components are linked via queues exchanging messages in memory.



The direct endpoint configuration components use the default "citrus" configuration namespace and schema definition. Include this namespace into your Spring bean configuration in order to use the Citrus configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as follows.

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus="http://www.citrusframework.org/schema/config"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/config
            http://www.citrusframework.org/schema/config/citrus-config.xsd">

    [...]

</beans>
```

Right now you are able to use the Citrus XML elements in order to define the direct endpoint components.

11.1. Channel endpoint

Citrus offers a direct endpoint component that is able to create producers and consumers. Producer and consumer send and receive messages both to and from a direct endpoint. By default, the endpoint is asynchronous when configured in the Citrus context.

Java

```
@Bean
public ChannelEndpoint helloEndpoint() {
    return new ChannelEndpointBuilder()
        .queue("helloChannel")
        .build();
}

@Bean
public MessageSelectingQueueChannel helloChannel() {
    return new MessageSelectingQueueChannel();
}
```

XML

```
<citrus:direct-endpoint id="helloEndpoint" queue="helloChannel"/>

<citrus:queue id="helloChannel"/>
```

The Citrus direct endpoint references a queue directly. Inside your test case you can reference the Citrus endpoint as usual to send and receive messages.

The message sender is now ready to publish messages on the defined queue. The communication is supposed to be asynchronous, so the producer is not able to process any reply message. We will deal with synchronous communication and reply messages later in this chapter. You can reference the id of the endpoint in a send and receive test action.

Java

```
when(send("helloEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(receive("helloEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<send endpoint="helloEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</send>

<receive endpoint="helloEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</receive>
```

You can send and receive messages from the same direct endpoint. As usual the receiver connects to the message destination and waits for messages to arrive. The user can set a receive timeout which is set to 5000 milliseconds by default. In case no message was received in this time frame the receiver raises timeout errors and the test fails.

11.2. Synchronous direct endpoints

The synchronous direct producer publishes messages and waits synchronously for the response to arrive on a reply queue destination. The reply queue name is set in the message headers. The counterpart in this communication must send its reply to that queue. The basic configuration for a synchronous direct endpoint component looks like follows:

Java

```
@Bean
public ChannelSyncEndpoint helloEndpoint() {
    return new ChannelSyncEndpointBuilder()
        .queue("helloChannel")
        .pollingInterval(1000)
        .build();
}
```

XML

```
<citrus:direct-sync-endpoint id="helloSyncEndpoint"
                             queue="helloChannel"
                             polling-interval="1000"/>
```

Synchronous direct endpoints usually do poll for synchronous reply messages for processing the reply messages. The poll interval is an optional setting in order to manage the amount of reply message handshake attempts. When the endpoint was able to receive the reply message synchronously the test case can verify the reply.

In case all polling attempts have failed the action raises a timeout error, and the test will fail.



By default, the direct endpoint uses temporary reply queue destinations. The temporary reply queues are only used once for a single communication handshake. The temporary reply queue is deleted automatically.

When sending a message to this endpoint in the first place the producer will wait synchronously for the response message to arrive on the reply queue. You can receive the reply message in your test case using the same endpoint component. So we have two actions on the same endpoint, first send then receive.

Java

```
when(send("helloSyncEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(receive("helloSyncEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<send endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</send>

<receive endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</receive>
```

This is how you handle synchronous communication as a sender. You publish messages to a queue and wait for reply messages on a temporary reply queue. The next section deals with the same synchronous communication, but now Citrus will receive a request and send a synchronous reply message to a temporary reply queue.

As usual the reply queue name is stored in the message headers. Citrus handles this synchronous communication with the same synchronous direct endpoint component. The handling of temporary reply destinations is done automatically behind the scenes.

So we have again two actions in our test case, but this time first receive then send.

Java

```
when(receive("helloSyncEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(send("helloSyncEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<receive endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</receive>

<send endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</send>
```

11.3. Message selectors

A queue can hold multiple messages at the same time. Usually you receive messages using first-in-first-out pattern. Message selectors enable you to select messages form that queue so you can pick messages form a queue based on a selector evaluation.

Citrus introduces a special message queue implementation that supports message selectors.

Java

```
@Bean
public MessageSelectingQueueChannel helloChannel() {
    return new MessageSelectingQueueChannel();
}
```

XML

```
<citrus:queue id="orderChannel" capacity="5"/>
```

We can add a capacity attribute for this queue. A receive test action makes use of message selectors on header values as described in message-selector.

In addition to that we have implemented other message filter possibilities on message queues that we discuss in the next sections.

11.4. Payload matching selector

You can select messages based on the payload content. Either you define the expected payload as an exact match in the selector or you make use of Citrus validation matchers which is more adequate in most scenarios.

Assume there are two different plain text messages living on a message queue waiting to be picked up by a consumer.

```
Hello, welcome!
```

```
GoodBye, see you next time!
```

The tester would like to pick up the message starting with **GoodBye** in our test case. The other messages should be left on the queue as we are not interested in it right now. We can define a payload matching selector in the receive action like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("payload", "@startsWith(GoodBye)@"))
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<receive endpoint="orderChannelEndpoint">
    <selector>
        <element name="payload" value="@startsWith(GoodBye)@"/>
    </selector>
    <message>
        <payload>GoodBye, see you next time!</payload>
    </message>
</receive>
```

The Citrus receiver picks up the **GoodBye** from the queue selected via the payload matching expression defined in the selector element. Of course, you can also combine message header selectors and payload matching selectors as shown in this example below where a message header **sequenceId** is added to the selection logic.

Java

```
Map<String, String> selectorMap = new HashMap<>();
selectorMap.put("payload", "@startsWith(GoodBye)@");
selectorMap.put("sequenceId", "1234");

when(receive("orderChannelEndpoint")
        .selector(selector)
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<selector>
    <element name="payload" value="@startsWith(GoodBye)@"/>
    <element name="sequenceId" value="1234"/>
</selector>
```

11.5. Root QName selector

As a special payload matching selector you can use the XML root QName of your message as selection criteria when dealing with XML message content. Let's see how this works in a small example:

We have two different XML messages on a message queue waiting to be picked up by a consumer.

```
<HelloMessage xmlns="http://citrusframework.org/schema">Hello Citrus</HelloMessage>
```

```
<GoodbyeMessage xmlns="http://citrusframework.org/schema">Goodbye
Citrus</GoodbyeMessage>
```

We would like to pick up the **GoodbyeMessage** in our test case. The **HelloMessage** should be left on the message queue as we are not interested in it right now. We can define a root qname message selector in the receive action like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("root-qname", "GoodbyeMessage"))
        .message()
        .body("<GoodbyeMessage xmlns=\"http://citrusframework.org/schema\">Goodbye
Citrus</GoodbyeMessage>"));
```

XML

```
<receive endpoint="orderChannelEndpoint">
    <selector>
        <element name="root-qname" value="GoodbyeMessage"/>
    </selector>
    <message>
        <payload>
            <GoodbyeMessage xmlns="http://citrusframework.org/schema">Goodbye
Citrus</GoodbyeMessage>
        </payload>
    </message>
</receive>
```

The Citrus receiver picks up the **GoodbyeMessage** from the queue selected via the root qname of the XML message payload. Of course, you can also combine message header selectors and root qname selectors as shown in this example below where a message header **sequenceId** is added to the selection logic.

Java

```
Map<String, String> selectorMap = new HashMap<>();
selectorMap.put("root-qname", "GoodbyeMessage");
selectorMap.put("sequenceId", "1234");

when(receive("orderChannelEndpoint")
        .selector(selector)
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<selector>
    <element name="root-qname" value="GoodbyeMessage"/>
    <element name="sequenceId" value="1234"/>
</selector>
```

As we deal with XML qname values, we can also use namespaces in our selector root qname selection.

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("root-qname",
"{http://citrusframework.org/schema}GoodbyeMessage"))
        .message()
        .body("<GoodbyeMessage xmlns=\"http://citrusframework.org/schema\">Goodbye
Citrus</GoodbyeMessage>"));
```

XML

```
<selector>
    <element name="root-qname"
value="{http://citrusframework.org/schema}GoodbyeMessage"/>
</selector>
```

11.6. Xpath selector

It is also possible to evaluate some XPath expression on the message payload in order to select a message from a message queue. The XPath expression outcome must match an expected value and only then the message is consumed from the queue.

The syntax for the XPath expression is to be defined as the element name like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("xpath://Order/status", "pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element name="xpath://Order/status" value="pending"/>
</selector>
```

The message selector looks for order messages with **status="pending"** in the message payload. This means that following messages would get accepted/declined by the message selector.

```
<Order><status>pending</status></Order> <!-- ACCEPTED -->
<Order><status>finished</status></Order> <!-- NOT ACCEPTED -->
```

Of course, you can also use XML namespaces in your XPath expressions when selecting messages from queues.

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("xpath://ns1:Order/ns1:status", "pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element name="xpath://ns1:Order/ns1:status" value="pending"/>
</selector>
```

Namespace prefixes must match the incoming message - otherwise the XPath expression will not work as expected. In our example the message should look like this:

```
<ns1:Order
xmlns:ns1="http://citrus.org/schema"><ns1:status>pending</ns1:status></ns1:Order>
```

Knowing the correct XML namespace prefix is not always easy. If you are not sure which namespace prefix to choose Citrus ships with a dynamic namespace replacement for XPath expressions. The XPath expression looks like this and is most flexible:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap(

"xpath://{http://citrus.org/schema}:Order/{http://citrus.org/schema}:status",
"pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element
name="xpath://{http://citrus.org/schema}:Order/{http://citrus.org/schema}:status"
                value="pending"/>
</selector>
```

This will match all incoming messages regardless the XML namespace prefix that is used.

11.7. JsonPath selector

It is also possible to evaluate some JsonPath expression on the message payload in order to select a message from a message queue. The JsonPath expression outcome must match an expected value and only then the message is consumed from the queue.

The syntax for the JsonPath expression is to be defined as the element name like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("jsonPath:$.order.status", "pending"))
        .message()
        .body("{ \"order\": { \"status\": \"pending\" } }"));
```

XML

```
<selector>
    <element name="jsonPath:$.order.status" value="pending"/>
</selector>
```

The message selector looks for order messages with **status="pending"** in the message payload. This means that following messages would get accepted/declined by the message selector.

```
{ "order": { "status": "pending" } } //ACCEPTED
{ "order": { "status": "finished" } } //NOT ACCEPTED
```

Chapter 12. JMS support

Citrus provides support for sending and receiving JMS messages. We have to separate between synchronous and asynchronous communication. So in this chapter we explain how to work with JMS message endpoints for synchronous and asynchronous communication



The JMS components in Citrus are kept in a separate Maven module. If not already done so you have to include the module as Maven dependency to your project

Maven module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-jms</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Citrus provides a "citrus-jms" configuration namespace and schema definition for JMS related components and features. Include this namespace into your Spring configuration in order to use the Citrus JMS configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as follows.

Spring configuration namespace

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-jms="http://www.citrusframework.org/schema/jms/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/jms/config
           http://www.citrusframework.org/schema/jms/config/citrus-jms-config.xsd">

    [...]

</beans>
```

Now you are able to use customized Citrus XML elements in order to define the JMS endpoint components.

12.1. JMS endpoints

By default, Citrus JMS endpoints are asynchronous. Asynchronous messaging means that the endpoint will not wait for a response message after sending a message.

The test case itself should not know about JMS transport details like queue names or connection credentials. This information is stored in the endpoint component configuration that lives in the basic project configuration files in Citrus. So let us have a look at a simple JMS message endpoint

configuration in Citrus.

Java

```
@Bean
public JmsEndpoint helloServiceEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Request.Queue")
        .timeout(10000L)
        .build();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceEndpoint"
          destination-name="Citrus.HelloService.Request.Queue"
          timeout="10000"/>
```

The endpoint component receives a unique id as well as a JMS destination name. This can be a queue or topic destination. JMS topics are described later on in this chapter. For now the timeout setting completes the first JMS endpoint component definition example.



In addition to the `destination-name` attribute you can also provide a reference to a destination implementation.

Java

```
@Bean
public JmsEndpoint helloServiceEndpoint() {
    return new JmsEndpointBuilder()
        .destination(helloServiceQueue())
        .build();
}

@Bean
public ActiveMQQueue helloServiceQueue() {
    return new ActiveMQQueue("Citrus.HelloService.Request.Queue");
}
```

XML

```
<citrus-jms:endpoint id="helloServiceEndpoint"
                     destination="helloServiceQueue"/>

<amq:queue id="helloServiceQueue" physicalName="Citrus.HelloService.Request.Queue"/>
```

The destination attribute references to a JMS destination object in the same Spring application context. In the example above we used the ActiveMQ queue destination component. The

destination reference can also refer to a JNDI lookup for instance.

The endpoint needs a JMS connection factory for connecting to a JMS message broker. The connection factory is also added as component bean to the Citrus project (e.g. in the Spring application context).

Java

```
@Bean
public ActiveMQConnectionFactory connectionFactory() {
    ActiveMQConnectionFactory factory = new ActiveMQConnectionFactory();
    factory.setBrokerURL("tcp://localhost:61616");
    return factory;
}
```

XML

```
<bean id="connectionFactory"
class="org.apache.activemq.artemis.jms.client.ActiveMQConnectionFactory">
  <property name="brokerURL" value="tcp://localhost:61616"/>
</bean>
```

The JMS connection factory receives the broker URL and is able to hold many other connection specific options. In this example we use the Apache ActiveMQ connection factory implementation as we want to use the ActiveMQ message broker. Citrus works with a bean id **connectionFactory**. All Citrus JMS component will automatically recognize this connection factory.



The configuration makes it very easy to connect to other JMS broker implementations, too (e.g. Apache ActiveMQ, TIBCO Enterprise Messaging Service, IBM Websphere MQ). Just add the required connection factory implementation as **connectionFactory** bean.



All JMS endpoint components in Citrus will automatically load the factory named **connectionFactory**. You can use the **connection-factory** endpoint attribute in order to use another connection factory instance with different bean names.

Java

```
@Bean
public JmsEndpoint helloServiceEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Request.Queue")
        .connectionFactory(myConnectionFactory())
        .build();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceEndpoint"
      destination-name="Citrus.HelloService.Request.Queue"
      connection-factory="myConnectionFactory"/>
```

As an alternative to that you may want to use a special Spring JMS template implementation as custom bean in your endpoint.

Java

```
@Bean
public JmsEndpoint helloServiceEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Request.Queue")
        .jmsTemplate(myJmsTemplate())
        .build();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceEndpoint"
              destination-name="Citrus.HelloService.Request.Queue"
              jms-template="myJmsTemplate"/>
```

The endpoint is now ready to be used inside a test case. You can send or receive messages using this endpoint. The test actions reference the JMS endpoint using its unique identifier. When sending a message the message endpoint creates a JMS message producer and will simply publish the message to the defined JMS destination. As the communication is asynchronous by default the producer does not wait for a synchronous response.

When receiving messages the endpoint creates a JMS consumer on the JMS destination. The endpoint then acts as a message driven listener. This means that the message consumer connects to the given destination and waits for messages to arrive.

Java

```
when(send("helloServiceEndpoint")
        .message()
        .body("..."));

then(receive("helloServiceEndpoint")
        .message()
        .body("..."));
```

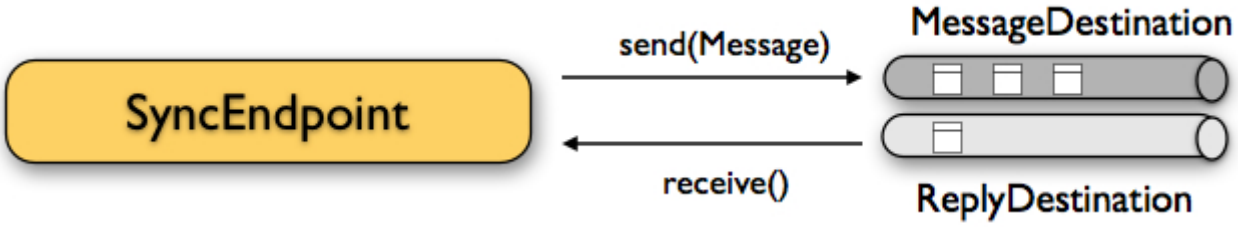
XML

```
<testcase name="jmsMessagingTest">
    <actions>
        <send endpoint="helloServiceEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </send>

        <receive endpoint="helloServiceEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </receive>
    </actions>
</testcase>
```

12.2. JMS synchronous endpoints

When using synchronous message endpoints Citrus will manage a reply destination for receiving a synchronous response message on the reply destination. The following figure illustrates that we now have two destinations in our communication scenario.



The synchronous message endpoint component is similar to the asynchronous variant that has been discussed before. The only difference is that the endpoint will automatically manage a reply destination behind the scenes. By default, Citrus uses temporary reply destinations that get automatically deleted after the communication handshake is done. Again we need to use a JMS connection factory in the configuration as the component needs to connect to a JMS message broker.

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsSyncEndpointBuilder()
        .destination("Citrus.HelloService.InOut.Queue")
        .build();
}
```

XML

```
<citrus-jms:sync-endpoint id="helloServiceSyncEndpoint"
          destination-name="Citrus.HelloService.InOut.Queue"
          timeout="10000"/>
```

The synchronous component defines a target destination which again is either a queue or topic destination. The endpoint will create the temporary reply destinations on its own. As soon as the endpoint has published a request message it waits synchronously for the response message to arrive at the reply destination. You can receive this reply message in your test case by referencing this same endpoint in a receiving test action. The timeout setting defines how long the endpoint waits for the synchronous reply. In case no reply message arrives in time a message timeout error is raised respectively.

See the following example test case which references the synchronous message endpoint in its send and receive test action in order to send out a message and wait for the synchronous response.

Java

```
when(send("helloServiceSyncEndpoint")
        .message()
        .body("..."));

then(receive("helloServiceSyncEndpoint")
        .message()
        .body("..."));
```

XML

```
<testcase name="jmsSyncMessagingTest">
    <actions>
        <send endpoint="helloServiceSyncEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </send>

        <receive endpoint="helloServiceSyncEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </receive>
    </actions>
</testcase>
```

We initiated the synchronous communication by sending a message on the synchronous endpoint. The second step then receives the synchronous message on the temporary reply destination that was automatically created for you.

If you rather want to define a static reply destination you can do so, too. The static reply destination is not deleted after the communication handshake. You may need to work with message selectors then in order to pick the right response message that belongs to a specific communication handshake. You can define a static reply destination on the synchronous endpoint component as follows.

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsSyncEndpointBuilder()
        .destination("Citrus.HelloService.InOut.Queue")
        .replyDestination("Citrus.HelloService.Reply.Queue")
        .build();
}
```

XML

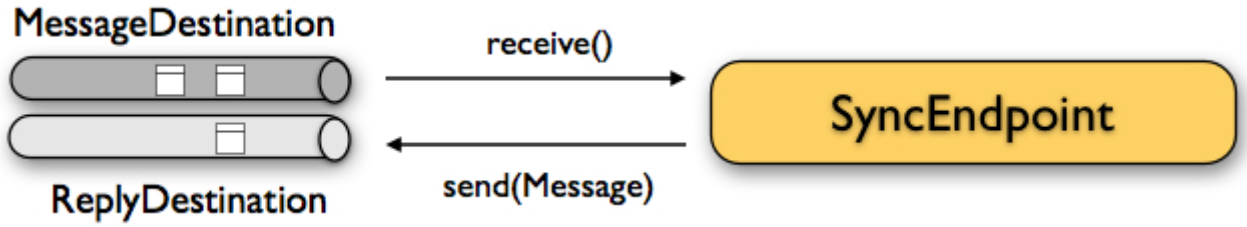
```
<citrus-jms:sync-endpoint id="helloServiceSyncEndpoint"
          destination-name="Citrus.HelloService.InOut.Queue"
          reply-destination-name="Citrus.HelloService.Reply.Queue"
          timeout="10000"/>
```

Instead of using the **reply-destination-name** feel free to use the destination reference with **reply-destination** attribute. Again you can use a JNDI lookup then to reference a destination object.



Be aware of permissions that are mandatory for creating temporary destinations. Citrus tries to create temporary queues on the JMS message broker. Following from that the Citrus JMS user has to have the permission to do so. Be sure that the user has the sufficient rights when using temporary reply destinations.

Up to now we have sent a message and waited for a synchronous response in the next step. Now it is also possible to switch the directions of send and receive actions. Then we have the situation where Citrus receives a JMS message first and then Citrus is in charge of providing a proper synchronous response message to the initial sender.



In this scenario the foreign message producer has stored a dynamic JMS reply queue destination to the JMS header. So Citrus has to send the reply message to this specific reply destination, which is dynamic of course. Fortunately the heavy lift is done with the JMS endpoint and we do not have to change anything in our configuration. Again we just define a synchronous message endpoint in the application context.

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.InOut.Queue")
        .build();
}
```

XML

```
<citrus-jms:sync-endpoint id="helloServiceSyncEndpoint"
      destination-name="Citrus.HelloService.InOut.Queue"
      timeout="10000"/>
```

Now the only thing that changes here is that we first receive a message in our test case on this endpoint. The second step is a send message action that references this same endpoint and we are done. Citrus automatically manages the reply destinations for us.

Java

```
when(receive("helloServiceSyncEndpoint")
        .message()
        .body("..."));

then(send("helloServiceSyncEndpoint")
        .message()
        .body("..."));
```

XML

```
<testcase name="jmsSyncMessagingTest">
  <actions>
        <receive endpoint="helloServiceSyncEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </receive>

        <send endpoint="helloServiceSyncEndpoint">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </send>
    </actions>
</testcase>
```

12.3. JMS topics

Up to now we have used JMS queue destinations on our endpoints. Citrus is also able to connect to JMS topic destinations. In contrary to JMS queues which represents the **point-to-point** communication JMS topics use **publish-subscribe** mechanism in order to spread messages over JMS.

A JMS topic producer publishes messages to the topic, while the topic accepts multiple message subscriptions and delivers the message to all subscribers.

The Citrus JMS endpoints offer the attribute **'pub-sub-domain'**. Once this attribute is set to **true** Citrus will use JMS topics instead of queue destinations.



When using JMS topics in your project you may want to configure a `jakarta.jms.TopicConnectionFactory` instead of a `jakarta.jms.QueueConnectionFactory`.

See the following example where the publish-subscribe attribute is set to true in JMS message endpoint components.

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Topic")
        .pubSubDomain(true)
        .build();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceTopicEndpoint"
            destination="Citrus.HelloService.Topic"
            pub-sub-domain="true"/>
----
```

When using JMS topics you will be able to subscribe several test actions to the topic destination and receive a message multiple times as all subscribers will receive the message. Also other applications besides Citrus are also able to consume messages with a topic subscription. This allows Citrus and other software components to coexist in a test environment.

12.3.1. JMS topic subscriber

By default, Citrus does not deal with durable subscribers when using JMS topics. This means that messages that were sent in advance to the message subscription are not delivered to the Citrus message endpoint. Following from that racing conditions may cause problems when using JMS topic endpoints in Citrus.

Be sure to start the Citrus subscription before messages are sent to the topic. Otherwise, you may lose some messages that were sent in advance to the subscription. By default Citrus will use a subscription per receive action using the JMS endpoint in the test cases. This means that the topic subscription is started and stopped per receive action when the action is performed inside a test case.

In order to solve racing conditions for messages that are sent prior to the subscription you can also use an `auto-start` setting on the JMS endpoint component. This causes Citrus to start/stop the subscription based on the endpoint lifecycle instead of linking the subscription to the receive action. When the endpoint is ready the subscription is started and all incoming message events are cached and stored to an internal in-memory message channel for later consumption in the tests.

Here is the endpoint configuration with `auto-start` enabled.

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Topic")
        .pubSubDomain(true)
        .autoStart(true)
        .build();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceTopicEndpoint"
            destination="helloServiceTopic"
            pub-sub-domain="true"
            auto-start="true"/>
```



The `auto-start` option is only valid in combination with `pub-sub-domain` enabled. Other combinations may be ignored or lead to configuration failure at start-up.

Now with `auto-start` set to `true` the Citrus JMS endpoint will setup a subscription at the very beginning when the endpoint is loaded in the project. The internal message channel name is derived from the JMS endpoint id and follows the pattern:

`{citrus-jms:endpoint@id}":subscriber.inbound"`

The in-memory channel id is the combined result of the JMS endpoint id and the postfix `:subscriber.inbound`. In our example this would be `helloServiceTopicEndpoint:subscriber.inbound`. Now all messages sent to the topic in advance to the tests are cached and ready for consumption and verification in the test.

In the test nothing really changes for you. You simply use a receive test action on the JMS endpoint as you would have done before. In the background Citrus will automatically receive the messages from the in memory cache. This mechanism enables us to not loose any messages that were sent to the topic in prior to Citrus firing up the test cases.



There is a small downside of the `auto-start` topic subscriber. As incoming events are cached internally you will not be able to receive the same topic event in multiple receive actions within the Citrus project. If you need to receive the topic message in several places within Citrus you need to set up several JMS topic endpoints with `auto-start` enabled. In case you just have one receive action at a time you are good to go with the `auto-start` subscriber as it is described here.

12.4. JMS topic durable subscription

When using durable subscriptions on JMS message brokers the message events on a topic are preserved for a subscriber even if the subscriber is inactive. This means that the subscriber may

not loose any message events on that particular topic as the subscription is durable and all events are stored for later consumption.

In case you want to activate durable subscriptions on the Citrus JMS endpoint use the `durable-subscription` setting in the configuration:

Java

```
@Bean
public JmsSyncEndpoint helloServiceSyncEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Topic")
        .pubSubDomain(true)
        .durableSubscription(true)
        .autoStart(true)
        .build();
}

@Bean SingleConnectionFactory topicConnectionFactory() {
    ActiveMQConnectionFactory factory = new ActiveMQConnectionFactory();
    factory.setBrokerURL("tcp://localhost:61616");
    factory.setClientID("citrusDurableConnectionFactory");
    factory.setWatchTopicAdvisories(false);

    return new SingleConnectionFactory(factory);
}
```

XML

```
<citrus-jms:endpoint id="helloServiceTopicEndpoint"
            connection-factory="topicConnectionFactory"
            destination="helloServiceTopic"
            pub-sub-domain="true"
            durable-subscription="true"
            auto-start="true"/>

<bean class="org.apache.activemq.artemis.core.server.embedded.EmbeddedActiveMQ" init-
method="start" destroy-method="stop">
  <property name="SecurityManager" ref="securityManager"/>
</bean>

<bean id="topicConnectionFactory"
class="org.springframework.jms.connection.SingleConnectionFactory">
  <constructor-arg>
    <bean
class="org.apache.activemq.artemis.jms.client.ActiveMQTopicConnectionFactory">
      <property name="brokerURL" value="tcp://localhost:61616"/>
      <property name="clientID" value="citrusDurableConnectionFactory"/>
    </bean>
  </constructor-arg>
</bean>

<bean id="securityManager"
class="org.apache.activemq.artemis.spi.core.security.ActiveMQJAASSecurityManager">
  <constructor-arg
value="org.apache.activemq.artemis.spi.core.security.jaas.InVMLoginModule"/>
  <constructor-arg>
    <bean class="org.apache.activemq.artemis.core.config.impl.SecurityConfiguration">
      <constructor-arg name="users">
        <map>
          <entry key="citrus" value="citrus"/>
        </map>
      </constructor-arg>
      <constructor-arg name="roles">
        <map>
          <entry key="citrus">
            <list>
              <value>citrus</value>
            </list>
          </entry>
        </map>
      </constructor-arg>
      <property name="DefaultUser" value="citrus"/>
    </bean>
  </constructor-arg>
</bean>
```

The durable subscription in Citrus implies that the subscriber is started when the endpoint configuration is done. All messages received on that subscription are cached internally until the receive action in the test case is performed for actual message consumption. The `auto-start` setting is required to be enabled for this reason when using durable subscriptions.

By default, Citrus is using the JMS endpoint subscriber name as durable subscription name (e.g. **helloServiceTopicEndpoint:subscriber**). You can overwrite the durable subscriber name with `durable-subscriber-name` setting on the endpoint.

In addition to that you need to add a client id on the connection factory so the message broker is able to identify the durable subscription with the client address. Also we use the `SingleConnectionFactory` implementation of Spring as a connection factory wrapper so we do not fail because of multiple connections with the same durable subscriber id.

12.5. JMS topic purging

As mentioned earlier, topic subscribers with `autoStart=true` consume all messages sent to the topic during the test run. Durable topic subscribers even consume messages sent outside the test run. Both cache the consumed messages internally in an in-memory-message-queue. To ensure that tests start with a clean in-memory-message-queue, it is necessary to purge the topic-subscribtion JmsEndpoint (which is defined in the code examples above) before the test begins. Endpoint-purging is explained in detail in this section: Purging Endpoints.

12.6. JMS message headers

The JMS specification defines a set of special message header entries that can go into your JMS message. These JMS headers are stored differently in a JMS message header than other custom header entries do. This is why these special header values should be set in a special syntax that we discuss in the next paragraphs.

Java

```
when(receive("helloServiceSyncEndpoint")
        .message()
        .header("citrus_jms_correlationId", "${correlationId}")
        .header("citrus_jms_messageId", "${messageId}")
        .header("citrus_jms_redelivered", "${redelivered}")
        .header("citrus_jms_timestamp", "${timestamp}")
        .body("..."));
```

XML

```
<header>
    <element name="citrus_jms_correlationId" value="${correlationId}"/>
    <element name="citrus_jms_messageId" value="${messageId}"/>
    <element name="citrus_jms_redelivered" value="${redelivered}"/>
    <element name="citrus_jms_timestamp" value="${timestamp}"/>
</header>
```

As you see all JMS specific message headers use the `citrus_jms_` prefix. This prefix comes from Spring Integration message header mappers that take care of setting those headers in the JMS message header properly.

Typing of message header entries may also be of interest in order to meet the JMS standards of typed message headers. For instance the following message header is of type double and is therefore transferred via JMS as a double value.

Java

```
when(receive("jmsEndpoint")
        .message()
        .header("amount", 19.75D)
        .body("..."));
```

XML

```
<header>
    <element name="amount" value="19.75" type="double"/>
</header>
```

12.7. Dynamic destination names

Usually you set the target destination as property on the JMS endpoint component. In some cases it might be useful to set the target destination in a more dynamic way during the test run. You can do this by adding a special message header named **citrus_jms_destination_name**. This header is automatically interpreted by the Citrus JMS endpoint and is set as the target destination before a message is sent.

Java

```
when(send("jmsEndpoint")
        .message()
        .header("citrus_jms_destination_name", "dynamic.destination.name")
        .body("..."));
```

XML

```
<send endpoint="jmsEndpoint">
    <message>
        ...
    </message>
    <header>
        <element name="citrus_jms_destination_name" value="dynamic.destination.name"/>
    </header>
</send>
```

This action above will send the message to the destination "*dynamic.destination.name*" no matter

what default destination is set on the referenced endpoint component named *jmsEndpoint*. The dynamic destination name setting also supports test variables. This means you can use variables and functions in the destination name, too.

Another possibility for dynamic JMS destinations is given with the dynamic endpoints.

12.8. SOAP over JMS

When sending SOAP messages you have to deal with proper envelope, body and header construction. In Citrus you can add a special message converter that performs the heavy lift for you. Just add the message converter to the JMS endpoint as shown in the next program listing:

Java

```
@Bean
public JmsSyncEndpoint helloServiceSoapJmsEndpoint() {
    return new JmsEndpointBuilder()
        .destination("Citrus.HelloService.Request.Queue")
        .messageConverter(soapJmsMessageConverter())
        .build();
}

@Bean
public SoapJmsMessageConverter soapJmsMessageConverter() {
    return new SoapJmsMessageConverter();
}
```

XML

```
<citrus-jms:endpoint id="helloServiceSoapJmsEndpoint"
        destination-name="Citrus.HelloService.Request.Queue"
        message-converter="soapJmsMessageConverter"/>

<bean id="soapJmsMessageConverter"
class="org.citrusframework.jms.message.SoapJmsMessageConverter"/>
```

With this message converter you can skip the SOAP envelope completely in your test case. You just deal with the message body payload and the header entries. The rest is done by the message converter. So you get proper SOAP messages on the producer and consumer side.

Chapter 13. Apache Kafka Support

Kafka is a distributed streaming platform that enables you to publish and subscribe to streams of records, similar to a message queue or enterprise messaging system. Citrus provides support for publishing/consuming records to/from a Kafka topic. Citrus acts as producer or consumer as the Citrus Kafka endpoint can be used bidirectional. In the current version Citrus supports asynchronous communication only.



The Kafka components in Citrus are shipped in a separate Maven module. If not already done so you have to include the module as Maven dependency to your project

Maven dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-kafka</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

In case you are using XML Spring configuration files Citrus provides a "citrus-kafka" configuration namespace and schema definition for Kafka related components and features. Include this namespace into your Spring XML configuration in order to use the Citrus Kafka configuration elements. The namespace URI and schema location are added to the Spring bean root element.

Spring bean configuration namespace

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-kafka="http://www.citrusframework.org/schema/kafka/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/kafka/config
           http://www.citrusframework.org/schema/kafka/config/citrus-kafka-
config.xsd">

    [...]

</beans>
```

Now you are able to use customized Citrus XML elements in order to define the Kafka endpoint components. In case you are using the Spring Java configuration with `@Bean` annotations you do not require this step.

13.1. Kafka Endpoint

By default, Citrus Kafka endpoints are asynchronous. Asynchronous messaging means that the endpoint will not wait for any response message after sending or receiving a message.

The endpoint component configuration holds transport specific configuration details such as topic names and server connectivity settings. So let us have a look at a simple Kafka message endpoint configuration in Citrus.

Java

```
@Bean
public KafkaEndpoint helloKafkaEndpoint() {
    return new KafkaEndpointBuilder()
                    .topic("hello")
                    .server("localhost:9092")
                    .build();
}
```

XML

```
<citrus-kafka:endpoint id="helloKafkaEndpoint"
          topic="hello"
          server="localhost:9092"/>
```

The endpoint component receives a unique id as well as a Kafka topic name. The bootstrap server url that points to the Kafka message brokers completes our first Kafka endpoint component definition. With this configuration you will be able to send and receive records on the given topic.

By default, the endpoint uses the topic partition `0`. The consumer on this endpoint is automatically added to a consumer group `citrus_kafka_group`. You can customize these settings on the endpoint.

Java

```
@Bean
public KafkaEndpoint helloKafkaEndpoint() {
    return new KafkaEndpointBuilder()
                    .topic("hello")
                    .server("localhost:9092")
                    .partition(1)
                    .consumerGroup("citrus_group")
                    .build();
}
```

XML

```
<citrus-kafka:endpoint id="helloKafkaEndpoint"
          topic="hello"
          server="localhost:9092"
          partition="1"
          consumer-group="citrus_group"/>
```

The endpoint is now ready to be used inside a test case. The test simply references the endpoint by its name when sending or receiving.



The `org.apache.kafka.clients.consumer.KafkaConsumer<K,V>` is **not thread-safe**. When using selective message consumption, it is recommended to configure `useThreadSafeConsumer` respectively `thread-safe-consumer` for the Kafka endpoint. Otherwise, you cannot execute tests in parallel.



For effective message consumption, it is additionally advisable to use random consumer groups. Both the Java and XML DSL support random consumer groups, when enabled: `randomConsumerGroup(true)` or `random-consumer-group="true"`.

In case of a send operation the endpoint creates a Kafka producer and will simply publish the records to the defined Kafka topic. As the communication is asynchronous by default, the producer does not wait for a response.

In case of a receive operation the endpoint creates a Kafka consumer instance in the defined (possibly random) consumer group.



By default, the consumer starts a subscription on the given topic and acts as a polling listener. This means that the message consumer connects to the given topic and polls one single record at a time. The action is the ready to perform the validation process that verifies the record, once it has been received. This approach requires looping to find specific Kafka messages, which can be highly inefficient, especially when the Kafka topic experiences high traffic. See Kafka message selectors for a more effective Kafka message selection.

13.1.1. Configuration

The following table shows all available settings on a Kafka endpoint in Citrus:

Property	Mandatory	Default	Description
`id`	Yes	-	Identifying name of the endpoint. Only required for XML configuration.

Property	Mandatory	Default	Description
`topic`	No	-	Default topic to use with this endpoint. Multiple topics are supported by using a comma delimited list of names (e.g. `topic1,topic2,topicN`). If not specified the test case send operation needs to set the topic as message header information.
`server`	No	`localhost:9092`	A comma delimited list of host/port pairs to use for establishing the initial connection to the Kafka cluster. Usually it is only required to connect to one Kafka server instance in the cluster. Kafka then makes sure that the endpoint is automatically introduced to all other servers in the cluster. This list only impacts the initial hosts used to discover the full set of servers.
`timeout`	No	`5000`	Timeout in milliseconds. For producers the timeout is set as time to wait for the message to be accepted by the cluster. For consumers the timeout is used for polling records on a specific topic.
`message-converter`	No	`org.citrusframework.kafka.message.KafkaMessageConverter`	Converter maps internal Citrus message objects to `ProducerRecord`/`ConsumerRecord` objects. The converter implementation takes care of message key, value, timestamp and special message headers.
`header-mapper`	No	`org.citrusframework.kafka.message.KafkaMessageHeaderMapper`	Header mapper maps Kafka record information (e.g. topic name, timestamp, message key) to internal message headers (`org.citrusframework.kafka.message.KafkaMessageHeaders`) and vice versa.
`auto-commit`	No	`true`	When this setting is enabled the consumer will automatically commit consumed records so the offset pointer on the Kafka topic is set to the next record.
`auto-commit-interval`	No	`1000`	Interval in milliseconds the auto commit operation on consumed records is performed.

Property	Mandatory	Default	Description
offset-reset	No	`earliest`	When consuming records from a topic partition and the current offset does not exist on that partition Kafka will automatically seek to a valid offset position on that partition. The `offset-reset` setting sets where to find the new position (`latest`, `earliest`, `none`). If `none` is set the consumer will receive an exception instead of resetting the offset to a valid position.
`partition`	No	`0`	Partition id that the consumer will be assigned to.
`consumer-group`	No	`citrus_kafka_group`	Consumer group name. Please keep in mind that records are load balanced across consumer instances with the same consumer group name set. So you might run into message timeouts when using multiple Kafka endpoints with the same consumer group name.
`random-consumer-group`	No	`false`	Whether to use random consumer group names. Note that these will all be prefixed by `citrus_kafka_` and end with a random 10 characters alphabetic suffix.
`key-serializer`	No	`org.apache.kafka.common.serialization.StringSerializer`	Serializer implementation that converts message key values. By default, keys are serialized to String values.
`key-deserializer`	No	`org.apache.kafka.common.serialization.StringDeserializer`	Deserializer implementation that converts message key values. By default, keys are deserialized as String values.
`value-serializer`	No	`org.apache.kafka.common.serialization.StringSerializer`	Serializer implementation that converts record values. By default values are serialized to String values.
`value-deserializer`	No	`org.apache.kafka.common.serialization.StringDeserializer`	Deserializer implementation that converts record values. By default, values are deserialized as String values.

Property	Mandatory	Default	Description
client-id	No	citrus_kafka_[producer/consumer]_{randomUUID}	An id string to pass to the server when producing/consuming records. Used as logical application name to be included in server-side request logging.
consumer-properties	No	-	Map of consumer property settings to apply to the Kafka consumer configuration. This enables you to overwrite any consumer setting with respective property key value pairs.
producer-properties	No	-	Map of producer property settings to apply to the Kafka producer configuration. This enables you to overwrite any producer setting with respective property key value pairs.
thread-safe-consumer	No	false	Whether to use separate org.apache.kafka.clients.consumer.KafkaConsumer<K,V>'s per thread. Required for parallel test execution, because the consumer is **not thread-safe**.

13.1.2. Producer and Consumer Properties

The Citrus Kafka endpoint component is also able to receive a map of Kafka producer and consumer properties. These property settings overwrite any predefined setting on the producer/consumer instance created by the endpoint. You can use the Kafka property keys with respective values for producer and consumer config maps.

Java

```
@Bean
public KafkaEndpoint helloKafkaEndpoint() {
    return new KafkaEndpointBuilder()
                    .consumerProperties(getConsumerProps())
                    .producerProperties(getProducerProps())
                    .build();
}

private Map<String, Object> getProducerProps() {
    // ...
}

private Map<String, Object> getConsumerProps() {
    // ...
}
```

XML

```
<citrus-kafka:endpoint id="helloKafkaEndpoint"
                               consumer-properties="consumerProps"
                               producer-properties="producerProps"/>


<util:map id="producerProps">
  <entry key="bootstrap.servers" value="localhost:9093,localhost:9094"/>
  <entry key="retries" value="10" value-type="java.lang.Integer"/>
  <entry key="max.request.size" value="1024" value-type="java.lang.Integer"/>
  <entry key="ssl.keystore.location" value="/path/to/keystore.jks"/>
  <entry key="ssl.kestore.password" value="secr3t"/>
</util:map>

<util:map id="consumerProps">
  <entry key="bootstrap.servers" value="localhost:9093,localhost:9094"/>
  <entry key="session.timeout.ms" value="10000" value-type="java.lang.Integer"/>
  <entry key="enable.auto.commit" value="true" value-type="java.lang.Boolean"/>
  <entry key="ssl.truststore.location" value="/path/to/truststore.jks"/>
  <entry key="ssl.truststore.password" value="secr3t"/>
</util:map>
```

13.2. Kafka Synchronous Endpoints

Not implemented yet.

13.3. Kafka Message Headers

The Kafka Citrus integration defines a set of special message header entries that are either used to

manipulate the endpoint behavior or as validation object. These Kafka specific headers are stored with a header key prefix `citrus_kafka_*`. You can set or verify those headers in send and receive actions as follows:

Java

```
send(helloKafkaEndpoint)
    .message()
    .header(KafkaMessageHeaders.TOPIC, "my.very.special.topic")
    .header(KafkaMessageHeaders.MESSAGE_KEY, "myKey")
    .header(KafkaMessageHeaders.PARTITION, 1);
```

XML

```
<header>
    <element name="citrus_kafka_topic" value="my.very.special.topic"/>
    <element name="citrus_kafka_messageKey" value="myKey"/>
    <element name="citrus_kafka_partition" value="1" />
</header>
```

The header entries above are used in a send operation in order to overwrite the topic destination, to set the record key and to specify the target partition of the producer record. These settings do only apply for the very specific send operation. Default values on the Kafka endpoint are overwritten respectively.



Typing of message header entries may also be of interest in order to meet the Kafka standards. For instance the following message key is of type `java.lang.Integer` and is therefore transferred via Kafka's key-serializer as an integer value. You need to set the header type to `integer` and use a `org.apache.kafka.common.serialization.IntegerSerializer` as key-serializer on the Kafka endpoint configuration.

Java

```
send(helloKafkaEndpoint)
    .message()
    .header(KafkaMessageHeaders.MESSAGE_KEY, 1L);
```

XML

```
<header>
    <element name="citrus_kafka_messageKey" value="1" type="integer"/>
</header>
```

In case of a receiving operation message headers are valuable validation objects that can be used to verify the message content with an expected behavior.

Java

```
receive(helloKafkaEndpoint)
    .message()
    .header(KafkaMessageHeaders.TIMESTAMP, Matchers.greaterThan(0))
    .header(KafkaMessageHeaders.TOPIC, "my.expected.topic")
    .header(KafkaMessageHeaders.MESSAGE_KEY, "myKey")
    .header(KafkaMessageHeaders.PARTITION, 1)
    .header(KafkaMessageHeaders.OFFSET, Matchers.greaterThanOrEqualTo(0));
```

XML

```
<header>
    <element name="citrus_kafka_timestamp" value="@assertThat(greaterThan(0))@"/>
    <element name="citrus_kafka_topic" value="my.expected.topic"/>
    <element name="citrus_kafka_messageKey" value="myKey"/>
    <element name="citrus_kafka_partition" value="1"/>
    <element name="citrus_kafka_offset"
value="@assertThat(greaterThanOrEqualTo(0))@"/>
</header>
```

These are the available Kafka message headers in Citrus:

Header	Name	Type	Description
KafkaMessageHeaders.TIMESTAMP	citrus_kafka_timestamp	java.lang.Long	Record timestamp value
KafkaMessageHeaders.TOPIC	citrus_kafka_topic	java.lang.String	Topic name
KafkaMessageHeaders.MESSAGE_KEY	citrus_kafka_messageKey	java.lang.Object	Record key
KafkaMessageHeaders.PARTITION	citrus_kafka_partition	java.lang.Integer	Topic partition id
KafkaMessageHeaders.OFFSET	citrus_kafka_offset	java.lang.Long	Record offset on partition

13.4. Kafka Message

Citrus also provides a Kafka message implementation that you can use on any send and receive operation. This enables you to set special message headers in a more comfortable way when using the Java fluent API:

Use message objects

```
send(helloKafkaEndpoint)
    .message(new KafkaMessage("sayHello")
                    .topic("my.very.special.topic")
                    .messageKey("myKey")
                    .partition(1));
```

The message implementation provides fluent API builder methods for each Kafka specific header.

Additionally, when receiving messages, you might want to use Kafka message selectors.

13.5. Kafka Message Selector

The Kafka Message Selector feature allows you to selectively receive messages from a Kafka topic based on specific criteria. This powerful functionality enables you to filter Kafka messages by different criteria, e.g. based on headers or record keys. Additionally, the defined time window for message retrieval significantly improves the performance. Imagine a large Kafka topic with thousands of events. Looking through all of these would require an immense amount of resources and time. Instead, selective message consumption starts at an offset `Ox = OT-n`. Where `T` is the current timestamp and `n` is the maximum timespan in which the wanted event is expected to have been published.

13.5.1. Basic Usage

The Kafka Message Selector can be used in various ways, depending on your preferred syntax and test framework.

Java

```
then(
    receive(kafkaEndpoint)
        .selector(
            kafkaMessageFilter()
                .eventLookbackWindow(Duration.ofSeconds(1L))
                .kafkaMessageSelector(kafkaHeaderEquals("key", "value"))
                .build()
        )
);
```

Java 2

```
then(
    kafkaEndpoint.findKafkaEventHeaderEquals(Duration.ofSeconds(1L), "key", "value")
);
```

XML

```
<receive endpoint="helloKafkaEndpoint">
    <description>Receive selective Kafka message</description>
    <selector>
      <element name="header-filter-key" value="key"/>
      <element name="header-filter-value" value="value"/>
      <element name="event-lookback-window" value="PT1S"/>
    </selector>
</receive>
```

13.5.2. Configuration

Java DSL	XML DSL	Description
`eventLookbackWindow`	`event-lookback-window`	This defines how far back in time the selector should search for messages. When using XML configuration, the event lookback window must be specified as an ISO-8601 duration string. For example, `PT1S` represents a duration of 1 second.
`kafkaMessageSelector`	See Selector Types	This specifies the criteria for message selection. In the examples, we're using `kafkaHeaderEquals("key", "value")`, which selects messages where a header with the `key` "key" exactly matches the `value` "value".
`pollTimeout`	`poll-timeout`	The timeout duration for each poll operation when consuming messages from Kafka. This value determines how long the consumer will wait for new records in each poll cycle. It is not the overall receive action timeout! When using XML configuration, the poll timeout must be specified as an ISO-8601 duration string. For example, `PT0.100S` represents a duration of 1 millisecond.

13.5.3. Selector Types

Message Header

The framework provides two main types of message header selectors. From within the Java DSL, these two can be easily invoked using statically provided methods:

1. `kafkaHeaderEquals`: Matches messages where the specified header `key` exactly equals the given `value`.
2. `kafkaHeaderContains`: Matches messages where the specified header `key` contains the given `value` as a substring.

More advanced users might want to do pre- or suffix matching. That is also possible.

Java

```
then(
    receive(kafkaWithRandomConsumerGroupEndpoint)
        .selector(
            kafkaMessageFilter()
                .eventLookbackWindow(Duration.ofSeconds(1L))
                .kafkaMessageSelector(
                    KafkaMessageByHeaderSelector.builder()
                        .key("key")
                        .value("prefix")
                        .matchingMechanism(STARTS_WITH)
                        .build()
                )
                .build()
        )
);
```

Note that if the specified `key` is `null`, all headers in the record will be matched against the `value`. If the `value` is `null` however, all headers with the exact `key` match.

Java DSL	XML DSL	Description
`key`	`header-filter-key`	Key-filter being applied to Kafka messages. Matches exact if specified, all keys if `null` or empty.
`value`	`header-filter-value`	Value-filter being applied to Kafka messages. Matches all values if `null` or empty. Otherwise matches as specified by strategy.

Java DSL	XML DSL	Description
`valueMatchingStrategy`	`header-filter-comparator`	Specifies how the `value` is being matched. Must be one of `EQUALS`, `CONTAINS`, `STARTS_WITH` or `ENDS_WITH`. It defaults to `EQUALS`, if not specified.

Message Key

The framework also provides selectors that filter on the Kafka record `key()`. From within the Java DSL, these can be invoked using statically provided methods:

1. `kafkaKeyEquals`: Matches messages where the record key exactly equals the given value.
2. `kafkaKeyContains`: Matches messages where the record key contains the given value as a substring.

More advanced users might want to do pre- or suffix matching. That is also possible.

Java

```
then(
    receive(kafkaEndpoint)
        .selector(
            kafkaMessageFilter()
                .eventLookbackWindow(Duration.ofSeconds(1L))
                .kafkaMessageSelector(kafkaKeyEquals("order-42"))
                .build()
        )
);
```

Java 2

```
then(
    kafkaEndpoint.findKafkaEventKeyEquals(Duration.ofSeconds(1L), "order-42")
);
```

XML

```
<receive endpoint="helloKafkaEndpoint">
    <description>Receive Kafka message by record key</description>
    <selector>
      <element name="key-filter-value" value="order-42"/>
      <element name="event-lookback-window" value="PT1S"/>
    </selector>
</receive>
```

`citrus_kafka_messageKey` is accepted as a shorthand for `key-filter-value` and defaults to `EQUALS` matching:

XML

```
<receive endpoint="helloKafkaEndpoint">
    <selector>
      <element name="citrus_kafka_messageKey" value="order-42"/>
      <element name="event-lookback-window" value="PT1S"/>
    </selector>
</receive>
```

Java DSL	XML DSL	Description
	`key`	
`key-filter-value` or `citrus_kafka_messageKey`		Value-filter being applied to the Kafka record key. Compared against `String.valueOf(record.key())`. `citrus_kafka_messageKey` is a shorthand and always uses `EQUALS`.
`valueMatchingStrategy`	`key-filter-comparator`	Specifies how the record key is being matched. Must be one of `EQUALS`, `CONTAINS`, `STARTS_WITH` or `ENDS_WITH`. It defaults to `EQUALS`, if not specified.

Custom Selector Strategy

Header-based and record-key selectors are built into Citrus and live side by side under Selector Types. Use `kafkaHeaderEquals` / `kafkaHeaderContains` or `kafkaKeyEquals` / `kafkaKeyContains` for those cases. A custom strategy is only needed when you want a predicate those built-ins do not cover.

In addition to the default Kafka message selection strategies in Citrus, it additionally allows you to define custom message selectors.

A custom selector strategy consists of two parts:

- A **predicate** to determine if a custom selector applies.
- A **constructor function** that returns the appropriate `KafkaMessageSelector`.

You register your strategy using the `KafkaMessageSelectorFactory#setCustomStrategies` method, typically within your test setup:

Java

```
kafkaWithRandomConsumerGroupEndpoint.getEndpointConfiguration()
    .getKafkaMessageSelectorFactory()
    .setCustomStrategies(
        factoryWithKafkaMessageSelector(
            messageSelectors -> messageSelectors.containsKey(TOPIC_FILTER_KEY),
            messageSelectors -> new KafkaMessageByTopicSelector(
                (String) messageSelectors.get(TOPIC_FILTER_KEY)
            )
        )
    );
```

Here, the `TOPIC_FILTER_KEY` is a custom key used to recognize and extract the expected Kafka topic from the selector.

The `KafkaMessageByTopicSelector` is an implementation of `KafkaMessageSelector` and matches messages by topic name:

Java

```
record KafkaMessageByTopicSelector(String topic) implements
KafkaMessageSelector<String> {
    static final String TOPIC_FILTER_KEY = "topic-filter";

    @Override
    public boolean matches(ConsumerRecord<Object, Object> consumerRecord) {
        return nonNull(consumerRecord.topic()) &&
consumerRecord.topic().equals(topic);
    }

    @Override
    public Map<String, String> asSelector() {
        return Map.of(TOPIC_FILTER_KEY, topic);
    }
}
```

- The `matches` method defines the filter logic.
- The `asSelector` method returns a key-value map representation of the selector.

13.5.4. Best Practices

Set Appropriate Lookback Window: Choose a lookback window that balances between finding the desired message and performance. A larger window might find older messages but could impact performance.

Combine with Other Citrus Features: The Kafka Message Selector can be combined with other Citrus testing features for comprehensive Kafka integration testing.

13.6. Dynamic Kafka Endpoints

As we have seen before the topic name can be overwritten in each send and receive operation by specifying the `citrus_kafka_topic` message header. In addition to that you can make use of completely dynamic Kafka endpoints, too.

By default, dynamic Kafka endpoints automatically use a unique consumer group for each endpoint created via the `kafka:` URI scheme. This prevents multiple dynamic endpoints from sharing the same consumer group and accidentally stealing messages from each other (point-to-point semantics). The consumer group name follows the pattern `citrus_kafka_group_{index}` where `{index}` is an auto-incrementing counter.

You can disable this behavior by setting the `citrus.kafka.dynamic.consumer.group` property to `false` (see Kafka Settings). When disabled, dynamic endpoints fall back to the default consumer group `citrus_kafka_group`. You can always override the consumer group on a per-endpoint basis by specifying the `consumerGroup` parameter in the endpoint URI (e.g. `kafka:my-topic?consumerGroup=my-group`).

The dynamic endpoint is created on the fly with respective settings. So you can use the `kafka` endpoint component in your test as follows:

Java

```
send("kafka:hello")
    .message()
    .body("foo")
    .header(KafkaMessageHeaders.MESSAGE_KEY, 1);
```

XML

```
<send endpoint="kafka:hello">
    <message>
        ...
    </message>
    <header>
        <element name="citrus_kafka_messageKey" value="1"/>
    </header>
</send>
```

This action above will create a dynamic Kafka endpoint and publish the message to the `hello` topic. The dynamic endpoint url uses the `kafka:` scheme and gives the topic name as resource path. In addition to that the dynamic endpoint url is able to set multiple parameters such as `server`. Let's have a look at this in a small example.

Java

```
send("kafka:hello?server=localhost:9091")
    .message(new KafkaMessage("foo"));
```

XML

```
<send endpoint="kafka:hello?server=localhost:9091">
    <message>
        ...
    </message>
</send>
```

You can add multiple parameters to the endpoint url in order to set properties on the dynamic endpoint. You can read more about dynamic endpoints in chapter dynamic endpoints.

13.7. Embedded Kafka Server

The Kafka message broker is composed of a Kafka server that uses some default configuration options. Citrus provides an embedded server (**for testing purpose only!**) that is able to start within your integration test environment. The server cluster is configured with one single Kafka node. You can define server ports and broker properties such as topics, number of partitions and broker ids. Given topics are automatically added via admin client on the Kafka server with given amount of partitions.

You can add the embedded server component to the Spring application context as normal Spring bean. The server will automatically start and stop within the application context lifecycle. The server log directory is located in the Java temp directory and is automatically deleted on JVM exit.

See the following configuration how to use the embedded server component:

Java

```
@Bean
public EmbeddedKafkaServer kafkaServer() {
    return new EmbeddedKafkaServerBuilder()
                    .topics("foo", "bar")
                    .kafkaServerPort(9091)
                    .build();
}
```

XML

```
<citrus-kafka:embedded-server id="kafkaServer"
                              topics="foo,bar"
                              kafka-server-port="9091"/>
```

The embedded server component provides following properties to set:

Name	Type	Description
topics	java.lang.String	Comma delimited list of topic names that automatically will be created on the server.

Name	Type	Description
kafka-server-port	java.lang.Integer	Port of the embedded Kafka server
broker-properties	java.util.Map	Map of broker property key-value pairs that overwrite the default broker properties. For a list of available properties please review the official Kafka documentation.
partitions	java.lang.Integer	Number of partitions to create for each topic
log-dir-path	java.lang.String	Path to server log directory. The server will create its data directory in this directory. By default, the Java temp directory is used.
auto-delete-logs	java.lang.Boolean	Auto delete server log directories on exit. Default is true.

13.8. Kafka Settings

Citrus provides global settings for the Kafka endpoint module. These settings can be configured via Java system properties or environment variables.

System Property	Environment Variable	Default	Description
`citrus.kafka.dynamic.consumer.group`	`CITRUS_KAFKA_DYNAMIC_CONSUMER_GROUP`	`true`	When enabled, dynamic Kafka endpoint URIs (e.g. `kafka:my-topic`) automatically use a unique consumer group for each endpoint. This ensures each consumer independently receives all messages from its subscribed topic. When disabled, dynamic endpoints share the default consumer group `citrus_kafka_group`.

Chapter 14. Http REST support

REST APIs have gained more and more significance regarding client-server interfaces with message exchange over Http. Http is a synchronous protocol by nature so the client blocks in order to wait for the server response synchronously. Citrus is able to connect with Http services and REST APIs on both client and server side with a powerful JSON message data support.

In the next sections you will learn how to invoke Http services as a client and how to handle REST Http requests in a test case. The chapter deals with setting up an Http server in order to accept client requests and provide proper Http responses with GET, PUT, DELETE or POST request method.



The http components in Citrus are kept in a separate Maven module. So you should add the module as Maven dependency to your project accordingly.

Http module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-http</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

14.1. Http REST client

On the client side Citrus uses a simple Http message client component connecting to the server. The **request-url** attribute defines the Http server endpoint URL to connect to. You can reference this client in your test case in order to send and receive messages. Citrus as client waits for the response message from the server. After that the response message goes through the validation process as usual.

Spring Bean

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .requestMethod(HttpMethod.GET)
        .contentType("application/xml")
        .charset("UTF-8")
        .timeout(60000L)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-http="http://www.citrusframework.org/schema/http/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/http/config
           http://www.citrusframework.org/schema/http/config/citrus-http-config.xsd">

    <citrus-http:client id="httpClient"
          request-url="http://localhost:8080/hello"
          request-method="GET"
          content-type="application/xml"
          charset="UTF-8"
          timeout="60000"/>
</beans>
```



Citrus provides a customized Http configuration schema for the Spring XML application context configuration files. Simply include the `citrus-http` namespace in the configuration XML files as seen in the example above.

The **request-method** defines the Http method to use. In addition to that we can specify the content-type of the request we are about to send. The charset is also added to the content-type header. In case you do not want to set the charset at all please specify an empty string as the default value is *UTF-8*.



The endpoint's default content type is applied only to request methods that carry a body (POST, PUT, DELETE, PATCH). Requests using GET, HEAD, OPTIONS or TRACE are sent without a content type header unless the test explicitly sets one.

The client builds the Http request and sends it to the Http server. While the client is waiting for the synchronous Http response to arrive we are able to poll several times for the response message in our test case. As usual you can use the same client endpoint in your test case to send and receive messages synchronously. In case the reply message comes in too late according to the timeout settings a respective timeout error is raised.

Http defines several request methods that a client can use to access Http server resources. In the example client above we are using **GET** as default request method. Of course you can overwrite this setting in a test case action by setting the Http request method inside the sending test action. The Http client component can be used as normal endpoint in a sending test action. Use something like this in your test:

Java

```
send("httpClient")
    .message()
    .body("Hello HttpServer")
    .header(HttpMessageHeaders.HTTP_REQUEST_METHOD, HttpMethod.POST.name())
```

XML

```
<send endpoint="httpClient">
    <message>
        <payload>
            <TestMessage>
                <Text>Hello HttpServer</Text>
            </TestMessage>
        </payload>
    </message>
    <header>
        <element name="citrus_http_method" value="POST"/>
    </header>
</send>
```



Citrus uses the Spring REST template mechanism for sending out Http requests. This means you have great customizing opportunities with a special REST template configuration. You can think of basic Http authentication, read timeouts and special message factory implementations. Just use the custom REST template attribute in client configuration like this:

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .requestMethod(HttpMethod.GET)
        .contentType("text/plain")
        .restTemplate(customizedRestTemplate())
        .timeout(60000L)
        .build();
}

@Bean
public RestTemplate customizedRestTemplate() {
    RestTemplate restTemplate = new RestTemplate();

    StringHttpMessageConverter converter = new StringHttpMessageConverter();
    converter.setSupportedMediaTypes(Collections.singletonList("text/plain"));

    restTemplate.setMessageConverters(Collections.singletonList(converter));

    restTemplate.setErrorHandler(customErrorHandler());

    HttpComponentsClientHttpRequestFactory requestFactory = new
HttpComponentsClientHttpRequestFactory();
    requestFactory.setReadTimeout(9000L);

    restTemplate.setRequestFactory(requestFactory);

    return restTemplate;
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/hello"
                    request-method="GET"
                    content-type="text/plain"
                    rest-template="customizedRestTemplate"/>

<!-- Customized rest template -->
<bean name="customizedRestTemplate"
class="org.springframework.web.client.RestTemplate">
  <property name="messageConverters">
    <util:list id="converter">
      <bean class="org.springframework.http.converter.StringHttpMessageConverter">
        <property name="supportedMediaTypes">
          <util:list id="types">
            <value>text/plain</value>
          </util:list>
        </property>
      </bean>
    </util:list>
  </property>
  <property name="errorHandler">
    <!-- Custom error handler -->
    <ref bean ="customErrorHandler"/>
  </property>
  <property name="requestFactory">
    <bean
class="org.springframework.http.client.HttpComponentsClientHttpRequestFactory">
      <property name="readTimeout" value="9000"/>
    </bean>
  </property>
</bean>
```

Up to now we have used a generic **send** test action to send Http requests as a client. This is completely valid strategy as the Citrus Http client is a normal message endpoint.

In order to simplify the Http usage in a test case Citrus also provides special test action implementations for Http.



These Http specific actions are located in a separate XML namespace. In case you are writing XML test cases you need to add this namespace to our test case XML first.

Add Citrus Http action namespace

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:http="http://www.citrusframework.org/schema/http/testcase"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/http/testcase
            http://www.citrusframework.org/schema/http/testcase/citrus-http-
testcase.xsd">

      [...]

    </beans>
```

The test case is now ready to use the specific Http test actions.

Java

```
http().client("httpClient")
        .send()
        .post("/customer")
        .message()
        .body("<customer>" +
                "<id>citrus:randomNumber()</id>" +
                "<name>testuser</name>" +
              "</customer>")
        .contentType(MediaType.APPLICATION_XML_VALUE)
        .accept(MediaType.APPLICATION_XML_VALUE)
        .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:send-request client="httpClient">
  <http:POST path="/customer">
    <http:headers content-type="application/xml" accept="application/xml">
      <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
    </http:headers>
    <http:body>
      <http:data>
        <![CDATA[
          <customer>
            <id>citrus:randomNumber()</id>
            <name>testuser</name>
          </customer>
        ]]>
      </http:data>
    </http:body>
  </http:POST>
</http:send-request>
```

The action above uses several Http specific settings such as the request method **POST** as well as the **content-type** and **accept** headers. As usual the send action needs a target Http client endpoint component. We can specify a request **path** attribute that added as relative path to the base uri used on the client.

When using a **GET** request we can specify some request uri parameters.

Java

```
http().client("httpClient")
        .send()
        .get("/customer/${custom_header_id}")
        .message()
        .contentType(MediaType.APPLICATION_XML_VALUE)
        .accept(MediaType.APPLICATION_XML_VALUE)
        .queryParam("type", "active");
```

XML

```
<http:send-request client="httpClient">
  <http:GET path="/customer/${custom_header_id}">
    <http:params content-type="application/xml" accept="application/xml">
      <http:param name="type" value="active"/>
    </http:params>
  </http:GET>
</http:send-request>
```

The send action above uses a **GET** request on the endpoint uri `http://localhost:8080/customer/1234?type=active`.

Of course when sending Http client requests we are also interested in receiving Http response messages. We want to validate the success response with Http status code.

Java

```
http().client("httpClient")
        .receive()
        .response(HttpStatus.OK.value())
        .message()
        .body("<customer>" +
                "<id>citrus:randomNumber()</id>" +
                "<name>testuser</name>" +
              "</customer>")
        .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:receive-response client="httpClient">
  <http:headers status="200" reason-phrase="OK" version="HTTP/1.1">
    <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
  </http:headers>
  <http:body>
    <http:data>
      <![CDATA[
          <customerResponse>
            <success>true</success>
          </customerResponse>
      ]]>
    </http:data>
  </http:body>
</http:receive-response>
```

The **receive-response** test action also uses a client component. We can expect response status code information such as **status** and **reason-phrase** . Of course Citrus will raise a validation exception in case Http status codes mismatch.



By default, the client component will add the **Accept** http header and set its value to a list of all supported encodings on the host operating system. This list can get quite big so you may want to not set this default accept header. The setting is done in the Spring RestTemplate:

Java

```
@Bean
public RestTemplate customizedRestTemplate() {
    RestTemplate restTemplate = new RestTemplate();

    StringHttpMessageConverter converter = new StringHttpMessageConverter();
    converter.setWriteAcceptCharset(false);

    restTemplate.setMessageConverters(Collections.singletonList(converter));

    return restTemplate;
}
```

XML

```
<bean name="customizedRestTemplate"
class="org.springframework.web.client.RestTemplate">
    <property name="messageConverters">
        <util:list id="converter">
            <bean
class="org.springframework.http.converter.StringHttpMessageConverter">
                <property name="writeAcceptCharset" value="false"/>
            </bean>
        </util:list>
    </property>
</bean>
```

Add this custom RestTemplate configuration and set it to the client component with **rest-template** property. Fortunately the Citrus client component provides a separate setting **default-accept-header** which is a Boolean setting. By default, this setting is set to **true** so the default accept header is automatically added to all requests. If you set this flag to **false** the header is not set:

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .requestMethod(HttpMethod.GET)
        .contentType("text/plain")
        .defaultAcceptHeader(false)
        .timeout(60000L)
        .build();
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/hello"
                    request-method="GET"
                    content-type="text/plain"
                    default-accept-header="false"/>
```

Of course, you can set the **Accept** header on each send operation in order to tell the server what kind of content types are supported in response messages.



For requests that must carry a Playwright browser session (the cookies and HTTP credentials of a logged-in browser), use an Http client whose transport is the browser. See API requests with the browser session.

Now we can send and receive messages as Http client with specific test actions. Now let's move on to the Http server.

14.2. Http client interceptors

The client component is able to add custom interceptors that participate in the request/response processing. The interceptors need to implement the common interface **org.springframework.http.client.ClientHttpRequestInterceptor**.

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .requestMethod(HttpMethod.GET)
        .interceptor(new LoggingClientInterceptor())
        .build();
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/hello"
                    request-method="GET"
                    interceptors="clientInterceptors"/>

<util:list id="clientInterceptors">
  <bean class="org.citrusframework.http.interceptor.LoggingClientInterceptor"/>
</util:list>
```

The sample above adds the Citrus logging client interceptor that logs requests and responses exchanged with that client component. You can add custom interceptor implementations here in

order to participate in the request/response message processing.

14.3. Http REST server

Receiving Http requests requires an Http server listening on a port on your local machine. Citrus offers an embedded Http server which is capable of handling incoming Http requests. The server accepts client connections and must provide a proper Http response. In the next section you will see how to simulate server side Http REST service with Citrus.

The Http server provides the following global settings configurable via system properties or environment variables:

Table 2. System properties

System property	Description
citrus.http.server.response.cache.size	Number of response messages the server keeps in its in-memory cache for message tracing (default="100")
citrus.http.server.use.default.filters	Enable/disable default servlet filters (e.g. request caching filter) on the server's servlet context (default="true")

Table 3. Environment variables

Environment variable	Description
CITRUS_HTTP_SERVER_RESPONSE_CACHE_SIZE	Number of response messages the server keeps in its in-memory cache for message tracing (default="100")
CITRUS_HTTP_SERVER_USE_DEFAULT_FILTERS	Enable/disable default servlet filters (e.g. request caching filter) on the server's servlet context (default="true")

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .build();
}
```

XML

```
<citrus-http:server id="httpServer"
                port="8080"
                auto-start="true"/>
```

Citrus uses an embedded Jetty server that will automatically start when the Citrus context is loaded (auto-start="true"). The basic connector is listening on port **8080** for requests. Test cases can interact with this server instance via message channels by default. The server provides an inbound channel that holds incoming request messages. The test case can receive those requests from the channel with a normal receive test action. In a second step the test case can provide a synchronous response message as reply which will be automatically sent back to the Http client as response.



The figure above shows the basic setup with inbound channel and reply channel. You as a tester should not worry about this too much. By default, you as a tester just use the server as synchronous endpoint in your test case. This means that you simply receive a message from the server and send a response back.

Java

```
receive("httpServer")
    .message()
    .body("...");

send("httpServer")
    .message()
    .body("...")
    .header(HttpMessageHeaders.HTTP_STATUS_CODE, 200);
```

XML

```
<receive endpoint="httpServer">
    <message>
        <data>
          [...]
        </data>
    </message>
</receive>

<send endpoint="httpServer">
    <message>
        <data>
          [...]
        </data>
    </message>
</send>
```

As you can see we reference the server id in both receive and send actions. The Citrus server instance will automatically send the response back to the calling Http client. In most cases this is exactly what we want to do - send back a response message that is specified inside the test. The Http

server component by default uses a channel endpoint adapter in order to forward all incoming requests to an in-memory message channel. This is done completely behind the scenes. The Http server component provides some more customization possibilities when it comes to endpoint adapter implementations. This topic is discussed in a separate section endpoint-adapter. Up to now we keep it simple by synchronously receiving and sending messages in the test case.



The default channel endpoint adapter automatically creates an inbound message channel where incoming messages are stored too internally. So if you need to clean up a server that has already stored some incoming messages you can do this easily by purging the internal message channel. The message channel follows a naming convention **{serverName}.inbound** where **{serverName}** is the Spring bean name of the Citrus server endpoint component. If you purge this internal channel in a before test nature you are sure that obsolete messages on a server instance get purged before each test is executed.

So let's get back to our mission of providing response messages as server to connected clients. As you might know Http REST works with some characteristic properties when it comes to send and receive messages. For instance a client can send different request methods GET, POST, PUT, DELETE, HEAD and so on. The Citrus server may verify this method when receiving client requests. Therefore we have introduced special Http test actions for server communication. Have a look at a simple example:

Java

```
http().server("httpServer")
    .receive()
    .post("/test")
    .message()
    .contentType("application/xml")
    .accept("application/xml")
    .body("<testRequestMessage>" +
            "<text>Hello HttpServer</text>" +
          "</testRequestMessage>")
    .header("Authorization", "Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==")
    .header("X-CustomHeaderId", "${custom_header_id}")
    .extract(extract().fromHeaders()
                .header("X-MessageId", "message_id"));

http().server("httpServer")
    .send()
    .response(HttpStatus.OK.value())
    .message()
    .contentType("application/xml")
    .body("<testResponseMessage>" +
            "<text>Hello Citrus</text>" +
          "</testResponseMessage>")
    .header("X-CustomHeaderId", "${custom_header_id}")
    .header("X-MessageId", "${message_id}");
```

XML

```
<http:receive-request server="helloHttpServer">
  <http:POST path="/test">
    <http:headers content-type="application/xml" accept="application/xml, */*">
      <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
      <http:header name="Authorization" value="Basic
c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==">
    </http:headers>
    <http:body>
    <http:data>
      <![CDATA[
        <testRequestMessage>
          <text>Hello HttpServer</text>
        </testRequestMessage>
      ]]>
    </http:data>
    </http:body>
  </http:POST>
  <http:extract>
    <http:header name="X-MessageId" variable="message_id"/>
  </http:extract>
</http:receive-request>

<http:send-response server="helloHttpServer">
  <http:headers status="200" reason-phrase="OK" version="HTTP/1.1">
    <http:header name="X-MessageId" value="${message_id}"/>
    <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
    <http:header name="Content-Type" value="application/xml"/>
  </http:headers>
  <http:body>
  <http:data>
    <![CDATA[
      <testResponseMessage>
        <text>Hello Citrus</text>
      </testResponseMessage>
    ]]>
  </http:data>
  </http:body>
</http:send-response>
```

We receive a client request and validate that the request method is **POST** on request path **/test** . Now we can validate special message headers such as **content-type** . In addition to that we can check custom headers and basic authorization headers. As usual the optional message body is compared to an expected message template. The custom **X-MessageId** header is saved to a test variable **message_id** for later usage in the response.

The response message defines Http typical entities such as **status** and **reason-phrase**. Here the tester can simulate **404 NOT_FOUND** errors or similar other status codes that get sent back to the client. In our example everything is **OK** and we send back a response body and some custom

header entries.

That is basically how Citrus simulates Http server operations. We receive the client request and validate the request properties. Then we send back a response with an Http status code.

This completes the server actions on Http message transport. Now we continue with some more Http specific settings and features.

14.4. Http headers

When dealing with Http request/response communication we always deal with Http specific headers. The Http protocol defines a group of header attributes that both client and server need to be able to handle. You can set and validate these Http headers in Citrus quite easy. Let us have a look at a client operation in Citrus where some Http headers are explicitly set before the request is sent out.

Java

```
http().client("httpClient")
    .send()
    .post()
    .message()
    .contentType("application/xml")
    .accept("application/xml")
    .body("<testRequestMessage>" +
            "<text>Hello HttpServer</text>" +
          "</testRequestMessage>")
    .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:send-request client="httpClient">
  <http:POST>
    <http:headers>
        <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
        <http:header name="Content-Type" value="application/xml"/>
        <http:header name="Accept" value="application/xml"/>
    </http:headers>
    <http:body>
        <http:payload>
            <testRequestMessage>
                <text>Hello HttpServer</text>
            </testRequestMessage>
        </http:payload>
    </http:body>
  </http:POST>
</http:send-request>
```

We are able to set custom headers (**X-CustomHeaderId**) that go directly into the Http header

section of the request. In addition to that testers can explicitly set Http reserved headers such as **Content-Type** . Fortunately you do not have to set all headers on your own. Citrus will automatically set the required Http headers for the request. So we have the following Http request which is sent to the server:

Sample Http request

```
POST /test HTTP/1.1
Accept: application/xml
Content-Type: application/xml
X-CustomHeaderId: 123456789
Accept-Charset: macroman
User-Agent: Jakarta Commons-HttpClient/3.1
Host: localhost:8091
Content-Length: 175
<testRequestMessage>
    <text>Hello HttpServer</text>
</testRequestMessage>
```

On server side testers are interested in validating the Http headers. Within Citrus receive action you simply define the expected header entries. The Http specific headers are automatically available for validation as you can see in this example:

Java

```
http().server("httpServer")
    .receive()
    .post()
    .message()
    .contentType("application/xml")
    .accept("application/xml")
    .body("<testRequestMessage>" +
            "<text>Hello HttpServer</text>" +
          "</testRequestMessage>")
    .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:receive-request server="httpServer">
  <http:POST>
    <http:headers>
        <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
        <http:header name="Content-Type" value="application/xml"/>
        <http:header name="Accept" value="application/xml"/>
    </http:headers>
    <http:body>
        <http:payload>
            <testRequestMessage>
                <text>Hello HttpServer</text>
            </testRequestMessage>
        </http:payload>
    </http:body>
  </http:POST>
</http:receive-request>
```

The test checks on custom headers and Http specific headers to meet the expected values.

Now that we have accepted the client request and validated the contents we are able to send back a proper Http response message. Same thing here with Http specific headers. The Http protocol defines several headers marking the success or failure of the server operation. In the test case you can set those headers for the response message with conventional Citrus header names. See the following example to find out how that works for you.

Java

```
http().server("httpServer")
    .send()
    .response(HttpStatus.OK.value())
    .message()
    .contentType("application/xml")
    .body("<testResponseMessage>" +
            "<text>Hello Citrus</text>" +
          "</testResponseMessage>")
    .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:send-response server="httpServer">
    <http:headers status="200" reason-phrase="OK">
        <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
        <http:header name="Content-Type" value="application/xml"/>
    </http:headers>
    <http:body>
        <http:payload>
            <testResponseMessage>
                <text>Hello Citrus</text>
            </testResponseMessage>
        </http:payload>
    </http:body>
</http:send-response>
```

Once more we set the custom header entry (**X-CustomHeaderId**) and an Http reserved header (**Content-Type**) for the response message. On top of this we are able to set the response status for the Http response. We use the reserved header names **status** in order to mark the success of the server operation. With this mechanism we can easily simulate different server behaviour such as Http error response codes (e.g. 404 - Not found, 500 - Internal error). Let us have a closer look at the generated response message:

Sample Http response

```
HTTP/1.1 200 OK
Content-Type: application/xml;charset=UTF-8
Accept-Charset: macroman
Content-Length: 205
Server: Jetty(7.0.0.pre5)
<testResponseMessage>
    <text>Hello Citrus Client</text>
</testResponseMessage>
```



You do not have to set the reason phrase all the time. It is sufficient to only set the Http status code. Citrus will automatically add the proper reason phrase for well known Http status codes.

The only thing that is missing right now is the validation of Http status codes when receiving the server response in a Citrus test case. It is very easy as you can use the Citrus reserved header names for validation, too.

Java

```
http().client("httpClient")
    .receive()
    .response(HttpStatus.OK.value())
    .message()
    .contentType("application/xml")
    .body("<testResponseMessage>" +
            "<text>Hello Citrus</text>" +
          "</testResponseMessage>")
    .header("X-CustomHeaderId", "${custom_header_id}");
```

XML

```
<http:receive-response client="httpClient">
    <http:headers status="200" reason-phrase="OK" version="HTTP/1.1">
        <http:header name="X-CustomHeaderId" value="${custom_header_id}"/>
    </http:headers>
    <http:body>
        <http:payload>
            <testResponseMessage>
                <text>Hello Citrus</text>
            </testResponseMessage>
        </http:payload>
    </http:body>
</http:receive-response>
```



Be aware of the slight differences in request URI and context path. The context path gives you the web application context path within the servlet container for your web application. The request URI always gives you the complete path that was called for this request.

As you can see we are able to validate all parts of the initial request endpoint URI the client was calling. This completes the Http header processing within Citrus. On both client and server side Citrus is able to set and validate Http specific header entries which is essential for simulating Http communication.

14.5. Http query parameter

Up to now we have used some of the basic Citrus reserved Http header names (status, version, reason-phrase). In Http RESTful services some other header names are essential for validation. These are request attributes like query parameters, context path and request URI. The Citrus server side REST message controller will automatically add all this information to the message header for you. So all you need to do is validate the header entries in your test.

The next example receives an Http GET method request on server side. Here the GET request does not have any message payload, so the validation just works on the information given in the message header. We assume the client to call `http://localhost:8080/app/users?id=123456789`. As a

tester we need to validate the request method, request URI, context path and the query parameters.

Java

```
http()
    .server(httpServer)
    .receive()
    .get("/api/users")
    .message()
    .queryParam("id", "123456789")
    .contentType(MediaType.APPLICATION_XML_VALUE)
    .accept(MediaType.APPLICATION_XML_VALUE)
    .header("Host", "localhost:8080");
```

XML

```
<http:receive-request server="httpServer">
  <http:GET path="/api/users" context-path="/app">
    <http:params>
        <http:param name="id" value="123456789"/>
    </http:params>
    <http:headers content-type="application/xml" accept="application/xml">
        <http:header name="Host" value="localhost:8080"/>
    </http:headers>
    <http:body>
        <http:data></http:data>
    </http:body>
  </http:GET>
</http:receive-request>
```

The http server is able to validate incoming Http query parameters. You can add as many parameters as you would like to verify. Each parameter value is able to use test variables and validation matcher expressions as usual.

On the client side we are able to send query parameters in our request.

Java

```
http()
    .client(httpClient)
    .send()
    .get("/api/users")
    .message()
    .queryParam("id", "123456789")
    .contentType(MediaType.APPLICATION_XML_VALUE)
    .accept(MediaType.APPLICATION_XML_VALUE);
```

XML

```
<http:send-request client="httpClient">
  <http:GET path="/app/users">
    <http:params>
        <http:param name="id" value="123456789"/>
    </http:params>
    <http:headers content-type="application/xml" accept="application/xml"/>
    <http:body>
        <http:data></http:data>
    </http:body>
  </http:GET>
</http:send-request>
```

The parameters are automatically added to the request URL that is configured on the `httpClient` component.

14.6. Http server interceptors

The server component is able to add custom interceptors that participate in the request/response processing. The interceptors need to implement the common interface **org.springframework.web.servlet.HandlerInterceptor**.

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .interceptor(new LoggingHandlerInterceptor())
        .build();
}
```

XML

```
<citrus-http:server id="httpServer"
                    port="8080"
                    auto-start="true"
                    interceptors="serverInterceptors"/>

<util:list id="serverInterceptors">
  <bean class="org.citrusframework.http.interceptor.LoggingHandlerInterceptor"/>
</util:list>
```

The sample above adds the Citrus logging handler interceptor that logs requests and responses exchanged with that server component. You can add custom interceptor implementations here in order to participate in the request/response message processing.

14.7. Http form urlencoded data

HTML form data can be sent to the server using different methods and content types. One of them is a POST method with **x-www-form-urlencoded** body content. The form data elements are sent to the server using key-value pairs POST data where the form control name is the key and the control data is the url encoded value.

Form urlencoded form data content could look like this:

Form urlencoded data

```
password=s%21cr%21t&username=foo
```

As you can see the form data is automatically encoded. In the example above we transmit two form controls **password** and **username** with respective values **scrt** and **foo** . In case we would validate this form data in Citrus we are able to do this with plaintext message validation.

Java

```
receive("httpServer")
    .message()
    .type(MessageType.PLAINTEXT)
    .body("password=s%21cr%21t&username=${username}")
    .header(HttpMessageHeaders.HTTP_REQUEST_METHOD, HttpMethod.POST.name())
    .header(HttpMessageHeaders.HTTP_REQUEST_URI, "/form-test")
    .header(HttpMessageHeaders.HTTP_CONTENT_TYPE, "application/x-www-form-
urlencoded");

send("httpServer")
    .message()
    .header(HttpMessageHeaders.HTTP_STATUS_CODE, 200);
```

XML

```
<receive endpoint="httpServer">
  <message type="plaintext">
    <data>
      <![CDATA[
        password=s%21cr%21t&username=${username}
      ]]>
    </data>
  </message>
  <header>
    <element name="citrus_http_method" value="POST"/>
    <element name="citrus_http_request_uri" value="/form-test"/>
    <element name="Content-Type" value="application/x-www-form-urlencoded"/>
  </header>
</receive>

<send endpoint="httpServer">
  <message>
    <data></data>
  </message>
  <header>
    <element name="citrus_http_status_code" value="200"/>
  </header>
</send>
```

Obviously validating these key-value pair character sequences can be hard especially when having HTML forms with lots of form controls. This is why Citrus provides a special message validator for **x-www-form-urlencoded** contents. First of all we have to add **citrus-http** module as dependency to our project if we have not done so yet. After that we can add the validator implementation to the list of message validators used in Citrus.

Java

```
@Bean
public FormUrlEncodedMessageValidator formUrlEncodedMessageValidator() {
    return new FormUrlEncodedMessageValidator();
}
```

XML

```
<citrus:message-validators>
  <citrus:validator
class="org.citrusframework.http.validation.FormUrlEncodedMessageValidator"/>
</citrus:message-validators>
```

Now we are able to receive the urlencoded form data message in a test.

Java

```
receive("httpServer")
    .message()
    .type("x-www-form-urlencoded")
    .body("<form-data xmlns=\"http://www.citrusframework.org/schema/http/message\">\n"
+
            "<content-type>application/x-www-form-urlencoded</content-type>\n" +
            "<action>/form-test</action>\n" +
            "<controls>\n" +
                "<control name=\"password\">\n" +
                    "<value>${password}</value>\n" +
                "</control>\n" +
                "<control name=\"username\">\n" +
                    "<value>${username}</value>\n" +
                "</control>\n" +
            "</controls>\n" +
            "</form-data>")
    .header(HttpMessageHeaders.HTTP_REQUEST_METHOD, HttpMethod.POST.name())
    .header(HttpMessageHeaders.HTTP_REQUEST_URI, "/form-test")
    .header(HttpMessageHeaders.HTTP_CONTENT_TYPE, "application/x-www-form-
urlencoded");
```

XML

```
<receive endpoint="httpServer">
  <message type="x-www-form-urlencoded">
    <payload>
      <form-data xmlns="http://www.citrusframework.org/schema/http/message">
        <content-type>application/x-www-form-urlencoded</content-type>
        <action>/form-test</action>
        <controls>
          <control name="password">
            <value>${password}</value>
          </control>
          <control name="username">
            <value>${username}</value>
          </control>
        </controls>
      </form-data>
    </payload>
  </message>
  <header>
    <element name="citrus_http_method" value="POST"/>
    <element name="citrus_http_request_uri" value="/form-test"/>
    <element name="Content-Type" value="application/x-www-form-urlencoded"/>
  </header>
</receive>
```

We use a special message type **x-www-form-urlencoded** so the new message validator will take

action. The form url encoded message validator is able to handle a special XML representation of the form data. This enables the very powerful XML message validation capabilities of Citrus such as ignoring elements and usage of test variables inline.

Each form control is translated to a control element with respective name and value properties. The form data is validated in a more comfortable way as the plaintext message validator would be able to offer.

14.8. Http error handling

So far we have received response messages with Http status code **200 OK** . How to deal with server errors like **404 Not Found** or **500 Internal server error** ? The default Http message client error strategy is to propagate server error response messages to the receive action for validation. We simply check on Http status code and status text for error validation.

Java

```
http().client("httpClient")
        .send()
        .post()
        .message()
        .body("<testRequestMessage>" +
                "<text>Hello HttpServer</text>" +
              "</testRequestMessage>");

http().client("httpClient")
        .receive()
        .response(HttpStatus.FORBIDDEN.value());
```

XML

```
<http:send-request client="httpClient">
    <http:POST>
        <http:body>
            <http:payload>
                <testRequestMessage>
                    <text>Hello HttpServer</text>
                </testRequestMessage>
            </http:payload>
        </http:body>
    </http:POST>
</http:send-request>

<http:receive-response client="httpClient">
    <http:headers status="403" reason-phrase="FORBIDDEN"/>
    <http:body>
        <http:data><![CDATA[]]></http:data>
    </http:body>
</http:receive-response>
```

The message data can be empty depending on the server logic for these error situations. If we receive additional error information as message payload just add validation assertions as usual.

Instead of receiving such empty messages with checks on Http status header information we can change the error strategy in the message sender component in order to automatically raise exceptions on response messages other than **200 OK** . Therefore we go back to the Http message sender configuration for changing the error strategy.

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/test")
        .errorHandlingStrategy(ErrorHandlingStrategy.THROWS_EXCEPTION)
        .build();
}
```

XML

```
<citrus-http:client id="httpClient"
                  request-url="http://localhost:8080/test"
                  error-strategy="throwsException"/>
```

Now we expect an exception to be thrown because of the error response. Following from that we have to change our test case. Instead of receiving the error message with receive action we assert the client exception and check on the Http status code and status text.

Java

```
assertException()
    .exception(HttpClientErrorException.class)
    .message("403 Forbidden")
    .when(
        http().client("httpClient")
            .send()
            .post()
            .message()
            .body("<testRequestMessage>" +
                    "<text>Hello HttpServer</text>" +
                  "</testRequestMessage>")
    );
```

XML

```
<assert exception="org.springframework.web.client.HttpClientErrorException"
           message="403 Forbidden">
    <when>
        <http:send-request client="httpClient">
            <http:body>
                <http:payload>
                    <testRequestMessage>
                        <text>Hello HttpServer</text>
                    </testRequestMessage>
                </http:payload>
            </http:body>
        </http:send-request>
    </when>
</assert>
```

Both ways of handling Http error messages on client side are valid for expecting the server to raise Http error codes. Choose the preferred way according to your test project requirements.

14.9. Http client basic authentication

As a client you may have to authenticate properly in order to access a resource on the server. In many cases this will be basic authentication with username/password where the credentials are transmitted in the request header section with base64 encoding.

The easiest approach to set the **Authorization** header for a basic authentication Http request would be to explicitly set the header in the send action definition. Of course, you have to use the correct basic authentication header syntax with base64 encoding for the username:password phrase. See how it works in this simple example.

Java

```
http().client("httpClient")
    .send()
    .get()
    .message()
    .header("Authorization", "Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==");
```

XML

```
<http:send-request client="httpClient">
  <http:GET>
    <http:headers>
      <http:header name="Authorization" value="Basic
c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==" />
    </http:headers>
  </http:GET>
</http:send-request>
```

Citrus will add this header to the Http requests and the server will read the **Authorization** username and password. For more convenient base64 encoding you can also use a Citrus function `citrus:encodeBase64(username:password)`. See functions-encode-base64 for more details.

Instead of adding the Http **Authorization** header manually you can also enable the basic authentication on the Http client endpoint. Simply configure the basic authentication credentials on the Citrus Http client component. Just see the following example and learn how to do that.

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/test")
        .authentication(HttpAuthentication.basic("username", "password"))
        .build();
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/test"
                    request-factory="clientRequestFactory"/>

<bean id="clientRequestFactory"
    class="org.citrusframework.http.security.HttpClientRequestFactoryBean">
    <constructor-arg>
        <bean class="org.citrusframework.http.security.BasicAuthentication">
            <constructor-arg value="username"/>
            <constructor-arg value="password"/>
        </bean>
    </constructor-arg>
</bean>
```

In case you need a more powerful configuration of the basic authentication you can also directly use the request factory as a configuration item for basic authentication.

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("http://localhost:8080/test")
        .requestFactory(basicAuthFactory())
        .build();
}

@Bean
public BasicAuthClientHttpRequestFactory basicAuthFactory() {
    BasicAuthClientHttpRequestFactory factory = new
BasicAuthClientHttpRequestFactory();

    AuthScope scope = new AuthScope("localhost", "8080", "", "basic");
    factory.setAuthScope(scope);

    UsernamePasswordCredentials credentials = new
UsernamePasswordCredentials("username", "password");
    factory.setCredentials(credentials);

    return factory;
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/test"
                    request-factory="basicAuthFactory"/>

<bean id="basicAuthFactory"
    class="org.citrusframework.http.client.BasicAuthClientHttpRequestFactory">
  <property name="authScope">
      <bean class="org.apache.hc.client5.http.auth.AuthScope">
        <constructor-arg value="localhost"/>
        <constructor-arg value="8080"/>
      </bean>
  </property>
  <property name="credentials">
    <bean class="org.apache.hc.client5.http.auth.UsernamePasswordCredentials">
        <constructor-arg value="username"/>
        <constructor-arg value="password"/>
    </bean>
  </property>
</bean>
```

The advantage of setting the basic authentication on the Http client is obvious. All test actions that reference the client component will automatically use the basic authentication.

The above configuration results in Http client requests with authentication headers properly set for basic authentication. The client request factory takes care of adding the proper basic authentication header to each request that is sent with this Citrus message sender. Citrus uses preemptive authentication. The message sender only sends a single request to the server with all authentication information set in the message header. The request which determines the authentication scheme on the server is skipped. This is why you have to add some auth scope in the client request factory so Citrus can set up an authentication cache within the Http context in order to have preemptive authentication.

As a result of the basic auth client request factory the following example request that is created by the Citrus Http client has the **Authorization** header set. This is done now automatically for all requests with this Http client.

Sample request with basic authentication

```
POST /test HTTP/1.1
Accept: application/xml
Content-Type: application/xml
Accept-Charset: iso-8859-1, us-ascii, utf-8
Authorization: Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==
User-Agent: Jakarta Commons-HttpClient/3.1
Host: localhost:8080
Content-Length: 175
<testRequestMessage>
  <text>Hello HttpServer</text>
</testRequestMessage>
```

14.10. Http server basic authentication

The Citrus Http server is able to configure basic authentication so clients need to authenticate properly when accessing server resources.

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .authentication("/foo/*", HttpAuthentication.basic("username", "password"))
        .build();
}
```

XML

```
<citrus-http:server id="basicAuthHttpServer"
                port="8080"
                auto-start="true"
                security-handler="securityHandlerFactory"/>

<bean id="securityHandlerFactory"
class="org.citrusframework.http.security.HttpSecurityHandlerFactoryBean">
    <constructor-arg>
        <bean class="org.citrusframework.http.security.BasicAuthentication">
            <constructor-arg value="username"/>
            <constructor-arg value="password"/>
            <property name="resourcePath" value="/foo/*"/>
        </bean>
    </constructor-arg>
</bean>
```

In case you need a more powerful server configuration you can also set the security handler directly.

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .securityHandler(securityHandler())
        .build();
}

@Bean
public SecurityHandlerFactory securityHandler() {
    SecurityHandlerFactory factory = new SecurityHandlerFactory();

    User user = new User();
    user.setName("citrus");
    user.setPassword("secret");
    user.setRoles("CitrusRole");
    factory.setUsers(Collections.singletonList(user));

    factory.setConstraints(Collections.singletonMap("/foo/*",
                                    new BasicAuthConstraint("CitrusRole")));

    return factory;
}
```

XML

```
<citrus-http:server id="basicAuthHttpServer"
                port="8080"
                auto-start="true"
                security-handler="securityHandler"/>

<bean id="securityHandler"
class="org.citrusframework.http.security.SecurityHandlerFactory">
    <property name="users">
        <list>
            <bean class="org.citrusframework.http.security.User">
                <property name="name" value="citrus"/>
                <property name="password" value="secret"/>
                <property name="roles" value="CitrusRole"/>
            </bean>
        </list>
    </property>
    <property name="constraints">
        <map>
            <entry key="/foo/*">
                <bean class="org.citrusframework.http.security.BasicAuthConstraint">
                    <constructor-arg value="CitrusRole"/>
                </bean>
            </entry>
        </map>
    </property>
</bean>
```

We have set a security handler on the server web container with a constraint on all resources with `/foo/*`. Following from that the server requires basic authentication for these resources. The granted users and roles are specified within the security handler bean definition. Connecting clients have to set the basic auth Http header properly using the correct user and role for accessing the Citrus server now.

You can customize the security handler for your very specific needs (e.g. load users and roles with JDBC from a database). Just have a look at the code base and inspect the settings and properties offered by the security handler interface.



This mechanism is not restricted to basic authentication only. With other settings you can also set up digest or form-based authentication constraints very easy.

14.11. Http client SSL

You can use SSL when establishing a client connection with the server. The Citrus Http client supports a secure connection with these settings.

Java

```
@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
        .requestUrl("https://localhost:8080/test")
        .secured(HttpSecureConnection.ssl())
        .build();
}
```

XML

```
<citrus-http:client id="httpClient"
                    request-url="http://localhost:8080/test"
                    connection-manager="sslConnectionManager"/>

<bean id="sslConnectionManager"
    class="org.citrusframework.http.security.HttpClientConnectionManagerFactory">
    <constructor-arg>
        <bean class="org.citrusframework.http.security.SSLConnection"/>
    </constructor-arg>
</bean>
```

The example Http client uses SSL secure connection. By default, the client will accept any server certificates and hostnames often referred to as trust-all-strategy.

You may want to also use a proper trust store to explicitly accept server certificates. The client by default allows self-signed certificates.

Java

```
@Bean
public HttpClient sslHttpClient() {
    return new HttpClientBuilder()
        .requestUrl("https://localhost:8080/test")
        .secured(HttpSecureConnection.ssl()
            .trustStore("path/to/server.jks", "trustStorePassword"))
        .build();
}
```

XML

```
<citrus-http:client id="sslHttpClient"
                    request-url="http://localhost:8080/test"
                    connection-manager="sslConnectionManager"/>

<bean id="sslConnectionManager"
    class="org.citrusframework.http.security.HttpClientConnectionManagerFactory">
    <constructor-arg>
        <bean class="org.citrusframework.http.security.SSLConnection">
            <constructor-arg value="path/to/server.jks"/>
            <constructor-arg value="trustStorePassword"/>
        </bean>
    </constructor-arg>
</bean>
```

You need to provide a key store file (.jks) and a password to access the certificates located in the store. The client now will do proper SSL handshake with the server in order to use secure connections.

14.12. Http server SSL

The Citrus Http server is able to configure a SSL connector so clients need to use secure connections to access the server resources. The connector

Java

```
@Bean
public HttpServer sslHttpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .secured(8443, HttpSecureConnection.ssl()
            .keyStore("path/to/server.jks", "keyStorePassword"))
        .build();
}
```

XML

```
<citrus-http:server id="sslHttpServer"
                port="8080"
                auto-start="true"
                connector="serverConnectorFactory"/>

<bean id="serverConnectorFactory"
class="org.citrusframework.http.security.HttpServerConnectorFactory">
    <constructor-arg>
        <bean class="org.citrusframework.http.security.SSLConnection">
            <constructor-arg value="path/to/server.jks"/>
            <constructor-arg value="trustStorePassword"/>
            <property name="securePort" value="8443"/>
        </bean>
    </constructor-arg>
</bean>
```

The server now uses a `securePort` and a server certificate. In most cases the certificate in a test environment is self-signed so please make sure to allow self-signed certificates in your clients.

14.13. Http cookies

Cookies hold any kind of information and are saved as test information on the client side. Http servers are able to instruct the client (browser) to save a new cookie with name, value and some attributes. This is usually done with a *"Set-Cookie"* message header set on the server response message. Citrus is able to add those cookie information in a server response.

Java

```
Cookie cookie = new Cookie("Token", "${messageId}");
cookie.setPath("/test/cookie.py");
cookie.setSecure(false);
cookie.setDomain("citrusframework.org");
cookie.setMaxAge(86400);

http().server("httpServer")
    .receive()
    .post()
    .message()
    .body("Some request data")
    .header("Operation", "sayHello");

http().server("httpServer")
    .send()
    .response(HttpStatus.OK.value())
    .message()
    .body("Some response body")
    .header("Operation", "sayHello")
    .cookie(cookie);
```

XML

```
<http:receive-request server="httpServer">
    <http:POST>
        <http:headers>
            <http:header name="Operation" value="getCookie"/>
        </http:headers>
    <http:body>
        <http:data>
            <![CDATA[
            Some request data
            ]]>
        </http:data>
    </http:body>
    </http:POST>
</http:receive-request>

<http:send-response server="httpServer">
    <http:headers status="200" reason-phrase="OK" version="HTTP/1.1">
        <http:header name="Operation" value="getCookie"/>
        <http:cookie name="Token"
                     value="${messageId}"
                     secure="false"
                     domain="citrusframework.org"
                     path="/test/cookie.py"
                     max-age="86400"/>
    </http:headers>
    <http:body>
        <http:data>
            <![CDATA[
            Some response body
            ]]>
        </http:data>
    </http:body>
</http:send-response>
```

The sample above receives an Http request with method POST and some request data. The server response is specified with *Http 200 OK* and some additional cookie information. The cookie is part of the message header specification and gets a name and value as well as several other attributes. This response will result in an Http response with the *"Set-Cookie"* header set:

Set cookie header

```
Set-Cookie:Token=5877643571;Path=/test/cookie.py;Domain=citrusframework.org;Max-
Age=86400
```

As you can see test variables are replaced before the cookie is added to the response. The client now is able to receive the cookie information for validation:

Java

```
Cookie cookie = new Cookie("Token", "${messageId}");
cookie.setPath("/test/cookie.py");
cookie.setSecure(false);
cookie.setDomain("citrusframework.org");
cookie.setMaxAge(86400);

http().client("echoHttpClient")
    .receive()
    .response(HttpStatus.OK.value())
    .message()
    .body("Some response body")
    .header("Operation", "sayHello")
    .cookie(cookie);
```

XML

```
<http:receive-response server="echoHttpClient">
  <http:headers status="200" reason-phrase="OK" version="HTTP/1.1">
  <http:header name="Operation" value="getCookie"/>
  <http:cookie name="Token"
               value="${messageId}"
               secure="false"
               domain="citrusframework.org"
               path="/test/cookie.py"
               max-age="86400"/>
  </http:headers>
  <http:body>
    <http:data>
      <![CDATA[
        Some response body
      ]]>
    </http:data>
  </http:body>
</http:receive-response>
```

Once again the cookie information is added to the header specification. The Citrus message validation will make sure that the cookie information is present with all specified attributes.

In all further actions the client is able to continue to send the cookie information with name and value:

Java DSL

```
http().client("echoHttpClient")
    .send()
    .post()
    .fork(true)
    .message()
    .body("Some other request data")
    .header("Operation", "sayHello")
    .cookie(new Cookie("Token", "${messageId}"));
```

XML

```
<http:send-request client="echoHttpClient" fork="true">
  <http:POST>
    <http:headers>
      <http:header name="Operation" value="sayHello"/>
      <http:cookie name="Token" value="${messageId}"/>
    </http:headers>
    <http:body>
      <http:data>
        <![CDATA[
          Some other request data
        ]]>
      </http:data>
    </http:body>
  </http:POST>
</http:send-request>
```

The cookie now is only specified with name and value as the cookie now goes to the *"Cookie"* request message header.

Cookie token

```
Cookie:Token=5877643571
```

Of course the Citrus Http server can now also validate the cookie information in a request validation:

Java

```
http().server("httpServer")
    .receive()
    .post()
    .message()
    .body("Some other request data")
    .header("Operation", "sayHello")
    .cookie(new Cookie("Token", "${messageId}"));
```

XML

```
<http:receive-request client="httpServer">
  <http:POST>
    <http:headers>
      <http:header name="Operation" value="sayHello"/>
      <http:cookie name="Token" value="${messageId}"/>
    </http:headers>
    <http:body>
      <http:data>
        <![CDATA[
          Some other request data
        ]]>
      </http:data>
    </http:body>
  </http:POST>
</http:receive-request>
```

The Citrus message validation will make sure that the cookie is set in the request with respective name and value.

14.14. Http Gzip compression

Gzip is a very popular compression mechanism for optimizing the message transportation for large content. The Citrus http client and server components support gzip compression out of the box. This means that you only need to set the specific encoding headers in your http request/response message.

Accept-Encoding=gzip

Setting for clients when requesting gzip compressed response content. The Http server must support gzip compression then in order to provide the response as zipped byte stream. The Citrus http server component automatically recognizes this header in a request and applies gzip compression to the response.

Content-Encoding=gzip

When an http server sends compressed message content to the client this header is set to **gzip** in order to mark the compression. The Http client must support gzip compression then in order to unzip the message content. The Citrus http client component automatically recognizes this header in a response and applies gzip unzip logic before passing the message to the test case.

The Citrus client and server automatically take care of gzip compression when those headers are set. In the test case you do not need to zip or unzip the content then as it is automatically done before.

This means that you can request gzipped content from a server with just adding the message header **Accept-Encoding** in your http request operation.

Java

```
http().client("gzipClient")
    .send()
    .post()
    .message()
    .body("Some other request data")
    .contentType("text/html")
    .header("Accept-Encoding", "gzip")
    .header("Accept", "text/plain");

http().client("gzipClient")
    .receive()
    .response(HttpStatus.OK.value())
    .message()
    .type(MessageType.PLAINTEXT)
    .contentType("text/plain")
    .body("${text}");
```

XML

```
<http:send-request client="gzipClient">
  <http:POST>
    <http:headers content-type="text/html">
      <http:header name="Accept-Encoding" value="gzip"/>
      <http:header name="Accept" value="text/plain"/>
    </http:headers>
  </http:POST>
</http:send-request>

<http:receive-response client="gzipClient">
    <http:headers status="200" reason-phrase="OK">
      <http:header name="Content-Type" value="text/plain"/>
    </http:headers>
    <http:body type="plaintext">
      <http:data>${text}</http:data>
    </http:body>
</http:receive-response>
```

On the server side if we receive a message and the response should be compressed with Gzip we just have to set the **Content-Encoding** header in the response operation.

Java

```
http().server("httpServer")
    .receive()
    .post()
    .message()
    .body("Some other request data")
    .contentType("text/html")
    .header("Accept-Encoding", "gzip")
    .header("Accept", "text/plain");

http().server("httpServer")
    .send()
    .response(HttpStatus.OK.value())
    .message()
    .contentType("text/plain")
    .body("${text}");
```

XML

```
<http:receive-request server="httpServer">
    <http:POST path="/echo">
      <http:headers>
        <http:header name="Content-Type" value="text/html"/>
        <http:header name="Accept-Encoding" value="gzip"/>
        <http:header name="Accept" value="text/plain"/>
      </http:headers>
    </http:POST>
</http:receive-request>

<http:send-response server="httpServer">
    <http:headers status="200" reason-phrase="OK">
      <http:header name="Content-Encoding" value="gzip"/>
      <http:header name="Content-Type" value="text/plain"/>
    </http:headers>
    <http:body>
      <http:data>${text}</http:data>
    </http:body>
</http:send-response>
```

So the Citrus server will automatically add gzip compression to the response for us.

Of course you can also send gzipped content as a client. Then you would just set the **Content-Encoding** header to **gzip** in your request. The client will automatically apply compression for you.

14.15. Http servlet filters

The Citrus http server component supports custom servlet filters that take part in handling an incoming request/response communication. This might be useful when customizing the basic

server behavior such as custom zip/unzip mechanisms. The custom servlet filters are referenced in the http server component as follows:

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .filters(filters())
        .filterMappings(filterMappings())
        .build();
}

public Map<String, Filter> filters() {
    Map<String, Filter> filters = new HashMap<>();

    filters.put("request-caching-filter", new RequestCachingServletFilter());
    filters.put("gzip-filter", new GzipServletFilter());

    return filters;
}

public Map<String, String> filterMappings() {
    Map<String, String> filterMappings = new HashMap<>();

    filterMappings.put("request-caching-filter", "/*");
    filterMappings.put("gzip-filter", "/gzip/*");

    return filterMappings;
}
```

XML

```
<citrus-http:server id="httpServer"
                    port="8080"
                    filters="filters"
                    filter-mappings="filterMappings"/>

<util:map id="filters">
    <entry key="request-caching-filter">
        <bean class="org.citrusframework.http.servlet.RequestCachingServletFilter"/>
    </entry>
    <entry key="gzip-filter">
        <bean class="org.citrusframework.http.servlet.GzipServletFilter"/>
    </entry>
</util:map>

<util:map id="filterMappings">
    <entry key="request-caching-filter" value="/*"/>
    <entry key="gzip-filter" value="/gzip/*"/>
</util:map>
```

The map of filters are specified as normal Spring configuration entries. The server component uses the attribute `filters` to reference a set of custom servlet filters. The map holds one to many servlet filter beans each given a name that is also referenced in the respective servlet mappings. The servlet mappings specify when to apply those filters.

This way you can set a very custom servlet filter chain for each request/response communication. As usual the filter implementations can participate in the request and response handling process.

Citrus provides several default servlet implementations that are automatically added to each http server component these implementations are:

org.citrusframework.http.servlet.RequestCachingServletFilter

caches incoming request data so input streams can be read multiple times during request processing (important when request logging is enabled)

org.citrusframework.http.servlet.GzipServletFilter

applies Gzip compression when according headers are set and client explicitly asks for compressed request/response communication

By the time you define some custom servlet filters or mappings to the server component Citrus will not apply default servlet filters. This means you always need to construct the whole servlet filter chain including default servlet filters mentioned above.

14.16. Http servlet context customization

The Citrus Http server uses Spring application context loading on startup. For high customizations you can provide a custom servlet context file which holds all custom configurations as Spring beans for the server. Here is a sample servlet context with some basic Spring MVC components and the

central HttpMessageController which is responsible for handling incoming requests (GET, PUT, DELETE, POST, etc.).

Java

```
@Bean
public RequestMappingHandlerMapping citrusHandlerMapping() {
    return new RequestMappingHandlerMapping();
}

@Bean
public RequestMappingHandlerAdapter citrusMethodHandlerAdapter() {
    RequestMappingHandlerAdapter adapter = new RequestMappingHandlerAdapter();


adapter.setMessageConverters(Collections.singletonList(citrusHttpMessageConverter()));

    return adapter;
}

@Bean
public DelegatingHttpEntityMessageConverter citrusHttpMessageConverter() {
    return new DelegatingHttpEntityMessageConverter();
}

@Bean
public HttpMessageController citrusHttpMessageController() {
    HttpMessageController controller = new HttpMessageController();

    controller.setEndpointAdapter(new EmptyResponseEndpointAdapter());

    return controller;
}
```

XML

```
<bean id="citrusHandlerMapping"
class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapp
ing"/>

<bean id="citrusMethodHandlerAdapter"
class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdap
ter">
  <property name="messageConverters">
    <util:list id="converters">
      <ref bean="citrusHttpMessageConverter"/>
    </util:list>
  </property>
</bean>

<bean id="citrusHttpMessageConverter"
class="org.citrusframework.http.message.DelegatingHttpEntityMessageConverter"/>

<bean id="citrusHttpMessageController"
class="org.citrusframework.http.controller.HttpMessageController">
  <property name="endpointAdapter">
      <bean

class="org.citrusframework.base.endpoint.adapter.EmptyResponseEndpointAdapter"/>
  </property>
</bean>
```

The beans above are responsible for proper Http server configuration. In general you do not need to adjust those beans, but we have the possibility to do so which gives us a great customization and extension points. The important part is the endpoint adapter definition inside the HttpMessageController. Once a client request was accepted the adapter is responsible for generating a proper response to the client.

You can add the custom servlet context as file resource to the Citrus Http server component. Just use the **context-config-location** attribute as follows:

Java

```
@Bean
public HttpServer httpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .contextConfigLocation("classpath:org/citrusframework/http/custom-servlet-
context.xml")
        .build();
}
```

XML

```
<citrus-http:server id="helloHttpServer"
      port="8080"
      auto-start="true"
      context-config-location="classpath:org/citrusframework/http/custom-servlet-
context.xml"/>
```

Chapter 15. GraphQL support

GraphQL APIs are usually served over HTTP: a client posts a query or mutation with variables and the server answers with `data` and/or `errors`. Citrus is able to call GraphQL APIs as a client and to simulate GraphQL APIs as a server. GraphQL endpoints run on top of the Citrus Http client and server, so every Http setting (request factory, timeouts, interceptors, TLS, authentication) keeps working.



The GraphQL support is located in the module **citrus-graphql**. The module builds on Spring GraphQL, so adding it brings `spring-graphql`, `graphql-java` and `reactor-core` to your test classpath. Projects that already use Spring GraphQL should keep both on compatible versions.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-graphql</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The GraphQL test actions are available as `graphql()` in the Citrus Java DSL, as `<graphql>` element in XML tests and as `graphql:` in YAML tests.

15.1. GraphQL client and server

A GraphQL client needs the request URL of the API. The GraphQL path defaults to `/graphql`. When the request URL already has a path of its own (e.g. `http://localhost:8080/api/graphql`) the URL is used as it is.

Java

```
@BindToRegistry
private final GraphQlServer bookServer = GraphQlEndpoints.graphql().server()
        .port(8080)
        .autoStart(true)
        .schema("classpath:graphql/schema.graphqls")
        .build();

@BindToRegistry
private final GraphQlClient bookClient = GraphQlEndpoints.graphql().client()
        .requestUrl("http://localhost:8080")
        .schema("classpath:graphql/schema.graphqls")
        .build();
```

Annotations

```
@CitrusEndpoint
@GraphQlServerConfig(port = 8080, autoStart = true, schema =
"classpath:graphql/schema.graphqls")
private GraphQlServer bookServer;

@CitrusEndpoint
@GraphQlClientConfig(requestUrl = "http://localhost:8080", schema =
"classpath:graphql/schema.graphqls")
private GraphQlClient bookClient;
```

XML

```
<endpoints>
  <graphql>
    <server name="bookServer" port="8080" auto-start="true"/>
  </graphql>
  <graphql>
    <client name="bookClient" request-url="http://localhost:8080"/>
  </graphql>
</endpoints>
```

YAML

```
endpoints:
  - graphql:
      server:
        name: "bookServer"
        port: 8080
        autoStart: true
  - graphql:
      client:
        name: "bookClient"
        requestUrl: "http://localhost:8080"
```

Clients can also be created from endpoint URIs, `graphql://localhost:8080/graphql` for Http and `graphqls://api.example.com/graphql` for Https. URI parameters such as `strict=false` or `requestMethod=GET` set the respective client settings. The endpoint catalog offers the builders as `CitrusEndpoints.graphql().client()` and `CitrusEndpoints.graphql().server()`.

Setting	Default	Description
path	`/graphql`	GraphQL path. On the client it is appended to the request URL unless the URL has a path of its own.
schema	none	One or more SDL resources. Queries are validated against the merged schema (see Schema validation).

Setting	Default	Description
strict	`true`	GraphQL checks (see Strict checks). Each action can override the setting.
requestMethod	`POST`	Client only: `POST` with a JSON body or `GET` with query parameters.

All other client and server settings are the ones of the Citrus Http client and server.



In a test class that implements `TestActionSupport` the method `graphql()` refers to the GraphQL test actions. Use the fully qualified `GraphQlEndpoints.graphql()` to create endpoints there.

15.2. Calling a GraphQL API

The client sends a GraphQL document with an optional operation name and variables. The document is given inline or as a resource. Citrus expressions such as `${id}` are resolved in the document, the operation name and the variables.

Java

```
@CitrusTest
public void readBook() {
    variable("id", "42");

    $(graphql().client(bookClient)
        .send()
        .query("query Book($id: ID!) { book(id: $id) { title } }")
        .operationName("Book")
        .variable("id", "${id}"));

    $(graphql().client(bookClient)
        .receive()
        .data("$.book.title", "Dune")
        .message()
        .status(200));
}
```

XML

```
<graphql client="bookClient">
  <send-request operation-name="Book">
    <query>query Book($id: ID!) { book(id: $id) { title } }</query>
    <variable name="id" value="${id}"/>
  </send-request>
</graphql>

<graphql client="bookClient">
  <receive-response status="200">
    <data path="$.book.title" value="Dune"/>
  </receive-response>
</graphql>
```

YAML

```
- graphql:
    client: "bookClient"
    sendRequest:
      operationName: "Book"
      query: "query Book($id: ID!) { book(id: $id) { title } }"
      variable:
        - name: "id"
          value: "${id}"
- graphql:
    client: "bookClient"
    receiveResponse:
      status: 200
      data:
        - path: "$.book.title"
          value: "Dune"
```

The request is a POST with the JSON body `{"query": …, "operationName": …, "variables": …}`, the `Content-Type: application/json` header and `Accept: application/graphql-response+json, application/json`. Use `queryResource("classpath:graphql/book.graphql")` (XML: `<query file="…"/>`, YAML: `queryResource`) to load the document from a file.

Queries can also be sent with Http GET. The client then encodes `query`, `operationName` and `variables` as query parameters. Only queries can be sent with GET, mutations are rejected before anything is sent.

Java

```
$(graphql().client(bookClient)
    .send()
    .get()
    .query("query Books($first: Int) { books(first: $first) { title } }")
    .variable("first", 10));
```

XML

```
<graphql client="bookClient">
  <send-request method="GET">
    <query>query Books($first: Int) { books(first: $first) { title } }</query>
    <variables>{"first": 10}</variables>
  </send-request>
</graphql>
```



With GET the variables are part of the request URI and therefore appear in logs and reports.

On the response, `data(path, value)` checks a value by JSONPath relative to `$.data` and `data(json)` compares the whole `data` object. Validation matchers and `@ignore@` work as in any JSON validation, and so do `jsonPath()` validations on the whole response, variable extraction, header and status checks. Check the Http status with `.message().status(…)`.



`method(…)`, `path(…)` and other Http builder methods return the Http builder type. Call the GraphQL methods such as `query(…)` or `operationName(…)` first in a chain.

15.2.1. Variables and their JSON type

Variables keep their JSON type: `variable("first", 10)` sends the number `10`, `variable("draft", true)` the boolean `true` and strings stay strings. A value taken from a Citrus variable is a string, so `variable("first", "${n}")` sends `"10"`, which a server expecting `Int` rejects. To send a typed value from a Citrus variable use the variables JSON object: it is resolved first and parsed afterwards.

```
$(graphql().client(bookClient)
    .send()
    .query("query Books($first: Int) { books(first: $first) { title } }")
    .variables("{\"first\": ${n}}"));
```

In XML and YAML the single `variable` values are strings; use `variables` for numbers, booleans, lists and objects.

15.3. GraphQL errors

GraphQL servers usually answer with Http 200 even when the `errors` array is filled. Citrus therefore fails a GraphQL receive action when the response contains errors the test did not expect, whatever the Http status:

```
GraphQL response contains 1 unexpected error: [NOT_FOUND] Book not found at book - use
expectError(...) or expectErrors() if errors are intended
```

A test that expects errors says so. `expectErrors()` passes when at least one error is present.

`expectError(…)` passes when any error of the response matches every field that is set: `message`, `path` and the `extensions.code`. The fields support validation matchers, and the expectation can be repeated. When nothing matches, the failure lists all received errors. Data checks and error expectations can be combined for partial results.

Java

```
$(graphql().client(bookClient)
    .receive()
    .expectError(error().code("NOT_FOUND").message("@contains('not
found')@").path("book"))
    .data("{\"book\": null}"));
```

XML

```
<graphql client="bookClient">
  <receive-response>
    <expect-error code="NOT_FOUND" message="@contains('not found')@" path="book"/>
    <data>{"book": null}</data>
  </receive-response>
</graphql>
```

YAML

```
- graphql:
    client: "bookClient"
    receiveResponse:
      expectError:
        - code: "NOT_FOUND"
          message: "@contains('not found')@"
          path: "book"
      data:
        - json: '{"book": null}'
```

The `error()` factory is `org.citrusframework.api.graphql.GraphQlError.error()`. Paths are given as segments in Java (`path("books", 1, "title")`) and as string (`books[1].title`) in XML and YAML.

15.4. Simulating a GraphQL API

The GraphQL server receives the operations of the client under test and replies with data and/or errors. GET and POST requests look the same to the test: a GET request is presented as the equivalent JSON body, the Http method and the original query string stay available in the `citrus_http_method` and `citrus_http_query_params` headers.

Java

```
$(graphql().server(bookServer)
    .receive()
    .operationName("Book")
    .query("query Book($id: ID!) { book(id: $id) { title } }")
    .variable("id", "@isNumber()@"));

$(graphql().server(bookServer)
    .send()
    .data("{\"book\": {\"title\": \"Dune\", \"author\": null}}")
    .error(error().message("Author unavailable").path("book",
"author").code("PARTIAL")));
```

XML

```
<graphql server="bookServer">
  <receive-request operation-name="Book">
    <query>query Book($id: ID!) { book(id: $id) { title } }</query>
    <variable name="id" value="@isNumber()@"/>
  </receive-request>
</graphql>

<graphql server="bookServer">
  <send-response>
    <data>{"book": {"title": "Dune", "author": null}}</data>
    <error message="Author unavailable" path="book.author" code="PARTIAL"/>
  </send-response>
</graphql>
```

YAML

```
- graphql:
    server: "bookServer"
    receiveRequest:
      operationName: "Book"
      query: "query Book($id: ID!) { book(id: $id) { title } }"
      variable:
        - name: "id"
          value: "@isNumber()@"
- graphql:
    server: "bookServer"
    sendResponse:
      data: '{"book": {"title": "Dune", "author": null}}'
      error:
        - message: "Author unavailable"
          path: "book.author"
          code: "PARTIAL"
```

The expected operation name is the name of the operation the request executes: the given `operationName`, or the name of the single operation in the document when the client omits it. Queries are compared ignoring whitespace, comments and formatting. Variables that the test does not check are ignored; `variables(json)` compares the whole object.

The reply has Http status 200 unless the test sets another one. Its content type is `application/graphql-response+json` when the client accepts it and `application/json` otherwise; a content type set on the message wins.

15.5. Schema validation

When an endpoint has a schema, GraphQL documents are validated against it: on the client before the request is sent, on the server when the request arrives. The schema may be split across several SDL files; types and type extensions are merged. Custom scalars need no configuration. The schema is loaded once per endpoint, a missing file or an SDL syntax error fails the first action that needs the schema with the file name and position.

```
GraphQL document does not match the schema: (line 1, column 17) Validation error
(FieldUndefined@[book/isbn]) : Field 'isbn' in type 'Book' is undefined
```



Federation SDL that uses directives such as `@key` without declaring them does not load. Add the directive declarations to the schema files or disable the checks for the action with `strict(false)`.

15.6. Strict checks

The `strict` setting enforces the GraphQL rules on everything Citrus sends or receives. It is enabled by default, set on the endpoint and can be overridden per action with `strict(false)` (XML and YAML: `strict="false"` / `strict: false`).

Action	strict (default)	not strict
Client send	Parses the document, checks the operation name and the schema, rejects mutations over GET. Fails before anything is sent.	Sends the document as written, e.g. to test the server's own validation.
Client receive	The body must be a JSON object with `data` and/or `errors`, every error with a `message`.	No shape check, e.g. for an Html error page of a proxy; check the Http status instead.
Server receive	Parses the incoming document and checks it against the schema.	Accepts any request; the raw query can be checked with validation matchers.
Server send	The reply must have `data` and/or `errors`, every error with a `message`.	Sends the message body as set, e.g. to test how the client under test copes with a malformed reply.

Unexpected GraphQL errors fail a client receive in both modes.

Java

```
$(graphql().client(bookClient)
    .send()
    .strict(false)
    .query("{ book(id: 1) { isbn } }"));

$(graphql().client(bookClient)
    .receive()
    .expectError(error().message("@contains('isbn')@"))
    .message()
    .status(400));
```

XML

```
<graphql client="bookClient">
  <send-request strict="false">
    <query>{ book(id: 1) { isbn } }</query>
  </send-request>
</graphql>

<graphql client="bookClient">
  <receive-response status="400">
    <expect-error message="@contains('isbn')@"/>
  </receive-response>
</graphql>
```

15.7. GraphQL message headers

Header	Description
`citrus_graphql_operation_name`	Name of the executed operation, on sent and received requests.
`citrus_graphql_operation_type`	`query` or `mutation`, set when the document was parsed (strict checks enabled).
`citrus_graphql_kind`	`request` or `response`, set on messages received through GraphQL actions.

The headers can be validated and extracted like any other message header.

Chapter 16. SOAP WebServices

SOAP Web Services over HTTP is a widely used communication scenario in modern enterprise applications. A SOAP Web Service client is posting a SOAP request via HTTP to a server. SOAP via HTTP is a synchronous message protocol by default so the client is waiting synchronously for the response message. Citrus provides both SOAP client and server components in order to meet both directions of this scenario. The components used are very similar to the HTTP components that were discussed in the sections before.



The SOAP WebService components in Citrus are kept in a separate Maven module. So you should add the module as Maven dependency to your project accordingly.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-ws</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

16.1. SOAP client

Citrus is able to form a proper SOAP request in order to pass it to the server via HTTP and validate the respective SOAP response message. Let us see how a message client for SOAP looks like in the Spring configuration:

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .timeout(60000L)
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .timeout(60000L)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="soapClient"
                        request-url="http://localhost:8090/test"
                        timeout="60000"/>
</beans>
```



In order to use the Citrus SOAP WebService support in a Spring beans XML application context you need to include the specific XML configuration schema provided by Citrus. See the `citrus-ws` prefix - in Spring XML configuration sample above.

The client component uses the **request-url** in order to access the server resource. The client will automatically build a proper SOAP request message including the SOAP envelope, SOAP header and the message payload as SOAP body. This means that you as a tester do not care about SOAP envelope specific logic in the test case. The client endpoint component saves the synchronous SOAP response so the test case can receive this message with a normal receive test action.

In detail you as a tester just send and receive using the same client endpoint reference just as you would do with a synchronous JMS or channel communication. In case no response message is available in time according to the timeout settings Citrus raises a timeout error and the test will fail.



The SOAP client component uses a SoapMessageFactory implementation in order to create the SOAP messages. This is a Spring bean added to the Citrus Spring application context. Spring offers several reference implementations as message factories so you can choose one of them (e.g. for SOAP 1.1 or 1.2 implementations).

Citrus Bean

```
@BindToRegistry
public SaajSoapMessageFactory messageFactory() {
    return new SaajSoapMessageFactory();
}

@BindToRegistry
public SaajSoapMessageFactory soap12MessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_12);
    return messageFactory;
}
```

Spring Bean

```
@Bean
public SaajSoapMessageFactory messageFactory() {
    return new SaajSoapMessageFactory();
}

@Bean
public SaajSoapMessageFactory soap12MessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_12);
    return messageFactory;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Default SOAP Message Factory (SOAP 1.1) -->
    <bean id="messageFactory"
class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory"/>

    <!-- SOAP 1.2 Message Factory -->
    <bean id="soap12MessageFactory"
class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory">
      <property name="soapVersion">
        <util:constant static-
field="org.springframework.ws.soap.SoapVersion.SOAP_12"/>
      </property>
    </bean>
</beans>
```

By default, Citrus will search for a bean with id **'messageFactory'** . In case you intend to use different identifiers you need to tell the SOAP client component which message factory to use:

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageFactory(soap12MessageFactory())
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageFactory(soap12MessageFactory())
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="soapClient"
         request-url="http://localhost:8090/test"
         message-factory="soap12MessageFactory"/>
</beans>
```



Up to now we have used a static endpoint request url for the SOAP message sender. Besides that we can use dynamic endpoint uri in configuration. We just use an endpoint uri resolver instead of the static request url like this:

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageFactory(soap12MessageFactory())
            .endpointResolver(dynamicEndpointResolver())
            .build();
}

@BindToRegistry
public DynamicEndpointUriResolver dynamicEndpointResolver() {
    return new DynamicEndpointUriResolver();
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageFactory(soap12MessageFactory())
            .endpointResolver(dynamicEndpointResolver())
            .build();
}

@Bean
public DynamicEndpointUriResolver dynamicEndpointResolver() {
    return new DynamicEndpointUriResolver();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="soapClient"
              request-url="http://localhost:8090/test"
             endpoint-resolver="dynamicEndpointResolver"
             message-factory="soap12MessageFactory"/>

  <bean id="dynamicEndpointResolver"
       class="org.citrusframework.endpoint.resolver.DynamicEndpointUriResolver"/>
</beans>
```

The **dynamicEndpointResolver** bean must implement the EndpointUriResolver interface in order to resolve dynamic endpoint uri values. Citrus offers a default implementation, the **DynamicEndpointUriResolver**, which uses a specific message header for setting the dynamic endpoint uri for each message. The message header needs to specify the header **citrus_endpoint_uri** with a valid request uri. Just like this:

Java

```
.header("citrus_endpoint_uri", "http://localhost:${port}/${context}")
```

XML

```
<headers>
  <header name="citrus_endpoint_uri"
          value="http://localhost:${port}/${context}" />
</headers>
```

YAML

```
headers:
  - name: citrus_endpoint_uri
    value: "http://localhost:${port}/${context}"
```

Spring XML

```
<header>
  <element name="citrus_endpoint_uri"
           value="http://localhost:${port}/${context}" />
</header>
```

As you can see you can use dynamic test variables then in order to build the request uri to use. The SOAP client evaluates the endpoint uri header and sends the message to this server resource. You can use a different uri value then in different test cases and send actions.

16.2. SOAP client interceptors

The client component is able to add custom interceptors that participate in the request/response processing. The interceptors need to implement the common interface **org.springframework.ws.client.support.interceptor.ClientInterceptor**.

Citrus Bean

```
@BindToRegistry
public WebServiceClient secureSoapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/services/ws/todolist")
            .interceptors(clientInterceptors())
            .build();
}

public List<ClientInterceptor> clientInterceptors() {
    Wss4jSecurityInterceptor wss4jSecurityInterceptor = new
Wss4jSecurityInterceptor();
    wss4jSecurityInterceptor.setSecurementActions("Timestamp UsernameToken");
    wss4jSecurityInterceptor.setSecurementUsername("admin");
    wss4jSecurityInterceptor.setSecurementPassword("secret");
    return Arrays.asList(wss4jSecurityInterceptor, new LoggingClientInterceptor());
}
```

Spring Bean

```
@Bean
public WebServiceClient secureSoapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/services/ws/todolist")
            .interceptors(clientInterceptors())
            .build();
}

public List<ClientInterceptor> clientInterceptors() {
    Wss4jSecurityInterceptor wss4jSecurityInterceptor = new
Wss4jSecurityInterceptor();
    wss4jSecurityInterceptor.setSecurementActions("Timestamp UsernameToken");
    wss4jSecurityInterceptor.setSecurementUsername("admin");
    wss4jSecurityInterceptor.setSecurementPassword("secret");
    return Arrays.asList(wss4jSecurityInterceptor, new LoggingClientInterceptor());
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="secureSoapClient"
                    request-url="http://localhost:8080/services/ws/todolist"
                    interceptors="clientInterceptors"/>

  <util:list id="clientInterceptors">
    <bean class="org.springframework.ws.soap.security.wss4j.Wss4jSecurityInterceptor">
      <property name="securementActions" value="Timestamp UsernameToken"/>
      <property name="securementUsername" value="admin"/>
      <property name="securementPassword" value="secret"/>
    </bean>
    <bean class="org.citrusframework.ws.interceptor.LoggingClientInterceptor"/>
  </util:list>
</beans>
```

The sample above adds Wss4J WsSecurity interceptors in order to add security constraints to the request messages.



When customizing the interceptor chain all default interceptors (like logging interceptor) are lost. You need to add these interceptors explicitly as shown with the *org.citrusframework.ws.interceptor.LoggingClientInterceptor* which is able to log request/response messages during communication.

16.3. SOAP server

Every client needs a server to talk to. When receiving SOAP messages we require a web server instance listening on a port. Citrus is using an embedded Jetty server instance in combination with the Spring Web Service API in order to accept SOAP request calls as a server. See how the Citrus SOAP server is configured in the Spring configuration.

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:server id="soapServer"
            port="8080"
            auto-start="true"/>
</beans>
```

The server component is able to start automatically when application starts up. In the example above the server is listening for requests on port ***8080*** . This setup uses the standard connector configuration for the Jetty server. For detailed customization the Citrus Jetty server configuration also supports explicit connector configurations (@connector and @connectors attributes). For more information please see the Jetty connector documentation.

Test cases interact with this server instance via message channels by default. The server component provides an inbound channel that holds incoming request messages. The test case can receive those requests from the channel with a normal receive test action. In a second step the test case can provide a synchronous response message as reply which will be automatically sent back to the calling SOAP client as response.



The figure above shows the basic setup with inbound channel and reply channel. You as a tester should not worry about this too much. By default you as a tester just use the server as synchronous endpoint in your test case. This means that you simply receive a message from the server and send a response back.

Java

```
@CitrusTest
public void soapServerTest() {
    when(receive().endpoint(soapServer)
            .message()
                .body("""
                    [...]
                """)
    );

    then(send().endpoint(soapServer)
            .message()
                .body("""
                    [...]
                """)
    );
}
```

XML

```
<test name="SoapServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="soapServer">
        <message>
            <body>
                [...]
            </body>
        </message>
      </receive>
      <send endpoint="soapServer">
        <message>
            <body>
                [...]
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SoapServerTest
actions:
  - receive:
      endpoint: "soapServer"
      message:
        body: |
          [...]
  - send:
      endpoint: "soapServer"
      message:
        body: |
          [...]
```

Spring XML

```
<testcase name="soapServerTest">
    <actions>
        <receive endpoint="soapServer">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </receive>

        <send endpoint="soapServer">
            <message>
                <data>
                  [...]
                </data>
            </message>
        </send>
    </actions>
</testcase>
```

As you can see we reference the server id in both receive and send actions. The Citrus server instance will automatically send the response back to the calling client. In most cases this is what you need to simulate a SOAP server instance in Citrus. Of course we have some more customization possibilities that we will go over later on. These customizations are optional so you can also skip the next description on endpoint adapters if you are happy with just what you have learned about the SOAP server component in Citrus.

Just like the HTTP server component the SOAP server component by default uses the channel endpoint adapter in order to forward all incoming requests to an in-memory message channel. This is done completely behind the scenes. The Citrus configuration has become a lot easier here so you do not have to configure this by default. When nothing else is set the test case does not worry about that setting on the server and just uses the server id reference as synchronous endpoint.



The default channel endpoint adapter automatically creates an inbound message channel where incoming messages are stored too internally. So if you need to clean up a server that has already stored some incoming messages you can do this easily by purging the internal message channel. The message channel follows a naming convention **{serverName}.inbound** where **{serverName}** is the Spring bean name of the Citrus server endpoint component. If you purge this internal channel in a before test nature you are sure that obsolete messages on a server instance get purged before each test is executed.

However, we do not want to lose the great extendability and customizing capabilities of the Citrus server component. This is why you can optionally define the endpoint adapter implementation used by the Citrus SOAP server. We provide several message endpoint adapter implementations for different simulation strategies. With these endpoint adapters you should be able to generate proper SOAP response messages for the client in various ways. Before we have a closer look at the different adapter implementations we want to show how you can set a custom endpoint adapter on the server component.

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .endpointAdapter(emptyResponseEndpointAdapter())
            .build();
}

@BindToRegistry
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .endpointAdapter(emptyResponseEndpointAdapter())
            .build();
}

@Bean
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/ws/config
           http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:server id="soapServer"
        port="8080"
        auto-start="true"
        endpoint-adapter="emptyResponseEndpointAdapter"/>

  <citrus:empty-response-adapter id="emptyResponseEndpointAdapter"/>
</beans>
```

With this endpoint adapter configuration above we change the Citrus server behavior from scratch. Now the server automatically sends back an empty SOAP response message every time. Setting a custom endpoint adapter implementation with custom logic is as easy as defining a custom endpoint adapter Spring bean and reference it in the server attribute. You can read more about endpoint adapters in endpoint-adapter.

16.4. SOAP send and receive

Citrus provides test actions for sending and receiving messages of all kind. Different message content and different message transports are available to these send and receive actions. When using SOAP message transport we might need to set special information on those messages. These are special SOAP headers, SOAP faults and so on. So we have created a special SOAP test actions for all your SOAP related send and receive operations.



In a Spring XML DSL test the **ws:** Citrus namespace is added to your test case, so you can use special send and receive operations in the test.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .send()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                    [...]
                """)
    );

    then(soap()
        .server(soapServer)
        .receive()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                    [...]
                """)
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <send-request>
          <message soap-action="MySoapService/sayHello">
            <body>
              [...]
            </body>
          </message>
        </send-request>
      </soap>

      <soap server="soapServer">
        <receive-request>
          <message soap-action="MySoapService/sayHello">
            <body>
              [...]
            </body>
          </message>
        </receive-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      sendRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              [...]
  - soap:
      server: "soapServer"
      receiveRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:send endpoint="soapClient" soap-action="MySoapService/sayHello">
            <message>
                [...]
            </message>
        </ws:send>

        <ws:receive endpoint="soapServer" soap-action="MySoapService/sayHello">
            <message>
                [...]
            </message>
        </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

The special SOAP actions include send/receive actions for both client and server that simplify the usage with SOAP envelopes.

SOAP client actions

send-request	SOAP client sending SOAP request message content.
receive-response	SOAP client receiving the SOAP response message content.
assert-fault	SOAP client expecting a SOAP fault from the server as a response message.

SOAP server actions

receive-request	SOAP server receiving a SOAP request message.
send-response	SOAP server sending a SOAP response message.
send-fault	SOAP server sending a SOAP fault message as a response.

The special SOAP related send and receive actions can coexist with normal Citrus actions in a test. In fact, you can mix those action types in your test case.

In Spring XML DSL test cases all SOAP test actions use a special namespace `http://www.citrusframework.org/schema/ws/testcase`.

The Java DSL provides a special SOAP related fluent method API with the **soap()** entry method.

16.5. SOAP headers

SOAP defines several header variations that we discuss in the following sections. First of all we deal with the special **SOAP action** header. In case we need to set this SOAP action header we simply need to use the special ***soap-action*** attribute in our test. The special header key in combination with an underlying SOAP client endpoint component constructs the SOAP action in the SOAP message.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .send()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                    [...]
                """)
    );

    then(soap()
        .server(soapServer)
        .receive()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                    [...]
                """)
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <send-request>
          <message soap-action="MySoapService/sayHello">
            <body>
              [...]
            </body>
          </message>
        </send-request>
      </soap>

      <soap server="soapServer">
        <receive-request>
          <message soap-action="MySoapService/sayHello">
            <body>
              [...]
            </body>
          </message>
        </receive-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      sendRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              [...]
  - soap:
      server: "soapServer"
      receiveRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
      <ws:send endpoint="soapClient" soap-action="MySoapService/sayHello">
        <message>
            [...]
        </message>
      </ws:send>

      <ws:receive endpoint="soapServer" soap-action="MySoapService/sayHello">
        <message>
            [...]
        </message>
      </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

The SOAP action header is added to the message before sending and validated when used in a receive operation.



The **soap-action** attribute is defined in the special SOAP namespace in Citrus. We recommend to use this namespace for all your send and receive operations that deal with SOAP message content. However, you can also set the special SOAP action header when not using the special SOAP namespace: Just set this header in your test action:

Java

```
.header("citrus_soap_action", "sayHello")
```

XML

```
<headers>
  <header name="citrus_soap_action"
          value="sayHello" />
</headers>
```

YAML

```
headers:
  - name: citrus_soap_action
    value: "sayHello"
```

Spring XML

```
<header>
    <element name="{http://citrusframework.org/sayHello}h1:Operation"
value="sayHello"/>
</header>
```

Secondly a SOAP message is able to contain customized SOAP headers. These are key-value pairs where the key is a qualified name (QName) and the value a normal String value.

Java

```
.header("{http://citrusframework.org/sayHello}h1:Operation", "sayHello")
.header("{http://citrusframework.org/sayHello}h1:Request", "HelloRequest")
```

XML

```
<headers>
  <header name="{http://citrusframework.org/sayHello}h1:Operation"
          value="sayHello" />
  <header name="{http://citrusframework.org/sayHello}h1:Request"
          value="HelloRequest" />
</headers>
```

YAML

```
headers:
  - name: "{http://citrusframework.org/sayHello}h1:Operation"
    value: "sayHello"
  - name: "{http://citrusframework.org/sayHello}h1:Request"
    value: "HelloRequest"
```

Spring XML

```
<header>
    <element name="{http://citrusframework.org/sayHello}h1:Operation"
value="sayHello"/>
    <element name="{http://citrusframework.org/sayHello}h1:Request"
value="HelloRequest"/>
</header>
```

The key is defined as qualified QName character sequence which has a mandatory XML namespace and a prefix along with a header name. Last but not least a SOAP header can contain whole XML fragment values. The next example shows how to set these XML fragments as SOAP header in Citrus:

Java

```
.header("""
  <User xmlns="http://citrusframework.org/schemas/sayHello">
      <UserId>123456789</UserId>
      <Handshake>S123456789</Handshake>
  </User>
""")
```

XML

```
<headers>
  <header>
    <data>
      <![CDATA[
          <User xmlns="http://citrusframework.org/schemas/sayHello">
              <UserId>123456789</UserId>
              <Handshake>S123456789</Handshake>
          </User>
      ]]>
    </data>
  </header>
</headers>
```

YAML

```
headers:
  - data: |
      <User xmlns="http://citrusframework.org/schemas/sayHello">
          <UserId>123456789</UserId>
          <Handshake>S123456789</Handshake>
      </User>
```

Spring XML

```
<header>
    <data>
      <![CDATA[
          <User xmlns="http://citrusframework.org/schemas/sayHello">
              <UserId>123456789</UserId>
              <Handshake>S123456789</Handshake>
          </User>
      ]]>
    </data>
</header>
```

You can also use external file resources to set this SOAP header XML fragment as shown in this last example code:

Java

```
.header(Resources.of("classpath:request-soap-header.xml"))
```

XML

```
<headers>
  <header file="classpath:request-soap-header.xml"/>
</headers>
```

YAML

```
headers:
  - file: "classpath:request-soap-header.xml"
```

Spring XML

```
<header>
    <resource file="classpath:request-soap-header.xml"/>
</header>
```

This completes the SOAP header possibilities for sending SOAP messages with Citrus. Of course you can also use these variants in SOAP message header validation. You define expected SOAP headers, SOAP action and XML fragments and Citrus will match incoming requests to that. Just use **citrus_soap_action** header key in your receiving message action and you validate this SOAP header accordingly.

When validating SOAP header XML fragments you need to define the whole XML header fragment as expected header data like this:

Java

```
@CitrusTest
public void soapHeaderTest() {
    then(soap()
        .server(soapServer)
        .receive()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                <ResponseMessage xmlns="http://citrusframework.org/schema">
                  <resultCode>OK</resultCode>
                </ResponseMessage>
                """)
                .header("""
                <SOAP-ENV:Header
                    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
                    <customHeader xmlns="http://citrusframework.org/headerschema">
                        <correlationId>${correlationId}</correlationId>
                        <applicationId>${applicationId}</applicationId>
                        <trackingId>${trackingId}</trackingId>
                        <serviceId>${serviceId}</serviceId>
                        <interfaceVersion>1.0</interfaceVersion>
                        <timestamp>@ignore@</timestamp>
                    </customHeader>
                </SOAP-ENV:Header>
                """)
                .header("citrus_soap_action", "doResponse")
    );
}
```

XML

```
<test name="SoapHeaderTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <receive-request>
          <message soap-action="MySoapService/sayHello">
            <headers>
              <header>
                <data>
                <![CDATA[
                <SOAP-ENV:Header
                    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
                    <customHeader xmlns="http://citrusframework.org/headerschema">
                        <correlationId>${correlationId}</correlationId>
                        <applicationId>${applicationId}</applicationId>
                        <trackingId>${trackingId}</trackingId>
                        <serviceId>${serviceId}</serviceId>
                        <interfaceVersion>1.0</interfaceVersion>
                        <timestamp>@ignore@</timestamp>
                    </customHeader>
                </SOAP-ENV:Header>
                ]]>
                </data>
              </header>
              <header name="citrus_soap_action" value="doResponse"/>
            </headers>
            <body>
              <data>
              <![CDATA[
                <ResponseMessage xmlns="http://citrusframework.org/schema">
                  <resultCode>OK</resultCode>
                </ResponseMessage>
              ]]>
              </data>
            </body>
          </message>
        </receive-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapHeaderTest
actions:
  - soap:
      server: "soapServer"
      receiveRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              <ResponseMessage xmlns="http://citrusframework.org/schema">
                <resultCode>OK</resultCode>
              </ResponseMessage>
          headers:
            - name: citrus_soap_action
              value: doResponse
            - data: |
                <SOAP-ENV:Header
                    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
                    <customHeader xmlns="http://citrusframework.org/headerschema">
                        <correlationId>${correlationId}</correlationId>
                        <applicationId>${applicationId}</applicationId>
                        <trackingId>${trackingId}</trackingId>
                        <serviceId>${serviceId}</serviceId>
                        <interfaceVersion>1.0</interfaceVersion>
                        <timestamp>@ignore@</timestamp>
                    </customHeader>
                </SOAP-ENV:Header>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapHeaderTest">
    <actions>
      <receive endpoint="soapServer">
        <message>
            <data>
            <![CDATA[
            <ResponseMessage xmlns="http://citrusframework.org/schema">
              <resultCode>OK</resultCode>
            </ResponseMessage>
            ]]>
            </data>
        </message>
        <header>
            <data>
            <![CDATA[
            <SOAP-ENV:Header
                xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
                <customHeader xmlns="http://citrusframework.org/headerschema">
                    <correlationId>${correlationId}</correlationId>
                    <applicationId>${applicationId}</applicationId>
                    <trackingId>${trackingId}</trackingId>
                    <serviceId>${serviceId}</serviceId>
                    <interfaceVersion>1.0</interfaceVersion>
                    <timestamp>@ignore@</timestamp>
                </customHeader>
            </SOAP-ENV:Header>
            ]]>
            </data>
            <element name="citrus_soap_action" value="doResponse"/>
        </header>
      </receive>
    </actions>
  </testcase>
</spring:beans>
```

As you can see the SOAP XML header validation can combine header element and XML fragment validation. This is also likely to be used when dealing with WS-Security message headers.

16.6. SOAP HTTP mime headers

Besides the SOAP specific header elements the HTTP mime headers (e.g. Content-Type, Content-Length, Authorization) might be candidates for validation, too. When using HTTP as transport layer the SOAP message may define those mime headers. The tester is able to send and validate these headers inside the test case, although these HTTP headers are located outside the SOAP envelope. Let us first of all speak about validating the HTTP mime headers. This feature is not enabled by default. We have enabled this in our SOAP server configuration.

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .handleMimeHeaders(true)
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .handleMimeHeaders(true)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:server id="soapServer"
        port="8080"
        auto-start="true"
        handle-mime-headers="true"/>
</beans>
```

With this configuration Citrus will handle all available mime headers and pass those to the test case for normal header validation.

Java

```
@CitrusTest
public void soapServerTest() {
    then(soap()
        .server(soapServer)
        .receiveRequest()
            .message()
                .soapAction("MySoapService/sayHello")
                .body("""
                <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                </SoapMessageRequest>
                """)
                .header("Content-Type", "application/xml")
    );
}
```

XML

```
<test name="SoapServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <receive-request>
          <message soap-action="MySoapService/sayHello">
            <body>
              <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                </SoapMessageRequest>
            </body>
            <headers>
              <header name="Content-Type" value="application/xml"/>
            </headers>
          </message>
        </receive-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapServerTest
actions:
  - soap:
      server: "soapServer"
      receiveRequest:
        message:
          soapAction: "MySoapService/sayHello"
          body:
            data: |
              <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Validate mime headers</Operation>
              </SoapMessageRequest>
          headers:
            - name: Content-Type
              value: application/xml
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapServerTest">
    <actions>
      <ws:receive endpoint="soapServer">
        <message>
            <payload>
                <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                </SoapMessageRequest>
            </payload>
        </message>
        <header>
            <element name="Content-Type" value="application/xml"/>
        </header>
      </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

The validation of these HTTP mime headers is as usual now that we have enabled the mime header handling in the SOAP server. The transport HTTP headers are available in the header just like the normal SOAP header elements do. So you can validate the headers as usual.

So much for receiving and validating HTTP mime message headers with SOAP communication. Now we want to send special mime headers on client side. We overwrite or add mime headers to our sending action. We mark some headers with following prefix ***"citrus_http"***_ . This tells the SOAP client to add these headers to the HTTP header section outside the SOAP envelope. Keep in mind that header elements without this prefix go right into the SOAP header section by default.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .send()
            .message()
                .header("citrus_http_operation", "foo")
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <send-request>
          <message>
            <headers>
              <header name="citrus_http_operation" value="foo"/>
            </headers>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      sendRequest:
        message:
          headers:
            - name: citrus_http_operation
              value: foo
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
      <ws:send endpoint="soapClient">
        [...]
        <header>
          <element name="citrus_http_operation" value="foo"/>
        </header>
        [...]
      </ws:send>
    </actions>
  </testcase>
</spring:beans>
```

The listing above defines an HTTP mime header **operation** . The header prefix ***citrus_http_*** is cut off before the header goes into the HTTP header section. With this feature we can decide where exactly our header information is located in our resulting client message.

16.7. SOAP Envelope handling

By default Citrus will remove the SOAP envelope in message converter. Following from that the Citrus test case is independent from SOAP message formats and is not bothered with handling of SOAP envelope at all. This is great in most cases but sometimes it might be mandatory to also see the whole SOAP envelope inside the test case receive action. Therefore you can keep the SOAP envelope for incoming messages by configuration on the SOAP server side.

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .keepSoapEnvelope(true)
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .keepSoapEnvelope(true)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:server id="soapServer"
        port="8080"
        auto-start="true"
        keep-soap-envelope="true"/>
</beans>
```

With this configuration Citrus will handle all available mime headers and pass those to the test case for normal header validation.

Java

```
@CitrusTest
public void soapServerTest() {
    then(soap()
        .server(soapServer)
        .receive()
            .message()
                .body("""
                <SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                  <SOAP-ENV:Header/>
                  <SOAP-ENV:Body>
                    <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                    </SoapMessageRequest>
                  </SOAP-ENV:Body>
                </SOAP-ENV:Envelope>
                """)
    );
}
```

XML

```
<test name="SoapServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <receive-request>
          <message>
            <body>
              <SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                  <SOAP-ENV:Header/>
                  <SOAP-ENV:Body>
                    <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                    </SoapMessageRequest>
                  </SOAP-ENV:Body>
              </SOAP-ENV:Envelope>
            </body>
          </message>
        </receive-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapServerTest
actions:
  - soap:
      server: "soapServer"
      receiveRequest:
        message:
          body:
            data: |
              <SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                  <SOAP-ENV:Header/>
                  <SOAP-ENV:Body>
                    <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Validate mime headers</Operation>
                    </SoapMessageRequest>
                  </SOAP-ENV:Body>
              </SOAP-ENV:Envelope>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapServerTest">
    <actions>
      <ws:receive endpoint="soapServer">
        <message>
          <payload>
            <SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
              <SOAP-ENV:Header/>
              <SOAP-ENV:Body>
                <SoapMessageRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                <Operation>Validate mime headers</Operation>
                </SoapMessageRequest>
              </SOAP-ENV:Body>
            </SOAP-ENV:Envelope>
          </payload>
        </message>
      </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

So now you are able to validate the whole SOAP envelope as is. This might be of interest in very special cases. As mentioned by default the Citrus server will automatically remove the SOAP envelope and translate the SOAP body to the message payload for straight forward validation inside the test cases.

16.8. SOAP server interceptors

The Citrus SOAP server supports the concept of interceptors in order to add custom logic to the request/response processing steps. The interceptors need to implement a common interface: **org.springframework.ws.server.EndpointInterceptor**. We are able to customize the interceptor chain on the server component as follows:

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .interceptors(serverInterceptors())
            .build();
}

public List<SoapEndpointInterceptor> serverInterceptors() {
    SoapMustUnderstandEndpointInterceptor mustUnderstandInterceptor = new
SoapMustUnderstandEndpointInterceptor();
    mustUnderstandInterceptor.setAcceptedHeaders("{http://docs.oasis-
open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd}Security");

    Wss4jSecurityInterceptor wss4jSecurityInterceptor = new
Wss4jSecurityInterceptor();
    wss4jSecurityInterceptor.setValidationActions("Timestamp UsernameToken");
    wss4jSecurityInterceptor.setValidationCallbackHandler(passwordCallbackHandler());

    return Arrays.asList(
            mustUnderstandInterceptor,
            new LoggingEndpointInterceptor(),
            wss4jSecurityInterceptor
    );
}

public SimplePasswordValidationCallbackHandler passwordCallbackHandler() {
    SimplePasswordValidationCallbackHandler handler = new
SimplePasswordValidationCallbackHandler();
    handler.setUsersMap(Collections.singletonMap("admin", "secret"));
    return handler;
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .interceptors(serverInterceptors())
            .build();
}

public List<SoapEndpointInterceptor> serverInterceptors() {
    SoapMustUnderstandEndpointInterceptor mustUnderstandInterceptor = new
SoapMustUnderstandEndpointInterceptor();
    mustUnderstandInterceptor.setAcceptedHeaders("{http://docs.oasis-
open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd}Security");

    Wss4jSecurityInterceptor wss4jSecurityInterceptor = new
Wss4jSecurityInterceptor();
    wss4jSecurityInterceptor.setValidationActions("Timestamp UsernameToken");
    wss4jSecurityInterceptor.setValidationCallbackHandler(passwordCallbackHandler());

    return Arrays.asList(
            mustUnderstandInterceptor,
            new LoggingEndpointInterceptor(),
            wss4jSecurityInterceptor
    );
}

public SimplePasswordValidationCallbackHandler passwordCallbackHandler() {
    SimplePasswordValidationCallbackHandler handler = new
SimplePasswordValidationCallbackHandler();
    handler.setUsersMap(Collections.singletonMap("admin", "secret"));
    return handler;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <citrus-ws:server id="secureSoapServer"
                      port="8080"
                      auto-start="true"
                      interceptors="serverInterceptors"/>

    <util:list id="serverInterceptors">
      <bean
class="org.citrusframework.ws.interceptor.SoapMustUnderstandEndpointInterceptor">
        <property name="acceptedHeaders">
          <list>
            <value>{http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
wssecurity-secext-1.0.xsd}Security</value>
          </list>
        </property>
      </bean>
      <bean class="org.citrusframework.ws.interceptor.LoggingEndpointInterceptor"/>
      <bean
class="org.springframework.ws.soap.security.wss4j.Wss4jSecurityInterceptor">
        <property name="validationActions" value="Timestamp UsernameToken"/>
        <property name="validationCallbackHandler">
          <bean id="passwordCallbackHandler"
class="org.springframework.ws.soap.security.wss4j.callback.SimplePasswordValidationCal
lbackHandler">
            <property name="usersMap">
              <map>
                <entry key="admin" value="secret"/>
              </map>
            </property>
          </bean>
        </property>
      </bean>
    </util:list>
</beans>
```

The custom interceptors are used to enable WsSecurity features on the soap server component via Wss4J.



When customizing the interceptor chain of the soap server component all default interceptors (like logging interceptors) are lost. You can see that we had to add the *org.citrusframework.ws.interceptor.LoggingEndpointInterceptor* explicitly in order to log request/response messages for the server communication.

16.9. SOAP 1.2

By default, Citrus components use SOAP 1.1 version. Fortunately SOAP 1.2 is supported the same way. As we already mentioned before the Citrus SOAP components do use a SOAP message factory for creating messages in SOAP format.

Citrus Bean

```
@BindToRegistry
public SaajSoapMessageFactory soapMessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_11);
    return messageFactory;
}

@BindToRegistry
public SaajSoapMessageFactory soap12MessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_12);
    return messageFactory;
}
```

Spring Bean

```
@Bean
public SaajSoapMessageFactory soapMessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_11);
    return messageFactory;
}

@Bean
public SaajSoapMessageFactory soap12MessageFactory() {
    SaajSoapMessageFactory messageFactory = new SaajSoapMessageFactory();
    messageFactory.setSoapVersion(SoapVersion.SOAP_12);
    return messageFactory;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <!-- SOAP 1.1 Message Factory -->
    <bean id="soapMessageFactory"
class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory">
      <property name="soapVersion">
        <util:constant static-
field="org.springframework.ws.soap.SoapVersion.SOAP_11"/>
      </property>
    </bean>

    <!-- SOAP 1.2 Message Factory -->
    <bean id="soap12MessageFactory"
class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory">
      <property name="soapVersion">
        <util:constant static-
field="org.springframework.ws.soap.SoapVersion.SOAP_12"/>
      </property>
    </bean>
</beans>
```

As you can see the SOAP message factory can either create SOAP 1.1 or SOAP 1.2 messages. This is how Citrus can create both SOAP 1.1 and SOAP 1.2 messages. Of course you can have multiple message factories configured in your project. Just set the message factory on a WebService client or server component in order to define which version should be used.

Citrus Bean

```
@BindToRegistry
public WebServiceClient soap12Client() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/echo")
            .timeout(10000L)
            .messageFactory(soap12MessageFactory())
            .build();
}

@BindToRegistry
public WebServiceServer soap12Server() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .messageFactory(soap12MessageFactory())
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceClient soap12Client() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/echo")
            .timeout(10000L)
            .messageFactory(soap12MessageFactory())
            .build();
}

@Bean
public WebServiceServer soap12Server() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .messageFactory(soap12MessageFactory())
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="soap12Client"
            request-url="http://localhost:8080/echo"
            message-factory="soap12MessageFactory"
            timeout="10000"/>

  <citrus-ws:server id="soap12Server"
          port="8080"
          auto-start="true"
          root-parent-context="true"
          message-factory="soap12MessageFactory"/>
</beans>
```

By default, Citrus components do connect with a message factory called **messageFactory** no matter what SOAP version this factory is using.

16.10. SOAP faults

SOAP faults describe a failed communication in SOAP WebServices world. Citrus is able to send and receive SOAP fault messages. On server side Citrus can simulate SOAP faults with fault-code, fault-reason, fault-actor and fault-detail. On client side Citrus is able to handle and validate SOAP faults in response messages. The next section describes how to deal with SOAP faults in Citrus.

16.11. Send SOAP faults

As Citrus simulates SOAP server endpoints you also need to think about sending a SOAP fault to the calling client. In case Citrus receives a SOAP request as a server you can respond with a proper SOAP fault if necessary.

Please keep in mind that we use the citrus-ws extension for sending SOAP faults in our test case, as shown in this very simple example:

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .server(soapServer)
            .sendFault()
                .message()
                    .faultActor("SERVER")
                    .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                    .faultString("Invalid request")
                    .faultDetail("""
                    <FaultDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                        <MessageId>${messageId}</MessageId>
                        <CorrelationId>${correlationId}</CorrelationId>
                        <ErrorCode>TEC-1000</ErrorCode>
                        <Text>Invalid request</Text>
                    </FaultDetail>
                    """)
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <send-fault>
          <message fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                   fault-string="Invalid request"
                   fault-actor="SERVER">
            <fault-detail>
              <content>
                <![CDATA[
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>${messageId}</MessageId>
                    <CorrelationId>${correlationId}</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
                ]]>
              </content>
            </fault-detail>
          </message>
        </send-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      server: "soapServer"
      sendFault:
        message:
          faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
          faultString: "Invalid request"
          faultActor: "SERVER"
          faultDetails:
            - content: |
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>${messageId}</MessageId>
                    <CorrelationId>${correlationId}</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
      <ws:send-fault endpoint="soapServer">
        <ws:fault>
            <ws:fault-code>{http://citrusframework.org/faults}citrus:TEC-
1000</ws:fault-code>
            <ws:fault-string>Invalid request</ws:fault-string>
            <ws:fault-actor>SERVER</ws:fault-actor>
            <ws:fault-detail>
                <![CDATA[
                    <FaultDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                        <MessageId>${messageId}</MessageId>
                        <CorrelationId>${correlationId}</CorrelationId>
                        <ErrorCode>TEC-1000</ErrorCode>
                        <Text>Invalid request</Text>
                    </FaultDetail>
                ]]>
            </ws:fault-detail>
        </ws:fault>
      </ws:send-fault>
    </actions>
  </testcase>
</spring:beans>
```

The example generates a simple SOAP fault that is sent back to the calling client. The fault-actor and the fault-detail elements are optional. Same with the soap action declared in the special Citrus header ***citrus_soap_action*** . In the sample above the fault-detail data is placed inline as XML data. As an alternative to that you can also set the fault-detail via external file resource. Just use the ***file*** attribute as fault detail instead of the inline CDATA definition.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .server(soapServer)
            .sendFault()
                .message()
                    .faultActor("SERVER")
                    .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                    .faultString("Invalid request")
                    .faultDetail(Resources.of("classpath:myFaultDetail.xml"))
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <send-fault>
          <message fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                   fault-string="Invalid request"
                   fault-actor="SERVER">
            <fault-detail>
              <resource file="classpath:myFaultDetail.xml"/>
            </fault-detail>
          </message>
        </send-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      server: "soapServer"
      sendFault:
        message:
          faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
          faultString: "Invalid request"
          faultActor: "SERVER"
          faultDetails:
            - file: "classpath:myFaultDetail.xml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
      <ws:send-fault endpoint="soapServer">
        <ws:fault>
            <ws:fault-code>{http://citrusframework.org/faults}citrus:TEC-
1000</ws:fault-code>
            <ws:fault-string>Invalid request</ws:fault-string>
            <ws:fault-actor>SERVER</ws:fault-actor>
            <ws:fault-detail file="classpath:myFaultDetail.xml"/>
        </ws:fault>
      </ws:send-fault>
    </actions>
  </testcase>
</spring:beans>
```

The generated SOAP fault looks like follows:

```
HTTP/1.1 500 Internal Server Error
Accept: text/xml, text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2
SOAPAction: "sayHello"
Content-Type: text/xml; charset=utf-8
Content-Length: 680
Server: Jetty(7.0.0.pre5)

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <SOAP-ENV:Body>
        <SOAP-ENV:Fault>
            <faultcode xmlns:citrus="http://citrusframework.org/faults">citrus:TEC-
1000</faultcode>
            <faultstring xml:lang="en">Invalid request</faultstring>
            <detail>
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>9277832563</MessageId>
                    <CorrelationId>4346806225</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
            </detail>
        </SOAP-ENV:Fault>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

16.12. Receive SOAP faults

In case you receive SOAP response messages as a client endpoint you may need to handle and validate SOAP faults in error situations. Citrus can validate SOAP faults with fault-code, fault-actor, fault-string and fault-detail values.

As a client we send out a request and receive a SOAP fault as response. By default the client sending action in Citrus throws a specific exception when the SOAP response is a SOAP fault element. This exception is called ***SoapFaultClientException*** coming from the Spring API. You as a tester can assert this kind of exception in a test case in order to expect the SOAP error.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(assertException()
            .exception(SoapFaultClientException.class)
            .when(
              send()
                .endpoint("soapClient")
                .message()
                    .body("""
                    <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/soap">
                        <Message>This is invalid</Message>
                    </SoapFaultForcingRequest>
                    """)
            )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <assert exception="org.springframework.ws.soap.client.SoapFaultClientException">
        <when>
          <send endpoint="soapClient">
            <message>
              <body>
                <SoapFaultForcingRequest
                  xmlns="http://citrusframework.org/schemas/soap">
                    <Message>This is invalid</Message>
                </SoapFaultForcingRequest>
              </body>
            </message>
          </send>
        </when>
      </assert>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - assert:
      exception: "org.springframework.ws.soap.client.SoapFaultClientException"
      when:
        send:
          endpoint: soapClient
          message:
            body: |
              <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/soap">
                <Message>This is invalid</Message>
              </SoapFaultForcingRequest>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <assert
exception="org.springframework.ws.soap.client.SoapFaultClientException">
          <when>
            <send endpoint="soapClient">
                <message>
                    <payload>
                        <SoapFaultForcingRequest
                          xmlns="http://citrusframework.org/schemas/soap">
                            <Message>This is invalid</Message>
                        </SoapFaultForcingRequest>
                    </payload>
                </message>
            </send>
          </when>
        </assert>
    </actions>
  </testcase>
</spring:beans>
```

The SOAP message sending action is surrounded by a simple assert action. The asserted exception

class is the ***SoapFaultClientException*** that we have mentioned before. This means that the test expects the exception to be thrown during the communication. In case the exception is missing the test fails.

So far we have used the Citrus core capabilities of asserting an exception. This basic assertion test action is not able to offer direct access to the SOAP fault-code and fault-string values for validation. The basic assert action simply has no access to the actual SOAP fault elements. Fortunately we can use the **citrus-ws** namespace again which offers a special assert action implementation especially designed for SOAP faults in this case.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .client(soapClient)
            .assertFault()
                .faultActor("SERVER")
                .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                .faultString("Invalid request")
            .when(
              send()
                .endpoint(soapClient)
                .message()
                    .body("""
                    <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/soap">
                        <Message>This is invalid</Message>
                    </SoapFaultForcingRequest>
                    """)
            )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <assert-fault fault-actor="SERVER"
                      fault-string="Invalid request"
                      fault-code="{http://citrusframework.org/faults}citrus:TEC-1000">
          <when>
            <send endpoint="soapClient">
              <message>
                <body>
                    <SoapFaultForcingRequest
                      xmlns="http://citrusframework.org/schemas/soap">
                        <Message>This is invalid</Message>
                    </SoapFaultForcingRequest>
                </body>
              </message>
            </send>
          </when>
        </assert-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      client: "soapClient"
      assertFault:
        faultActor: "SERVER"
        faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
        faultString: "Invalid request"
        when:
          - send:
              endpoint: soapClient
              message:
                body: |
                  <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/soap">
                    <Message>This is invalid</Message>
                  </SoapFaultForcingRequest>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:assert-fault fault-code="{http://citrusframework.org/faults}TEC-1001"
                  fault-string="Invalid request">
                  fault-actor="SERVER">
          <ws:when>
            <send endpoint="soapClient">
                <message>
                    <payload>
                        <SoapFaultForcingRequest
                          xmlns="http://citrusframework.org/schemas/soap">
                            <Message>This is invalid</Message>
                        </SoapFaultForcingRequest>
                    </payload>
                </message>
            </send>
          </ws:when>
        </ws:assert-fault>
    </actions>
  </testcase>
</spring:beans>
```

The special assert action offers several attributes to validate the expected SOAP fault. Namely these are **"fault-code"**, **"fault-string"** and **"fault-actor"** . The **fault-code** is defined as a QName string and is mandatory for the validation. The fault assertion also supports test variable replacement as usual.

The time you use SOAP fault validation you need to tell Citrus how to validate the SOAP faults. Citrus needs an instance of a ***SoapFaultValidator*** that we need to add to the Spring application context. By default Citrus is searching for a bean with the id **'soapFaultValidator'** .

Citrus Bean

```
@BindToRegistry
public SimpleSoapAttachmentValidator soapFaultValidator() {
    return new SimpleSoapAttachmentValidator();
}
```

Spring Bean

```
@Bean
public SimpleSoapAttachmentValidator soapFaultValidator() {
    return new SimpleSoapAttachmentValidator();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <bean id="soapFaultValidator"
class="org.citrusframework.ws.validation.SimpleSoapAttachmentValidator"/>
</beans>
```

Citrus offers several reference implementations for these SOAP fault validators. These are:

- org.citrusframework.ws.validation.SimpleSoapAttachmentValidator
- org.citrusframework.ws.validation.SimpleSoapFaultValidator
- org.citrusframework.ws.validation.XmlSoapFaultValidator

Please see the API documentation for details on the available reference implementations. Of course you can also define your own SOAP validator logic (would be great if you could share your ideas!). In the test case you can explicitly choose the validator to use:

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .client(soapClient)
            .assertFault()
                .faultActor("SERVER")
                .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                .faultString("Invalid request")
                .validator(mySpecialSoapFaultValidator)
            .when(
              // [...]
            )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <assert-fault fault-actor="SERVER"
                      fault-string="Invalid request"
                      fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                      validator="mySpecialSoapFaultValidator">
          <when>
            [...]
          </when>
        </assert-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      client: "soapClient"
      assertFault:
        faultActor: "SERVER"
        faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
        faultString: "Invalid request"
        validator: "mySpecialSoapFaultValidator"
        when:
          [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:assert-fault fault-code="{http://citrusframework.org/faults}TEC-1001"
                         fault-string="Invalid request"
                         fault-validator="mySpecialSoapFaultValidator">
           [...]
        </ws:assert-fault>
    </actions>
  </testcase>
</spring:beans>
```



Another important thing to notice when asserting SOAP faults is the fact, that Citrus needs to have a ***SoapMessageFactory*** available in the Spring application context. If you deal with SOAP messaging in general you will already have such a bean in the context.

Citrus Bean

```
@BindToRegistry
public SaajSoapMessageFactory messageFactory() {
    return new SaajSoapMessageFactory();
}
```

Spring Bean

```
@Bean
public SaajSoapMessageFactory messageFactory() {
    return new SaajSoapMessageFactory();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="messageFactory"
class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory"/>
</beans>
```

Choose one of Spring's reference implementations or some other implementation as SOAP message factory. Citrus will search for a bean with id **'messageFactory'** by default.

Citrus is also able to validate SOAP fault details. See the following example for understanding how to do it:

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .client(soapClient)
            .assertFault()
                .faultActor("SERVER")
                .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                .faultString("Invalid request")
                .faultDetail("""
                <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                  <ErrorCode>TEC-1000</ErrorCode>
                  <Text>Invalid request</Text>
                </FaultDetail>
                """)
            .when(
              // [...]
            )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <assert-fault fault-actor="SERVER"
                      fault-string="Invalid request"
                      fault-code="{http://citrusframework.org/faults}citrus:TEC-1000">
          <fault-detail>
            <content>
              <![CDATA[
                  <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                      <Text>Invalid request</Text>
                  </FaultDetail>
              ]]>
            </content>
          </fault-detail>
          <when>
            [...]
          </when>
        </assert-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      client: "soapClient"
      assertFault:
        faultActor: "SERVER"
        faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
        faultString: "Invalid request"
        faultDetails:
          - content: |
              <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                  <ErrorCode>TEC-1000</ErrorCode>
                  <Text>Invalid request</Text>
              </FaultDetail>
        when:
          [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:assert-fault fault-code="{http://citrusframework.org/faults}TEC-1001"
                         fault-string="Invalid request">
            <ws:fault-detail>
              <![CDATA[
                  <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                      <Text>Invalid request</Text>
                  </FaultDetail>
              ]]>
            </ws:fault-detail>
            <ws:when>
                [...]
            </ws:when>
        </ws:assert-fault>
    </actions>
  </testcase>
</spring:beans>
```

The expected SOAP fault detail content is simply added to the **ws:assert** action. The ***SoapFaultValidator*** implementation defined in the Spring application context is responsible for checking the SOAP fault detail with validation algorithm. The validator implementation checks the detail content to meet the expected template. Citrus provides some default ***SoapFaultValidator*** implementations. Supported algorithms are pure String comparison (**org.citrusframework.ws.validation.SimpleSoapFaultValidator**) as well as XML tree walk-through (**org.citrusframework.ws.validation.XmlSoapFaultValidator**).

When using the XML validation algorithm you have the complete power as known from normal message validation in receive actions. This includes schema validation or ignoring elements for instance. On the fault-detail element you are able to add some validation settings such as **schema-validation=enabled/disabled**, custom **schema-repository** and so on.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .client(soapClient)
            .assertFault()
                .faultActor("SERVER")
                .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                .faultString("Invalid request")
                .schemaValidation(false)
                .faultDetail("""
                <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                  <ErrorCode>TEC-1000</ErrorCode>
                  <Text>Invalid request</Text>
                </FaultDetail>
                """)
            .when(
              // [...]
            )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <assert-fault fault-actor="SERVER"
                      fault-string="Invalid request"
                      fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                      schema-validation="false">
          <fault-detail>
            <content>
              <![CDATA[
                  <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                      <Text>Invalid request</Text>
                  </FaultDetail>
              ]]>
            </content>
          </fault-detail>
          <when>
            [...]
          </when>
        </assert-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      client: "soapClient"
      assertFault:
        faultActor: "SERVER"
        faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
        faultString: "Invalid request"
        schemaValidation: false
        faultDetails:
          - content: |
              <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                  <ErrorCode>TEC-1000</ErrorCode>
                  <Text>Invalid request</Text>
              </FaultDetail>
        when:
          [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:assert-fault fault-code="{http://citrusframework.org/faults}TEC-1001"
                         fault-string="Invalid request">
            <ws:fault-detail schema-validation="false">
              <![CDATA[
                  <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                      <Text>Invalid request</Text>
                  </FaultDetail>
              ]]>
            </ws:fault-detail>
            <ws:when>
                [...]
            </ws:when>
        </ws:assert-fault>
    </actions>
  </testcase>
</spring:beans>
```

Please see also the Citrus API documentation for available validator implementations and validation algorithms.

So far we have used assert action wrapper in order to catch SOAP fault exceptions and validate the SOAP fault content. Now we have an alternative way of handling SOAP faults in Citrus. With exceptions the send action aborts and we do not have a receive action for the SOAP fault. This might be inadequate if we need to validate the SOAP message content (SOAPHeader and SOAPBody) coming with the SOAP fault. Therefore the web service message sender component offers several fault strategy options. In the following we discuss the propagation of SOAP fault as messages to the receive action as we would do with normal SOAP messages.

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .faultStrategy(ErrorHandlingStrategy.PROPAGATE)
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .faultStrategy(ErrorHandlingStrategy.PROPAGATE)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

  <citrus-ws:client id="soapClient"
                  request-url="http://localhost:8090/test"
                  fault-strategy="propagateError"/>
</beans>
```

We have configured a fault strategy **propagateError** so the message sender will not raise client exceptions but inform the receive action with SOAP fault message contents. By default, the fault strategy raises client exceptions (fault-strategy= **throwsException**).

So now that we do not raise exceptions we can leave out the assert action wrapper in our test. Instead we simply use a receive action and validate the SOAP fault like this.

Java

```
@CitrusTest
public void soapFaultTest() {
    when(send()
            .endpoint(soapClient)
            .message()
                .body("""
                 <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/soap">
                     <Message>This is invalid</Message>
                 </SoapFaultForcingRequest>
                 """)
    );

    then(receive()
             .endpoint(soapClient)
             .message()
                 .body("""
                 <SOAP-ENV:Fault xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                     <SOAP-ENV:faultcode
xmlns:CITRUS="http://citrus.org/soap">CITRUS:${soapFaultCode}</SOAP-ENV:faultcode>
                     <SOAP-ENV:faultstring xml:lang="en">${soapFaultString}</SOAP-
ENV:faultstring>
                 </SOAP-ENV:Fault>
                 """));
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="soapClient">
            <message>
                <body>
                    <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                        <Message>This is invalid</Message>
                    </SoapFaultForcingRequest>
                </body>
            </message>
        </send>

        <receive endpoint="soapClient" timeout="5000">
            <message>
                <body>
                    <SOAP-ENV:Fault xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                        <SOAP-ENV:faultcode
xmlns:CITRUS="http://citrus.org/soap">CITRUS:${soapFaultCode}</SOAP-ENV:faultcode>
                        <SOAP-ENV:faultstring xml:lang="en">${soapFaultString}</SOAP-
ENV:faultstring>
                    </SOAP-ENV:Fault>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - send:
      endpoint: "soapClient"
      message:
        body:
          data: |
            <SoapFaultForcingRequest xmlns="http://citrusframework.org/schemas/soap">
                <Message>This is invalid</Message>
            </SoapFaultForcingRequest>

  - receive:
      endpoint: "soapClient"
      timeout: 5000
      message:
        body:
          data: |
            <SOAP-ENV:Fault xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                <SOAP-ENV:faultcode
xmlns:CITRUS="http://citrus.org/soap">CITRUS:${soapFaultCode}</SOAP-ENV:faultcode>
                <SOAP-ENV:faultstring xml:lang="en">${soapFaultString}</SOAP-
ENV:faultstring>
            </SOAP-ENV:Fault>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <send endpoint="soapClient">
            <message>
                <payload>
                    <SoapFaultForcingRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                        <Message>This is invalid</Message>
                    </SoapFaultForcingRequest>
                </payload>
            </message>
        </send>

        <receive endpoint="soapClient" timeout="5000">
            <message>
                <payload>
                    <SOAP-ENV:Fault xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
                        <SOAP-ENV:faultcode
xmlns:CITRUS="http://citrus.org/soap">CITRUS:${soapFaultCode}</SOAP-ENV:faultcode>
                        <SOAP-ENV:faultstring xml:lang="en">${soapFaultString}</SOAP-
ENV:faultstring>
                    </SOAP-ENV:Fault>
                </payload>
            </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

So choose the preferred way of handling SOAP faults either by asserting client exceptions or propagating fault messages to the `receive` action on a SOAP client.

16.13. Multiple SOAP fault details

SOAP fault messages can hold multiple SOAP fault detail elements. In the previous sections we have used SOAP fault details in sending and receiving actions as single element. In order to meet the

SOAP specification Citrus is also able to handle multiple SOAP fault detail elements in a message. You just use multiple fault-detail elements in your test action like this:

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .server(soapServer)
            .sendFault()
                .message()
                    .faultActor("SERVER")
                    .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                    .faultString("Invalid request")
                    .faultDetail("""
                    <FaultDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                        <MessageId>${messageId}</MessageId>
                        <CorrelationId>${correlationId}</CorrelationId>
                        <ErrorCode>TEC-1000</ErrorCode>
                        <Text>Invalid request</Text>
                    </FaultDetail>
                    """)
                    .faultDetail("""
                    <ErrorDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                        <ErrorCode>TEC-1000</ErrorCode>
                    </ErrorDetail>
                    """)
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <send-fault>
          <message fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                   fault-string="Invalid request"
                   fault-actor="SERVER">
            <fault-detail>
              <content>
                <![CDATA[
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>${messageId}</MessageId>
                    <CorrelationId>${correlationId}</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
                ]]>
              </content>
              <content>
                <![CDATA[
                <ErrorDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <ErrorCode>TEC-1000</ErrorCode>
                </ErrorDetail>
                ]]>
              </content>
            </fault-detail>
          </message>
        </send-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      server: "soapServer"
      sendFault:
        message:
          faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
          faultString: "Invalid request"
          faultActor: "SERVER"
          faultDetails:
            - content: |
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>${messageId}</MessageId>
                    <CorrelationId>${correlationId}</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
            - content: |
                <ErrorDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <ErrorCode>TEC-1000</ErrorCode>
                </ErrorDetail>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:send-fault endpoint="soapServer">
            <ws:fault>
                <ws:fault-code>{http://citrusframework.org/faults}citrus:TEC-
1000</ws:fault-code>
                <ws:fault-string>Invalid request</ws:fault-string>
                <ws:fault-actor>SERVER</ws:fault-actor>
                <ws:fault-detail>
                    <![CDATA[
                        <FaultDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                            <MessageId>${messageId}</MessageId>
                            <CorrelationId>${correlationId}</CorrelationId>
                            <ErrorCode>TEC-1000</ErrorCode>
                            <Text>Invalid request</Text>
                        </FaultDetail>
                    ]]>
                </ws:fault-detail>
                <ws:fault-detail>
                    <![CDATA[
                        <ErrorDetail
xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                            <ErrorCode>TEC-1000</ErrorCode>
                        </ErrorDetail>
                    ]]>
                </ws:fault-detail>
            </ws:fault>
            <ws:header>
                <ws:element name="citrus_soap_action" value="sayHello"/>
            </ws:header>
        </ws:send-fault>
    </actions>
  </testcase>
</spring:beans>
```

This will result in following SOAP envelope message:

```
HTTP/1.1 500 Internal Server Error
Accept: text/xml, text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2
SOAPAction: "sayHello"
Content-Type: text/xml; charset=utf-8
Content-Length: 680
Server: Jetty(7.0.0.pre5)

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <SOAP-ENV:Body>
        <SOAP-ENV:Fault>
            <faultcode xmlns:citrus="http://citrusframework.org/faults">citrus:TEC-
1000</faultcode>
            <faultstring xml:lang="en">Invalid request</faultstring>
            <detail>
                <FaultDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <MessageId>9277832563</MessageId>
                    <CorrelationId>4346806225</CorrelationId>
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
                <ErrorDetail xmlns="http://citrusframework.org/schemas/sayHello.xsd">
                    <ErrorCode>TEC-1000</ErrorCode>
                </ErrorDetail>
            </detail>
        </SOAP-ENV:Fault>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Of course, we can also expect several fault detail elements when receiving a SOAP fault.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(soap()
            .client(soapClient)
            .assertFault()
                .faultActor("SERVER")
                .faultCode("{http://citrusframework.org/faults}citrus:TEC-1000")
                .faultString("Invalid request")
                .faultDetail("""
                <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
                """)
                .faultDetail("""
                <ErrorDetail xmlns="http://citrusframework.org/schemas/soap">
                    <ErrorCode>TEC-1000</ErrorCode>
                </ErrorDetail>
                """)
                .when(send()
                    .endpoint(soapClient)
                    // [...]
                )
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapServer">
        <assert-fault fault-code="{http://citrusframework.org/faults}citrus:TEC-1000"
                      fault-string="Invalid request">
            <fault-detail>
              <content>
                <![CDATA[
                <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                    <ErrorCode>TEC-1000</ErrorCode>
                    <Text>Invalid request</Text>
                </FaultDetail>
                ]]>
              </content>
              <content>
                <![CDATA[
                <ErrorDetail xmlns="http://citrusframework.org/schemas/soap">
                    <ErrorCode>TEC-1000</ErrorCode>
                </ErrorDetail>
                ]]>
              </content>
            </fault-detail>
            <when>
              <send endpoint="soapClient">
                [...]
              </send>
            </when>
        </assert-fault>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - soap:
      client: "soapClient"
      assertFault:
        faultCode: "{http://citrusframework.org/faults}citrus:TEC-1000"
        faultString: "Invalid request"
        faultDetails:
          - content: |
              <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                <ErrorCode>TEC-1000</ErrorCode>
                <Text>Invalid request</Text>
              </FaultDetail>
          - content: |
              <ErrorDetail xmlns="http://citrusframework.org/schemas/soap">
                <ErrorCode>TEC-1000</ErrorCode>
              </ErrorDetail>
        when:
          - send:
              endpoint: soapClient
              # [...]
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <ws:assert-fault fault-code="{http://citrusframework.org/faults}TEC-1001"
                  fault-string="Invalid request">
            <ws:fault-detail schema-validation="false">
              <![CDATA[
                  <FaultDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                      <Text>Invalid request</Text>
                  </FaultDetail>
              ]]>
            </ws:fault-detail>
            <ws:fault-detail>
              <![CDATA[
                  <ErrorDetail xmlns="http://citrusframework.org/schemas/soap">
                      <ErrorCode>TEC-1000</ErrorCode>
                  </ErrorDetail>
              ]]>
            </ws:fault-detail>
            <ws:when>
                <send endpoint="soapClient">
                    [...]
                </send>
            </ws:when>
        </ws:assert-fault>
    </actions>
  </testcase>
</spring:beans>
```

As you can see we can individually use validation settings for each fault detail. In the example above we disabled schema validation for the first fault detail element.

16.14. Send HTTP error codes with SOAP

The SOAP server logic in Citrus is able to simulate pure HTTP error codes such as `404` "Not found" or `500` "Internal server error". The good thing is that the Citrus server is able to receive a request for proper validation in a `receive` action and then simulate HTTP errors on demand.

The mechanism on HTTP error code simulation is not different to the usual SOAP request/response handling in Citrus. We receive the request as usual, and we provide a response. The HTTP error situation is simulated according to the special HTTP header **citrus_http_status** in the Citrus SOAP response definition. In case this header is set to a value other than 200 OK the Citrus SOAP server sends an empty SOAP response with HTTP error status code set accordingly.

Java

```
@CitrusTest
public void soapFaultTest() {
    then(send()
            .endpoint(soapServer)
            .message()
            .header("citrus_http_status_code", 500L)
    );
}
```

XML

```
<test name="SoapFaultTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="soapServer">
          <message>
            <body>
              <data/>
            </body>
            <headers>
              <header name="citrus_http_status_code" value="500"/>
            </headers>
          </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SoapFaultTest
actions:
  - send:
      endpoint: "soapServer"
      message:
        body:
          data: {}
        headers:
          - name: "citrus_http_status_code"
            value: 500
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapFaultTest">
    <actions>
        <send endpoint="soapServer">
            <message>
                <data/>
            </message>
            <header>
                <element name="citrus_http_status_code" value="500"/>
            </header>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

The SOAP response must be empty and the HTTP status code is set to a value other than 200, like 500. This results in an HTTP error sent to the calling client with error 500 "Internal server error".

16.15. SOAP attachment support

Citrus is able to add attachments to a SOAP request on client and server side. As usual you can validate the SOAP attachment content on a received SOAP message. The next chapters describe how to handle SOAP attachments in Citrus.

16.16. Send SOAP attachments

As client Citrus is able to add attachments to the SOAP message. I think it is best to go straight into an example in order to understand how it works.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .send()
            .message()
                .body("""
                <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Read the attachment</Operation>
                </SoapMessageWithAttachment>
                """)
                .attachment("MySoapAttachment", "text/plain",
Resources.of("classpath:org/citrusframework/ws/soapAttachment.txt"))
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <send-request>
          <message>
            <body>
              <payload>
                <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Read the attachment</Operation>
                </SoapMessageWithAttachment>
              </payload>
            </body>
            <attachment content-id="MySoapAttachment" content-type="text/plain">
              <resource file="classpath:org/citrusframework/ws/soapAttachment.txt"/>
            </attachment>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      sendRequest:
        message:
          body:
            data: |
              <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Read the attachment</Operation>
              </SoapMessageWithAttachment>
          attachments:
            - contentId: MySoapAttachment
              contentType: "text/plain"
              resource: "classpath:org/citrusframework/ws/soapAttachment.txt"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:send endpoint="soapClient">
            <message>
                <payload>
                    <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                        <Operation>Read the attachment</Operation>
                    </SoapMessageWithAttachment>
                </payload>
            </message>
            <ws:attachment content-id="MySoapAttachment" content-type="text/plain">
                <ws:resource
file="classpath:org/citrusframework/ws/soapAttachment.txt"/>
            </ws:attachment>
        </ws:send>
    </actions>
  </testcase>
</spring:beans>
```



In the previous chapters you may have already noticed the **citrus-ws** namespace that stands for the SOAP extensions in Citrus. Please include the **citrus-ws** namespace in your test case as described earlier in this chapter so you can use the attachment support.

The special send action of the SOAP extension namespace is aware of SOAP attachments. The attachment content usually consists of a **content-id**, a **content-type** and the actual content as plain text or binary content. Inside the test case you can use external file resources or inline CDATA sections for the attachment content. As you are familiar with Citrus you may know this already from other actions.

Citrus will construct a SOAP message with the SOAP attachment. Currently only one attachment per message is supported.

16.17. Receive SOAP attachments

When Citrus calls SOAP WebServices as a client we may receive SOAP responses with attachments. The tester can validate those received SOAP messages with attachment content quite easy. As usual let us have a look at an example first.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .receive()
            .message()
                .body("""
                <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Read the attachment</Operation>
                </SoapMessageWithAttachment>
                """)
                .attachment("MySoapAttachment", "text/plain",
Resources.of("classpath:org/citrusframework/ws/soapAttachment.txt"))
                .attachmentValidator(mySoapAttachmentValidator)
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <receive-response attachment-validator="mySoapAttachmentValidator">
          <message>
            <body>
              <payload>
                <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                    <Operation>Read the attachment</Operation>
                </SoapMessageWithAttachment>
              </payload>
            </body>
            <attachment content-id="MySoapAttachment" content-type="text/plain">
              <resource file="classpath:org/citrusframework/ws/soapAttachment.txt"/>
            </attachment>
          </message>
        </receive-response>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      receiveResponse:
        validator: mySoapAttachmentValidator
        message:
          body:
            data: |
              <SoapMessageWithAttachment
xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Read the attachment</Operation>
              </SoapMessageWithAttachment>
          attachments:
            - contentId: MySoapAttachment
              contentType: "text/plain"
              resource: "classpath:org/citrusframework/ws/soapAttachment.txt"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:receive endpoint="soapClient">
            <message>
                <payload>
                    <SoapMessageWithAttachmentRequest
xmlns="http://citrusframework.org/schemas/sample.xsd">
                        <Operation>Read the attachment</Operation>
                    </SoapMessageWithAttachmentRequest>
                </payload>
            </message>
            <ws:attachment content-id="MySoapAttachment"
                           content-type="text/plain"
                           validator="mySoapAttachmentValidator">
                <ws:resource
file="classpath:org/citrusframework/ws/soapAttachment.txt"/>
            </ws:attachment>
        </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

Again we use the Citrus SOAP extension namespace with the specific receive action that is aware of SOAP attachment validation. The tester can validate the **content-id**, the **content-type** and the attachment content. Instead of using the external file resource you could also define an expected attachment template directly in the test case as inline CDATA section.



The **ws:attachment** element specifies a validator instance. This validator determines how to validate the attachment content. SOAP attachments are not limited to XML content. Plain text content and binary content is possible, too. So each SOAP attachment validating action can use a different ***SoapAttachmentValidator*** instance which is responsible for validating and comparing received attachments to expected template attachments. In the Citrus configuration the validator is set as normal Spring bean with the respective identifier.

Citrus Bean

```
@BindToRegistry
public SimpleSoapAttachmentValidator soapAttachmentValidator() {
    return new SimpleSoapAttachmentValidator();
}

@BindToRegistry
public MySoapAttachmentValidator mySoapAttachmentValidator() {
    return new MySoapAttachmentValidator();
}
```

Spring Bean

```
@Bean
public SimpleSoapAttachmentValidator soapAttachmentValidator() {
    return new SimpleSoapAttachmentValidator();
}

@Bean
public MySoapAttachmentValidator mySoapAttachmentValidator() {
    return new MySoapAttachmentValidator();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="soapAttachmentValidator"
class="org.citrusframework.ws.validation.SimpleSoapAttachmentValidator"/>
    <bean id="mySoapAttachmentValidator"
class="com.company.ws.validation.MySoapAttachmentValidator"/>
</beans>
```

You can define several validator instances in the Citrus configuration. The validator with the general id **"soapAttachmentValidator"** is the default validator for all actions that do not explicitly set a validator instance. Citrus offers a set of reference validator implementations. The ***SimpleSoapAttachmentValidator*** will use a simple plain text comparison. Of course you are able to add individual validator implementations, too.

16.18. SOAP MTOM support

MTOM (Message Transmission Optimization Mechanism) enables you to send and receive large SOAP message content using streamed data handlers. This optimizes the resource allocation on

server and client side where not all data is loaded into memory when marshalling/unmarshalling the message payload data. In detail MTOM-enabled messages do have a XOP package inside the message payload replacing the actual large content data. The content is then streamed aas separate attachment. Server and client can operate with a data handler providing access to the streamed content. This is very helpful when using large binary content inside a SOAP message for instance.

Citrus is able to both send and receive MTOM enabled SOAP messages on client and server. Just use the **mtom-enabled** flag when sending a SOAP message:

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapMtomClient)
        .send()
            .message()
                .mtomEnabled(true)
                .body("""
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                </image:addImage>
                """)
                .attachment("IMAGE", "application/octet-stream",
Resources.of("classpath:org/citrusframework/hugeImageData.png"))
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapMtomClient">
        <send-request>
          <message mtom-enabled="true">
            <body>
              <payload>
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                </image:addImage>
              </payload>
            </body>
            <attachment content-id="IMAGE" content-type="application/octet-stream">
              <resource file="classpath:org/citrusframework/hugeImageData.png"/>
            </attachment>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapMtomClient"
      sendRequest:
        message:
          mtomEnabled: true
          body:
            data: |
              <image:addImage xmlns:image="http://citrusframework.org/imageService/">
                <image>cid:IMAGE</image>
              </image:addImage>
          attachments:
            - contentId: IMAGE
              contentType: "application/octet-stream"
              resource: "classpath:org/citrusframework/hugeImageData.png"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:send endpoint="soapMtomClient" mtom-enabled="true">
          <message>
            <data>
              <![CDATA[
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                </image:addImage>
              ]]>
            </data>
          </message>
          <ws:attachment content-id="IMAGE" content-type="application/octet-stream">
            <ws:resource file="classpath:org/citrusframework/hugeImageData.png"/>
          </ws:attachment>
        </ws:send>
   </actions>
  </testcase>
</spring:beans>
```

As you can see the example above sends a SOAP message that contains a large binary image content. The actual binary image data is referenced with a content id marker **cid:IMAGE** inside the message payload. The actual image content is added as attachment with a separate file resource. Important is here the **content-id** which matches the id marker in the SOAP message payload (**IMAGE**).

Citrus builds a proper SOAP MTOM enabled message automatically adding the XOP package inside the message. The binary data is sent as separate SOAP attachment accordingly. The resulting SOAP message looks like this:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header></SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <image:addImage xmlns:image="http://citrusframework.org/imageService/">
      <image><xop:Include xmlns:xop="http://www.w3.org/2004/08/xop/include"
href="cid:IMAGE"/></image>
    </image:addImage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

On the server side Citrus is also able to handle MTOM enabled SOAP messages. In a server receive action you can specify the MTOM SOAP attachment content as follows.

Java

```
@CitrusTest
public void soapServerTest() {
    when(soap()
        .server(soapMtomServer)
        .receive()
            .message()
                .schemaValidation(false)
                .mtomEnabled(true)
                .body("""
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image><xop:Include
xmlns:xop="http://www.w3.org/2004/08/xop/include" href="cid:IMAGE"/></image>
                </image:addImage>
                """)
                .attachment("IMAGE", "application/octet-stream",
Resources.of("classpath:org/citrusframework/hugeImageData.png"))
    );
}
```

XML

```
<test name="SoapServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap server="soapMtomServer">
        <receive-request>
          <message mtom-enabled="true" schema-validation="false">
            <body>
              <payload>
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image><xop:Include
xmlns:xop="http://www.w3.org/2004/08/xop/include" href="cid:IMAGE"/></image>
                </image:addImage>
              </payload>
            </body>
            <attachment content-id="IMAGE" content-type="application/octet-stream">
              <resource file="classpath:org/citrusframework/hugeImageData.png"/>
            </attachment>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapServerTest
actions:
  - soap:
      server: "soapMtomServer"
      receiveRequest:
        message:
          schemaValidation: false
          mtomEnabled: true
          body:
            data: |
              <image:addImage xmlns:image="http://citrusframework.org/imageService/">
                <image><xop:Include xmlns:xop="http://www.w3.org/2004/08/xop/include"
href="cid:IMAGE"/></image>
              </image:addImage>
          attachments:
            - contentId: IMAGE
              contentType: "application/octet-stream"
              resource: "classpath:org/citrusframework/hugeImageData.png"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapServerTest">
    <actions>
        <ws:receive endpoint="soapMtomServer" mtom-enabled="true">
          <message schema-validation="false">
            <data>
              <![CDATA[
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image><xop:Include
xmlns:xop="http://www.w3.org/2004/08/xop/include" href="cid:IMAGE"/></image>
                </image:addImage>
              ]]>
            </data>
          </message>
          <ws:attachment content-id="IMAGE" content-type="application/octet-stream">
            <ws:resource file="classpath:org/citrusframework/hugeImageData.png"/>
          </ws:attachment>
        </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

We define the MTOM attachment content as separate SOAP attachment. The **content-id** is referenced somewhere in the SOAP message payload data. At runtime Citrus will add the XOP package definition automatically and perform validation on the message and its streamed MTOM attachment data.

Next thing that we have to talk about is inline MTOM data. This means that the content should be added as either **base64Binary** or **hexBinary** encoded String data directly to the message content. See the following example that uses the **mtom-inline** setting:

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapMtomClient)
        .send()
            .message()
                .mtomEnabled(true)
                .body("""
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                  <icon>cid:ICON</icon>
                </image:addImage>
                """)
                .attachment(image())
                .attachment(icon())
    );
}

private SoapAttachment image() {
    SoapAttachment attachment = new SoapAttachment("IMAGE", "application/octet-
stream", Resources.of("classpath:org/citrusframework/hugeImageData.png"));
    attachment.setMtomInline(true);
    attachment.setEncodingType("base64Binary");
    return attachment;
}

private SoapAttachment icon() {
    SoapAttachment attachment = new SoapAttachment("ICON", "application/octet-stream",
Resources.of("classpath:org/citrusframework/icon.ico"));
    attachment.setMtomInline(true);
    attachment.setEncodingType("hexBinary");
    return attachment;
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapMtomClient">
        <send-request>
          <message mtom-enabled="true">
            <body>
              <payload>
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                </image:addImage>
              </payload>
            </body>
            <attachment content-id="IMAGE" content-type="application/octet-stream"
mtom-inline="true" encoding-type="base64Binary">
              <resource file="classpath:org/citrusframework/hugeImageData.png"/>
            </attachment>
            <attachment content-id="ICON" content-type="application/octet-stream"
mtom-inline="true" encoding-type="hexBinary">
              <resource file="classpath:org/citrusframework/icon.ico"/>
            </attachment>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapMtomClient"
      sendRequest:
        message:
          mtomEnabled: true
          body:
            data: |
              <image:addImage xmlns:image="http://citrusframework.org/imageService/">
                <image>cid:IMAGE</image>
                <icon>cid:ICON</icon>
              </image:addImage>
          attachments:
            - contentId: IMAGE
              contentType: "application/octet-stream"
              mtomInline: true
              encodingType: base64Binary
              resource: "classpath:org/citrusframework/hugeImageData.png"
            - contentId: ICON
              contentType: "application/octet-stream"
              mtomInline: true
              encodingType: hexBinary
              resource: "classpath:org/citrusframework/icon.ico"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:send endpoint="soapMtomClient" mtom-enabled="true">
          <message>
            <data>
              <![CDATA[
                <image:addImage
xmlns:image="http://citrusframework.org/imageService/">
                  <image>cid:IMAGE</image>
                  <icon>cid:ICON</icon>
                </image:addImage>
              ]]>
            </data>
          </message>
          <ws:attachment content-id="IMAGE" content-type="application/octet-stream"
                    mtom-inline="true" encoding-type="base64Binary">
            <ws:resource file="classpath:org/citrusframework/image.png"/>
          </ws:attachment>
          <ws:attachment content-id="ICON" content-type="application/octet-stream"
                    mtom-inline="true" encoding-type="hexBinary">
            <ws:resource file="classpath:org/citrusframework/icon.ico"/>
          </ws:attachment>
        </ws:send>
    </actions>
  </testcase>
</spring:beans>
```

The listing above defines two inline MTOM attachments. The first attachment **cid:IMAGE** uses the encoding type **base64Binary** which is the default. The second attachment **cid:ICON** uses **hexBinary** encoding. Both attachments are added as inline data before the message is sent. The final SOAP message looks like follows:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header></SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <image:addImage xmlns:image="http://citrusframework.org/imageService/">

<image>VGhpcyBpcyBhIGJpbmFyeSBpbWFnZSBhdHRhY2htZW50IQpWYXJpYWJsZXMgJXt0ZXN0fSBzaG91bGQ
gbm90IGJlIHJlcGxhY2VkIQ==</image>

<icon>5468697320697320612062696E6172792069636F6E20617474616363686D656E74210A566172696162
6C657320257B746573747D2073686F756C64206E6F74206265207265706C6163656421</icon>
    </image:addImage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The image content is a base64Binary String and the icon a hexBinary String. Of course this mechanism also is supported in receive actions on the server side where the expected message content is added as inline MTOM data before validation takes place.

16.19. SOAP client basic authentication

As a SOAP client you may have to use basic authentication in order to access a server resource. Basic authentication via HTTP stands for username/password authentication where the credentials are transmitted in the HTTP request header section as base64 encoded entry. As Citrus uses the Spring WebService stack we can use the basic authentication support there. We set the user credentials on the HttpClient message sender which is used inside the Spring ***WebServiceTemplate*** .

Citrus provides a comfortable way to set the HTTP message sender with basic authentication credentials on the ***WebServiceTemplate*** . Just see the following example and learn how to do that.

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageSender(basicAuthClient())
            .build();
}

public HttpComponents5MessageSender basicAuthClient() {
    HttpComponents5MessageSender messageSender = new HttpComponents5MessageSender();
    messageSender.setAuthScope(new AuthScope("http", "localhost", 8090, null,
"basic"));
    messageSender.setCredentials(new UsernamePasswordCredentials("someUsername",
"somePassword"));
    return messageSender;
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageSender(basicAuthClient())
            .build();
}

public HttpComponents5MessageSender basicAuthClient() {
    HttpComponents5MessageSender messageSender = new HttpComponents5MessageSender();
    messageSender.setAuthScope(new AuthScope("http", "localhost", 8090, null,
"basic"));
    messageSender.setCredentials(new UsernamePasswordCredentials("someUsername",
"somePassword"));
    return messageSender;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <citrus-ws:client id="soapClient"
                      request-url="http://localhost:8090/test"
                      message-sender="basicAuthClient"/>

    <bean id="basicAuthClient"
class="org.springframework.ws.transport.http.HttpComponents5MessageSender">
      <property name="authScope">
          <bean class="org.apache.hc.client5.http.auth.AuthScope">
            <constructor-arg value="http"/>
            <constructor-arg value="localhost"/>
            <constructor-arg value="8090"/>
            <constructor-arg>
              <null/>
            </constructor-arg>
            <constructor-arg value="basic"/>
          </bean>
      </property>
      <property name="credentials">
        <bean class="org.apache.hc.client5.http.auth.UsernamePasswordCredentials">
            <constructor-arg value="someUsername"/>
            <constructor-arg value="somePassword"/>
        </bean>
      </property>
    </bean>
</beans>
```

The above configuration results in SOAP requests with authentication headers properly set for basic authentication. The special message sender takes care of adding the proper basic authentication header to each request that is sent with this Citrus message sender. By default, preemptive authentication is used. The message sender only sends a single request to the server with all authentication information set in the message header. The request which determines the authentication scheme on the server is skipped. This is why you have to add some auth scope so Citrus can set up an authentication cache within the HTTP context in order to have preemptive authentication.



You can also skip the message sender configuration and set the **Authorization** header on each request in your send action definition on your own. Be aware of setting the header as HTTP mime header using the correct prefix and take care of using the correct basic authentication with base64 encoding for the **username:password** phrase.

Java

```
.header("citrus_http_Authorization", "Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==")
```

XML

```
<headers>
  <header name="citrus_http_Authorization"
          value="Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==" />
</headers>
```

YAML

```
headers:
  - name: citrus_http_Authorization
    value: "Basic c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA=="
```

Spring XML

```
<header>
    <element name="citrus_http_Authorization" value="Basic
c29tZVVzZXJuYW1lOnNvbWVQYXNzd29yZA==">
</header>
```

For base64 encoding you can also use a Citrus function, see functions-encode-base64

16.20. SOAP server basic authentication

When providing SOAP WebService server functionality Citrus can also set basic authentication so all clients need to authenticate properly when accessing the server resource.

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .securityHandler(basicSecurityHandler())
            .build();
}

private SecurityHandlerFactory basicSecurityHandler() {
    SecurityHandlerFactory securityHandlerFactory = new SecurityHandlerFactory();
    securityHandlerFactory.setUsers(Collections.singletonList(new User("citrus",
"secret", new String[] {"CitrusRole"})));
    securityHandlerFactory.setContraints(Collections.singletonMap("/foo/*", new
BasicAuthConstraint("CitrusRole")));
    return securityHandlerFactory;
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .port(8080)
            .autoStart(true)
            .securityHandler(basicSecurityHandler())
            .build();
}

private SecurityHandlerFactory basicSecurityHandler() {
    SecurityHandlerFactory securityHandlerFactory = new SecurityHandlerFactory();
    securityHandlerFactory.setUsers(Collections.singletonList(new User("citrus",
"secret", new String[] {"CitrusRole"})));
    securityHandlerFactory.setContraints(Collections.singletonMap("/foo/*", new
BasicAuthConstraint("CitrusRole")));
    return securityHandlerFactory;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <citrus-ws:server id="soapServer"
                 port="8080"
                 auto-start="true"
                 security-handler="basicSecurityHandler"/>

    <bean id="securityHandler"
class="org.citrusframework.ws.security.SecurityHandlerFactory">
        <property name="users">
            <list>
                <bean class="org.citrusframework.ws.security.User">
                    <property name="name" value="citrus"/>
                    <property name="password" value="secret"/>
                    <property name="roles" value="CitrusRole"/>
                </bean>
            </list>
        </property>
        <property name="constraints">
            <map>
                <entry key="/foo/*">
                    <bean class="org.citrusframework.ws.security.BasicAuthConstraint">
                        <constructor-arg value="CitrusRole"/>
                    </bean>
                </entry>
            </map>
        </property>
    </bean>
</beans>
```

We have set a security handler on the server web container with a constraint on all resources with `/foo/*`. Following from that the server requires basic authentication for these resources. The granted users and roles are specified within the security handler bean definition. Connecting clients have to set the basic auth HTTP header properly using the correct user and role for accessing the Citrus server now.

You can customize the security handler for your very specific needs (e.g. load users and roles with JDBC from a database). Just have a look at the code base and inspect the settings and properties

offered by the security handler interface.



This mechanism is not restricted to basic authentication only. With other settings you can also set up digest or form-based authentication constraints very easy.

16.21. WS-Addressing support

The web service stack offers a lot of different technologies and standards within the context of SOAP WebServices. We speak of WS-* specifications in particular. One of these specifications deals with addressing. On client side you may add wsa header information to the request in order to give the server instructions how to deal with SOAP faults for instance.

In Citrus WebService client you can add those header information using the common configuration like this:

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageConverter(wsAddressingMessageConverter())
            .build();
}

public WsAddressingMessageConverter wsAddressingMessageConverter() {
    WsAddressingHeaders wsAddressingHeaders = new WsAddressingHeaders();
    wsAddressingHeaders.setVersion(WsAddressingVersion.VERSION200408);
    wsAddressingHeaders.setAction("http://citrus.sample/sayHello");
    wsAddressingHeaders.setFrom("http://citrus.sample/client");
    wsAddressingHeaders.setTo("http://citrus.sample/server");
    wsAddressingHeaders.setFaultTo("http://citrus.sample/fault/resolver");

    return new WsAddressingMessageConverter(wsAddressingHeaders);
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8090/test")
            .messageConverter(wsAddressingMessageConverter())
            .build();
}

public WsAddressingMessageConverter wsAddressingMessageConverter() {
    WsAddressingHeaders wsAddressingHeaders = new WsAddressingHeaders();
    wsAddressingHeaders.setVersion(WsAddressingVersion.VERSION200408);
    wsAddressingHeaders.setAction("http://citrus.sample/sayHello");
    wsAddressingHeaders.setFrom("http://citrus.sample/client");
    wsAddressingHeaders.setTo("http://citrus.sample/server");
    wsAddressingHeaders.setFaultTo("http://citrus.sample/fault/resolver");

    return new WsAddressingMessageConverter(wsAddressingHeaders);
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <citrus-ws:client id="soapClient"
                      request-url="http://localhost:8090/test"
                      message-converter="wsAddressingMessageConverter"/>

    <bean id="wsAddressingMessageConverter"
class="org.citrusframework.ws.message.converter.WsAddressingMessageConverter">
      <constructor-arg>
        <bean id="wsAddressing200408"
class="org.citrusframework.ws.addressing.WsAddressingHeaders">
            <property name="version" value="VERSION200408"/>
            <property name="action" value="http://citrus.sample/sayHello"/>
            <property name="to" value="http://citrus.sample/server"/>
            <property name="from">
                <bean
class="org.springframework.ws.soap.addressing.core.EndpointReference">
                    <constructor-arg value="http://citrus.sample/client"/>
                </bean>
            </property>
            <property name="replyTo">
                <bean
class="org.springframework.ws.soap.addressing.core.EndpointReference">
                    <constructor-arg value="http://citrus.sample/client"/>
                </bean>
            </property>
            <property name="faultTo">
                <bean
class="org.springframework.ws.soap.addressing.core.EndpointReference">
                    <constructor-arg value="http://citrus.sample/fault/resolver"/>
                </bean>
            </property>
        </bean>
      </constructor-arg>
    </bean>
</beans>
```

The WsAddressing header values will be used for all request messages that are sent with the soap

client component *soapClient*. You can overwrite the WsAddressing header in each send test action in your test though. Just set the special WsAddressing message header on your request. You can use the following message header names in order to overwrite the default addressing headers specified in the message converter configuration (also see the class *org.citrusframework.ws.addressing.WsAddressingMessageHeaders*).

citrus_soap_ws_addressing_messageId	addressing message id as URI
citrus_soap_ws_addressing_from	addressing from endpoint reference as URI
citrus_soap_ws_addressing_to	addressing to URI
citrus_soap_ws_addressing_action	addressing action URI
citrus_soap_ws_addressing_replyTo	addressing reply to endpoint reference as URI
citrus_soap_ws_addressing_faultTo	addressing fault to endpoint reference as URI

When using these message headers you are able to explicitly overwrite the WsAddressing headers. Test variables are supported of course when specifying the values. Most of the values are parsed to a URI value at the end so please make sure to use correct URI String representations.



The WS-Addressing specification knows several versions. Supported versions are:

VERSION10	WS-Addressing 1.0 May 2006
VERSION200408	August 2004 edition of the WS-Addressing specification

The addressing headers find a place in the SOAP message header with respective namespaces and values. A possible SOAP request with WS addressing headers looks like follows:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing">
        <wsa:To SOAP-ENV:mustUnderstand="1">http://citrus.sample/server</wsa:To>
        <wsa:From>
            <wsa:Address>http://citrus.sample/client</wsa:Address>
        </wsa:From>
        <wsa:ReplyTo>
            <wsa:Address>http://citrus.sample/client</wsa:Address>
        </wsa:ReplyTo>
        <wsa:FaultTo>
            <wsa:Address>http://citrus.sample/fault/resolver</wsa:Address>
        </wsa:FaultTo>
        <wsa:Action>http://citrus.sample/sayHello</wsa:Action>
        <wsa:MessageID>urn:uuid:4c4d8af2-b402-4bc0-a2e3-ad33b910e394</wsa:MessageID>
    </SOAP-ENV:Header>
    <SOAP-ENV:Body>
        <cit:HelloRequest xmlns:cit="http://citrus/sample/sayHello">
            <cit:Text>Hello Citrus!</cit:Text>
        </cit:HelloRequest>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```



By default, when not set explicitly on the message headers the WsAddressing message id property is automatically generated for each request. You can set the message id generation strategy in the Spring application context message converter configuration:

Citrus Bean

```
public WsAddressingMessageConverter wsAddressingMessageConverter() {
    WsAddressingHeaders wsAddressingHeaders = new WsAddressingHeaders();
    wsAddressingHeaders.setVersion(WsAddressingVersion.VERSION200408);
    wsAddressingHeaders.setAction("http://citrus.sample/sayHello");
    wsAddressingHeaders.setFrom("http://citrus.sample/client");
    wsAddressingHeaders.setTo("http://citrus.sample/server");
    wsAddressingHeaders.setFaultTo("http://citrus.sample/fault/resolver");

    WsAddressingMessageConverter messageConverter = new
WsAddressingMessageConverter(wsAddressingHeaders);
    messageConverter.setMessageIdStrategy(new UuidMessageIdStrategy());
    return messageConverter;
}
```

Spring Bean

```
public WsAddressingMessageConverter wsAddressingMessageConverter() {
    WsAddressingHeaders wsAddressingHeaders = new WsAddressingHeaders();
    wsAddressingHeaders.setVersion(WsAddressingVersion.VERSION200408);
    wsAddressingHeaders.setAction("http://citrus.sample/sayHello");
    wsAddressingHeaders.setFrom("http://citrus.sample/client");
    wsAddressingHeaders.setTo("http://citrus.sample/server");
    wsAddressingHeaders.setFaultTo("http://citrus.sample/fault/resolver");

    WsAddressingMessageConverter messageConverter = new
WsAddressingMessageConverter(wsAddressingHeaders);
    messageConverter.setMessageIdStrategy(new UuidMessageIdStrategy());
    return messageConverter;
}
```

Spring XML

```
<bean id="wsAddressingMessageConverter"
class="org.citrusframework.ws.message.converter.WsAddressingMessageConverter">
  <property name="messageIdStrategy">
    <bean
class="org.springframework.ws.soap.addressing.messageid.UuidMessageIdStrategy"/>
  </property>
</bean>
```

By default, the strategy will create a new Java UUID for each request. The strategy also uses a common resource name prefix *urn:uuid:*. You can overwrite the message id any time for each request explicitly by setting the message header *citrus_soap_ws_addressing_messageId* with a respective value on the message in your test.

16.22. SOAP client fork mode

SOAP over HTTP uses synchronous communication by nature. This means that sending a SOAP message in Citrus over HTTP will automatically block further test actions until the synchronous HTTP response has been received. In test cases this synchronous blocking might cause problems for several reasons. A simple reason would be that you need to do further test actions in parallel to the synchronous HTTP SOAP communication (e.g. simulate another backend system in the test case).

You can separate the SOAP send action from the rest of the test case by using the **"fork"** mode. The SOAP client will automatically open a new Java Thread for the synchronous communication and the test is able to continue with execution although the synchronous HTTP SOAP response has not arrived yet.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .send()
            .fork(true)
            .message()
                .body("""
                <SoapRequest xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Do something</Operation>
                </SoapRequest>
                """)
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <send-request fork="true">
          <message>
            <body>
              <payload>
                <SoapRequest xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Do something</Operation>
                </SoapRequest>
              </payload>
            </body>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      sendRequest:
        fork: true
        message:
          body:
            data: |
              <SoapRequest xmlns="http://citrusframework.org/schemas/sample.xsd">
                <Operation>Do something</Operation>
              </SoapRequest>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
      <ws:send endpoint="soapClient" fork="true">
          <message>
            <payload>
                <SoapRequest xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Do something</Operation>
                </SoapRequest>
            </payload>
          </message>
      </ws:send>
    </actions>
  </testcase>
</spring:beans>
```

With the **"fork"** mode enabled the test continues with execution while the sending action waits for the synchronous response in a separate Java Thread. You could reach the same behaviour with a complex <parallel>/<sequential> container construct, but forking the send action is much more straight forward.



It is highly recommended to use a proper **"timeout"** setting on the SOAP receive action when using fork mode. The forked send operation might take some time and the corresponding receive action might run into failure as the response was has not been received yet. The result would be a broken test because of the missing response message. A proper **"timeout"** setting for the receive action solves this problem as the action waits for this time period and occasionally repeatedly asks for the SOAP response message. The following listing sets the receive timeout to 10 seconds, so the action waits for the forked send action to deliver the SOAP response in time.

Java

```
@CitrusTest
public void soapClientTest() {
    when(soap()
        .client(soapClient)
        .receive()
            .timeout(10000L)
            .message()
                .body("""
                <SoapResponse xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Did something</Operation>
                  <Success>true</Success>
                </SoapResponse>
                """)
    );
}
```

XML

```
<test name="SoapClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <soap client="soapClient">
        <receive-response timeout="10000">
          <message>
            <body>
              <payload>
                <SoapResponse xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Did something</Operation>
                  <Success>true</Success>
                </SoapResponse>
              </payload>
            </body>
          </message>
        </send-request>
      </soap>
    </actions>
</test>
```

YAML

```
name: SoapClientTest
actions:
  - soap:
      client: "soapClient"
      receiveResponse:
        timeout: 10000
        message:
          body:
            data: |
              <SoapResponse xmlns="http://citrusframework.org/schemas/sample.xsd">
                <Operation>Did something</Operation>
                <Success>true</Success>
              </SoapResponse>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ws="http://www.citrusframework.org/schema/ws/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/ws/testcase
          http://www.citrusframework.org/schema/ws/testcase/citrus-ws-testcase.xsd">

  <testcase name="soapClientTest">
    <actions>
        <ws:receive endpoint="soapClient" timeout="10000">
          <message>
            <payload>
                <SoapResponse xmlns="http://citrusframework.org/schemas/sample.xsd">
                  <Operation>Did something</Operation>
                  <Success>true</Success>
                </SoapResponse>
            </payload>
          </message>
        </ws:receive>
    </actions>
  </testcase>
</spring:beans>
```

16.23. SOAP servlet context customization

For highly customized SOAP server components in Citrus you can define a full servlet context configuration file. Here you have the full power to add Spring endpoint mappings and custom endpoint implementations. You can set the custom servlet context as external file resource on the

server component:

Citrus Bean

```
@BindToRegistry
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .contextConfigLocation("classpath:citrus-ws-servlet.xml")
            .build();
}
```

Spring Bean

```
@Bean
public WebServiceServer soapServer() {
    return new WebServiceServerBuilder()
            .contextConfigLocation("classpath:citrus-ws-servlet.xml")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">

    <citrus-ws:server id="soapServer"
          context-config-location="classpath:citrus-ws-servlet.xml"/>
</beans>
```

Now let us have a closer look at the context-config-location attribute. This configuration defines the Spring application context file for endpoints, request mappings and other SpringWS specific information. Please see the official SpringWS documentation for details on this Spring based configuration. You can also just copy the following example application context which should work for you in general.

citrus-ws-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
```

```
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="loggingInterceptor"

class="org.springframework.ws.server.endpoint.interceptor.PayloadLoggingInterceptor">
      <description>
          This interceptor logs the message payload.
      </description>
  </bean>

  <bean id="helloServicePayloadMapping"

class="org.springframework.ws.server.endpoint.mapping.PayloadRootQNameEndpointMapping"
>
      <property name="mappings">
          <props>
              <prop
                  key="{http://citrusframework.org/schemas/sayHello}HelloRequest">
                  helloServiceEndpoint
              </prop>
          </props>
      </property>
      <property name="interceptors">
          <list>
              <ref bean="loggingInterceptor"/>
          </list>
      </property>
  </bean>

  <bean id="helloServiceEndpoint"
class="org.citrusframework.ws.server.WebServiceEndpoint">
      <property name="endpointAdapter" ref="staticResponseEndpointAdapter"/>
  </bean>

  <citrus:static-response-adapter id="staticResponseEndpointAdapter">
      <citrus:payload>
          <![CDATA[
              <HelloResponse xmlns="http://citrusframework.org/schemas/sayHello">
                  <MessageId>123456789</MessageId>
                  <CorrelationId>CORR123456789</CorrelationId>
                  <User>WebServer</User>
                  <Text>Hello User</Text>
              </HelloResponse>
          ]]>
      </citrus:payload>
      <citrus:header>
          <citrus:element
name="{http://citrusframework.org/schemas/samples/sayHello.xsd}ns0:Operation"
                  value="sayHelloResponse"/>
          <citrus:element
```

```
name="{http://citrusframework.org/schemas/samples/sayHello.xsd}ns0:Request"
                  value="HelloRequest"/>
          <citrus:element name="citrus_soap_action"
                  value="sayHello"/>
      </citrus:header>
  </citrus:static-response-adapter>
</beans>
```

The program listing above describes a normal SpringWS request mapping with endpoint configurations. The mapping is responsible to forward incoming requests to the endpoint which will handle the request and provide a proper response message. First of all we add a logging interceptor to the context so all incoming requests get logged to the console first. Then we use a payload mapping (PayloadRootQNameEndpointMapping) in order to map all incoming ***'HelloRequest'*** SOAP messages to the ***'helloServiceEndpoint'*** . Endpoints are of essential nature in Citrus SOAP WebServices implementation. They are responsible for processing a request in order to provide a proper response message that is sent back to the calling client. Citrus uses the endpoint in combination with a message endpoint adapter implementation.



The endpoint works together with the message endpoint adapter that is responsible for providing a response message for the client. The various message endpoint adapter implementations in Citrus were already discussed in endpoint-adapter.

In this example the ***'helloServiceEndpoint'*** uses the ***'static-response-adapter'*** which is always returning a static response message. In most cases static responses will not fit the test scenario and you will have to respond more dynamically.

Regardless of which message endpoint adapter setup you are using in your test case the endpoint transforms the response into a proper SOAP message. You can add as many request mappings and endpoints as you want to the server context configuration. So you are able to handle different request types with one single Jetty server instance.

That's it for connecting with SOAP WebServices! We saw how to send and receive SOAP messages with Jetty and Spring WebServices. Have a look at the samples coming with your Citrus archive in order to learn more about the SOAP message handling.

Chapter 17. Apache Camel support

Apache Camel is a fantastic Open Source integration framework that empowers you to quickly and easily integrate various systems consuming or producing data.

Camel implements the enterprise integration patterns for building mediation and routing rules in your software. With the Camel support in Citrus you are able to directly interact with the 300+ Camel components through route definitions. You can call Camel routes and receive synchronous response messages from a Citrus test. You can also simulate the Camel route endpoint with receiving messages and providing simulated response messages.



The camel components in Citrus are located in a separate Maven module. So you should add the module as Maven dependency to your project accordingly.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-camel</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

For XML configurations Citrus provides a special Camel configuration schema that is used in Spring configuration files. You have to include the `citrus-camel` namespace in your Spring configuration XML files as follows.

XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:citrus="http://www.citrusframework.org/schema/config"
      xmlns:citrus-camel="http://www.citrusframework.org/schema/camel/config"
      xsi:schemaLocation="
          http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/config
          http://www.citrusframework.org/schema/config/citrus-config.xsd
          http://www.citrusframework.org/schema/camel/config
          http://www.citrusframework.org/schema/camel/config/citrus-camel-config.xsd">

      [...]

      </beans>
```

Now you are ready to use the Camel configuration elements using the citrus-camel namespace prefix.

The next sections explain the Citrus capabilities while working with Apache Camel.

17.1. Camel endpoint

Camel and Citrus both use the endpoint pattern in order to define message destinations. Users can interact with these endpoints when creating the mediation and routing logic. The Citrus endpoint component for Camel interaction is defined as follows in your Citrus Spring configuration.

Java

```
@Bean
public CamelEndpoint helloCamelEndpoint() {
    return new CamelEndpointBuilder()
        .endpointUri("direct:hello")
        .build();
}
```

XML

```
<citrus-camel:endpoint id="helloCamelEndpoint"
      endpoint-uri="direct:hello"/>
```

The endpoint defines an endpoint uri that will be called when messages are sent/received using the endpoint producer or consumer. The endpoint uri refers to a Camel route that is located inside a Camel context. The Camel route should use the respective endpoint uri as `from` definition.

Java

```
@Bean
public CamelContext camelContext() {
    CamelContext context = new DefaultCamelContext();

    context.addRoutes(new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:hello")
                .routeId("helloRoute")
                .to("log:org.citrusframework.camel?level=INFO")
                .to("seda:greetings");
        }
    });

    return context;
}
```

XML

```
<camelContext id="camelContext" xmlns="http://camel.apache.org/schema/spring">
  <route id="helloRoute">
    <from uri="direct:hello"/>
    <to uri="log:org.citrusframework.camel?level=INFO"/>
    <to uri="seda:greetings-feed"/>
  </route>
</camelContext>
```

In the example above the Camel context is placed as Spring bean. The context defines a Camel route the Citrus test case can interact with.

The Camel context is automatically referenced in the Citrus Camel endpoint. This is because Citrus will automatically look for a Camel context in the Spring bean configuration.

In case you have multiple Camel context instances in your configuration you can explicitly link the endpoint to a context with `camel-context="camelContext"`.

Java

```
@Bean
public CamelEndpoint helloCamelEndpoint() {
    return new CamelEndpointBuilder()
        .camelContext(specialCamelContext)
        .endpointUri("direct:hello")
        .build();
}
```

XML

```
<citrus-camel:endpoint id="helloCamelEndpoint"
      camel-contxt="specialCamelContext"
      endpoint-uri="direct:hello"/>
```

This explicitly binds the endpoint to the context named "*specialCamelContext*". This configuration would be the easiest setup to use Camel with Citrus as you can add the Camel context straight to the Spring bean application context and interact with it in Citrus. Of course, you can also import your Camel context and routes from other Spring bean context files, or you can start the Camel context routes with Java code.

In the example the Camel route is listening on the route endpoint uri `direct:hello`. Incoming messages will be logged to the console using a `log` Camel component. After that the message is forwarded to a `seda` Camel component which is a simple queue in memory.

The Citrus endpoint can interact with this sample route definition sending messages to the `direct:hello` endpoint. The endpoint configuration holds the endpoint uri information that tells Citrus how to access the Camel route. This endpoint uri can be any Camel endpoint uri that is used in a Camel route.

The Camel routes support asynchronous and synchronous message communication patterns. By default, Citrus uses asynchronous communication with Camel routes. This means that the Citrus producer sends the exchange message to the route endpoint uri and is finished immediately. There is no synchronous response to await. In contrary to that the synchronous endpoint will send and receive a synchronous message on the Camel destination route. This message exchange pattern is discussed in a later section in this chapter.

For now, we have a look on how to use the Citrus Camel endpoint in a test case in order to send a message to the Camel route:

Java

```
send(helloCamelEndpoint)
    .message()
    .body("Hello from Citrus!");
```

XML

```
<send endpoint="helloCamelEndpoint">
  <message type="plaintext">
    <payload>Hello from Citrus!</payload>
  </message>
</send>
```

You can use the very same Citrus Camel endpoint component to receive messages in your test case, too. In this situation you would receive a message from the route endpoint. This is especially designed for queueing endpoint routes such as the Camel seda component. In our example Camel route above the seda Camel component is called with the endpoint uri **seda:greetings-feed**.

This means that the Camel route is sending a message to the `seda` component. Citrus is able to receive this route message with an endpoint component like this:

Java

```
@Bean
public CamelEndpoint greetingsFeed() {
    return new CamelEndpointBuilder()
        .endpointUri("seda:greetings-feed")
        .build();
}
```

XML

```
<citrus-camel:endpoint id="greetingsFeed"
    endpoint-uri="seda:greetings-feed"/>
```

You can use the Citrus camel endpoint in your test case receive action in order to consume the message on the seda component.

Java

```
receive(greetingsFeed)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Hello from Citrus!");
```

XML

```
<receive endpoint="greetingsFeed">
  <message type="plaintext">
    <payload>Hello from Citrus!</payload>
  </message>
</receive>
```



Instead of defining a static Citrus camel component you could also use the dynamic endpoint components in Citrus. This would enable you to send your message directly using the endpoint uri **direct:news** in your test case. Read more about this in dynamic-endpoint-components.

Citrus is able to send and receive messages with Camel route endpoint uri. This enables you to invoke a Camel route. The Camel components used is defined by the endpoint uri as usual. When interacting with Camel routes you might need to send back some response messages in order to simulate boundary applications. We will discuss the synchronous communication in the next section.

17.2. Synchronous Camel endpoint

The synchronous Camel producer sends a message to a route and waits synchronously for the response to arrive. In Camel this communication is represented with the exchange pattern **InOut**. The basic configuration for a synchronous Camel endpoint component looks like follows:

Java

```
@Bean
public CamelSyncEndpoint helloCamelEndpoint() {
    return new CamelSyncEndpointBuilder()
        .endpointUri("direct:hello")
        .timeout(1000L)
        .pollingInterval(300L)
        .build();
}
```

XML

```
<citrus-camel:sync-endpoint id="helloCamelEndpoint"
      endpoint-uri="direct:hello"
      timeout="1000"
      polling-interval="300"/>
```

Synchronous endpoints poll for synchronous reply messages to arrive. The poll interval is an optional setting in order to manage the amount of reply message handshake attempts. Once the endpoint was able to receive the reply message synchronously the test case can receive the reply. In case the reply message is not available in time we raise some timeout error and the test will fail.

In a first test scenario we write a test case that sends a message to the synchronous endpoint and waits for the synchronous reply message to arrive. So we have two actions on the same Citrus endpoint, first send then receive.

Java

```
send(helloCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Hello from Citrus!");

receive(helloCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("This is the reply from Apache Camel!");
```

XML

```
<send endpoint="helloCamelEndpoint">
  <message type="plaintext">
    <payload>Hello from Citrus!</payload>
  </message>
</send>

<receive endpoint="helloCamelEndpoint">
  <message type="plaintext">
    <payload>This is the reply from Apache Camel!</payload>
  </message>
</receive>
```

The next variation deals with the same synchronous communication, but send and receive roles are switched. Now Citrus receives a message from a Camel route and has to provide a reply message. We handle this synchronous communication with the same synchronous Apache Camel endpoint component. Only difference is that we initially start the communication by receiving a message from the endpoint. Knowing this Citrus is able to send a synchronous response back. Again just use the same endpoint reference in your test case. So we have again two actions in our test case, but this time first receive then send.

Java

```
receive(helloCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Hello from Apache Camel!");

send(helloCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("This is the reply from Citrus!");
```

XML

```
<receive endpoint="helloCamelEndpoint">
  <message type="plaintext">
    <payload>Hello from Apache Camel!</payload>
  </message>
</receive>

<send endpoint="helloCamelEndpoint">
  <message type="plaintext">
    <payload>This is the reply from Citrus!</payload>
  </message>
</send>
```

This is pretty simple. Citrus takes care of setting the Camel exchange pattern **InOut** while using synchronous communications. The Camel routes do respond and Citrus is able to receive the synchronous messages accordingly. With this pattern you can interact with Camel routes where Citrus simulates synchronous clients and consumers.

17.3. Camel exchange headers

Camel uses exchanges when sending and receiving messages to and from routes. These exchanges hold specific information on the communication outcome. Citrus automatically converts these exchange information to special message header entries. You can validate those exchange headers then easily in your test case:

Java

```
receive(greetingsFeed)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Hello from Camel!")
    .header("citrus_camel_route_id", "greetings")
    .header("citrus_camel_exchange_id", "ID-local-50532-1402653725341-0-3")
    .header("citrus_camel_exchange_failed", false)
    .header("citrus_camel_exchange_pattern", "InOnly")
    .header("CamelCorrelationId", "ID-local-50532-1402653725341-0-1")
    .header("CamelToEndpoint", "seda://greetings-feed");
```

XML

```
<receive endpoint="greetingsFeed">
  <message type="plaintext">
    <payload>Hello from Camel!</payload>
  </message>
  <header>
    <element name="citrus_camel_route_id" value="greetings"/>
    <element name="citrus_camel_exchange_id" value="ID-local-50532-1402653725341-0-
3"/>
    <element name="citrus_camel_exchange_failed" value="false"/>
    <element name="citrus_camel_exchange_pattern" value="InOnly"/>
    <element name="CamelCorrelationId" value="ID-local-50532-1402653725341-0-1"/>
    <element name="CamelToEndpoint" value="seda://greetings-feed"/>
  </header>
</receive>
```

In addition to the Camel specific exchange information the Camel exchange does also hold some custom properties. These properties such as **CamelToEndpoint** or **CamelCorrelationId** are also added automatically to the Citrus message header so can expect them in a `receive` message action.

17.4. Camel exception handling

Let us suppose the following route definition:

Java

```
@Bean
public CamelContext camelContext() {
    CamelContext context = new DefaultCamelContext();

    context.addRoutes(new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:hello")
                .routeId("helloRoute")
                .to("log:org.citrusframework.camel?level=INFO")
                .to("seda:greetings-feed")
                .onException(CitrusRuntimeException.class)
                    .to("seda:exceptions");
        }
    });

    return context;
}
```

XML

```
<camelContext id="camelContext" xmlns="http://camel.apache.org/schema/spring">
  <route id="helloRoute">
    <from uri="direct:hello"/>
    <to uri="log:org.citrusframework.camel?level=INFO"/>
    <to uri="seda:greetings-feed"/>
    <onException>
      <exception>org.citrusframework.exceptions.CitrusRuntimeException</exception>
      <to uri="seda:exceptions"/>
    </onException>
  </route>
</camelContext>
```

The route has an exception handling block defined that is called as soon as the exchange processing ends up in some error or exception. With Citrus you can also simulate an exchange exception when sending back a synchronous response to a calling route.

Java

```
send(helloCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Something went wrong!")
    .header("citrus_camel_exchange_exception", CitrusRuntimeException.class)
    .header("citrus_camel_exchange_exception_message", "Something went wrong!")
    .header("citrus_camel_exchange_failed", true);
```

XML

```
<send endpoint="greetingsFeed">
  <message type="plaintext">
    <payload>Something went wrong!</payload>
  </message>
  <header>
    <element name="citrus_camel_exchange_exception"
                value="org.citrusframework.exceptions.CitrusRuntimeException"/>
    <element name="citrus_camel_exchange_exception_message" value="Something went
wrong!"/>
    <element name="citrus_camel_exchange_failed" value="true"/>
  </header>
</send>
```

This message as response to the **seda:greetings-feed** route would cause Camel to enter the exception handling in the route definition. The exception handling is activated and calls the error handling route endpoint **seda:exceptions** . Of course Citrus would be able to receive such an exception exchange validating the exception handling outcome.

In such failure scenarios the Camel exchange holds the exception information (**CamelExceptionCaught**) such as causing exception class and error message. These headers are present in an error scenario and can be validated in Citrus when receiving error messages as follows:

Java

```
receive(errorCamelEndpoint)
    .message()
    .type(MessageType.PLAINTEXT)
    .body("Something went wrong!")
    .header("citrus_camel_route_id", "helloRoute")
    .header("citrus_camel_exchange_failed", true)
    .header("CamelExceptionCaught",
"org.citrusframework.exceptions.CitrusRuntimeException: Something went wrong!");
```

XML

```
<receive endpoint="errorCamelEndpoint">
  <message type="plaintext">
    <payload>Something went wrong!</payload>
  </message>
  <header>
    <element name="citrus_camel_route_id" value="helloRoute"/>
    <element name="citrus_camel_exchange_failed" value="true"/>
    <element name="CamelExceptionCaught"
        value="org.citrusframework.exceptions.CitrusRuntimeException: Something went
wrong!"/>
  </header>
</receive>
```

This completes the basic exception handling in Citrus when using the Camel endpoints.

17.5. Camel context handling

In the previous samples we have used the Camel context as Spring bean context that is automatically loaded when Citrus starts up. Now when using a single Camel context instance Citrus is able to automatically pick this Camel context for route interaction. If you use more than one Camel context you have to tell the Citrus endpoint component which context to use. The endpoint offers an optional attribute called `camel-context`.

Java

```
@Bean
public CamelEndpoint newsCamelEndpoint() {
    return new CamelEndpointBuilder()
        .camelContext(newsContext())
        .endpointUri("direct:news")
        .build();
}

@Bean
public CamelContext newsContext() {
    CamelContext context = new DefaultCamelContext();

    context.addRoutes(new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:news")
                .routeId("newsRoute")
                .to("log:org.citrusframework.camel?level=INFO")
                .to("seda:news-feed");
        }
    });

    return context;
}

@Bean
public CamelContext helloContext() {
    CamelContext context = new DefaultCamelContext();

    context.addRoutes(new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:hello")
                .routeId("helloRoute")
                .to("log:org.citrusframework.camel?level=INFO")
                .to("seda:greetings");
        }
    });

    return context;
}
```

XML

```
<citrus-camel:endpoint id="newsCamelEndpoint"
    camel-context="newsContext"
    endpoint-uri="direct:news"/>

<camelContext id="newsContext" xmlns="http://camel.apache.org/schema/spring">
    <route id="newsRoute">
      <from uri="direct:news"/>
      <to uri="log:org.citrusframework.camel?level=INFO"/>
      <to uri="seda:news-feed"/>
    </route>
</camelContext>

<camelContext id="helloContext" xmlns="http://camel.apache.org/schema/spring">
  <route id="helloRoute">
    <from uri="direct:hello"/>
    <to uri="log:org.citrusframework.camel?level=INFO"/>
    <to uri="seda:greetings"/>
  </route>
</camelContext>
```

In the example above we have two Camel context instances loaded. The endpoint has to pick the context to use with the attribute **camel-context** which resides to the Spring bean id of the Camel context.

17.6. Camel route actions

Since Citrus 2.4 we introduced some Camel specific test actions that enable easy interaction with Camel routes and the Camel context.



In XML the Camel route test actions do follow a specific XML namespace. This means you have to add this namespace to the test case when using the actions.

XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:camel="http://www.citrusframework.org/schema/camel/testcase"
      xsi:schemaLocation="
          http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/camel/testcase
          http://www.citrusframework.org/schema/camel/testcase/citrus-camel-
testcase.xsd">

  [...]

</beans>
```

Once you have added the special Camel namespace with prefix `camel` you are ready to start using the Camel test actions in your test case.

Create Camel routes

You can create a new Camel route as part of the test using this test action.

Java

```
public class CamelRouteActionIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void createCamelRoute() {
        $(camel().camelContext(camelContext)
            .route()
            .create(new RouteBuilder() {
                @Override
                public void configure() throws Exception {
                    from("direct:messages")
                        .routeId("message-tokenizer")
                        .split().tokenize(" ")
                        .to("seda:words");
                }
            }));
    }
}
```

XML

```
<testcase name="CamelRouteIT">
  <actions>
      <camel:create-routes>
        <routeContext xmlns="http://camel.apache.org/schema/spring">
          <route id="message-tokenizer">
            <from uri="direct:messages"/>
            <split>
              <tokenize token=" "/>
              <to uri="seda:words"/>
            </split>
          </route>
        </routeContext>
      </camel:create-routes>
  </actions>
</testcase>
```

In the example above we have used the **camel:create-route** test action that will create new Camel routes at runtime in the Camel context. The target Camel context is referenced with an automatic context lookup.



The default Camel context name in this lookup is "*citrusCamelContext*". You can set this default context name via environment variables (*CITRUS_CAMEL_CONTEXT_NAME*) or system properties (*citrus.camel.context.name*).

If no specific settings are set Citrus will automatically try to look up the Camel context with name "*citrusCamelContext*" in the Spring bean configuration. All route operations will target this Camel context then.

In addition to that you can skip this lookup and directly reference a target Camel context with the action attribute **camel-context** (used in the second action above).

Remove Camel routes

You can remove routes from the Camel context as part of the test.

Java

```
public class CamelRouteActionIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void removeCamelRoutes() {
        $(camel().camelContext(camelContext)
            .route()
            .remove("route_1", "route_2", "route_3"));
    }
}
```

XML

```
<testcase name="CamelRouteIT">
  <actions>
      <camel:remove-routes camel-context="camelContext">
        <route id="route_1"/>
        <route id="route_2"/>
        <route id="route_3"/>
      </camel:remove-routes>
  </actions>
</testcase>
```

Start/stop routes

Next operation we will discuss is the start and stop of existing Camel routes:

Java

```
public class CamelRouteActionIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void startThenStopCamelRoutes() {
        $(camel().camelContext(camelContext)
            .route()
            .start("route_1"));

        $(camel().camelContext(camelContext)
            .route()
            .stop("route_2", "route_3"));
    }
}
```

XML

```
<testcase name="CamelRouteIT">
  <actions>
      <camel:start-routes camel-context="camelContext">
        <route id="route_1"/>
      </camel:start-routes>

      <camel:stop-routes camel-context="camelContext">
        <route id="route_2"/>
        <route id="route_3"/>
      </camel:stop-routes>
  </actions>
</testcase>
```

Starting and stopping Camel routes at runtime is important when temporarily Citrus need to receive a message on a Camel endpoint URI. We can stop a route, use a Citrus camel endpoint instead for validation and start the route after the test is done. This way we can also simulate errors and failure scenarios in a Camel route interaction.

Verify route

You can verify that a Camel route exists in the context and optionally assert its current status (e.g. `Started`, `Stopped`). This is useful when you need to wait for or assert that a route has reached a certain state before continuing with the test.

Java

```
public class CamelRouteActionIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void verifyRouteStatus() {
        $(camel().camelContext(camelContext)
            .route()
            .verify("route_1")
            .status(ServiceStatus.Started));
    }
}
```

XML

```
<testcase name="CamelRouteIT">
  <actions>
      <camel:verify-route route="route_1" status="Started"/>
  </actions>
</testcase>
```

YAML

```
actions:
  - camel:
      verifyRoute:
        route: "route_1"
        status: "Started"
```

When no `status` is given, the action only verifies that the route exists in the Camel context. The action throws an exception when the route does not exist or when the actual status does not match the expected value.

17.7. Camel route statistics verification

The verify route stats action accesses Camel route management statistics using the `camel-management` dependency (`ManagedCamelContext`). You can verify individual route metrics like completed and failed exchange counts, or compare the full route statistics JSON returned by `dumpStatsAsJSon()`.



The `camel-management` dependency must be on the classpath to use this action. For JSON stats comparison, `citrus-validation-json` is also required.

17.7.1. Verifying individual metrics

Java

```
public class CamelRouteStatsIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void verifyRouteStats() {
        $(camel().camelContext(camelContext)
            .route()
            .verifyRouteStats("route_1")
            .completed(5)
            .failed(0));
    }
}
```

XML

```
<testcase name="CamelRouteStatsIT">
  <actions>
      <camel:verify-route-stats route="route_1" completed="5" failed="0"/>
  </actions>
</testcase>
```

YAML

```
actions:
  - camel:
      verifyRouteStats:
        route: "route_1"
        completed: 5
        failed: 0
```

The action verifies the number of completed and/or failed exchanges on the given route. Both `completed` and `failed` are optional - you can verify one or both.

17.7.2. Verifying route statistics JSON

You can also verify the full route statistics JSON using the Citrus JSON message validator. The expected JSON supports partial matching - you only need to specify the fields you want to verify.

Java

```
public class CamelRouteStatsJsonIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void verifyRouteStatsJson() {
        $(camel().camelContext(camelContext)
            .route()
            .verifyRouteStats("route_1")
            .stats("{\"exchangesCompleted\": 5, \"exchangesFailed\": 0}"));
    }
}
```

XML

```
<testcase name="CamelRouteStatsJsonIT">
  <actions>
      <camel:verify-route-stats route="route_1">
          <stats>{"exchangesCompleted": 5, "exchangesFailed": 0}</stats>
      </camel:verify-route-stats>
  </actions>
</testcase>
```

YAML

```
actions:
  - camel:
      verifyRouteStats:
        route: "route_1"
        stats: '{"exchangesCompleted": 5, "exchangesFailed": 0}'
```

You can combine individual metric checks with JSON stats comparison in a single action.

17.8. Camel route statistics reset

The reset route stats action resets Camel route statistics using the `camel-management` dependency (`ManagedCamelContext`). It performs a deep reset via `ManagedRouteMBean.reset(true)`, which clears all counters including processor-level statistics. This is useful when you need a clean baseline before verifying route statistics in tests that share a Camel context.



The `camel-management` dependency must be on the classpath to use this action.

Java

```
public class CamelResetRouteStatsIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void resetRouteStats() {
        $(camel().camelContext(camelContext)
            .route()
            .resetRouteStats("route_1"));
    }
}
```

XML

```
<testcase name="CamelResetRouteStatsIT">
  <actions>
      <camel:reset-route-stats route="route_1"/>
  </actions>
</testcase>
```

YAML

```
actions:
  - camel:
      resetRouteStats:
        route: "route_1"
```

You can combine the reset action with the verify action to ensure a clean baseline before making assertions:

Java

```
$(camel().camelContext(camelContext)
    .route()
    .resetRouteStats("route_1"));

// ... trigger some exchanges ...

$(camel().camelContext(camelContext)
    .route()
    .verifyRouteStats("route_1")
    .completed(1)
    .failed(0));
```

XML

```
<camel:reset-route-stats route="route_1"/>
<!-- ... trigger some exchanges ... -->
<camel:verify-route-stats route="route_1" completed="1" failed="0"/>
```

YAML

```
- camel:
    resetRouteStats:
      route: "route_1"
# ... trigger some exchanges ...
- camel:
    verifyRouteStats:
      route: "route_1"
      completed: 1
      failed: 0
```

17.9. Camel controlbus actions

The Camel controlbus component is a good way to access route statistics and route status information within a Camel context. Citrus provides controlbus test actions to easily access the controlbus operations at runtime.

Java

```
public class CamelControlBusIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void createCamelRoute() {
        $(camel().camelContext(camelContext)
            .controlbus()
            .route("route_1")
            .status()
            .result(ServiceStatus.Stopped));

        $(camel().camelContext(camelContext)
            .controlbus()
            .route("route_1")
            .start());

        $(camel().camelContext(camelContext)
            .controlbus()
            .route("route_1")
            .status()
            .result(ServiceStatus.Started));
    }
}
```

XML

```
<testcase name="CamelControlBusIT">
  <actions>
    <camel:control-bus camel-context="camelContext">
      <camel:route id="route_1" action="status"/>
      <camel:result>Stopped</camel:result>
    </camel:control-bus>

    <camel:control-bus>
      <camel:route id="route_1" action="start"/>
    </camel:control-bus>

    <camel:control-bus camel-context="camelContext">
      <camel:route id="route_1" action="status"/>
      <camel:result>Started</camel:result>
    </camel:control-bus>

    <camel:control-bus>
      <camel:language type="simple">${camelContext.stop()}</camel:language>
    </camel:control-bus>

    <camel:control-bus camel-context="camelContext">
      <camel:language
type="simple">${camelContext.getRouteController().getRouteStatus('route_3')}</camel:la
nguage>
      <camel:result>Started</camel:result>
    </camel:control-bus>
  </actions>
</testcase>
```

The example test case shows the controlbus access. As already mentioned you can explicitly reference a target Camel context with `camel-context="camelContext"`. In case no specific context is referenced Citrus will automatically lookup a target Camel context with the default context name "*citrusCamelContext*".

Camel provides two different ways to specify operations and parameters. The first option is the use of an **action** attribute. The Camel route id has to be specified as mandatory attribute. As a result the controlbus action will be executed on the target route during test runtime. This way we can also start and stop Camel routes in a Camel context.

In case a controlbus operation has a result such as the **status** action we can specify a control result that is compared. Citrus will raise validation exceptions when the results differ.

The second option for executing a controlbus action is the language expression. We can use Camel language expressions on the Camel context for accessing a controlbus operation. Also, here we can define an optional outcome as expected result.

Java

```
public class CamelControlBusIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void createCamelRoute() {
        $(camel().camelContext(camelContext)
            .controlbus()
            .language("simple", "${camelContext.getRouteStatus('my_route')}")
            .result(ServiceStatus.Stopped));

        $(camel().camelContext(camelContext)
            .controlbus()
            .language("simple", "${camelContext.stop()}"));
    }
}
```

XML

```
<testcase name="CamelControlBusIT">
  <actions>
    <camel:control-bus camel-context="camelContext">
      <camel:language
type="simple">${camelContext.getRouteStatus('my_route')}</camel:language>
      <camel:result>Started</camel:result>
    </camel:control-bus>

    <camel:control-bus>
      <camel:language type="simple">${camelContext.stop()}</camel:language>
    </camel:control-bus>
  </actions>
</testcase>
```

17.10. Camel CLI support

The Apache Camel project offers a great CLI tooling that allows you to run Camel integrations directly from the command line. Citrus supports two ways to launch the Camel CLI:

- **JBang** (default) — uses JBang to resolve and run the Camel CLI application.
- **Camel Launcher jar** — uses a standalone Camel Launcher jar file executed via `java -jar`.

17.10.1. Camel JBang setup

By default, Citrus uses JBang to launch the Camel CLI. You can install the Camel CLI tooling on your

local machine as a JBang application.

```
jbang app install camel@apache/camel
```

The `--help` option will show you all available commands that you can run with Camel CLI.

17.10.2. Camel Launcher setup

As an alternative to JBang you can use the Camel Launcher jar to run Camel CLI commands. The Camel Launcher is a standalone executable jar that does not require JBang to be installed.

To use the Camel Launcher jar, configure the following settings:

System properties

```
citrus.camel.cli.type=launcher
citrus.camel.cli.launcher.jar.path=/path/to/camel-launcher.jar
```

Environment variables

```
export CITRUS_CAMEL_CLI_TYPE=launcher
export CITRUS_CAMEL_CLI_LAUNCHER_JAR_PATH=/path/to/camel-launcher.jar
```

When configured, Citrus will execute Camel CLI commands as `java -jar <path-to-jar> <command> <args>` instead of using JBang.

17.10.3. Run Camel integrations

One of the available commands in Camel CLI is the `run` command that takes a Camel route as an input and runs the route locally without any prior project setup.

Citrus is able to use this capability in a test to run a Camel integration in a separate process.

Given the following Camel route written in YAML:

Sample Camel route.yaml

```
- from:
    uri: "timer:tick"
    parameters:
      period: "1000"
      includeMetadata: true
    steps:
      - setBody:
          simple: "{{greeting}} #${header.CamelTimerCounter}"
      - to: "log:info"
```

The Camel route uses a timer component to periodically log a message to the output. The message is

given as a property placeholder `greeting` and may be set in the `application.properties`.

application.properties

```
greeting=Hello from Apache Camel!
```

The Citrus test is able to run this route with Camel CLI in a separate process:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .run()
                .integration(Resources.fromClasspath("route.yaml",
CamelCliTest.class))
                .withSystemProperty("greeting", "Hello from Apache Camel!"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        run:
          integration:
            file: "classpath:route.yaml"
            systemProperties:
              properties:
                - name: greeting
                  value: Hello from Apache Camel!
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <run>
          <integration file="classpath:route.yaml">
            <system-properties>
              <property name="greeting" value="Hello from Apache Camel!"/>
            </system-properties>
          </integration>
        </run>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This starts the Camel integration with Camel CLI. The test waits for the integration to report running status. You can skip the waiting state with the `waitForRunningState=false` option.



The test action has an `autoRemove` option that is enabled by default. This means that the test will automatically stop and remove the Camel integration after the test.



You may choose to add the `--stub` option to the Camel CLI run command. This allows you to stub endpoints in the Camel integration route. The stubbed endpoint uses a local internal message queue instead of connecting to the real endpoint. This is very useful when an endpoint raises quite complex connection efforts. For instance, you may stub an SQL endpoint or a message broker connection endpoint such as Kafka or JMS when running the Camel integration. You may access the messages processed by the stubbed endpoint in the test with a `cmd receive` test action that is described later in this guide.

17.10.4. Stop Camel integrations

You may want to explicitly stop the Camel integration. You can do this with the following action:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .stop()
                .integration("route")
        );
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        stop:
          integration: "route"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <stop integration="route"/>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The Camel integration to stop is identified by its name. Usually this is the file name without the file extension. You can also specify a name when running the integration and reference this name in the stop action.

17.10.5. Verify Camel integrations

Each Camel integration reports a state (running or stopped) that Citrus is able to verify. In addition to that the running Camel integration produces a log output that Citrus is able to scan for expected messages.

You can do all of this in the `verify` action.

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .verify("route")
                .waitForLogMessage("Hello from Apache Camel! #10"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        verify:
          integration: "route"
          logMessage: "Hello from Apache Camel! #10"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <verify integration="route" log-message="Hello from Apache Camel! #10"/>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The action above verifies that the Camel integration is running. Also, the action waits for a log message `Hello from Apache Camel! #10`. The action periodically polls the produced log output and searches for the expected log message.

The action provides options such as `maxAttempts` and `pollingInterval` to stop the waiting period. In case the log message is not found after the maximum number of attempts the test action fails the test.

17.10.6. Send messages

The Camel CLI tooling allows you to send messages to running Camel integrations. You can use the `send` command to send messages to any running route.

Give the following Camel integration that is running in Camel CLI:

echo.yaml

```
- from:
    uri: "direct:echo"
    steps:
      - transform:
          simple: "${body.toUpperCase()}"
      - to: "log:info"
```

The Camel route listens for messages on the `direct:echo` endpoint. With Camel CLI you can send messages to this route using the following command:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSendMessage() {
        when(camel().cli().cmd()
                    .send()
                    .integration("echo")
                    .body("Hello Camel"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
    cli:
      cmd:
        send:
          integration: "echo"
          body:
            data: "Hello Camel"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <cmd>
          <send integration="echo">
            <body>
              <data>Hello Camel</data>
            </body>
          </send>
        </cmd>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The send action connects with the running Camel integration called `echo` and send the message body `Hello Camel` to the route. The send operation automatically uses the `direct:echo` endpoint because there is only one single route.

In case you have multiple routes running with multiple endpoints you can specify the endpoint with the `endpoint` attribute.

Also, you may set some additional message headers.

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSendMessage() {
        when(camel().cli().cmd()
                    .send()
                    .integration("echo")
                    .endpoint("direct:echo")
                    .header("user", "christoph")
                    .body("Hello Camel"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
    cli:
      cmd:
        send:
          integration: "echo"
          endpoint: "direct:echo"
          headers:
            user: christoph
          body:
            data: "Hello Camel"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <cmd>
          <send integration="echo" endpoint="direct:echo">
            <headers>
              <header name="user" value="christoph"/>
            </headers>
            <body>
              <data>Hello Camel</data>
            </body>
          </send>
        </cmd>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

You can specify a send timeout with the `timeout` attribute (by default this is set to 20 seconds).

The `reply=true/false` option makes the send operation wait for a response message from the Camel integration. The response data is logged to the output.

You can also use the Camel CLI send command without a running Camel integration. This can be used to send messages to any support Camel endpoint. Just use the endpoint uri setting on the send operation like this:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSendMessage() {
        when(camel().cli().cmd()
                    .send()
                    .uri("kafka:echo?server=localhost:9092")
                    .header("user", "christoph")
                    .body("Hello Camel"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
    cli:
      cmd:
        send:
          uri: "kafka:echo?server=localhost:9092"
          headers:
            user: christoph
          body:
            data: "Hello Camel"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <cmd>
          <send uri="kafka:echo?server=localhost:9092">
            <headers>
              <header name="user" value="christoph"/>
            </headers>
            <body>
              <data>Hello Camel</data>
            </body>
          </send>
        </cmd>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



You can also use the Kamelets sources to send messages with Camel CLI. The Kamelets provide easy to use properties for comfortable connection options.

Please see the Camel CLI send command help for more configuration options.

Camel CLI cmd send options

```
camel cmd send --help
```

17.10.7. Receive messages

The Camel CLI tooling allows you to access stubbed messages in a running Camel integration. You can use the `receive` command to list messages that an endpoint in Camel has been processing.

Give the following Camel integration that is running in Camel CLI:

echo.yaml

```
- from:
    uri: "direct:echo"
    steps:
      - transform:
          simple: "${body.toUpperCase()}"
      - to: "kafka:inbox?brokers=localhost:9092"
```

The Camel route listens for messages on the `direct:echo` endpoint and connects with a Kafka message broker. With Camel CLI you can start the route with a stubbed Kafka endpoint like this:

Run Camel integration with --stub option

```
camel run my-route.yaml --stub=kafka
```

This makes Camel use an internal message queue instead of connecting with the Kafka message broker. The messages received by the stubbed Kafka component in Camel can be accessed with the `receive` command:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSendMessage() {
        then(camel().cli().cmd()
                    .receive()
                    .integration("my-route")
                    .tail(1)
                    .endpoint("kafka:inbox"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        cmd:
          receive:
            integration: "my-route"
            tail: 1
            endpoint: "kafka:inbox"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <cmd>
          <receive integration="my-route" tail="1" endpoint="kafka:inbox"/>
        </cmd>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The `receive` action connects with the running Camel integration called `my-route` and receives messages on the stubbed `kafka:inbox` endpoint. The `tail` setting defines how many messages that have been received before will be included. This means a setting of `tail=1` includes the last message that was processed by the stubbed endpoint. By default, the operation will wait only for new messages to arrive (tail=0).

The test action monitors the `receive` command output and waits for messages to arrive. You can use the `grep` option to wait for very specific messages that match the given expression. Also, you can specify the `since` option that allows you to include messages that have arrived in a given time span (e.g. 5s, 2m, 1h)

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldSendMessage() {
        then(camel().cli().cmd()
                    .receive()
                    .integration("my-route")
                    .tail(1)
                    .since("1h")
                    .grep("HELLO CAMEL")
                    .endpoint("kafka:inbox"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        cmd:
          receive:
            integration: "my-route"
            tail: 1
            since: 1h
            grep: HELLO CAMEL
            endpoint: "kafka:inbox"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <cmd>
          <receive integration="my-route"
                   tail="1"
                   since="1h"
                   grep="HELLO CAMEL"
                   endpoint="kafka:inbox"/>
        </cmd>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The `grep`, `tail` and `since` options help you to verify that a specific message has been processed on the stubbed endpoint. The test action polls the command output for the message and throws a timeout error in case the message does not arrive in time. You can use the `maxAttempts` and `delayBetweeAttempts` settings to specify the waiting and polling configuration.

Please see the Camel CLI receive command help for more configuration options.

Camel CLI cmd receive options

```
camel cmd receive --help
```

17.10.8. Manage Camel plugins

Camel CLI supports plugins that extend the CLI with additional commands and capabilities. Citrus provides test actions to add and delete plugins as part of a test.

Add a plugin

Java

```
when(camel().cli()
    .plugin()
    .add()
    .pluginName("my-plugin")
    .withArg("--version", "1.0.0"));
```

YAML

```
- camel:
    cli:
      plugin:
        add:
          name: "my-plugin"
          args:
            - "--version"
            - "1.0.0"
```

XML

```
<camel>
  <cli>
    <plugin>
      <add name="my-plugin" args="--version 1.0.0"/>
    </plugin>
  </cli>
</camel>
```

The action checks if the plugin is already installed by name. When the plugin is already installed and no `--version` or `--gav` arguments are provided, the action skips the installation. When `--version` or `--gav` arguments are present, the action removes the existing plugin and reinstalls it with the new arguments.



The test action has an `autoRemove` option that is disabled by default. When enabled, the test will automatically delete the plugin after the test using the `context.doFinally()` mechanism. You can enable it globally via the `citrus.camel.cli.auto.remove.plugins` system property or `CITRUS_CAMEL_CLI_AUTO_REMOVE_PLUGINS` environment variable.

Delete a plugin

You can explicitly delete a plugin in your test. This is useful in `finally` blocks to clean up plugins that were installed during the test.

Java

```
when(camel().cli()
    .plugin()
    .delete()
    .pluginName("my-plugin"));
```

YAML

```
- camel:
    cli:
      plugin:
        delete:
          name: "my-plugin"
```

XML

```
<camel>
  <cli>
    <plugin>
      <delete name="my-plugin"/>
    </plugin>
  </cli>
</camel>
```

17.10.9. Camel CLI settings

The Camel CLI support uses the following environment settings. Settings use the `citrus.camel.cli.` prefix. The older `citrus.camel.jbang.` prefix is still supported as a fallback.

Table 4. System properties

System property	Description
citrus.camel.cli.type	The launcher type to use: `jbang` (default) or `launcher`. When set to `launcher`, Citrus uses the Camel Launcher jar instead of JBang.
citrus.camel.cli.launcher.jar.path	Path to the Camel Launcher jar file. Required when `citrus.camel.cli.type` is set to `launcher`. Falls back to the `camel.launcher.jar` system property.
citrus.camel.cli.app	Camel JBang app name. (default="camel@apache/camel")
citrus.camel.cli.version	Camel version to use. (default="latest")
citrus.camel.cli.work.dir	Local working directory for the Camel CLI.
citrus.camel.cli.kamelets.version	Kamelets version used by the Camel CLI runtime.

System property	Description
citrus.camel.cli.kamelets.local.dir	Local Kamelets directory.
citrus.camel.cli.dump.integration.output	When set to true, Camel CLI process output for integrations will be redirected to a file in the current working directory. (default="false")
citrus.camel.cli.verbose	When set to true, Camel CLI will print detailed messages. (default="false")
citrus.camel.cli.auto.remove.resources	When set to true, Camel CLI resources created during the test are automatically removed after the test. (default="true")
citrus.camel.cli.auto.remove.plugins	When set to true, Camel CLI plugins added during the test are automatically removed after the test. (default="false")
citrus.camel.cli.wait.for.running.state	When set to true, Camel CLI will automatically wait for each integration created to be in running state. (default="true")
citrus.camel.cli.trust.url	Comma-separated list of trust URLs for JBang. (default="https://github.com/apache/camel/")

Table 5. Environment variables

Environment variable	Description
CITRUS_CAMEL_CLI_TYPE	The launcher type to use: `jbang` (default) or `launcher`.
CITRUS_CAMEL_CLI_LAUNCHER_JAR_PATH	Path to the Camel Launcher jar file.
CITRUS_CAMEL_CLI_APP	Camel JBang app name.
CITRUS_CAMEL_CLI_VERSION	Camel version to use.
CITRUS_CAMEL_CLI_WORK_DIR	Local working directory for the Camel CLI.
CITRUS_CAMEL_CLI_KAMELETS_VERSION	Kamelets version used by the Camel CLI runtime.
CITRUS_CAMEL_CLI_KAMELETS_LOCAL_DIR	Local Kamelets directory.
CITRUS_CAMEL_CLI_DUMP_INTEGRATION_OUTPUT	When set to true, integration output is redirected to a file.
CITRUS_CAMEL_CLI_VERBOSE	When set to true, Camel CLI will print detailed messages.
CITRUS_CAMEL_CLI_AUTO_REMOVE_RESOURCES	When set to true, resources are automatically removed after the test.
CITRUS_CAMEL_CLI_AUTO_REMOVE_PLUGINS	When set to true, plugins are automatically removed after the test.
CITRUS_CAMEL_CLI_WAIT_FOR_RUNNING_STATE	When set to true, waits for integrations to reach running state.

Environment variable	Description
CITRUS_CAMEL_CLI_TRUST_URL	Comma-separated list of trust URLs for JBang.

17.11. Camel CLI Kubernetes support

The Camel CLI integrations are run locally by default as a separate JBang process. In addition to that you may also run the integration in a Kubernetes namespace.

This uses the Camel CLI Kubernetes plugin that is able to build a container image and deploy it to a connected Kubernetes namespace.

17.11.1. Deploy the Camel integration

The Camel CLI Kubernetes plugin is able to build a container image from an integration and deploy the integration with a generated Kubernetes manifest.



The following test actions call the Camel Kubernetes plugin. A prerequisite to this is that you are connected to a Kubernetes cluster and a namespace. It is essential to have a Kubernetes command line tooling installed and ready for usage on your local machine.

Citrus provides a test action to call the Camel CLI Kubernetes plugin:

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .kubernetes()
                .run()
                .integration(Resources.fromClasspath("route.yaml",
CamelCliTest.class))
                .runtime("quarkus")
                .clusterType("kind")
                .imageBuilder("docker")
                .imageRegistry("localhost:5000")
                .withProperty("greeting=Hello from Apache Camel!")
        );
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
       kubernetes:
          run:
            integration:
              file: "classpath:route.yaml"
            runtime: "quarkus"
            clusterType: "kind"
            imageBuilder: "docker"
            imageRegistry: "localhost:5000"
            properties:
              - name: greeting
                value: Hello from Apache Camel!
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <kubernetes>
          <run runtime="quarkus"
               cluster-type="kind"
               image-builder="docker"
               image-registry="localhost:5000">
            <integration file="classpath:route.yaml"/>
            <properties>
              <property name="greeting" value="Hello from Apache Camel!"/>
            </properties>
          </run>
        </kubernetes>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The Kubernetes `run` action receives detailed configuration on how to build the container image and where to push the resulting image. You may choose the target cluster type and use an external or internal image registry (e.g. quay.io or localhost:5000).

The Camel CLI Kubernetes plugin will perform several steps to create the deployment. First of all the plugin creates a project export for the given Camel integration and builds the container image.

Also, the plugin generates a Kubernetes manifest that holds all required resources to deploy the integration to a Kubernetes cluster. The Kubernetes deployment is automatically performed as a last step in this action.

The result will be a new Kubernetes deployment in the current namespace that uses the freshly built container image.



Please make sure to connect to a Kubernetes cluster and choose a proper namespace before running the test action. The Kubernetes client will automatically use the current cluster and namespace.



The action has an `autoRemove` setting that is enabled by default. This means that the Kubernetes deployment is automatically stopped and removed after the test.

17.11.2. Delete integrations from Kubernetes

You can explicitly remove the Camel integration from Kubernetes.

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .kubernetes()
                .delete()
                .integration("route")
        );
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        kubernetes:
          delete:
            integration: "route"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <kubernetes>
          <delete>
            <integration name="route"/>
          </delete>
        </kubernetes>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This deletes the Kubernetes deployment and all resources listed in the Camel integration Kubernetes manifest.

17.11.3. Verify integration on Kubernetes

Citrus is able to verify the state of a running Camel integration in Kubernetes. This is the status of the Kubernetes deployment and its pods and containers. Also, Citrus is able to scan the log output of a Camel integration that is running in Kubernetes.

Java

```
public class CamelCliTest extends TestNGCitrusSupport implements TestActionSupport {

    @Test
    @CitrusTest
    public void shouldRunCamelCli() {
        when(camel().cli()
                .kubernetes()
                .verify("route")
                .waitForLogMessage("Hello from Apache Camel! #10"));
    }
}
```

YAML

```
name: "CamelCliTest"
actions:
  - camel:
      cli:
        kubernetes:
          verify:
            integration: "route"
            logMessage: "Hello from Apache Camel! #10"
```

XML

```
<test name="CamelCliTest" xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <cli>
        <kubernetes>
          <verify integration="route" log-message="Hello from Apache Camel! #10"/>
        </kubernetes>
      </cli>
    </camel>
  </actions>
</test>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The action makes sure to find the Kubernetes deployment and the running Pod. The action waits for the expected log message and fails when the message is not available in the log output after the maximum number of attempts.

17.12. Camel infrastructure services

The Apache Camel project maintains a huge set of test infrastructure services that you can also start and stop during a Citrus test. See this documentation about the Camel test infra services.

The Camel infra services usually start Docker/Podman containers and/or use Testcontainers.

See this list of supported infrastructure services in Apache Camel:

Service	Service	Service
arangodb	artemis (amqp, mqtt, persistent)	aws (dynamodb, event-bridge, kinesis, s3, sns, sqs, sts, ...)
azure	cassandra	chat-script
couchbase	couchdb	elasticsearch
fhir	ftp	ftps
google (pub-sub)	hashicorp (vault)	hazelcast
hive-mq (sparkplug)	kafka (confluent, redpanda, strimzi)	microprofile (lra)
milvus	minio	mongodb
mosquitto	nats	ollama
openldap	postgres	pulsar
qdrant	rabbitmq	redis
rocketmq	sftp	smb
solr	torch-serve	xmpp
zookeeper		

You can start/stop such an infra service via test actions in Citrus.

17.12.1. Run Camel infra services

Java

```
public class CamelInfraIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTest
    public void createInfraService() {
        $(camel()
            .infra()
            .run()
            .service("postgres"));
    }
}
```

YAML

```
name: "CamelInfraTest"
actions:
  - camel:
      infra:
        run:
          service: postgres
```

XML

```
<test name="CamelInfraIT"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <infra>
        <run service="postgres"/>
      </infra>
    </camel>
  </actions>
</testcase>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

Once the infrastructure service is started Citrus exposes connection settings as test variables. You can use the exposed connection settings to create proper clients that connect to the services.

The exposed connection settings follow a naming pattern that looks like this:

- `CITRUS_CAMEL_INFRA_SERVICE_<SERVICE_NAME>_<PROPERTY_NAME>`
- `CITRUS_CAMEL_INFRA_SERVICE_<SERVICE_NAME>_<IMPLEMENTATION>_<PROPERTY_NAME>`

For instance a infrastructure service called `foo` that exposes the settings `url` and `port` expose the following test variables.

Variable
CITRUS_CAMEL_INFRA_SERVICE_FOO_URL
CITRUS_CAMEL_INFRA_SERVICE_FOO_PORT

You can now use these test variables when connecting to the service. Please see the individual Apache Camel infrastructure services documentation to see which properties get exposed by the service.

Some infrastructure services provide multiple implementations (e.g. Kafka provides both Redpanda and Strimzi).

You may need to choose the implementation as follows:

Java

```
public class CamelInfraIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTest
    public void createInfraService() {
        $(camel()
            .infra()
            .run()
            .service("kafka")
            .implementation("strimzi"));
    }
}
```

YAML

```
name: "CamelInfraTest"
actions:
  - camel:
      infra:
        run:
          service: kafka
          implementation: strimzi
```

XML

```
<test name="CamelInfraIT"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <infra>
        <run service="kafka" implementation="strimzi"/>
      </infra>
    </camel>
  </actions>
</testcase>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

By default, the infrastructure services get automatically removed after the test. You can disable the auto removal when running the service (`auto-remove` setting).

17.12.2. Fixed port configuration

By default, Camel infra services use random ports for better test isolation. In some cases you may want to use a fixed port instead (e.g. when an external tool expects a service on a well-known port, or when debugging locally).

You can configure a fixed port directly on the run action:

Java

```
public class CamelInfraIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTest
    public void createInfraService() {
        $(camel()
            .infra()
            .run()
            .service("postgres")
            .port(5432));
    }
}
```

YAML

```
name: "CamelInfraTest"
actions:
  - camel:
      infra:
        run:
          service: postgres
          port: 5432
```

Setting a specific port automatically enables fixed port mode. You can also enable fixed port mode without specifying a concrete port number — this tells the service to use its own default port rather than a random one:

Java

```
$(camel()
    .infra()
    .run()
    .service("postgres")
    .fixedPort(true));
```

YAML

```
name: "CamelInfraTest"
actions:
  - camel:
      infra:
        run:
          service: postgres
          fixedPort: true
```

The fixed port settings can also be configured globally via system properties or environment variables (see Camel infra settings).

17.12.3. Stop Camel infra services

To explicitly stop a running infrastructure service use the following test action.

Java

```
public class CamelInfraIT extends TestNGCitrusSpringSupport {

    @Test
    @CitrusTest
    public void createInfraService() {
        $(camel()
            .infra()
            .stop()
            .service("postgres"));
    }
}
```

YAML

```
name: "CamelInfraTest"
actions:
  - camel:
      infra:
        stop:
          service: postgres
```

XML

```
<test name="CamelInfraIT"
      xmlns="http://citrusframework.org/schema/xml/testcase"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://citrusframework.org/schema/xml/testcase
http://citrusframework.org/schema/xml/testcase/citrus-testcase.xsd">
  <actions>
    <camel>
      <infra>
        <stop service="postgres"/>
      </infra>
    </camel>
  </actions>
</testcase>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

17.12.4. Camel infra settings

The Camel infrastructure services support the following environment settings.

Table 6. System properties

System property	Description
citrus.camel.infra.auto.remove.services	When set to true the test will automatically stop and remove Camel infra services that have been started. (default="true")
citrus.camel.infra.dump.service.output	When set to true infra service output will be redirected to a file in the current working directory. (default="false")
citrus.camel.infra.port	When set infra services use this port as a fixed port. (default="-1", meaning random port)
citrus.camel.infra.fixedPort	When set to true infra services use fixed ports. Otherwise, services use random ports for better isolation. (default="false")
citrus.camel.infra.<service>.port	Sets a fixed port for a specific service. The service name is the full service name (e.g. `citrus.camel.infra.postgres.port=5432`).

Table 7. Environment variables

Environment variable	Description
CITRUS_CAMEL_INFRA_AUTO_REMOVE_SERVICES	When set to true the test will automatically stop and remove Camel infra services that have been started. (default="true")
CITRUS_CAMEL_INFRA_DUMP_SERVICE_OUTPUT	When set to true infra service output will be redirected to a file in the current working directory. (default="false")
CITRUS_CAMEL_INFRA_PORT	When set infra services use this port as a fixed port. (default="-1", meaning random port)
CITRUS_CAMEL_INFRA_FIXED_PORT	When set to true infra services use fixed ports. Otherwise, services use random ports for better isolation. (default="false")
CITRUS_CAMEL_INFRA_<SERVICE>_PORT	Sets a fixed port for a specific service. The service name uses upper case and non-alphanumeric characters are replaced with underscores (e.g. `CITRUS_CAMEL_INFRA_POSTGRES_PORT=5432`).

17.13. Camel endpoint DSL

Since Camel 3 the endpoint DSL provides a convenient way to construct an endpoint uri. In Citrus

you can use the Camel endpoint DSL to send/receive messages in a test.

Java

```
$(send(camel().endpoint(seda("test")::getRawUri))
        .message()
        .body("Citrus rocks!"));
```

The fluent endpoint DSL in Camel allows to build the endpoint uri. The `camel().endpoint(seda("test")::getRawUri)` builds the endpoint uri `seda:test`. The endpoint DSL provides all settings and properties that you can set for a Camel endpoint component.

17.14. Camel processor support

Camel implements the concept of processors as enterprise integration pattern. A processor is able to add custom logic to a Camel route. Each processor is able to access the Camel exchange that is being processed in the current route. The processor is able to change the message content (body, headers) as well as the exchange information.

The send/receive operations in Citrus also implement the processor concept. With the Citrus Camel support you can use the very same Camel processor also in a Citrus test action.

Message processor on send

```
public class CamelMessageProcessorIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void shouldProcessMessages() {
        $(send(camel().endpoint(seda("test")::getRawUri))
                .message()
                .body("Citrus rocks!")
                .process(processor()
                    .camel()
                    .camelContext(camelContext)
                    .process(exchange -> exchange
                            .getMessage()

.setBody(exchange.getMessage().getBody(String.class).toUpperCase())))
        );
    }
}
```

The example above uses a Camel processor to change the exchange and the message content before the message is sent to the endpoint. This way you can apply custom changes to the message as part of the test action.

Message processor on receive

```
public class CamelMessageProcessorIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void shouldProcessMessages() {
        $(send(camel().endpoint(seda("test")::getRawUri))
                .message()
                .body("Citrus rocks!"));

        $(receive(camel().endpoint(seda("test")::getRawUri))
                .process(processor()
                        .camel(camelContext)
                        .process(exchange -> exchange
                                .getMessage()

.setBody(exchange.getMessage().getBody(String.class).toUpperCase())))
                .message()
                .type(MessageType.PLAINTEXT)
                .body("CITRUS ROCKS!"));
    }
}
```

The Camel processors are very powerful. In particular, you can apply transformations of multiple kind.

Transform processor

```
public class CamelTransformIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void shouldTransformMessageReceived() {
        $(send(camel().endpoint(seda("hello")::getRawUri))
                .message()
                .body("{\"message\": \"Citrus rocks!\"}")
        );

        $(receive(camel().endpoint(seda("hello")::getRawUri))
                .transform(
                    camel()
                        .camelContext(camelContext)
                        .transform()
                        .jsonpath("$.message"))
                .message()
                .type(MessageType.PLAINTEXT)
                .body("Citrus rocks!"));
    }
}
```

The transform pattern is able to change the message content before a message is received/sent in Citrus. The example above applies a JsonPath expression as part of the message processing. The JsonPath expression evaluates `$.message` on the Json payload and saves the result as new message body content. The following message validation expects the plaintext value `Citrus rocks!`.

The message processor is also able to apply a complete route logic as part of the test action.

Route processor

```
public class CamelRouteProcessorIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void shouldProcessRoute() {
        CamelRouteProcessor.Builder beforeReceive = camel(camelContext).route(route ->
                route.choice()
                    .when(jsonpath("$.greeting[?(@.language == 'EN')]"))
                        .setBody(constant("Hello!"))
                    .when(jsonpath("$.greeting[?(@.language == 'DE')]"))
                        .setBody(constant("Hallo!"))
                    .otherwise()
                        .setBody(constant("Hi!")));

        $(send(camel().endpoint(seda("greetings")::getRawUri))
                .message()
                .body("{" +
                        "\"greeting\": {" +
                            "\"language\": \"EN\"" +
                        "}" +
                      "}")
        );

        $(receive("camel:" + camel().endpoints().seda("greetings").getUri())
                .process(beforeReceive)
                .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello!"));
    }
}
```

With the complete route logic you have the full power of Camel ready to be used in your send/receive test action. This enables many capabilities as Camel implements the enterprise integration patterns such as split, choice, enrich and many more.

17.15. Camel data format support

Camel uses the concept of data format to transform message content in form of marshal/unmarshal operations. You can use the data formats supported in Camel in Citrus, too.

Data format marshal/unmarshal

```
public class CamelDataFormatIT extends TestNGCitrusSpringSupport {

    @Autowired
    private CamelContext camelContext;

    @Test
    @CitrusTest
    public void shouldApplyDataFormat() {
        when(send(camel().endpoint(seda("data")::getRawUri))
                .message()
                .body("Citrus rocks!")
                .transform(camel(camelContext)
                        .marshal()
                        .base64())
        );

        then(receive("camel:" + camel().endpoints().seda("data").getUri())
                .transform(camel(camelContext)
                        .unmarshal()
                        .base64())
                .transform(camel(camelContext)
                        .convertBodyTo(String.class))
                .message()
                .type(MessageType.PLAINTEXT)
                .body("Citrus rocks!"));
    }
}
```

The example above uses the `base64` data format provided in Camel to marshal/unmarshal the message content to/from a base64 encoded String. Camel provides support for many data formats as you can see in the documentation on data formats.

Chapter 18. Message channel support

Message channels represent the in memory messaging solution in Citrus. Producer and consumer components are linked via channels exchanging messages in memory. The transport comes from the Spring Integration project.

This opens up a lot of great possibilities to interact with the Spring Integration transport adapters for FTP, TCP/IP and so on. In addition to that the message channel support provides us a good way to exchange messages in memory.



The message channel configuration components use the "citrus-si" configuration namespace and schema definition. Include this namespace into your Spring configuration in order to use the Citrus configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as follows.

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus-si="http://www.citrusframework.org/schema/spring-
integration/config"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/spring-integration/config
            http://www.citrusframework.org/schema/spring-integration/config/citrus-
spring-integration-config.xsd">

    [...]

</beans>
```

Right now you are able to use customized Citrus XML elements in order to define the Spring Integration endpoint components.

18.1. Channel endpoint

Citrus offers a channel endpoint component that is able to create producers and consumers. Producer and consumer send and receive messages both to and from a channel endpoint. By default, the endpoint is asynchronous when configured in the Citrus context. With this component you are able to access message channels directly:

Java

```
@Bean
public ChannelEndpoint helloEndpoint() {
    return new ChannelEndpointBuilder()
        .channel("helloChannel")
        .build();
}

@Bean
public MessageSelectingQueueChannel helloChannel() {
    return new MessageSelectingQueueChannel();
}
```

XML

```
<citrus-si:channel-endpoint id="helloEndpoint" channel="helloChannel"/>

<citrus-si:channel id="helloChannel"/>
```

The Citrus channel endpoint references a Spring Integration channel directly. Inside your test case you can reference the Citrus endpoint as usual to send and receive messages.

The Citrus channel endpoint also supports a customized message channel template that will actually send the messages. The customized template might give you access to special configuration possibilities.

Java

```
@Bean
public ChannelEndpoint helloEndpoint() {
    return new ChannelEndpointBuilder()
        .channel("helloChannel")
        .messagingTemplate(messagingTemplate())
        .build();
}
```

XML

```
<citrus-si:channel-endpoint id="helloEndpoint"
                            channel="helloChannel"
                            message-channel-template="myMessageChannelTemplate"/>
```

The message sender is now ready to publish messages on the defined channel. The communication is supposed to be asynchronous, so the producer is not able to process any reply message. We will deal with synchronous communication and reply messages later in this chapter. You can reference the id of the endpoint in a send and receive test action.

Java

```
when(send("helloEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(receive("helloEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<send endpoint="helloEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</send>

<receive endpoint="helloEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</receive>
```

You can send and receive messages from the same Spring Integration message channel endpoint. As usual the receiver connects to the message destination and waits for messages to arrive. The user can set a receive timeout which is set to 5000 milliseconds by default. In case no message was received in this time frame the receiver raises timeout errors and the test fails.

18.2. Synchronous channel endpoints

The synchronous channel producer publishes messages and waits synchronously for the response to arrive on a reply channel destination. The reply channel name is set in the message headers. The

counterpart in this communication must send its reply to that channel. The basic configuration for a synchronous channel endpoint component looks like follows:

Java

```
@Bean
public ChannelSyncEndpoint helloEndpoint() {
    return new ChannelSyncEndpointBuilder()
        .channel("helloChannel")
        .pollingInterval(1000)
        .build();
}
```

XML

```
<citrus-si:channel-sync-endpoint id="helloSyncEndpoint"
                            channel="helloChannel"
                            polling-interval="1000"/>
```

Synchronous message channel endpoints usually do poll for synchronous reply messages for processing the reply messages. The poll interval is an optional setting in order to manage the amount of reply message handshake attempts. When the endpoint was able to receive the reply message synchronously the test case can verify the reply.

In case all polling attempts have failed the action raises a timeout error, and the test will fail.



By default, the channel endpoint uses temporary reply channel destinations. The temporary reply channels are only used once for a single communication handshake. The temporary reply channel is deleted automatically.

When sending a message to this endpoint in the first place the producer will wait synchronously for the response message to arrive on the reply channel. You can receive the reply message in your test case using the same endpoint component. So we have two actions on the same endpoint, first send then receive.

Java

```
when(send("helloSyncEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(receive("helloSyncEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<send endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</send>

<receive endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</receive>
```

This is how you handle synchronous communication as a sender. You publish messages to a channel and wait for reply messages on a temporary reply channel. The next section deals with the same synchronous communication, but now Citrus will receive a request and send a synchronous reply message to a temporary reply channel.

As usual the reply channel name is stored in the message headers. Citrus handles this synchronous communication with the same synchronous channel endpoint component. The handling of temporary reply destinations is done automatically behind the scenes.

So we have again two actions in our test case, but this time first receive then send.

Java

```
when(receive("helloSyncEndpoint")
        .message()
        .body("<v1:HelloRequest
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello World!</v1:Text>" +
            "</v1:HelloRequest>"));

then(send("helloSyncEndpoint")
        .message()
        .body("<v1:HelloResponse
xmlns:v1=\"http://citrusframework.org/schemas/HelloService.xsd\">" +
                "<v1:Text>Hello Citrus!</v1:Text>" +
            "</v1:HelloResponse>"));
```

XML

```
<receive endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloRequest
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello World!</v1:Text>
            </v1:HelloRequest>
        </payload>
    </message>
</receive>

<send endpoint="helloSyncEndpoint">
    <message>
        <payload>
            <v1:HelloResponse
xmlns:v1="http://citrusframework.org/schemas/HelloService.xsd">
                <v1:Text>Hello Citrus!</v1:Text>
            </v1:HelloResponse>
        </payload>
    </message>
</send>
```

18.3. Message selectors

A channel can hold multiple messages at the same time. Usually you receive messages using first-in-first-out pattern. Message selectors enable you to select messages form that channel so you can pick messages form a channel based on a selector evaluation.

Citrus introduces a special queue message channel implementation that support message selectors.

Java

```
@Bean
public MessageSelectingQueueChannel helloChannel() {
    return new MessageSelectingQueueChannel();
}
```

XML

```
<citrus-si:channel id="orderChannel" capacity="5"/>
```

The Citrus message channel implementation extends the queue channel implementation from Spring Integration. So we can add a capacity attribute for this channel. A receive test action makes use of message selectors on header values as described in message-selector.

In addition to that we have implemented other message filter possibilities on message channels that we discuss in the next sections.

18.4. Payload matching selector

You can select messages based on the payload content. Either you define the expected payload as an exact match in the selector or you make use of Citrus validation matchers which is more adequate in most scenarios.

Assume there are two different plain text messages living on a message channel waiting to be picked up by a consumer.

```
Hello, welcome!
```

```
GoodBye, see you next time!
```

The tester would like to pick up the message starting with **GoodBye** in our test case. The other messages should be left on the channel as we are not interested in it right now. We can define a payload matching selector in the receive action like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("payload", "@startsWith(GoodBye)@"))
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<receive endpoint="orderChannelEndpoint">
    <selector>
        <element name="payload" value="@startsWith(GoodBye)@"/>
    </selector>
    <message>
        <payload>GoodBye, see you next time!</payload>
    </message>
</receive>
```

The Citrus receiver picks up the **GoodBye** from the channel selected via the payload matching expression defined in the selector element. Of course, you can also combine message header selectors and payload matching selectors as shown in this example below where a message header **sequenceId** is added to the selection logic.

Java

```
Map<String, String> selectorMap = new HashMap<>();
selectorMap.put("payload", "@startsWith(GoodBye)@");
selectorMap.put("sequenceId", "1234");

when(receive("orderChannelEndpoint")
        .selector(selector)
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<selector>
    <element name="payload" value="@startsWith(GoodBye)@"/>
    <element name="sequenceId" value="1234"/>
</selector>
```

18.5. Root QName selector

As a special payload matching selector you can use the XML root QName of your message as selection criteria when dealing with XML message content. Let's see how this works in a small example:

We have two different XML messages on a message channel waiting to be picked up by a consumer.

```
<HelloMessage xmlns="http://citrusframework.org/schema">Hello Citrus</HelloMessage>
```

```
<GoodbyeMessage xmlns="http://citrusframework.org/schema">Goodbye
Citrus</GoodbyeMessage>
```

We would like to pick up the **GoodbyeMessage** in our test case. The **HelloMessage** should be left on the message channel as we are not interested in it right now. We can define a root qname message selector in the receive action like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("root-qname", "GoodbyeMessage"))
        .message()
        .body("<GoodbyeMessage xmlns=\"http://citrusframework.org/schema\">Goodbye
Citrus</GoodbyeMessage>"));
```

XML

```
<receive endpoint="orderChannelEndpoint">
    <selector>
        <element name="root-qname" value="GoodbyeMessage"/>
    </selector>
    <message>
        <payload>
            <GoodbyeMessage xmlns="http://citrusframework.org/schema">Goodbye
Citrus</GoodbyeMessage>
        </payload>
    </message>
</receive>
```

The Citrus receiver picks up the **GoodbyeMessage** from the channel selected via the root qname of the XML message payload. Of course, you can also combine message header selectors and root qname selectors as shown in this example below where a message header **sequenceId** is added to the selection logic.

Java

```
Map<String, String> selectorMap = new HashMap<>();
selectorMap.put("root-qname", "GoodbyeMessage");
selectorMap.put("sequenceId", "1234");

when(receive("orderChannelEndpoint")
        .selector(selector)
        .message()
        .body("GoodBye, see you next time!"));
```

XML

```
<selector>
    <element name="root-qname" value="GoodbyeMessage"/>
    <element name="sequenceId" value="1234"/>
</selector>
```

As we deal with XML qname values, we can also use namespaces in our selector root qname selection.

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("root-qname",
"{http://citrusframework.org/schema}GoodbyeMessage"))
        .message()
        .body("<GoodbyeMessage xmlns=\"http://citrusframework.org/schema\">Goodbye
Citrus</GoodbyeMessage>"));
```

XML

```
<selector>
    <element name="root-qname"
value="{http://citrusframework.org/schema}GoodbyeMessage"/>
</selector>
```

18.6. Xpath selector

It is also possible to evaluate some XPath expression on the message payload in order to select a message from a message channel. The XPath expression outcome must match an expected value and only then the message is consumed from the channel.

The syntax for the XPath expression is to be defined as the element name like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("xpath://Order/status", "pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element name="xpath://Order/status" value="pending"/>
</selector>
```

The message selector looks for order messages with **status="pending"** in the message payload. This means that following messages would get accepted/declined by the message selector.

```
<Order><status>pending</status></Order> <!-- ACCEPTED -->
<Order><status>finished</status></Order> <!-- NOT ACCEPTED -->
```

Of course, you can also use XML namespaces in your XPath expressions when selecting messages

from channels.

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("xpath://ns1:Order/ns1:status", "pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element name="xpath://ns1:Order/ns1:status" value="pending"/>
</selector>
```

Namespace prefixes must match the incoming message - otherwise the XPath expression will not work as expected. In our example the message should look like this:

```
<ns1:Order
xmlns:ns1="http://citrus.org/schema"><ns1:status>pending</ns1:status></ns1:Order>
```

Knowing the correct XML namespace prefix is not always easy. If you are not sure which namespace prefix to choose Citrus ships with a dynamic namespace replacement for XPath expressions. The XPath expression looks like this and is most flexible:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap(

"xpath://{http://citrus.org/schema}:Order/{http://citrus.org/schema}:status",
"pending"))
        .message()
        .body("<Order><status>pending</status></Order>"));
```

XML

```
<selector>
    <element
name="xpath://{http://citrus.org/schema}:Order/{http://citrus.org/schema}:status"
                value="pending"/>
</selector>
```

This will match all incoming messages regardless the XML namespace prefix that is used.

18.7. JsonPath selector

It is also possible to evaluate some JsonPath expression on the message payload in order to select a message from a message channel. The JsonPath expression outcome must match an expected value and only then the message is consumed from the channel.

The syntax for the JsonPath expression is to be defined as the element name like this:

Java

```
when(receive("orderChannelEndpoint")
        .selector(Collections.singletonMap("jsonPath:$.order.status", "pending"))
        .message()
        .body("{ \"order\": { \"status\": \"pending\" } }"));
```

XML

```
<selector>
    <element name="jsonPath:$.order.status" value="pending"/>
</selector>
```

The message selector looks for order messages with **status="pending"** in the message payload. This means that following messages would get accepted/declined by the message selector.

```
{ "order": { "status": "pending" } } //ACCEPTED
{ "order": { "status": "finished" } } //NOT ACCEPTED
```

Chapter 19. WebSocket support

The WebSocket message protocol builds on top of Http standard and brings bidirectional communication to the Http client-server world. Citrus is able to send and receive messages with WebSocket connections as client and server. The Http server implementation is now able to define multiple WebSocket endpoints. The new Citrus WebSocket client is able to publish and consumer messages via bidirectional WebSocket protocol.

The new WebSocket support is located in the module **citrus-websocket** . Therefore we need to add this module to our project as dependency when we are about to use the WebSocket features in Citrus.

```
<dependency>
    <groupId>org.citrusframework</groupId>
    <artifactId>citrus-websocket</artifactId>
    <version>${citrus.version}</version>
</dependency>
```

As Citrus provides a customized WebSocket configuration schema for the Spring application context configuration files we have to add name to the top level **beans** element. Simply include the websocket-config namespace in the configuration XML files as follows.

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:citrus="http://www.citrusframework.org/schema/config"
      xmlns:citrus-websocket="http://www.citrusframework.org/schema/websocket/config"
      xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/config
            http://www.citrusframework.org/schema/config/citrus-config.xsd
            http://www.citrusframework.org/schema/websocket/config
            http://www.citrusframework.org/schema/websocket/config/citrus-websocket-
config.xsd">

    [...]

</beans>
```

Now our project is ready to use the Citrus WebSocket support. First of all let us send a message via WebSocket connection to some server.

19.1. WebSocket client

On the client side Citrus offers a client component that goes directly to the Spring bean application context. The client needs a server endpoint uri. This is a WebSocket protocol endpoint uri.

Citrus Bean

```
@BindToRegistry
public WebSocketClient helloWebSocketClient() {
    return new WebSocketClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .timeout(5000L)
        .build();
}
```

Spring Bean

```
@Bean
public WebSocketClient helloWebSocketClient() {
    return new WebSocketClientBuilder()
        .requestUrl("http://localhost:8080/hello")
        .timeout(5000L)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-websocket="http://www.citrusframework.org/schema/websocket/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/websocket/config
           http://www.citrusframework.org/schema/websocket/config/citrus-websocket-
config.xsd">

    <citrus-websocket:client id="helloWebSocketClient"
                     url="http://localhost:8080/hello"
                     timeout="5000"/>
</beans>
```

The **url** defines the endpoint to send messages to. The server has to be a WebSocket ready web server that supports Http connection upgrade for WebSocket protocols. WebSocket by its nature is an asynchronous bidirectional protocol. This means that the connection between client and server remains open and both server and client can send and receive messages. So when the Citrus client is waiting for a message we need a timeout that stops the asynchronous waiting. The receiving test action and the test case will fail when such a timeout is raised.

The WebSocket client will automatically open a connection to the server and ask for a connection upgrade to WebSocket protocol. This handshake is done once when the connection to the server is

established. After that the client can push messages to the server and on the other side the server can push messages to the client. Now let's first push some messages to the server:

```
<send endpoint="helloWebSocketClient">
  <message>
      <payload>
          <TestMessage>
              <Text>Hello WebSocketServer</Text>
          </TestMessage>
      </payload>
  </message>
</send>
```

The connection handshake and the connection upgrade is done automatically by the client. After that the message is pushed to the server. As WebSocket is a bidirectional protocol we can also receive messages on the WebSocket client. These messages are pushed from server to all connected clients.

```
<receive endpoint="helloWebSocketClient">
  <message>
      <payload>
          <TestMessage>
              <Text>Hello WebSocketClient</Text>
          </TestMessage>
      </payload>
  </message>
</receive>
```

We just use the very same client endpoint component in a message receive action. The client will wait for messages from the server and once received perform the well known message validation. Here we expect some XML message payload. This completes the client side as we are able to push and consumer messages via WebSocket connections.



Up to now we have used static WebSocket endpoint URIs in our client component configurations. This can be done with a more powerful dynamic endpoint URI in WebSocket client. Similar to the endpoint resolving mechanism in SOAP you can dynamically set the called endpoint uri at test runtime through message header values. By default Citrus will check a specific header entry for dynamic endpoint URI which is simply defined for each message sending action inside the test.

The **dynamicEndpointResolver** bean must implement the EndpointUriResolver interface in order to resolve dynamic endpoint uri values. Citrus offers a default implementation, the **DynamicEndpointUriResolver**, which uses a specific message header for setting dynamic endpoint uri. The message header needs to specify the header **citrus_endpoint_uri** with a valid request uri.

```
<header>
  <element name="citrus_endpoint_uri"
value="ws://localhost:8080/customers/${customerId}"/>
</header>
```

The specific send action above will send its message to the dynamic endpoint (ws://localhost:8080/customers/${customerId}[ws://localhost:8080/customers/${customerId}]) which is set in the header **citrus_endpoint_uri** .

19.2. WebSocket server endpoints

On the server side Citrus has a Http server implementation that we can easily start during test runtime. The Http server accepts connections from clients and also supports WebSocket upgrade strategies. This means clients can ask for an upgrade to the WebSocket standard. In this handshake the server will upgrade the connection to WebSocket and afterwards client and server can exchange messages over this connection. This means the connection is kept alive and multiple messages can be exchanged. Let's see how WebSocket endpoints are added to a Http server component in Citrus.

Citrus Bean

```
@BindToRegistry
public WebSocketServer helloWebSocketServer() {
    return new WebSocketServerBuilder()
        .port(8080)
        .webSockets(Arrays.asList(
            webSocket1(),
            webSocket2()
        ))
        .autoStart(true)
        .build();
}

private WebSocketEndpoint webSocket1() {
    WebSocketServerEndpointConfiguration webSocketConfig = new
WebSocketServerEndpointConfiguration();
    webSocketConfig.setName("websocket1");
    webSocketConfig.setEndpointUri("/test1");
    return new WebSocketEndpoint(webSocketConfig);
}

private WebSocketEndpoint webSocket2() {
    WebSocketServerEndpointConfiguration webSocketConfig = new
WebSocketServerEndpointConfiguration();
    webSocketConfig.setName("websocket2");
    webSocketConfig.setEndpointUri("/test2");
    webSocketConfig.setTimeout(10000L);
    return new WebSocketEndpoint(webSocketConfig);
}
```

Spring Bean

```
@Bean
public WebSocketServer helloWebSocketServer() {
    return new WebSocketServerBuilder()
        .port(8080)
        .webSockets(Arrays.asList(
            webSocket1(),
            webSocket2()
        ))
        .autoStart(true)
        .build();
}

private WebSocketEndpoint webSocket1() {
    WebSocketServerEndpointConfiguration webSocketConfig = new
WebSocketServerEndpointConfiguration();
    webSocketConfig.setName("websocket1");
    webSocketConfig.setEndpointUri("/test1");
    return new WebSocketEndpoint(webSocketConfig);
}

private WebSocketEndpoint webSocket2() {
    WebSocketServerEndpointConfiguration webSocketConfig = new
WebSocketServerEndpointConfiguration();
    webSocketConfig.setName("websocket2");
    webSocketConfig.setEndpointUri("/test2");
    webSocketConfig.setTimeout(10000L);
    return new WebSocketEndpoint(webSocketConfig);
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-websocket="http://www.citrusframework.org/schema/websocket/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/websocket/config
           http://www.citrusframework.org/schema/websocket/config/citrus-websocket-
config.xsd">

    <citrus-websocket:server id="helloWebSocketServer"
            port="8080"
            auto-start="true">
        <citrus-websocket:endpoints>
            <citrus-websocket:endpoint ref="websocket1"/>
            <citrus-websocket:endpoint ref="websocket2"/>
        </citrus-websocket:endpoints>
    </citrus-websocket:server>

    <citrus-websocket:endpoint id="websocket1" path="/test1"/>
    <citrus-websocket:endpoint id="websocket2" path="/test2" timeout="10000"/>
</beans>
```

The embedded Jetty WebSocket server component in Citrus now is able to define multiple WebSocket endpoints. The WebSocket endpoints match to a request path on the server and are referenced by a unique id. Each WebSocket endpoint can follow individual timeout settings. In a test we can use these endpoints directly to receive messages.

Java

```
@CitrusTest
public void httpWebSocketServerTest() {
    when(receive()
            .endpoint(websocket1)
            .message()
                .body("..."))
    );

    then(send()
            .endpoint(websocket1)
            .message()
                .body("..."))
    );
}
```

XML

```
<test name="HttpWebSocketServerTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="websocket1">
        <message>
            <body>
              <data>...</data>
            </body>
        </message>
      </receive>

      <send endpoint="websocket1">
        <message>
            <body>
              <data>...</data>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: HttpWebSocketServerTest
actions:
  - receive:
      endpoint: "websocket1"
      message:
        body:
          data: {}
  - send:
      endpoint: "websocket1"
      message:
        body:
          data: {}
```

Spring XML

```
<testcase name="httpWebSocketServerTest">
    <actions>
        <receive endpoint="websocket1">
            <message>
                <data>...</data>
            </message>
        </receive>

        <send endpoint="websocket1">
            <message>
                <data>...</data>
            </message>
        </send>
    </actions>
</testcase>
```

As you can see we reference the endpoint id in both receive and send actions. Each WebSocket endpoint holds one or more open connections to its clients. Each message that is sent is pushed to all connected clients. Each client can send messages to the WebSocket endpoint.

The WebSocket endpoint component handles connection handshakes automatically and caches all open sessions in memory. By default all connected clients will receive the messages pushed from server. This is done completely behind the scenes. The Citrus server is able to handle multiple WebSocket endpoints with different clients connected to it at the same time. This is why we have to choose the WebSocket endpoint on the server by its identifier when sending and receiving messages.

With this WebSocket endpoints we change the Citrus server behavior so that clients can upgrade to WebSocket connection. Now we have a bidirectional connection where the server can push messages to the client and vice versa.

19.3. WebSocket headers

The WebSocket standard defines some default headers to use during connection upgrade. These headers are made available to the test case in both directions. Citrus will handle these header values with special care when WebSocket support is activated on a server or client. Now WebSocket messages can also be split into multiple pieces. Each message part is pushed separately to the server but still is considered to be a single message payload. The server has to collect and aggregate all messages until a special message header **isLast** is set in one of the message parts.

The Citrus WebSocket client can slice messages into several parts.

Java

```
@CitrusTest
public void webSocketClientTest() {
    when(send()
            .endpoint(webSocketClient)
            .message()
                .type(MessageType.JSON)
                .body("""
                [
                  {
                    "event" : "client_message_1",
                    "timestamp" : "citrus:currentDate()"
                  },
                """)
                .header("citrus_websocket_is_last", false)
    );

    then(send()
            .endpoint(webSocketClient)
            .message()
                .type(MessageType.JSON)
                .body("""
                  {
                    "event" : "client_message_2",
                    "timestamp" : "citrus:currentDate()"
                  }
                ]
                """)
                .header("citrus_websocket_is_last", true)
    );
}
```

XML

```
<test name="WebSocketClientTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="webSocketClient">
        <message type="json">
            <body>
              <data>
              [
                {
                  "event" : "client_message_1",
                  "timestamp" : "citrus:currentDate()"
                },
              </data>
            </body>
            <headers>
              <header name="citrus_websocket_is_last" value="false"/>
            </headers>
        </message>
      </send>

      <send endpoint="webSocketClient">
        <message type="json">
            <body>
              <data>
                {
                  "event" : "client_message_2",
                  "timestamp" : "citrus:currentDate()"
                }
              ]
              </data>
            </body>
            <headers>
              <header name="citrus_websocket_is_last" value="true"/>
            </headers>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: WebSocketClientTest
actions:
  - send:
      endpoint: "webSocketClient"
      message:
        type: JSON
        body:
          data: |
            [
              {
                "event" : "client_message_1",
                "timestamp" : "citrus:currentDate()"
              },
        headers:
          - name: citrus_websocket_is_last
            value: false
  - send:
      endpoint: "webSocketClient"
      message:
        type: JSON
        body:
          data: |
              {
                "event" : "client_message_2",
                "timestamp" : "citrus:currentDate()"
              }
            ]
        headers:
          - name: citrus_websocket_is_last
            value: true
```

Spring XML

```
<testcase name="webSocketClientTest">
    <actions>
        <send endpoint="webSocketClient">
            <message type="json">
                <data>
                [
                    {
                        "event" : "client_message_1",
                        "timestamp" : "citrus:currentDate()"
                    },
                </data>
            </message>
            <header>
                <element name="citrus_websocket_is_last" value="false"/>
            </header>
        </send>

        <sleep milliseconds="500"/>

        <send endpoint="webSocketClient">
            <message type="json">
                <data>
                    {
                        "event" : "client_message_2",
                        "timestamp" : "citrus:currentDate()"
                    }
                  ]
                </data>
            </message>
            <header>
                <element name="citrus_websocket_is_last" value="true"/>
            </header>
        </send>
    </actions>
</testcase>
```

The test above has two separate send operations both sending to a WebSocket endpoint. The first sending action sets the header **citrus_websocket_is_last** to **false** which indicates that the message is not complete yet. The 2nd send action pushes the rest of the message to the server and set the **citrus_websocket_is_last** header to **true** . Now the server is able to aggregate the message pieces to a single message payload. The result is a valida JSON array with both events in it.

```
[
  {
    "event" : "client_message_1",
    "timestamp" : "2015-01-01"
  },
  {
    "event" : "client_message_2",
    "timestamp" : "2015-01-01"
  }
]
```

Now the server part in Citrus is able to handle these sliced messages, too. The server will automatically aggregate those message parts before passing it to the test case for validation.

Chapter 20. Mail support

Sending and receiving mails is the next interest we are going to talk about. When dealing with mail communication you most certainly need to interact with some sort of IMAP or POP mail server. But in Citrus we do not want to manage mails in a personal inbox. We just need to be able to exchange mail messages the persisting in a user inbox is not part of our business.

This is why Citrus provides **just** an SMTP mail server which accepts mail messages from clients. Once the SMTP server has accepted an incoming mail it forwards those data to the running test case. In the test case you can receive the incoming mail message and perform message validation as usual. The mail sending part is easy as Citrus offers a mail client that connects to some SMTP server for sending mails to the outside world.



The mail components in Citrus are kept in a separate Maven module. So you should check that the module is available as Maven dependency in your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-mail</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Read the next section in order to find out more about the mail message support in Citrus.

20.1. Mail client

The mail sending part is quite easy and straight forward. We just need to send a mail message to some SMTP server. So Citrus provides a mail client that sends out mail messages.

Citrus Bean

```
@BindToRegistry
public MailClient simpleMailClient() {
    return new MailClientBuilder()
            .host("localhost")
            .port(25025L)
            .build();
}
```

Spring Bean

```
@Bean
public MailClient simpleMailClient() {
    return new MailClientBuilder()
            .host("localhost")
            .port(25025L)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:citrus="http://www.citrusframework.org/schema/config"
      xmlns:citrus-mail="http://www.citrusframework.org/schema/mail/config"
      xsi:schemaLocation="
          http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/config
          http://www.citrusframework.org/schema/config/citrus-config.xsd
          http://www.citrusframework.org/schema/mail/config
          http://www.citrusframework.org/schema/mail/config/citrus-mail-config.xsd">

    <citrus-mail:client id="simpleMailClient"
          host="localhost"
          port="25025"/>
</beans>
```

This is how a Citrus mail client component is defined in the Spring application context. You can use this client referenced by its id or name in your test case in a message sending action. The client defines a host and port attribute which should connect the client to some SMTP server instance.

We all know mail message contents. The mail message has some general properties set by the user:

from The message sender mail address

to The message recipient mail address. You can add multiple recipients by using a comma separated list.

cc Copy recipient mail address. You can add multiple recipients by using a comma separated list.

bcc Blind copy recipient mail address. You can add multiple recipients by using a comma separated list.

subject Some subject used as mail headline.

As a tester you are able to set these properties in your test case. Citrus defines an XML mail message representation that you can use inside your send action. Let us have a look at this:

Java

```
@CitrusTest
public void mailClientTest() {
    given(send()
            .endpoint(simpleMailClient)
            .message()
            .body("""
            <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
            </mail-message>
            """));
}
```

XML

```
<test name="MailClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="simpleMailClient">
        <message>
            <body>
                <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <from>christoph@citrusframework.com</from>
                    <to>dev@citrusframework.com</to>
                    <cc></cc>
                    <bcc></bcc>
                    <subject>This is a test mail message</subject>
                    <body>
                        <contentType>text/plain; charset=utf-8</contentType>
                        <content>Hello Citrus mail server!</content>
                    </body>
                </mail-message>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: MailClientTest
actions:
  - send:
      endpoint: "simpleMailClient"
      message:
        body: |
          <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
          </mail-message>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailClientTest">
    <actions>
        <send endpoint="simpleMailClient">
            <message>
                <payload>
                    <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <from>christoph@citrusframework.com</from>
                        <to>dev@citrusframework.com</to>
                        <cc></cc>
                        <bcc></bcc>
                        <subject>This is a test mail message</subject>
                        <body>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>Hello Citrus mail server!</content>
                        </body>
                    </mail-message>
                </payload>
            </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

The basic XML mail message representation defines a list of basic mail properties such as **from**, **to** or **subject** . In addition to that we define a text body which is either plain text or HTML. You can

specify the content type of the mail body very easy (e.g. text/plain or text/html). By default, Citrus uses **text/plain** content type.

Now when dealing with mail messages you often come to use multipart structures for attachments. In Citrus you can define attachment content as base64 character sequence. The Citrus mail client will automatically create a proper multipart mail mime message using the content types and body parts specified.

Java

```
@CitrusTest
public void mailClientTest() {
    given(send()
            .endpoint(simpleMailClient)
            .message()
            .body("""
            <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                    <attachments>
                        <attachment>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>This is attachment data</content>
                            <fileName>attachment.txt</fileName>
                        </attachment>
                    </attachments>
                </body>
            </mail-message>
            """));
}
```

XML

```
<test name="MailClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="simpleMailClient">
        <message>
            <body>
                <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <from>christoph@citrusframework.com</from>
                    <to>dev@citrusframework.com</to>
                    <cc></cc>
                    <bcc></bcc>
                    <subject>This is a test mail message</subject>
                    <body>
                        <contentType>text/plain; charset=utf-8</contentType>
                        <content>Hello Citrus mail server!</content>
                        <attachments>
                            <attachment>
                                <contentType>text/plain; charset=utf-8</contentType>
                                <content>This is attachment data</content>
                                <fileName>attachment.txt</fileName>
                            </attachment>
                        </attachments>
                    </body>
                </mail-message>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: MailClientTest
actions:
  - send:
      endpoint: "simpleMailClient"
      message:
        body: |
          <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                    <attachments>
                        <attachment>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>This is attachment data</content>
                            <fileName>attachment.txt</fileName>
                        </attachment>
                    </attachments>
                </body>
          </mail-message>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailClientTest">
    <actions>
        <send endpoint="simpleMailClient">
            <message>
                <payload>
                    <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <from>christoph@citrusframework.com</from>
                        <to>dev@citrusframework.com</to>
                        <cc></cc>
                        <bcc></bcc>
                        <subject>This is a test mail message</subject>
                        <body>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>Hello Citrus mail server!</content>
                            <attachments>
                                <attachment>
                                    <contentType>text/plain; charset=utf-
8</contentType>
                                    <content>This is attachment data</content>
                                    <fileName>attachment.txt</fileName>
                                </attachment>
                            </attachments>
                        </body>
                    </mail-message>
                </payload>
            </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

That completes the basic mail client capabilities. But wait we have not talked about error scenarios where mail communication results in error. When running into mail error scenarios we have to handle the error respectively with exception handling. When the mail server responded with errors Citrus will raise mail exceptions automatically and your test case fails accordingly.

As a tester you can catch and assert these mail exceptions verifying your error scenario.

Java

```
@CitrusTest
public void mailClientTest() {
    given(assertException()
            .exception(MailSendException.class)
            .when(send()
                .endpoint(simpleMailClient)
                .message()
                .body("""
                <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    [...]
                </mail-message>
                """)
            ));
}
```

XML

```
<test name="MailClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <assert exception="org.springframework.mail.MailSendException">
        <when>
          <send endpoint="simpleMailClient">
            <message>
                <body>
                    <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                        [...]
                    </mail-message>
                </body>
            </message>
          </send>
      </when>
      </assert>
    </actions>
</test>
```

YAML

```
name: MailClientTest
actions:
  - assert:
      exception: "org.springframework.mail.MailSendException"
      when:
        send:
          endpoint: "simpleMailClient"
          message:
            body: |
              <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    [...]
              </mail-message>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailClientTest">
    <actions>
        <assert exception="org.springframework.mail.MailSendException">
            <when>
                <send endpoint="simpleMailClient">
                    <message>
                        <payload>
                            <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                                [...]
                            </mail-message>
                        </payload>
                    </message>
                </send>
            </when>
        </assert>
    </actions>
  </testcase>
</spring:beans>
```

We assert the ***MailSendException*** from Spring to be thrown while sending the mail message to the SMTP server. With exception message validation you are able to expect very specific mail send errors on the client side. This is how you can handle some sort of error situation returned by the mail server. Speaking of mail servers we need to also talk about providing a mail server endpoint in Citrus for clients. This is part of our next section.

20.2. Mail server

Consuming mail messages is a more complicated task as we need to have some sort of server that

clients can connect to. In your mail client software you typically point to some IMAP or POP inbox and receive mails from that endpoint. In Citrus we do not want to manage a whole personal mail inbox such as IMAP or POP would provide. We just need an SMTP server endpoint for clients to send mails to. The SMTP server accepts mail messages and forwards those to a running test case for further validation.



We have no user inbox where incoming mails are stored. The mail server just forwards incoming mails to the running test for validation. After the test the incoming mail message is gone.

And this is exactly what the Citrus mail server is capable of. The server is a very lightweight SMTP server. All incoming mail client connections are accepted by default and the mail data is converted into a Citrus XML mail interface representation. The XML mail message is then passed to the running test for validation.

Let us have a look at the Citrus mail server component and how you can add it to the Spring application context.

Citrus Bean

```
@BindToRegistry
public MailServer simpleMailServer() {
    return new MailServerBuilder()
            .port(25025L)
            .autoStart(true)
            .build();
}
```

Spring Bean

```
@Bean
public MailServer simpleMailServer() {
    return new MailServerBuilder()
            .port(25025L)
            .autoStart(true)
            .build();
}
```

Spring XML

```
<citrus-mail:server id="simpleMailServer"
      port="25025"
      auto-start="true"/>
```

The mail server component receives several properties such as **port** or **auto-start** . Citrus starts an in-memory SMTP server that clients can connect to.

In your test case you can then receive the incoming mail messages on the server in order to

perform the well known XML validation mechanisms within Citrus. The message header and the payload contain all mail information so you can verify the content with expected templates as usual:

Java

```
@CitrusTest
public void mailServerTest() {
    when(receive()
            .endpoint(simpleMailServer)
            .message()
            .body("""
            <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
            </mail-message>
            """)
            .header("citrus_mail_from", "christoph@citrusframework.com")
            .header("citrus_mail_to", "dev@citrusframework.com")
            .header("citrus_mail_subject", "This is a test mail message")
            .header("citrus_mail_content_type", "text/plain; charset=utf-8"));
}
```

XML

```
<test name="MailServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="simpleMailServer">
        <message>
            <body>
                <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <from>christoph@citrusframework.com</from>
                    <to>dev@citrusframework.com</to>
                    <cc></cc>
                    <bcc></bcc>
                    <subject>This is a test mail message</subject>
                    <body>
                        <contentType>text/plain; charset=utf-8</contentType>
                        <content>Hello Citrus mail server!</content>
                    </body>
                </mail-message>
            </body>
            <headers>
                <header name="citrus_mail_from"
value="christoph@citrusframework.com"/>
                <header name="citrus_mail_to" value="dev@citrusframework.com"/>
                <header name="citrus_mail_subject" value="This is a test mail
message"/>
                <header name="citrus_mail_content_type" value="text/plain;
charset=utf-8"/>
            </headers>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: MailServerTest
actions:
  - receive:
      endpoint: "simpleMailServer"
      message:
        body: |
          <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
          </mail-message>
        headers:
        - name: "citrus_mail_from"
          value: "christoph@citrusframework.com"
        - name: "citrus_mail_to"
          value: "dev@citrusframework.com"
        - name: "citrus_mail_subject"
          value: "This is a test mail message"
        - name: "citrus_mail_content_type"
          value: "text/plain; charset=utf-8"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailServerTest">
    <actions>
        <receive endpoint="simpleMailServer">
            <message>
                <payload>
                    <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <from>christoph@citrusframework.com</from>
                        <to>dev@citrusframework.com</to>
                        <cc></cc>
                        <bcc></bcc>
                        <subject>This is a test mail message</subject>
                        <body>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>Hello Citrus mail server!</content>
                        </body>
                    </mail-message>
                </payload>
            </message>
            <header>
                <element name="citrus_mail_from"
value="christoph@citrusframework.com"/>
                <element name="citrus_mail_to" value="dev@citrusframework.com"/>
                <element name="citrus_mail_subject" value="This is a test mail
message"/>
                <element name="citrus_mail_content_type" value="text/plain;
charset=utf-8"/>
            </header>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

The general mail properties such as **from**, **to**, **subject** are available as elements in the mail payload and in the message header information. The message header names do start with a common Citrus mail prefix **citrus_mail** . Following from that you can verify these special mail message headers in your test as shown above. Citrus offers following mail headers:

- citrus_mail_from
- citrus_mail_to
- citrus_mail_cc
- citrus_mail_bcc
- citrus_mail_subject
- citrus_mail_replyTo

- citrus_mail_date

In addition to that Citrus converts the incoming mail data to a special XML mail representation which is passed as message payload to the test. The mail body parts are represented as body and optional attachment elements. As this is plain XML you can verify the mail message content as usual using Citrus variables, functions and validation matchers.

Regardless of how the mail message has passed the validation the Citrus SMTP mail server will automatically respond with success codes (SMTP 250 OK) to the calling client. This is the basic Citrus mail server behavior where all client connections are accepted an all mail messages are responded with SMTP 250 OK response codes.

Now in more advanced usage scenarios the tester may want to control the mail communication outcome. User can force some error scenarios where mail clients are not accepted or mail communication should fail with some SMTP error state for instance.

By using a more advanced mail server setup the tester gets more power to sending back mail server response codes to the mail client. Just use the advanced mail adapter implementation in your mail server component configuration:

Citrus Bean

```
@BindToRegistry
public MailServer advancedMailServer() {
    return new MailServerBuilder()
            .port(25025L)
            .autoAccept(false)
            .splitMultipart(true)
            .autoStart(true)
            .build();
}
```

Spring Bean

```
@Bean
public MailServer advancedMailServer() {
    return new MailServerBuilder()
            .port(25025L)
            .autoAccept(false)
            .splitMultipart(true)
            .autoStart(true)
            .build();
}
```

Spring XML

```
<citrus-mail:server id="advancedMailServer"
      auto-accept="false"
      split-multipart="true"
      port="25025"
      auto-start="true"/>
```

We have disabled the **auto-accept** mode on the mail server. This means that we have to do some additional steps in your test case to accept the incoming mail message first. So we can decide in our test case whether to accept or decline the incoming mail message for a more powerful test. You accept/decline a mail message with a special XML accept request/response exchange in your test case:

Java

```
@CitrusTest
public void mailServerTest() {
    when(receive()
            .endpoint(advancedMailServer)
            .message()
            .body("""
            <accept-request
xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
            </accept-request>
            """));
}
```

XML

```
<test name="MailServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="advancedMailServer">
        <message>
            <body>
                <accept-request
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <from>christoph@citrusframework.com</from>
                    <to>dev@citrusframework.com</to>
                </accept-request>
            </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: MailServerTest
actions:
  - receive:
      endpoint: "advancedMailServer"
      message:
        body: |
          <accept-request xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
          </accept-request>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailServerTest">
    <actions>
        <receive endpoint="advancedMailServer">
            <message>
                <payload>
                    <accept-request
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <from>christoph@citrusframework.com</from>
                        <to>dev@citrusframework.com</to>
                    </accept-request>
                </payload>
            </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

So before receiving the actual mail message we receive this simple accept-request in our test. The accept request gives us the message **from** and **to** resources of the mail message. Now the test decides to also decline a mail client connection. You can simulate that the server does not accept the mail client connection by sending back a negative accept response.

Java

```
@CitrusTest
public void mailServerTest() {
    given(send()
            .endpoint(advancedMailClient)
            .message()
            .body("""
            <accept-response
xmlns="http://www.citrusframework.org/schema/mail/message">
                <accept>true</accept>
            </accept-response>
            """));
}
```

XML

```
<test name="MailServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="advancedMailClient">
        <message>
            <body>
                <accept-response
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <accept>true</accept>
                </accept-response>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: MailServerTest
actions:
  - send:
      endpoint: "advancedMailClient"
      message:
        body: |
          <accept-response xmlns="http://www.citrusframework.org/schema/mail/message">
                <accept>true</accept>
          </accept-response>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailServerTest">
    <actions>
        <send endpoint="advancedMailServer">
            <message>
                <payload>
                    <accept-response
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <accept>true</accept>
                    </accept-response>
                </payload>
            </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

Depending on the accept outcome the mail client will receive an error response with proper error codes. If you accept the mail message with a positive accept response the next step in your test receives the actual mail message as we have seen it before in this chapter.

Now besides not accepting a mail message in the first place you can als simulate another error scenario with the mail server. In this scenario the mail server should respond with some sort of SMTP error code after accepting the message. This is done with a special mail response message like this:

Java

```
@CitrusTest
public void mailServerTest() {
    when(receive()
            .endpoint(advancedMailServer)
            .message()
            .body("""
            <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
            </mail-message>
            """));

    then(send()
            .endpoint(advancedMailServer)
            .message()
            .body("""
            <mail-response xmlns="http://www.citrusframework.org/schema/mail/message">
                <code>443</code>
                <message>Failed!</message>
            </mail-response>
            """));
}
```

XML

```
<test name="MailServerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="advancedMailServer">
        <message>
            <body>
                <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <from>christoph@citrusframework.com</from>
                    <to>dev@citrusframework.com</to>
                    <cc></cc>
                    <bcc></bcc>
                    <subject>This is a test mail message</subject>
                    <body>
                        <contentType>text/plain; charset=utf-8</contentType>
                        <content>Hello Citrus mail server!</content>
                    </body>
                </mail-message>
            </body>
        </message>
      </receive>

      <send endpoint="advancedMailServer">
        <message>
            <body>
                <mail-response
xmlns="http://www.citrusframework.org/schema/mail/message">
                    <code>443</code>
                    <message>Failed!</message>
                </mail-response>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: MailServerTest
actions:
  - receive:
      endpoint: "advancedMailServer"
      message:
        body: |
          <mail-message xmlns="http://www.citrusframework.org/schema/mail/message">
                <from>christoph@citrusframework.com</from>
                <to>dev@citrusframework.com</to>
                <cc></cc>
                <bcc></bcc>
                <subject>This is a test mail message</subject>
                <body>
                    <contentType>text/plain; charset=utf-8</contentType>
                    <content>Hello Citrus mail server!</content>
                </body>
          </mail-message>
  - send:
      endpoint: "advancedMailServer"
      message:
        body: |
          <mail-response xmlns="http://www.citrusframework.org/schema/mail/message">
                <code>443</code>
                <message>Failed!</message>
          </mail-response>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
  <testcase name="mailServerTest">
    <actions>
        <receive endpoint="advancedMailServer">
            <message>
                <payload>
                    <mail-message
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <from>christoph@citrusframework.com</from>
                        <to>dev@citrusframework.com</to>
                        <cc></cc>
                        <bcc></bcc>
                        <subject>This is a test mail message</subject>
                        <body>
                            <contentType>text/plain; charset=utf-8</contentType>
                            <content>Hello Citrus mail server!</content>
                        </body>
                    </mail-message>
                </payload>
            </message>
        </receive>

        <send endpoint="advancedMailServer">
            <message>
                <payload>
                    <mail-response
xmlns="http://www.citrusframework.org/schema/mail/message">
                        <code>443</code>
                        <message>Failed!</message>
                    </mail-response>
                </payload>
            </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

As you can see from the example above we first accept the connection and receive the mail content as usual. Now the test returns a negative mail response with some error code reason set. The Citrus SMTP communication will then fail and the calling mail client receives the respective error.

If you skip the negative mail response the server will automatically respond with positive SMTP response codes to the calling client.

Chapter 21. FTP support

With Citrus it is possible to start your own ftp server for accepting incoming client requests. You can also use Citrus as an FTP client to send FTP commands. The next sections deal with FTP connectivity.



The FTP components in Citrus are maintained in their own Maven module. So you should add the module as Maven dependency to your project accordingly.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-ftp</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

21.1. FTP client

We want to use Citrus to connect to some FTP server as a client sending commands such as creating a directory or listing files. Citrus offers a client component doing exactly this FTP client connection.

Spring Bean

```
@Bean
public FtpClient ftpClient() {
    return CitrusEndpoints.ftp()
            .client()
            .port(2222)
            .username("citrus")
            .password("admin")
            .timeout(10000L)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ftp="http://www.citrusframework.org/schema/ftp/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/http/config
           http://www.citrusframework.org/schema/ftp/config/citrus-ftp-config.xsd">

    <citrus-ftp:client id="ftpClient"
          host="localhost"
          port="2222"
          username="admin"
          password="admin"
          timeout="10000"/>
</beans>
```



Citrus provides a customized FTP configuration schema for the Spring XML application context configurations. Simply include the `citrus-ftp` namespace in the configuration XML files as seen in the example above.

The configuration above describes a Citrus ftp client connected to a ftp server with `ftp://localhost:22222`. For authentication username and password are defined as well as the global connection timeout. The client will automatically send username and password for proper authentication to the server when opening a new connection.

21.1.1. FTP client commands

In a test case you are now able to use the client to push commands to the server.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(send(ftpClient)
        .message(FtpMessage.command(FTPCmd.MKD).arguments("test"))
    );

    CommandResult result = new CommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(257));
    result.setReplyString("@contains(\"/test\" created)@");

    $(receive(ftpClient)
        .message(FtpMessage.result(result))
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>MKD</ftp:signal>
                            <ftp:arguments>test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </body>
            </message>
        </send>

        <receive endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>257</ftp:reply-code>
                            <ftp:reply-string>257 "/test" created.</ftp:reply-string>
                        </ftp:command-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - send:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:command xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:signal>MKD</ftp:signal>
            <ftp:arguments>test</ftp:arguments>
          </ftp:command>
  - receive:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
            <ftp:reply-code>257</ftp:reply-code>
            <ftp:reply-string>257 "/test" created.</ftp:reply-string>
          </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <send endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>MKD</ftp:signal>
                            <ftp:arguments>test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </message>
            </send>

            <receive endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>257</ftp:reply-code>
                            <ftp:reply-string>257 "/test" created.</ftp:reply-string>
                        </ftp:command-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

As you can see most of the ftp communication parameters are specified in a ftp command message. Citrus automatically converts those information to proper FTP commands and response messages.

21.1.2. Store files

The client is able to store files on the server using file transfer.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(send(ftpClient)
        .message(FtpMessage.put("test/hello.txt", DataType.ASCII).arguments(""))
    );

    PutCommandResult result = new PutCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains(Transfer complete)@");

    $(receive(ftpClient)
        .message(FtpMessage.result(result))
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:put-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:file path="test/hello.txt" type="ASCII"/>
                            <ftp:target path="/test/hello.txt"/>
                        </ftp:put-command>
                    </payload>
                </body>
            </message>
        </send>

        <receive endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:put-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Transfer
complete')@</ftp:reply-string>
                        </ftp:put-command-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - send:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:put-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:file path="test/hello.txt" type="ASCII"/>
            <ftp:target path="/test/hello.txt"/>
          </ftp:put-command>
  - receive:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:put-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
            <ftp:reply-code>226</ftp:reply-code>
            <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
          </ftp:put-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <send endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:put-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:file path="test/hello.txt" type="ASCII"/>
                            <ftp:target path="/test/hello.txt"/>
                        </ftp:put-command>
                    </payload>
                </message>
            </send>

            <receive endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:put-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Transfer
complete')@</ftp:reply-string>
                        </ftp:put-command-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

The file store operation uses the put command as message payload when sending the file request. The file content is loaded from external file resource. You can choose the transfer type `ASCII` and `BINARY`. When the file is stored on server side we receive a success result message with respective reply code and string for validation.

21.1.3. Retrieve files

We are able to retrieve files from an FTP server. We need to specify the target file path that we want to get on the server user home directory.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(send(ftpClient)
        .message(FtpMessage.get("test/hello.txt", "target/test/hello.txt",
DataType.ASCII))
    );

    $(receive(ftpClient)

.message(FtpMessage.result(getRetrieveFileCommandResult("target/test/hello.txt", new
ClassPathResource("test/hello.txt"))))
    );
}

private GetCommandResult getRetrieveFileCommandResult(String path, Resource content)
throws IOException {
    GetCommandResult result = new GetCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains('Transfer complete')@");

    GetCommandResult.File entryResult = new GetCommandResult.File();
    entryResult.setPath(path);
    entryResult.setData(FileUtils.readToString(content));
    result.setFile(entryResult);

    return result;
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:get-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:file path="test/hello.txt" type="ASCII"/>
                            <ftp:target path="target/test/hello.txt"/>
                        </ftp:get-command>
                    </payload>
                </body>
            </message>
        </send>

        <receive endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:get-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Transfer
complete')@</ftp:reply-string>
                            <ftp:file path="target/test/hello.txt">

<ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                            </ftp:file>
                        </ftp:get-command-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - send:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:get-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:file path="test/hello.txt" type="ASCII"/>
            <ftp:target path="target/test/hello.txt"/>
          </ftp:get-command>
  - receive:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:get-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
            <ftp:reply-code>226</ftp:reply-code>
            <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
            <ftp:file path="target/test/hello.txt">
              <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
            </ftp:file>
          </ftp:get-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <send endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:get-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:file path="test/hello.txt" type="ASCII"/>
                            <ftp:target path="target/test/hello.txt"/>
                        </ftp:get-command>
                    </payload>
                </message>
            </send>

            <receive endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:get-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Transfer
complete')@</ftp:reply-string>
                            <ftp:file path="target/test/hello.txt">

<ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                            </ftp:file>
                        </ftp:get-command-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

When file transfer is complete we are able to verify the file content in a command result. The file content is provided as data string.

21.1.4. List files

Listing files on the server is possible with the list command.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(send(ftpClient)
        .message(FtpMessage.list("test"))
    );

    $(receive(ftpClient)
        .message(FtpMessage.result(getListCommandResult("hello.txt")))
    );
}

private ListCommandResult getListCommandResult(String ... fileNames) {
    ListCommandResult result = new ListCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains('Closing data connection')@");

    ListCommandResult.Files expectedFiles = new ListCommandResult.Files();

    for (String fileName : fileNames) {
        ListCommandResult.Files.File entry = new ListCommandResult.Files.File();
        entry.setPath(fileName);
        expectedFiles.getFiles().add(entry);
    }

    result.setFiles(expectedFiles);

    return result;
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:list-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:target path="test" />
                        </ftp:list-command>
                    </payload>
                </body>
            </message>
        </send>

        <receive endpoint="ftpClient">
            <message>
                <body>
                    <payload>
                        <ftp:list-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Closing data
connection')@</ftp:reply-string>
                            <ftp:files>
                              <ftp:file path="hello.txt"/>
                            </ftp:files>
                        </ftp:list-command-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - send:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:list-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:target path="test" />
          </ftp:list-command>
  - receive:
      endpoint: "ftpClient"
      message:
        body: |
          <ftp:list-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
            <ftp:reply-code>226</ftp:reply-code>
            <ftp:reply-string>@contains('Closing data connection')@</ftp:reply-string>
            <ftp:files>
              <ftp:file path="hello.txt"/>
            </ftp:files>
          </ftp:list-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <send endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:list-command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:target path="test" />
                        </ftp:list-command>
                    </payload>
                </message>
            </send>

            <receive endpoint="ftpClient">
                <message>
                    <payload>
                        <ftp:list-command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                            <ftp:reply-code>226</ftp:reply-code>
                            <ftp:reply-string>@contains('Closing data
connection')@</ftp:reply-string>
                            <ftp:files>
                              <ftp:file path="hello.txt"/>
                            </ftp:files>
                        </ftp:list-command-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

Listing files results in a command result that gives us the list of files on the server directory. We are able to verify that list with respective file paths.

21.2. FTP server

Now that we are able to access FTP as a client we might also want to simulate the server side. Therefore Citrus offers a server component that is listening on a port for incoming FTP connections. The server has a default home directory on the local file system specified. But you can also define home directories per user. For now let us have a look at the server configuration component:

Citrus Bean

```
@BindToRegistry
public FtpServer ftpListServer() {
    return CitrusEndpoints.ftp()
            .server()
            .port(2222)
            .autoLogin(true)
            .autoStart(true)
            .autoHandleCommands(Stream.of(FTPCmd.MKD.getCommand(),
                                          FTPCmd.PORT.getCommand(),

FTPCmd.TYPE.getCommand()).collect(Collectors.joining(",")))
            .userManagerProperties(new
ClassPathResource("citrus.ftp.user.properties"))
            .build();
}
```

Spring Bean

```
@Bean
public FtpServer ftpListServer() {
    return CitrusEndpoints.ftp()
            .server()
            .port(2222)
            .autoLogin(true)
            .autoStart(true)
            .autoHandleCommands(Stream.of(FTPCmd.MKD.getCommand(),
                                          FTPCmd.PORT.getCommand(),

FTPCmd.TYPE.getCommand()).collect(Collectors.joining(",")))
            .userManagerProperties(new
ClassPathResource("citrus.ftp.user.properties"))
            .build();
}
```

Spring XML

```
<citrus-ftp:server id="ftpServer"
      port="2222"
      auto-start="true"
      auto-handle-commands="MKD,PORT,TYPE"
      user-manager-properties="classpath:ftp.server.properties"/>
```

The ftp server configuration is quite simple. The server starts automatically and binds to a port. With `autoLogin` and `autoHandleCommands` we can specify the behavior of the server. When `autoLogin` is enabled the server will automatically accept user login requests. With `autoHandleCommands` we can set a list of commands that should also be handled automatically so we do not have to verify those commands in a test case. The server will automatically respond with a positive command result

then.

The user configuration is read from a **user-manager-property** file. Let us have a look at the content of this user management file:

```
# Password is "admin"
ftpserver.user.admin.userpassword=c7ad44cbad762a5da0a452f9e854fdc1e0e7a52a38015f23f3ea
b1d80b931dd472634dfac71cd34ebc35d16ab7fb8a90c81f975113d6c7538dc69dd8de9077ec
ftpserver.user.admin.homedirectory=target/ftp/user/admin
ftpserver.user.admin.enableflag=true
ftpserver.user.admin.writepermission=true
ftpserver.user.admin.maxloginnumber=0
ftpserver.user.admin.maxloginperip=0
ftpserver.user.admin.idletime=0
ftpserver.user.admin.uploadrate=0
ftpserver.user.admin.downloadrate=0

ftpserver.user.anonymous.userpassword=
ftpserver.user.anonymous.homedirectory=target/ftp/user/anonymous
ftpserver.user.anonymous.enableflag=true
ftpserver.user.anonymous.writepermission=false
ftpserver.user.anonymous.maxloginnumber=20
ftpserver.user.anonymous.maxloginperip=2
ftpserver.user.anonymous.idletime=300
ftpserver.user.anonymous.uploadrate=4800
ftpserver.user.anonymous.downloadrate=4800
```

The FTP server defines two accounts `citrus` and `anonymous`. Clients may authenticate to the server using these credentials. Based on the user account we can set a user workspace home directory. The server will save incoming stored files to this directory and the server will read retrieved files from that home directory.

In case you want to setup some files in that directory in order to provide it to clients, please copy those files to that home directory prior to the test.

The ftp-client connects to the server using the user credentials and is then able to store and retrieve files in a test.

You are able to define as many user for the ftp server as you like. In addition to that you have plenty of configuration possibilities per user. Citrus uses the Apache ftp server implementation. So for more details on configuration capabilities please consult the official Apache ftp server documentation.

The following listings show how to handle incoming commands representing different file operation such as store and retrieve. In the test we indicate the server response that we would link the server to respond with. Positive command results accept the client command and execute the command. As we have a fully qualified ftp server running the client can store, retrieve files and create and change directories. All incoming commands result in a file system change in the user home directory. So stored files are stored in that working directory and retrieved files are read

form that directory. In the test case we only receive the commands for validation purpose and to indicate server success or failure response.

21.2.1. FTP server commands

Now we would like to use the server in a test case. Each operation that arrives on the server is automatically forwarded to the test case for validation. This means that we can verify any command on the server by using a normal receive action in our test.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(receive(ftpServer)
        .message(FtpMessage.command(FTPCmd.MKD).arguments("test"))
    );

    $(send(ftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>MKD</ftp:signal>
                            <ftp:arguments>/test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </body>
            </message>
        </receive>

        <send endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - receive:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:signal>MKD</ftp:signal>
            <ftp:arguments>/test</ftp:arguments>
          </ftp:command>
  - send:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
          </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <receive endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>MKD</ftp:signal>
                            <ftp:arguments>/test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </message>
            </receive>

            <send endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The receive action uses the command signal and argument for validation. In the sample above we receive a `MKD` signal with argument `/test` which implies a create directory command. The server respectively the test case is now able to simulate the response for this command. We respond with a success command result. Following from that the Citrus FTP server implementation will create that directory in the user home directory and respond to the client with a proper success message.

Of course you can also simulate error scenarios here. Just respond in the test with a negative command result.

21.2.2. Store files

Clients are able to store files on the server component. Each file store operation is executed in the user home directory when the command result is successful. In a test you can verify the `STOR` signal coming from the client.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(receive(ftpServer)
        .message(FtpMessage.command(FTPCmd.STOR).arguments("/test/hello.txt"))
    );

    $(send(ftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>STOR</ftp:signal>
                            <ftp:arguments>/test/hello.txt</ftp:arguments>
                        </ftp:command>
                    </payload>
                </body>
            </message>
        </receive>

        <send endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - receive:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:signal>STOR</ftp:signal>
            <ftp:arguments>/test/hello.txt</ftp:arguments>
          </ftp:command>
  - send:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
          </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <receive endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>STOR</ftp:signal>
                            <ftp:arguments>/test/hello.txt</ftp:arguments>
                        </ftp:command>
                    </payload>
                </message>
            </receive>

            <send endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

After that you should find a new file in the user home directory with the given file path. The file transfer is automatically handled by the Citrus FTP server component.

21.2.3. Retrieve files

Clients should be able to get files from the server by using get/retrieve commands. In the request the client needs to give the target file path based on the user home directory.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(receive(ftpServer)
        .message(FtpMessage.command(FTPCmd.RETR).arguments("/test/hello.txt"))
    );

    $(send(ftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>RETR</ftp:signal>
                            <ftp:arguments>/test/hello.txt</ftp:arguments>
                        </ftp:command>
                    </payload>
                </body>
            </message>
        </receive>

        <send endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - receive:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:signal>RETR</ftp:signal>
            <ftp:arguments>/test/hello.txt</ftp:arguments>
          </ftp:command>
  - send:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
          </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <receive endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>RETR</ftp:signal>
                            <ftp:arguments>/test/hello.txt</ftp:arguments>
                        </ftp:command>
                    </payload>
                </message>
            </receive>

            <send endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The file request is verified with proper signal and arguments. When the server command result is positive the Citrus FTP server will transfer the file content to the calling client.

21.2.4. List files

When clients request for listing files on the server we get a list command on the server.

Java

```
@CitrusTest
public void ftpMessageTest() {
    $(receive(ftpServer)
        .message(FtpMessage.command(FTPCmd.LIST).arguments("test"))
    );

    $(send(ftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="FtpMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>LIST</ftp:signal>
                            <ftp:arguments>test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </body>
            </message>
        </receive>

        <send endpoint="ftpServer">
            <message>
                <body>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: FtpMessageTest
actions:
  - receive:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:signal>LIST</ftp:signal>
            <ftp:arguments>test</ftp:arguments>
          </ftp:command>
  - send:
      endpoint: "ftpServer"
      message:
        body: |
          <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
            <ftp:success>true</ftp:success>
          </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="FtpMessageTest">
        <actions>
            <receive endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:signal>LIST</ftp:signal>
                            <ftp:arguments>test</ftp:arguments>
                        </ftp:command>
                    </payload>
                </message>
            </receive>

            <send endpoint="ftpServer">
                <message>
                    <payload>
                        <ftp:command-result
xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
                            <ftp:success>true</ftp:success>
                        </ftp:command-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

As you can see the list command is verified with proper signal and arguments that specifies the target folder to list the files for. When the command result is positive the FTP server implementation will send back a proper list command result for that given directory in the user home directory.

Chapter 22. SFTP/SCP support

With Citrus it is possible to start your own sftp server for accepting incoming client requests. You can also use Citrus as a SFTP client to send FTP commands. The next sections deal with SFTP connectivity.



The SFTP components in Citrus are maintained in their own Maven module. So you should add the module as Maven dependency to your project accordingly.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-ftp</artifactId>
  <version>2.7.6</version>
</dependency>
```

22.1. SFTP client

We want to use Citrus to connect to some FTP server using secure file transfer and authentication. Citrus offers a client component doing exactly this SFTP client connection.

Citrus Bean

```
@BindToRegistry
public SftpClient sftpClient() {
    return CitrusEndpoints.sftp()
            .client()
            .strictHostChecking(false)
            .port(2222)
            .username("citrus")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .build();
}
```

Spring Bean

```
@Bean
public SftpClient sftpClient() {
    return CitrusEndpoints.sftp()
            .client()
            .strictHostChecking(false)
            .port(2222)
            .username("citrus")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-sftp="http://www.citrusframework.org/schema/sftp/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/sftp/config
           http://www.citrusframework.org/schema/sftp/config/citrus-sftp-config.xsd">

    <citrus-sftp:client id="sftpClient"
           strict-host-checking="false"
           port="2222"
           username="citrus"
           private-key-path="classpath:ssh/citrus.priv"
           timeout="10000"/>
</beans>
```



Citrus provides a customized SFTP configuration schema for the Spring XML application context configurations. Simply include the `citrus-sftp` namespace in the configuration XML files as seen in the example above.

The configuration above describes a Citrus sftp client. The sftp-client connects to the server using the user credentials and/or the private key authentication. The client will automatically authenticate to the server when opening a new connection. As the Citrus stfp client supports various authentication methods, the following order is default: `publickey,password,keyboard-interactive`. You're able to configure the default order with the `preferred-authentications` attribute in XML or the `preferredAuthentications` method in Java.

Citrus Bean

```
@BindToRegistry
public SftpClient sftpClient() {
    return CitrusEndpoints.sftp()
            .client()
            .strictHostChecking(false)
            .port(2222)
            .username("citrus")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .preferredAuthentications("publickey,password,gssapi-with-mic,keyboard-
interactive")
            .build();
}
```

Spring Bean

```
@Bean
public SftpClient sftpClient() {
    return CitrusEndpoints.sftp()
            .client()
            .strictHostChecking(false)
            .port(2222)
            .username("citrus")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .preferredAuthentications("publickey,password,gssapi-with-mic,keyboard-
interactive")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-sftp="http://www.citrusframework.org/schema/sftp/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/sftp/config
           http://www.citrusframework.org/schema/sftp/config/citrus-sftp-config.xsd">

    <citrus-sftp:client id="sftpClient"
           strict-host-checking="false"
           port="2222"
           username="citrus"
           private-key-path="classpath:ssh/citrus.priv",
           preferred-authentications="publickey,password,gssapi-with-mic,keyboard-
interactive"
           timeout="10000"/>
</beans>
```

22.1.1. SFTP client commands

In a test case you are now able to use the client to push commands to the server.

Java

```
@CitrusTest
public void sftpTest() {
    when(send()
        .endpoint(sftpClient)
        .message(FtpMessage.command(FTPCmd.MKD).arguments("test"))
    );

    CommandResult result = new CommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(257));
    result.setReplyString("257 Pathname created");

    then(receive()
        .endpoint(sftpClient)
        .message(FtpMessage.result(result))
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:command>
                <ftp:signal>MKD</ftp:signal>
                <ftp:arguments>test</ftp:arguments>
              </ftp:command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>257</ftp:reply-code>
                <ftp:reply-string>257 Pathname created</ftp:reply-string>
              </ftp:command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - send:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:command>
              <ftp:signal>MKD</ftp:signal>
              <ftp:arguments>test</ftp:arguments>
            </ftp:command>
  - receive:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>257</ftp:reply-code>
              <ftp:reply-string>257 Pathname created</ftp:reply-string>
            </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <send endpoint="sftpClient">
          <message>
            <payload>
              <ftp:command>
                <ftp:signal>MKD</ftp:signal>
                <ftp:arguments>test</ftp:arguments>
              </ftp:command>
            </payload>
          </message>
        </send>

        <receive endpoint="sftpClient">
          <message>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>257</ftp:reply-code>
                <ftp:reply-string>257 Pathname created</ftp:reply-string>
              </ftp:command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

As you can see most of the sftp communication parameters are specified in a ftp command message. Citrus automatically converts those information to proper FTP commands and response messages.

22.1.2. Store files

The client is able to store files on the server using file transfer.

Java

```
@CitrusTest
public void sftpTest() {
    when(send()
        .endpoint(sftpClient)
        .message(FtpMessage.put("test/hello.txt", DataType.ASCII).arguments(""))
    );

    PutCommandResult result = new PutCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains(Transfer complete)@");

    then(receive()
        .endpoint(sftpClient)
        .message(FtpMessage.result(result))
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:put-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:put-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              </ftp:put-command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - send:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:put-command>
              <ftp:file path="test/hello.txt" type="ASCII"/>
              <ftp:target path="/test/hello.txt"/>
            </ftp:put-command>
  - receive:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:put-command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>226</ftp:reply-code>
              <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
            </ftp:put-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <send endpoint="sftpClient">
          <message>
            <payload>
              <ftp:put-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </message>
        </send>

        <receive endpoint="sftpClient">
          <message>
            <payload>
              <ftp:put-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              </ftp:put-command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

The file store operation uses the put command as message payload when sending the file request. The file content is loaded from external file resource. You can choose the transfer type `ASCII` and `BINARY`. When the file is stored on server side we receive a success result message with respective reply code and string for validation.

22.1.3. Retrieve files

We are able to retrieve files from a SFTP server. We need to specify the target file path that we want to get on the server user home directory.

Java

```
@CitrusTest
public void sftpTest() {
    when(send()
        .endpoint(sftpClient)
        .message(FtpMessage.get("test/hello.txt", "target/test/hello.txt",
DataType.ASCII))
    );

    then(receive()
        .endpoint(sftpClient)

.message(FtpMessage.result(getRetrieveFileCommandResult("target/test/hello.txt", new
ClassPathResource("test/hello.txt"))))
    );
}

private GetCommandResult getRetrieveFileCommandResult(String path, Resource content)
throws IOException {
    GetCommandResult result = new GetCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains('Transfer complete')@");

    GetCommandResult.File entryResult = new GetCommandResult.File();
    entryResult.setPath(path);
    entryResult.setData(FileUtils.readToString(content));
    result.setFile(entryResult);

    return result;
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:get-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="target/test/hello.txt"/>
              </ftp:get-command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:get-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
                <ftp:file path="target/test/hello.txt">
                  <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                </ftp:file>
              </ftp:get-command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - send:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:get-command>
              <ftp:file path="test/hello.txt" type="ASCII"/>
              <ftp:target path="target/test/hello.txt"/>
            </ftp:get-command>
  - receive:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:get-command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>226</ftp:reply-code>
              <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              <ftp:file path="target/test/hello.txt">
                <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
              </ftp:file>
            </ftp:get-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <send endpoint="sftpClient">
          <message>
            <payload>
              <ftp:get-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="target/test/hello.txt"/>
              </ftp:get-command>
            </payload>
          </message>
        </send>

        <receive endpoint="sftpClient">
          <message>
            <payload>
              <ftp:get-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
                <ftp:file path="target/test/hello.txt">
                  <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                </ftp:file>
              </ftp:get-command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

When file transfer is complete we are able to verify the file content in a command result. The file content is provided as data string.

22.1.4. List files

Listing files on the server is possible with the list command.

Java

```
@CitrusTest
public void sftpTest() {
    when(send()
        .endpoint(sftpClient)
        .message(FtpMessage.list("test"))
    );

    then(receive()
        .endpoint(sftpClient)
        .message(FtpMessage.result(getListCommandResult("hello.txt")))
    );
}

private ListCommandResult getListCommandResult(String ... fileNames) {
    ListCommandResult result = new ListCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains('Closing data connection')@");

    ListCommandResult.Files.File currentDir = new ListCommandResult.Files.File();
    currentDir.setPath(".");
    expectedFiles.getFiles().add(currentDir);

    ListCommandResult.Files.File parentDir = new ListCommandResult.Files.File();
    parentDir.setPath("..");
    expectedFiles.getFiles().add(parentDir);

    ListCommandResult.Files expectedFiles = new ListCommandResult.Files();

    for (String fileName : fileNames) {
        ListCommandResult.Files.File entry = new ListCommandResult.Files.File();
        entry.setPath(fileName);
        expectedFiles.getFiles().add(entry);
    }

    result.setFiles(expectedFiles);

    return result;
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:list-command>
                <ftp:target path="test" />
              </ftp:list-command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="sftpClient">
        <message>
          <body>
            <payload>
              <ftp:list-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>150</ftp:reply-code>
                <ftp:reply-string>List files complete</ftp:reply-string>
                <ftp:files>
                  <ftp:file path="."/>
                  <ftp:file path=".."/>
                  <ftp:file path="hello.txt"/>
                </ftp:files>
              </ftp:list-command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - send:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:list-command>
              <ftp:target path="test" />
            </ftp:list-command>
  - receive:
      endpoint: sftpClient
      message:
        body:
          data: |
            <ftp:list-command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>150</ftp:reply-code>
              <ftp:reply-string>List files complete</ftp:reply-string>
              <ftp:files>
                <ftp:file path="."/>
                <ftp:file path=".."/>
                <ftp:file path="hello.txt"/>
              </ftp:files>
            </ftp:list-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <send endpoint="sftpClient">
          <message>
            <payload>
              <ftp:list-command>
                <ftp:target path="test" />
              </ftp:list-command>
            </payload>
          </message>
        </send>

        <receive endpoint="sftpClient">
          <message>
            <payload>
              <ftp:list-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>150</ftp:reply-code>
                <ftp:reply-string>List files complete</ftp:reply-string>
                <ftp:files>
                  <ftp:file path="."/>
                  <ftp:file path=".."/>
                  <ftp:file path="hello.txt"/>
                </ftp:files>
              </ftp:list-command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

Listing files results in a command result that gives us the list of files on the server directory. We are able to verify that list with respective file paths.

22.2. SFTP server

Now that we are able to access SFTP as a client we might also want to simulate the server side. Therefore Citrus offers a server component that is listening on a port for incoming SFTP

connections. The server has a default home directory on the local file system specified. But you can also define home directories per user. For now let us have a look at the server configuration component:

Citrus Bean

```
@BindToRegistry
public SftpServer sftpServer() {
    return CitrusEndpoints.sftp()
            .server()
            .port(2222)
            .autoStart(true)
            .user("citrus")
            .password("admin")
            .allowedKeyPath("classpath:ssh/citrus_pub.pem")
            .build();
}
```

Spring Bean

```
@Bean
public SftpServer sftpServer() {
    return CitrusEndpoints.sftp()
            .server()
            .port(2222)
            .autoStart(true)
            .user("citrus")
            .password("admin")
            .allowedKeyPath("classpath:ssh/citrus_pub.pem")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-sftp="http://www.citrusframework.org/schema/sftp/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/sftp/config
           http://www.citrusframework.org/schema/sftp/config/citrus-sftp-config.xsd">


    <citrus-sftp:server id="sftpServer"
                   port="2222"
                   auto-start="true"
                   user="citrus"
                   password="admin"
                   allowed-key-path="classpath:ssh/citrus_pub.pem"/>
</beans>
```

The **sftpServer** is a small but fully qualified SFTP server implementation in Citrus. The server receives a `user` that defines the user account and its home directory. All commands will be performed in this user home directory. You can set the user home directory using the `userHomePath` attribute on the server. By default this is a directory located in `${user.dir}/target/{serverName}/home/{user}`.

In case you want to setup some files in that directory in order to provide it to clients, please copy those files to that home directory prior to the test. The server adds the public key to the list of allowed keys.

The following listings show how to handle incoming commands representing different file operation such as store and retrieve. In the test we indicate the server response that we would link the server to respond with. Positive command results accept the client command and execute the command. As we have a fully qualified sftp server running the client can store, retrieve files and create and change directories. All incoming commands result in a file system change in the user home directory. So stored files are stored in that working directory and retrieved files are read form that directory. In the test case we only receive the commands for validation purpose and to indicate server success or failure response.

22.2.1. SFTP server commands

Now we would like to use the server in a test case. Each operation that arrives on the server is automatically forwarded to the test case for validation. This means that we can verify any command on the server by using a normal receive action in our test.

Java

```
@CitrusTest
public void sftpTest() {
    when(receive()
        .endpoint(sftpServer)
        .message(FtpMessage.command(FTPCmd.MKD).arguments("test"))
    );

    then(send()
        .endpoint(sftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <receive endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command>
                <ftp:signal>MKD</ftp:signal>
                <ftp:arguments>test</ftp:arguments>
              </ftp:command>
            </payload>
          </body>
        </message>
      </receive>
      <send endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - receive:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command>
              <ftp:signal>MKD</ftp:signal>
              <ftp:arguments>test</ftp:arguments>
            </ftp:command>
  - send:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command-result>
              <ftp:success>true</ftp:success>
            </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <receive endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command>
                <ftp:signal>MKD</ftp:signal>
                <ftp:arguments>/test</ftp:arguments>
              </ftp:command>
            </payload>
          </message>
        </receive>

        <send endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

The `receive` action uses the command signal and argument for validation. In the sample above we receive a `MKD` signal with argument `/test` which implies a create directory command. The server respectively the test case is now able to simulate the response for this command. We respond with a success command result. Following from that the Citrus SFTP server implementation will create that directory in the user home directory and respond to the client with a proper success message.

Of course, you can also simulate error scenarios here. Just respond in the test with a negative command result.

22.2.2. Store files

Clients are able to store files on the server component. Each file store operation is executed in the user home directory when the command result is successful. In a test you can verify the `STOR` signal

coming from the client.

Java

```
@CitrusTest
public void sftpTest() {
    when(receive()
        .endpoint(sftpServer)
        .message(put("@ignore@","/test/hello.txt", DataType.ASCII))
    );

    then(send()
        .endpoint(sftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <receive endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:put-command>
                <ftp:signal>STOR</ftp:signal>
                <ftp:file path="@ignore@" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </body>
        </message>
      </receive>
      <send endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - receive:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:put-command>
              <ftp:signal>STOR</ftp:signal>
              <ftp:file path="@ignore@" type="ASCII"/>
              <ftp:target path="/test/hello.txt"/>
            </ftp:put-command>
  - send:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command-result>
              <ftp:success>true</ftp:success>
            </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <echo>
          <message>Store file on server</message>
        </echo>

        <receive endpoint="sftpServer">
          <message>
            <payload>
              <ftp:put-command>
                <ftp:signal>STOR</ftp:signal>
                <ftp:file path="@ignore@" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </message>
        </receive>

        <send endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

After that you should find a new file in the user home directory with the given file path. The file transfer is automatically handled by the Citrus SFTP server component.

22.2.3. Retrieve files

Clients should be able to get files from the server by using get/retrieve commands. In the request the client needs to give the target file path based on the user home directory.

Java

```
@CitrusTest
public void sftpTest() {
    when(receive()
        .endpoint(sftpServer)
        .message(FtpMessage.get("/test/hello.txt", "@ignore@", DataType.ASCII))
    );

    then(send()
        .endpoint(sftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <receive endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:get-command>
                <ftp:signal>RETR</ftp:signal>
                <ftp:file path="/test/hello.txt" type="ASCII"/>
                <ftp:target path="@ignore@"/>
              </ftp:get-command>
            </payload>
          </body>
        </message>
      </receive>
      <send endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - receive:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:get-command>
              <ftp:signal>RETR</ftp:signal>
              <ftp:file path="/test/hello.txt" type="ASCII"/>
              <ftp:target path="@ignore@"/>
            </ftp:get-command>
  - send:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command-result>
              <ftp:success>true</ftp:success>
            </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <echo>
          <message>Retrieve file from server</message>
        </echo>

        <receive endpoint="sftpServer">
          <message>
            <payload>
              <ftp:get-command>
                <ftp:signal>RETR</ftp:signal>
                <ftp:file path="/test/hello.txt" type="ASCII"/>
                <ftp:target path="@ignore@"/>
              </ftp:get-command>
            </payload>
          </message>
        </receive>

        <send endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

The file request is verified with proper signal and arguments. When the server command result is positive the Citrus SFTP server will transfer the file content to the calling client.

22.2.4. List files

When clients request for listing files on the server we get a list command on the server.

Java

```
@CitrusTest
public void sftpTest() {
    when(receive()
        .endpoint(sftpServer)
        .message(FtpMessage.command(FTPCmd.LIST).arguments("test"))
    );

    then(send()
        .endpoint(sftpServer)
        .message(FtpMessage.success())
    );
}
```

XML

```
<test name="SftpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <receive endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command>
                <ftp:signal>LIST</ftp:signal>
                <ftp:arguments>test</ftp:arguments>
              </ftp:command>
            </payload>
          </body>
        </message>
      </receive>
      <send endpoint="sftpServer">
        <message>
          <body>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: SftpTest
actions:
  - receive:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command>
              <ftp:signal>LIST</ftp:signal>
              <ftp:arguments>test</ftp:arguments>
            </ftp:command>
  - send:
      endpoint: sftpServer
      message:
        body:
          data: |
            <ftp:command-result>
              <ftp:success>true</ftp:success>
            </ftp:command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="SftpTest">
    <actions>
        <receive endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command>
                <ftp:signal>LIST</ftp:signal>
                <ftp:arguments>test</ftp:arguments>
              </ftp:command>
            </payload>
          </message>
        </receive>

        <send endpoint="sftpServer">
          <message>
            <payload>
              <ftp:command-result>
                <ftp:success>true</ftp:success>
              </ftp:command-result>
            </payload>
          </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

As you can see the list command is verified with proper signal and arguments that specifies the target folder to list the files for. When the command result is positive the SFTP server implementation will send back a proper list command result for that given directory in the user home directory.

22.3. SCP client

We want to use Citrus to connect to some FTP server using secure file copy with SCP. Citrus offers a client component doing exactly this SCP client connection.

Citrus Bean

```
@BindToRegistry
public ScpClient scpClient() {
    return CitrusEndpoints.scp()
            .client()
            .port(2222)
            .username("citrus")
            .password("admin")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .build();
}
```

Spring Bean

```
@Bean
public ScpClient scpClient() {
    return CitrusEndpoints.scp()
            .client()
            .port(2222)
            .username("citrus")
            .password("admin")
            .privateKeyPath("classpath:ssh/citrus.priv")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-scp="http://www.citrusframework.org/schema/scp/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/scp/config
           http://www.citrusframework.org/schema/scp/config/citrus-scp-config.xsd">

    <citrus-scp:client id="scpClient"
               port="2222"
               username="citrus"
               password="admin"
               private-key-path="classpath:ssh/citrus.priv"/>
</beans>
```



Citrus provides a customized SCP configuration schema for the Spring XML application context configuration files. Simply include the `citrus-scp` namespace in the configuration XML files as seen in the example above.

The configuration above describes a Citrus scp client. The scp-client connects to the server using the

user credentials and/or the private key authentication. The client will automatically authenticate to the server when opening a new connection.

22.3.1. Store files

The client is able to store files on the server using file transfer.

Java

```
@CitrusTest
public void scpTest() {
    when(send()
        .endpoint(scpClient)
        .message(FtpMessage.put("test/hello.txt", DataType.ASCII).arguments(""))
    );

    PutCommandResult result = new PutCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains(Transfer complete)@");

    then(receive()
        .endpoint(scpClient)
        .message(FtpMessage.result(result))
    );
}
```

XML

```
<test name="ScpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="scpClient">
        <message>
          <body>
            <payload>
              <ftp:put-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="scpClient">
        <message>
          <body>
            <payload>
              <ftp:put-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              </ftp:put-command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: ScpTest
actions:
  - send:
      endpoint: scpClient
      message:
        body:
          data: |
            <ftp:put-command>
              <ftp:file path="test/hello.txt" type="ASCII"/>
              <ftp:target path="/test/hello.txt"/>
            </ftp:put-command>
  - receive:
      endpoint: scpClient
      message:
        body:
          data: |
            <ftp:put-command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>226</ftp:reply-code>
              <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
            </ftp:put-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="ScpTest">
    <actions>
        <send endpoint="scpClient">
          <message>
            <payload>
              <ftp:put-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="/test/hello.txt"/>
              </ftp:put-command>
            </payload>
          </message>
        </send>

        <receive endpoint="scpClient">
          <message>
            <payload>
              <ftp:put-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              </ftp:put-command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

The file store operation uses the put command as message payload when sending the file request. The file content is loaded from external file resource. You can choose the transfer type `ASCII` and `BINARY`. When the file is stored on server side we receive a success result message with respective reply code and string for validation.

22.3.2. Retrieve files

We are able to retrieve files from a SFTP server. We need to specify the target file path that we want to get on the server user home directory.

Java

```
@CitrusTest
public void scpTest() {
    when(send()
        .endpoint(scpClient)
        .message(FtpMessage.get("test/hello.txt", "target/test/hello.txt",
DataType.ASCII))
    );

    then(receive()
        .endpoint(scpClient)

.message(FtpMessage.result(getRetrieveFileCommandResult("target/test/hello.txt", new
ClassPathResource("test/hello.txt"))))
    );
}

private GetCommandResult getRetrieveFileCommandResult(String path, Resource content)
throws IOException {
    GetCommandResult result = new GetCommandResult();
    result.setSuccess(true);
    result.setReplyCode(String.valueOf(226));
    result.setReplyString("@contains('Transfer complete')@");

    GetCommandResult.File entryResult = new GetCommandResult.File();
    entryResult.setPath(path);
    entryResult.setData(FileUtils.readToString(content));
    result.setFile(entryResult);

    return result;
}
```

XML

```
<test name="ScpTest" xmlns="http://citrusframework.org/schema/xml/testcase"

xmlns:ftp="http://www.citrusframework.org/schema/ftp/message">
    <actions>
      <send endpoint="scpClient">
        <message>
          <body>
            <payload>
              <ftp:get-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="target/test/hello.txt"/>
              </ftp:get-command>
            </payload>
          </body>
        </message>
      </send>
      <receive endpoint="scpClient">
        <message>
          <body>
            <payload>
              <ftp:get-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
                <ftp:file path="target/test/hello.txt">
                  <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                </ftp:file>
              </ftp:get-command-result>
            </payload>
          </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: ScpTest
actions:
  - send:
      endpoint: scpClient
      message:
        body:
          data: |
            <ftp:get-command>
              <ftp:file path="test/hello.txt" type="ASCII"/>
              <ftp:target path="target/test/hello.txt"/>
            </ftp:get-command>
  - receive:
      endpoint: scpClient
      message:
        body:
          data: |
            <ftp:get-command-result>
              <ftp:success>true</ftp:success>
              <ftp:reply-code>226</ftp:reply-code>
              <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
              <ftp:file path="target/test/hello.txt">
                <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
              </ftp:file>
            </ftp:get-command-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:ftp="http://www.citrusframework.org/schema/ftp/message"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/ftp/message
              http://www.citrusframework.org/schema/ftp/citrus-ftp-message.xsd">
  <testcase name="ScpTest">
    <actions>
        <send endpoint="scpClient">
          <message>
            <payload>
              <ftp:get-command>
                <ftp:file path="test/hello.txt" type="ASCII"/>
                <ftp:target path="target/test/hello.txt"/>
              </ftp:get-command>
            </payload>
          </message>
        </send>

        <receive endpoint="scpClient">
          <message>
            <payload>
              <ftp:get-command-result>
                <ftp:success>true</ftp:success>
                <ftp:reply-code>226</ftp:reply-code>
                <ftp:reply-string>@contains('Transfer complete')@</ftp:reply-string>
                <ftp:file path="target/test/hello.txt">
                  <ftp:data>citrus:readFile('classpath:test/hello.txt')</ftp:data>
                </ftp:file>
              </ftp:get-command-result>
            </payload>
          </message>
        </receive>
    </actions>
  </testcase>
</spring:beans>
```

When file transfer is complete we are able to verify the file content in a command result. The file content is provided as data string.

Chapter 23. File support

In chapter message-channels we have discussed the native Spring Integration channel support which enables Citrus to interact with all Spring Integration messaging adapter implementations. This is a fantastic way to extend Citrus for additional transports. This interaction now comes handy when writing and reading files from the file system in Citrus.

23.1. Write files

We want to use the Spring Integration file adapter for both reading and writing files with a local directory. Citrus can easily connect to this file adapter implementation with its message channel support. Citrus message sender and receiver speak to message channels that are connected to the Spring Integration file adapters.

Citrus Bean

```
@BindToRegistry
public ChannelEndpointAdapter fileEndpoint() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("fileChannel");
    return new ChannelEndpointAdapter(endpointConfiguration);
}

@BindToRegistry
public MessageChannel fileChannel() {
    return new DirectChannel();
}

@BindToRegistry
public FileWritingMessageHandler fileOutboundAdapter() {
    FileWritingMessageHandler messageHandler = new FileWritingMessageHandler(new
File("some/directory"));
    messageHandler.setOutboundChannel(fileChannel());
    return messageHandler;
}
```

Spring Bean

```
@Bean
public ChannelEndpointAdapter fileEndpoint() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("fileChannel");
    return new ChannelEndpointAdapter(endpointConfiguration);
}

@Bean
public MessageChannel fileChannel() {
    return new DirectChannel();
}

@Bean
public FileWritingMessageHandler fileOutboundAdapter() {
    FileWritingMessageHandler messageHandler = new FileWritingMessageHandler(new
File("some/directory"));
    messageHandler.setOutboundChannel(fileChannel());
    return messageHandler;
}
```

Spring XML

```
<?xml version="1.0" encoding="UTF-8"?>
        <beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus="http://www.citrusframework.org/schema/config"
        xmlns:citrus-si="http://www.citrusframework.org/schema/spring-
integration/config"
        xmlns:si="http://www.springframework.org/schema/integration"
        xmlns:si-file="http://www.springframework.org/schema/integration/file"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/config
            http://www.citrusframework.org/schema/config/citrus-config.xsd
            http://www.citrusframework.org/schema/spring-integration/config
            http://www.citrusframework.org/schema/spring-integration/config/citrus-
spring-integration-config.xsd
            http://www.springframework.org/schema/integration
            http://www.springframework.org/schema/integration/spring-integration.xsd
            http://www.springframework.org/schema/integration/file
            http://www.springframework.org/schema/integration/file/spring-integration-
file.xsd">

    <citrus-si:channel-endpoint id="fileEndpoint" channel="fileChannel"/>

    <si-file:outbound-channel-adapter id="fileOutboundAdapter"
          channel="fileChannel"
          directory="file:some/directory"/>

    <si:channel id="fileChannel"/>
</beans>
```

The configuration above describes a Citrus message channel endpoint connected to a Spring Integration outbound file adapter that writes messages to a storage directory. With this combination you are able to write files to a directory in your Citrus test case. The test case uses the channel endpoint in its send action and the endpoint interacts with the Spring Integration file adapter so sending out the file.



The Spring Integration file adapter configuration component adds new namespaces (`si`, `si-file`, `citrus-si`) to the Spring XML application context. Please see the individual XML schema locations for details of the supported component in these namespaces.

23.2. Read files

The next program listing shows a possible inbound file communication. So the Spring Integration file inbound adapter will read files from a storage directory and publish the file contents to a message channel. Citrus can then receive those files as messages in a test case via the channel endpoint and validate the file contents for instance.

Citrus Bean

```
@BindToRegistry
public ChannelEndpointAdapter fileEndpoint() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("fileChannel");
    return new ChannelEndpointAdapter(endpointConfiguration);
}

@BindToRegistry
public MessageChannel fileChannel() {
    return new DirectChannel();
}

@BindToRegistry
public FileReadingMessageSource fileInboundAdapter() {
    FileReadingMessageSource messageSource = new FileReadingMessageSource();
    messageSource.setDirectory(new File("some/directory"));
    messageSource.setInboundChannel(fileChannel());
    return messageSource;
}
```

Spring Bean

```
@Bean
public ChannelEndpointAdapter fileEndpoint() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("fileChannel");
    return new ChannelEndpointAdapter(endpointConfiguration);
}

@Bean
public MessageChannel fileChannel() {
    return new DirectChannel();
}

@Bean
public FileReadingMessageSource fileInboundAdapter() {
    FileReadingMessageSource messageSource = new FileReadingMessageSource();
    messageSource.setDirectory(new File("some/directory"));
    messageSource.setInboundChannel(fileChannel());
    return messageSource;
}
```

Spring XML

```
<?xml version="1.0" encoding="UTF-8"?>
        <beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus="http://www.citrusframework.org/schema/config"
        xmlns:citrus-si="http://www.citrusframework.org/schema/spring-
integration/config"
        xmlns:si="http://www.springframework.org/schema/integration"
        xmlns:si-file="http://www.springframework.org/schema/integration/file"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/config
            http://www.citrusframework.org/schema/config/citrus-config.xsd
            http://www.citrusframework.org/schema/spring-integration/config
            http://www.citrusframework.org/schema/spring-integration/config/citrus-
spring-integration-config.xsd
            http://www.springframework.org/schema/integration
            http://www.springframework.org/schema/integration/spring-integration.xsd
            http://www.springframework.org/schema/integration/file
            http://www.springframework.org/schema/integration/file/spring-integration-
file.xsd">

  <citrus-si:channel-endpoint id="fileEndpoint" channel="fileChannel"/>

  <si-file:inbound-channel-adapter  id="fileInboundAdapter"
        channel="fileChannel"
        directory="file:some/directory">
    <si:poller fixed-rate="100"/>
  </si-file:inbound-channel-adapter>

  <si:channel id="fileChannel">
    <si:queue capacity="25"/>
    <si:interceptors>
        <bean
class="org.springframework.integration.transformer.MessageTransformingChannelIntercept
or">
            <constructor-arg>
                <bean
class="org.springframework.integration.file.transformer.FileToStringTransformer"/>
            </constructor-arg>
        </bean>
    </si:interceptors>
  </si:channel>

</beans>
```



The file inbound adapter constructs Java file objects as the message payload by default. Citrus can only work on String message payloads. So we need a file transformer that converts the file objects to String payloads representing the file's content.

This file adapter example shows how easy Citrus can work hand in hand with Spring Integration adapter implementations. The message channel support is a fantastic way to extend the transport and protocol support in Citrus by connecting with the very good Spring Integration adapter implementations. Have a closer look at the Spring Integration project for more details and other adapter implementations that you can use with Citrus integration testing.

Chapter 24. Selenium support

Selenium is a very popular tool for testing user interfaces with browser automation. Citrus is able to integrate with the Selenium Java API in order to execute Selenium commands.



The Selenium test components in Citrus are kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-selenium</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

24.1. Selenium browser

Selenium uses browser automation in order to simulate the user interact with web applications. You can configure the Selenium browser and web driver as Spring bean.

Citrus Bean

```
@BindToRegistry
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("firefox")
            .startPage("http://citrusframework.org")
            .build();
}
```

Spring Bean

```
@Bean
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("firefox")
            .startPage("http://citrusframework.org")
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-selenium="http://www.citrusframework.org/schema/selenium/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/selenium/config
           http://www.citrusframework.org/schema/selenium/config/citrus-selenium-
config.xsd">

  <citrus-selenium:browser id="seleniumBrowser"
                type="firefox"
                start-page="http://citrusframework.org"/>
</beans>
```



Citrus provides a "citrus-selenium" configuration namespace and schema definition for Selenium related Spring bean components and actions. Include this namespace into your Spring configuration in order to use the Citrus Selenium configuration elements. The namespace URI and schema location are added to the Spring configuration XML.

The Selenium browser component supports different browser types for the commonly used browsers out in the wild.

- **htmlunit**
- **firefox**
- **safari**
- **chrome**
- **googlechrome**
- **internet explorer**
- **edge**
- **custom**

Html unit is the default browser type and represents a headless browser that executed without displaying the graphical user interface. The optional **version** setting chooses which browser Html unit emulates: **FIREFOX** (default), **FIREFOX_ESR**, **CHROME** or **EDGE**.

In case you need a totally different browser or you need to customize the Selenium web driver you can use the *type="custom"* in combination with a web driver reference:

Citrus Bean

```
@BindToRegistry
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("custom")
            .webDriver(operaWebDriver())
            .build();
}

@BindToRegistry
public OperaDriver operaWebDriver() {
    return new OperaDriver();
}
```

Spring Bean

```
@Bean
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("custom")
            .webDriver(operaWebDriver())
            .build();
}

@Bean
public OperaDriver operaWebDriver() {
    return new OperaDriver();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-selenium="http://www.citrusframework.org/schema/selenium/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/selenium/config
           http://www.citrusframework.org/schema/selenium/config/citrus-selenium-
config.xsd">

  <citrus-selenium:browser id="seleniumBrowser"
               type="custom"
               web-driver="operaWebDriver"/>

  <bean id="operaWebDriver" class="org.openqa.selenium.opera.OperaDriver"/>
</beans>
```

Now Citrus is using the customized Selenium web driver implementation.



When using Firefox as browser you may also want to set the optional properties **firefox-profile** and **version**.

Citrus Bean

```
@BindToRegistry
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("firefox")
            .firefoxProfile(firefoxProfile())
            .version("FIREFOX_38")
            .startPage("http://citrusframework.org")
            .build();
}

@BindToRegistry
public FirefoxProfile firefoxProfile() {
    return new FirefoxProfile();
}
```

Spring Bean

```
@Bean
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("firefox")
            .firefoxProfile(firefoxProfile())
            .version("FIREFOX_38")
            .startPage("http://citrusframework.org")
            .build();
}

@Bean
public FirefoxProfile firefoxProfile() {
    return new FirefoxProfile();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-selenium="http://www.citrusframework.org/schema/selenium/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/selenium/config
           http://www.citrusframework.org/schema/selenium/config/citrus-selenium-
config.xsd">

  <citrus-selenium:browser id="mySeleniumBrowser"
               type="firefox"
               firefox-profile="firefoxProfile"
               version="FIREFOX_38"
               start-page="http://citrusframework.org"/>

  <bean id="firefoxProfile" class="org.openqa.selenium.firefox.FirefoxProfile"/>
</beans>
```

Now Citrus is able to execute Selenium operations as a user.

24.1.1. Browser options

The Selenium browser supports headless mode, custom browser arguments, a fixed window size as well as page load and script timeouts. Headless mode and browser arguments apply to Chrome, Edge and Firefox and are passed to the browser via its driver options. The window size and the timeouts are applied to the started web driver, so these settings work for any browser type.

Citrus Bean

```
@BindToRegistry
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
             .type("chrome")
             .headless(true)
             .argument("--disable-gpu")
             .argument("--no-sandbox")
             .windowSize("1280x800")
             .pageLoadTimeout(10000L)
             .scriptTimeout(5000L)
             .build();
}
```

Spring Bean

```
@Bean
public SeleniumBrowser seleniumBrowser() {
    return new SeleniumBrowserBuilder()
            .type("chrome")
            .headless(true)
            .argument("--disable-gpu")
            .argument("--no-sandbox")
            .windowSize("1280x800")
            .pageLoadTimeout(10000L)
            .scriptTimeout(5000L)
            .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-selenium="http://www.citrusframework.org/schema/selenium/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/selenium/config
           http://www.citrusframework.org/schema/selenium/config/citrus-selenium-
config.xsd">

  <citrus-selenium:browser id="seleniumBrowser"
                type="chrome"
                headless="true"
                arguments="--disable-gpu,--no-sandbox"
                window-size="1280x800"
                page-load-timeout="10000"
                script-timeout="5000"/>
</beans>
```

The browser arguments are comma-separated in the XML configuration. The window size uses the form `WIDTHxHEIGHT` (e.g. `1280x800`). Both timeouts are given in milliseconds.



Safari does not support headless mode or custom browser arguments. When these options are set on a Safari browser Citrus logs a warning and ignores the options. The window size and timeout settings work with Safari as usual.

24.2. Selenium actions

We have several Citrus test actions each representing a Selenium command. These actions can be part of a Citrus test case.

Java

```
@Autowired
private SeleniumBrowser seleniumBrowser;

@CitrusTest
public void seleniumTest() {
    selenium().start(seleniumBrowser);

    selenium().navigate("http://localhost:8080");

    selenium().find().element(By.id("header")
                .tagName("h1")
                .enabled(true)
                .displayed(true)
                .text("Welcome!")
                .style("font-size", "40px"));

    selenium().click().element(By.linkText("Click Me!"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <start browser="webBrowser"/>
      </selenium>

      <selenium>
        <navigate page="http://localhost:8080"/>
      </selenium>

      <selenium>
        <find>
          <element tag-name="h1"/>
          <validate text="Welcome!">
            <styles>
              <style name="font-size" value="40px"/>
            </styles>
          </validate>
        </find>
      </selenium>

      <selenium>
        <click>
          <element id="ok-button"/>
        </click>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      browser: webBrowser
      start: {}
  - selenium:
      navigate:
        page: "http://localhost:8080"
  - selenium:
      find:
        element:
          tag-name: "h1"
        validate:
          text: "Welcome!"
          styles:
            - name: "font-size"
              value: "40px"
  - selenium:
      click:
        element:
          id: "ok-button"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">

    <testcase name="SeleniumTest">
        <actions>
          <selenium:start browser="webBrowser"/>

          <selenium:navigate page="http://localhost:8080"/>

          <selenium:find>
            <selenium:element tag-name="h1" text="Welcome!">
              <selenium:styles>
                <selenium:style name="font-size" value="40px"/>
              </selenium:styles>
            </selenium:element>
          </selenium:find>

          <selenium:click>
            <selenium:element id="ok-button"/>
          </selenium:click>
        </actions>
    </testcase>
</spring:beans>
```



In Spring XML test cases there is special selenium namespace with prefix **selenium:** for special Selenium related test actions.

In this very simple example we first start the Selenium browser instance. After that we can continue to use Selenium commands without browser attribute explicitly set. Citrus knows which browser instance is currently active and will automatically use this opened browser instance. Next in this example we find some element on the displayed page by its tag-name and text. We also validate the element style *font-size* to meet the expected value *40px* in this step.

In addition to that the example performs a click operation on the element with the id *ok-button*. Selenium supports element find operations on different properties:

id finds element based on the *id* attribute

name	finds element based on the *name* attribute
tag-name	finds element based on the *tag name*
class-name	finds element based on the css *class name*
link-text	finds link element based on the *link-text*
xpath	finds element based on XPath evaluation in the DOM

Based on that we can execute several Selenium commands in a test case and validate the results such as web elements. Citrus supports the following Selenium commands with respective test actions:

selenium:start	Start the browser instance
selenium:find	Finds element on current page and validates element properties
selenium:verify	Verifies page url, title, element state or value on the current page
selenium:extract	Extracts page url, title, element state or value into a test variable
selenium:click	Performs click operation on element
selenium:hover	Performs hover operation on element
selenium:double-click	Performs double click operation on element
selenium:right-click	Performs right click (context click) operation on element
selenium:drag-and-drop	Drags a source element and drops it onto a target element
selenium:press	Presses one key or a key combination on an element or the focused element
selenium:navigate	Navigates to new page url (including history back, forward and refresh)
selenium:set-input	Finds input element and sets value
selenium:check-input	Finds checkbox element and sets/unsets value

selenium:dropdown-select	Finds dropdown element and selects or deselects single or multiple options by value, visible text or index
selenium:fill-form	Fills form with given values and optionally submits form afterwards. Supports setting multiple fields at the same time.
selenium:page	Instantiate page object with dependency injection and execute page action with verification
selenium:open	Open new window
selenium:close	Close window by given name
selenium:switch	Switch focus to window with given name
selenium:frame	Switch focus to frame by element, index or name, back to parent or default content
selenium:wait-until	Wait for element presence, visibility, clickability, text, url or title
selenium:alert	Access current alert dialog (with action *access* or *dismiss*)
selenium:screenshot	Makes screenshot of current page
selenium:store-file	Store file to temporary browser directory
selenium:get-stored-file	Gets stored file from temporary browser directory
selenium:javascript	Execute Javascript code in browser
selenium:cookies	Add, read, verify or delete browser cookies
selenium:clear-cache	Clears cookies and web storage in current browser
selenium:stop	Stops the browser instance

Now let's have a closer look at the different Selenium test actions supported in Citrus.

24.3. Start/stop browser

You can start and stop the browser instance with a test action. This instantiates a new browser window and prepares everything for interacting with the web interface.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().start(seleniumBrowser);
    // Do something in browser
    selenium().stop(seleniumBrowser);
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <start browser="seleniumBrowser"/>
      </selenium>
      <!-- Do something in browser -->
      <selenium>
        <stop browser="seleniumBrowser"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      browser: seleniumBrowser
      start: {}
  # Do something in browser
  - selenium:
      browser: seleniumBrowser
      stop: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
      <selenium:start browser="seleniumBrowser"/>
      <!-- Do something in browser -->
      <selenium:stop browser="seleniumBrowser"/>
    </actions>
  </testcase>
</spring:beans>
```

After starting a browser instance Citrus will automatically use this very same browser instance in all further Selenium actions. This mechanism is based on a test variable (**selenium_browser**) that is automatically set. All other test actions are able to load the current browser instance by reading this test variable before execution. In case you need to explicitly use a different browser instance than the active instance you can add the **browser** attribute to all Selenium test actions.



It is a good idea to start and stop the browser instance before each test case. This makes sure that tests are also executable in single run and it always sets up a new browser instance so tests will not influence each other.

24.4. Find

The find element test action searches for an element on the current page. The element is specified by one of the following settings:

id	finds element based on the *id* attribute
name	finds element based on the *name* attribute
tag-name	finds element based on the *tag name*
class-name	finds element based on the css *class name*
link-text	finds link element based on the *link-text*

xpath	finds element based on XPath evaluation in the DOM

The find element action will automatically fail in case there is no such element on the current page. In case the element is found you can add additional attributes and properties for further element validation:

The element selector and the expected tag name are independent settings. The selector (one of *id*, *name*, *tag-name*, *class-name*, *link-text* or *xpath*) decides which element is looked up on the page, while the *tagName* / *validate-tag-name* setting asserts the tag name of the element that has been found. In Java you use the `tagName` method on the find builder, in the Spring XML configuration you set the `validate-tag-name` attribute on the `selenium:element` (for example `<selenium:element id="ok-button" validate-tag-name="button"/>`).



In earlier versions the `tag-name` attribute of the Spring XML `selenium:element` also acted as the expected tag name when combined with another selector (for example `<selenium:element class-name="clickable" tag-name="button"/>`). That dual meaning no longer applies: `tag-name` is only a selector now, so move the expected tag name to `validate-tag-name` to keep the assertion (for example `<selenium:element class-name="clickable" validate-tag-name="button"/>`). Without this change the tag name is no longer validated.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().find().element(By.tagName("h1"))
        .text("Welcome!")
        .style("font-size", "40px");

    selenium().find().element(By.id("ok-button"))
        .tagName("button")
        .enabled(true)
        .displayed(true)
        .text("Ok")
        .style("color", "red")
        .attribute("type", "submit");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <find>
          <element tag-name="h1"/>
          <validate text="Welcome!">
            <styles>
              <style name="font-size" value="40px"/>
            </styles>
          </validate>
        </find>
      </selenium>

      <selenium>
        <find>
          <element id="ok-button"/>
          <validate text="Ok" enabled="true" displayed="true">
            <attributes>
              <attribute name="type" value="submit"/>
            </attributes>
          </validate>
        </find>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      find:
        element:
          tag-name: "h1"
        validate:
          text: "Welcome!"
          styles:
            - name: "font-size"
              value: "40px"
  - selenium:
      find:
        element:
          id: "ok-button"
        validate:
          text: "Ok"
          displayed: true
          enabled: true
          attributes:
            - name: "type"
              value: "submit"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:find>
          <selenium:element tag-name="h1" text="Welcome!">
            <selenium:styles>
              <selenium:style name="font-size" value="40px"/>
            </selenium:styles>
          </selenium:element>
        </selenium:find>

        <selenium:find>
          <selenium:element id="ok-button" text="Ok" enabled="true" displayed="true">
            <selenium:attributes>
              <selenium:attribute name="type" value="submit"/>
            </selenium:attributes>
          </selenium:element>
        </selenium:find>
    </actions>
  </testcase>
</spring:beans>
```

The example above finds the **h1** element by its tag name and validates the text and css style attributes. Secondly the **ok-button** is validated with expected enabled, displayed, text, style and attribute values. The elements must be present on the current page and all expected element properties have to match. Otherwise the test action and the test case is failing with validation errors.

24.5. Verify

The verify action checks the current page or a web element. In contrast to the find action the verify action focuses on a single check that is selected with the **check** setting. The expected value supports Citrus validation matchers (e.g. `@endsWith('index.html')@`).

Supported checks are:

url	current page url matches the expected value (requires the *expected* setting)

title	current page title matches the expected value (requires the *expected* setting)
present	element is present in the DOM (displayed or not)
absent	element is not present in the DOM
displayed	element is present in the DOM and displayed
hidden	element is not displayed or not present at all
enabled	element is enabled
disabled	element is disabled
selected	element is selected (e.g. a checkbox or radio button)
not-selected	element is not selected
text	element text matches the expected value (requires the *expected* setting)
value	element value matches the expected value (requires the *expected* setting)
attribute	element attribute matches the expected value (requires the *attribute* and *expected* settings)
css-value	element css style property matches the expected value (requires the *css-property* and *expected* settings)
count	number of elements that match the element selector matches the expected value (requires the *expected* setting)

The element to check is selected with one of the usual element selectors. The *url* and *title* checks do not require an element at all. Every failing check reports what has been checked, the expected value and the actual value.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().verify()
            .url("@endsWith('/index.html')@");

    selenium().verify()
            .title("Citrus");

    selenium().verify()
            .text("Welcome!")
            .element(By.id("header"));

    selenium().verify()
            .attribute("href", "https://citrusframework.org")
            .element(By.id("docs"));

    selenium().verify()
            .count("3")
            .element(By.className("order"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <verify check="url" expected="@endsWith('/index.html')@"/>
      </selenium>

      <selenium>
        <verify check="title" expected="Citrus"/>
      </selenium>

      <selenium>
        <verify check="text" expected="Welcome!">
          <element id="header"/>
        </verify>
      </selenium>

      <selenium>
        <verify check="attribute" attribute="href"
expected="https://citrusframework.org">
          <element id="docs"/>
        </verify>
      </selenium>

      <selenium>
        <verify check="count" expected="3">
          <element class-name="order"/>
        </verify>
      </selenium>
    </actions>
</test>
```

YAML

```yaml
name: SeleniumTest
actions:
  - selenium:
      verify:
        check: "url"
        expected: "@endsWith('/index.html')@"
  - selenium:
      verify:
        check: "text"
        expected: "Welcome!"
        element:
          id: "header"
  - selenium:
      verify:
        check: "attribute"
        attribute: "href"
        expected: "https://citrusframework.org"
        element:
          id: "docs"
  - selenium:
      verify:
        check: "count"
        expected: "3"
        element:
          class-name: "order"
```

24.6. Extract

The extract action reads the current page or a web element and stores the value in a test variable. The variable is then available in the test context with the usual Citrus variable syntax `${variable-name}`.

Supported values are:

url	current page url
title	current page title
page-source	current page source
text	element text
value	element value
attribute	element attribute value (requires the *attribute* setting)

css-value	element css style property value (requires the *css-property* setting)
count	number of elements that match the element selector

The element to extract from is selected with one of the usual element selectors. The *url*, *title* and *page-source* values do not require an element at all.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().extract()
            .title()
            .variable("pageTitle");

    selenium().extract()
            .text()
            .variable("headerText")
            .element(By.id("header"));

    selenium().extract()
            .count()
            .variable("orderCount")
            .element(By.className("order"));

    selenium().extract()
            .attribute("href")
            .variable("docsHref")
            .element(By.id("docs"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <extract value="title" variable="pageTitle"/>
      </selenium>

      <selenium>
        <extract value="text" variable="headerText">
          <element id="header"/>
        </extract>
      </selenium>

      <selenium>
        <extract value="count" variable="orderCount">
          <element class-name="order"/>
        </extract>
      </selenium>

      <selenium>
        <extract value="attribute" attribute="href" variable="docsHref">
          <element id="docs"/>
        </extract>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      extract:
        value: "title"
        variable: "pageTitle"
  - selenium:
      extract:
        value: "text"
        variable: "headerText"
        element:
          id: "header"
  - selenium:
      extract:
        value: "count"
        variable: "orderCount"
        element:
          class-name: "order"
  - selenium:
      extract:
        value: "attribute"
        attribute: "href"
        variable: "docsHref"
        element:
          id: "docs"
```



In XML and YAML test cases the element selectors resolve test variables, so a dynamic locator value such as `<element id="${element-id}"/>` uses the current value of the variable when the action is executed.

24.7. Click

The action performs a click operation on the element.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().click().element(By.linkText("Click Me!"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <click>
          <element>
            <property name="link-text" value="Click Me!"/>
          </element>
        </click>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      click:
        element:
          property:
            name: "link-text"
            value: "Click Me!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:click>
          <selenium:element link-text="Click Me!"/>
        </selenium:click>
    </actions>
  </testcase>
</spring:beans>
```

24.8. Hover

The action performs a hover operation on the element.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().hover().element(By.linkText("Find Me!"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <hover>
          <element>
            <property name="link-text" value="Find Me!"/>
          </element>
        </hover>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      hover:
        element:
          property:
            name: "link-text"
            value: "Find Me!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:hover>
          <selenium:element link-text="Find Me!"/>
        </selenium:hover>
    </actions>
  </testcase>
</spring:beans>
```

24.9. Double click

The action performs a double click operation on the element.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().doubleClick().element(By.id("double-click-me"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <double-click>
          <element id="double-click-me"/>
        </double-click>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      double-click:
        element:
          id: "double-click-me"
```

24.10. Right click

The action performs a right click (context click) operation on the element.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().rightClick().element(By.id("right-click-me"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <right-click>
          <element id="right-click-me"/>
        </right-click>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      right-click:
        element:
          id: "right-click-me"
```

24.11. Drag and drop

The action drags a source element and drops it onto a target element. The source element is set on the action and the target element is set as a separate target setting.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().dragAndDrop()
            .element(By.id("drag-source"))
            .target(By.id("drop-target"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <drag-and-drop>
          <element id="drag-source"/>
          <target id="drop-target"/>
        </drag-and-drop>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      drag-and-drop:
        element:
          id: "drag-source"
        target:
          id: "drop-target"
```



The drag and drop action uses the WebDriver mouse actions. This does not fire the native HTML5 drag events (*dragstart*, *drag*, *drop*) in the browser. Applications that rely on these events need to use Javascript actions or a dedicated WebDriver implementation.

24.12. Press keys

The action presses a single key or a combination of keys. The keys are sent to a located element or to the currently focused element when no element is set on the action.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().press("ENTER")
            .element(By.id("username"));

    selenium().press("CONTROL+A");

    selenium().press()
            .keys("TAB");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <press keys="ENTER">
          <element id="username"/>
        </press>
      </selenium>

      <selenium>
        <press keys="TAB"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      press:
        keys: "ENTER"
        element:
          id: "username"
  - selenium:
      press:
        keys: "TAB"
```

Use the Selenium key names (e.g. *ENTER*, *TAB*, *ESCAPE*, *ARROW_DOWN*) for special keys and a single character for regular text. Keys are combined with the `+` character (e.g. `CONTROL+A`). An unknown key name raises an error naming the key.

24.13. Form input actions

The following actions are used to access form input elements such as text fields, checkboxes and dropdown lists.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().setInput("Citrus")
            .element(By.name("username"));

    selenium().checkInput(true)
            .element(By.xpath("//input[@type='checkbox']"));

    selenium().select("happy")
            .element(By.id("user-mood"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <set-input value="Citrus">
          <element name="username"/>
        </set-input>
      </selenium>

      <selenium>
        <check-input checked="true">
          <element xpath="//input[@type='checkbox']"/>
        </set-input>
      </selenium>

      <selenium>
        <dropdown-select option="happy">
          <element id="user-mood"/>
        </dropdown-select>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      set-input:
        value: "Citrus"
        element:
          name: "username"
  - selenium:
      check-input:
        checked: true
        element:
          xpath: |
            xpath="//input[@type='checkbox']"
  - selenium:
      dropdown-select:
        option: "happy"
        element:
          id: "user-mood"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:set-input value="Citrus">
          <selenium:element name="username"/>
        </selenium:set-input>

        <selenium:check-input checked="true">
          <selenium:element xpath="//input[@type='checkbox']"/>
        </selenium:check-input>

        <selenium:dropdown-select option="happy">
          <selenium:element id="user-mood"/>
        </selenium:dropdown-select>
    </actions>
  </testcase>
</spring:beans>
```

The actions above select dropdown options and set user input on text fields and checkboxes. As usual the form elements are selected by some properties such as ids, names or xpath expressions.

When selecting multiple dropdown options Citrus holds the platform specific modifier key: **Command** on macOS and **Control** on all other operating systems. So multiple selection behaves the same way on every platform.

The dropdown select action selects options by their value by default. Use the `by` setting to select options by their visible `text` or by their zero based `index` instead. The same setting applies when deselecting options. Use the deselect all setting to unselect all options at once or provide a list of deselect options in order to unselect a set of options.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().select("Germany")
            .byText()
            .element(By.id("countries"));

    selenium().select("1")
            .byIndex()
            .element(By.id("countries"));

    selenium().select()
            .deselectAll()
            .element(By.id("languages"));

    selenium().select()
            .byText()
            .deselect("German")
            .element(By.id("languages"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <dropdown-select option="Germany" by="text">
          <element id="countries"/>
        </dropdown-select>
      </selenium>

      <selenium>
        <dropdown-select option="1" by="index">
          <element id="countries"/>
        </dropdown-select>
      </selenium>

      <selenium>
        <dropdown-select deselect-all="true">
          <element id="languages"/>
        </dropdown-select>
      </selenium>

      <selenium>
        <dropdown-select by="text">
          <element id="languages"/>
          <deselect-options>
            <option>German</option>
          </deselect-options>
        </dropdown-select>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      dropdown-select:
        option: "Germany"
        by: "text"
        element:
          id: "countries"
  - selenium:
      dropdown-select:
        option: "1"
        by: "index"
        element:
          id: "countries"
  - selenium:
      dropdown-select:
        deselect-all: true
        element:
          id: "languages"
  - selenium:
      dropdown-select:
        by: "text"
        element:
          id: "languages"
        deselect-options: ["German"]
```

24.14. Fill form

The fill form action sets multiple form fields with given values and optionally submits the form afterwards.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().fillForm()
        .field("username", "foo_user")
        .field("password", "${secret}")
        .submit("save");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <fill-form submit="save">
          <fields>
            <field id="username" value="foo_user"/>
            <field id="password" value="${secret}"/>
          </fields>
        </fill-form>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      fill-form:
        submit: "save"
        fields:
          - id: "username"
            value: "foo_user"
          - id: "password"
            value: "${secret}"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/selenium/testcase
              http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:fill-form submit="save">
          <selenium:fields>
            <selenium:field id="username" value="foo_user"/>
            <selenium:field id="password" value="${secret}"/>
          </selenium:fields>
        </selenium:fill-form>
    </actions>
  </testcase>
</spring:beans>
```

The actions above set the form fields identified by its id and sets the given values. The actions supports input fields, checkboxes and drop-down select controls. Optionally the form gets submitted by clicking on the given submit button once all fields are set.

The user also has the opportunity to set the form fields via Json object where each property in the Json object represents a form field.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().fillForm()
        .fromJson("""
        {
          "username": "foo_user",
          "password": "secret"
        }
        """)
        .submit(By.id("save"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <fill-form submit="save">
          <json>
          {
            "username": "foo_user",
            "password": "secret"
          }
          </json>
        </fill-form>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      fill-form:
        submit: "save"
        json: |
          {
            "username": "foo_user",
            "password": "secret"
          }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:fill-form submit="save">
            <selenium:json>
              {
                "username": "foo_user",
                "password": "secret"
              }
            </selenium:json>
        </selenium:fill-form>
    </actions>
  </testcase>
</spring:beans>
```

This will set the form fields `username` and `password` with the given values before submitting the form.

24.15. Page actions

Page objects are a well known pattern when using Selenium. The page objects define elements that the page is working with. In addition to that the page objects define actions that can be executed from outside. This object-oriented approach for accessing pages and their elements is a very good idea. Let's have a look at a sample page object.

```
public class UserFormPage implements WebPage {

    @FindBy(id = "userForm")
    private WebElement form;

    @FindBy(id = "username")
    private WebElement userName;

    /**
     * Sets the username.
     */
    public void setUserName(String value, TestContext context) {
        userName.clear();
        userName.sendKeys(value);
    }

    /**
     * Submits the form.
     */
    public void submit(TestContext context) {
        form.submit();
    }
}
```

As you can see the page object is a Java POJO that implements the **WebPage** interface. The page defines **WebElement** members. These are automatically injected by Citrus and Selenium based on the **FindBy** annotation. Now the test case is able to load that page object and execute some action methods on the page such as *setUserName* or *submit*.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().page(UserFormPage.class)
            .argument("Citrus")
            .execute("setUserName");

    selenium().page(UserFormPage.class)
            .execute("submit");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <page action="setUserName"
              type="org.citrusframework.selenium.pages.UserFormPage">
          <arguments>
            <argument>Citrus</argument>
          </arguments>
        </page>
      </selenium>
      <selenium>
        <page action="submit"
              type="org.citrusframework.selenium.pages.UserFormPage"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      page:
        type: "org.citrusframework.selenium.pages.UserFormPage"
        action: "setUserName"
        argument: Citrus
  - selenium:
      page:
        type: "org.citrusframework.selenium.pages.UserFormPage"
        action: "submit"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:page action="setUserName"
            type="org.citrusframework.selenium.pages.UserFormPage">
          <selenium:arguments>
            <selenium:argument>Citrus</selenium:argument>
          </selenium:arguments>
        </selenium:page>

        <selenium:page action="submit"
            type="org.citrusframework.selenium.pages.UserFormPage"/>
    </actions>
  </testcase>
</spring:beans>
```

The page object class is automatically loaded and instantiated with dependency injection for all *FindBy* annotated web elements. After that the action method is executed. The action methods can also have method parameters as seen in *setUserName*. The value parameter is automatically set when calling the method.

Methods can also use the optional parameter *TestContext*. With this context you can access the current test context with all test variables for instance. This method parameter should always be the last parameter.

24.16. Page validation

We can also use page object for validation purpose. The page object is loaded and instantiated as described in previous section. Then the page validator is called. The validator performs assertions and validation operations with the page object. Let’s see a sample page validator:

```
public class UserFormValidator implements PageValidator<UserFormPage> {

    @Override
    public void validate(UserFormPage webPage, SeleniumBrowser browser, TestContext
context) {
        Assert.isTrue(webPage.getUserName() != null);

Assert.isTrue(StringUtils.hasText(webPage.getUserName().getAttribute("value")));
    }
}
```

The page validator is called with the web page instance, the browser and the test context. The validator should assert page objects and web elements for validation purpose. In a test case we can call the validator to validate the page.

Java

```
@Autowired
private UserFormValidator userFormValidator;

@CitrusTest
public void seleniumTest() {
    selenium().page(UserFormPage.class)
            .execute("validate")
            .validator(userFormValidator);
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <page action="validate"
              validator="userFormValidator"
              type="org.citrusframework.selenium.pages.UserFormPage"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      page:
        type: "org.citrusframework.selenium.pages.UserFormPage"
        action: "validate"
        validator: "userFormValidator"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <spring:bean id ="userFormValidator"
class="org.citrusframework.selenium.pages.UserFormValidator"/>

  <testcase name="SeleniumTest">
    <actions>
        <selenium:page type="org.citrusframework.selenium.pages.UserFormPage"
               action="validate"
               validator="userFormValidator"/>
    </actions>
  </testcase>
</spring:beans>
```

Instead of using a separate validator class you can also put the validation method to the page object itself. Then page object and validation is done within the same class:

```
public class UserFormPage implements WebPage, PageValidator<UserFormPage> {

    @FindBy(id = "userForm")
    private WebElement form;

    @FindBy(id = "username")
    private WebElement userName;

    /**
     * Sets the username.
     */
    public void setUserName(String value, TestContext context) {
        userName.clear();
        userName.sendKeys(value);
    }

    /**
     * Submits the form.
     */
    public void submit(TestContext context) {
        form.submit();
    }

    @Override
    public void validate(UserFormPage webPage, SeleniumBrowser browser, TestContext
context) {
        Assert.isTrue(userName != null);
        Assert.isTrue(StringUtils.hasText(userName.getAttribute("value")));
        Assert.isTrue(form != null);
    }
}
```

24.17. Wait

Sometimes it is required to wait for an element to appear or disappear on the current page. The wait action will wait a given time for the element status to meet a condition such as *visible*, *hidden*, *present*, *absent* or *clickable*. The action also supports waiting for the element text, the current page url or the page title.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().waitUntil()
            .hidden()
            .element(By.id("info-dialog"));
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <wait until="hidden">
          <element id="info-dialog"/>
        </wait>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      wait-until:
        condition: "hidden"
        element:
          id: "info-dialog"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:wait until="hidden">
            <selenium:element id="info-dialog"/>
        </selenium:wait>
    </actions>
  </testcase>
</spring:beans>
```

The example waits for the element *info-dialog* to disappear. The time to wait is 5000 milliseconds by default. You can set the timeout on the action. Due to Selenium limitations the minimum wait time is 1000 milliseconds.

The wait action supports the following conditions:

visible	element is present in the DOM and displayed
hidden	element is not displayed or not present at all
present	element is present in the DOM (displayed or not)
absent	element is not present in the DOM
clickable	element is displayed and enabled
text	element text contains the expected value (requires the *expected* setting)
url	current page url contains the expected value (requires the *expected* setting)
title	current page title contains the expected value (requires the *expected* setting)

The element is looked up while waiting, so elements that appear later on the page are found as well. The *url* and *title* conditions do not require an element at all. An unknown condition raises an error that lists the supported conditions.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().waitUntil()
            .clickable()
            .element(By.id("submit-button"))
            .timeout(10000L);

    selenium().waitUntil()
            .text("Ok")
            .element(By.id("dialog"));

    selenium().waitUntil()
            .url("citrusframework.org");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <wait until="clickable" timeout="10000">
          <element id="submit-button"/>
        </wait>
      </selenium>

      <selenium>
        <wait until="text" expected="Ok">
          <element id="dialog"/>
        </wait>
      </selenium>

      <selenium>
        <wait until="url" expected="citrusframework.org"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      wait-until:
        condition: "clickable"
        timeout: 10000
        element:
          id: "submit-button"
  - selenium:
      wait-until:
        condition: "text"
        expected: "Ok"
        element:
          id: "dialog"
  - selenium:
      wait-until:
        condition: "url"
        expected: "citrusframework.org"
```

24.18. Navigate

The action navigates to a new page either by using a new relative path or a complete new Http URL.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().navigate("http://localhost:8080");

    selenium().navigate("help");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <navigate page="http://localhost:8080"/>
      </selenium>

      <selenium>
        <navigate page="help"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      navigate:
        page: "http://localhost:8080"
  - selenium:
      navigate:
        page: "help"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:navigate page="http://localhost:8080"/>

        <selenium:navigate page="help"/>
    </actions>
  </testcase>
</spring:beans>
```

The sample above describes a new page with new Http URL. The browser will navigate to this new page. All further Selenium actions are performed on this new page. The second navigation action opens the relative page *help* so the new page URL is `http://localhost:8080/help`.

Navigation is always done on the active browser window. You can manage the opened windows as described in next section.

24.19. Window actions

Selenium is able to manage multiple windows. So you can open, close and switch active windows in a Citrus test.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().open().window("my_window");
    selenium().focus().window("my_window");
    selenium().close().window("my_window");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <open-window name="my_window"/>
        <switch-window name="my_window"/>
        <close-window name="my_window"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      open-window:
        name: "my_window"
  - selenium:
      switch-window:
        name: "my_window"
  - selenium:
      close-window:
        name: "my_window"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:open-window name="my_window"/>
        <selenium:switch-window name="my_window"/>
        <selenium:close-window name="my_window"/>
    </actions>
  </testcase>
</spring:beans>
```

When a new window is opened Selenium creates a window handle for us. This window handle is saved as test variable using a given window name. So after opening the window you can access the

window by its name in further actions. All upcoming Selenium actions will take place in this new active window. Of course the test actions will fail as soon as the window with that given name is missing. Citrus uses default window names that are automatically used as test variables:

selenium_active_window	the active window handle
selenium_last_window	the last window handle when switched to other window

24.20. Frame

Web pages may embed other pages in frames. The frame action switches the focus to a frame so that all upcoming Selenium actions take place inside that frame. Switch back to the parent frame or to the default (top-level) content when done.

The frame is located by a web element, by its zero-based index or by its name or id attribute. Exactly one target must be set on the action.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().frame()
            .element(By.id("menu-frame"));

    selenium().frame()
            .index(0);

    selenium().frame()
            .nameOrId("menu-frame");

    selenium().frame()
            .parent();

    selenium().frame()
            .defaultContent();
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <frame>
          <element id="menu-frame"/>
        </frame>
      </selenium>

      <selenium>
        <frame index="0"/>
      </selenium>

      <selenium>
        <frame name="menu-frame"/>
      </selenium>

      <selenium>
        <frame parent="true"/>
      </selenium>

      <selenium>
        <frame default="true"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      frame:
        element:
          id: "menu-frame"
  - selenium:
      frame:
        index: 0
  - selenium:
      frame:
        name: "menu-frame"
  - selenium:
      frame:
        parent: true
  - selenium:
      frame:
        default: true
```

In case the frame element or the frame with the given index, name or id cannot be found the test

action fails with an error. The action also fails when no frame target has been set at all.

24.21. Alert

We are able to access the alert dialog on the current page. Citrus will validate the displayed dialog text and accept or dismiss of the dialog.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().alert()
                .text("Hello!")
                .accept();
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <alert text="Hello!" accept="true"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      alert:
        accept: true
        text: "Hello!"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:alert accept="true">
          <selenium:alert-text>Hello!</selenium:alert-text>
        </selenium:alert>
    </actions>
  </testcase>
</spring:beans>
```

The alert dialog text is validated when expected text is given on the test action. The user can decide to accept or dismiss the dialog. After that the dialog should be closed. In case the test action fails to find an open alert dialog the test action raises runtime errors and the test will fail.

24.22. Make screenshot

You can execute this action in case you want to take a screenshot of the current page. This action only works with browsers that actually display the user interface. The action will not have any effect when executed with Html unit web driver in headless mode.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().screenhsot();

    selenium().screenhsot("target");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <screenshot/>
      </selenium>

      <selenium>
        <screenshot output-dir="target"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      screenshot: {}
  - selenium:
      screenshot:
        output-dir: "target"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:screenshot/>

        <selenium:screenshot output-dir="target"/>
    </actions>
  </testcase>
</spring:beans>
```

The test action has an optional parameter *output-dir* which represents the output directory where the screenshot is saved to.

24.23. Temporary storage (Firefox)

Important This action only works with Firefox web driver! Other browsers are not working with the temporary download storage.

The browser uses a temporary storage for downloaded files. We can access this temporary storage during a test case.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().store("classpath:download/file.txt");
    selenium().getStored("file.txt");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <store-file file-path="classpath:download/file.txt"/>
        <get-stored-file file-name="file.txt"/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      store-file:
        file-path: "classpath:download/file.txt"
  - selenium:
      get-stored-file:
        file-name: "file.txt"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:store-file file-path="classpath:download/file.txt"/>
        <selenium:get-stored-file file-name="file.txt"/>
    </actions>
  </testcase>
</spring:beans>
```

As you can see the test case is able to store new files to the temporary browser storage. We have to give the file path as classpath or file system path. When reading the temporary file storage we need to specify the file name that we want to access in the temporary storage. The temporary storage is not capable of subdirectories all files are stored directly to the storage in one single directory.

In case the stored file is not found by that name the test action fails with respective errors. On the other hand when the file is found in temporary storage Citrus will automatically create a new test variable **selenium_download_file** which contains the file name as value.

24.24. Javascript

The javascript action executes a given piece of Javascript code in the current browser. Optionally the action validates that the browser reported expected Javascript errors while the script was running.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().javascript("alert('This is awesome!')")
        .errors("Something went wrong");

    selenium().javascript("return document.title;")
        .variable("pageTitle");
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <javascript>
          <script>alert('This is awesome!')</script>
          <errors>
            <error>Something went wrong</error>
          </errors>
        </javascript>
      </selenium>

      <selenium>
        <javascript variable="pageTitle">
          <script>return document.title;</script>
        </javascript>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      javaScript:
        script: "alert('This is awesome!')"
        errors:
          - Something went wrong
  - selenium:
      javaScript:
        script: "return document.title;"
        variable: "pageTitle"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:javascript>
          <selenium:script>alert('This is awesome!')</selenium:script>
          <selenium:errors>
            <selenium:error>Something went wrong</selenium:error>
          </selenium:errors>
        </selenium:javascript>
        <selenium:javascript variable="pageTitle">
          <selenium:script>return document.title;</selenium:script>
        </selenium:javascript>
    </actions>
  </testcase>
</spring:beans>
```

Before executing the script Citrus installs a small error collector in the current page. Any Javascript error raised while the script is running is collected and exposed to the test as the **selenium_jsErrors** test variable. When the action declares expected errors with the *errors* setting the test automatically fails in case one of these expected errors has not been raised by the browser.

When the action declares a *variable* name the script result is stored as a test variable. A script result of `null` or `undefined` is stored as an empty string.

24.25. Cookies

The cookies action manages the browser cookies of the current session. It adds a cookie, reads a cookie value into a test variable, verifies a cookie value or presence, and deletes single cookies or all cookies.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().cookies()
            .add("session", "citrus")
            .path("/")
            .httpOnly(true);

    selenium().cookies()
            .read("session", "sessionValue");

    selenium().cookies()
            .verify("session", "citrus");

    selenium().cookies()
            .verifyAbsent("unknown-cookie");

    selenium().cookies()
            .delete("session");

    selenium().cookies()
            .deleteAll();
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <cookies command="add" name="session" value="citrus" path="/" http-
only="true"/>
      </selenium>

      <selenium>
        <cookies command="read" name="session" variable="sessionValue"/>
      </selenium>

      <selenium>
        <cookies command="verify" name="session" expected="citrus"/>
      </selenium>

      <selenium>
        <cookies command="verify-absent" name="unknown-cookie"/>
      </selenium>

      <selenium>
        <cookies command="delete" name="session"/>
      </selenium>

      <selenium>
        <cookies command="delete-all"/>
      </selenium>
    </actions>
</test>
```

YAML

```yaml
name: SeleniumTest
actions:
  - selenium:
      cookies:
        command: "add"
        name: "session"
        value: "citrus"
        path: "/"
        http-only: true
  - selenium:
      cookies:
        command: "read"
        name: "session"
        variable: "sessionValue"
  - selenium:
      cookies:
        command: "verify"
        name: "session"
        expected: "citrus"
  - selenium:
      cookies:
        command: "verify-absent"
        name: "unknown-cookie"
  - selenium:
      cookies:
        command: "delete"
        name: "session"
  - selenium:
      cookies:
        command: "delete-all"
```

When adding a cookie the optional *domain*, *path*, *expiry* (epoch seconds), *secure*, *http-only* and *same-site* settings control the cookie attributes. The verify command checks the cookie value and supports validation matchers (e.g. `@contains('citrus')@`). Reading or verifying a missing cookie fails the test with an error naming the cookie.

24.26. Clear browser cache

Clearing the browser cache deletes all cookies of the current session and clears the browser's local and session web storage for the current origin. Please note that the browser HTTP cache cannot be cleared through the WebDriver API, so previously cached responses may still be served.

Java

```
@CitrusTest
public void seleniumTest() {
    selenium().clearCache();
}
```

XML

```
<test name="SeleniumTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <selenium>
        <clear-cache/>
      </selenium>
    </actions>
</test>
```

YAML

```
name: SeleniumTest
actions:
  - selenium:
      clear-cache: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:selenium="http://www.citrusframework.org/schema/selenium/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/schema/selenium/testcase
          http://www.citrusframework.org/schema/selenium/testcase/citrus-selenium-
testcase.xsd">
  <testcase name="SeleniumTest">
    <actions>
        <selenium:clear-cache/>
    </actions>
  </testcase>
</spring:beans>
```

Chapter 25. Playwright support

Playwright enables reliable end-to-end testing for modern web applications. Citrus integrates with the Playwright Java API in order to drive real browsers (Chromium, Firefox, WebKit) from Citrus tests: contexts and pages, locators, dialogs and downloads, storage state, tracing, and console/network capture.



The Playwright connector in Citrus is kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-playwright</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The connector is Spring-free and only requires `citrus-api`, `citrus-base` and the Playwright driver.

25.1. Playwright browser

The `PlaywrightBrowser` endpoint manages the Playwright runtime, browser launch, default context and observability settings. You can configure it as a Citrus bean.

Citrus Bean

```
@BindToRegistry
public PlaywrightBrowser playwrightBrowser() {
    return PlaywrightEndpoints.playwright()
            .browser()
            .browserType("chromium")
            .headless(true)
            .baseUrl("http://localhost:8080")
            .build();
}
```

Spring Bean

```
@Bean
public PlaywrightBrowser playwrightBrowser() {
    return PlaywrightEndpoints.playwright()
            .browser()
            .browserType("chromium")
            .headless(true)
            .baseUrl("http://localhost:8080")
            .build();
}
```

Redaction is configured entirely through the Citrus-wide log masking settings: name patterns come from `citrus.logger.mask.keywords` (default `password,secret,secretKey` - add browser-relevant entries such as `authorization`, `token`, or `cookie` when relying on defaults) and the mask value follows `citrus.logger.mask.value`. Endpoint-specific `secretPatterns(…)` extend the defaults per browser and additionally scrub their literal occurrences from free text.

Remote browsers are supported via WebSocket or CDP endpoints (`connectWsEndpoint(…)` / `connectOverCdpEndpoint(…)`), and extra secret name/value patterns can be supplied with `secretPatterns(…)` so diagnostics redact sensitive values.

25.2. Browser annotation config

When using `@CitrusEndpoint` annotations on test fields the browser is configured with `@PlaywrightBrowserConfig`:

```
@CitrusEndpoint
@PlaywrightBrowserConfig(browserType = "firefox", headless = false)
private PlaywrightBrowser browser;
```

25.3. Playwright actions

All actions live behind a single Java DSL entry point:

```
import static
org.citrusframework.playwright.actions.PlaywrightActionBuilder.playwright;
```

Start a browser once and it becomes the ambient target of the current test thread — no `.browser(…)` repetition:

```
$(playwright().start().browserType("chromium"));            // starts and binds
$(playwright().start().browserType(PlaywrightBrowserType.FIREFOX)); // or via enum

$(playwright().open().url("http://localhost:8080"));
$(playwright().extract().pageCount().variable("pageCount"));

$(playwright().stop());                                     // unbinds and closes
```

The browser for an action resolves in this order; the first match wins and an ambiguous or missing browser fails fast with a report of all attempted strategies:

1. explicit `.browser(…)` binding on the action
2. the ambient browser bound to the current thread by `start()`
3. Citrus variable `citrus_playwright_browser` (endpoint name)
4. a unique `PlaywrightBrowser` bean in the application context

25.4. Browser engines

The engine can be given as a raw string (`"chromium"`, `"firefox"`, `"webkit"`) or as the type-safe `PlaywrightBrowserType` enum — on both the endpoint builder and the `start()` action:

```
PlaywrightBrowser browser = PlaywrightEndpoints.playwright()
        .browser()
        .browserType(PlaywrightBrowserType.WEBKIT)
        .build();
```

25.5. Interaction actions

```
$(playwright().click().locator("#save"));
$(playwright().doubleClick().locator(LocatorSpec.css("#row").nth(0)));
$(playwright().hover().locator("#menu"));
$(playwright().fill().locator("#username").value("${username}"));
$(playwright().press().locator("#username").value("Enter"));
$(playwright().select().locator("#gender").values("male", "female"));
$(playwright().check().locator("#agree"));
$(playwright().upload().locator("#avatar").file("uploads/avatar.png"));

$(playwright().navigate().back());
$(playwright().waitFor().locator("#dialog").hidden());
$(playwright().waitFor().networkIdle());
```

Locators accept CSS selectors by default and richer variants via locator specs:

```
$(playwright().click().locator(LocatorSpec.text("Save")));
$(playwright().click().locator(LocatorSpec.role("button").name("Submit")));
$(playwright().click().locator(LocatorSpec.testId("submit-button").last()));
$(playwright().click().locator(LocatorSpec.css("button.action").visible()));
```

`visible()` narrows a locator to visible elements only. It is the recommended replacement for the `:visible` CSS pseudo-class. Unlike `first()`, `last()` and `nth(…)` it is a **filter** rather than a positional selection, so it combines with them and is applied first — `.visible().nth(1)` selects the second **visible** match.

Pointer actions can opt out of Playwright's automatic scroll-into-view:

```
$(playwright().click().locator("#save").scroll("none"));
$(playwright().check().locator("#agree").scroll("none"));
```

`scroll` accepts `auto` (the default) and `none`, and is supported by `click`, `doubleClick`, `rightClick`, `hover`, `tap`, `check` and `uncheck`. Setting it on any other command fails at build time, because the underlying Playwright option does not exist for them.

Waiting can also be driven by a predicate evaluated against the matched element:

```
$(playwright().waitFor().locator("#grid").function("el => el.children.length > 10"));
```

25.6. Frames

A frame selector targets one iframe. Omitting it searches every frame in the page subtree, which avoids chaining through nested iframes:

```
$(playwright().frame().frame("#checkout").click("#submit"));
$(playwright().frame().click("#submit"));
```

Without a selector the inner locator must match in exactly one frame; an ambiguous match fails with an error naming the locator and the scope that was searched.

25.7. Verify and extract

Verify page, element, storage, console, network or download state — checks fail fast with sanitized expected/actual values:

```
$(playwright().verify().locator("#message").text("Citrus:ready"));
$(playwright().verify().url("http://localhost:8080/home"));
$(playwright().verify().storageLocal("theme", "dark"));
$(playwright().verify().consoleContains("Started"));
```

Two ARIA snapshot checks are available and they differ in kind. `ariaSnapshotContains` compares a substring of the rendered snapshot; `ariaSnapshotMatches` uses Playwright's own structural matcher:

```
$(playwright().verify().locator("#panel").ariaSnapshotContains("heading"));
$(playwright().verify().locator("#panel").ariaSnapshotMatches("""
    - heading "Title"
    - button "Save"
    """));
```

Both route their failure message through the configured secret redaction before it is reported. `ariaSnapshotMatches` accepts a `timeout` in milliseconds for slow-rendering regions; the substring check has no timeout because it reads an already captured snapshot:

```
$(playwright().verify().locator("#panel").ariaSnapshotMatches("- heading
\"Title\"").timeout(5000));
```

Extract values into test variables:

```
$(playwright().extract().locator("#message").text().variable("message"));
$(playwright().extract().title().variable("pageTitle"));
$(playwright().extract().cookie("session").variable("sessionCookie"));
```

25.8. JavaScript and screenshots

```
$(playwright().javascript().script("window.title").variable("jsResult"));
$(playwright().screenshot().path("target/playwright/screen.png").variable("screenshotP
ath"));
```

Screenshots support PNG, JPEG and WebP. The format is taken from the file extension unless `type` is set explicitly, in which case the explicit value wins. `quality` applies to the lossy formats only — 100 is lossless for WebP — and setting it for a PNG fails at build time:

```
$(playwright().screenshot().path("target/playwright/screen.webp").quality(50));
$(playwright().screenshot().path("target/playwright/full.png").fullPage(true));
$(playwright().screenshot().path("target/playwright/shot.bin").type("webp"));
```

25.9. Drag and drop

`drop` dispatches `dragenter`, `dragover` and `drop` with a synthetic data transfer, so upload zones can be exercised without a real file manager. Supply either a file or clipboard-like data, not both:

```
$(playwright().drop().locator("#dropzone").file("note.txt", "text/plain", "hello"));
$(playwright().drop().locator("#dropzone")
        .data("text/plain", "hello world")
        .data("text/uri-list", "https://example.com"));
```

A drop can carry the driver's own options: a `timeout` in milliseconds and a drop `position` relative to the element's top-left corner. Both are unset by default, which keeps the no-options `drop` call:

```
$(playwright().drop().locator("#dropzone").file("note.txt", "text/plain",
"hello").timeout(5000));
$(playwright().drop().locator("#dropzone").data("text/plain", "hello").position(10,
20));
```

25.10. Tracing, HAR and screencast

Tracing can additionally capture an aria snapshot and a screenshot on **every** action. Both are off by default because they add a capture to each step and make traces substantially larger — turn them on for a failing scenario rather than for a whole suite:

```
$(playwright().tracing().start().screenshots(true).snapshots(true)
        .ariaSnapshots(true).screenSnapshots(true));
$(playwright().tracing().stop().path("target/playwright/trace.zip"));
```

HAR recording is a first-class tracing command. Response bodies are **omitted by default** so that credentials and payloads are not written into the artifact; widening that is deliberate:

```
$(playwright().tracing().startHar("target/playwright/session.har").harUrlFilter("**/ap
i/**"));
$(playwright().tracing().stopHar());
$(playwright().tracing().startHar("target/playwright/full.har").harContent("embed"));
```

A screencast records the page as a video and can annotate it as a walkthrough. Nothing is recorded until the action starts it:

```
$(playwright().screencast().start("target/playwright/run.webm"));
$(playwright().screencast().showActions().position("top-right").fontSize(18));
$(playwright().screencast().showChapter("Checkout").chapterDescription("Applies the
coupon"));
$(playwright().screencast().stop());
```



The chapter text is set with `chapterDescription(…)`, not `description(…)`. The latter is the standard Citrus action description shared by every action builder.

A recording can carry custom HTML overlays, for example to narrate a walkthrough, and the recording size can be chosen up front. Overlay content is test-supplied HTML rendered into the page — do not interpolate untrusted data into it. Overlays stay visible until they are hidden again:

```
$(playwright().screencast().start("target/playwright/run.webm").size(1280, 720));
$(playwright().screencast().showOverlay("<div>Step 3 — checkout</div>"));
$(playwright().screencast().showOverlays());
$(playwright().screencast().hideOverlays());
$(playwright().screencast().stop());
```

Recording is expensive. Keep it to the scenarios whose output you intend to review. The real-time frame callback is Java DSL only, because a callback has no declarative representation:

```
$(playwright().screencast().start("target/playwright/run.webm")
        .onFrame(frame -> sendToVisionModel(frame.data())));
```

25.11. Storage state

Saved storage state can include the origin private file system so it can be restored into a later browser context:

```
$(playwright().storage().saveState("target/state.json").opfs(true));
$(playwright().storage().saveState("target/state.json").indexedDB(true).credentials(tr
ue));
$(playwright().storage().restoreState("target/state.json"));
```

`indexedDB` and `credentials` default to off, like `opfs`. Note that `credentials(true)` writes credential material into the storage-state file on disk — keep it off unless the restored session genuinely needs those credentials.

25.12. Dialogs

Besides accepting and dismissing a dialog, a test can assert that one was closed:

```
$(playwright().dialog().accept().triggerScript("window.confirm('Delete?')"));
$(playwright().dialog().verifyClosed().triggerScript("window.alert('Saved')"));
```

25.13. Console messages, page errors and response details

Uncaught page JavaScript errors are read directly from the page — there is no capture step, the call observes the errors present at that moment. Assert that a page stayed clean, match an error, or extract the report into a variable.

Console message retrieval reads the capture registry by default. With `filter` set to `since-navigation` it instead reads directly from the page, returning only messages since the last navigation:

Java

```
$(playwright().console().verifyNoPageErrors());
$(playwright().console().verifyPageErrorsContain("TypeError"));
$(playwright().console().pageErrors().variable("pageErrors"));

$(playwright().console().report().variable("consoleReport"));
$(playwright().console().report().filter("since-
navigation").variable("recentConsole"));
$(playwright().console().verifyContains("Started"));
```

XML

```
<playwright>
  <console command="verify-no-page-errors"/>
  <console command="page-errors-report" variable="pageErrors"/>
  <console command="report" filter="since-navigation" variable="recentConsole"/>
</playwright>
```

A waited-for page response can additionally verify which server answered and over which TLS protocol, and the structured snapshot stored in a variable carries the same details. On plain HTTP the driver reports no security details, so the TLS fields are absent rather than empty:

Java

```
$(playwright().network().waitForResponse().urlContains("/api/orders")
        .triggerScript("() => fetch('/api/orders')")
        .verifyServerAddress("127.0.0.1", 8080)
        .verifySecurityDetails("TLSv1.3")
        .variable("response"));
```

XML

```
<playwright>
  <network command="wait-for-response" url-contains="/api/orders" server-
ip="127.0.0.1"
      server-port="8080" security-protocol="TLSv1.3" variable="response"/>
</playwright>
```

25.14. HTTP credentials and passkeys

A browser endpoint can carry several origin-scoped HTTP credentials. The first entry matching the request origin is used, and an entry without an origin matches any request:

```
@CitrusEndpoint
@PlaywrightBrowserConfig(
    browserType = "chromium",
    httpCredentials = {
        @HttpCredential(origin = "https://api.example.com", username = "api", password
= "${apiPassword}"),
        @HttpCredential(username = "fallback", password = "${fallbackPassword}")
    })
private PlaywrightBrowser browser;
```

An explicitly configured `contextOptions` still wins, so the raw driver escape hatch keeps working unchanged.

Each credential can control when it is sent with `send`: `always` or `unauthorized`. Unset means the driver default. Use `unauthorized` against servers that require an unauthenticated first request:

```
@HttpCredential(origin = "https://api.example.com", username = "api",
        password = "${apiPassword}", send = "unauthorized")
```

WebAuthn passkeys are driven through a virtual authenticator, so passkey login flows need no hardware:

```
$(playwright().credentials().create("example.com")
        .id("${credentialId}").userHandle("${userHandle}")
        .privateKey("${privateKey}").publicKey("${publicKey}"));
$(playwright().credentials().install());
$(playwright().credentials().read("storedCredential"));
$(playwright().credentials().delete("example.com"));
```

Only the credential id is written to the test variable. Private key material never reaches the test context, the logs, or the failure evidence.

25.15. API requests with the browser session

A test often needs to call the application's HTTP API **as the user the browser is logged in as**: to seed data before a UI step, or to check through the API what the UI claims to have saved. The connector provides a Citrus `HttpClient` whose requests travel through a Playwright browser context, the same way Playwright's own `APIRequestContext` (`context.request()`) does. You write the requests and verify the responses with the standard `http()` actions; only the client is different.

When to use which. Use a browser-session client when the request must carry the browser's session, or must hand a session back to the browser (an API login that the UI then continues). Use a plain `HttpClient` for everything else. The two are used the same way in a test, and you can mix them freely.

25.15.1. Declaring the client

Citrus Bean

```
@BindToRegistry
private final PlaywrightBrowser browser = PlaywrightEndpoints.playwright()
        .browser()
        .baseUrl("http://localhost:8080")
        .build();

@BindToRegistry
private final HttpClient browserApi = PlaywrightEndpoints.playwright()
        .apiClient()
        .browser(browser)
        .build();
```

Annotation

```
@CitrusEndpoint
@PlaywrightApiClientConfig(browser = "browser")
private HttpClient browserApi;
```

Spring XML

```
<bean id="browserTransport"
class="org.citrusframework.playwright.http.PlaywrightClientHttpRequestFactory">
  <constructor-arg ref="browser"/>
</bean>

<citrus-http:client id="browserApi"
                    request-url="http://localhost:8080"
                    request-factory="browserTransport"/>
```

The builder and the annotation offer the same options, except where the table shows no attribute. Building the client fails when no browser is bound, when a browser name cannot be resolved, or when there is neither a request URL nor a browser `baseUrl`.

Builder method	Annotation attribute	Effect and default
`browser(PlaywrightBrowser)` / `browser(String)`	`browser`	The browser endpoint whose context carries the requests. Required; a name is resolved as a bean
`context(String)`	`context`	Binds every request to a named browser context. Default: the browser's **current** context at the time each request is sent
`requestUrl(String)`	`requestUrl`	Base URL that request paths are appended to. Default: the browser endpoint's `baseUrl`

Builder method	Annotation attribute	Effect and default
`timeout(long)`	`timeout`	Request timeout in milliseconds, also used as the client timeout. Default: the driver's 30 seconds
`maxRedirects(int)`	`maxRedirects`	Redirects to follow. `0` returns the redirect response itself. Default: 20
`maxRetries(int)`	`maxRetries`	Retries after a connection reset (`ECONNRESET`), with a backoff starting at 250 ms. Other failures are never retried. Default: 0
`ignoreHttpsErrors(boolean)`	`ignoreHttpsErrors`	Accepts invalid TLS certificates for this client. It can only widen the browser context's own `ignoreHTTPSErrors` setting, never narrow it. Default: false
`handleCookies(boolean)`	`handleCookies`	Maps `Set-Cookie` response headers to message cookies, as on any Http client. Default: false
`errorHandlingStrategy(…)`	—	What a non-2xx response does, as on any Http client. Default: `PROPAGATE`, the response is delivered for the test to verify
`name(String)`	the field name	Endpoint name

For full control over the Http client, plug the transport into the regular builder. The request URL is then yours to set:

```
HttpClient custom = new HttpClientBuilder()
        .requestUrl("http://localhost:8080")
        .requestFactory(new PlaywrightClientHttpRequestFactory(browser)
                .context("admin")
                .timeout(10_000)
                .maxRedirects(0)
                .maxRetries(2)
                .ignoreHttpsErrors(true))
        .defaultAcceptHeader(false)
        .build();
```

25.15.2. Sending requests and verifying responses

Log in through the UI, then call the API as that user:

Java

```
$(playwright().open().url("http://localhost:8080/login"));
$(playwright().fill().locator("#username").value("alice"));
$(playwright().click().locator("#submit"));

$(http().client(browserApi)
        .send()
        .post("/api/orders")
        .message()
        .contentType("application/json")
        .body("{\"item\":\"${item}\"}"));

$(http().client(browserApi)
        .receive()
        .response(HttpStatus.CREATED)
        .message()
        .type(MessageType.JSON)
        .validate(validation().jsonPath().expression("$.item", "${item}")));
```

XML

```
<http client="browserApi">
  <send-request>
    <GET path="/api/me"/>
  </send-request>
</http>

<http client="browserApi">
  <receive-response>
    <response status="200" content-type="application/json">
      <body>
        <data>{"user":"alice"}</data>
      </body>
    </response>
  </receive-response>
</http>
```

YAML

```
- http:
    client: "browserApi"
    sendRequest:
      GET:
        path: "/api/me"
- http:
    client: "browserApi"
    receiveResponse:
      response:
        status: "200"
        contentType: "application/json"
        body:
          data: |
            {"user":"alice"}
```

Form and multipart bodies are encoded by the Http client before they reach the browser, so they work exactly as on a plain client:

```
MultiValueMap<String, Object> parts = new LinkedMultiValueMap<>();
parts.add("name", "citrus");
parts.add("file", new ClassPathResource("uploads/avatar.png"));

$(http().client(browserApi)
        .send()
        .post("/api/upload")
        .message()
        .contentType(MediaType.MULTIPART_FORM_DATA_VALUE)
        .body(parts));
```

25.15.3. What travels with a request

The browser context's cookies, including HttpOnly ones, go out with every request, and a `Set-Cookie` on a response is stored in the context, so the next page navigation sends it. The browser context's HTTP settings apply as well: extra HTTP headers, user agent, HTTP credentials, proxy, client certificates and `ignoreHTTPSErrors`. Configure them on the browser endpoint (`contextOptions(…)`, `httpCredentials(…)`), and they hold for pages and API requests alike. Bind a client to a named context with `context("admin")` to test several users side by side.

Web storage is **not** part of the session. If the application keeps a bearer token in `localStorage`, read it and send it yourself:

```
$(playwright().storage().local().read("token").variable("token"));
$(http().client(browserApi)
        .send()
        .get("/api/profile")
        .message()
        .header("Authorization", "Bearer ${token}"));
```

A request that sets a `Cookie` header fails before it is sent. The driver would use that header **instead of** the browser's cookies and silently drop the session, so add cookies to the context with `playwright().cookies().add(…)` instead.

Browser-session requests are not part of the page's network traffic. `network().route(…)` stubs do not intercept them, network capture and HAR recording do not list them, and the context trace does not record them. To stub the backend an API request talks to, use a Citrus Http server.

25.15.4. Transport details

Each response carries the details the driver reports as headers you can verify or extract like any other. A detail the driver does not report, such as TLS over plain HTTP or the DNS phase for an IP address, is left out rather than faked.

Header	Value
`citrus_playwright_api_url`	Final URL of the response, after the redirects the driver followed
`citrus_playwright_api_server_ip`, `citrus_playwright_api_server_port`	Address of the server that answered
`citrus_playwright_api_tls_protocol`, `_tls_subject_name`, `_tls_issuer`	TLS details of the connection
`citrus_playwright_api_tls_valid_from`, `_tls_valid_to`	Certificate validity, in epoch seconds
`citrus_playwright_api_timing_start_time`	Request start, in epoch milliseconds
`citrus_playwright_api_timing_domain_lookup_start`, `_domain_lookup_end`, `_connect_start`, `_secure_connection_start`, `_connect_end`, `_request_start`, `_response_start`, `_response_end`	Phases in milliseconds relative to the start time; `_response_end` is the total duration

```
$(http().client(browserApi)
        .receive()
        .response(HttpStatus.OK)
        .message()
        .header("citrus_playwright_api_tls_protocol", "@startsWith('TLS')@")
        .header("citrus_playwright_api_timing_response_end", "@lowerThan(500)@")
        .extract(extractor().fromHeaders().header("citrus_playwright_api_url",
"finalUrl")));
```

25.15.5. Playwright API requests in Citrus

Every capability of Playwright's `APIRequestContext` and `APIResponse` has a Citrus counterpart, except where the table says otherwise.

Playwright	Citrus browser-session client
Requests	
`get`, `post`, `put`, `patch`, `delete`, `head`, `fetch` with a method	`http().client(browserApi).send().get(…)`, `.post(…)`, `.put(…)`, `.patch(…)`, `.delete(…)`, `.head(…)`, `.options(…)`, `.trace(…)`
Request URL and `baseURL`	Client `requestUrl` (default: the browser `baseUrl`) plus the request path; `.uri(…)` sends to another absolute URL
`setQueryParam(…)`	`.queryParam(name, value)`
`setHeader(…)`	`.message().header(name, value)`, `.contentType(…)`, `.accept(…)`
`setData(…)`	`.message().body(…)` with text, a resource or a JSON document
`setForm(…)`	`.message().contentType("application/x-www-form-urlencoded").body(multiValueMap)`
`setMultipart(…)`	`.message().contentType("multipart/form-data").body(multiValueMap)`
`setTimeout(…)`	Client `timeout(…)`
`setMaxRedirects(…)`	Client `maxRedirects(…)`
`setMaxRetries(…)`	Client `maxRetries(…)`
`setIgnoreHTTPSErrors(…)`	Client `ignoreHttpsErrors(true)`, or the browser context's `ignoreHTTPSErrors`
`setFailOnStatusCode(true)`	Client `errorHandlingStrategy(ErrorHandlingStrategy.THROWS_EXCEPTION)`
`fetch(Request)`, replaying a captured page request	Not supported
Session	
Cookies of the browser context, in both directions	Automatic

Playwright	Citrus browser-session client
Context `extraHTTPHeaders`, `userAgent`, `httpCredentials`, `proxy`, `clientCertificates`	Automatic, from the browser endpoint's context configuration
`storageState()`	`playwright().storage().saveState(…)`; it holds the same cookies
`Playwright.request().newContext()`, a request without the browser session	A plain `HttpClient`
Responses	
`status()`, `ok()`	`.response(HttpStatus.CREATED)`, or the `citrus_http_status_code` header
`statusText()`	Not available. `citrus_http_reason_phrase` holds the standard reason phrase of the status code, as on any Http client
`headers()`, `headersArray()`	Message headers: `.message().header(…)`, `extractor().fromHeaders()`. Repeated headers arrive comma-joined (`Set-Cookie: A=1; Path=/,B=2; Path=/`), or as separate cookies with `handleCookies(true)`
`body()`, `text()`	The message body, validated and extracted like any other; binary bodies follow the client's binary media types, as on any Http client. A compressed body arrives decompressed, without `Content-Encoding` and with a `Content-Length` that matches it
`url()`	`citrus_playwright_api_url`
`serverAddr()`	`citrus_playwright_api_server_ip`, `citrus_playwright_api_server_port`
`securityDetails()`	`citrus_playwright_api_tls_*`
`timing()`	`citrus_playwright_api_timing_*`
`dispose()`	Automatic after each response
Context	
`tracing()`	Not supported. The context trace does not record API requests; Citrus message tracing and reports do
`dispose()`	Follows the browser endpoint's lifecycle

25.15.6. Http client settings, threads and logging

The client is a regular `HttpClient`, so message conversion, header mapping, interceptors, the error handling strategy and `handleCookies` behave exactly as on any other client. Settings of the replaced transport do not apply: the HttpComponents `httpClient`, connection pooling and a custom `restTemplate`. Replacing the `requestFactory` detaches the client from the browser.



A browser-session client runs on the browser endpoint's thread, like every Playwright action. A forked send (`fork(true)`) runs on another thread and therefore fails.

The cookies and credentials that the browser adds never appear in Citrus logs: the driver attaches them after Citrus has logged the request, and a failed request is reported without the driver's call log, which lists them. The rest of the exchange is logged like any Http exchange, masked by the Citrus-wide `citrus.logger.mask.keywords` (see Playwright browser). Add keywords such as `token` for response bodies that carry credentials.

Request URLs are not secret. As with every Http client, the request URL, query parameters included, appears in the request log and in the message of a failed request (Spring's `I/O error on GET request for "…"`). Keep secrets out of query parameters and send them in headers or bodies instead.

25.16. XML and YAML DSL

The connector registers a `playwright` action namespace so declarative test cases can use the same features. Loading requires the standard `citrus-yaml` / `citrus-xml` modules on the classpath.

YAML

```
actions:
  - playwright:
      browser: "playwrightBrowser"
      start:
        browser-type: "chromium"

  - playwright:
      open:
        url: "index.html"

  - playwright:
      fill:
        element:
          css: "#username"
        value: "${username}"

  - playwright:
      verify:
        element:
          text: "Welcome"
        check: "hidden"

  - playwright:
      click:
        element:
          css: ".action"
          visible: true
        scroll: "none"
```

```
  - playwright:
      screenshot:
        path: "target/playwright/screen.webp"
        quality: 50

  - playwright:
      drop:
        element:
          css: "#dropzone"
        content-type: "text/plain"
        value: "hello world"

  - playwright:
      screencast:
        command: "start"
        path: "target/playwright/run.webm"

  - playwright:
      credentials:
        command: "install"

  - playwright:
      stop: {}
```

XML

```
<playwright browser="playwrightBrowser">
  <open url="index.html"/>
</playwright>

<playwright>
  <fill value="${username}">
    <element css="#username"/>
  </fill>
</playwright>

<playwright>
  <verify check="text" expected="Welcome">
    <element css="#welcome"/>
  </verify>
</playwright>
```

Supported element action keys: `start`, `stop`, `open`, `navigate` (direction), `click`, `double-click`, `right-click`, `hover`, `focus`, `tap`, `fill`, `clear`, `press`, `check`, `uncheck`, `select`, `upload`, `wait` (condition), `screenshot`, `screencast`, `credentials`, `drop`, `javaScript`/`javascript`, `verify` (check + expected) and `extract` (value + variable). Element locators accept one of `css`, `xpath`, `text`, `role` (plus optional `name`) or `test-id`, narrowed by `nth`, `first` or `last`.

The same applies to XML:

```
<playwright>
  <click scroll="none">
    <element css=".action" visible="true"/>
  </click>
</playwright>

<playwright>
  <screenshot path="target/playwright/screen.webp" quality="50"/>
</playwright>

<playwright>
  <drop content-type="text/plain" value="hello world">
    <element css="#dropzone"/>
  </drop>
</playwright>

<playwright>
  <screencast command="start" path="target/playwright/run.webm"/>
</playwright>

<playwright>
  <credentials command="install"/>
</playwright>
```

Browser context, page, network and diagnostics actions are available declaratively as well. Actions with several modes take a `command`; commands and option values are case-insensitive and accept `_` in place of `-`. The attribute or key names are the same in XML and YAML:

Key	Commands	Options
`console`	`capture`, `clear`, `report`, `verify-contains`	`text`, `variable`
`context`	`create`, `switch`, `close`	`alias`, `storage-state`
`cookies`	`add`, `clear`, `read`, `verify`	`name`, `value`, `url`, `domain`, `path`, `expires`, `http-only`, `secure`, `same-site`, `variable`
`dialog`	`accept`, `dismiss`, `verify-closed`	`prompt-text`, `message`, `type`, `trigger-script`
`download`	—	`element`, `trigger-script`, `save-as`, `path-variable`, `filename-variable`
`emulate`	—	`width` + `height`, `latitude` + `longitude`, `accuracy`, `color-scheme`, `locale`, `timezone`, `user-agent`, `context`
`enter`	—	Alias of `fill`
`frame`	`fill`, `click`, `verify-text`	`selector`, `element`, `value`

Key	Commands	Options
network	capture, clear, report, verify-url-contains, route-abort, route-fulfill, route-continue-header, unroute, wait-for-response	pattern, text, body, content-type, status, header (XML) / headers (YAML), header-name, header-value, url, url-contains, url-matches, timeout, element, trigger-script, include-body, variable
page	create, switch, close	alias, context, index, title, url-contains
page-object	—	type, method, arguments, validator
pdf	—	path, print-background, variable
permissions	grant, clear	permissions
storage	set, read, verify, remove, clear, save-state, restore-state	scope (local, session), key, value, path, opfs, variable
tracing	start, stop, start-har, stop-har	path, screenshots, snapshots, sources, aria-snapshots, screen-snapshots, har-content, har-mode, har-url-filter, variable

page with command="switch" takes exactly one of alias, index, title or url-contains. For tracing, path is the trace archive written on stop and the HAR file written on start-har. On network, status is the fulfilled response status for route-fulfill and the expected response status for wait-for-response.

YAML

```
actions:
  - playwright:
      network:
        command: "route-fulfill"
        pattern: "**/api/users"
        body: "[]"
        content-type: "application/json"
        headers:
          - name: "X-Test"
            value: "1"

  - playwright:
      tracing:
        command: "start-har"
        path: "target/playwright/session.har"
        har-url-filter: "**/api/**"

  - playwright:
      permissions:
        command: "grant"
        permissions:
          - "geolocation"
          - "clipboard-read"

  - playwright:
      page-object:
        type: "com.example.LoginPage"
        method: "login"
        arguments:
          - "${username}"
```

XML

```
<playwright>
  <network command="route-fulfill" pattern="**/api/users" body="[]" content-
type="application/json">
    <header name="X-Test" value="1"/>
  </network>
</playwright>

<playwright>
  <frame command="click" selector="#checkout">
    <element role="button" name="Pay"/>
  </frame>
</playwright>

<playwright>
  <storage command="save-state" path="target/state.json" opfs="true"/>
</playwright>

<playwright>
  <permissions command="grant" permissions="geolocation, clipboard-read"/>
</playwright>
```

In XML, list values such as `select` and `upload values`, `permissions` and `page-object arguments` are separated by commas; each entry is trimmed, so values may contain spaces. Page-object and validator types are fully qualified class names.



Options that take a Java object or a callback stay Java DSL only: `context().options(…)` with a `Browser.NewContextOptions`, raw function locators, and the screencast frame callback.

25.17. Failure evidence

On test failure the endpoint can persist diagnostic artifacts to `target/playwright/`: screenshot (`failure.png`), page source (`page.html`), trace (`trace.zip`), console messages (`console.log`), network log (`network.log`) and a Markdown summary (`failure-summary.md`). Enable them per endpoint:

```
PlaywrightEndpoints.playwright()
        .browser()
        .captureFailureScreenshot(true)
        .captureFailurePageSource(true)
        .captureFailureTrace(true)
        .build();
```

25.18. Browser runtimes

Integration tests that drive a real browser require the Playwright browser binaries. Install them

with the `playwright-runtimes` Maven profile, which fetches just the engine under test (`chromium` by default, selectable with `-Dplaywright.browser=firefox|webkit`):

```
mvn verify -pl connectors/citrus-playwright -Pplaywright-runtimes
```

Be aware that the Playwright Java bindings install browsers themselves on the first `Playwright.create()` call. They check the **whole** registered set and fetch whatever is missing, so Chromium, Firefox and WebKit are pulled even when the profile has already installed Chromium - and the download runs inside the test. Set `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` to forbid that implicit install. The profile is then the only source of binaries, and the browser tests skip when the runtime is missing instead of downloading it:

```
PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1 mvn verify -pl connectors/citrus-playwright
```

The CI build combines the two: it pins `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` and activates `playwright-runtimes`, so the browser binaries are always explicit and cached.

Chapter 26. Vert.x event bus support

Vert.x is an application platform for the JVM that provides a network event bus for lightweight scalable messaging solutions. The Citrus Vert.x components do participate on that event bus messaging as producer or consumer. With these components you can access Vert.x instances available in your network in order to test those Vert.x applications in some integration test scenario.



The Vert.x components in Citrus are kept in a separate Maven module. So you should add the module as Maven dependency to your project accordingly.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-vertx</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The next sections discuss sending and receiving operations on the Vert.x event bus with Citrus.

26.1. Vert.x endpoint

As usual Citrus uses an endpoint component in order to specify some message destination to send and receive messages to and from. The Vert.x endpoint component is defined as follows in your Citrus Spring configuration.

Citrus Bean

```
@BindToRegistry
public VertxEndpoint simpleVertxEndpoint() {
    return new VertxEndpointBuilder()
        .host("localhost")
        .port(5001)
        .pubSubDomain(false)
        .address("news-feed")
        .build();
}

private VertxInstanceFactory vertxInstanceFactory() {
    return new CachingVertxInstanceFactory();
}
```

Spring Bean

```
@Bean
public VertxEndpoint simpleVertxEndpoint() {
    return new VertxEndpointBuilder()
        .host("localhost")
        .port(5001)
        .pubSubDomain(false)
        .address("news-feed")
        .build();
}

private VertxInstanceFactory vertxInstanceFactory() {
    return new CachingVertxInstanceFactory();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-vertx="http://www.citrusframework.org/schema/vertx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/vertx/config
           http://www.citrusframework.org/schema/vertx/config/citrus-vertx-
config.xsd">

    <citrus-vertx:endpoint id="simpleVertxEndpoint"
          host="localhost"
          port="5001"
          pub-sub-domain="false"
          address="news-feed"/>

    <bean id="vertxInstanceFactory"
class="org.citrusframework.vertx.factory.CachingVertxInstanceFactory"/>
</beans>
```



Citrus provides a special Vert.x configuration schema that is used in our Spring Xml bean configurations. You have to include the `citrus-vertx` namespace in your Spring configuration XML files as seen in the example above.

The endpoint holds some general information how to access the Vert.x event bus. Host and port values define the Vert.x Hazelcast cluster hostname and port. Citrus starts a new Vert.x instance using this cluster. So all other Vert.x instances connected to this cluster host will receive the event bus messages from Citrus during the test. In your test case you can use this endpoint component

referenced by its id or name in order to send and receive messages on the event bus address **news-feed** . In Vert.x the event bus address defines the destination for event consumers to listen on. As already mentioned cluster hostname and port are optional, so Citrus will use **localhost** and a new random port on the cluster host if nothing is specified.

The Vert.x event bus supports publish-subscribe and point-to-point message communication patterns. By default the **pubSubDomain** in Citrus is false so the event bus sender will initiate a point-to-point communication on the event bus address. This means that only one single consumer on the event bus address will receive the message. If there are more consumers on the address the first to come wins and receives the message. In contrary to that the publish-subscribe scenario would deliver the message to all available consumers on the event bus address simultaneously. You can enable the **pubSubDomain** on the Vert.x endpoint component for this communication pattern.

The Vert.x endpoint needs an instance factory implementation in order to create the embedded Vert.x instance. By default the bean name **vertxInstanceFactory** is recognized by all Vert.x endpoint components. We will talk about Vert.x instance factories in more detail later on in this chapter.

As message content you can send and receive JSON objects or simple character sequences to the event bus. Let us have a look at a simple sample sending action that uses the new Vert.x endpoint component:

Java

```
@CitrusTest
public void vertxTest() {
    then(send()
            .endpoint(simpleVertxEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello from Citrus!")
    );
}
```

XML

```
<test name="VertxTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="simpleVertxEndpoint">
        <message type="plaintext">
            <body>
              <data>Hello from Citrus!</data>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: VertxTest
actions:
  - send:
      endpoint: "simpleVertxEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "Hello from Citrus!"
```

Spring XML

```
<testcase name="vertxTest">
    <actions>
        <send endpoint="simpleVertxEndpoint">
          <message type="plaintext">
            <payload>Hello from Citrus!</payload>
          </message>
        </send>
    </actions>
</testcase>
```

As the Vert.x Citrus endpoint is bidirectional you can also receive messages from the event bus.

Java

```
@CitrusTest
public void vertxTest() {
    then(receive()
            .endpoint(simpleVertxEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello from Vert.x!")
                .header("citrus_vertx_address", "news-feed")
    );
}
```

XML

```
<test name="VertxTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="simpleVertxEndpoint">
        <message type="plaintext">
            <body>
              <data>Hello from Vert.x!</data>
            </body>
            <headers>
              <header name="citrus_vertx_address" value="news-feed"/>
            </headers>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: VertxTest
actions:
  - receive:
      endpoint: "simpleVertxEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "Hello from Vert.x!"
        headers:
          - name: citrus_vertx_address
            value: news-feed
```

Spring XML

```
<testcase name="vertxTest">
    <actions>
        <receive endpoint="simpleVertxEndpoint">
          <message type="plaintext">
            <payload>Hello from Vert.x!</payload>
          </message>
          <header>
            <element name="citrus_vertx_address" value="news-feed"/>
          </header>
        </receive>
    </actions>
</testcase>
```

Citrus automatically adds some special message headers to the message, so you can validate the Vert.x event bus address. This completes the simple send and receive operations on a Vert.x event bus. Now let's move on to synchronous endpoints where Citrus waits for a reply on the event bus.

26.2. Synchronous Vert.x endpoint

The synchronous Vert.x event bus producer sends a message and waits synchronously for the response to arrive on some reply address destination. The reply address name is generated automatically and set in the request message header attributes so the receiving counterpart in this communication can send its reply to that event bus address. The basic configuration for a synchronous Vert.x endpoint component looks like follows:

Citrus Bean

```
@BindToRegistry
public VertxSnycEndpoint vertxSyncEndpoint() {
    return new VertxSyncEndpointBuilder()
        .timeout(1000)
        .address("hello")
        .pollingInterval(300)
        .build();
}

private VertxInstanceFactory vertxInstanceFactory() {
    return new CachingVertxInstanceFactory();
}
```

Spring Bean

```
@Bean
public VertxSnycEndpoint vertxSyncEndpoint() {
    return new VertxSyncEndpointBuilder()
        .timeout(1000)
        .address("hello")
        .pollingInterval(300)
        .build();
}

private VertxInstanceFactory vertxInstanceFactory() {
    return new CachingVertxInstanceFactory();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-vertx="http://www.citrusframework.org/schema/vertx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/vertx/config
           http://www.citrusframework.org/schema/vertx/config/citrus-vertx-
config.xsd">

    <citrus-vertx:sync-endpoint id="vertxSyncEndpoint"
          address="hello"
          timeout="1000"
          polling-interval="300"/>
</beans>
```

Synchronous endpoints poll for synchronous reply messages to arrive on the event bus reply address. The poll interval is an optional setting in order to manage the amount of reply message handshake attempts. Once the endpoint was able to receive the reply message synchronously the test case can receive the reply. In case all message handshake attempts do fail because the reply message is not available in time we raise some timeout error and the test will fail.



The Vert.x endpoint uses temporary reply address destinations. The temporary reply address in generated and is only used once for a single communication handshake. After that the reply address is dismissed again.

When sending a message to the synchronous Vert.x endpoint the producer will wait synchronously for the response message to arrive on the reply address. You can receive the reply message in your test case using the same endpoint component. So we have two actions on the same endpoint, first send then receive.

Java

```
@CitrusTest
public void vertxTest() {
    when(send()
            .endpoint(vertxSyncEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello from Citrus!")
    );

    then(receive()
            .endpoint(vertxSyncEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello from Vert.x!")
    );
}
```

XML

```
<test name="VertxTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="vertxSyncEndpoint">
        <message type="plaintext">
            <body>
              <data>Hello from Citrus!</data>
            </body>
        </message>
      </send>

      <receive endpoint="vertxSyncEndpoint">
        <message type="plaintext">
            <body>
              <data>Hello from Vert.x!</data>
            </body>
        </message>
      </receive>
    </actions>
</test>
```

YAML

```
name: VertxTest
actions:
  - send:
      endpoint: "vertxSyncEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "Hello from Citrus!"
  - receive:
      endpoint: "vertxSyncEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "Hello from Vert.x!"
```

Spring XML

```
<testcase name="vertxTest">
    <actions>
        <send endpoint="vertxSyncEndpoint">
          <message type="plaintext">
            <payload>Hello from Citrus!</payload>
          </message>
        </send>

        <receive endpoint="vertxSyncEndpoint">
          <message type="plaintext">
            <payload>Hello from Vert.x!</payload>
          </message>
        </receive>
    </actions>
</testcase>
```

In the last section we saw that synchronous communication is based on reply messages on temporary reply event bus address. We saw that Citrus is able to send messages to event bus address and wait for reply messages to arrive. This next section deals with the same synchronous communication, but send and receive roles are switched. Now Citrus receives a message and has to send a reply message to a temporary reply address.

We handle this synchronous communication with the same synchronous Vert.x endpoint component. Only difference is that we initially start the communication by receiving a message from the endpoint. Knowing this Citrus is able to send a synchronous response back. Again just use the same endpoint reference in your test case. The handling of the temporary reply address is done automatically behind the scenes. So we have again two actions in our test case, but this time first receive then send.

Java

```
@CitrusTest
public void vertxTest() {
    when(receive()
            .endpoint(vertxSyncEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("Hello from Vert.x!")
    );

    then(send()
            .endpoint(vertxSyncEndpoint)
            .message()
                .type(MessageType.PLAINTEXT)
                .body("This is the reply from Citrus!")
    );
}
```

XML

```
<test name="VertxTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <receive endpoint="vertxSyncEndpoint">
        <message type="plaintext">
            <body>
              <data>Hello from Vert.x!</data>
            </body>
        </message>
      </receive>

      <send endpoint="vertxSyncEndpoint">
        <message type="plaintext">
            <body>
              <data>This is the reply from Citrus!</data>
            </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: VertxTest
actions:
  - receive:
      endpoint: "vertxSyncEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "Hello from Vert.x!"
  - send:
      endpoint: "vertxSyncEndpoint"
      message:
        type: PLAINTEXT
        body:
          data: "This is the reply from Citrus!"
```

Spring XML

```
<testcase name="vertxTest">
    <actions>
        <receive endpoint="vertxSyncEndpoint">
          <message type="plaintext">
            <payload>Hello from Vert.x!</payload>
          </message>
        </receive>

        <send endpoint="vertxSyncEndpoint">
          <message type="plaintext">
            <payload>This is the reply from Citrus!</payload>
          </message>
        </send>
    </actions>
</testcase>
```

The synchronous message endpoint for Vert.x event bus communication will handle all reply address destinations and provide those behind the scenes.

26.3. Vert.x instance factory

Citrus starts an embedded Vert.x instance at runtime in order to participate in the Vert.x cluster. Within this cluster multiple Vert.x instances are connected via the event bus. For starting the Vert.x event bus Citrus uses a cluster hostname and port definition. You can customize this cluster host in order to connect to a very special cluster in your network.

Now Citrus needs to manage the Vert.x instances created during the test run. By default Citrus will look for an instance factory bean named **vertxInstanceFactory** . You can choose the factory implementation to use in your project. By default you can use the caching factory implementation that caches the Vert.x instances so we do not connect more than one Vert.x instance to the same

cluster host. Citrus offers following instance factory implementations:

org.citrusframework.vertx.factory.CachingVertxInstanceFactory

default implementation that reuses the Vert.x instance based on given cluster host and port. With this implementation we ensure to connect a single Citrus Vert.x instance to a cluster host.

org.citrusframework.vertx.factory.SingleVertxInstanceFactory

creates a single Vert.x instance and reuses this instance for all endpoints. You can also set your very custom Vert.x instance via configuration for custom Vert.x instantiation.

The instance factory implementations do implement the ***VertxInstanceFactory*** interface. So you can also provide your very special implementation. By default, Citrus looks for a bean named **vertxInstanceFactory** but you can also define your very special factory implementation onm an endpoint component. The Vert.x instance factory is set on the Vert.x endpoint as follows:

Citrus Bean

```
@BindToRegistry
public VertxEndpoint vertxHelloEndpoint() {
    return new VertxEndpointBuilder()
        .address("news-feed")
        .vertxInstanceFactory(singleVertxInstanceFactory())
        .build();
}

private VertxInstanceFactory singleVertxInstanceFactory() {
    return new CachingVertxInstanceFactory();
}
```

Spring Bean

```
@Bean
public VertxEndpoint vertxHelloEndpoint() {
    return new VertxEndpointBuilder()
        .address("news-feed")
        .vertxInstanceFactory(singleVertxInstanceFactory())
        .build();
}

private VertxInstanceFactory singleVertxInstanceFactory() {
    return new SingleVertxInstanceFactory();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-vertx="http://www.citrusframework.org/schema/vertx/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/vertx/config
       http://www.citrusframework.org/schema/vertx/config/citrus-vertx-config.xsd">

    <citrus-vertx:endpoint id="vertxHelloEndpoint"
          address="news-feed"
          vertx-factory="singleVertxInstanceFactory"/>

    <bean id="singleVertxInstanceFactory"
          class="org.citrusframework.vertx.factory.SingleVertxInstanceFactory"/>
</beans>
```

Chapter 27. JDBC support

Database communication is an essential part of many applications, when persistent data storage is required. May it be orders, customer data, product recommendations or product information, if persistent storage is in place, the data contains a certain business value. Therefore it's important that your software handles your persistent storage the right way. To ensure that, Citrus offers a JDBC server endpoint that allows you to verify the communication between your application and a real database server which is accessible via the Citrus-JDBC-Driver.

To enable the JDBC support for your test project, you'll have to add the following dependency.

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-jdbc</artifactId>
  <version>${citrus.version}</version>
  <scope>test</scope>
</dependency>
```

27.1. The Citrus-JDBC-Driver

The Citrus-JDBC-Driver is a JDBC conform driver that realizes the communication with the Citrus-JDBC-Server. To be able to use Citrus-JDBC within a CI/CD setup, we recommend to ensure that your software is able to exchange the JDBC driver by configuration. The Citrus-JDBC-Driver is available on maven central under `org.citrusframework.citrus-db-driver`. After the driver has been downloaded and configured to be used by your application, please make sure that the driver configuration matches the Citrus-JDBC-Server configuration in your tests.

Example jdbc driver configuration

```
<systemProperties>
    <systemProperty>
      <name>todo.persistence.type</name>
      <value>jdbc</value>
    </systemProperty>
    <systemProperty>
      <name>todo.jdbc.driverClassName</name>
      <value>org.citrusframework.db.driver.JdbcDriver</value>
    </systemProperty>
    <systemProperty>
      <name>todo.jdbc.url</name>
      <value>jdbc:citrus:http://localhost:3306/testdb</value>
    </systemProperty>
</systemProperties>
```

Citrus-JDBC-Server matching the driver config

```
@Bean
public JdbcServer jdbcServer() {
    return CitrusEndpoints.jdbc()
        .server()
        .host("localhost")
        .databaseName("testdb")
        .port(3306)
        .build();
}
```

27.2. The Citrus-JDBC-Server

To setup a JDBC endpoint within your tests, just configure it as any other endpoint via e.g. XML, Spring Bean or by Citrus Annotations.

Spring Bean

```
@Bean
public JdbcServer jdbcServer() {
    return CitrusEndpoints.jdbc()
        .server()
        .host("localhost")
        .databaseName("testdb")
        .port(4567)
        .timeout(10000L)
        .autoStart(true)
        .autoTransactionHandling(false)
        .build();
}
```

Spring XML

```
<citrus-jdbc:server id="testServer"
                      host="citrus-jdbc-test-server"
                      port="4567"
                      database-name="test-db"
                      max-connections="50"
                      auto-start="true"/>
```

Citrus Annotations

```
@CitrusEndpoint
@JdbcServerConfig(
            databaseName = "testdb",
            autoStart = true,
            port = 4567)
private JdbcServer jdbcServer;
```

After that configuration has been done, you'll be able to use the Server within your tests to receive and send messages from or to your system under test.

```
@Test
@CitrusTest
public void testAddTodoEntry() {
    variable("todoName", "citrus:concat('todo_', citrus:randomNumber(4))");
    variable("todoDescription", "Description: ${todoName}");

    http()
        .client(todoClient)
        .send()
        .post("/todolist")
        .fork(true)
        .contentType("application/x-www-form-urlencoded")
        .payload("title=${todoName}&description=${todoDescription}");

    receive(jdbcServer)
        .messageType(MessageType.JSON)
        .message(JdbcMessage.execute(
            "@startsWith('INSERT INTO todo_entries (id, title, description, done)
VALUES (?, ?, ?, ?)')@"));①

    send(jdbcServer)
        .message(JdbcMessage.result().rowsUpdated(1));②

    http()
        .client(todoClient)
        .receive()
        .response(HttpStatus.FOUND.value());

    http()
        .client(todoClient)
        .send()
        .get("/todolist")
        .fork(true)
        .accept("text/html");

    receive(jdbcServer)
            .message(JdbcMessage.execute("SELECT id, title, description FROM
todo_entries"));③
```

```
    send(jdbcServer)
        .messageType(MessageType.JSON)
        .message(JdbcMessage.result().dataSet("[ {" +
                    "\"id\": \"" + UUID.randomUUID().toString() + "\"," +
                    "\"title\": \"${todoName}\"," +
                    "\"description\": \"${todoDescription}\"," +
                    "\"done\": \"false\"" +
                "} ]"));④

    http()
        .client(todoClient)
        .receive()
        .response(HttpStatus.OK.value())
        .messageType(MessageType.XHTML)
        .xpath("(//xh:li[@class='list-group-item']/xh:span)[last()]", "${todoName}");
}
```

① Expects a `INSERT` statement matching the given expression.

② Responds with a result set stating, that one row has been updated/created.

③ Expects a `SELECT` statement matching the given statement.

④ Responds with the DataSet specified as JSON string.

27.2.1. Transactions

When it comes to complex modifications of your database, transactions are commonly used. Citrus is able to verify the behavior of your system under test concerning start, commit and rollback actions of transactions. The verification of transactions has to be enabled in the server Citrus-JDBC-Server configuration. For more information, please have a look at the Configuration section.

Verifying transaction commit

```
receive(jdbcServer)
    .message(JdbcMessage.startTransaction());①

receive(jdbcServer)
    .message(JdbcMessage.execute("@startsWith('INSERT INTO todo_entries (id, title,
description, done) VALUES (?, ?, ?, ?)')@"));

send(jdbcServer)
    .message(JdbcMessage.result().rowsUpdated(1));

receive(jdbcServer)
    .message(JdbcMessage.commitTransaction());②
```

① Verify, that the transaction has been started.

② Verify, that the modification of the database has been committed.

It is also possible to simulate an erroneous modification including the verification of a rollback.

Verifying transaction rollback

```
receive(jdbcServer)
    .message(JdbcMessage.startTransaction());①

receive(jdbcServer)
    .message(JdbcMessage.execute("@startsWith('INSERT INTO todo_entries (id, title,
description, done) VALUES (?, ?, ?, ?)')@"));

send(jdbcServer)
    .message(JdbcMessage.result().exception("Could not execute something"));

receive(jdbcServer)
    .message(JdbcMessage.rollbackTransaction());②
```

① Verify, that the transaction has been started.

② Verify, that a rollback occurred after the database exception has been send.

27.2.2. Prepared statements

Because prepared statements work slightly different than simple database queries, the validation of those is also slightly different. Currently, Citrus offers you the possibility to verify that your application has created the correct prepared statement, that it was executed with the correct parameters and that it has been closed.

Verifying prepared statement

```
receive(jdbcServer)
    .message(JdbcMessage.createPreparedStatement("INSERT INTO todo_entries (id, title,
description, done) VALUES (?, ?, ?, ?)"));①

receive(jdbcServer)
    .message(JdbcMessage.execute(
        "INSERT INTO todo_entries (id, title, description, done) VALUES (?, ?, ?, ?) -
(1,sample,A sample todo,false)"));②

receive(jdbcServer)
    .message(JdbcMessage.closeStatement());③
```

① Verify that the given prepared statement has been created.

② Verify that the statement has been executed with the parameters `1,sample,A sample todo,false`.

③ Verify that the statement has been closed.

Please notice, that the verification of `createPreparedStatement` and `closeStatement` messages has to be activated via configuration. For more information, please have a look at the Configuration section.

27.2.3. Callable statements / stored procedures

As well as prepared statements, callable statements are different from simple queries. Callable statements are used on jdbc level to access stored procedures, functions, etc. on the database server.

Verifying callable statement

```
receive(jdbcServer)
    .message(JdbcMessage.createCallableStatement("{CALL limitedToDoList(?)}"));①

receive(jdbcServer)
    .message(JdbcMessage.execute("{CALL limitedToDoList(?)} - (1)"));②

send(jdbcServer)
    .messageType(MessageType.XML)
    .message(JdbcMessage.result().dataSet("" +
            "<dataset>" +
                "<row>" +
                    "<id>1</id>"+
                    "<title>sample</title>"+
                    "<description>A sample todo</description>" +
                    "<done>false</done>" +
                 "</row>" +
                "</dataset>"));

receive(jdbcServer)
    .message(JdbcMessage.closeStatement());③
```

① Verify that the given callable statement has been created.

② Verify that the statement has been executed with the parameter 1.

③ Verify that the statement has been closed.

As you might have noticed, callable statements and prepared statements have nearly the same workflow in Citrus. The only difference is the creation of the statement itself. It is also the case that the verification of `createCallableStatement` and `closeStatement` messages has to be activated via configuration. For more information, please have a look at the Configuration section.

27.2.4. Configuration

As already mentioned, you're able to configure the JDBC endpoint in different ways (XML, Spring Bean, etc.). The following properties are available to configure the server for your test scenario.

Property	Mandatory	Default	Description
id	Yes		Only required for XML configuration.

Property	Mandatory	Default	Description
auto connect	No	true	Determines whether the server should automatically accept connection related messages or validate them. This includes `openConnection` and `closeConnection`.
auto create statement	No	true	Determines whether the server should automatically accept statement related messages or validate them. This includes `createStatement`, `createPreparedStatement`, `createCallableStatement` and `closeStatement`.
auto transaction handling	No	true	Determines whether the server should automatically accept transaction related messages or validate them. This includes `startTransaction`, `commitTransaction` and `rollbackTransaction`.
auto handle queries	No	Collection of system queries for different databases	Determines whether the server should automatically respond with a positive answer for matching queries, e.g. `SELECT USER FROM DUAL`. You can override the currently defined validation queries when setting `citrus.jdbc.auto.handle.query` system property within the citrus-application.properties. The property value is expected to be a semicolon separated list of regex patterns. Every query can be specified as a regular expression, e.g. `SELECT.*FROM DUAL;SELECT \\w;`.
host	Yes		The hostname of the server. There has to be a valid route between the test suite, the system under test and the database server.
port	No	4567	The port the server listens to.
database name	Yes		The database name to work on
max connections	No	20	The maximum amount of open connections to be accepted by the server.
polling interval	No	500	Polling interval when waiting for synchronous reply message to arrive.

Property	Mandatory	Default	Description
timeout	No	5000	Send/receive timeout setting
debug logging	No	false	Determines whether the inbound channel debug logging should be enabled.

In addition, there are advanced configuration possibilities to customize the behavior of the JDBC server.

Property	Mandatory	Default	Description
correlator	No	DefaultMessageCorrelator	A MessageCorrelator implementation to identify messages.
endpoint adapter	No	JdbcEndpointAdapterController	An Endpoint adapter implementation creating the messages for validation.

27.3. JdbcMessage

The JdbcMessage class is the central location to specifying your expected inbound and outbound communication for the JDBC endpoint.

Message	receive/send	Description
`JdbcMessage.openConnection(Properties properties)`	receive	States that a connection has been opened with the given properties. The evaluation of connections has to be enabled via the endpoint configuration.
`JdbcMessage.closeConnection()`	receive	States that the connection has been closed. The evaluation of connections has to be enabled via the endpoint configuration.
`JdbcMessage.createStatement()`	receive	States that a statement has been created. The evaluation of statement handling has to be enabled via the endpoint configuration.
`JdbcMessage.createPreparedStatement(String sql)`	receive	States that a SQL statement matching the given expression has been created. The evaluation of statement handling has to be enabled via the endpoint configuration.

Message	receive/send	Description
`JdbcMessage.createCallableStatement(String sql)`	receive	States that a callable statement referencing a function or procedure that is matching the given expression has been created. The evaluation of statement handling has to be enabled via the endpoint configuration.
`JdbcMessage.closeStatement()`	receive	States that a statement has been closed. The evaluation of statement handling has to be enabled via the endpoint configuration.
`JdbcMessage.execute(String sql)`	receive	States that a SQL statement matching the given expression has been executed.
`JdbcMessage.startTransaction()`	receive	States that a transaction start has been received. The evaluation of transaction handling has to be enabled via the endpoint configuration.
`JdbcMessage.commitTransaction()`	receive	States that a commit for a transaction has been received. The evaluation of transaction handling has to be enabled via the endpoint configuration.
`JdbcMessage.rollbackTransaction()`	receive	States that a rollback of the transaction has been received. The evaluation of transaction handling has to be enabled via the endpoint configuration.
`JdbcMessage.result()`	send	Sends a positive empty result to the system under test.
`JdbcMessage.result(boolean success)`	send	Sends empty result to the system under test which is a success or a failure based on the given boolean value.
`JdbcMessage.exception(String exceptionText)`	send	Sends an exception to the system under test. Regarding the driver documentation, the error will be an SQLException.

Message	receive/send	Description
`JdbcMessage.rowsUpdated(int number)`	send	Sends a positive result to the system under test where the payload is the number of updated rows.
`JdbcMessage.dataSet(DataSet dataSet)`	send	Sends a positive result to the system under test where the payload is the specified DataSet.
`JdbcMessage.dataSet(String dataSet)`	send	Sends a positive result to the system under test where the payload is the specified DataSet. To use this, you have to specify the format of the dataSet String. Please refer to the section DataSet parsing.
`JdbcMessage.dataSet(Resource dataSet)`	send	Sends a positive result to the system under test where the payload is the content of the specified resource. To use this, you have to specify the format of the dataSet String. Please refer to the section DataSet parsing.
`JdbcMessage.success()`	send	Sends a positive empty result to the system under test.
`JdbcMessage.error()`	send	Sends an empty error result to the system under test.

27.3.1. DataSet parsing

Citrus provides different ways to prepare the response DataSets for your system under test. You can specify your DataSets as Java Objects, as XML or JSON Strings or as resource file containing your XML or JSON DataSet as text.

Java dataset creation example

```
Row sheldon = new Row();
sheldon.getValues().put("id", "1");
sheldon.getValues().put("name", "Sheldon");
sheldon.getValues().put("profession", "physicist");

Row leonard = new Row();
leonard.getValues().put("id", "2");
leonard.getValues().put("name", "Leonard");
leonard.getValues().put("profession", "physicist");
leonard.getValues().put("email", "leo@bigbangtheory.org");

Row penny = new Row();
penny.getValues().put("id", "3");
penny.getValues().put("name", "Penny");
penny.getValues().put("profession", "this_and_that");

Table table = new Table("user");
table.getRows().add(sheldon);
table.getRows().add(leonard);
table.getRows().add(penny);

DataSet userDataSet = new TableDataSetProducer(table).produce();

send(jdbcServer).message(JdbcMessage.result().dataSet(userDataSet));
```

If you use the XML or JSON notation as string or within a resource, you'll have to specify that in your test setup.

Java json dataset creation example

```
receive(jdbcServer)
                .message(JdbcMessage.execute("SELECT id, title, description FROM
todo_entries"));

send(jdbcServer)
        .messageType(MessageType.JSON)①
        .message(JdbcMessage.result().dataSet("[ {" +
                    "\"id\": \"" + UUID.randomUUID().toString() + "\"," +
                    "\"title\": \"${todoName}\"," +
                    "\"description\": \"${todoDescription}\"," +
                    "\"done\": \"false\"" +
                "} ]"));
```

① Tells Citrus that the response has to be interpreted as JSON.

Java xml dataset creation example

```
receive(jdbcServer)
        .message(JdbcMessage.execute("SELECT id, title, description FROM
todo_entries"));
send(jdbcServer)
        .messageType(MessageType.XML)①
        .message(JdbcMessage.result().dataSet("" +
                "<dataset>" +
                    "<row>" +
                        "<id>${todoId}</id>"+
                        "<title>${todoName}</title>"+
                        "<description>${todoDescription}</description>" +
                        "<done>false</done>" +
                     "</row>" +
                "</dataset>"));
```

① Tells Citrus that the response has to be interpreted as XML.



Technically it is not required to specify the messages as `MessageType.XML`, because the default message type in citrus currently **is** XML. Nevertheless we highly recommend to specify the message type. This will ensure that your tests sustain future changes.

Chapter 28. Docker support

Citrus provides configuration components and test actions for interaction with a Docker daemon. The Citrus docker client component will execute Docker commands for container management such as start, stop, build, inspect and so on. The Docker client by default uses the Docker remote REST API. As a user you can execute Docker commands as part of a Citrus test and validate possible command results.



The Docker test components in Citrus are kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-docker</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Citrus provides a "citrus-docker" configuration namespace and schema definition for Docker related components and actions. Include this namespace into your Spring configuration in order to use the Citrus Docker configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as follows.

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-docker="http://www.citrusframework.org/schema/docker/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/docker/config
           http://www.citrusframework.org/schema/docker/config/citrus-docker-
config.xsd">

    [...]

</beans>
```

After that you are able to use customized Citrus XML elements in order to define the Spring beans.

28.1. Docker client

Citrus operates with the Docker remote REST API in order to interact with the Docker daemon. The Docker client is defined as Spring bean component in the configuration as follows:

Citrus Bean

```
@BindToRegistry
public DockerClient dockerClient() {
    return new DockerClientBuilder().build();
}
```

Spring Bean

```
@Bean
public DockerClient dockerClient() {
    return new DockerClientBuilder().build();
}
```

Spring XML

```
<citrus-docker:client id="dockerClient"/>
```

The Docker client component above is using all default configuration values. By default, Citrus is searching the system properties as well as environment variables for default Docker settings such as:

DOCKER_HOST	tcp://localhost:2376
DOCKER_CERT_PATH	~/.docker/machine/machines/default
DOCKER_TLS_VERIFY	1
DOCKER_MACHINE_NAME	default

In case these settings are not settable in your environment you can also use explicit settings in the Docker client component:

Citrus Bean

```
@BindToRegistry
public DockerClient dockerClient() {
    return new DockerClientBuilder()
            .url("tcp://localhost:2376")
            .version("1.20")
            .username("user")
            .password("s!cr!t")
            .email("user@foo.bar")
            .registry("https://index.docker.io/v1/")
            .certPath("/path/to/some/cert/directory")
            .configPath("/path/to/some/config/directory")
            .build();
}
```

Spring Bean

```
@Bean
public DockerClient dockerClient() {
    return new DockerClientBuilder()
            .url("tcp://localhost:2376")
            .version("1.20")
            .username("user")
            .password("s!cr!t")
            .email("user@foo.bar")
            .registry("https://index.docker.io/v1/")
            .certPath("/path/to/some/cert/directory")
            .configPath("/path/to/some/config/directory")
            .build();
}
```

Spring XML

```
<citrus-docker:client id="dockerClient"
            url="tcp://localhost:2376"
            version="1.20"
            username="user"
            password="s!cr!t"
            email="user@foo.bar"
            registry="https://index.docker.io/v1/"
            cert-path="/path/to/some/cert/directory"
            config-path="/path/to/some/config/directory"/>
```

Now Citrus is able to access the Docker remote API for executing commands such as start, stop, build, inspect and so on.

28.2. Docker commands

We have several Citrus test actions each representing a Docker command. These actions can be part of a test case where you can manage Docker containers inside the test.

Java

```
@CitrusTest
public void dockerTest() {
    when(docker().ping());

    then(docker().version()
        .validateCommandResult(new CommandResultCallback<Version>() {
            @Override
            public void doWithCommandResult(Version version, TestContext context) {
                Assert.assertEquals(version.getApiVersion(), "1.21");
            }
    }));
}
```

XML

```
<test name="DockerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <docker>
        <ping/>
      </docker>

      <docker>
        <version/>
      </docker>
    </actions>
</test>
```

YAML

```
name: DockerTest
actions:
  - docker:
      ping: {}

  - docker:
      version: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:docker="http://www.citrusframework.org/schema/docker/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/docker/testcase
              http://www.citrusframework.org/schema/docker/testcase/citrus-docker-
testcase.xsd">
  <testcase name="DockerTest">
    <actions>
      <docker:ping></docker:ping>

      <docker:version>
        <docker:expect>
          <docker:result>
            <![CDATA[
              {
                "Version":"1.8.3",
                "ApiVersion":"1.21",
                "GitCommit":"@ignore@",
                "GoVersion":"go1.4.2",
                "Os":"darwin",
                "Arch":"amd64",
                "KernelVersion":"@ignore@"
              }
            ]]>
          </docker:result>
        </docker:expect>
      </docker:version>
    </actions>
  </testcase>
</spring:beans>
```



We added a special docker namespace in Spring XML with the prefix **docker:** so now we can start to add Docker test actions to the test case:

In this very simple example we first ping the Docker daemon to make sure we have connectivity up and running. After that we get the Docker version information. The second action shows an important concept when executing Docker commands in Citrus. As a tester we might be interested in validating the command result. So we can specify an optional **docker:result** which is usually in JSON data format. As usual we can use test variables here and ignore some values explicitly such as the **GitCommit** value.

Based on that we can execute several Docker commands in a test case:

Java

```
@CitrusTest
public void dockerTest() {
    variable("imageId", "busybox");
    variable("containerName", "citrus_box");

    given(docker().pull()
        .image("${imageId}")
        .tag("latest"));

    when(docker().create()
        .image("${imageId}")
        .containerName("${containerName}")
        .cmd("top")
        .validateCommandResult(new CommandResultCallback<CreateContainerResponse>() {
            @Override
            public void doWithCommandResult(CreateContainerResponse result,
TestContext context) {
                Assert.assertNull(result.getWarnings());
                context.getVariables().put("containerId", result.getId());
            }
        })
    );

    then(docker().start()
        .containerName("${containerName}")
    );
}
```

XML

```
<test name="DockerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <docker>
        <pull/>
      </docker>

      <docker>
        <create/>
      </docker>

      <docker>
        <start/>
      </docker>
    </actions>
</test>
```

YAML

```
name: DockerTest
actions:
  - docker:
      pull: {}

  - docker:
      create: {}

  - docker:
      start: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:docker="http://www.citrusframework.org/schema/docker/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/docker/testcase
              http://www.citrusframework.org/schema/docker/testcase/citrus-docker-
testcase.xsd">
  <testcase name="DockerTest">
    <variables>
      <variable name="imageId" value="busybox"/>
      <variable name="containerName" value="citrus_box"/>
    </variables>

    <actions>
      <docker:pull image="${imageId}"
                   tag="latest"/>

      <docker:create image="${imageId}"
                     name="${containerName}"
                     cmd="top">
        <docker:expect>
            <docker:result>
                <![CDATA[
                  {"Id":"@variable(containerId)@","Warnings":null}
                ]]>
            </docker:result>
        </docker:expect>
      </docker:create>

      <docker:start container="${containerName}"/>
    </actions>
</testcase>
```

In this example we pull a Docker image, build a new container out of this image and start the container. As you can see each Docker command action offers attributes such as **container**, **image** or **tag** . These are command settings that are available on the Docker command specification. Read more about the Docker commands and the specific settings in official Docker API reference guide.

Citrus supports the following Docker commands with respective test actions:

- **docker:pull**
- **docker:build**
- **docker:create**
- **docker:start**
- **docker:stop**
- **docker:wait**
- **docker:ping**
- **docker:version**
- **docker:inspect**
- **docker:remove**
- **docker:info**

Some of the Docker commands can be executed both on container and image targets such as **docker:inspect** or **docker:remove** . The command action then offers both **container** and **image** attributes so the user can choose the target of the command operation to be a container or an image.

The Java DSL Docker commands provide an optional Docker **CommandResultCallback** that is called with the unmarshalled command result object. In the example above the *Version* model object is passed as argument to the callback. So the tester can access the command result and validate its properties with assertions.

By default, Citrus tries to find a Docker client component within the Citrus Spring application context. If not present Citrus will instantiate a default docker client with all default settings. You can also explicitly set the docker client instance when using the Java DSL Docker command actions:

Java DSL

```
@Autowired
private DockerClient dockerClient;

@CitrusTest
public void dockerTest() {
    docker().client(dockerClient).version()
        .validateCommandResult(new CommandResultCallback<Version>() {
            @Override
            public void doWithCommandResult(Version version, TestContext context) {
                Assert.assertEquals(version.getApiVersion(), "1.20");
            }
    });

    docker().client(dockerClient).ping();

    docker().client(dockerClient).start("my_container");
}
```

Chapter 29. SSH support

In the spirit of other Citrus mock services, there is support for simulating an external SSH server as well as for connecting to SSH servers as a client during the test execution. Citrus translates SSH requests and responses to simple XML documents for better validation with the common Citrus mechanisms.

This means that the Citrus test case does not deal with pure SSH protocol commands. Instead of this we use the powerful XML validation capabilities in Citrus when dealing with the simple XML documents that represent the SSH request/response data.

Let us clarify this with a little example. Once the real SSH server daemon is fired up within Citrus we accept an SSH EXEC request for instance. The request is translated into an XML message of the following format:

```
<ssh-request xmlns="http://www.citrusframework.org/schema/ssh/message">
  <command>cat - | sed -e 's/Hello/Hello SSH/'</command>
  <stdin>Hello World</stdin>
</ssh-request>
```

This message can be validated with the usual Citrus mechanism in a receive test action. If you do not know how to do this, please read one of the sections about XML message validation in this reference guide first. Now after having received this request message the respective SSH response should be provided as appropriate answer. This is done with a message sending action on a reply handler as it is known from synchronous http message communication in Citrus for instance. The SSH XML representation of a response message looks like this:

```
<ssh-response xmlns="http://www.citrusframework.org/schema/ssh/message">
  <stdout>Hello SSH World</stdout>
  <stderr></stderr>
  <exit>0</exit>
</ssh-response>
```

Besides simulating a full-featured SSH server, Citrus also provides SSH client functionality. This client uses the same request message pattern, which is translated into a real SSH call to an SSH server. The SSH response received is also translated into an XML message as shown above so we can validate it with known validation mechanisms in Citrus.

Both, SSH server and client along with their configuration options are described in the following two sections.

29.1. SSH Client

A Citrus SSH client is useful for testing against a real SSH server. So Citrus is able to invoke SSH commands on the external server and validate the SSH response accordingly. The test case does not deal with the pure SSH protocol within this communication. The Citrus SSH client component

expects a customized XML representation and automatically translates these request messages into a real SSH call to a specific host. Once the synchronous SSH response was received the result gets translated back to the XML response message representation. On this translated response we can easily apply the validation steps by the usual Citrus means.

The SSH client components receive its configuration in the Spring application context as usual. We can use the special SSH module namespace for easy configuration:

Citrus Bean

```
@BindToRegistry
public SshClient sshClient() {
    return new SshClientBuilder()
        .host("localhost")
        .port(9072)
        .user("roland")
        .privateKeyPath("classpath:org/citrusframework/ssh/test_user.priv")
        .strictHostChecking(false)
        .build();
}
```

Spring Bean

```
@Bean
public SshClient sshClient() {
    return new SshClientBuilder()
        .host("localhost")
        .port(9072)
        .user("roland")
        .privateKeyPath("classpath:org/citrusframework/ssh/test_user.priv")
        .strictHostChecking(false)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-ssh="http://www.citrusframework.org/schema/ssh/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/ssh/config
           http://www.citrusframework.org/schema/ssh/config/citrus-ssh-config.xsd">

    <citrus-ssh:client id="sshClient"
           port="9072"
           user="roland"
           private-key-path="classpath:org/citrusframework/ssh/test_user.priv"
           strict-host-checking="false"
           host="localhost"/>
</beans>
```



Similar to the other Citrus modules (http, soap), a Citrus SSH server and client is configured in Citrus Spring application context. There is a dedicated **ssh** namespace available for all ssh Citrus components. The namespace declaration goes into the Spring XML bean configuration top-level element as usual.

The SSH client receives several attributes, these are:

id	Id identifying the bean and used as reference from with test descriptions. (e.g. id="sshClient")
host	Host to connect to for sending an SSH Exec request. Default is 'localhost' (e.g. host="localhost") port: Port to use. Default is 2222 (e.g. port="9072")
private-key-path	Path to a private key, which can be either a plain file path or a class resource if prefixed with 'classpath' (e.g. private-key-path="classpath:test_user.priv")
private-key-password	Optional password for the private key (e.g. password="s!cr!t")
user	User used for connecting to the SSH server (e.g. user="roland")
password	Password used for password based authentication. Might be combined with "private-key-path" in which case both authentication mechanism are tried (e.g. password="ps!st)

strict-host-checking	Whether the host key should be verified by looking it up in a 'known_hosts' file. Default is false (e.g. strict-host-checking="true")
known-hosts-path	Path to a known hosts file. If prefixed with 'classpath:' this file is looked up as a resource in the classpath (e.g. known-hosts-path="/etc/ssh/known_hosts")
command-timeout	Timeout in milliseconds for how long to wait for the SSH command to complete. Default is 5 minutes (e.g. command-timeout="300000")
connection-timeout	Timeout in milliseconds for how long to for a connectiuon to connect. Default is 1 minute (e.g. connection-timeout="60000")
actor	Actor used for switching groups of actions (e.g. actor="ssh-mock")

Once defines as client component in the Spring application context test cases can reference the client in every send test action.

```
<send endpoint="sshClient">
  <message>
    <payload>
        <ssh-request xmlns="http://www.citrusframework.org/schema/ssh/message">
          <command>shutdown</command>
          <stdin>input</stdin>
        </ssh-request>
    </payload>
  </message>
</send>

<receive endpoint="sshClient">
  <message>
    <payload>
        <ssh-response xmlns="http://www.citrusframework.org/schema/ssh/message">
            <stdout>Hello Citrus</stdout>
            <stderr/>
            <exit>0</exit>
        </ssh-response>
    </payload>
  </message>
</receive>
```

As you can see we use usual send and receive test actions. The XML SSH representation helps us to specify the request and response data for validation. This way you can call SSH commands against an external SSH server and validate the response data.

29.2. SSH Server

Now that we have used Citrus on the client side we can also use Citrus SSH server module in order to provide a full stacked SSH server daemon. We can accept SSH client connections and provide proper response messages as an answer.

Given the above SSH module namespace declaration, adding a new SSH server is quite simple:

Citrus Bean

```
@BindToRegistry
public SshServer sshServer() {
    return new SshServerBuilder()
        .port(9072)
        .user("roland")
        .allowedKeyPath("classpath:org/citrusframework/ssh/test_user_pub.pem")
        .endpointAdapter(sshEndpointAdapter())
        .autoStart(true)
        .build();
}
```

Spring Bean

```
@Bean
public SshServer sshServer() {
    return new SshServerBuilder()
        .port(9072)
        .user("roland")
        .allowedKeyPath("classpath:org/citrusframework/ssh/test_user_pub.pem")
        .endpointAdapter(sshEndpointAdapter())
        .autoStart(true)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-ssh="http://www.citrusframework.org/schema/ssh/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/ssh/config
           http://www.citrusframework.org/schema/ssh/config/citrus-ssh-config.xsd">

    <citrus-ssh:server id="sshServer"
             allowed-key-path="classpath:org/citrusframework/ssh/test_user_pub.pem"
             user="roland"
             port="9072"
             auto-start="true"
             endpoint-adapter="sshEndpointAdapter"/>
</beans>
```

The **endpoint-adapter** is the handler which receives the SSH request as messages (in the request format described above). Endpoint adapter implementations are fully described in http-serverAll adapters described there are supported in SSH server module, too.

The `<citrus-ssh:server>` supports the following attributes:

SSH Server Attributes:

id	Name of the SSH server which identifies it unique within the Citrus Spring context (e.g. id="sshServer")
host-key-path	Path to PEM encoded key pair (public and private key) which is used as host key. By default, a standard, pre-generate, fixed keypair is used. The path can be specified either as a file path, or, if prefixed with **classpath:** is looked up from within the classpath. The path the is relative from to the top-level package, so no leading slash should be used (e.g. hist-key-path="/etc/citrus_ssh_server.pem)
user-home-path	Path to user home directory. If not set ${user.dir}/target/{serverName}/home/{user} is used by default.
user	User which is allowed to connect (e.g. user="roland")

allowed-key-path	Path to an SSH public key stored in PEM format. These are the keys, which are allowed to connect to the SSH server when publickey authentication is used. It seves the same purpose as authorized_keys for standard SSH installations. The path can be specified either as a file path, or, if prefixed with **classpath:** is looked up from within the classpath. The path the is relative from to the top-level package, so no leading slash should be used (e.g. allowed-key-path="classpath:test_user_pub.pem)
password	Password which should be used when password authentication is used. Both publickey authentication and password based authentication can be used together in which case both methods are tried in turn (e.g. password="s!cr!t")
host	Host address (e.g. localhost)
port	Port on which to listen. The SSH server will bind on localhost to this port (e.g. port="9072")
auto-start	Whether to start this SSH server automatically. Default is **true** . If set to **false**, a test action is responsible for starting/stopping the server (e.g. auto-start="true")
endpoint-adapter	Bean reference to an endpoint adapter which processes the incoming SSH request. The message format for the request and response are described above (e.g. endpoint-adapter="sshEndpointAdapter")

Once the SSH server component is added to the Spring application context with a proper endpoint adapter like the MessageChannel forwarding adapter we can receive incoming requests in a test case and provide a respone message for the client.

```
<receive endpoint="sshServer">
  <message>
    <payload>
        <ssh-request xmlns="http://www.citrusframework.org/schema/ssh/message">
           <command>shutdown</command>
           <stdin>input</stdin>
        </ssh-request>
    </payload>
  </message>
</receive>

<send endpoint="sshServer">
  <message>
    <payload>
        <ssh-response xmlns="http://www.citrusframework.org/schema/ssh/message">
            <stdout>Hello Citrus</stdout>
            <exit>0</exit>
        </ssh-response>
    </payload>
  </message>
</send>
```

Chapter 30. RMI support

RMI stands for Remote Method Invocation and is a standard way of calling Java method interfaces where caller and callee (client and server) are not located within the same JVM. So the object passed to the method as argument as well as the method return value are transmitted over the wire.

As a client Citrus is able to connect to some RMI registry that exposes some remote interfaces. As a server Citrus implements such an RMI registry and handles incoming method calls with providing the respective return value.



The RMI components in Citrus are kept in a separate Maven module. So you should check that the module is available as Maven dependency in your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-rmi</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Read the next section in order to find out more about the RMI message support in Citrus.

30.1. RMI client

On the client side we want to call e remote interface. We need to specify the method to call as well as all method arguments. The respective method return value is receivable within the test case for validation. Citrus provides a client component for RMI that sends out service invocation calls.

Citrus Bean

```
@BindToRegistry
public RmiClient rmiClient1() {
    return new RmiClientBuilder()
        .host("localhost")
        .port(1099)
        .binding("newsService")
        .build();
}

@BindToRegistry
public RmiClient rmiClient2() {
    return new RmiClientBuilder()
        .serverUrl("rmi://localhost:1099/newsService")
        .build();
}
```

Spring Bean

```
@Bean
public RmiClient rmiClient1() {
    return new RmiClientBuilder()
        .host("localhost")
        .port(1099)
        .binding("newsService")
        .build();
}

@Bean
public RmiClient rmiClient2() {
    return new RmiClientBuilder()
        .serverUrl("rmi://localhost:1099/newsService")
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-rmi="http://www.citrusframework.org/schema/rmi/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/rmi/config
           http://www.citrusframework.org/schema/rmi/config/citrus-rmi-config.xsd">

    <citrus-rmi:client id="rmiClient1"
          host="localhost"
          port="1099"
          binding="newsService"/>

    <citrus-rmi:client id="rmiClient2"
          server-url="rmi://localhost:1099/newsService"/>
</beans>
```



As usual, Citrus provides a customized rmi configuration schema that is used in Spring XML bean configurations. Simply include the `citrus-rmi` namespace in the configuration XML files as seen in the example above.

The client component in the Spring application context receives host and port configuration of a valid RMI service registry. Either by specifying a proper server url or by giving host, port and binding properties. The service binding is the name of the service that we would like to address in the registry. Now we are ready to use this client referenced by its id or name in a test case for a message sending action.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(send(rmiClient)
        .message(RmiMessage.invocation(NewsService.class, "getNews"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="rmiClient">
            <message>
                <body>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <remote>org.citrusframework.rmi.remote.NewsService</remote>
                          <method>getNews</method>
                        </service-invocation>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - send:
      endpoint: "rmiClient"
      message:
        body: |
          <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
              <remote>org.citrusframework.rmi.remote.NewsService</remote>
              <method>getNews</method>
          </service-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <send endpoint="rmiClient">
                <message>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <remote>org.citrusframework.rmi.remote.NewsService</remote>
                          <method>getNews</method>
                        </service-invocation>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

We are using the usual Citrus send message action referencing the **rmiClient** as endpoint. The message payload is a special Citrus message that defines the service invocation. We define the **remote** interface as well as the **method** to call. Citrus RMI client component will be able to interpret this message content and call the service method.

The method return value is receivable for validation using the very same client endpoint.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(receive(rmiClient)
        .message(RmiMessage.result("This is news from RMI!"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="rmiClient">
            <message>
                <body>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <object type="java.lang.String" value="This is news from
RMI!"/>
                        </service-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - receive:
      endpoint: "rmiClient"
      message:
        body: |
          <service-result xmlns="http://www.citrusframework.org/schema/rmi/message">
              <object type="java.lang.String" value="This is news from RMI!"/>
          </service-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <receive endpoint="rmiClient">
                <message>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <object type="java.lang.String" value="This is news from
RMI!"/>
                        </service-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

In the sample above we receive the service result and expect a **java.lang.String** object return value. The return value content is also validated within the service result payload.

Of course we can also deal with method arguments.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(send(rmiClient)
        .message(RmiMessage.invocation(NewsService.class, "setNews")
              .argument("This is breaking news!"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="rmiClient">
            <message>
                <body>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">

<remote>org.citrusframework.rmi.remote.NewsService</remote>
                            <method>setNews</method>
                            <args>
                              <arg value="This is breaking news!"/>
                            </args>
                        </service-invocation>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - send:
      endpoint: "rmiClient"
      message:
        body: |
          <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                <remote>org.citrusframework.rmi.remote.NewsService</remote>
                <method>setNews</method>
                <args>
                  <arg value="This is breaking news!"/>
                </args>
          </service-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <send endpoint="rmiClient">
                <message>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">

<remote>org.citrusframework.rmi.remote.NewsService</remote>
                            <method>setNews</method>
                            <args>
                              <arg value="This is breaking news!"/>
                            </args>
                        </service-invocation>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

This completes the basic remote service call. Citrus invokes the remote interface method and validates the method return value. As a tester you might also face errors and exceptions when calling the remote interface method. You can catch and assert these remote exceptions verifying your error scenario.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(assertException()
        .exception(java.rmi.RemoteException.class)
        .when(
            send(rmiClient)
                .message(RmiMessage.invocation(NewsService.class, "setNews")
                        .argument("This is breaking news!"))
        )
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <assert exception="java.rmi.RemoteException">
            <when>
                <send endpoint="rmiClient">
                    <message>
                        <body>
                            <payload>
                                <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">

<remote>org.citrusframework.rmi.remote.NewsService</remote>
                                    <method>setNews</method>
                                    <args>
                                      <arg value="This is breaking news!"/>
                                    </args>
                                </service-invocation>
                            </payload>
                        </body>
                    </message>
                </send>
            </when>
        </assert>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - assert:
      exception: "java.rmi.RemoteException"
      when:
        - send:
            endpoint: "rmiClient"
            message:
              body: |
                <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                      <!-- ... -->
                </service-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <assert exception="java.rmi.RemoteException">
                <when>
                    <send endpoint="rmiClient">
                        <message>
                            <payload>
                                <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                                    <!-- ... -->
                                </service-invocation>
                            </payload>
                        </message>
                    </send>
                </when>
            </assert>
        </actions>
    </testcase>
</spring:beans>
```

We assert the ***RemoteException*** to be thrown while calling the remote service method. This is how you can handle some sort of error situation while calling remote services. In the next section we will handle RMI communication where Citrus provides the remote interfaces.

30.2. RMI server

On the server side Citrus needs to provide remote interfaces with methods callable for clients. This means that Citrus needs to support all your remote interfaces with method arguments and return values. The Citrus RMI server is able to bind your remote interfaces to a service registry. All incoming RMI client method calls are automatically accepted and the method arguments are converted into a Citrus XML service invocation representation. The RMI method call is then passed to the running test for validation.

Let us have a look at the Citrus RMI server component and how you can add it to the Spring application context.

Citrus Bean

```
@BindToRegistry
public RmiServer rmiServer() {
    return new RmiServerBuilder()
        .host("localhost")
        .port(1099)
        .remoteInterfaces(org.citrusframework.rmi.remote.NewsService.class)
        .binding("newsService")
        .createRegistry(true)
        .autoStart(true)
        .build();
}
```

Spring Bean

```
@Bean
public RmiServer rmiServer() {
    return new RmiServerBuilder()
        .host("localhost")
        .port(1099)
        .remoteInterfaces(org.citrusframework.rmi.remote.NewsService.class)
        .binding("newsService")
        .createRegistry(true)
        .autoStart(true)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-rmi="http://www.citrusframework.org/schema/rmi/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/rmi/config
           http://www.citrusframework.org/schema/rmi/config/citrus-rmi-config.xsd">

    <citrus-rmi:server id="rmiServer"
          host="localhost"
          port="1099"
          interface="org.citrusframework.rmi.remote.NewsService"
          binding="newService"
          create-registry="true"
          auto-start="true"/>
</beans>
```

The RMI server component uses properties such as **host** and **port** to define the service registry. By

default Citrus will connect to this service registry and bind its remote interfaces to it. With the attribute **create-registry** Citrus can also create the registry for you.

You have to give Citrus the fully qualified remote interface name so Citrus can bind it to the service registry and handle incoming method calls properly. In your test case you can then receive the incoming method calls on the server in order to perform validation steps.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(receive(rmiServer)
        .message(RmiMessage.invocation(NewsService.class, "getNews"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="rmiServer">
            <message>
                <body>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <remote>org.citrusframework.rmi.remote.NewsService</remote>
                          <method>getNews</method>
                        </service-invocation>
                    </payload>
                </body>
                <headers>
                    <header name="citrus_rmi_interface"
value="org.citrusframework.rmi.remote.NewsService"/>
                    <header name="citrus_rmi_method" value="getNews"/>
                </headers>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - receive:
      endpoint: "rmiServer"
      message:
        body: |
          <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
              <remote>org.citrusframework.rmi.remote.NewsService</remote>
              <method>getNews</method>
          </service-invocation>
        headers:
          - name: citrus_rmi_interface
            value: "org.citrusframework.rmi.remote.NewsService"
          - name: citrus_rmi_method
            value: "getNews"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <receive endpoint="rmiServer">
                <message>
                    <payload>
                        <service-invocation
xmlns="http://www.citrusframework.org/schema/rmi/message">
                          <remote>org.citrusframework.rmi.remote.NewsService</remote>
                          <method>getNews</method>
                        </service-invocation>
                    </payload>
                </message>
                <header>
                    <element name="citrus_rmi_interface"
value="org.citrusframework.rmi.remote.NewsService"/>
                    <element name="citrus_rmi_method" value="getNews"/>
                </header>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

As you can see Citrus converts the incoming service invocation to a special XML representation which is passed as message payload to the test. As this is plain XML you can verify the RMI message content as usual using Citrus variables, functions and validation matchers.

Since we have received the method call we need to provide some return value for the client. As

usual we can specify the method return value with some XML representation.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(send(rmiServer)
        .message(RmiMessage.result("This is news from RMI!"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="rmiServer">
            <message>
                <body>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                            <object type="java.lang.String" value="This is news from
RMI!"/>
                        </service-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - send:
      endpoint: "rmiServer"
      message:
        body: |
          <service-result xmlns="http://www.citrusframework.org/schema/rmi/message">
            <object type="java.lang.String" value="This is news from RMI!"/>
          </service-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <send endpoint="rmiServer">
                <message>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                            <object type="java.lang.String" value="This is news from
RMI!"/>
                        </service-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The service result is defined as object with a **type** and **value** . The Citrus RMI remote interface method will return this value to the calling client. This would complete the successful remote service invocation. At this point we also have to think of choosing to raise some remote exception as service outcome.

Java

```
@CitrusTest
public void rmiMessageTest() {
    $(send(rmiServer)
        .message(RmiMessage.exception("Something went wrong"))
    );
}
```

XML

```
<test name="RmiMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="rmiServer">
            <message>
                <body>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                            <exception>Something went wrong</exception>
                        </service-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: RmiMessageTest
actions:
  - send:
      endpoint: "rmiServer"
      message:
        body: |
          <service-result xmlns="http://www.citrusframework.org/schema/rmi/message">
            <exception>Something went wrong</exception>
          </service-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="RmiMessageTest">
        <actions>
            <send endpoint="rmiServer">
                <message>
                    <payload>
                        <service-result
xmlns="http://www.citrusframework.org/schema/rmi/message">
                            <exception>Something went wrong</exception>
                        </service-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

In the example above Citrus will not return some object as service result but raise a **java.rmi.RemoteException** with respective error message as specified in the test case. The calling client will receive the exception accordingly.

Chapter 31. JMX support

JMX is a standard Java API for making beans accessible to others in terms of management and remote configuration. JMX is the short term for Java Management Extensions and is often used in JEE application servers to manage bean attributes and operations from outside (e.g. another JVM). A managed bean server hosts multiple managed beans for JMX access. Remote connections to JMX can be realized with RMI (Remote method invocation) capabilities.

Citrus is able to connect to JMX managed beans as client and server. As a client Citrus can invoke managed bean operations and read write managed bean attributes. As a server Citrus is able to expose managed beans as mbean server. Clients can access those Citrus managed beans and get proper response objects as result. Doing so you can use the JVM platform managed bean server or some RMI registry for providing remote access.



The JMX components in Citrus are kept in a separate Maven module. So you should check that the module is available as Maven dependency in your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-jmx</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Next sections describe the JMX message support in Citrus in more detail.

31.1. JMX client

On the client side we want to call some managed bean by either accessing managed attributes with read/write or by invoking a managed bean operation. For proper mbean server connectivity we should specify a client component for JMX that sends out mbean invocation calls.

Citrus Bean

```
@BindToRegistry
public JmxClient jmxClient() {
    return new JmxClientBuilder()
        .serverUrl("platform")
        .build();
}
```

Spring Bean

```
@Bean
public JmxClient jmxClient() {
    return new JmxClientBuilder()
        .serverUrl("platform")
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jmx="http://www.citrusframework.org/schema/jmx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jmx/config
           http://www.citrusframework.org/schema/jmx/config/citrus-jmx-config.xsd">

    <citrus-jmx:client id="jmxClient"
            server-url="platform"/>
</beans>
```



As usual Citrus provides a customized jmx configuration schema that is used in Spring XML bean configurations. Simply include the `citrus-jmx` namespace in the configuration XML files as seen in the example above.

The client component specifies the target managed bean server that we want to connect to. In this example we are using the JVM platform mbean server. This means we are able to access all JVM managed beans such as Memory, Threading and Logging. In addition to that we can access all custom managed beans that were exposed to the platform mbean server.

In most cases you may want to access managed beans on a different JVM or application server. So we need some remote connection to the foreign mbean server.

Citrus Bean

```
@BindToRegistry
public JmxClient jmxClient() {
    return new JmxClientBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .username("user")
        .password("s!cr!t")
        .autoReconnect(true)
        .delayOnReconnect(5000L)
        .build();
}
```

Spring Bean

```
@Bean
public JmxClient jmxClient() {
    return new JmxClientBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .username("user")
        .password("s!cr!t")
        .autoReconnect(true)
        .delayOnReconnect(5000L)
        .build();
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jmx="http://www.citrusframework.org/schema/jmx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jmx/config
           http://www.citrusframework.org/schema/jmx/config/citrus-jmx-config.xsd">

    <citrus-jmx:client id="jmxClient"
          server-url="service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi"
          username="user"
          password="s!cr!t"
          auto-reconnect="true"
          delay-on-reconnect="5000"/>
</beans>
```

In this example above we connect to a remote mbean server via RMI using the default RMI registry

localhost:1099 and the service name **jmxrmi** . Citrus is able to handle different remote transport protocols. Just define those in the **server-url** .

Now that we have set up the client component we can use it in a test case to access a managed bean.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(send(jmxClient)
        .message(JmxMessage.invocation("java.lang:type=Memory")
            .attribute("Verbose"))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="jmxClient">
            <message>
                <body>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>java.lang:type=Memory</mbean>
                          <attribute name="Verbose"/>
                        </mbean-invocation>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - send:
      endpoint: "jmxClient"
      message:
        body: |
          <mbean-invocation xmlns="http://www.citrusframework.org/schema/jmx/message">
              <mbean>java.lang:type=Memory</mbean>
              <attribute name="Verbose"/>
          </mbean-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <send endpoint="jmxClient">
                <message>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>java.lang:type=Memory</mbean>
                          <attribute name="Verbose"/>
                        </mbean-invocation>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

As you can see we just used a normal send action referencing the jmx client component that we have just added. The message payload is an XML representation of the managed bean access. This is a special Citrus XML representation. Citrus will convert this XML payload to the actuel managed bean access. In the example above we try to access a managed bean with object name **java.lang:type=Memory** . The object name is defined in JMX specification and consists of a key **java.lang:type** and a value **Memory** . So we identify the managed bean on the server by its type.

Now that we have access to the managed bean we can read its managed attributes such as **Verbose** . This is a boolean type attribute so the mbean invocation result will be a respective Boolean object. We can validate the managed bean attribute access in a receive action.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(receive(jmxClient)
        .message(JmxMessage.result(false))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="jmxClient">
            <message>
                <body>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <object type="java.lang.Boolean" value="false"/>
                        </mbean-result>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - receive:
      endpoint: "jmxClient"
      message:
        body: |
          <mbean-result xmlns="http://www.citrusframework.org/schema/jmx/message">
              <object type="java.lang.Boolean" value="false"/>
          </mbean-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <receive endpoint="jmxClient">
                <message>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <object type="java.lang.Boolean" value="false"/>
                        </mbean-result>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

In the sample above we receive the mbean result and expect a **java.lang.Boolean** object return value. The return value content is also validated within the mbean result payload.

Some managed bean attributes might also be settable for us. So we can define the attribute access as write operation by specifying a value in the send action payload.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(send(jmxClient)
        .message(JmxMessage.invocation("java.lang:type=Memory")
            .attribute("Verbose", true))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="jmxClient">
            <message>
                <body>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>java.lang:type=Memory</mbean>
                          <attribute name="Verbose" value="true"
type="java.lang.Boolean"/>
                        </mbean-invocation>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - send:
      endpoint: "jmxClient"
      message:
        body: |
          <mbean-invocation xmlns="http://www.citrusframework.org/schema/jmx/message">
              <mbean>java.lang:type=Memory</mbean>
              <attribute name="Verbose" value="true" type="java.lang.Boolean"/>
          </mbean-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <send endpoint="jmxClient">
                <message>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>java.lang:type=Memory</mbean>
                          <attribute name="Verbose" value="true"
type="java.lang.Boolean"/>
                        </mbean-invocation>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

Now we have write access to the managed attribute **Verbose** . We do specify the value and its type **java.lang.Boolean** . This is how we can set attribute values on managed beans.

Last not least we are able to access managed bean operations.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(send(jmxClient)
        .message(JmxMessage.invocation("org.citrusframework.jmx.mbean:type=HelloBean")
            .operation("sayHello")
            .parameter("Hello JMX!"))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="jmxClient">
            <message>
                <body>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
                          <operation name="sayHello">
                            >parameter>
                              >param type="java.lang.String" value="Hello JMX!"/>
                            >/parameter>
                          >/operation>
                        </mbean-invocation>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - send:
      endpoint: "jmxClient"
      message:
        body: |
          <mbean-invocation xmlns="http://www.citrusframework.org/schema/jmx/message">
              <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
              <operation name="sayHello">
                >parameter>
                  >param type="java.lang.String" value="Hello JMX!"/>
                >/parameter>
              >/operation>
          </mbean-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <send endpoint="jmxClient">
                <message>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
                          <operation name="sayHello">
                            >parameter>
                              >param type="java.lang.String" value="Hello JMX!"/>
                            >/parameter>
                          >/operation>
                        </mbean-invocation>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

In the example above we access a custom managed bean and invoke its operation **sayHello** . We are also using operation parameters for the invocation. This should call the managed bean operation and return its result if any as usual.

This completes the basic JMX managed bean access as client. Now we also want to discuss the server side were Citrus is able to provide managed beans for others

31.2. JMX server

The server side is always a little bit more tricky because we need to simulate custom managed bean access as a server. First of all Citrus provides a server component that specifies the connection properties for clients such as transport protocols, ports and mbean object names. Let's create a new server that accepts incoming requests via RMI on a remote registry **localhost:1099** .

Citrus Bean

```
@BindToRegistry
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        helloBean(),
        newsBean()
    );
}

private ManagedBeanDefinition helloBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.HelloBean.class);
    return managedBeanDefinition;
}

private ManagedBeanDefinition newsBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.NewsBean.class);
    managedBeanDefinition.setObjectDomain("org.citrusframework.news");
    managedBeanDefinition.setObjectName("name=News");
    return managedBeanDefinition;
}
```

Spring Bean

```
@Bean
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        helloBean(),
        newsBean()
    );
}

private ManagedBeanDefinition helloBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.HelloBean.class);
    return managedBeanDefinition;
}

private ManagedBeanDefinition newsBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.NewsBean.class);
    managedBeanDefinition.setObjectDomain("org.citrusframework.news");
    managedBeanDefinition.setObjectName("name=News");
    return managedBeanDefinition;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jmx="http://www.citrusframework.org/schema/jmx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jmx/config
           http://www.citrusframework.org/schema/jmx/config/citrus-jmx-config.xsd">

    <citrus-jmx:server id="jmxServer"
        server-url="service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi">
        <citrus-jmx:mbeans>
            <citrus-jmx:mbean type="org.citrusframework.jmx.mbean.HelloBean"/>
            <citrus-jmx:mbean type="org.citrusframework.jmx.mbean.NewsBean"
objectDomain="org.citrusframework.news" objectName="name=News"/>
        </citrus-jmx:mbeans>
    </citrus-jmx:server>
</beans>
```

As usual, we define a **server-url** that controls the JMX connector access to the mbean server. In this example above we open a JMX RMI connector for clients using the registry **localhost:1099** and the service name **jmxrmi** By default Citrus will not attempt to create this registry automatically so the registry has to be present before the server start up. With the optional server property **create-registry** set to **true** you can auto create the registry when the server starts up. These properties do only apply when using a remote JMX connector server.

Besides using the whole server-url as property we can also construct the connection by host, port, protocol and binding properties.

Citrus Bean

```
@BindToRegistry
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .host("localhost")
        .port(1099)
        .protocol("rmi")
        .binding("jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        helloBean(),
        newsBean()
    );
}

private ManagedBeanDefinition helloBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.HelloBean.class);
    return managedBeanDefinition;
}

private ManagedBeanDefinition newsBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.NewsBean.class);
    managedBeanDefinition.setObjectDomain("org.citrusframework.news");
    managedBeanDefinition.setObjectName("name=News");
    return managedBeanDefinition;
}
```

Spring Bean

```
@Bean
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .host("localhost")
        .port(1099)
        .protocol("rmi")
        .binding("jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        helloBean(),
        newsBean()
    );
}

private ManagedBeanDefinition helloBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.HelloBean.class);
    return managedBeanDefinition;
}

private ManagedBeanDefinition newsBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setType(org.citrusframework.jmx.mbean.NewsBean.class);
    managedBeanDefinition.setObjectDomain("org.citrusframework.news");
    managedBeanDefinition.setObjectName("name=News");
    return managedBeanDefinition;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jmx="http://www.citrusframework.org/schema/jmx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jmx/config
           http://www.citrusframework.org/schema/jmx/config/citrus-jmx-config.xsd">

    <citrus-jmx:server id="jmxServer"
          host="localhost"
          port="1099"
          protocol="rmi"
          binding="jmxrmi">
        <citrus-jmx:mbeans>
            <citrus-jmx:mbean type="org.citrusframework.jmx.mbean.HelloBean"/>
            <citrus-jmx:mbean type="org.citrusframework.jmx.mbean.NewsBean"
objectDomain="org.citrusframework.news" objectName="name=News"/>
        </citrus-jmx:mbeans>
    </citrus-jmx:server>
</beans>
```

On last thing to mention is that we could have also used **platform** as server-url in order to use the JVM platform mbean server instead.

Now that we clarified the connectivity we need to talk about how to define the managed beans that are available on our JMX mbean server. This is done as nested **mbean** configuration elements. Here the managed bean definitions describe the managed bean with its objectDomain, objectName, operations and attributes. The most convenient way of defining such managed bean definitions is to give a bean type which is the fully qualified class name of the managed bean. Citrus will use the package name and class name for proper objectDomain and objectName construction.

Let's have a closer look at the first mbean definition in the example above. So the first managed bean is defined by its class name **org.citrusframework.jmx.mbean.HelloBean** and therefore is accessible using the objectName **org.citrusframework.jmx.mbean:type=HelloBean** . In addition to that Citrus will read the class information such as available methods, getters and setters for constructing a proper MBeanInfo. In the second managed bean definition in our example we have used additional custom objectDomain and objectName values. So the **NewsBean** will be accessible with **org.citrusframework.news:name=News** on the managed bean server.

This is how we can define the bindings of managed beans and what clients need to search for when finding and accessing the managed beans on the server. When clients try to find the managed beans they have to use proper objectNames accordingly. ObjectNames that are not defined on the server will be rejected with managed bean not found error.

Right now we have to use the qualified class name of the managed bean in the definition. What happens if we do not have access to that mbean class or if there is not managed bean interface available at all? Citrus provides a generic managed bean that is able to handle any managed bean interaction. The generic bean implementation needs to know the managed operations and attributes though. So let's define a new generic managed bean on our server:

Citrus Bean

```
@BindToRegistry
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        fooBean()
    );
}

private ManagedBeanDefinition fooBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setObjectDomain("foo.object.domain");
    managedBeanDefinition.setObjectName("type=FooBean");

    ManagedBeanInvocation.Operation fooOp = new ManagedBeanInvocation.Operation();
    fooOp.setName("fooOp");

    ManagedBeanInvocation.Parameter fooOpParams = new
ManagedBeanInvocation.Parameter();
    OperationParam p1 = new OperationParam();
    p1.setType(String.class.getName());
    p1.setType(Integer.class.getName());
    fooOpParams.getParameter().add(p1);
    fooOp.setParameter(fooOpParams);

    ManagedBeanInvocation.Operation barOp = new ManagedBeanInvocation.Operation();
    barOp.setName("barOp");

    managedBeanDefinition.getOperations().add(fooOp);
    managedBeanDefinition.getOperations().add(barOp);

    ManagedBeanInvocation.Attribute fooAttr = new ManagedBeanInvocation.Attribute();
    fooAttr.setName("fooAttr");
    fooAttr.setType(String.class.getName());

    ManagedBeanInvocation.Attribute barAttr = new ManagedBeanInvocation.Attribute();
    barAttr.setName("barAttr");
    barAttr.setType(Boolean.class.getName());

    managedBeanDefinition.getAttributes().add(fooAttr);
    managedBeanDefinition.getAttributes().add(barAttr);

    return managedBeanDefinition;
}
```

Spring Bean

```
@Bean
public JmxServer jmxServer() {
    return new JmxServerBuilder()
        .serverUrl("service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi")
        .mbeans(jmxBeans())
        .build();
}

private List<ManagedBeanDefinition> jmxBeans() {
    return Arrays.asList(
        fooBean()
    );
}

private ManagedBeanDefinition fooBean() {
    ManagedBeanDefinition managedBeanDefinition = new ManagedBeanDefinition();
    managedBeanDefinition.setObjectDomain("foo.object.domain");
    managedBeanDefinition.setObjectName("type=FooBean");

    ManagedBeanInvocation.Operation fooOp = new ManagedBeanInvocation.Operation();
    fooOp.setName("fooOp");

    ManagedBeanInvocation.Parameter fooOpParams = new
ManagedBeanInvocation.Parameter();
    OperationParam p1 = new OperationParam();
    p1.setType(String.class.getName());
    p1.setType(Integer.class.getName());
    fooOpParams.getParameter().add(p1);
    fooOp.setParameter(fooOpParams);

    ManagedBeanInvocation.Operation barOp = new ManagedBeanInvocation.Operation();
    barOp.setName("barOp");

    managedBeanDefinition.getOperations().add(fooOp);
    managedBeanDefinition.getOperations().add(barOp);

    ManagedBeanInvocation.Attribute fooAttr = new ManagedBeanInvocation.Attribute();
    fooAttr.setName("fooAttr");
    fooAttr.setType(String.class.getName());

    ManagedBeanInvocation.Attribute barAttr = new ManagedBeanInvocation.Attribute();
    barAttr.setName("barAttr");
    barAttr.setType(Boolean.class.getName());

    managedBeanDefinition.getAttributes().add(fooAttr);
    managedBeanDefinition.getAttributes().add(barAttr);

    return managedBeanDefinition;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-jmx="http://www.citrusframework.org/schema/jmx/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd
           http://www.citrusframework.org/schema/jmx/config
           http://www.citrusframework.org/schema/jmx/config/citrus-jmx-config.xsd">

    <citrus-jmx:server id="jmxServer"
        server-url="service:jmx:rmi:///jndi/rmi://localhost:1099/jmxrmi">
        <citrus-jmx:mbeans>
            <citrus-jmx:mbean name="fooBean" objectDomain="foo.object.domain"
objectName="type=FooBean">
                <citrus-jmx:operations>
                    <citrus-jmx:operation name="fooOp">
                        <citrus-jmx:parameter>
                            <citrus-jmx:param type="java.lang.String"/>
                            <citrus-jmx:param type="java.lang.Integer"/>
                        </citrus-jmx:parameter>
                    </citrus-jmx:operation>
                    <citrus-jmx:operation name="barOp"/>
                </citrus-jmx:operations>
                <citrus-jmx:attributes>
                    <citrus-jmx:attribute name="fooAttr" type="java.lang.String"/>
                    <citrus-jmx:attribute name="barAttr" type="java.lang.Boolean"/>
                </citrus-jmx:attributes>
            </citrus-jmx:mbean>
        </citrus-jmx:mbeans>
    </citrus-jmx:server>
</beans>
```

The generic bean definition needs to define all operations and attributes that are available for access. Up to now we are restricted to using Java base types when defining operation parameter and attribute return types. There is actually no way to define more complex return types. Nevertheless Citrus is now able to expose the managed bean for client access without having to know the actual managed bean implementation.

Now we can use the server component in a test case to receive some incoming managed bean access.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(receive(jmxServer)
        .message(JmxMessage.invocation("org.citrusframework.jmx.mbean:type=HelloBean")
            .operation("sayHello")
            .parameter("Hello JMX!"))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="jmxServer">
            <message>
                <body>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
                          <operation name="sayHello">
                            >parameter>
                              >param type="java.lang.String" value="Hello JMX!"/>
                            >/parameter>
                          </operation>
                        </mbean-invocation>
                    </payload>
                </body>
            </message>
        </receive>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - receive:
      endpoint: "jmxServer"
      message:
        body: |
          <mbean-invocation xmlns="http://www.citrusframework.org/schema/jmx/message">
              <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
              <operation name="sayHello">
                >parameter>
                  >param type="java.lang.String" value="Hello JMX!"/>
                >/parameter>
              </operation>
          </mbean-invocation>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <receive endpoint="jmxServer">
                <message>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.jmx.mbean:type=HelloBean</mbean>
                          <operation name="sayHello">
                            >parameter>
                              >param type="java.lang.String" value="Hello JMX!"/>
                            >/parameter>
                          </operation>
                        </mbean-invocation>
                    </payload>
                </message>
            </receive>
        </actions>
    </testcase>
</spring:beans>
```

In this very first example we expect a managed bean access to the bean **org.citrusframework.jmx.mbean:type=HelloBean** . We further expect the operation **sayHello** to be called with respective parameter values. Now we have to define the operation result that will be returned to the calling client as operation result.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(send(jmxServer)
        .message(JmxMessage.result("Hello from JMX!"))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="jmxServer">
            <message>
                <body>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                            <object type="java.lang.String" value="Hello from JMX!"/>
                        </mbean-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - send:
      endpoint: "jmxServer"
      message:
        body: |
          <mbean-result xmlns="http://www.citrusframework.org/schema/jmx/message">
            <object type="java.lang.String" value="Hello from JMX!"/>
          </mbean-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <send endpoint="jmxServer">
                <message>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                            <object type="java.lang.String" value="Hello from JMX!"/>
                        </mbean-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The operation returns a String **Hello from JMX!** . This is how we can expect operation calls on managed beans. Now we already have seen that managed beans also expose attributes. The next example is handling incoming attribute read access.

Java

```
@CitrusTest
public void jmxMessageTest() {
    $(receive(jmxServer)
        .message(JmxMessage.invocation("org.citrusframework.news:name=News")
            .attribute("newsCount"))
    );

    $(send(jmxServer)
        .message(JmxMessage.result(100))
    );
}
```

XML

```
<test name="JmxMessageTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="jmxServer">
            <message>
                <body>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.news:name=News</mbean>
                            >attribute name="newsCount"/>
                        </mbean-invocation>
                    </payload>
                </body>
            </message>
        </receive>
        <send endpoint="jmxServer">
            <message>
                <body>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                            <object type="java.lang.Integer" value="100"/>
                        </mbean-result>
                    </payload>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: JmxMessageTest
actions:
  - receive:
      endpoint: "jmxServer"
      message:
        body: |
          <mbean-invocation xmlns="http://www.citrusframework.org/schema/jmx/message">
              <mbean>org.citrusframework.news:name=News</mbean>
                >attribute name="newsCount"/>
          </mbean-invocation>
  - send:
      endpoint: "jmxServer"
      message:
        body: |
          <mbean-result xmlns="http://www.citrusframework.org/schema/jmx/message">
            <object type="java.lang.Integer" value="100"/>
          </mbean-result>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="JmxMessageTest">
        <actions>
            <receive endpoint="jmxServer">
                <message>
                    <payload>
                        <mbean-invocation
xmlns="http://www.citrusframework.org/schema/jmx/message">
                          <mbean>org.citrusframework.news:name=News</mbean>
                            >attribute name="newsCount"/>
                        </mbean-invocation>
                    </payload>
                </message>
            </receive>

            <send endpoint="jmxServer">
                <message>
                    <payload>
                        <mbean-result
xmlns="http://www.citrusframework.org/schema/jmx/message">
                            <object type="java.lang.Integer" value="100"/>
                        </mbean-result>
                    </payload>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The `receive` action expects read access to the **NewsBean** attribute **newsCount** and returns a result object of type **java.lang.Integer** . This way we can expect all attribute access to our managed beans. Write operations will have an attribute value specified.

This completes the JMX server capabilities with managed bean access on operations and attributes.

Chapter 32. Zookeeper support

Citrus provides configuration components and test actions for interacting with Zookeeper. The Citrus Zookeeper client component executes commands like create-node, check node-exists, delete-node, get node-data or set node-data. As a user you can execute Zookeeper commands as part of a Citrus test and validate possible command results.



The Zookeeper test components in Citrus are kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
      <groupId>org.citrusframework</groupId>
      <artifactId>citrus-zookeeper</artifactId>
      <version>${citrus.version}</version>
      </dependency>
```

32.1. Zookeeper client

Before you can interact with a Zookeeper server you have to configure the Zookeeper client. A sample configuration is provided below describing the configuration options available:

Citrus Bean

```
@BindToRegistry
public ZooClient zookeeperClient() {
    ZooClientConfig configuration = new ZooClientConfig();
    configuration.setUrl("http://localhost:21118");
    configuration.setTimeout(2000L);
    return new ZooClient(configuration);
}
```

Spring Bean

```
@Bean
public ZooClient zookeeperClient() {
    ZooClientConfig configuration = new ZooClientConfig();
    configuration.setUrl("http://localhost:21118");
    configuration.setTimeout(2000L);
    return new ZooClient(configuration);
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-zookeeper="http://www.citrusframework.org/schema/zookeeper/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/zookeeper/config
           http://www.citrusframework.org/schema/zookeeper/config/citrus-zookeeper-
config.xsd">

    <citrus-zookeeper:client id="zookeeperClient"
                             url="http://localhost:21118"
                             timeout="2000"/>
</beans>
```



Citrus provides a "citrus-zookeeper" configuration namespace and schema definition for Zookeeper related components and actions. Include this namespace into your Spring XML bean configuration in order to use the Citrus zookeeper configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as seen in the example above.

This is a typical client configuration for connecting to a Zookeeper server. Now you are able to execute several commands. These commands will be sent to the Zookeeper server for execution.

32.2. Zookeeper commands

See below all available Zookeeper commands that a Citrus client is able to execute.

```
info: Retrieves the current state of the client connection
create: Creates a znode in a specified path of the ZooKeeper namespace
delete: Deletes a znode from a specified path of the ZooKeeper namespace
exists: Checks if a znode exists in the path
children: Gets a list of children of a znode
get: Gets the data associated with a znode
set: Sets/writes data into the data field of a znode
```

The next example shows some of these features in action.

Java

```
@CitrusTest
public void testZookeeper() {
    variable("randomString", "citrus:randomString(10)");

    zookeeper()
        .create("/${randomString}", "foo")
        .acl("OPEN_ACL_UNSAFE")
        .mode("PERSISTENT")
        .validateCommandResult(new CommandResultCallback<ZooResponse>() {
            @Override
            public void doWithCommandResult(ZooResponse result, TestContext context) {
                Assert.assertEquals(result.getResponseData().get("path"),
context.replaceDynamicContentInString("/${randomString}"));
            }
        });

    zookeeper()
        .get("/${randomString}")
        .validateCommandResult(new CommandResultCallback<ZooResponse>() {
            @Override
            public void doWithCommandResult(ZooResponse result, TestContext context) {
                Assert.assertEquals(result.getResponseData().get("version"), 0);
            }
        });

    zookeeper()
        .set("/${randomString}", "bar");
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:zookeeper="http://www.citrusframework.org/schema/zookeeper/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/zookeeper/testcase
              http://www.citrusframework.org/schema/zookeeper/testcase/citrus-
zookeeper-testcase.xsd">

  <testcase name="ScpTest">
    <actions>
        <zookeeper:create zookeeper-client="zookeeperClient" path="/${randomString}"
acl="OPEN_ACL_UNSAFE" mode="PERSISTENT">
          <zookeeper:data>foo</zookeeper:data>
          <zookeeper:expect>
            <zookeeper:result>
              <![CDATA[
                {
                  "responseData":{
                      "path":"/${randomString}"
                  }
                }
              ]]>
            </zookeeper:result>
          </zookeeper:expect>
        </zookeeper:create>

        <zookeeper:get zookeeper-client="zookeeperClient" path="/${randomString}">
          <zookeeper:expect>
            <zookeeper:result>
              <![CDATA[
              {
                "responseData":{
                  "data":"foo"
                }
              }
              ]]>
            </zookeeper:result>
          </zookeeper:expect>
        </zookeeper:getData>

        <zookeeper:set zookeeper-client="zookeeperClient" path="/${randomString}">
          <zookeeper:data>bar</zookeeper:data>
        </zookeeper:setData>
    </actions>
  </testcase>
</spring:beans>
```



We added the Zookeeper specific namespace with prefix **zookeeper:** in the Spring XML application context, so now we can start to add special Zookeeper related test actions to the test case.

The examples above create a new znode in Zookeeper using a **randomString** as path. We can get and set the data with expecting and validating the result of the Zookeeper server. This is basically the idea of integrating Zookepper operations to a Citrus test. This opens the gate to manage Zookeeper related entities within a Citrus test. We can manipulate and validate the znodes on the Zookeeper instance.

Zookeeper keeps its nodes in a hierarchical storage. This means a znode can have children and we can add and remove those. In Citrus you can get all children of a znode and manage those within the test:

Java

```
@CitrusTest
public void testZookeeper() {
    zookeeper()
        .create("/${randomString}/child1", "")
        .acl("OPEN_ACL_UNSAFE")
        .mode("PERSISTENT")
        .validateCommandResult(new CommandResultCallback<ZooResponse>() {
            @Override
            public void doWithCommandResult(ZooResponse result, TestContext context) {
                Assert.assertEquals(result.getResponseData().get("path"),
context.replaceDynamicContentInString("/${randomString}/child1"));
            }
        });

    zookeeper()
        .create("/${randomString}/child2", "")
        .acl("OPEN_ACL_UNSAFE")
        .mode("PERSISTENT")
        .validateCommandResult(new CommandResultCallback<ZooResponse>() {
            @Override
            public void doWithCommandResult(ZooResponse result, TestContext context) {
                Assert.assertEquals(result.getResponseData().get("path"),
context.replaceDynamicContentInString("/${randomString}/child2"));
            }
        });

    zookeeper()
        .children("/${randomString}")
        .validateCommandResult(new CommandResultCallback<ZooResponse>() {
            @Override
            public void doWithCommandResult(ZooResponse result, TestContext context) {

Assert.assertEquals(result.getResponseData().get("children").toString(), "[child1,
child2]");
            }
        });
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xmlns:zookeeper="http://www.citrusframework.org/schema/zookeeper/testcase"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
              http://www.citrusframework.org/schema/zookeeper/testcase
```

```
              http://www.citrusframework.org/schema/zookeeper/testcase/citrus-
zookeeper-testcase.xsd">

  <testcase name="ScpTest">
    <actions>
        <zookeeper:create zookeeper-client="zookeeperClient"
path="/${randomString}/child1" acl="OPEN_ACL_UNSAFE" mode="EPHEMERAL">
          <zookeeper:data></zookeeper:data>
          <zookeeper:expect>
            <zookeeper:result>
              <![CDATA[
                {
                  "responseData":{
                      "path":"/${randomString}/child1"
                  }
                }
              ]]>
            </zookeeper:result>
          </zookeeper:expect>
        </zookeeper:create>

        <zookeeper:create zookeeper-client="zookeeperClient"
path="/${randomString}/child2" acl="OPEN_ACL_UNSAFE" mode="EPHEMERAL">
          <zookeeper:data></zookeeper:data>
          <zookeeper:expect>
            <zookeeper:result>
              <![CDATA[
                {
                  "responseData":{
                      "path":"/${randomString}/child2"
                  }
                }
              ]]>
            </zookeeper:result>
          </zookeeper:expect>
        </zookeeper:create>

        <zookeeper:children zookeeper-client="zookeeperClient"
path="/${randomString}">
          <zookeeper:expect>
            <zookeeper:result>
              <![CDATA[
                {
                  "responseData":{
                      "children":["child1","child2"]
                  }
                }
              ]]>
            </zookeeper:result>
          </zookeeper:expect>
        </zookeeper:children>
```

```
    </actions>
  </testcase>
</spring:beans>
```

Chapter 33. Dynamic endpoint components

Endpoints represent the central components in Citrus to send or receive a message on some destination. Usually endpoints get defined in the basic Citrus Spring application context configuration as Spring bean components. In some cases this might be over engineering as the tester just wants to send or receive a message. In particular this is done when doing sanity checks in server endpoints while debugging a certain scenario.

With endpoint components you are able to create the Citrus endpoint for sending and receiving a message at test runtime. There is no additional configuration or Spring bean component needed. You just use the endpoint uri in a special naming convention and Citrus will create the endpoint for you. Let us see a first example of this scenario:

```
<testcase name="DynamicEndpointTest">
    <actions>
        <send endpoint="jms:Hello.Queue?timeout=10000">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </send>

        <receive endpoint="jms:Hello.Response.Queue?timeout=5000">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </receive>
    </actions>
</testcase>
```

As you can see the endpoint uri just goes into the test case action in substitution to the usual endpoint reference name. Instead of referencing a bean id that points to the previously configured Citrus endpoint we use the endpoint uri directly. The endpoint uri should give all information to create the endpoint at runtime. In the example above we use a keyword **jms:** which tells Citrus that we need to create a JMS message endpoint. Secondly we give the JMS destination name **Hello.Queue** which is a mandatory part of the endpoint uri when using the JMS component. The optional timeout parameter completed the uri. Citrus is able to create the JMS endpoint at runtime sending the message to the defined destination via JMS.

Of course this mechanism is not limited to JMS endpoints. We can use all default Citrus message transports in the endpoint uri. Just pick the right keyword that defines the message transport to use. Here is a list of supported keywords:

jms	Creates a JMS endpoint for sending and receiving message to a queue or topic

channel	Creates a channel endpoint for sending and receiving messages using an in memory Spring Integration message channel
http	Creates an HTTP client for sending a request to some server URL synchronously waiting for the response message
ws	Creates a Web Socket client for sending messages to or receiving messages from a Web Socket server
soap	Creates a SOAP WebService client that send a proper SOAP message to the server URL and waits for the synchronous response to arrive
ssh	Creates a new ssh client for publishing a command to the server
mail	or smtp: Creates a new mail client for sending a mail mime message to an SMTP server
camel	Creates a new Apache Camel endpoint for sending and receiving Camel exchanges both to and from Camel routes.
vertx	or eventbus: Creates a new Vert.x instance sending and receiving messages with the network event bus
rmi	Creates a new RMI client instance sending and receiving messages for method invocation on remote interfaces
jmx	Creates a new JMX client instance sending and receiving messages to and from a managed bean server.

Depending on the message transport we have to add mandatory parameters to the endpoint uri. In the JMS example we had to specify the destination name. The mandatory parameters are always part of the endpoint uri. Optional parameters can be added as key value pairs to the endpoint uri. The available parameters depend on the endpoint keyword that you have chosen. See these example endpoint uri expressions:

```
    jms:queuename?connectionFactory=specialConnectionFactory&timeout=10000
    jms:topic:topicname?connectionFactory=topicConnectionFactory

jms:sync:queuename?connectionFactory=specialConnectionFactory&pollingInterval=100&repl
yDestination=myReplyDestination

    channel:channelName
    channel:sync:channelName
    channel:channelName?timeout=10000&channelResolver=myChannelResolver

    http:localhost:8088/test
```

```
    http://localhost:8088/test

http:localhost:8088?requestMethod=GET&timeout=10000&errorHandlingStrategy=throwsExcept
ion&requestFactory=myRequestFactory
    http://localhost:8088/test?requestMethod=DELETE&customParam=foo

    websocket:localhost:8088/test
    websocket://localhost:8088/test
    ws:localhost:8088/test
    ws://localhost:8088/test

    soap:localhost:8088/test

soap:localhost:8088?timeout=10000&errorHandlingStrategy=propagateError&messageFactory=
myMessageFactory

    mail:localhost:25000
    smtp://localhost:25000

smtp://localhost?timeout=10000&username=foo&password=1234&mailMessageMapper=myMapper

    ssh:localhost:2200

ssh://localhost:2200?timeout=10000&strictHostChecking=true&user=foo&password=12345678

    rmi://localhost:1099/someService
    rmi:localhost/someService&timeout=10000

    jmx:rmi:///jndi/rmi://localhost:1099/someService
    jmx:platform&timeout=10000

    camel:direct:address
    camel:seda:address
    camel:jms:queue:someQueue?connectionFactory=myConnectionFactory

camel:activemq:queue:someQueue?concurrentConsumers=5&destination.consumer.prefetchSize
=50
    camel:controlbus:route?routeId=myRoute&action=status

    vertx:addressName
    vertx:addressName?port=10105&timeout=10000&pubSubDomain=true
    vertx:addressName?vertxInstanceFactory=vertxFactory
```

The optional parameters get directly set as endpoint configuration. You can use primitive values as well as Spring bean id references. Citrus will automatically detect the target parameter type and resolve the value to a Spring bean in the application context if necessary. If you use some unknown parameter Citrus will raise an exception at runtime as the endpoint could not be created properly.

In synchronous communication we have to reuse endpoint components in order to receive synchronous messages on reply destinations. This is a problem when using dynamic endpoints as

the endpoints get created at runtime. Citrus uses a caching of endpoints that get created at runtime. Following from that we have to use the exact same endpoint uri in your test case in order to get the cached endpoint instance. With this little trick synchronous communication will work just as it is done with static endpoint components. Have a look at this sample test:

```
<testcase name="DynamicEndpointTest">
    <actions>
        <send endpoint="jms:sync:Hello.Sync.Queue">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </send>

        <receive endpoint="jms:sync:Hello.Sync.Queue">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </receive>
    </actions>
</testcase>
```

As you can see we used the exact dynamic endpoint uri in both send and receive actions. Citrus is then able to reuse the same dynamic endpoint and the synchronous reply will be received as expected. However the reuse of exactly the same endpoint uri might get annoying as we also have to copy endpoint uri parameters and so on.

```
<testcase name="DynamicEndpointTest">
    <actions>
        <send endpoint="http://localhost:8080/HelloService?user=1234567">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </send>

        <receive endpoint="http://localhost:8080/HelloService?user=1234567">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </receive>
    </actions>
</testcase>
```

We have to use the exact same endpoint uri when receiving the synchronous service response. This is not very straight forward. This is why Citrus also supports dynamic endpoint names. With a special endpoint uri parameter called **endpointName** you can name the dynamic endpoint. In a corresponding receive action you just use the endpoint name as reference which makes life easier:

```
<testcase name="DynamicEndpointTest">
    <actions>
        <send endpoint="http://localhost:8080/HelloService?endpointName=myHttpClient">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </send>

        <receive endpoint="http://localhost?endpointName=myHttpClient">
            <message>
                <payload>
                [...]
                </payload>
            </message>
        </receive>
    </actions>
</testcase>
```

So we can reference the dynamic endpoint with the given name. The internal **endpointName** uri parameter is automatically removed before sending out messages. Once again the dynamic endpoint uri mechanism provides a fast way to write test cases in Citrus with less configuration.

But you should consider to use the static endpoint components defined in the basic Spring bean application context for endpoints that are heavily reused in multiple test cases.

Chapter 34. Endpoint adapter

Endpoint adapter help to customize the behavior of a Citrus server such as HTTP or SOAP web servers. As the servers get started with the Citrus context they are ready to receive incoming client requests. Now there are different ways to process these incoming requests and to provide a proper response message. By default the server will forward the incoming request to an in-memory message channel where a test can receive the message and provide a synchronous response. This message channel handling is done automatically behind the scenes so the tester does not care about these things. The tester just uses the server directly as endpoint reference in the test case. This is the default behaviour. In addition to that you can define custom endpoint adapters on the Citrus server in order to change this default behavior.

You set the custom endpoint adapter directly on the server configuration as follows:

Citrus Bean

```
@BindToRegistry
public HttpServer helloHttpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .endpointAdapter("emptyResponseEndpointAdapter")
        .build();
}

@BindToRegistry
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring Bean

```
@Bean
public HttpServer helloHttpServer() {
    return new HttpServerBuilder()
        .port(8080)
        .autoStart(true)
        .endpointAdapter("emptyResponseEndpointAdapter")
        .build();
}

@Bean
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring XML

```
<citrus-http:server id="helloHttpServer"
    port="8080"
    auto-start="true"
    endpoint-adapter="emptyResponseEndpointAdapter"/>

<citrus:empty-response-adapter id="emptyResponseEndpointAdapter"/>
```

Now let us have a closer look at the provided endpoint adapter implementations.

34.1. Empty response endpoint adapter

This is the simplest endpoint adapter you can think of. It simply provides an empty success response using the HTTP response code **200** . The adapter does not need any configurations or properties as it simply responds with an empty HTTP response.

Citrus Bean

```
@BindToRegistry
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring Bean

```
@Bean
public EmptyResponseEndpointAdapter emptyResponseEndpointAdapter() {
    return new EmptyResponseEndpointAdapter();
}
```

Spring XML

```
<citrus:empty-response-adapter id="emptyResponseEndpointAdapter"/>
```

34.2. Static response endpoint adapter

The next more complex endpoint adapter will always return a static response message.

Citrus Bean

```
@BindToRegistry
public StaticResponseEndpointAdapter staticEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
          <MessageId>123456789</MessageId>
          <CorrelationId>Cx1x123456789</CorrelationId>
          <Text>Hello User</Text>
    </HelloResponse>
    """);

    endpointAdapter.getMessageHeader().put("Operation", "sayHello");
    endpointAdapter.getMessageHeader().put("MessageId", "123456789");
    return endpointAdapter;
}
```

Spring Bean

```
@Bean
public StaticResponseEndpointAdapter staticEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
          <MessageId>123456789</MessageId>
          <CorrelationId>Cx1x123456789</CorrelationId>
          <Text>Hello User</Text>
    </HelloResponse>
    """);

    endpointAdapter.getMessageHeader().put("Operation", "sayHello");
    endpointAdapter.getMessageHeader().put("MessageId", "123456789");
    return endpointAdapter;
}
```

Spring XML

```
<citrus:static-response-adapter id="staticEndpointAdapter">
    <citrus:payload>
        <![CDATA[
          <HelloResponse
            xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
              <MessageId>123456789</MessageId>
              <CorrelationId>Cx1x123456789</CorrelationId>
              <Text>Hello User</Text>
          </HelloResponse>
        ]]>
    </citrus:payload>
    <citrus:header>
        <citrus:element name="Operation" value="sayHello"/>
        <citrus:element name="MessageId" value="123456789"/>
    </citrus:header>
 </citrus:static-response-adapter>
```

The endpoint adapter is configured with a static message payload and static response header values. The response to the client is therefore always the same. You can add dynamic values by using Citrus functions such as **randomString** or **randomNumber**. Also, we are able to use values of the actual request message that has triggered the response adapter. The request is available via the local message store. In combination with Xpath or JsonPath functions we can map values from the actual request.

Citrus Bean

```
@BindToRegistry
public StaticResponseEndpointAdapter staticEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
          <MessageId>citrus:randomNumber(10)</MessageId>
          <CorrelationId>citrus:xpath(citrus:message(request.body()),
'/hello:HelloRequest/hello:CorrelationId')</CorrelationId>
          <Text>Hello User</Text>
    </HelloResponse>
    """);

    endpointAdapter.getMessageHeader().put("Operation", "sayHello");
    endpointAdapter.getMessageHeader().put("MessageId", "123456789");
    return endpointAdapter;
}
```

Spring Bean

```
@Bean
public StaticResponseEndpointAdapter staticEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
          <MessageId>citrus:randomNumber(10)</MessageId>
          <CorrelationId>citrus:xpath(citrus:message(request.body()),
'/hello:HelloRequest/hello:CorrelationId')</CorrelationId>
          <Text>Hello User</Text>
    </HelloResponse>
    """);

    endpointAdapter.getMessageHeader().put("Operation", "sayHello");
    endpointAdapter.getMessageHeader().put("MessageId", "123456789");
    return endpointAdapter;
}
```

Spring XML

```
<citrus:static-response-adapter id="staticEndpointAdapter">
    <citrus:payload>
        <![CDATA[
          <HelloResponse
            xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
              <MessageId>citrus:randomNumber(10)</MessageId>
              <CorrelationId>citrus:xpath(citrus:message(request.body()),
'/hello:HelloRequest/hello:CorrelationId')</CorrelationId>
              <Text>Hello User</Text>
          </HelloResponse>
        ]]>
    </citrus:payload>
    <citrus:header>
        <citrus:element name="Operation" value="sayHello"/>
        <citrus:element name="MessageId" value="citrus:randomNumber(10)"/>
    </citrus:header>
 </citrus:static-response-adapter>
```

The example above maps the **CorrelationId** of the **HelloRequest** message to the response with Xpath function. The local message store automatically has the message named **request** stored so we can access the payload with this message name.



XML is namespace specific so we need to use the namespace prefix **hello** in the Xpath expression. The namespace prefix should evaluate to a global namespace entry in the global Citrus namespace.

34.3. Request dispatching endpoint adapter

The idea behind the request dispatching endpoint adapter is that the incoming requests are dispatched to several other endpoint adapters. The decision which endpoint adapter should handle the actual request is done depending on some adapter mapping. The mapping is done based on the payload or header data of the incoming request. A mapping strategy evaluates a mapping key using the incoming request. You can think of an XPath expression that evaluates to the mapping key for instance. The endpoint adapter that maps to the mapping key is then called to handle the request.

So the request dispatching endpoint adapter is able to dynamically call several other endpoint adapters based on the incoming request message at runtime. This is very powerful. The next example uses the request dispatching endpoint adapter with a XPath mapping key extractor.

Citrus Bean

```
@BindToRegistry
public RequestDispatchingEndpointAdapter dispatchingEndpointAdapter() {
    RequestDispatchingEndpointAdapter endpointAdapter = new
RequestDispatchingEndpointAdapter();

    XPathPayloadMappingKeyExtractor mappingKeyExtractor = new
XPathPayloadMappingKeyExtractor();
    mappingKeyExtractor.setXpathExpression("//TestMessage/Operation/*");
    endpointAdapter.setMappingKeyExtractor(mappingKeyExtractor);

    SimpleMappingStrategy mappingStrategy = new SimpleMappingStrategy();
    mappingStrategy.setAdapterMappings("sayHello", helloEndpointAdapter());
    endpointAdapter.setMappingStrategy(mappingStrategy);
    return endpointAdapter;
}

@BindToRegistry
public StaticResponseEndpointAdapter helloEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
        <MessageId>123456789</MessageId>
        <Text>Hello User</Text>
    </HelloResponse>
    """);
    return endpointAdapter;
}
```

Spring Bean

```
@Bean
public RequestDispatchingEndpointAdapter dispatchingEndpointAdapter() {
    RequestDispatchingEndpointAdapter endpointAdapter = new
RequestDispatchingEndpointAdapter();

    XPathPayloadMappingKeyExtractor mappingKeyExtractor = new
XPathPayloadMappingKeyExtractor();
    mappingKeyExtractor.setXpathExpression("//TestMessage/Operation/*");
    endpointAdapter.setMappingKeyExtractor(mappingKeyExtractor);

    SimpleMappingStrategy mappingStrategy = new SimpleMappingStrategy();
    mappingStrategy.setAdapterMappings("sayHello", helloEndpointAdapter());
    endpointAdapter.setMappingStrategy(mappingStrategy);
    return endpointAdapter;
}

@Bean
public StaticResponseEndpointAdapter helloEndpointAdapter() {
    StaticResponseEndpointAdapter endpointAdapter = new
StaticResponseEndpointAdapter();
    endpointAdapter.setMessagePayload("""
    <HelloResponse
        xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
        <MessageId>123456789</MessageId>
        <Text>Hello User</Text>
    </HelloResponse>
    """);
    return endpointAdapter;
}
```

Spring XML

```
<citrus:dispatching-endpoint-adapter id="dispatchingEndpointAdapter"
         mapping-key-extractor="mappingKeyExtractor"
         mapping-strategy="mappingStrategy"/>

<bean id="mappingStrategy"
  class="org.citrusframework.endpoint.adapter.mapping.SimpleMappingStrategy">
    <property name="adapterMappings">
      <map>
          <entry key="sayHello" ref="helloEndpointAdapter"/>
      </map>
    </property>
</bean>

<bean id="mappingKeyExtractor"

class="org.citrusframework.xml.endpoint.adapter.mapping.XPathPayloadMappingKeyExtracto
r">
    <property name="xpathExpression" value="//TestMessage/Operation/*"/>
</bean>

<citrus:static-response-adapter id="helloEndpointAdapter">
    <citrus:payload>
        <![CDATA[
            <HelloResponse
                xmlns="http://citrusframework.org/schemas/samples/sayHello.xsd">
                <MessageId>123456789</MessageId>
                <Text>Hello User</Text>
            </HelloResponse>
        ]]>
    </citrus:payload>
</citrus:static-response-adapter>
```

The XPath mapping key extractor expression decides for each request which mapping key to use in order to find a proper endpoint adapter through the mapping strategy. The endpoint adapters available in the application context are mapped via their bean id. For instance an incoming request with a matching element ***//TestMessage/Operation/sayHello*** would be handled by the endpoint adapter bean that is registered in the mapping strategy as "sayHello" key. The available endpoint adapters are configured in the same Spring application context.

Citrus provides several default mapping key extractor implementations.

HeaderMappingKeyExtractor	Reads a special header entry and uses its value as mapping key
SoapActionMappingKeyExtractor	Uses the soap action header entry as mapping key

XPathPayloadMappingKeyExtractor	Evaluates a XPath expression on the request payload and uses the result as mapping key

In addition to that we need a mapping strategy. Citrus provides following default implementations.

SimpleMappingStrategy	Simple key value map with endpoint adapter references
BeanNameMappingStrategy	Loads the endpoint adapter Spring bean with the given id matching the mapping key
ContextLoadingMappingStrategy	Same as BeanNameMappingStrategy but loads a separate application context defined by external file resource

34.4. Channel endpoint adapter

The channel connecting endpoint adapter is the default adapter used in all Citrus server components. Indeed, this adapter also provides the most flexibility. This adapter forwards incoming requests to a channel destination. The adapter is waiting for a proper response on a reply destination synchronously. With the channel endpoint components you can read the requests on the channel and provide a proper response on the reply destination.

This special adapter comes with the **citrus-spring-integration** module. So you have to add the module to your project as a dependency. The Maven module for **citrus-spring-integration** goes to the Maven POM file as normal project dependency.

Maven module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-spring-integration</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Now you can add the Spring integration channel endpoint adapter component to the bean registry.

Citrus Bean

```
@BindToRegistry
public ChannelEndpointAdapter channelEndpointAdapter() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("inbound.channel");
    endpointConfiguration.setTimeout(2500L);
    return new ChannelEndpointAdapter(endpointConfiguration);
    return endpointAdapter;
}
```

Spring Bean

```
@Bean
public ChannelEndpointAdapter channelEndpointAdapter() {
    ChannelSyncEndpointConfiguration endpointConfiguration = new
ChannelSyncEndpointConfiguration();
    endpointConfiguration.setChannelName("inbound.channel");
    endpointConfiguration.setTimeout(2500L);
    return new ChannelEndpointAdapter(endpointConfiguration);
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus-si="http://www.citrusframework.org/schema/spring-
integration/config"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/spring-integration/config
            http://www.citrusframework.org/schema/spring-integration/config/citrus-
spring-integration-config.xsd">

  <citrus-si:channel-endpoint-adapter id="channelEndpointAdapter"
              channel-name="inbound.channel"
              timeout="2500"/>

</beans>
```



For the Spring XML bean configuration Citrus provides a `citrus-si` configuration namespace and schema definition for Spring integration related components and features. Include this namespace into your Spring configuration in order to use the Citrus Spring integration configuration elements. The namespace URI and schema location are added to the Spring configuration XML file.

34.5. JMS endpoint adapter

Another powerful endpoint adapter is the JMS connecting adapter implementation. This adapter forwards incoming requests to a JMS destination and waits for a proper response on a reply destination. A JMS endpoint can access the requests internally and provide a proper response on the reply destination. So this adapter is very flexible to provide proper response messages.

This special adapter comes with the **citrus-jms** module. So you have to add the module to your project as a dependency. The Maven module for **citrus-jms** goes to the Maven POM file as normal project dependency.

Maven module dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-jms</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

Now you can add the JMS endpoint adapter component to the bean registry.

Citrus Bean

```
@BindToRegistry
public JmsEndpointAdapter jmsEndpointAdapter() {
    JmsSyncEndpointConfiguration endpointConfiguration = new
JmsSyncEndpointConfiguration();
    endpointConfiguration.setDestinationName("JMS.Queue.Requests.In");
    endpointConfiguration.setReplyDestinationName("JMS.Queue.Requests.In");
    endpointConfiguration.setConnectionFactory(jmsConnectionFactory());
    endpointConfiguration.setTimeout(2500L);
    return new JmsEndpointAdapter(endpointConfiguration);
}

@BindToRegistry
public ActiveMQConnectionFactory jmsConnectionFactory() {
    ActiveMQConnectionFactory connectionFactory = new ActiveMQConnectionFactory();
    connectionFactory.setBrokerURL("tcp://localhost:61616");
    return connectionFactory;
}
```

Spring Bean

```
@Bean
public JmsEndpointAdapter jmsEndpointAdapter() {
    JmsSyncEndpointConfiguration endpointConfiguration = new
JmsSyncEndpointConfiguration();
    endpointConfiguration.setDestinationName("JMS.Queue.Requests.In");
    endpointConfiguration.setReplyDestinationName("JMS.Queue.Requests.In");
    endpointConfiguration.setConnectionFactory(jmsConnectionFactory());
    endpointConfiguration.setTimeout(2500L);
    return new JmsEndpointAdapter(endpointConfiguration);
}

@Bean
public ActiveMQConnectionFactory jmsConnectionFactory() {
    ActiveMQConnectionFactory connectionFactory = new ActiveMQConnectionFactory();
    connectionFactory.setBrokerURL("tcp://localhost:61616");
    return connectionFactory;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:citrus-jms="http://www.citrusframework.org/schema/jms/config"
        xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.citrusframework.org/schema/jms/config
        http://www.citrusframework.org/schema/jms/config/citrus-jms-config.xsd">

    <citrus-jms:endpoint-adapter id="jmsEndpointAdapter"
                  destination-name="JMS.Queue.Requests.In"
                  reply-destination-name="JMS.Queue.Response.Out"
                  connection-factory="jmsConnectionFactory"
                  timeout="2500"/>

    <bean id="jmsConnectionFactory"
class="org.apache.activemq.artemis.jms.client.ActiveMQConnectionFactory">
      <property name="brokerURL" value="tcp://localhost:61616"/>
    </bean>
</beans>
```



For the Spring XML bean configuration Citrus provides a `citrus-jms` configuration namespace and schema definition for JMS related components and features. Include this namespace into your Spring configuration in order to use the Citrus JMS configuration elements. The namespace URI and schema location are added to the Spring configuration XML file.

Chapter 35. Connectors

Connectors are quite similar to endpoints. The modules connect Citrus to a certain technology or framework rather than implementing a message transport (client and server) like endpoints usually do.

Connectors typically provide a client side only implementation that enable Citrus to interact with a service or framework (e.g. Docker, Kubernetes, Knative, Selenium, OpenAPI specification).

Chapter 36. OpenAPI support

OpenAPI is a popular specification language to describe HTTP APIs and its exposure to clients. Citrus is able to leverage the specification to auto generate client and server request/response message data. The generated message data follows the rules of a given operation in the specification. In particular, the message body is generated from the given Json schema rules in that specification. This way users may do contract-driven testing where the client and the server ensure the conformity with the contract to obey to the same specification rules.



The OpenAPI support in Citrus get enabled by adding a separate Maven module as dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-openapi</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

36.1. OpenAPI settings

The OpenAPI support in Citrus provides several settings that control validation, data generation, and specification handling. These settings can be configured via system properties or environment variables.

Table 8. System properties

System property	Description
citrus.openapi.generate.optional.fields	Enable/disable generation of optional fields in request/response data (default="true")
citrus.openapi.validation.enabled.request	Enable/disable OpenAPI validation for outgoing request messages (default="true")
citrus.openapi.validation.enabled.response	Enable/disable OpenAPI validation for incoming response messages (default="true")
citrus.openapi.neglect.base.path	When enabled, the base path defined in the OpenAPI specification is ignored when matching request paths (default="false")
citrus.openapi.request.fill.random.values	Controls auto-fill of random values in request data: `REQUIRED`, `ALL`, `NONE` (default="REQUIRED")
citrus.openapi.response.fill.random.values	Controls auto-fill of random values in response data: `REQUIRED`, `ALL`, `NONE` (default="REQUIRED")
citrus.openapi.validation.policy	Validation policy when parsing OpenAPI specifications: `REPORT`, `STRICT`, `IGNORE` (default="REPORT")

Table 9. Environment variables

Environment variable	Description
CITRUS_OPENAPI_GENERATE_OPTIONAL_FIELDS	Enable/disable generation of optional fields in request/response data (default="true")
CITRUS_OPENAPI_VALIDATION_ENABLED_REQUEST	Enable/disable OpenAPI validation for outgoing request messages (default="true")
CITRUS_OPENAPI_VALIDATION_ENABLED_RESPONSE	Enable/disable OpenAPI validation for incoming response messages (default="true")
CITRUS_OPENAPI_NEGLECT_BASE_PATH	When enabled, the base path defined in the OpenAPI specification is ignored when matching request paths (default="false")
CITRUS_OPENAPI_REQUEST_FILL_RANDOM_VALUES	Controls auto-fill of random values in request data: `REQUIRED`, `ALL`, `NONE` (default="REQUIRED")
CITRUS_OPENAPI_RESPONSE_FILL_RANDOM_VALUES	Controls auto-fill of random values in response data: `REQUIRED`, `ALL`, `NONE` (default="REQUIRED")
CITRUS_OPENAPI_VALIDATION_POLICY	Validation policy when parsing OpenAPI specifications: `REPORT`, `STRICT`, `IGNORE` (default="REPORT")

36.2. OpenAPI specification

The OpenAPI test actions in Citrus uses a specification which usually is a json or yaml document shared between the components.

Sometimes the specification gets exposed by a server application via HTTP endpoint. You can directly load the specification from the HTTP URL. Or you may just point the OpenAPI components to a local specification file.

Citrus supports OpenAPI on both client and server components so the next sections will describe the usage for each of those.

36.3. OpenAPI client

On the client side Citrus uses the OpenAPI specification to generate a proper HTTP request that is sent to the server. The user just gives a valid operationId from the specification every thing else is automatically generated. The Citrus client message will use the proper request path (e.g. `/petstore/v3/pet`) and Content-Type (e.g. `applicaiton/json`) according to the specification rules.

Of course, you can also validate the HTTP response message with auto generated validation. The user just gives the expected HTTP status code that is also described in the specification (e.g. 200 OK). The response data used as expected message content is then also generated from the specification.

As an example the following OpenAPI specification defines the operation `getPetById`.

petstore-v3.yaml

```
openapi: 3.0.2
info:
  title: Petstore
  version: 1.0.1
servers:
  - url: 'http://localhost/petstore/v3/'
paths:
  '/pet/{petId}':
    get:
      operationId: getPetById
      parameters:
        - name: petId
          description: ID of pet to return
          schema:
            format: int64
            type: integer
          in: path
          required: true
      responses:
        '200':
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Pet'
        '404':
          description: Pet not found
      summary: Find pet by ID
      description: Returns a single pet
# ...
```

The operation defines the HTTP GET request on `/pet/{petId}` and the response `200` OK that delivers the `#/components/schemas/Pet` Json object to the calling client as a response.

The Json schema for the pet defines all properties on the object.

Pet Json schema

```
Pet:
  required:
    - category
    - name
    - status
  type: object
  properties:
    id:
      format: int64
      type: integer
    category:
      $ref: '#/components/schemas/Category'
    name:
      type: string
      example: doggie
    photoUrls:
      type: array
      items:
        type: string
    tags:
      type: array
      items:
        $ref: '#/components/schemas/Tag'
    status:
      description: pet status in the store
      enum:
        - available
        - pending
        - sold
      type: string
# ...
```

In a testcase Citrus is able to leverage this information in order to send a proper request and validate the response based on the OpenAPI specification.

Java

```
private final HttpClient httpClient = new HttpClientBuilder()
            .requestUrl("http://localhost:%d".formatted(port))
            .build();

private final OpenApiSpecification petstoreSpec = OpenApiSpecification.from(
            Resources.create("classpath:org/citrusframework/openapi/petstore/petstore-
v3.json"));

@CitrusTest
public void openApiClientTest() {
    when(openapi(petstoreSpec)
                .client(httpClient)
                .send("getPetById"));

    then(openapi(petstoreSpec)
                .client(httpClient)
                .receive("getPetById", HttpStatus.OK));
}
```

XML

```
<test name="OpenApiClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="petstoreSpec"
value="classpath:org/citrusframework/openapi/petstore/petstore-v3.json"/>
    </variables>
    <actions>
        <openapi specification="${petstoreSpec}" client="httpClient">
          <send-request operation="getPetById"/>
        </openapi>

        <openapi specification="${petstoreSpec}" client="httpClient">
          <receive-response operation="getPetById" status="200"/>
        </openapi>
    </actions>
</test>
```

YAML

```
name: OpenApiClientTest
variables:
  - name: petstoreSpec
    value: classpath:org/citrusframework/openapi/petstore/petstore-v3.yaml
actions:
  - openapi:
      specification: ${petstoreSpec}
      client: "httpClient"
      sendRequest:
        operation: getPetById
  - openapi:
      specification: ${petstoreSpec}
      client: "httpClient"
      receiveResponse:
        operation: getPetById
        status: 200
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

In this very first example The client uses the OpenAPI specification to generate a proper GET HTTP request for the `getPetById` operation. The request is sent to the server using the request URL path `/petstore/v3/pet/${petId}` as declared in the OpenAPI specification.

The resulting HTTP response from the server is verified on the client by giving the operationId and the expected status `200`. The OpenAPI client generates the expected control message from the given Json schema in the OpenAPI specification.

The generated control message contains validation matchers and expressions as follows.

Generated control message body

```
{
  "id": "@isNumber()@",
  "name": "@notEmpty()@",
  "category": {
    "id": "@isNumber()@",
    "name": "@notEmpty()@"
  },
  "photoUrls": "@notEmpty()@",
  "tags":  "@ignore@",
  "status": "@matches(sold|pending|available)@"
}
```

This control message meets the rules defined by the OpenAPI Json schema specification for the pet object. For instance the enum field `status` is validated with a matching expression. In case the OpenAPI specification changes the generated control message will change accordingly.

This completes the client side OpenAPI support. Now let's have a closer look at the server side OpenAPI support in the next section.

36.4. OpenAPI server

On the server side Citrus is able to verify incoming requests based on the OpenAPI specification. The expected request message content as well as the expected resource URL path and the Content-Type are automatically validated.

Java

```
private final HttpServer httpServer = new HttpServerBuilder()
            .port(port)
            .timeout(5000L)
            .autoStart(true)
            .defaultStatus(HttpStatus.NO_CONTENT)
            .build();

private final OpenApiSpecification petstoreSpec = OpenApiSpecification.from(
            Resources.create("classpath:org/citrusframework/openapi/petstore/petstore-
v3.json"));

@CitrusTest
public void openApiClientTest() {
    when(openapi(petstoreSpec)
                .server(httpServer)
                .receive("addPet"));

    then(openapi(petstoreSpec)
                .server(httpServer)
                .send("addPet", HttpStatus.CREATED));
}
```

XML

```
<test name="OpenApiClientTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <variables>
        <variable name="petstoreSpec"
value="classpath:org/citrusframework/openapi/petstore/petstore-v3.json"/>
    </variables>
    <actions>
        <openapi specification="${petstoreSpec}" server="httpServer">
          <receive-request operation="addPet"/>
        </openapi>

        <openapi specification="${petstoreSpec}" server="httpServer">
          <send-response operation="addPet" status="200"/>
        </openapi>
    </actions>
</test>
```

YAML

```
name: OpenApiClientTest
variables:
  - name: petstoreSpec
    value: classpath:org/citrusframework/openapi/petstore/petstore-v3.yaml
actions:
  - openapi:
      specification: ${petstoreSpec}
      server: "httpServer"
      receiveRequest:
        operation: addPet
  - openapi:
      specification: ${petstoreSpec}
      server: "httpServer"
      sendResponse:
        operation: addPet
        status: 200
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The example above uses the `addPet` operation defined in the OpenAPI specification. The operation expects a HTTP POST request with a pet object as message payload. The OpenAPI server generates an expected Json message body according to the specification. This ensures that the incoming client request meets the Json schema rules for the pet object. Also, the server will verify the HTTP request method, the Content-Type header as well as the used resource path `/petstore/v3/pet`.

The given HTTP status code defines the response that should be sent by the server. The server will generate a proper response according to the OpenAPI specification. This also includes a potential response message body (e.g. pet object).

Chapter 37. JBang support

With JBang you can run your `.java` files directly in your shell - instantly without tedious setup.



The JBang support in Citrus gets enabled by adding a separate Maven module as a dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-jbang-connector</artifactId>
  <version>${citrus.version}</version>
</dependency>
```



You may wonder why is there another module named `citrus-jbang`! What is the difference compared to the `citrus-jbang-connector` module? The `citrus-jbang` module represents the Citrus JBang app that you can use in JBang to run Citrus tests without any prior setup. The `citrus-jbang-connector` module provides the JBang support to use JBang as part of a test case, for instance in the form of a test action that runs JBang scripts.

37.1. JBang action

The JBang test action runs a script or JBang application with a spawned process. You can call any JBang CLI command and run your JBang app. JBang is called with the Java process API so the JBang CLI binary is executed and the command output is saved for later reference. You can verify the command output with an expected output and you can also verify the exit code of the spawned process.

Java

```
@CitrusTest
public void jBangScriptTest() {
    when(jbang()
            .app("myApp")
            .command("getUsers")
            .arg("--username", "FooUser"));
}
```

XML

```
<test name="JBangScriptTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <jbang app="myApp" command="getUsers">
          <args>
            <arg name="--username" value="FooUser"/>
          </args>
        </jbang>
    </actions>
</test>
```

YAML

```
name: JBangScriptTest
actions:
  - jbang:
      app: "myApp"
      command: "getUsers"
      args:
        - name: "--username"
          value: "FooUser"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The test above calls the JBang app `myApp` with the `getUsers` command. The call is similar to a command line statement like this:

```
$ jbang myApp getUsers --username=FooUser
```

The app `myApp` represents a JBang application that has been installed previously in JBang (for instance from a GitHub repository that holds a `jbang-catalog.json`):

```
$ jbang app install myApp@github-user/repository
```

Chapter 38. Kubernetes support

Kubernetes is one of the hottest management platforms for containerized applications these days. Kubernetes lets you deploy, scale and manage your Docker containers on the platform. You get features like auto-scaling, self-healing, service discovery and load balancing.

Citrus provides interaction with the Kubernetes REST API to access the Kubernetes cluster and its resources within a Citrus test case.



The Kubernetes test components in Citrus are kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-kubernetes</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

38.1. Kubernetes client

Citrus operates with the Kubernetes remote REST API in order to interact with the Kubernetes platform. The Kubernetes client component holds the configuration to connect to a Kubernetes cluster:

Citrus Bean

```
@BindToRegistry
public KubernetesClient k8sClient() {
    return new KubernetesClientBuilder()
                .build();
}
```

Spring Bean

```
@Bean
public KubernetesClient k8sClient() {
    return new KubernetesClientBuilder()
                .build();
}
```

Spring XML

```
<citrus-k8s:client id="k8sClient"/>
```

The Kubernetes client is based on the Fabric8 Java Kubernetes client implementation. Following

from that the component can be configured in various ways. By default, the client reads the system properties as well as environment variables for default Kubernetes settings such as:

- **kubernetes.master** / **KUBERNETES_MASTER**
- **kubernetes.api.version** / **KUBERNETES_API_VERSION**
- **kubernetes.trust.certificates** / **KUBERNETES_TRUST_CERTIFICATES**

If you set these properties in your environment the client component will automatically pick up the configuration settings. Also, when using `kubectl` command line locally the client may automatically use the stored user authentication settings from there.

For a complete list of settings and explanation of those please refer to the Fabric8 client documentation.

In case you need to set the client configuration explicitly on your environment you can also use explicit settings on the Kubernetes client component:

Citrus Bean

```
@BindToRegistry
public KubernetesClient k8sClient() {
    return new KubernetesClientBuilder()
                .client(fabric8Client)
                .url("http://localhost:8843")
                .version("v1")
                .username("user")
                .password("s!cr!t")
                .namespace("user_namespace")
                .messageConverter(messageConverter)
                .jsonMapper("jsonMapper")
                .build();
}
```

Spring Bean

```
@Bean
public KubernetesClient k8sClient() {
    return new KubernetesClientBuilder()
                .client(fabric8Client)
                .url("http://localhost:8843")
                .version("v1")
                .username("user")
                .password("s!cr!t")
                .namespace("user_namespace")
                .messageConverter(messageConverter)
                .jsonMapper("jsonMapper")
                .build();
}
```

Spring XML

```
<citrus-k8s:client id="k8sClient"
              client="fabric8Client"
              url="http://localhost:8843"
              version="v1"
              username="user"
              password="s!cr!t"
              namespace="user_namespace"
              message-converter="messageConverter"
              json-mapper="jsonMapper"/>
```



You have the option to set an `oauthToken` for the client instead of using `username` and `password`.

Now Citrus is able to access the Kubernetes remote API for executing commands such as list-pods, watch-services and so on. Citrus provides a set of actions that perform a Kubernetes command via REST. The results usually get validated in the Citrus test as usual.

38.2. Kubernetes actions

Citrus provides a set of test actions specifically designed to manage resources on the Kubernetes cluster. Each test action requires a reference to the Kubernetes client. Usually this client is created once in the bean registry and used by many test actions.



All resources created on the Kubernetes cluster will be automatically removed by Citrus after the test. You can disable this default behavior via system property setting or environment variables (`citrus.kubernetes.auto.remove.resources=false` or `CITRUS_KUBERNETES_AUTO_REMOVE_RESOURCES=false`)

38.3. Create Labels

Resources in Kubernetes are able to expose labels as part of the resource metadata. You can set labels on existing Kubernetes resources with this test action:

Java

```
@CitrusTest
public void createLabelsTest() {
    given(kubernetes()
            .client(k8sClient)
            .pods()
            .addLabel("my-pod")
            .label("test", "citrus")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateLabelsTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-labels type="POD" resource="my-pod">
          <labels>
            <label name="test" value="citrus"/>
          </labels>
        </create-labels>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateLabelsTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createLabels:
        type: "POD"
        resource: "my-pod"
        labels:
          - name: "test"
            value: "citrus"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.4. Create Annotations

Resources in Kubernetes are able to expose annotations as part of the resource metadata. You can set annotations on existing Kubernetes resources with this test action:

Java

```
@CitrusTest
public void createAnnotationsTest() {
    given(kubernetes()
            .client(k8sClient)
            .pods()
            .addAnnotation("my-pod")
            .annotation("test", "citrus")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateAnnotationsTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-annotations type="POD" resource="my-pod">
          <annotations>
            <annotation name="test" value="citrus"/>
          </annotations>
        </create-annotations>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateAnnotationsTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createAnnotations:
        type: "POD"
        resource: "my-pod"
        labels:
          - name: "test"
            value: "citrus"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.5. Create Service

You can create a Kubernetes service resource with this test action:

Java

```
@CitrusTest
public void createServiceTest() {
    given(kubernetes()
            .client(k8sClient)
            .services()
            .create("my-service")
            .portMapping(80, 8080)
            .withPodSelector(Collections.singletonMap("test", "citrus"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateServiceTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-service name="my-service">
          <ports>
            <port-mapping port="80" target-port="8080"/>
          </ports>
          <selector>
            <label name="test" value="citrus"/>
          </selector>
        </create-service>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateServiceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createService:
        name: "my-service"
        ports:
          - port: "80"
            targetPort: "8080"
        selector:
          labels:
            - name: "test"
              value: "citrus"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The service is able to define port mappings as well as pod selectors based on labels. This exposes the service as a Kubernetes resource on the connected cluster in the given namespace.



The Kubernetes service test action will automatically bind this service to a local HttpServer instance on the given port mapping. As a result the HttpServer instance is created and started as part of the test action. The HttpServer instance is added to the bean registry for further reference. The default HttpServer bean name is `citrus-k8s-service` or the service name chosen for the Kubernetes service. You can reference the HttpServer in following test actions to receive requests from the Kubernetes service.



The behavior to auto create the HttpServer instance can be disabled via system property or environment variable setting (`citrus.kubernetes.auto.create.server.binding=false` or `CITRUS_KUBERNETES_AUTO_CREATE_SERVER_BINDING=false`) You can also customize the HttpServer instance by referencing a previously created server from the bean registry. In this case the auto creation is skipped for this test action and the custom server reference is used.

38.6. Connect with Service

Every Kubernetes Deployment is able to expose a service to other resources on the cluster so clients are able to call the service endpoint. Usually Kubernetes service exposes this endpoint to internal

clients only (e.g. service type is ClusterIP). Internal client means that the client connects from within the same cluster (e.g. from another pod). Following from that your test that runs on your local machine may not have access to the Kubernetes service endpoint.

The solution for that is to connect to the Kubernetes service as part of the Citrus test. This enables the access to such a Kubernetes service. The service connection uses a port forward to the Kubernetes service and the underlying Kubernetes pod. You can then use a local port on your machine to access the Kubernetes service.

Java

```
@CitrusTest
public void connectServiceTest() {
    given(kubernetes()
            .client(k8sClient)
            .services()
            .connect("my-service")
            .portMapping(8080, 31234)
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="ServiceConnectTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
  <actions>
    <kubernetes client="k8sClient" namespace="my-namespace">
      <connect>
        <service name="my-service" port="8080" local-port="31234"/>
      </connect>
    </kubernetes>
  </actions>
</test>
```

YAML

```
name: ServiceConnectTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      connect:
        service:
          name: "my-service"
          port: 8080
          localPort: 31234
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The service connect test action creates a new port forward using a local port on your machine connecting to the exposed port on the service. The user is able to define the port mapping with a fixed local port and the target exposed port on the Kubernetes service.



You can also skip the local port setting to choose a random local port.

This opens the connection of your local machine to the Kubernetes service so sending requests to the local endpoint will invoke the service on the Kubernetes cluster. The port forward is kept open for the rest of your test or until you explicitly disconnect.

The connect test action automatically creates a new Http client instance and stores this in the Citrus registry. You can use the client in following test actions to connect to the new port forward service endpoint. As usual, you can reference the client by its name in the Citrus registry. The name of the Http client instance that connects to the new port forward service uses a default pattern: `<service-name>.client`.

In our example the client is stored in the Citrus registry with the name `my-service.client`. You can use the client in a normal Http test action to call the new port forward service.

Java

```
@CitrusTest
public void connectServiceTest() {
    when(http()
        .client("my-service.client")
        .send()
        .get()
    );

    then(http()
        .client("my-service.client")
        .receive()
        .response(HttpStatus.OK.value())
    );
}
```

XML

```
<test name="ServiceConnectTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
  <actions>
    <http client="my-service.client">
      <send-request>
        <GET/>
      </send-request>
    </http>

    <http client="my-service.client">
      <receive-response>
        <response status="200" reason-phrase="OK"/>
      </receive-response>
    </http>
  </actions>
</test>
```

YAML

```
name: ServiceConnectTest
actions:
  - http:
      client: "my-service.client"
      sendRequest:
        GET: {}
  - http:
      client: "httpClient"
      receiveResponse:
        status: 200
        reasonPhrase: OK
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

Instead of using the default generated Http client name you can also explicitly specify a client name. Citrus will search create the client and add it to the Citrus registry with that name.



The Kubernetes service port forward may require special permissions on your Kubernetes cluster. Please make sure to connect with a proper user that is able to create the port forward. On Kubernetes platforms such as Minikube or Kind this is given by default. On other cluster types you may need to set up special permissions for your local Kubernetes user.



The port forward connection is automatically closed when the test ends. This is done with an explicit disconnect test action that is run in the finally section of the current test. In case you need to keep the port forward connection open you can disable the `auto-remove` option on the connect test action.

38.7. Disconnect from Service

By default, the port forward service connection is closed automatically when the test ends. In case you need to explicitly close the port forward connection you can do so with the following test action.

Java

```
@CitrusTest
public void connectServiceTest() {
    given(kubernetes()
            .client(k8sClient)
            .services()
            .disconnect("my-service")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="ServiceConnectTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
  <actions>
    <kubernetes client="k8sClient" namespace="my-namespace">
      <disconnect>
        <service name="my-service"/>
      </disconnect>
    </kubernetes>
  </actions>
</test>
```

YAML

```
name: ServiceConnectTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      disconnect:
        service:
          name: "my-service"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This closes the port forward to the Kubernetes service immediately.

38.8. Create ConfigMap

You can create a Kubernetes config map with this test action:

Java

```
@CitrusTest
public void createConfigMapTest() {
    given(kubernetes()
            .client(k8sClient)
            .configMaps()
            .create("my-config-map")
            .properties(Collections.singletonMap("username", "foo"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateConfigMapTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-config-map name="my-config-map">
          <properties>
            <property name="username" value="foo"/>
          </properties>
        </create-config-map>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateConfigMapTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createConfigMap:
        name: "my-config-map"
        properties:
          - name: username
            value: foo
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

It is also possible to load the properties from a file resource:

Java

```
@CitrusTest
public void createConfigMapTest() {
    given(kubernetes()
            .client(k8sClient)
            .configMaps()
            .create("my-config-map")
            .fromFile(Resources.create("application.properties"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateConfigMapTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-config-map name="my-config-map" file="application.properties"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateConfigMapTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createConfigMap:
        name: "my-config-map"
        file: "application.properties"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.9. Create Secret

You can create a Kubernetes secret with this test action:

Java

```
@CitrusTest
public void createSecretTest() {
    given(kubernetes()
            .client(k8sClient)
            .secrets()
            .create("my-secret")
            .properties(Collections.singletonMap("password", "top_secret"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateSecretTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-secret name="my-secret">
          <properties>
            <property name="password" value="top_secret"/>
          </properties>
        </create-secret>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateSecretTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createSecret:
        name: "my-secret"
        properties:
          - name: password
            value: top_secret
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

It is also possible to load the properties from a file resource:

Java

```
@CitrusTest
public void createSecretTest() {
    given(kubernetes()
            .client(k8sClient)
            .secrets()
            .create("my-secret")
            .fromFile(Resources.create("credentials.properties"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateSecretTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-secret name="my-secret" file="credentials.properties"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateSecretTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createSecret:
        name: "my-secret"
        file: "credentials.properties"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.10. Create Resources

You can create standards Kubernetes resources such as Pods as part of the test. The easiest way to define the resource specification is to give a YAML file that represents the resource.

Of course, you can use Citrus test variables in the resource specification.

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    name: my-pod
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

Java

```
@CitrusTest
public void createPodResourceTest() {
    given(kubernetes()
            .client(k8sClient)
            .resources()
            .create()
            .resource(Resources.create("pod.yaml"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreatePodResourceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-resource file="pod.yaml"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreatePodResourceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createResource:
        file: "pod.yaml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.11. Delete Resources

You can delete any resource on the Kubernetes cluster. Of course your client connection needs to have sufficient permissions to perform the delete operation. The Kubernetes resource that should be deleted is identified by its name and an optional namespace.

The next example deletes a service resource identified by its name.

Java

```
@CitrusTest
public void deleteServiceTest() {
    given(kubernetes()
            .client(k8sClient)
            .services()
            .delete("my-service")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="DeleteServiceTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <delete-service name="my-service"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: DeleteServiceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      deleteService:
        name: "my-service"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

As an alternative to that you can also use a resource specification to delete the resource.

Java

```
@CitrusTest
public void deletePodResourceTest() {
    given(kubernetes()
            .client(k8sClient)
            .resources()
            .delete()
            .resource(Resources.create("pod.yaml"))
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="DeletePodResourceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <delete-resource file="pod.yaml"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: DeletePodResourceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      deleteResource:
        file: "pod.yaml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.12. Create Custom Resource

Given the following Kubernetes custom resource:

foo.yaml

```
apiVersion: citrus.dev/v1
kind: Foo
metadata:
  name: my-foo
  labels:
    test: citrus
spec:
  message: Hello
```

You can create the Kubernetes custom resource with this test action:

Java

```
@CitrusTest
public void createCustomResourceTest() {
    given(kubernetes()
            .client(k8sClient)
            .customResources()
            .create()
            .kind("Foo")
            .apiVersion("citrus.dev/v1")
            .file("foo.yaml")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateCustomResourceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <create-custom-resource kind="Foo" api-version="citrus.dev/v1" file="foo.yaml"
/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateCustomResourceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createCustomResource:
        kind: "Foo"
        apiVersion: "citrus.dev/v1"
        file: "foo.yaml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



When you have access to a Fabric8 Java model that represents the custom resource you can use the Java type to specify the Kind and ApiVersion while creating the resource. Also, you can use a Java object instance directly to create the resource with the Fabric8 Kubernetes client.

38.13. Delete Custom Resource

When you want to delete a Kubernetes custom resource you need to specify the Kind as well as the ApiVersion in combination with the resource name on the cluster.

Java

```
@CitrusTest
public void createCustomResourceTest() {
    given(kubernetes()
            .client(k8sClient)
            .customResources()
            .delete()
            .kind("Foo")
            .apiVersion("citrus.dev/v1")
            .name("foo")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="DeleteCustomResourceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <delete-custom-resource kind="Foo" api-version="citrus.dev/v1" name="foo" />
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: DeleteCustomResourceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      deleteCustomResource:
        kind: "Foo"
        apiVersion: "citrus.dev/v1"
        name: "foo"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



When you have access to a Fabric8 Java model that represents the custom resource you can use the Java type to specify the Kind and ApiVersion while deleting the resource.

38.14. Verify Custom Resource

You can verify the ready condition status for a Kubernetes custom resource. Usually the custom resource exposes a Status sub-resource with that expose a ready condition. You can verify this condition as part of a test like this:

Java

```
@CitrusTest
public void verifyCustomResourceTest() {
    given(kubernetes()
            .client(k8sClient)
            .customResources()
            .verify("foo")
            .kind("Foo")
            .apiVersion("citrus.dev/v1")
            .isReady()
            .maxAttempts(2)
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="VerifyCustomResourceTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <verify-custom-resource kind="Foo" api-version="citrus.dev/v1" name="foo"
condition="Ready" />
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: VerifyCustomResourceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      verifyCustomResource:
        kind: "Foo"
        apiVersion: "citrus.dev/v1"
        name: "foo"
        condition: "Ready"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



Instead of specifying the resource name you can also use a label expression (name=value) to identify the custom resource on the cluster namespace. Citrus tries to find the resource with the matching label.

38.15. Verify Pod Status

Each Pod in Kubernetes exposes its Ready state condition as well as the phase (Running, Error, Initializing, Terminating, ...). You can verify this Pod status in your test case:

Java

```
@CitrusTest
public void verifyPodTest() {
    given(kubernetes()
            .client(k8sClient)
            .pods()
            .verify("my-pod")
            .isRunning()
            .maxAttempts(2)
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="VerifyPodTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <verify-pod name="my-pod" phase="Running" />
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: VerifyPodTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      verifyPod:
        name: "my-pod"
        phase: "Running"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



When asserting the `Running` phase the verification also checks for all containers in the Pod to expose the condition `Ready`.

38.16. Verify Pod Logs

You can read and verify the Pod logs as part of the test.

Java

```
@CitrusTest
public void verifyPodTest() {
    given(kubernetes()
            .client(k8sClient)
            .pods()
            .verify("my-pod")
            .isRunning()
            .waitForLogMessage("Something interesting")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="VerifyPodTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <verify-pod name="my-pod" phase="Running" log-message="Something
interesting"/>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: VerifyPodTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      verifyPod:
        name: "my-pod"
        phase: "Running"
        logMessage: "Something interesting"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This action scans the Pod logs for a message `Something interesting`. As soon as the log message has been located in the Pod logs the test action finishes with success. The action waits for the log message to appear in the Pod logs for given amount of time with a specified polling attempt configuration.

38.17. Kubernetes Citrus agent

The Citrus agent is a way to run tests as part of the Kubernetes platform. You can deploy the Citrus agent on the Kubernetes cluster. Usually the agent exposes a REST service endpoint so clients are then able to inject Citrus tests for execution.

The Citrus agent runs as part of the Kubernetes cluster, so it has access to Kubernetes resources on the same namespace. This means the tests that run with the agent are able to access resources such as Knative eventing brokers, Services, ConfigMaps, Secrets and many more.

38.17.1. Agent connect

You can connect to a Citrus agent service as part of your test like follows:

Java

```
@CitrusTest
public void agentConnectTest() {
    given(kubernetes()
            .client(k8sClient)
            .agent()
            .connect()
            .service("citrus-agent")
            .localPort(12345)
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="AgentConnectTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <agent>
          <connect name="citrus-agent" local-port="12345"/>
        </agent>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: AgentConnectTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      agent:
        connect:
          name: "citrus-agent"
          localPort: 12345
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The agent connect test action connects to the Citrus agent service on the given Kubernetes namespace. In case the Citrus agent application has been deployed already and the exposed service is available the test action connects to the service via local port forwarding (see also the section about connecting with a Kubernetes Service). This means that you will be able to trigger test runs

on the agent with calling REST operations on the local endpoint.

In case the Citrus agent application is not present on the Kubernetes cluster the connect test action will automatically create the deployment on the given namespace. The deployment includes a ConfigMap that holds the test case sources (e.g. in the form of a `test-jar` Java archive). The agent deployment specification will mount the test sources into the agent Pod via volume mounts, so you can run the tests from the `test-jar` Java archive.

Also, the action creates a new Kubernetes service that exposes the agent REST API and the Web UI. Once the connection is done you can start triggering test actions and test runs on the Citrus agent.

38.17.2. Agent run

When the local test is connected to the Kubernetes Citrus agent you can trigger the test execution on the agent.

Java

```
@CitrusTest
public void agentRunTest() {
    given(agent()
            .run()
            .sourceFile("classpath:my-test.groovy"));
}
```

XML

```
<test name="AgentRunTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <agent>
        <run>
          <source file="classpath:my-test.groovy"/>
        </run>
      </agent>
    </actions>
</test>
```

YAML

```
name: AgentRunTest
actions:
  - agent:
      source:
        file: "classpath:my-test.groovy"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The test source code can be any of the supported domain specific languages (e.g. XML, YAML, Java, Groovy, Cucumber). The example above loads a Groovy test source code file. The test actions will be sent to the Citrus agent and executed as a test within the agent runtime. The test results are validated for success outcome and in case the remote test execution fails the local Citrus agent run action will also fail accordingly.



When using XML and YAML tests you can also leverage the possibility to use inline test actions as part of the Citrus agent run action

YAML

```
name: AgentRunTest
actions:
  - agent:
      run:
        actions:
          - echo:
              message: Citrus rocks!
```

XML

```
<test name="AgentRunTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <agent>
        <run>
          <actions>
            <echo message="Citrus rocks!"/>
          </actions>
        </run>
      </agent>
    </actions>
</test>
```

All nested test actions within the Citrus agent run action will be run on the connected remote Citrus agent.

38.17.3. Agent disconnect

You can explicitly disconnect from the Citrus agent service. This terminates the local port forward and also removes the Citrus agent deployment and all dependent resources from the current Kubernetes namespace.

Java

```
@CitrusTest
public void agentDisconnectTest() {
    given(kubernetes()
            .client(k8sClient)
            .agent()
            .disconnect()
            .service("citrus-agent")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="AgentDisconnectTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <kubernetes client="k8sClient" namespace="my-namespace">
        <agent>
          <disconnect name="citrus-agent"/>
        </agent>
      </kubernetes>
    </actions>
</test>
```

YAML

```
name: AgentDisconnectTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      agent:
        disconnect:
          name: "citrus-agent"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

38.18. Spring bean config

Citrus provides a "citrus-kubernetes" configuration namespace and schema definition for Kubernetes related components and actions. Include this namespace into your Spring configuration in order to use the Citrus Kubernetes configuration elements.

38.18.1. Spring XML config namespace

The namespace URI and schema location are added to the Spring configuration XML file as follows.

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-k8s="http://www.citrusframework.org/schema/kubernetes/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/kubernetes/config
           http://www.citrusframework.org/schema/kubernetes/config/citrus-kubernetes-
config.xsd">

    [...]

</beans>
```

After that you are able to use customized Citrus XML elements in order to define the Spring beans.

38.18.2. Spring XML testcase namespace

We have several Citrus test actions each representing a Kubernetes command. These actions can be part of a test case where you can manage Kubernetes pods inside the test. As a prerequisite we have to enable the Kubernetes specific test actions in our XML test as follows:

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:k8s="http://www.citrusframework.org/schema/kubernetes/testcase"
        xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.citrusframework.org/schema/kubernetes/testcase
        http://www.citrusframework.org/schema/kubernetes/testcase/citrus-kubernetes-
testcase.xsd">

    [...]

</beans>
```

We added a special kubernetes namespace with prefix **k8s:** so now we can start to add Kubernetes test actions to the test case:

Spring XML

```
<testcase name="KubernetesCommandIT">
    <actions>
      <k8s:info client="myK8sClient">
        <k8s:validate>
          <k8s:result>{
            "result": {
              "clientVersion": "1.4.27",
              "apiVersion": "v1",
              "kind":"Info",
              "masterUrl": "${masterUrl}",
              "namespace": "test"
            }
          }</k8s:result>
        </k8s:validate>
      </k8s:info>
    </actions>
</testcase>
```

In this very simple example we first ping the Kubernetes REST API to make sure we have connectivity up and running. The info command connects the REST API and returns a list of status information of the Kubernetes client.

Based on that we can execute several Kubernetes commands in a test case and validate the Json results.

Citrus supports the following Kubernetes API commands with respective test actions:

- **k8s:info**
- **k8s:list-pods**
- **k8s:get-pod**
- **k8s:delete-pod**
- **k8s:list-services**
- **k8s:get-service**
- **k8s:delete-service**
- **k8s:list-namespaces**
- **k8s:list-events**
- **k8s:list-endpoints**
- **k8s:list-nodes**
- **k8s:list-replication-controllers**
- **k8s:watch-pods**
- **k8s:watch-services**
- **k8s:watch-namespaces**

- **k8s:watch-nodes**
- **k8s:watch-replication-controllers**

We will discuss these commands in detail in the following sections.

38.19. Kubernetes commands

The Citrus test may use Kubernetes commands that get run as test actions.

Java

```
@CitrusTest
public void kubernetesTest() {
    kubernetes().info()
        .validate(new CommandResultCallback<InfoResult>() {
            @Override
            public void doWithCommandResult(InfoResult info, TestContext context) {
                Assert.assertEquals(info.getApiVersion(), "v1");
            }
    });

    kubernetes().pods()
                .list()
                .withoutLabel("running")
                .label("app", "myApp");
}
```

Spring XML

```
<testcase name="KubernetesCommandIT">
    <actions>
      <k8s:info>
        <k8s:validate>
          <k8s:result>{
            "result": {
              "clientVersion": "1.4.27",
              "apiVersion": "v1",
              "kind":"Info",
              "masterUrl": "${masterUrl}",
              "namespace": "test"
            }
          }</k8s:result>
        </k8s:validate>
      </k8s:info>

      <k8s:list-pods>
        <k8s:validate>
          <k8s:result>{
            "result": {
              "apiVersion":"v1",
              "kind":"PodList",
              "metadata":"@ignore@",
              "items":[]
            }
          }</k8s:result>
          <k8s:element path="$.result.items.size()" value="0"/>
        </k8s:validate>
      </k8s:list-pods>
    </actions>
</testcase>
```

In this very simple example we first ping the Kubernetes REST API to make sure we have connectivity up and running. The info command connects the REST API and returns a list of status information of the Kubernetes client. After that we get the list of available Kubernetes pods.

As a tester we might be interested in validating the command results. So we can specify an optional **k8s:result** which is usually in Json format. With that we can apply the full Citrus Json validation power to the Kubernetes results. As usual, we can use test variables here and ignore some values explicitly such as the **metadata** value. Also, JsonPath expression validation and Json test message validation features in Citrus come in here to validate the results.

The Java DSL Kubernetes commands provide an optional **KubernetesCommandResultCallback** that is automatically called with the unmarshalled command result object. In the example above the *InfoResult* model object is passed as argument to the callback. So the tester can access the command result and validate its properties with assertions.

The option of Lamdba expressions allows us to use some syntactical sugar in Java when using the

KubernetesCommandResultCallback.

Java

```
@CitrusTest
public void kubernetesTest() {
    kubernetes().info()
                .validate((info, context) -> Assert.assertEquals(info.getApiVersion(),
"v1"));
}
```



By default, Citrus tries to find a Kubernetes client component within the Citrus Spring application context. If not present Citrus will instantiate a default kubernetes client with all default settings. You can also explicitly set the kubernetes client instance when using the Java DSL Kubernetes command actions:

Java

```
@Autowired
private KubernetesClient k8sClient;

@CitrusTest
public void kubernetesTest() {
    kubernetes().client(k8sClient)
                .info()
                .validate((info, context) -> Assert.assertEquals(info.getApiVersion(),
"v1"));
}
```

Spring XML

```
<testcase name="KubernetesCommandIT">
    <actions>
      <k8s:info client="k8sClient">
        <k8s:validate>
          <k8s:result>{
            "result": {
              "clientVersion": "1.4.27",
              "apiVersion": "v1",
              "kind":"Info",
              "masterUrl": "${masterUrl}",
              "namespace": "test"
            }
          }</k8s:result>
        </k8s:validate>
      </k8s:info>
    </actions>
</testcase>
```

38.19.1. Info command

The info command just gets the client connection settings and provides them as a Json result to the action.

Java

```
@CitrusTest
public void infoTest() {
    kubernetes().info()
                .validate((info, context) -> Assert.assertEquals(info.getApiVersion(),
"v1"));
}
```

Spring XML

```
<k8s:info client="myK8sClient">
  <k8s:validate>
    <k8s:result>{
      "result": {
        "clientVersion": "1.4.27",
        "apiVersion": "v1",
        "kind":"Info",
        "masterUrl": "${masterUrl}",
        "namespace": "test"
      }
    }</k8s:result>
  </k8s:validate>
</k8s:info>
```

38.19.2. List resources

We can list Kubernetes resources such as pods, services, endpoints and replication controllers. The list can be filtered by several properties such as

- label
- namespace

The test action is able to define respective filters to the list, so we get only pods the match the given attributes:

Java

```
@CitrusTest
public void listPodsTest() {
    kubernetes()
        .client(k8sClient)
        .pods()
        .list()
        .label("app=todo")
        .validate("$..status.phase", "Running")
        .validate((pods, context) -> {
            Assert.assertFalse(CollectionUtils.isEmpty(pods.getResult().getItems()));
        });
}
```

Spring XML

```
<k8s:list-pods label="app=todo">
    <k8s:validate>
      <k8s:result>{
        "result": {
          "apiVersion":"${apiVersion}",
          "kind":"PodList",
          "metadata":"@ignore@",
          "items":"@ignore@"
        }
      }</k8s:result>
      <k8s:element path="$.result.items.size()" value="1"/>
      <k8s:element path="$..status.phase" value="Running"/>
    </k8s:validate>
</k8s:list-pods>
```

As you can see we are able to give the pod label that is searched for in list of all pods. The list returned is validated either by giving an expected Json message or by adding JsonPath expressions with expected values to check.

In Java DSL we can add a validation result callback that is provided with the unmarshalled result object for validation. Besides *label* filtering we can also specify the *namespace* and the pod *name* to search for.

You can also define multiple labels as comma delimited list:

Java

```
@CitrusTest
public void listServicesTest() {
    kubernetes()
        .client(k8sClient)
        .services()
        .list()
        .label("stage!=test,provider=fabric8")
        .namespace("default");
}
```

Spring XML

```
<k8s:list-services label="stage!=test,provider=fabric8" namespace="default"/>
```

As you can see we have combined to label filters `stage!=test` and `provider=fabric8` on pods in namespace `default`. The first label filter is negated so the label `stage` should **not** be `test` here.

38.19.3. List nodes and namespaces

Nodes and namespaces are special resources that are not filtered by their namespace as they are more global resources. The rest is pretty similar to listing pods or services. We can add filters such as `name` and `label`.

Java

```
@CitrusTest
public void listPodsTest() {
    kubernetes()
        .client(k8sClient)
        .namespaces()
        .list()
        .label("provider=citrus")
        .validate((pods, context) -> {
            Assert.assertFalse(CollectionUtils.isEmpty(pods.getResult().getItems()));
        });
}
```

Spring XML

```
<k8s:list-namespaces label="provider=citrus">
    <k8s:validate>
      <k8s:element path="$.result.items.size()" value="1"/>
    </k8s:validate>
</k8s:list-namespaces>
```

38.19.4. Get resources

We can get a very special Kubernetes resource such as a pod or service for detailed validation of that resource. We need to specify a resource name in order to select the resource from list of available resources in Kubernetes.

Java

```
@CitrusTest
public void getPodsTest() {
    kubernetes()
        .client(k8sClient)
        .pods()
        .get("citrus_pod")
        .validate("$..status.phase", "Running")
        .validate((pod, context) -> {
            Assert.assertEquals(pods.getResult().getStatus().getPhase(), "Running");
        });
}
```

Spring XML

```
<k8s:get-pod name="citrus_pod">
    <k8s:validate>
      <k8s:result>{
      "result": {
        "apiVersion":"${apiVersion}",
        "kind":"Pod",
        "metadata": {
            "annotations":"@ignore@",
            "creationTimestamp":"@ignore@",
            "finalizers":[],
            "generateName":"@startsWith('hello-minikube-')@",
            "labels":{
              "pod-template-hash":"@ignore@",
              "run":"hello-minikube"
            },
            "name":"${podName}",
            "namespace":"default",
            "ownerReferences":"@ignore@",
            "resourceVersion":"@ignore@",
            "selfLink":"/api/${apiVersion}/namespaces/default/pods/${podName}",
            "uid":"@ignore@"
        },
        "spec": {
          "containers": [{
            "args":[],
            "command":[],
            "env":[],
            "image":"gcr.io/google_containers/echoserver:1.4",
```

```
            "imagePullPolicy":"IfNotPresent",
            "name":"hello-minikube",
            "ports":[{
              "containerPort":8080,
              "protocol":"TCP"
            }],
            "resources":{},
            "terminationMessagePath":"/dev/termination-log",
            "volumeMounts":"@ignore@"
          }],
          "dnsPolicy":"ClusterFirst",
          "imagePullSecrets":"@ignore@",
          "nodeName":"minikube",
          "restartPolicy":"Always",
          "securityContext":"@ignore@",
          "serviceAccount":"default",
          "serviceAccountName":"default",
          "terminationGracePeriodSeconds":30,
          "volumes":"@ignore@"
        },
        "status": "@ignore@"
      }
      }</k8s:result>
      <k8s:element path="$..status.phase" value="Running"/>
    </k8s:validate>
</k8s:get-pod>
```

As you can see we are able to get the complete pod information from Kubernetes. The result is validated with Json message validator in Citrus. This means we can use *@ignore@* as well as test variables and JsonPath expressions.

38.19.5. Create resources

We can create new Kubernetes resources within a Citrus test. This is very important in case we need to set up new pods or services for the test run. You can create new resources by giving a `.yaml` file holding all information how to create the new resource. See the following sample YAML for a new pod and service:

pod.yaml

```
kind: Pod
apiVersion: v1
metadata:
  name: hello-netty-${randomId}
  namespace: default
  labels:
    server: hello-netty
spec:
  containers:
    - name: hello-netty
      image: netty
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 8080
          protocol: TCP
  restartPolicy: Always
  terminationGracePeriodSeconds: 30
  dnsPolicy: ClusterFirst
  serviceAccountName: default
  serviceAccount: default
  nodeName: minikube
```

This YAML file specifies a new resource of kind *Pod*. We define the metadata as well as all containers that are part of this pod. The container is build from *jetty:9.3* Docker image that should be pulled automatically from Docker Hub registry. We also expose port 8080 as *containerPort* so the upcoming service configuration can provide this port to clients as Kubernetes service.

service.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: hello-netty
  namespace: default
  labels:
    service: hello-netty
spec:
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 31123
  selector:
    server: hello-netty
  type: NodePort
  sessionAffinity: None
```

The service resource maps the port *8080* and selects all pods with label *server=hello-netty*. This

makes the jetty container available to clients. The service type is *NodePort* which means that clients outside of Kubernetes are also able to access the service by using the port *nodePort=31123*. We can use Citrus functions such as *randomNumber* in the YAML files.

In the test case we can use these YAML files to create the resources in Kubernetes:

Java

```
@CitrusTest
public void createPodsTest() {
    kubernetes()
        .pods()
        .create(new ClassPathResource("templates/hello-netty-pod.yml"))
        .namespace("default");

    kubernetes()
        .services()
        .create(new ClassPathResource("templates/hello-service.yml"))
        .namespace("default");
}
```

Spring XML

```
<testcase name="KubernetesIT">
  <actions>
    <k8s:create-pod namespace="default">
      <k8s:template file="classpath:templates/hello-netty-pod.yml"/>
    </k8s:create-pod>

    <k8s:create-service namespace="default">
      <k8s:template file="classpath:templates/hello-service.yml"/>
    </k8s:create-service>
  </actions>
</testcase>
```

Creating new resources may take some time to finish. Kubernetes will have to pull images, build containers and start up everything. The `create` action is not waiting synchronously for all that to have happened. Therefore, we might add a list-pods action that waits for the new resources to appear.

Spring XML

```
<repeat-onerror-until-true condition="@assertThat(greaterThan(9))@" auto-sleep="1000">
  <k8s:list-pods label="server=hello-netty">
    <k8s:validate>
      <k8s:element path="$.result.items.size()" value="1"/>
      <k8s:element path="$..status.phase" value="Running"/>
    </k8s:validate>
  </k8s:list-pods>
</repeat-onerror-until-true>
```

With this repeat on error action we wait for the new *server=hello-netty* labeled pod to be in state *Running*.

38.19.6. Delete resources

With that command we are able to delete a resource in Kubernetes. Up to now deletion of pods and services is supported. We have to give a name of the resource that we want to delete.

Java

```
@CitrusTest
public void deletePodsTest() {
    kubernetes()
        .pods()
        .delete("citrus_pod")
        .validate((result, context) ->
Assert.assertTrue(result.getResult().getSuccess()));
}
```

Spring XML

```
<k8s:delete-pod name="citrus_pod">
    <k8s:validate>
      <k8s:element path="$.result.success" value="true"/>
    </k8s:validate>
</k8s:delete-pod>
```

38.19.7. Watch resources



The watch operation is still in experimental state and may face severe adjustments and improvements in near future.

When using a watch command we add a subscription to change events on a Kubernetes resources. So we can watch resources such as pods, services for future changes. Each change on that resource triggers a new watch event result that we can expect and validate.

Java

```
@CitrusTest
public void listPodsTest() {
    kubernetes()
        .pods()
        .watch()
        .label("provider=citrus")
        .validate((watchEvent, context) -> {
            Assert.assertFalse(watchEvent.hasError());
            Assert.assertEquals(((WatchEventResult) watchEvent).getAction(),
Watcher.Action.DELETED);
        });
}
```

Spring XML

```
<k8s:watch-pods label="provider=citrus">
    <k8s:validate>
      <k8s:element path="$.action" value="DELETED"/>
    </k8s:validate>
</k8s:watch-pods>
```



The watch command may be triggered several times for multiple changes on the respective Kubernetes resource. The watch action will always handle one single event result. The first event trigger is forwarded to the action validation. All further watch events on that same resource are ignored. This means that you may need multiple watch actions in your test case in case you expect multiple watch events to be triggered.

38.20. Kubernetes command bus

We have seen how to access the Kubernetes remote REST API by using special Citrus test actions in a test. As an alternative to that we can also use more generic send/receive actions in Citrus for accessing the Kubernetes API.

The next example shows this in action:

Java

```
@CitrusTest
public void kubernetesSendReceiveTest() {
    send(k8sClient)
        .message()
        .body("""
        { "command": "info" }
        """);

    receive(k8sClient)
        .message()
        .body("""
        {
          "command": "info",
          "result": {
            "clientVersion": "1.4.27",
            "apiVersion": "v1",
            "kind":"Info",
            "masterUrl": "${masterUrl}",
            "namespace": "test"
          }
        }
        """);

    send(k8sClient)
        .message()
        .body("""
        { "command": "list-pods" }
        """);

    receive(k8sClient)
        .message()
        .body("""
        {
          "command": "list-pods",
          "result": {
            "apiVersion":"v1",
            "kind":"PodList",
            "metadata":"@ignore@",
            "items":[]
          }
        }
        """)
        .validate(validation().json().expression("$.result.items.size()", 0));
}
```

Spring XML

```
<testcase name="KubernetesSendReceiveIT">
```

```
    <actions>
      <send endpoint="k8sClient">
        <message>
          <data>
            { "command": "info" }
          </data>
        </message>
      </send>

      <receive endpoint="k8sClient">
        <message type="json">
          <data>
          {
            "command": "info",
            "result": {
                "clientVersion": "1.4.27",
                "apiVersion": "v1",
                "kind":"Info",
                "masterUrl": "${masterUrl}",
                "namespace": "test"
            }
          }
          </data>
        </message>
      </receive>

      <echo>
        <message>List all pods</message>
      </echo>

      <send endpoint="k8sClient">
        <message>
          <data>
            { "command": "list-pods" }
          </data>
        </message>
      </send>

      <receive endpoint="k8sClient">
        <message type="json">
          <data>
          {
            "command": "list-pods",
            "result": {
                  "apiVersion":"v1",
                  "kind":"PodList",
                  "metadata":"@ignore@",
                  "items":[]
            }
          }
          </data>
```

```
          <validate path="$.result.items.size()" value="0"/>
        </message>
      </receive>
    </actions>
</testcase>
```

As you can see we can use the send/receive actions to call Kubernetes API commands and receive the respective results in Json format, too. This gives us the well known Json validation mechanism in Citrus in order to validate the results from Kubernetes. This way you can load Kubernetes resources verifying its state and properties. Of course JsonPath expressions also come in here in order to validate Json elements explicitly.

Chapter 39. Knative support

Knative lets you build Serverless and Event Driven applications on the cloud. The project knows the two main components Knative Serving and Knative Eventing.

Citrus as a test framework is able to interact with Knative to manage and leverage Serving and Eventing resources on a Kubernetes cluster.



The Knative support in Citrus is enabled by adding a separate Maven module as a dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-knative</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

39.1. Knative client

Citrus interacts with Knative resources on the Kubernetes cluster. This means we need to connect as a client to a Kubernetes cluster as part of the test. Citrus uses the Fabric8 Knative client implementation for this connection. You can create and configure the client once and reuse the client instance in multiple test cases.

Citrus Bean

```
@BindToRegistry
public KubernetesClient kubernetesClient() {
    return new KubernetesClientBuilder().build();
}

@BindToRegistry
public KnativeClient knativeClient() {
    return kubernetesClient().adapt(KnativeClient.class);
}
```

Spring Bean

```
@Bean
public KubernetesClient kubernetesClient() {
    return new KubernetesClientBuilder().build();
}

@Bean
public KnativeClient knativeClient() {
    return kubernetesClient().adapt(KnativeClient.class);
}
```

The Fabric8 Knative client is based on the Kubernetes client implementation. You can bind the client instances as beans to the Citrus bean registry. You may then reference and inject the client to your tests using the `@CitrusResource` annotation. Spring beans may be autowired to the test case as usual with `@Autowired`.

39.2. Knative broker

Citrus is able to create Knative Eventing broker resources as part of the test.

Java

```
@CitrusTest
public void knativeBrokerTest() {
    given(knative()
            .client(knativeClient)
            .brokers()
            .create("my-broker")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="KnativeBrokerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative client="knativeClient" namespace="test">
          <create-broker name="my-broker"/>
        </knative>
    </actions>
</test>
```

YAML

```
name: KnativeBrokerTest
actions:
  - knative:
      client: "knativeClient"
      namespace: "test"
      createBroker:
        name: "my-broker"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The test above creates a new Knative broker resource in the given namespace. In the following you can connect to the broker to produce and consume events.



Citrus automatically removes the created Knative resources after the test. This is the default behavior that makes Citrus delete resources such as the Knative broker even when the test has finished in failure state. However, you can disable the auto removal by setting this system property or environment variable: `citrus.knative.auto.remove.resources=false` or `CITRUS_KNATIVE_AUTO_REMOVE_RESOURCES=false`

You can also disable the auto removal of the Knative broker on the test action, too:

Java

```
@CitrusTest
public void knativeBrokerTest() {
    given(knative()
            .client(knativeClient)
            .brokers()
            .create("my-broker")
            .autoRemoveResources(false)
            .inNamespace("my-namespace"));
}
```

39.3. Verify broker status

The test may also verify that a broker is running and in a given status condition on a Kubernetes namespace. By default, the verify action checks that the broker condition `Ready` is set to `True`. You can use the `isReady()` convenience method to make this explicit, or specify a custom condition with `condition("Addressable")`. The expected status value defaults to `True` but can be changed with `status("False")` to verify that a condition is not met.

Java

```
@CitrusTest
public void knativeBrokerTest() {
    given(knative()
            .client(knativeClient)
            .brokers()
            .verify("my-broker")
            .isReady()
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="KnativeBrokerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative client="knativeClient" namespace="test">
          <verify-broker name="my-broker" condition="Ready"/>
        </knative>
    </actions>
</test>
```

YAML

```
name: KnativeBrokerTest
actions:
  - knative:
      client: "knativeClient"
      namespace: "test"
      verifyBroker:
        name: "my-broker"
        condition: "Ready"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

You can verify any status condition on the broker resource and also check for a specific status value.

Java

```
@CitrusTest
public void knativeBrokerTest() {
    given(knative()
            .client(knativeClient)
            .brokers()
            .verify("my-broker")
            .condition("Addressable")
            .status("False")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="KnativeBrokerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative client="knativeClient" namespace="test">
          <verify-broker name="my-broker" condition="Addressable" status="False"/>
        </knative>
    </actions>
</test>
```

YAML

```
name: KnativeBrokerTest
actions:
  - knative:
      client: "knativeClient"
      namespace: "test"
      verifyBroker:
        name: "my-broker"
        condition: "Addressable"
        status: "False"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

39.4. Send events

Once a Knative broker is available we can start to produce some events with a Citrus test. Knative uses the CloudEvent data format out of the box. The test is able to set the CloudEvent attributes as well as the event data when pushing the event to the broker.

Java

```
@CitrusTest
public void produceEventTest() {
    when(knative()
            .event()
            .send()
            .brokerUrl("http://my-cluster.svc.cluster.local/my-namespace/my-broker")
            .eventData("""
            { "message": "Hello Knative event!" }
            """));
}
```

XML

```
<test name="ProduceEventTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative namespace="test">
          <send-event broker="http://my-cluster.svc.cluster.local/my-namespace/my-
broker">
            <event>
              <data>{ "message": "Hello Knative event!" }</data>
            </event>
          </send-event>
        </knative>
    </actions>
</test>
```

YAML

```
name: ProduceEventTest
actions:
    - knative:
      namespace: "test"
      sendEvent:
        broker: "my-broker"
        event:
          data: |
            { "message": "Hello Knative event!" }
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This action produces a new Knative event with some CloudEvent attributes default values set. The

Http request to the Knative broker looks like this:

```
POST http://my-cluster.svc.cluster.local/my-namespace/my-broker
Accept:text/plain, application/json, application/*+json, */*
Host:broker-ingress.knative-eventing.svc.cluster.local
Content-Type:application/json
Ce-Id:2818d613-bc75-4b25-b570-b825bbe33378
Ce-Type:org.citrusframework.event.test
Ce-Specversion:1.0
Ce-Source:citrus-test
Content-Length:54

{ "message": "Hello Knative event!" }
```

You can see the CloudEvent attributes set with `Ce-*` headers. You can customize these attributes on the send event test action.

Java

```
@CitrusTest
public void produceEventTest() {
    when(knative()
            .event()
            .send()
            .brokerUrl("http://my-cluster.svc.cluster.local/my-namespace/my-broker")
            .attribute("ce-type", "org.citrusframework.knative.event")
            .eventData("""
            { "message": "Hello Knative event!" }
            """));
}
```

XML

```
<test name="ProduceEventTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative namespace="test">
          <send-event broker="http://my-cluster.svc.cluster.local/my-namespace/my-
broker">
            <event>
              <ce-attributes>
                <ce-attribute name="ce-type"
value="org.citrusframework.knative.event"/>
              </ce-attributes>
              <data>{ "message": "Hello Knative event!" }</data>
            </event>
          </send-event>
        </knative>
    </actions>
</test>
```

YAML

```
name: ProduceEventTest
actions:
    - knative:
      namespace: "test"
      sendEvent:
        broker: "my-broker"
        event:
          data: |
            { "message": "Hello Knative event!" }
          attributes:
            - name: ce-type
              value: "org.citrusframework.knative.event"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

39.5. Receive events

The Knative broker dispatches events by invoking services. Usually this service is an arbitrary Kubernetes Service resource or any addressable resource on the Kubernetes cluster. The events that should be consumed by a service gets specified in a Knative Trigger resource.

This means we need to consume Knative events with a Kubernetes service resource. Citrus provides

special Kubernetes support that enables you to create a Kubernetes service resource on the cluster. Read more about it in chapter Kubernetes support.

You can create the Kubernetes service as follows:

Java

```
@CitrusTest
public void createServiceTest() {
    given(kubernetes()
            .client(k8sClient)
            .service()
            .create("my-service")
            .portMapping(80, 8080)
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateServiceTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <kubernetes client="k8sClient" namespace="my-namespace">
          <create-service name="my-service">
            <ports>
              <port-mapping port="80" target-port="8080"/>
            </ports>
          </create-service>
        </kubernetes>
    </actions>
</test>
```

YAML

```
name: CreateServiceTest
actions:
  - kubernetes:
      client: "k8sClient"
      namespace: "my-namespace"
      createService:
        name: "my-service"
        ports:
          - port: "80"
            targetPort: "8080"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This creates the Kubernetes service named `my-service` in the namespace `my-namespace`.



The test requires a local HttServer instance to actually receive the requests on the given port mapping. Citrus automatically creates this local HttpServer instance. Please make sure to choose a free port on your host machine when defining the port mapping.



Before creating the HttpServer instance as part of the Kubernetes test action Citrus tries to resolve the HttpServer instance from the Citrus bean registry. This means you can also predefine the HttpServer instance as a usual bean in the registry. Just give the server name as an additional attribute to the Kubernetes service create test action so Citrus knows which server instance should be resolved via the bean registry.

This creates a proper Kubernetes service and binds it to the local port where the local HttpServer instance is listening for incoming requests. Before we can start to actually consume events we need to create a Knative Trigger resource. You can create the Trigger resource as part of the test, too:

Java

```
@CitrusTest
public void createTriggerTest() {
    given(knative()
            .client(knativeClient)
            .trigger()
            .create("my-trigger")
            .broker("my-broker")
            .service("my-service")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateTriggerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <knative client="knativeClient" namespace="my-namespace">
        <create-trigger name="my-trigger" broker="my-broker" service="my-service"/>
      </knative>
    </actions>
</test>
```

YAML

```
name: CreateTriggerTest
actions:
  - knative:
      client: "knativeClient"
      namespace: "my-namespace"
      createTrigger:
        name: "my-trigger"
        broker: "my-broker"
        service: "my-service"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The Trigger resource receives the broker name that should be used as a source of events and the service name as a subject to being invoked when an event gets dispatched. The trigger name and the namespace defines where the Trigger resource is created on the Kubernetes cluster.

You add a filter to the Trigger resource to filter events on CloudEvent attributes.

Java

```
@CitrusTest
public void createTriggerTest() {
    given(knative()
            .client(knativeClient)
            .trigger()
            .create("my-trigger")
            .broker("my-broker")
            .service("my-service")
            .filter("type", "my.very.specific.event.type")
            .inNamespace("my-namespace"));
}
```

XML

```
<test name="CreateTriggerTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <knative client="knativeClient" namespace="my-namespace">
        <create-trigger name="my-trigger" broker="my-broker" service="my-service">
          <filter>
            <attribute name="type" value="my.very.specific.event.type"/>
          </filter>
        </create-trigger>
      </knative>
    </actions>
</test>
```

YAML

```
name: CreateTriggerTest
actions:
  - knative:
      client: "knativeClient"
      namespace: "my-namespace"
      createTrigger:
        name: "my-trigger"
        broker: "my-broker"
        service: "my-service"
        filter:
          attributes:
            - name: type
              value: "my.very.specific.event.type"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The filter is set on the CloudeEvent type attribute so the trigger will only dispatch events with this specific type. The created Trigger resource looks like follows:

Knative Trigger

```
apiVersion: eventing.knative.dev/v1
kind: Trigger
metadata:
  name: my-trigger
spec:
  broker: default
  filter:
    attributes:
      type: my.very.specific.event.type
  subscriber:
    ref:
      apiVersion: v1
      kind: Service
      name: my-service
```

This completes all resources that we need to consume events from the Knative eventing broker. The following test action receives and verifies a Knative event with an expected CloudEvent message holding expected attributes and event data.

Java

```
@CitrusTest
public void consumeEventTest() {
    when(knative()
            .event()
            .receive()
            .serviceName("my-service")
            .attribute("ce-type", "org.citrusframework.knative.event")
            .eventData("""
            {
                "message": "Hello Knative event!"
            }
            """));
}
```

XML

```
<test name="ConsumeEventTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <knative namespace="my-namespace">
          <receive-event service="my-service">
            <event>
              <ce-attributes>
                <ce-attribute name="ce-type"
value="org.citrusframework.knative.event"/>
              </ce-attributes>
              <data>{ "message": "Hello Knative event!" }</data>
            </event>
          </receive-event>
        </knative>
    </actions>
</test>
```

YAML

```
name: ConsumeEventTest
actions:
    - knative:
      namespace: "my-namespace"
      receiveEvent:
        service: "my-service"
        event:
          data: |
            { "message": "Hello Knative event!" }
          attributes:
            - name: ce-type
              value: "org.citrusframework.knative.event"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

39.6. Knative channels

TODO

39.7. Knative subscription

TODO

Chapter 40. Testcontainers support

Testcontainers leverage Docker containers in your tests to provide lightweight infrastructure instances such as databases, message brokers and web browsers. Anything that you can run in a Docker container is subject to be managed by Testcontainers in your test. Citrus as a framework is able to start/stop your Testcontainers instances as part of the test.



The Testcontainers support is enabled by adding a separate Maven module as a project dependency

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-testcontainers</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

40.1. Start and stop Testcontainers

The Testcontainers test action runs a container and exposes its connection settings as test variables. You can start any Docker container image as a Testcontainers instance. The Testcontainers project provides many modules that represent different test infrastructure technologies.

Each of those modules can be started in Citrus with a special test action:

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .start()
            .image("busybox:latest")
            .containerName("busybox-container")
            .withCommand("echo", "Hello", "World")
            .withLabel("app", "citrus"));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <container name="busybox-container" image="busybox:latest" command="echo
Hello World">
              <labels>
                <label name="app" value="citrus"/>
              </labels>
            </container>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        container:
          name: "busybox-container"
          image: "busybox:latest"
          command: "echo Hello World"
          labels:
            - name: app
              value: citrus
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

In the test example uses the Docker image name `busybox:latest` to start the Testcontainer instance. You can set labels, annotations as well as the command that is run as entry point when starting the container.

You can also leverage the Testcontainers API to specify the container details:

Java

```
@CitrusTest
public void testcontainersTest() {
    GenericContainer<?> busyBox = new GenericContainer("busybox:latest")
                .withCommand("echo", "Hello World");

    given(testcontainers()
            .start()
            .container("my-container", busyBox));
}
```



The Citrus test automatically exposes connection details of the running Testcontainers instance. The connection settings get exposed in the form of test variables.

These are the test variables that get exposed representing connection settings for the Testcontainers instance.

Variable
CITRUS_TESTCONTAINERS_<CONTAINER_NAME>_HOST
CITRUS_TESTCONTAINERS_<CONTAINER_NAME>_PORT
CITRUS_TESTCONTAINERS_<CONTAINER_NAME>_CONTAINER_IP
CITRUS_TESTCONTAINERS_<CONTAINER_NAME>_CONTAINER_ID
CITRUS_TESTCONTAINERS_<CONTAINER_NAME>_CONTAINER_NAME

You can use these test variables in the test to access the details of the running Testcontainers instance. Special containers may expose additional connection settings such as user credentials, endpoint URLs, managed ports and so on. Read about this in the individual supported Testcontainers module sections that follow in this guide.



Each Testcontainers instance that is started from Citrus is automatically removed after the test. This is the default behavior. However, you can disable the auto removal by setting this system property or environment variable: `citrus.testcontainers.auto.remove.resources=false` or `CITRUS_TESTCONTAINERS_AUTO_REMOVE_RESOURCES=false`

You can also disable the auto removal of the started Testcontainers started by Citrus:

Java

```
@CitrusTest
public void testcontainersTest() {
    GenericContainer<?> busyBox = new GenericContainer("busybox:latest")
                .withCommand("echo", "Hello World");

    given(testcontainers()
            .start()
            .autoRemove(false)
            .container("my-container", busyBox));
}
```

This keeps the started Testcontainers instance running even after the test has finished. When the whole test suite is finished the Testcontainers library may automatically stop and remove the container.

You can explicitly stop a running Testcontainers instance by its name with this test action:

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .stop()
            .containerName("busybox-container"));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <stop name="busybox-container"/>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      stop:
        name: "busybox-container"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

This is how you can manage any Docker image as a Testcontainers instance. Citrus also provides special support for test infrastructure Testcontainers modules such as:

- PostgreSQL
- MongoDB
- LocalStack
- Kafka
- Redpanda

The following sections deal with these special Testcontainers modules.

40.2. PostgreSQL

You can start a PostgreSQL instance with the following test action.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .postgreSQL()
            .start()
            .initScript(Resources.create("db.init.sql")));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <postgresql>
              <init-script file="classpath:db.init.sql"/>
            </postgresql>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        postgresql:
          initScript:
            file: "db.init.sql"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

In the test example the PostgreSQL Testcontainers module is started. The given init script is used to initialize the database on startup.

Citrus exposes special test variables that represent connection settings for the PostgreSQL container:

Variable
CITRUS_TESTCONTAINERS_POSTGRESQL_HOST
CITRUS_TESTCONTAINERS_POSTGRESQL_CONTAINER_IP
CITRUS_TESTCONTAINERS_POSTGRESQL_CONTAINER_ID
CITRUS_TESTCONTAINERS_POSTGRESQL_CONTAINER_NAME
CITRUS_TESTCONTAINERS_POSTGRESQL_SERVICE_PORT
CITRUS_TESTCONTAINERS_POSTGRESQL_PORT
CITRUS_TESTCONTAINERS_POSTGRESQL_LOCAL_URL
CITRUS_TESTCONTAINERS_POSTGRESQL_SERVICE_LOCAL_URL
CITRUS_TESTCONTAINERS_POSTGRESQL_SERVICE_NAME
CITRUS_TESTCONTAINERS_POSTGRESQL_SERVICE_URL
CITRUS_TESTCONTAINERS_POSTGRESQL_URL
CITRUS_TESTCONTAINERS_POSTGRESQL_DRIVER
CITRUS_TESTCONTAINERS_POSTGRESQL_DB_NAME
CITRUS_TESTCONTAINERS_POSTGRESQL_USERNAME
CITRUS_TESTCONTAINERS_POSTGRESQL_PASSWORD

In addition to that the Citrus test action exposes a DataSource that connects to the PostgreSQL database and binds it as abean to the Citrus registry. This means that you can resolve the data

source bean via the test context reference resolver.



You can set the name of the exposed data source bean on each Citrus Testcontainers test action for PostgreSQL. By default, Citrus exposed the data source bean with the name `postgreSQL`.

40.3. MongoDB

You can start a MongoDB instance with the following test action.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .mongodb()
            .start());
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <mongodb/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        mongodb: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

In the test example the MongoDB Testcontainers module is started. Citrus exposes special test variables that represent connection settings for the MongoDB container:

Variable
CITRUS_TESTCONTAINERS_MONGODB_HOST
CITRUS_TESTCONTAINERS_MONGODB_CONTAINER_IP
CITRUS_TESTCONTAINERS_MONGODB_CONTAINER_ID
CITRUS_TESTCONTAINERS_MONGODB_CONTAINER_NAME
CITRUS_TESTCONTAINERS_MONGODB_LOCAL_URL
CITRUS_TESTCONTAINERS_MONGODB_SERVICE_PORT
CITRUS_TESTCONTAINERS_MONGODB_PORT
CITRUS_TESTCONTAINERS_MONGODB_SERVICE_LOCAL_URL
CITRUS_TESTCONTAINERS_MONGODB_SERVICE_NAME
CITRUS_TESTCONTAINERS_MONGODB_SERVICE_URL
CITRUS_TESTCONTAINERS_MONGODB_URL
CITRUS_TESTCONTAINERS_MONGODB_CONNECTION_STRING

You can use these test variables to connect to the MongoDB Testcontainers instance.

40.4. LocalStack

The LocalStack Testcontainers module allows you to start lightweight Amazon WebServices such as AWS S3, AWS SQS, AWS SNS or AWS Kinesis. You can start a LocalStack instance with the following test action.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .localstack()
            .start()
            .withService(LocalStackContainer.Service.S3));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <localstack services="S3"/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        localstack:
          services:
            - "S3"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

Each LocalStack instance receives a list of services that should be enabled. In the example above the AWS S3 service is enabled.

40.4.1. Container security settings

The LocalStack container uses some default dummy values for `secretKey` and `accessKey`. You may set these properties explicitly as well as the region when starting the container.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .localstack()
            .start()
            .secretKey("${secretKey}")
            .accessKey("${accessKey}")
            .region("us-east-1")
            .withService(LocalStackContainer.Service.S3));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <localstack
              secret-key="${secretKey}"
              access-key="${accessKey}"
              region="us-east-1"
              services="S3"/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        localstack:
          secretKey: '${secretKey}'
          accessKey: '${accessKey}'
          region: us-east-1
          services:
            - "S3"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

As you can see the settings support test variables so you may load these values from a local file or as a global variable via env setting.

You may also use one of these envVars for a global and secure way of setting these values:

- CITRUS_TESTCONTAINERS_LOCALSTACK_SECRET_KEY
- CITRUS_TESTCONTAINERS_LOCALSTACK_ACCESS_KEY
- CITRUS_TESTCONTAINERS_LOCALSTACK_REGION
- CITRUS_TESTCONTAINERS_LOCALSTACK_AUTH_TOKEN

In the `citrus-application.properties` you can set these keys:

- citrus.testcontainers.localstack.secret.key
- citrus.testcontainers.localstack.access.key
- citrus.testcontainers.localstack.region
- citrus.testcontainers.localstack.auth.token



Since mid 2026 newer versions of the LocalStack container require an auth token. You may obtain your personal token from the official LocalStack website.

You can use the `authToken()` setting when starting the container with the respective test action.

40.4.2. Exposed connection settings

Citrus exposes special test variables that represent connection settings for the LocalStack container:

Variable
CITRUS_TESTCONTAINERS_LOCALSTACK_HOST
CITRUS_TESTCONTAINERS_LOCALSTACK_CONTAINER_IP
CITRUS_TESTCONTAINERS_LOCALSTACK_CONTAINER_ID
CITRUS_TESTCONTAINERS_LOCALSTACK_CONTAINER_NAME
CITRUS_TESTCONTAINERS_LOCALSTACK_REGION
CITRUS_TESTCONTAINERS_LOCALSTACK_ACCESS_KEY
CITRUS_TESTCONTAINERS_LOCALSTACK_SECRET_KEY
CITRUS_TESTCONTAINERS_LOCALSTACK_SERVICE_PORT
CITRUS_TESTCONTAINERS_LOCALSTACK_SERVICE_LOCAL_URL

Variable
CITRUS_TESTCONTAINERS_LOCALSTACK_SERVICE_NAME
CITRUS_TESTCONTAINERS_LOCALSTACK_SERVICE_URL

For each enabled service on the LocalStack instance these variables are exposed:

Variable
CITRUS_TESTCONTAINERS_LOCALSTACK_<SERVICE_NAME>_URL
CITRUS_TESTCONTAINERS_LOCALSTACK_<SERVICE_NAME>_LOCAL_URL
CITRUS_TESTCONTAINERS_LOCALSTACK_<SERVICE_NAME>_PORT

40.4.3. Service client auto creation

The Citrus LocalStack container action is able to automatically create clients for the enabled services.

By default, Citrus creates the client instances and stores them in the bean Citrus registry. You can then reference the clients by their name.

The following services support auto creation of clients:

- S3 ("s3Client")
- SNS ("snsClient")
- SQS ("sqsClient")
- KINESIS (kinesisClient)
- DYNAMODB (dnyamodbClient)
- EVENT_BRIDGE("eventbridgeClient")

For instance the S3 service creates a client instance called `s3Client`. The clients are automatically configured with the container service settings such as endpointUri, accessKey, secretKey and region for S3.

The underlying client factory is able to handle specific options. For the S3 client factory for instance this is a comma separated list of bucket names to auto create S3 buckets.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .localstack()
            .start()
            .withService(LocalStackContainer.Service.S3))
            .withOption("buckets", "my_bucket_name,another_bucket_name");
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <localstack services="S3">
              <options>
                <option name="buckets" value="my_bucket_name,another_bucket_name"/>
              </options>
            </localstack>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        localstack:
          services:
            - "S3"
          options:
            buckets: "my_bucket_name,another_bucket_name"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

Please see this list of supported options on the individual client factories:

Client	Option	Description
s3Client	buckets	Comma separated list of bucket names that get auto created
sqsClient	queues	Comma separated list of queue names that get auto created
snsClient	topics	Comma separated list of topic names that get auto created
kinesisClient	streams	Comma separated list of stream names that get auto created

Client	Option	Description
kinesisClient	<stream_name>.shard.count	Shard count set on the given stream
dynamodbClient	tables	Comma separated list of table names that get auto created.
dynamodbClient	<table_name>.id	Id attribute name set on the table as a primary id.
dynamodbClient	<table_name>.view.type	View type set on the table stream specification (default=NEW_AND_OLD_IMAGES).
dynamodbClient	<table_name>.read.capacity	Read capacity set as provisioned throughput (default=1).
dynamodbClient	<table_name>.write.capacity	Write capacity set as provisioned throughput (default=1).
eventbridgeClient	eventBusNames	Comma separated list of event bus names that get auto created

40.4.4. Custom service clients

You can use the exposed connections settings in the test variables to connect to the LocalStack Testcontainers instance.

As an example this S3 client connects to the LocalStack Testcontainers instance and creates a test bucket:

Java

```
private S3Client createS3Client(TestContext context) {
    S3Client s3 = S3Client
            .builder()

.endpointOverride(URI.create(context.getVariable("${CITRUS_TESTCONTAINERS_LOCALSTACK_S
3_URL}")))
            .credentialsProvider(StaticCredentialsProvider.create(
                    AwsBasicCredentials.create(

context.getVariable("${CITRUS_TESTCONTAINERS_LOCALSTACK_ACCESS_KEY}"),

context.getVariable("${CITRUS_TESTCONTAINERS_LOCALSTACK_SECRET_KEY}"))
            ))
            .forcePathStyle(true)

.region(Region.of(context.getVariable("${CITRUS_TESTCONTAINERS_LOCALSTACK_REGION}")))
            .build();

    s3.createBucket(b -> b.bucket(context.getVariable("${aws.s3.bucketNameOrArn}")));

    return s3;
}
```



You can access the Citrus TesContext via `@CitrusResource` injection in your test class or method.

40.5. Kafka

You can start a Kafka instance with the following test action.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .kafka()
            .start());
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <kafka/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        kafka: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

The example above uses a special test actions to start the Kafka container.

40.5.1. Kafka container implementations

Citrus provides support for different Kafka container implementations to give you flexibility in choosing the most suitable option for your testing needs:

- **Confluent Platform** (default) - The Confluent Community Edition Kafka distribution
- **Apache Kafka** - The official Apache Kafka distribution
- **Apache Kafka Native** - The Apache Kafka distribution with native GraalVM compilation for faster startup
- **Strimzi** - A Kubernetes-native Kafka distribution based on Apache Kafka

By default, Citrus uses the Confluent Platform implementation. You can specify a different implementation using the `implementation()` method:

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .kafka()
            .start()
            .implementation(KafkaImplementation.STRIMZI));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <kafka implementation="STRIMZI"/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        kafka:
          implementation: "STRIMZI"
```

Available implementation values are:

- `CONFLUENT` - Uses the `confluentinc/cp-kafka` image (default)
- `APACHE` - Uses the `apache/kafka` image
- `APACHE_NATIVE` - Uses the `apache/kafka-native` image (GraalVM native compilation for faster startup)
- `STRIMZI` - Uses the `quay.io/strimzi/kafka` image
- `DEFAULT` - Same as `CONFLUENT`

Choosing between APACHE and APACHE_NATIVE

Both `APACHE` and `APACHE_NATIVE` implementations use the official Apache Kafka distribution, but there are important differences:

- **APACHE** uses the standard JVM-based `apache/kafka` container image

- **APACHE_NATIVE** uses the `apache/kafka-native` container image, which is compiled ahead-of-time using GraalVM native image technology

The main advantage of `APACHE_NATIVE` is significantly faster container startup time, which can reduce test execution time, especially when running multiple tests that start Kafka containers. However, it has the same runtime behavior and compatibility as the standard `APACHE` implementation.

Use `APACHE_NATIVE` when: * You want faster test execution with reduced container startup times * Your test suite frequently starts and stops Kafka containers * You are comfortable using GraalVM-compiled binaries

Use `APACHE` when: * You need the standard JVM-based Kafka distribution * You want to ensure full compatibility with the traditional Kafka deployment model

40.5.2. Configuration options

You can customize the Kafka container with additional settings:

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .kafka()
            .start()
            .implementation(KafkaImplementation.APACHE)
            .version("4.2.1")
            .port(9092));
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <kafka implementation="APACHE" version="4.2.1" port="9092"/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        kafka:
          implementation: "APACHE"
          version: "4.2.1"
          port: 9092
```

You may also use system properties or environment variables to configure the default Kafka container settings:

Setting	System Property	Environment Variable
Implementation	citrus.testcontainers.kafka.implementation	CITRUS_TESTCONTAINERS_KAFKA_IMPLEMENTATION
Confluent image name	citrus.testcontainers.kafka.confluent.image.name	CITRUS_TESTCONTAINERS_KAFKA_CONFLUENT_IMAGE_NAME
Confluent version	citrus.testcontainers.kafka.confluent.version	CITRUS_TESTCONTAINERS_KAFKA_CONFLUENT_VERSION
Apache image name	citrus.testcontainers.kafka.apache.image.name	CITRUS_TESTCONTAINERS_KAFKA_APACHE_IMAGE_NAME
Apache version	citrus.testcontainers.kafka.apache.version	CITRUS_TESTCONTAINERS_KAFKA_APACHE_VERSION
Apache Native image name	citrus.testcontainers.kafka.apache.native.image.name	CITRUS_TESTCONTAINERS_KAFKA_APACHE_NATIVE_IMAGE_NAME
Apache Native version	citrus.testcontainers.kafka.apache.native.version	CITRUS_TESTCONTAINERS_KAFKA_APACHE_NATIVE_VERSION
Strimzi image name	citrus.testcontainers.kafka.strimzi.image.name	CITRUS_TESTCONTAINERS_KAFKA_STRIMZI_IMAGE_NAME
Strimzi version	citrus.testcontainers.kafka.strimzi.version	CITRUS_TESTCONTAINERS_KAFKA_STRIMZI_VERSION
Service name	citrus.testcontainers.kafka.service.name	CITRUS_TESTCONTAINERS_KAFKA_SERVICE_NAME
Container name	citrus.testcontainers.kafka.container.name	CITRUS_TESTCONTAINERS_KAFKA_CONTAINER_NAME
Startup timeout (seconds)	citrus.testcontainers.kafka.startup.timeout	CITRUS_TESTCONTAINERS_KAFKA_STARTUP_TIMEOUT

40.5.3. Exposed connection settings

Citrus exposes special test variables that represent connection settings for the Kafka container:

Variable
CITRUS_TESTCONTAINERS_KAFKA_HOST
CITRUS_TESTCONTAINERS_KAFKA_CONTAINER_IP
CITRUS_TESTCONTAINERS_KAFKA_CONTAINER_ID
CITRUS_TESTCONTAINERS_KAFKA_CONTAINER_NAME
CITRUS_TESTCONTAINERS_KAFKA_LOCAL_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_KAFKA_SERVICE_PORT
CITRUS_TESTCONTAINERS_KAFKA_PORT
CITRUS_TESTCONTAINERS_KAFKA_SERVICE_LOCAL_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_KAFKA_SERVICE_NAME
CITRUS_TESTCONTAINERS_KAFKA_SERVICE_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_KAFKA_BOOTSTRAP_SERVERS

You can use these test variables to connect to the Kafka Testcontainers instance.

40.6. Redpanda

You can start a Redpanda instance with the following test action.

Java

```
@CitrusTest
public void testcontainersTest() {
    given(testcontainers()
            .redpanda()
            .start());
}
```

XML

```
<test name="TestcontainersTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <start>
            <redpanda/>
          </start>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: TestcontainersTest
actions:
  - testcontainers:
      start:
        redpanda: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```

In the test example the Redpanda Testcontainers module is started. Citrus exposes special test variables that represent connection settings for the Redpanda container:

Variable
CITRUS_TESTCONTAINERS_REDPANDA_HOST
CITRUS_TESTCONTAINERS_REDPANDA_CONTAINER_IP
CITRUS_TESTCONTAINERS_REDPANDA_CONTAINER_ID
CITRUS_TESTCONTAINERS_REDPANDA_CONTAINER_NAME
CITRUS_TESTCONTAINERS_REDPANDA_LOCAL_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_REDPANDA_SERVICE_PORT
CITRUS_TESTCONTAINERS_REDPANDA_PORT
CITRUS_TESTCONTAINERS_REDPANDA_SERVICE_LOCAL_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_REDPANDA_SERVICE_NAME
CITRUS_TESTCONTAINERS_REDPANDA_SERVICE_BOOTSTRAP_SERVERS
CITRUS_TESTCONTAINERS_REDPANDA_BOOTSTRAP_SERVERS

You can use these test variables to connect to the Redpanda Testcontainers instance.

40.7. Docker compose

The Testcontainers project provides the opportunity to interact with Docker compose. In Citrus you can start a Docker compose specification (`compose.yaml`) as part of the test.



By default, the Testcontainers compose container uses your local Docker compose executable binary. This means you need to have Docker compose installed on your machine. You can verify the installation with (`docker compose version`).



If for some reason you are not able to install Docker compose you may also disable the `useComposeBinary` setting in Citrus (e.g. via System property or environment variable). The Testcontainers library will then use arbitrary Docker containers in combination with an ambassador container that stars and manages the services defined in the Docker compose specification.

A Docker compose specification may look like this:

compose.yaml

```
services:
  web:
    image: nginx:latest
    ports:
      - '8080:80'
    volumes:
      - ./html:/usr/share/nginx/html:ro
```

The specification defines one or more services that are started as part of the `compose up` command:

Java

```
@CitrusTest
public void composeTest() {
    given(testcontainers()
            .compose()
            .up()
            .file("compose.yaml"));
}
```

XML

```
<test name="ComposeTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <compose>
            <up file="compose.yaml"/>
          </compose>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: ComposeTest
actions:
  - testcontainers:
      compose:
        up:
          file: "compose.yaml"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



The Docker containers started with Docker compose will be automatically stopped and removed after the test. You can change this behavior by setting one of the System property or environment variable settings: `citrus.testcontainers.compose.auto.remove.resources=false` or `CITRUS_TESTCONTAINERS_COMPOSE_AUTO_REMOVE_RESOURCES=false`

You may explicitly stop all Docker containers with the `compose down` command:

Java

```
@CitrusTest
public void composeTest() {
    given(testcontainers()
            .compose()
            .down());
}
```

XML

```
<test name="ComposeTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <testcontainers>
          <compose>
            <down/>
          </compose>
        </testcontainers>
    </actions>
</test>
```

YAML

```
name: ComposeTest
actions:
  - testcontainers:
      compose:
        down: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <!-- NOT SUPPORTED -->
</spring:beans>
```



The `compose down` test action searches for a single Testcontainers compose container that has been started before. In case you are operating with multiple `compose up` commands in a test you may identify each of those compose containers with a unique name and reference this name in the `compose down` test action.

Chapter 41. Functions

The test framework will offer several functions that are useful throughout the test execution. The functions will always return a string value that is ready for use as variable value or directly inside a text message.

A set of functions is usually combined to a function library. The library has a prefix that will identify the functions inside the test case. The default test framework function library uses a default prefix (citrus). You can write your own function library using your own prefix in order to extend the test framework functionality whenever you want.

The library is built in the Spring configuration and contains a set of functions that are of public use.

Citrus Bean

```
@BindToRegistry
public FunctionLibrary testFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("test");
    library.setPrefix("foo:");
    library.getMembers().put("randomNumber", new RandomNumberFunction());
    library.getMembers().put("randomString", new RandomStringFunction());
    library.getMembers().put("customFunction", customFunctionBean());
    return library;
}
```

Spring Bean

```
@Bean
public FunctionLibrary testFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("test");
    library.setPrefix("foo:");
    library.getMembers().put("randomNumber", new RandomNumberFunction());
    library.getMembers().put("randomString", new RandomStringFunction());
    library.getMembers().put("customFunction", customFunctionBean());
    return library;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd">

    <citrus:function-library id="testFunctionLibrary" prefix="foo:">
              <citrus:function name="randomNumber"
class="org.citrusframework.functions.RandomNumberFunction"/>
              <citrus:function name="randomString"
class="org.citrusframework.functions.RandomStringFunction"/>
              <citrus:function name="customFunction" ref="customFunctionBean"/>
              ...
    </citrus:function-library>
</beans>
```

As you can see the library defines one-to-many functions either referenced as normal Spring bean or by its implementing Java class name. Citrus constructs the library, and you are able to use the functions in your test case with the leading library prefix just like this:

```
foo:randomNumber()
foo:randomString()
foo:customFunction()
```



You can add custom function implementations and custom function libraries. Just use a custom prefix for your library. The default Citrus function library uses the **citrus:** prefix.In the next chapters the default functions offered by the framework will be described in detail.

41.1. concat()

The function will combine several string tokens to a single string value. This means that you can combine a static text value with a variable value for instance. A first example should clarify the usage:

Java

```
@CitrusTest
public void concatFunctionTest() {
    variable("date", "citrus:currentDate(yyyy-MM-dd)");
    variable("text", "Hello Test Framework!");

    then(echo().message("citrus:concat('Today is: ', ${date}, ' right!?')"));
    then(echo().message("citrus:concat('Text is: ', ${text})"));
}
```

XML

```
<test name="ConcatFunctionTest"
xmlns="http://citrusframework.org/schema/xml/testcase">
  <variables>
    <variable name="date" value="citrus:currentDate(yyyy-MM-dd)" />
    <variable name="text" value="Hello Test Framework!" />
  </variables>
  <actions>
    <echo>
      <message>
        citrus:concat('Today is: ', ${date}, ' right!?')
      </message>
    </echo>
    <echo>
      <message>
        citrus:concat('Text is: ', ${text})
      </message>
    </echo>
  </actions>
</test>
```

YAML

```
name: ConcatFunctionTest
variables:
  - name: date
    value: "citrus:currentDate(yyyy-MM-dd)"
  - name: text
    value: "Hello Test Framework!"
actions:
  - echo:
      message: "citrus:concat('Today is: ', ${date}, ' right!?')"
  - echo:
      message: "citrus:concat('Text is: ', ${text})"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <testcase name="concatFunctionTest">
        <variables>
            <variable name="date" value="citrus:currentDate(yyyy-MM-dd)" />
            <variable name="text" value="Hello Test Framework!" />
        </variables>
        <actions>
            <echo>
                <message>
                    citrus:concat('Today is: ', ${date}, ' right!?')
                </message>
            </echo>
            <echo>
                <message>
                    citrus:concat('Text is: ', ${text})
                </message>
            </echo>
        </actions>
    </testcase>
</spring:beans>
```

Please do not forget to mark static text with single quote signs. There is no limitation for string tokens to be combined.

```
citrus:concat('Text1', 'Text2', 'Text3', ${text}, 'Text5', …, 'TextN')
```

The function can be used wherever variables can be used. For instance when validating XML elements via XPath expressions in a `receive` test action.

Java

```
@CitrusTest
public void functionsTest() {
    when(receive()
        .endpoint(someEndpoint)
        .validate(validation().xpath()
                    .expression("//element/element", "citrus:concat('Cx1x',
${generatedId})"))
    );
}
```

XML

```
<test name="FunctionsTest" xmlns="http://citrusframework.org/schema/xml/testcase">
  <actions>
    <receive endpoint="someEndpoint">
      <validate path="//element/element" value="citrus:concat('Cx1x',
${generatedId})"/>
    </receive>
  </actions>
</test>
```

YAML

```
name: FunctionsTest
actions:
  - receive:
      endpoint: "soapClient"
      message:
        body:
          data: {}
        expression:
          - path: "//element/element"
            value: "citrus:concat('Cx1x', ${generatedId})"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
              http://www.springframework.org/schema/beans/spring-beans.xsd
              http://www.citrusframework.org/schema/testcase
              http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

  <testcase name="functionsTest">
    <actions>
      <receive endpoint="someEndpoint">
        <message>
            <validate path="//element/element" value="citrus:concat('Cx1x',
${generatedId})"/>
        </message>
      </receive>
    </actions>
  </testcase>
</spring:beans>
```

41.2. substring()

The function will have three parameters.

1. String to work on
2. Starting index
3. End index (optional)

Let us have a look at a simple example for this function:

```
<echo>
    <message>
        citrus:substring('Hello Test Framework', 6)
    </message>
</echo>
<echo>
    <message>
        citrus:substring('Hello Test Framework', 0, 5)
    </message>
</echo>
```

Function output:

```
Test Framework
Hello
```

41.3. stringLength()

The function will calculate the number of characters in a string representation and return the number.

```
<echo>
    <message>citrus:stringLength('Hello Test Framework')</message>
</echo>
```

Function output:

20

41.4. translate()

This function will replace regular expression matching values inside a string representation with a specified replacement string.

```
<echo>
    <message>
        citrus:translate('H.llo Test Fr.mework', '\.', 'a')
    </message>
</echo>
```

Note that the second parameter will be a regular expression. The third parameter will be a simple replacement string value.

Function output:

Hello Test Framework

41.5. substringBefore()

The function will search for the first occurrence of a specified string and will return the substring before that occurrence. Let us have a closer look in a simple example:

```
<echo>
    <message>
        citrus:substringBefore('Test/Framework', '/')
    </message>
</echo>
```

In the specific example the function will search for the '/' character and return the string before that index.

Function output:

Test

41.6. substringAfter()

The function will search for the first occurrence of a specified string and will return the substring after that occurrence. Let us clarify this with a simple example:

```
<echo>
    <message>
        citrus:substringAfter('Test/Framework', '/')
    </message>
</echo>
```

Similar to the substringBefore function the '/' character is found in the string. But now the remaining string is returned by the function meaning the substring after this character index.

Function output:

41.7. round()

This is a simple mathematical function that will round decimal numbers representations to their nearest non-decimal number.

```
<echo>
    <message>citrus:round('3.14')</message>
</echo>
```

Function output:

3

41.8. floor()

This function will round down decimal number values.

```
<echo>
    <message>citrus:floor('3.14')</message>
</echo>
```

Function output:

3.0

41.9. ceiling()

Similar to floor function, but now the function will round up the decimal number values.

```
<echo>
    <message>citrus:ceiling('3.14')</message>
</echo>
```

Function output:

4.0

41.10. randomNumber()

The random number function will provide you the opportunity to generate random number strings containing positive number letters. There is a singular Boolean parameter for that function describing whether the generated number should have exactly the amount of digits. Default value for this padding flag will be true.

Next example will show the function usage:

```
<variables>
    <variable name="rndNumber1" value="citrus:randomNumber(10)"/>
    <variable name="rndNumber2" value="citrus:randomNumber(10, true)"/>
    <variable name="rndNumber2" value="citrus:randomNumber(10, false)"/>
    <variable name="rndNumber3" value="citrus:randomNumber(3, false)"/>
</variables>
```

Function output:

```
8954638765
5003485980
6387650
65
```

41.11. randomNumberGenerator()

This function is the big brother of randomNumber it generates a random numeric value with customizable decimal precision and range. It provides options to control the minimum and maximum values, inclusivity, and numerical multiples of the generated number.

The function accepts up to six parameters, which influence the generated number:

Parameters

1. Decimal places (optional, default: 0)
    - Defines the number of decimal places in the generated number.
    - A value of 0 results in an integer output.
2. Minimum value (optional, default: -1,000,000)
    - The lower bound for the generated number.
3. Maximum value (optional, default: 1,000,000)
    - The upper bound for the generated number.
4. Exclude minimum (optional, default: false)
    - If true, the generated number will never be exactly the minimum value.
5. Exclude maximum (optional, default: false)
    - If true, the generated number will never be exactly the maximum value.
6. Multiple of (optional)
    - If provided, the generated number will be a multiple of this value.
    - If no valid multiple exists within the given range, the function may return Infinity.
7. Format pattern (optional)

- If no pattern is specified a plain string representation of the number will be returned.
- e.g. ",.000", " .000", " .#E0"

XML Example

```
<variables>
    <variable name="rndNum1" value="${citrus:randomNumberGenerator()}"/>
    <variable name="rndNum2" value="${citrus:randomNumberGenerator(2)}"/>
    <variable name="rndNum3" value="${citrus:randomNumberGenerator(3, -10, 10)}"/>
    <variable name="rndNum4" value="${citrus:randomNumberGenerator(0, 100, 200, true,
false)}"/>
    <variable name="rndNum5" value="${citrus:randomNumberGenerator(2, 0, 100, false,
true, 5)}"/>
    <variable name="rndNum5" value="${citrus:randomNumberGenerator(0, 10000000,
1000000000, false, false, null, #.###E0)}"/>
</variables>
```

Function Output Examples

```
157
23.78
-5.462
101
15.00
```

If no parameters are provided, the function generates a random integer between -1,000,000 and 1,000,000.

If decimal places are specified, the number will be formatted accordingly.

If multiple of is set, the generated number will always be a multiple of the given value within the range.

If minimum or maximum values are excluded, the function ensures that the output never equals these boundaries.

41.12. randomString()

This function will generate a random string representation with a defined length.

Parameters

1. Length of the generated string. (Required, no default value)
2. Case of the generated letters. (Optional, default: MIXED)
   - Possible values: UPPERCASE, LOWERCASE, MIXED.
3. Digits characters (0-9) included. (Optional, default: false)
   - If set to true, digits will be part of the output.

4. Minimum number of characters in the generated string. (Optional, default: length)
    - This value must be positive or zero and less than or equal to the total length.
    - If smaller than the length, the generated string will have a random length between the minimum and the specified length.

```
<variables>
    <variable name="rndString0" value="${citrus:randomString(10)}"/>
    <variable name="rndString1" value="citrus:randomString(10)"/>
    <variable name="rndString2" value="citrus:randomString(10, UPPERCASE)"/>
    <variable name="rndString3" value="citrus:randomString(10, LOWERCASE)"/>
    <variable name="rndString4" value="citrus:randomString(10, MIXED)"/>
    <variable name="rndString4" value="citrus:randomString(10, MIXED, true)"/>
    <variable name="rndString4" value="citrus:randomString(10, MIXED, true, 2)"/>
</variables>
```

Function output:

```
HrGHOdfAer
AgSSwedetG
JSDFUTTRKU
dtkhirtsuz
Vt567JkA32
Vt56
```

41.13. randomPattern()

This function generates a random string that matches the given regular expression pattern. It utilizes the Generex library to produce valid strings based on the provided regex.

The function takes one parameter, which is the regular expression pattern to generate the string from.

```
<variables>
    <variable name="rndPattern1" value="${citrus:randomPattern('[A-Z]{5}')}"/>
    <variable name="rndPattern2" value="${citrus:randomPattern('[0-9]{3}-[A-
Z]{2}')}"/>
    <variable name="rndPattern3" value="${citrus:randomPattern('[a-z]{4}[0-9]{2}')}"/>
    <variable name="rndPattern4" value="${citrus:randomPattern('[A-Za-z0-
9]{8,12}')}"/>
</variables>
```

Function output:

```
ABCDE
735-RT
bcda42
TgH56yJk12
```

The generated string strictly adheres to the provided regex pattern. The function supports character ranges, quantifiers, and grouping as per standard Java regex syntax. If an invalid regex is provided, an error will be thrown. An error might also be thrown for very complex regex, which cannot be handled by the Generex library.

41.14. randomEnumValue()

This function returns one of its supplied arguments. Furthermore, you can specify a custom function with a configured list of values (the enumeration). The function will randomly return an entry when called without arguments. This promotes code reuse and facilitates refactoring.

In the next sample the function is used to set a httpStatusCode variable to one of the given HTTP status codes (200, 401, 500)

```
<variable name="httpStatusCode" value="citrus:randomEnumValue('200', '401', '500')" />
```

As mentioned before you can define a custom function for your very specific needs in order to easily manage a list of predefined values like this:

Citrus Bean

```
@BindToRegistry
public FunctionLibrary myCustomFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("custom");
    library.setPrefix("custom:");
    library.getMembers().put("randomHttpStatusCode", randomHttpStatusCode());
    return library;
}

private RandomEnumValueFunction randomHttpStatusCode() {
    RandomEnumValueFunction function = new RandomEnumValueFunction();
    function.setValues(Arrays.asList("200", "500", "401"));
    return function;
}
```

Spring Bean

```
@Bean
public FunctionLibrary myCustomFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("custom");
    library.setPrefix("custom:");
    library.getMembers().put("randomHttpStatusCode", randomHttpStatusCode());
    return library;
}

private RandomEnumValueFunction randomHttpStatusCode() {
    RandomEnumValueFunction function = new RandomEnumValueFunction();
    function.setValues(Arrays.asList("200", "500", "401"));
    return function;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd">

    <citrus:function-library id="myCustomFunctionLibrary" prefix="custom:">
        <citrus:function name="randomHttpStatusCode"
ref="randomHttpStatusCodeFunction"/>
    </citrus:function-library>

    <bean id="randomHttpStatusCodeFunction"
class="org.citrusframework.functions.core.RandomEnumValueFunction">
      <property name="values">
        <list>
          <value>200</value>
          <value>500</value>
          <value>401</value>
        </list>
      </property>
    </bean>
</beans>
```

We have added a custom function library with a custom function definition. The custom function "randomHttpStatusCode" randomly chooses an HTTP status code each time it is called. Inside the test you can use the function like this:

```
<variable name="httpStatusCode" value="custom:randomHttpStatusCode()" />
```

41.15. currentDate()

This function will definitely help you when accessing the current date. Some examples will show the usage in detail:

```
<echo><message>citrus:currentDate()</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd'T'hh:mm:ss')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1y')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1M')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1d')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1h')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1m')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '+1s')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss', '-1y')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd HH:mm:ss','0h','UTC')</message></echo>
<echo><message>citrus:currentDate('yyyy-MM-dd
HH:mm:ss','0h','Europe/Berlin')</message></echo>
```

Note that the currentDate function provides three parameters: The first parameter describes the date format string. The second will define a date offset string containing year, month, days, hours, minutes or seconds that will be added or subtracted to or from the actual date value. The third parameter defines a timezone ID. If no timezone is defined, the system default timezone is used.

Function output:

```
01.09.2009
2009-09-01
2009-09-01 12:00:00
2009-09-01T12:00:00
2009-09-01 11:00:00
```

41.16. upperCase()

This function converts any string to upper case letters.

```
<echo>
    <message>citrus:upperCase('Hello Test Framework')</message>
</echo>
```

Function output:

41.17. lowerCase()

This function converts any string to lower case letters.

```
<echo>
    <message>citrus:lowerCase('Hello Test Framework')</message>
</echo>
```

Function output:

hello test framework

41.18. average()

The function will sum up all specified number values and divide the result through the number of values.

```
<variable name="avg" value="citrus:average('3', '4', '5')"/>
```

avg = **4.0**

41.19. minimum()

This function returns the minimum value in a set of number values.

```
<variable name="min" value="citrus:minimum('3', '4', '5')"/>
```

min = **3.0**

41.20. maximum()

This function returns the maximum value in a set of number values.

```
<variable name="max" value="citrus:maximum('3', '4', '5')"/>
```

max = **5.0**

41.21. sum()

The function will sum up all number values. The number values can also be negative.

```
<variable name="sum" value="citrus:sum('3', '4', '5')"/>
```

sum = **12.0**

41.22. absolute()

The function will return the absolute number value.

```
<variable name="abs" value="citrus:absolute('-3')"/>
```

abs = **3.0**

41.23. mapValue()

This function implementation maps string keys to string values. This is very helpful when the used key is randomly chosen at runtime and the corresponding value is not defined during the design time.

The following function library defines a custom function for mapping HTTP status codes to the corresponding messages:

Citrus Bean

```
@BindToRegistry
public FunctionLibrary myCustomFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("custom");
    library.setPrefix("custom:");
    library.getMembers().put("randomHttpStatusCode", randomHttpStatusCode());
    return library;
}

private RandomEnumValueFunction randomHttpStatusCode() {
    RandomEnumValueFunction function = new RandomEnumValueFunction();
    function.setValues(Arrays.asList("200", "500", "401"));
    return function;
}
```

Spring Bean

```
@Bean
public FunctionLibrary myCustomFunctionLibrary() {
    FunctionLibrary library = new FunctionLibrary();
    library.setName("custom");
    library.setPrefix("custom:");
    library.getMembers().put("getHttpStatusMessage", httpStatusMessageFunction());
    return library;
}

private MapValueFunction httpStatusMessageFunction() {
    MapValueFunction function = new MapValueFunction();
    function.getValues().put("200", "OK");
    function.getValues().put("401", "Unauthorized");
    function.getValues().put("500", "Internal Server Error");
    return function;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd">


    <citrus:function-library id="myCustomFunctionLibrary" prefix="custom:">
          <citrus-function name="getHttpStatusMessage"
ref="httpStatusMessageFunction"/>
    </citrus:function-library>

    <bean id="httpStatusMessageFunction"
class="org.citrusframework.functions.core.MapValueFunction">
      <property name="values">
        <map>
          <entry key="200" value="OK" />
          <entry key="401" value="Unauthorized" />
          <entry key="500" value="Internal Server Error" />
        </map>
      </property>
    </bean>
</beans>
```

In this example the function sets the variable httpStatusMessage to the 'Internal Server Error' string dynamically at runtime. The test only knows the HTTP status code and does not care about spelling

and message locales.

```
<variable name="httpStatusCodeMessage" value="custom:getHttpStatusMessage('500')" />
```

41.24. randomUUID()

The function will generate a random Java UUID.

```
<variable name="uuid" value="citrus:randomUUID()"/>
```

uuid = **98fbd7b0-832e-4b85-b9d2-e0113ee88356**

41.25. encodeBase64()

The function will encode a string to binary data using base64 hexadecimal encoding.

```
<variable name="encoded" value="citrus:encodeBase64('Hallo Testframework')"/>
```

encoded = **VGVzdCBGcmFtZXdvcms=**

It also has an optional charset parameter that is used for encoding the input string, with UTF-8 as its default value.

```
<variable name="encoded" value="citrus:encodeBase64('Hallo Testframework', 'UTF-8')"/>
```

41.26. decodeBase64()

The function will decode binary data to a character sequence using base64 hexadecimal decoding.

```
<variable name="decoded" value="citrus:decodeBase64('VGVzdCBGcmFtZXdvcms=')"/>
```

decoded = **Hallo Testframework**

It also has an optional charset parameter that is used for encoding the input string, with UTF-8 as its default value.

```
<variable name="decoded" value="citrus:decodeBase64('VGVzdCBGcmFtZXdvcms=', 'UTF-
8')"/>
```

41.27. escapeXml()

If you want to deal with escaped XML in your test case you may want to use this function. It automatically escapes all XML special characters.

```
<echo>
    <message>
        <![CDATA[
            citrus:escapeXml('<Message>Hallo Test Framework</Message>')
        ]]>
    </message>
</echo>
```

<Message>Hallo Test Framework</Message>

41.28. escapeJson()

If you want to deal with escaped JSON in your test case you may want to use this function. It automatically escapes all JSON special characters (especially double quotes).

```
<echo>
    <message>
        <![CDATA[
            {
                "kafkaClientConfiguration": {
                    "bootstrapServers": ["${kafkaBootstrapServer}"]
                },
                "messageBody": "citrus:escapeJson("{"exampleJson": "exampleValue"}")"
            }
        ]]>
    </message>
</echo>
```

41.29. cdataSection()

Usually we use CDATA sections to define message payload data inside a testcase. We might run into problems when the payload itself contains CDATA sections as nested CDATA sections are prohibited by XML nature. In this case the next function ships very usefull.

```
<variable name="cdata" value="citrus:cdataSection('payload')"/>
```

cdata = `<![CDATA[payload]]>`

41.30. digestAuthHeader()

Digest authentication is a commonly used security algorithm, especially in Http communication and SOAP WebServices. Citrus offers a function to generate a digest authentication principle used in the Http header section of a message.

```
<variable name="digest"
  value="citrus:digestAuthHeader('username', 'password', 'authRealm', 'acegi',
                            'POST', 'http://127.0.0.1:8080', 'citrus', 'md5')"/>
```

A possible digest authentication header value looks like this:

```
<Digest username=foo,realm=arealm,nonce=MTMzNT,
uri=http://127.0.0.1:8080,response=51f98c,opaque=b29a30,algorithm=md5>
```

You can use these digest headers in messages sent by Citrus like this:

```
<header>
  <element name="citrus_http_Authorization"
    value="vflig:digestAuthHeader('${username}','${password}','${authRealm}',

'${nonceKey}','POST','${uri}','${opaque}','${algorithm}')"/>
</header>
```

This will set a Http Authorization header with the respective digest in the request message. So your test is ready for client digest authentication.

41.31. localHostAddress()

Test cases may use the local host address for some reason (e.g. used as authentication principle). As the tests may run on different machines at the same time we can not use static host addresses. The provided function localHostAddress() reads the local host name dynamically at runtime.

```
<variable name="address" value="citrus:localHostAddress()"/>
```

A possible value is either the host name as used in DNS entry or an IP address value:

address = `<192.168.2.100>`

41.32. changeDate()

This function works with date values and manipulates those at runtime by adding or removing a date value offset. You can manipulate several date fields such as: year, month, day, hour, minute or second.

Let us clarify this with a simple example for this function:

```
<echo>
    <message>citrus:changeDate('01.01.2000', '+1y+1M+1d')</message>
</echo>
<echo>
    <message>citrus:changeDate(citrus:currentDate(), '-1M')</message>
</echo>
```

Function output:

```
02.02.2001
13.04.2013
```

As you can see the change date function works on static date values or dynamic variable values or functions like **citrus:currentDate()** . By default the change date function requires a date format such as the current date function ('dd.MM.yyyy'). You can also define a custom date format:

```
<echo>
    <message>citrus:changeDate('2000-01-10', '-1M-1d', 'yyyy-MM-dd')</message>
</echo>
```

Function output:

```
1999-12-09
```

With this you are able to manipulate all date values of static or dynamic nature at test runtime.

41.33. readFile()

The **readFile** function reads a file resource from given file path and loads the complete file content as function result. The file path can be a system file path as well as a classpath file resource. The file path can have test variables as part of the path or file name. In addition to that the file content can also have test variable values and other functions.

Let's see this function in action:

```
<echo>
    <message>citrus:readFile('classpath:some/path/to/file.txt')</message>
</echo>
<echo>
    <message>citrus:readFile(${filePath})</message>
</echo>
```

The function reads the file content and places the content at the position where the function has been called. This means that you can also use this function as part of Strings and message payloads for instance. This is a very powerful way to extract large message parts to separate file resources. Just add the **readFile** function somewhere to the message content and Citrus will load the extra file content and place it right into the message payload for you.

This function has a second and a third optional parameter that can be used for the following:

- 2nd parameter: a boolean value to indicate that the returned value should be base64 encoded. Defaults to false.

```
<message>citrus:readFile('classpath:some/path/to/file.txt', true)</message>
```

- 3rd parameter: a boolean value to indicate that a dynamic replacement (Citrus variables, functions, etc.) should be performed before the content is base64 encoded. Defaults to false.

```
<message>citrus:readFile('classpath:some/path/to/file.txt', true, true)</message>
```

41.34. message()

When messages are exchanged in Citrus the content is automatically saved to an in memory storage for further access in the test case. That means that functions and test actions can access the messages that have been sent or received within the test case. The **message** function loads a message content from that message store. The message is identified by its name. Receive and send actions usually define the message name. Now we can load the message payload with that name.

Let's see this function in action:

```
<echo>
    <message>citrus:message(myRequest.body())</message>
</echo>
```

The function above loads the message named **myRequest** from the local memory store. This requires a send or receive action to have handled the message before in the same test case.

Java

```
@CitrusTest
public void messageNameTest() {
    send("someEndpoint")
        .message()
        .name("myRequest")
        .body("Some payload");
}
```

XML

```
<test name="MessageNameTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <send endpoint="someEndpoint">
        <message name="myRequest">
          <body>
            <data>Some payload</data>
          </body>
        </message>
      </send>
    </actions>
</test>
```

YAML

```
name: MessageNameTest
actions:
  - send:
      endpoint: someEndpoint
      name: myRequest
      message:
        body:
          data: "Some payload"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="MessageNameTest">
    <actions>
        <send endpoint="someEndpoint">
          <message name="myRequest">
            <payload>Some payload</payload>
          </message>
        </send>
    </actions>
  </testcase>
</spring:beans>
```

The name of the message is important. Otherwise the message can not be found in the local message store. Note: a message can either be received or sent with a name in order to be stored in the local message store. The **message** function is then able to access the message by its name. In the first example the **body()** has been loaded. Of course we can also access header information.

Java

```
@CitrusTest
public void messageNameTest() {
    echo().message("citrus:message(myRequest.header('Operation'))");
}
```

XML

```
<test name="MessageNameTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <echo message="citrus:message(myRequest.header('Operation'))"/>
    </actions>
</test>
```

YAML

```
name: MessageNameTest
actions:
  - echo:
      message: "citrus:message(myRequest.header('Operation'))"
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="MessageNameTest">
    <actions>
        <echo>
            <message>citrus:message(myRequest.header('Operation'))</message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

The sample above loads the header **Operation** of the message.

In Java DSL the message store is also accessible over the TestContext.

41.35. xpath()

The **xpath** function evaluates a Xpath expressions on some XML source and returns the expression result as String.

Java

```
@CitrusTest
public void xpathTest() {
    echo()
        .message("""
        citrus:xpath('<message><id>1000</id></text>Some text
content</text></message>', '/message/id')
        """);
}
```

XML

```
<test name="XpathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <echo>
        <message>
        <![CDATA[
        citrus:xpath('<message><id>1000</id></text>Some text
content</text></message>', '/message/id')
        ]]>
        </message>
      </echo>
    </actions>
</test>
```

YAML

```
name: XpathTest
actions:
  - echo:
      message: |
        citrus:xpath('<message><id>1000</id></text>Some text
content</text></message>', '/message/id')
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="XpathTest">
    <actions>
        <echo>
            <message>
            <![CDATA[
            citrus:xpath('<message><id>1000</id></text>Some text
content</text></message>', '/message/id')
            ]]>
            </message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

The XML source is given as first function parameter and can be loaded in different ways. In the example above a static XML source has been used. We could load the XML content from external file or just use a test variable.

Java

```
@CitrusTest
public void xpathTest() {
    echo()
        .message("""
        citrus:xpath(citrus:readFile('some/path/to/file.xml'), '/message/id')
        """);
}
```

XML

```
<test name="XpathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <echo>
        <message>
        <![CDATA[
        citrus:xpath(citrus:readFile('some/path/to/file.xml'), '/message/id')
        ]]>
        </message>
      </echo>
    </actions>
</test>
```

YAML

```
name: XpathTest
actions:
  - echo:
      message: |
        citrus:xpath(citrus:readFile('some/path/to/file.xml'), '/message/id')
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="XpathTest">
    <actions>
        <echo>
            <message>
            <![CDATA[
            citrus:xpath(citrus:readFile('some/path/to/file.xml'), '/message/id')
            ]]>
            </message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

Also accessing the local message store is valid here:

Java

```
@CitrusTest
public void xpathTest() {
    echo()
        .message("""
        citrus:xpath(citrus:message(myRequest.body()), '/message/id')
        """);
}
```

XML

```
<test name="XpathTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
      <echo>
        <message>
        <![CDATA[
        citrus:xpath(citrus:message(myRequest.body()), '/message/id')
        ]]>
        </message>
      </echo>
    </actions>
</test>
```

YAML

```
name: XpathTest
actions:
  - echo:
      message: |
        citrus:xpath(citrus:message(myRequest.body()), '/message/id')
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
          xmlns:spring="http://www.springframework.org/schema/beans"
          xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
  <testcase name="XpathTest">
    <actions>
        <echo>
            <message>
            <![CDATA[
            citrus:xpath(citrus:message(myRequest.body()), '/message/id')
            ]]>
            </message>
        </echo>
    </actions>
  </testcase>
</spring:beans>
```

This combination is quite powerful as all previously exchanged messages in the test are automatically stored to the local message store. Reusing dynamic message values from other messages becomes very easy then.

41.36. jsonPath()

The **jsonPath** function evaluates a JsonPath expressions on some JSON source and returns the expression result as String.

```
<echo>
    <message><![CDATA[citrus:jsonPath('{ "message": { "id": 1000, "text": "Some text
content" } }', '$.message.id')]]></message>
</echo>
```

The JSON source is given as first function parameter and can be loaded in different ways. In the example above a static JSON source has been used. We could load the JSON content from external file or just use a test variable.

```
<echo>
    <message><![CDATA[citrus:jsonPath(${jsonSource}, '$.message.id')]]></message>
</echo>
```

The JSON source may also be specified in multiple parameters, in which case the arguments except the last one are concatenated with commas, and will be treated as the JSON source. The last parameter is always treated as the JSON path expression.

```
<echo>
    <message><![CDATA[citrus:jsonPath('{ "message": { "id": 1000', '"text": "Some text
content" } }', '$.message.id')]]></message>
</echo>
```

In the example above, the parts `{ "message": { "id": 1000` and `"text": "Some text content" } }` will form the JSON source as `{ "message": { "id": 1000, "text": "Some text content" } }`.

Also accessing the local message store is valid here:

```
<echo>
    <message><![CDATA[citrus:jsonPath(citrus:message(myRequest.body()),
'$.message.id')]]></message>
</echo>
```

This combination is quite powerful as all previously exchanged messages in the test are automatically stored to the local message store. Reusing dynamic message values from other messages becomes very easy then.

41.37. jsonPatch()

The **jsonPatch** function applies RFC 6902 JSON Patch operations to a JSON source and returns the modified JSON as String. It supports JSONPath notation for paths, which is automatically converted

to JSON Pointer format.

```
<echo>
    <message><![CDATA[citrus:jsonPatch('{ "message": { "id": 1000, "status": "DRAFT" }
}', 'replace', '$.message.status', 'PUBLISHED')]]></message>
</echo>
```

The first parameter is the JSON source, followed by patch operations in triplets: operation, path, and value. Valid operations are `add`, `remove`, `replace`, `move`, and `copy`.

The JSON source can be loaded from a test variable:

```
<echo>
    <message><![CDATA[citrus:jsonPatch(${jsonSource}, 'replace', '$.status',
'APPROVED')]]></message>
</echo>
```

Multiple patch operations can be chained by adding more triplets:

```
<echo>
    <message><![CDATA[citrus:jsonPatch(${jsonSource}, 'replace', '$.status',
'APPROVED', 'add', '$.metadata.timestamp', '2024-01-15', 'remove', '$.tempField',
'')]]></message>
</echo>
```

For the `remove` operation, the value parameter is ignored but must still be provided (use an empty string). For `move` and `copy` operations, the value parameter specifies the source path:

```
<echo>
    <message><![CDATA[citrus:jsonPatch(${jsonSource}, 'copy', '$.backup.name',
'$.name')]]></message>
</echo>
```

Paths can be specified in JSONPath notation (e.g., `$.items[0].name`) or JSON Pointer format (e.g., `/items/0/name`). To append to an array, use the `-` index:

```
<echo>
    <message><![CDATA[citrus:jsonPatch('{ "items": [1, 2, 3] }', 'add', '$.items/-',
'4')]]></message>
</echo>
```

Values can be plain strings or valid JSON structures:

```
<echo>
    <message><![CDATA[citrus:jsonPatch(${jsonSource}, 'add', '$.metadata',
'{"created": "2024-01-15", "author": "test"}')]]></message>
</echo>
```

Also accessing the local message store is valid here:

```
<echo>
    <message><![CDATA[citrus:jsonPatch(citrus:message(myRequest.body()), 'replace',
'$.message.status', 'PROCESSED')]]></message>
</echo>
```

This combination is quite powerful as all previously exchanged messages in the test are automatically stored to the local message store. Reusing dynamic message values from other messages becomes very easy, enabling dynamic modification of message content for subsequent test steps.

41.38. urlEncode()/urlDecode()

The **urlEncode** function takes a String and performs proper URL encoding. The result is a URL encoded String that is using proper character escaping for Http.

```
<echo>
    <message><![CDATA[citrus:urlEncode('foo@citrusframework', 'UTF-8')]]></message>
</echo>
```

The above function takes the String `foo@citrusframework.org` and performs proper URL encoding resulting in `foo%40citrusframework`.

Same logic applies to the `urlDecode()` function that will read an encoded String replacing all escaped characters to the normal String representation.

```
<echo>
    <message><![CDATA[citrus:urlDecode('foo%40citrusframework', 'UTF-8')]]></message>
</echo>
```

The `UTF-8` charset is used during URL encoding operation and is optional as the default is `UTF-8`.

41.39. systemProperty()

The **systemProperty** function resolves a System property expression at test runtime. The resulting value is returned as function result. In case the System property is not available in the JVM an optional default value is used. In case no default value is given the function will fail with errors.

```
<echo>
    <message><![CDATA[citrus:systemProperty('user.name', 'my-default')]]></message>
</echo>
```

41.40. env()

The **env** function can be used to access an environment specific property at test runtime. The environment property can be a variable set on the underlying operating system. Also the `env()` function is able to access the Spring environment settings (see `org.springframework.core.env.Environment`).

As the Spring environment is also able to resolve System properties you can use this function in this manner, too.

```
<echo>
    <message><![CDATA[citrus:env('USER_NAME', 'my-default')]]></message>
</echo>
```

The default value is optional and provides an error fallback in case the environment setting is not available. In case no default value is provided the function will fail with errors.

41.41. unixTimestamp()

unixTimestamp is a parameterless function that simply returns the current epoch timestamp as seconds.

```
<echo>
    <message><![CDATA[citrus:unixTimestamp()]]></message>
</echo>
```

Chapter 42. Validation Matchers

Message validation in Citrus is essential. The framework offers several validation mechanisms for different message types and formats. With test variables we are able to check for simple value equality. We ensure that message entries are equal to predefined expected values. Validation matcher add powerful assertion functionality on top of that. You just can use the predefined validation matcher functionalities in order to perform more complex assertions like **contains** or **isNumber** in your validation statements.

The following sections describe the Citrus default validation matcher implementations that are ready for usage. The matcher implementations should cover the basic assertions on character sequences and numbers. Of course you can add custom validation matcher implementations in order to meet your very specific validation assertions, too.

First of all let us have a look at a validation matcher statement in action so we understand how to use them in a test case.

```
<message>
    <payload>
        <RequestMessage>
            <MessageBody>
                <Customer>
                    <Id>@greaterThan(0)@</Id>
                    <Name>@equalsIgnoreCase('foo')@</Name>
                </Customer>
            </MessageBody>
        </RequestMessage>
    </payload>
</message>
```

The listing above describes a normal message validation block inside a receive test action. We use some inline message payload template as CDATA. As you know Citrus will compare the actual message payload to this expected template in DOM tree comparison. In addition to that you can simply include validation matcher statements. The message element **Id** is automatically validated to be a number greater than zero and the **Name** character sequence is supposed to match 'foo' ignoring case spelling considerations.

Please note the special validation matcher syntax. The statements are surrounded with '@' markers and are identified by some unique name. The optional parameters passed to the matcher implementation state the expected values to match.



You can use validation matcher with all validation mechanisms - not only with XML validation. Plaintext, JSON, SQL result set validation are also supported.

A set of validation matcher implementations is usually combined to a validation matcher library. The library has a prefix that will identify the validation matcher inside the test case. The default test framework validation matcher library uses a default prefix (citrus). You can write your own validation matcher library using your own prefix in order to extend the test framework

functionality whenever you want.

The library is built in the Spring configuration and contains a set of validation matchers that are of public use.

Citrus Bean

```
@BindToRegistry
public ValidationMatcherLibrary testMatcherLibrary() {
    ValidationMatcherLibrary library = new ValidationMatcherLibrary();
    library.setName("test");
    library.setPrefix("foo:");
    library.getMembers().put("isNumber", new IsNumberValidationMatcher());
    library.getMembers().put("contains", new ContainsValidationMatcher());
    library.getMembers().put("customMatcher", customMatcherBean());
    return library;
}
```

Spring Bean

```
@Bean
public ValidationMatcherLibrary testMatcherLibrary() {
    ValidationMatcherLibrary library = new ValidationMatcherLibrary();
    library.setName("test");
    library.setPrefix("foo:");
    library.getMembers().put("isNumber", new IsNumberValidationMatcher());
    library.getMembers().put("contains", new ContainsValidationMatcher());
    library.getMembers().put("customMatcher", customMatcherBean());
    return library;
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.citrusframework.org/schema/config
           http://www.citrusframework.org/schema/config/citrus-config.xsd">

    <citrus:validation-matcher-library id="testMatcherLibrary" prefix="foo:">
          <citrus:matcher name="isNumber"
class="org.citrusframework.base.validation.matcher.core.IsNumberValidationMatcher"/>
          <citrus:matcher name="contains"
class="org.citrusframework.base.validation.matcher.core.ContainsValidationMatcher"/>
          <citrus:matcher name="customMatcher" ref="customMatcherBean"/>
          ...
    </citrus:validation-matcher-library>
</beans>
```

As you can see the library defines one to many validation matcher members either referenced as normal Spring bean or by its implementing Java class name. Citrus constructs the library, and you are able to use the validation matcher in your test case with the leading library prefix just like this:

```
@foo:isNumber()@
@foo:contains()@
@foo:customMatcher()@
```



You can add custom validation matcher implementations and custom validation matcher libraries. Just use a custom prefix for your library. The default Citrus validation matcher library uses no prefix. See now the following sections describing the default validation matcher in Citrus.

42.1. ignore()

The ignore validation matcher is a special matcher that ignores the value and is always positive in its outcome. You should use the ignore validation matcher when only validating the pure existence of an element. The value is ignored but the element has to be present in the message payload.

```
<message>
    <payload>
        <RequestMessage>
            <MessageBody>
                <Customer>
                    <Id>@ignore()@</Id>
                    <Name>@equalsIgnoreCase('foo')@</Name>
                </Customer>
            </MessageBody>
        </RequestMessage>
    </payload>
</message>
```



The ignore validation matcher is the only validation matcher that is able to skip the function parameter body. So you can use both `@ignore()@` and `@ignore@`.

42.2. matchesXml()

The XML validation matcher implementation is the possibly most exciting one, as we can validate nested XML with full validation power (e.g. ignoring elements, variable support). The matcher checks a nested XML fragment to compare against expected XML. For instance we receive the following XML message payload for validation:

```
<GetCustomerMessage>
      <CustomerDetails>
          <Id>5</Id>
          <Name>Christoph</Name>
          <Configuration>
          <![CDATA[
            <config>
                <premium>true</premium>
                <last-login>2012-02-24T23:34:23</last-login>
                <link>http://citrusframework.org/customer/5</link>
            </config>
          ]]>
          </Configuration>
      </CustomerDetails>
</GetCustomerMessage>
```

As you can see the message payload contains some configuration as nested XML data in a CDATA section. We could validate this CDATA section as static character sequence comparison, true. But the <last-login> timestamp changes its value continuously. This breaks the static validation for CDATA elements in XML. Fortunately the new XML validation matcher provides a solution for us:

```
<message>
    <payload>
        <GetCustomerMessage>
            <CustomerDetails>
                <Id>5</Id>
                <Name>Christoph</Name>
                <Configuration>
                  citrus:cdataSection(@matchesXml('<config>
                    <premium>${isPremium}</premium>
                    <last-login>@ignore@</last-login>
                    <link>http://citrusframework.org/customer/5</link>
                  </config>')@)
                </Configuration>
            </CustomerDetails>
        </GetCustomerMessage>
    </payload>
</message>
```

With the validation matcher you are able to validate the nested XML with full validation power. Ignoring elements is possible and we can also use variables in our control XML.



Nested CDATA elements within other CDATA sections are not allowed by XML standard. This is why we create the nested CDATA section on the fly with the function cdataSection().

42.3. equalsIgnoreCase()

This matcher implementation checks for equality without any case spelling considerations. The matcher expects a single parameter as the expected character sequence to check for.

```
<value>@equalsIgnoreCase('foo')@</value>
```

42.4. contains()

This matcher searches for a character sequence inside the actual value. If the character sequence is not found somewhere the matcher starts complaining.

```
<value>@contains('foo')@</value>
```

The validation matcher also exists in a case-insensitive variant.

```
<value>@containsIgnoreCase('foo')@</value>
```

42.5. startsWith()

The matcher implementation asserts that the given value starts with a character sequence otherwise the matcher will arise some error.

```
<value>@startsWith('foo')@</value>
```

42.6. endsWith()

Ends with matcher validates a value to end with a given character sequence.

```
<value>@endsWith('foo')@</value>
```

42.7. matches()

You can check a value to meet a regular expression with this validation matcher. This is for instance very useful for email address validation.

```
<value>@matches('[a-z0-9]')@</value>
```

42.8. matchesDatePattern()

Date values are always difficult to check for equality. Especially when you have millisecond timestamps to deal with. Therefore the date pattern validation matcher should have some improvement for you. You simply validate the date format pattern instead of checking for total equality.

```
<value>@matchesDatePattern('yyyy-MM-dd')@</value>
```

The example listing uses a date format pattern that is expected. The actual date value is parsed according to this pattern and may cause errors in case the value is not a valid date matching the desired format.

42.9. isNumber()

Checking on values to be of numeric nature is essential. The actual value must be a numeric number otherwise the matcher raises errors. The matcher implementation does not evaluate any parameters.

```
<value>@isNumber()@</value>
```

42.10. lowerThan()

This matcher checks a number to be lower than a given threshold value.

```
<value>@lowerThan(5)@</value>
```

42.11. greaterThan()

The matcher implementation will check on numeric values to be greater than a minimum value.

```
<value>@greaterThan(5)@</value>
```

42.12. greaterThanOrEqualTo()

The matcher implementation will check on numeric values to be greater than or equal to a minimum value.

```
<value>@greaterThanOrEqualTo(5)@</value>
```

42.13. lowerThanOrEqualTo()

This matcher checks a number to be lower than or equal to a given threshold value.

```
<value>@lowerThanOrEqualTo(5)@</value>
```

42.14. isWeekday()

The matcher works on date values and checks that a given date evaluates to the expected day of the week. The user defines the expected day by its name in uppercase characters. The matcher fails in case the given date is another week day than expected.

```
<someDate>@isWeekday('MONDAY')@</someDate>
```

Possible values for the expected day of the week are: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY or SUNDAY.

The field value has to be a date value otherwise the matcher will fail to parse the date. The matcher requires a date format which is **dd.MM.yyyy** by default. You can change this date format as follows:

```
<someDate>@isWeekday(MONDAY('yyyy-MM-dd'))@</someDate>
```

Now the matcher uses the custom date format in order to parse the date value for evaluation. The validation matcher also works with date time values. In this case you have to give a valid date time format respectively (e.g. FRIDAY('yyyy-MM-dd'T'hh:mm:ss')).

42.15. variable()

This is a very special validation matcher. Instead of performing a validation logic you can save the actual value passed to the validation matcher as new test variable. This comes very handy as you can use the matcher wherever you want: JSON message payloads, XML message payloads, headers and so on.

```
<value>@variable('foo')@</value>
```

The validation matcher creates a new variable **foo** with the actual element value as variable value. When leaving out the control value the field name itself is used as variable name.

```
<date>@variable()@</date>
```

This creates a new variable **date** with the actual element value as variable value.

42.16. dateRange()

The matcher works on date values and checks that a given date is within the expected date range. The user defines the expected date range by specifying a from-date, a to-date and optionally a date format. The matcher fails when the given date lies outside the expected date range.

```
<someDate>@dateRange('01-12-2015', '31-12-2015', 'dd-MM-yyyy')@</someDate>
```

Possible valid values would be 'some date' >= '01-12-2015' and 'some date' <= '31-12-2015'

The date-format is optional and when omitted it is assumed that all dates match the default date format **yyyy-MM-dd** . When specifying a custom date format use Java's date format as a reference for valid date formats. Only dates were used in the example above but we could just as easily used date and time as shown in the example below

```
<someDate>@dateRange('2015.12.01 07:00:00', '2015.12.01 19:00:00', 'yyyy.MM.dd
HH:mm:ss')@</someDate>
```

42.17. assertThat()

Hamcrest is a very powerful matcher library with extraordinary matcher implementations. You can use Hamcrest matchers also as Citrus validation matchers.

```
<someValue>@assertThat(equalTo(foo))@</someValue>
```

In the listing above we are using the **equalTo()** matcher. All Hamcrest matchers are surrounded by an **assertThat** expression. You are able to combine several Hamcrest matchers then in order to construct very powerful validation logic. See the following examples on what is possible then:

```
<someValue>@assertThat(equalTo(value))@</someValue>
<someValue>@assertThat(not(equalTo(other))@</someValue>
<someValue>@assertThat(is(not(other)))@</someValue>
<someValue>@assertThat(not(is(other)))@</someValue>
<someValue>@assertThat(equalToIgnoringCase(VALUE))@</someValue>
<someValue>@assertThat(containsString(lue))@</someValue>
<someValue>@assertThat(not(containsString(other)))@</someValue>
<someValue>@assertThat(startsWith(val))@</someValue>
<someValue>@assertThat(endsWith(lue))@</someValue>
<someValue>@assertThat(anyOf(startsWith(val), endsWith(lue)))@</someValue>
<someValue>@assertThat(allOf(startsWith(val), endsWith(lue)))@</someValue>
<someValue>@assertThat(isEmptyString())@</someValue>
<someValue>@assertThat(not(isEmptyString()))@</someValue>
<someValue>@assertThat(isEmptyOrNullString())@</someValue>
<someValue>@assertThat(nullValue())@</someValue>
<someValue>@assertThat(notNullValue())@</someValue>
<someValue>@assertThat(empty())@</someValue>
<someValue>@assertThat(not(empty())@</someValue>
<someValue>@assertThat(greaterThan(4))@</someValue>
<someValue>@assertThat(allOf(greaterThan(4), lessThan(6),
not(lessThan(5)))@</someValue>
<someValue>@assertThat(is(not(greaterThan(5))))@</someValue>
<someValue>@assertThat(greaterThanOrEqualTo(5))@</someValue>
<someValue>@assertThat(lessThan(5))@</someValue>
<someValue>@assertThat(not(lessThan(1)))@</someValue>
<someValue>@assertThat(lessThanOrEqualTo(4))@</someValue>
<someValue>@assertThat(hasSize(5))@</someValue>
<someValue>@assertThat(closeTo(9.0))@</someValue>
<someValue>@assertThat(closeTo(9.0, 0.5))@</someValue>
<someValue>@assertThat(isIn(foo, bar))@</someValue>
<someValue>@assertThat(isOneOf(foo, bar))@</someValue>
```

Citrus will automatically perform validation matchers on the element value. Only if all matchers are satisfied the validation will pass.

42.18. ignoreNewLine()

This matcher implementation checks for equality with prior normalization of all new line characters. This includes new line types CR, LF and CRLF as well as multiple new lines in value and control value. So when using this validation matcher all new line characters are removed prior to checking for equality.

Let's assume that we have a value with new lines that we want to validate using the matcher implementation:

```
<value>This
is
a
value with lots of
new lines</value>
```

You can now skip all new line characters in your control value using the `ignoreNewLine` matcher.

```
<value>@ignoreNewLine('This is a value with lots of new lines')@</value>
```

As you can see the new line characters are not breaking the validation. The other whitespace characters remain untouched though.

42.19. trim()

This trim matcher will remove leading and trailing whitespaces before checking for equality.

Let's assume that we have a value with leading and trailing whitespaces:

```
<value>
This is a value   </value>
```

You can now skip all leading and trailing whitespaces in your control value.

```
<value>@trim('This is a value')@</value>
```

42.20. trimAllWhitespaces()

Sometimes it is necessary to check equality of some value without caring for whitespaces at all. The matcher implementation will remove all whitespaces before checking for equality.

```
<value>   some value   </value>
```

You can now skip all whitespaces in your control value.

```
<value>@trimAllWhitespaces('somevalue')@</value>
```

42.21. isUUIDv4()

This validation matcher checks if a valid UUID version 4 is present. Given the `<variable name="validUuid" value="653ce6fd-dca4-4672-bbc0-16e2b74b8b81"/>`, usage is as following.

Example `isUUIDv4()` validation

```
<actions>
    <send endpoint="helloEndpoint">
        <message>
            <data>Hello Citrus!</data>
        </message>
        <header>
            <element name="message-id" value="${validUuid}"/>
        </header>
    </send>

    <receive endpoint="helloEndpoint">
        <message type="plaintext">
            <data>Hello Citrus!</data>
        </message>
        <header>
            <element name="message-id" value="@isUUIDv4()@"/>
        </header>
    </receive>
</actions>
```

42.22. allOf()

The `allOf` matcher is a conjunction combinator — it takes two or more sub-matcher expressions and passes only when **all** of them pass. If any sub-matcher fails, the validation reports every individual failure in a single error message.

This is useful when a field value must simultaneously satisfy several conditions.

Syntax

```
@allOf(matcher1('param'), matcher2('param'), ...)@
```

XML DSL example

```
<receive endpoint="helloEndpoint">
    <message>
        <data>
            <![CDATA[
            {
                "version": "@allOf(startsWith('v'), contains('.'), matches('v[0-
9]+\\.[0-9]+'))@"
            }
            ]]>
        </data>
    </message>
</receive>
```

YAML DSL example

```
- receive:
    endpoint: "helloEndpoint"
    message:
      type: json
      body:
        data: "{\"version\": \"@allOf(startsWith('v'), contains('.'))@\"}"
```

If any sub-matcher fails, the error lists all failing sub-matchers:

```
allOf() failed for field 'version' with value 'hello': ['startsWith('v')' -> ...,
'contains('.')' -> ...]
```



`allOf()` requires at least two sub-expressions. Providing zero or one sub-expression raises a `CitrusRuntimeException`.

42.23. anyOf()

The `anyOf` matcher is a disjunction combinator — it takes two or more sub-matcher expressions and passes when **at least one** of them passes. Evaluation short-circuits as soon as one sub-matcher passes. If all sub-matchers fail, all individual failures are reported.

This is useful when a field value may take one of several valid forms.

Syntax

```
@anyOf(matcher1('param'), matcher2('param'), ...)@
```

XML DSL example

```
<receive endpoint="helloEndpoint">
    <message>
        <data>
            <![CDATA[
            {
                "status": "@anyOf(equalsIgnoreCase('ok'),
equalsIgnoreCase('success'))@"
            }
            ]]>
        </data>
    </message>
</receive>
```

YAML DSL example

```
- receive:
    endpoint: "helloEndpoint"
    message:
      type: json
      body:
        data: "{\"status\": \"@anyOf(equalsIgnoreCase('ok'),
equalsIgnoreCase('success'))@\"}"
```

If all sub-matchers fail, the error lists each failure:

```
anyOf() failed for field 'status' with value 'error': all matchers failed:
['equalsIgnoreCase('ok')' -> ..., 'equalsIgnoreCase('success')' -> ...]
```



`anyOf()` requires at least two sub-expressions. Providing zero or one sub-expression raises a `CitrusRuntimeException`.

42.24. not()

The `not` matcher is a negation combinator — it takes exactly one sub-matcher expression and inverts its result: it passes when the inner matcher **fails**, and fails when the inner matcher **passes**.

Syntax

```
@not(matcher('param'))@
```

XML DSL example

```
<receive endpoint="helloEndpoint">
    <message>
        <data>
            <![CDATA[
            {
                "message": "@not(contains('error'))@"
            }
            ]]>
        </data>
    </message>
</receive>
```

YAML DSL example

```
- receive:
    endpoint: "helloEndpoint"
    message:
      type: json
      body:
        data: "{\"message\": \"@not(contains('error'))@\"}"
```

If the inner matcher passes (meaning `not()` fails), the error is:

```
not() failed for field 'message': 'contains('error')' matched value 'fatal error' but
was expected not to
```



`not()` requires exactly one sub-expression. Providing zero or more than one sub-expression raises a `CitrusRuntimeException`.

42.25. Nesting combinators

All three combinators — `allOf`, `anyOf`, and `not` — can be nested inside each other to arbitrary depth. Any sub-expression that is itself a combinator is evaluated recursively.

Example: allOf with nested anyOf and not

```
<receive endpoint="helloEndpoint">
    <message>
        <data>
            <![CDATA[
            {
                "result": "@allOf(anyOf(startsWith('foo'), startsWith('bar')),
not(contains('error')))@"
            }
            ]]>
        </data>
    </message>
</receive>
```

This validates that `result` starts with either `foo` or `bar`, AND does not contain `error`.

Equivalent YAML

```
- receive:
    endpoint: "helloEndpoint"
    message:
      type: json
      body:
        data: "{\"result\": \"@allOf(anyOf(startsWith('foo'), startsWith('bar')),
not(contains('error')))@\"}"
```



For users who need richer matching logic (numeric comparisons, collection matchers, etc.), Citrus also provides Hamcrest integration via `@assertThat(allOf(…))@`. The built-in combinators described here are the simpler, zero-dependency alternative for string-based validations.

Chapter 43. Citrus agent

The Citrus agent provides a way to run tests as part of the Docker container environment and as part of the Kubernetes platform.

The agent ships as a web application and users are able to run Citrus tests on demand. Usually the agent application runs as a Docker container or Kubernetes deployment so that the agent is able to access internal platform services such as Knative eventing brokers or a service resources exposed only to the current Kubernetes namespace.

The agent application exposes a REST service API for clients to inject tests and trigger test runs.

43.1. Agent REST service API

As a web application the agent exposes several REST endpoints. Http clients are able to trigger test runs and inspect the remote test results.

At service startup the agent will automatically run all tests that can be found according to certain criteria. The agent performs a test scan looking for available tests on multiple sources applying multiple search criteria:

- name pattern: Tests that should be executed must match one of the given class name patterns (e.g. `.*IT`)
- packages: List of packages to scan in the agent classpath. All tests matching the given name patterns will be executed.
- source files: List of source file paths to test sources (e.g. XML, YAML, Cucumber feature). All sources found will be run.
- test-jar: A provided Java archive holding test classes for execution. All previous test filters (e.g. name patterns, package filters) get applied while scanning classes in this JAR.



A test-jar that holds all tests and its dependencies can be injected into the agent application classpath. When using a Docker container image this injection may be accomplished through Docker volumes. In Kubernetes the test-jar injection may be done via ConfigMaps and volume mounts on the citrus-agent deployment.

The citrus-agent may also run tests on demand rather than on startup. The agent provides a REST service endpoint to trigger a test run. Each call to the service API may use different settings and test run criteria (e.g. name patterns, packages, source files).

The agent REST API provides these operations:

Table 10. REST service operations

Operation	Method	Description
/health	GET	Provides general agent service status as a Json object (UP: true/false)

Operation	Method	Description
/run	GET	Triggers a new test run. The request query parameters may define the test run settings (e.g. name pattern, packages, source files). The operation blocks until the test run is finished and the response represents the test result summary using proper Http response codes for failure (500 INTERNAL_SERVER_ERROR) and success (200 OK).
/run	POST	Triggers a new test run. The request body may be a test configuration object that specifies the test run (e.g. name pattern, packages, source files). The operation blocks until the test run is finished and the response represents the test result summary using proper Http response codes for failure (500 INTERNAL_SERVER_ERROR) and success (200 OK).
/run	PUT	Same as GET/POST run operation but the call is not blocking until the test run is finished. Instead of this clients may inspect the test results calling/polling the `/results` service endpoint.
/results	GET	Get cumulated test results for this agent. This represents the overall test results for all tests that have been run on this agent.

Operation	Method	Description
/results/latest	GET	Get the most recent test results for the latest test run. This also includes results from a pending test run which may deliver partial test results (Http response code partial 206). This way clients can check on when the current test run is finished or still pending.
/results/clear	PUT	Clear the cumulated test results to start fresh.
/results/files	GET	Get test results files that have been created as part of the test run. This includes JUnit test reports and HTML Citrus test suite reports.

43.2. Configuration

The agent reads general settings on startup from system properties and environment variables. The environment settings are well suited for both usual Java runtime environment and containerized runtime environments such as Docker containers or Kubernetes deployments.

The Citrus agent application supports the following settings.

Table 11. System properties

System properties	Description
citrus.agent.name	The name of this agent instance. Default value is `citrus-agent`.
citrus.agent.server.port	The port to listen on for incoming client requests. Default port is `4567`.
citrus.agent.test.engine	The default engine used to run tests on this agent. One of junit, junit-jupiter, junit4, testng, cucumber. Default is `junit-jupiter`.
citrus.agent.time.to.live	The time to wait after the tests have run as part of the startup procedure. Default is `-1` which means indefinite.
citrus.agent.system.exit	Should use system exit to mark test failure (e.g. system exit = -1). Default is `false`.

System properties	Description
citrus.agent.skip.tests	Test execution should bee skipped at server startup. Default is `false` which means that all tests found are automatically run as part of the server startup procedure.
citrus.agent.includes	List of test name patterns to include in test runs. The server searches for tests matching these patterns when running all available tests. Default patterns are `["^.IT$", "^.*ITCase$", "^IT.$"]`
citrus.agent.packages	List of packages to include in test scan.
citrus.agent.default.properties	Comma separated list of key-value pairs that represent default system properties for a test run.
citrus.agent.test.sources	List of test source paths to run. This may be paths to XML, YAML, Cucumber feature files that the agent should run as part of a test run.
citrus.agent.config.class	Bean configuration class to load as part of a test run.
citrus.agent.test.jar	Path to a test Java archive that hold tests for a test run. Default value is `classpath:citrus-agent-tests.jar`. The test-jar execution is skipped when the file does not exist.

Same properties are settable via environment variables.

Table 12. Environment variables

Environment variable	Description
CITRUS_AGENT_NAME	The name of this agent instance. Default value is `citrus-agent`.
CITRUS_AGENT_SERVER_PORT	The port to listen on for incoming client requests. Default port is `4567`.
CITRUS_AGENT_TEST_ENGINE	The default engine used to run tests on this agent. One of junit, junit-jupiter, junit4, testng, cucumber. Default is `junit-jupiter`.
CITRUS_AGENT_TIME_TO_LIVE	The time to wait after the tests have run as part of the startup procedure. Default is `-1` which means indefinite.
CITRUS_AGENT_SYSTEM_EXIT	Should use system exit to mark test failure (e.g. system exit = -1). Default is `false`.

Environment variable	Description
CITRUS_AGENT_SKIP_TESTS	Test execution should bee skipped at server startup. Default is `false` which means that all tests found are automatically run as part of the server startup procedure.
CITRUS_AGENT_INCLUDES	List of test name patterns to include in test runs. The server searches for tests matching these patterns when running all available tests. Default patterns are `["^.IT$", "^.*ITCase$", "^IT.$"]`
CITRUS_AGENT_PACKAGES	List of packages to include in test scan.
CITRUS_AGENT_DEFAULT_PROPERTIES	Comma separated list of key-value pairs that represent default system properties for a test run.
CITRUS_AGENT_TEST_SOURCES	List of test source paths to run. This may be paths to XML, YAML, Cucumber feature files that the agent should run as part of a test run.
CITRUS_AGENT_CONFIG_CLASS	Bean configuration class to load as part of a test run.
CITRUS_AGENT_TEST_JAR	Path to a test Java archive that hold tests for a test run. Default value is `classpath:citrus-agent-tests.jar`. The test-jar execution is skipped when the file does not exist.

43.3. Web UI

The Citrus agent application provides a Web user interface for clients to inspect previous test runs. You can access the user interface pointing your browser to the exposed server port. By default, when running the agent service locally (e.g. via JBang) the exposed Web UI endpoint is http://localhost:4567.

When run on Docker or Kubernetes the exposed port may be mapped to a local port or Service resource that you can access with the browser.

43.4. Docker container image

The Citrus agent also ships as a Docker container image that you can use to run the agent as a Docker container or as part of a Kubernetes deployment. The Docker image is available on this Quay.io repository.

You can use the image in a Kubernetes deployment:

Citrus agent deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/managed-by: citrus
    app.kubernetes.io/name: citrus-agent
  name: citrus-agent
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: citrus-agent
  template:
    metadata:
      labels:
        app.kubernetes.io/managed-by: citrus
        app.kubernetes.io/name: citrus-agent
    spec:
      containers:
      - name: citrus-agent
        env:
        - name: CITRUS_AGENT_SERVER_PORT
          value: "8080"
        image: quay.io/citrusframework/citrus-agent:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

The Deployment may use the environment variables to customize the Citrus agent application. The container exposes the Http port so you can access the Web UI with a Service resource.



You can inject a test jar that contains the tests to run with the agent. Just use a ConfigMap with the archive as binary content and mount the JAR into the Citrus agent pod with volumes.

Test jar ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: citrus-agent-resources
  labels:
    app.kubernetes.io/managed-by: citrus
    app.kubernetes.io/name: citrus-agent
binaryData:
  citrus-agent-tests.jar: |
    [...]
```

Citrus agent with volumes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/managed-by: citrus
    app.kubernetes.io/name: citrus-agent
  name: citrus-agent
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: citrus-agent
  template:
    metadata:
      labels:
        app.kubernetes.io/managed-by: citrus
        app.kubernetes.io/name: citrus-agent
    spec:
      containers:
      - name: citrus-agent
        env:
        - name: CITRUS_AGENT_TEST_JAR
          value: /deployments/resources/citrus-agent-tests.jar
        - name: CITRUS_AGENT_SERVER_PORT
          value: "8080"
        image: quay.io/citrusframework/citrus-agent:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 8080
          name: http
          protocol: TCP
        volumeMounts:
        - mountPath: /deployments/resources
          name: resources
      volumes:
      - name: resources
        configMap:
          defaultMode: 420
          name: citrus-agent-resources
```

The volume mount adds the binary content from the ConfigMap to the Pod and the Citrus agent application will be able to run the tests from that Java archive.

You may also use a ConfigMap that holds other test sources such as XML, YAML, Groovy or Cucumber test files.

43.5. Agent JBang commands

The JBang integration is a perfect way to run the agent locally suited very well for rapid prototyping.

The Citrus JBang command line tooling supports several agent related commands:

Table 13. JBang agent commands

command	Description
agent start	Start the Citrus agent application as a local JBang process.
agent stop	Stop the current agent process.

43.6. Maven plugin

The Citrus agent provides a Maven plugin that you can include into your project build lifecycle. You can use the plugin to build a `test-jar` from your local test sources. The plugin will then start a new Citrus agent service (e.g. in Kubernetes) and run the tests from the `test-jar` artifact. The plugin waits for the remote test execution to finish and provides the test results to the local Maven build.

The Citrus agent Maven plugin provides these goals to execute:

Table 14. Goals

Goal	Build phase	Description
test-jar	package	Builds a test JAR artifact from your test scoped sources
start	pre-integration-test	Create and run a new Citrus agent application instance (locally via JBang, as a Docker container or as a Kubernetes deployment)
run	integration-test	Run the tests from the test-jar artifact in the agent service (e.g. the tests run as part of the Kubernetes platform)
stop	post-integration-test	Stop the agent service
verify	verify	Verify the remote test results and make sure that tests have executed successfully

Each Maven plugin goal is bound to a default Maven build lifecycle phase. Like the Maven failsafe plugin the Citrus agent plugin binds its goals to the Maven `integration-test` phases in the build lifecycle.

The agent plugin makes sure to run the `post-integration-test` phase even when tests are failing.

The plugin propagates the remote test execution results to the local Maven build so the `verify` goal is able to fail the build in case of failed tests.

A typical usage example for the Citrus agent Maven plugin looks like follows:

Maven POM

```
<plugin>
  <groupId>org.citrusframework.mvn</groupId>
  <artifactId>citrus-agent-maven-plugin</artifactId>
  <version>${project.version}</version>
  <configuration>
    <run>
      <async>true</async>
    </run>
    <kubernetes>
      <enabled>true</enabled>
    </kubernetes>
  </configuration>
  <executions>
    <execution>
      <id>citrus-agent-tests</id>
      <goals>
        <goal>test-jar</goal>
        <goal>start</goal>
        <goal>test</goal>
        <goal>stop</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

The example above uses the Citrus agent Maven plugin amd binds its individual goals to the Maven build lifecycle. In this example the Kubernetes support is enabled. This means that the Citrus agent service will be started as a Deployment on the current Kubernetes cluster. Please make sure to connect to the Kubernetes cluster before the Maven build (e.g. via `kubectl`).

The plugin uses the Kubernetes client to create the agent service as a deployment. It creates a ConfigMap from the built test-jar Java archive and makes sure the agent service has access to it (e.g. via volume mounts). It connects to the service and triggers the test run on the agent. It waits for the tes run to finish properly and the remote test results are propagated to the local Maven build. It verifies the test results and makes the local Maven build fail in case of test failures or errors. It automatically removes the Citrus agent service deployment and its related resources from the Kubernetes cluster.

When the local Maven build is successful you can be sure that all tests have run successful on the agent.

43.7. Message processors

Message processors allow you to manipulate message payloads before sending or after receiving messages. The `processor()` fluent API provides access to various message processors for JSON, XML, and SOAP message formats.

You apply a message processor on a send or receive action using the `.process()` method.

43.7.1. JSON pretty print

The JSON formatting message processor pretty-prints a JSON message payload with proper indentation.

Java

```
@CitrusTest
public void jsonPrettyPrintTest() {
    $(send("helloService")
        .message()
        .type(MessageType.JSON)
        .body("{\"name\":\"Citrus\",\"version\":\"4.6\"}")
        .process(processor().json().prettyPrint())
    );
}
```

The processor takes a compact JSON string and formats it with line breaks and indentation before sending. This is useful when you want to ensure that the transmitted JSON payload is human-readable.

You can also apply the pretty print processor on a received message.

Java

```
@CitrusTest
public void jsonPrettyPrintReceiveTest() {
    $(receive("helloService")
        .message()
        .type(MessageType.JSON)
        .body("{\"name\":\"Citrus\",\"version\":\"4.6\"}")
        .process(processor().json().prettyPrint())
    );
}
```

43.7.2. XML pretty print

The XML formatting message processor pretty-prints an XML message payload with proper indentation.

Java

```
@CitrusTest
public void xmlPrettyPrintTest() {
    $(send("helloService")
        .message()
        .type(MessageType.XML)
        .body("<root><element attribute='value'><sub-element>text</sub-
element></element></root>")
        .process(processor().xml().prettyPrint())
    );
}
```

The processor handles both XML `String` payloads and `org.w3c.dom.Document` object payloads.

43.7.3. SOAP pretty print

The SOAP formatting message processor extends the XML formatter with additional SOAP-specific handling. It pretty-prints both the SOAP message body and any SOAP fault detail elements.

Java

```
@CitrusTest
public void soapPrettyPrintTest() {
    $(send("soapService")
        .message()
        .type(MessageType.XML)
        .body("<root><element>text</element></root>")
        .process(processor().xml().soap().prettyPrint())
    );
}
```



The SOAP pretty print processor requires the `citrus-ws` module on the classpath.

Chapter 44. Data dictionaries

Data dictionaries in Citrus provide a new way to manipulate message payload data before a message is sent or received. The dictionary defines a set of keys and respective values. Just like every other dictionary it is used to translate things. In our case we translate message data elements.

You can translate common message elements that are used widely throughout your domain model. As Citrus deals with different types of message data (e.g. XML, JSON) we have different dictionary implementations that are described in the next sections.

44.1. XML data dictionaries

As you would expect XML data dictionaries can be used together with XML formatted message payloads. To do so a dictionary has to be added to the basic Citrus Spring application context which will make the dictionary visible to all test cases:

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("TestMessage.CorrelationId", "${correlationId}");
    dictionary.getMappings().put("TestMessage.User", "Christoph");
    dictionary.getMappings().put("TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("TestMessage.CorrelationId", "${correlationId}");
    dictionary.getMappings().put("TestMessage.User", "Christoph");
    dictionary.getMappings().put("TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="nodeMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="TestMessage.MessageId" value="${messageId}"/>
    <citrus:mapping path="TestMessage.CorrelationId" value="${correlationId}"/>
    <citrus:mapping path="TestMessage.User" value="Christoph"/>
    <citrus:mapping path="TestMessage.TimeStamp" value="citrus:currentDate()"/>
  </citrus:mappings>
</citrus:xml-data-dictionary>
```

As you can see the dictionary is nothing but a normal Spring bean definition. The **NodeMappingDataDictionary** implementation receives a map of key value pairs where the key is a message element path expression. For XML payloads the message element tree is traversed so the path expression is built for an exact message element inside the payload. When matched the respective value is set according to the value stored within the dictionary.

Alternatively the key-value mappings can be defined in an external file and a reference to the file can be provided:

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="nodeMappingDataDictionary">
  <citrus:mapping-file path="classpath:sample-dictionary.properties"/>
</citrus:xml-data-dictionary>
```

The mapping file content just looks like a normal property file in Java:

sample-dictionary.properties

```
TestMessage.MessageId=${messageId}
TestMessage.CorrelationId=${correlationId}
TestMessage.User=Christoph
TestMessage.TimeStamp=citrus:currentDate()
```

You can set any message element value inside the XML message payload. The path expression also supports XML attributes. Just use the attribute name as the last part of the path expression. Let us have a closer look at a sample XML message payload with attributes:

```
<TestMessage>
  <User name="Christoph" age="18"/>
</TestMessage>
```

Using this sample XML payload we can access the attributes in the data dictionary as follows:

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.User.name", "${userName}");
    dictionary.getMappings().put("TestMessage.User.age", "${userAge}");
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.User.name", "${userName}");
    dictionary.getMappings().put("TestMessage.User.age", "${userAge}");
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="nodeMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="TestMessage.User.name" value="${userName}"/>
    <citrus:mapping path="TestMessage.User.age" value="${userAge}"/>
  </citrus:mappings>
</citrus:xml-data-dictionary>
```

The **NodeMappingDataDictionary** implementation is easy to use and can be used with most basic

XML payloads.

For more complex XML payloads where more flexibility is required the XPath data dictionaries may be better suited:

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    return dictionary;
}
```

Spring XML

```
<citrus:xpath-data-dictionary id="xpathMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="//TestMessage/MessageId" value="${messageId}"/>
    <citrus:mapping path="//TestMessage/CorrelationId" value="${correlationId}"/>
    <citrus:mapping path="//TestMessage/User" value="Christoph"/>
    <citrus:mapping path="//TestMessage/User/@id" value="123"/>
    <citrus:mapping path="//TestMessage/TimeStamp" value="citrus:currentDate()"/>
  </citrus:mappings>
</citrus:xpath-data-dictionary>
```

As expected XPath mapping expressions are more powerful and can better handle complex scenarios with XML namespaces, attributes and node lists. Just like the node mapping dictionary the XPath mapping dictionary also supports variables, functions and an external mapping file.

XPath works fine with namespaces. In general it is good practice to define a namespace context where you map namespace URI values with prefix values. So your XPath expression is more precise and evaluation is strict. In Citrus the **NamespaceContextBuilder** which is also added as a normal Spring bean to the application context manages namespaces used in your XPath expressions. See our XML and XPath chapters in this documentation for detailed description how to accomplish fail-safe XPath expressions with namespaces.

This completes the XML data dictionary usage in Citrus. Later on we will see some more advanced data dictionary scenarios where we will discuss the usage of dictionary scopes and mapping strategies. But before that let us have a look at other message formats like JSON messages.

44.2. JSON data dictionaries

JSON data dictionaries complement with XML data dictionaries. As usual we have to add the JSON data dictionary to the basic Spring application context first. Once this is done the data dictionary automatically applies for all JSON message payloads in Citrus. This means that all JSON messages sent and received get translated with the JSON data dictionary implementation.

Citrus uses message types in order to evaluate which data dictionary may fit to the message that is currently processed. As usual you can define the message type directly in your test case as attribute inside the sending and receiving message action.

Let us see a simple dictionary for JSON data:

Citrus Bean

```
@BindToRegistry
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("TestMessage.CorrelationId", "${correlationId}");
    dictionary.getMappings().put("TestMessage.User", "Christoph");
    dictionary.getMappings().put("TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring Bean

```
@Bean
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("TestMessage.CorrelationId", "${correlationId}");
    dictionary.getMappings().put("TestMessage.User", "Christoph");
    dictionary.getMappings().put("TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring XML

```
<citrus:json-data-dictionary id="jsonMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="TestMessage.MessageId" value="${messageId}"/>
    <citrus:mapping path="TestMessage.CorrelationId" value="${correlationId}"/>
    <citrus:mapping path="TestMessage.User" value="Christoph"/>
    <citrus:mapping path="TestMessage.TimeStamp" value="citrus:currentDate()"/>
  </citrus:mappings>
</citrus:json-data-dictionary>
```

The message path expressions do look very similar to those used in XML data dictionaries. Here the

path expression keys do apply to the JSON object graph. See the following sample JSON data which perfectly applies to the dictionary expressions above.

```
{
  "TestMessage": {
      "MessageId": "1122334455",
      "CorrelationId": "100000001",
      "User": "Christoph",
      "TimeStamp": 1234567890
  }
}
```

The path expressions will match a very specific message element inside the JSON object graph. The dictionary will automatically set the message element values then. The path expressions are easy to use as you can traverse the JSON object graph very easy.

Of course the data dictionary does also support test variables, functions. Also very interesting is the usage of JSON arrays. A JSON array element is referenced in a data dictionary like this:

Citrus Bean

```
@BindToRegistry
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.Users[0]", "Christoph");
    dictionary.getMappings().put("TestMessage.Users[1]", "Julia");
    return dictionary;
}
```

Spring Bean

```
@Bean
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.Users[0]", "Christoph");
    dictionary.getMappings().put("TestMessage.Users[1]", "Julia");
    return dictionary;
}
```

Spring XML

```
<citrus:json-data-dictionary id="jsonMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="TestMessage.Users[0]" value="Christoph"/>
    <citrus:mapping path="TestMessage.Users[1]" value="Julia"/>
  </citrus:mappings>
</citrus:json-data-dictionary>
```

The **Users** element is a JSON array, so we can access the elements with index. Nesting JSON objects and arrays is also supported so you can also handle more complex JSON data.

The **JsonMappingDataDictionary** implementation is easy to use and fits the basic needs for JSON data dictionaries. The message element path expressions are very simple and do fit basic needs. However, when more complex JSON payloads apply for translation we might reach the boundaries here.

For more complex JSON message payloads JsonPath data dictionaries are very effective:

Citrus Bean

```
@BindToRegistry
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("$.TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("$..CorrelationId", "${correlationId}");
    dictionary.getMappings().put("$..Users[0]", "Christoph");
    dictionary.getMappings().put("$.TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring Bean

```
@Bean
public JsonPathMappingDataDictionary jsonMappingDataDictionary() {
    JsonPathMappingDataDictionary dictionary = new JsonPathMappingDataDictionary();
    dictionary.getMappings().put("$.TestMessage.MessageId", "${messageId}");
    dictionary.getMappings().put("$..CorrelationId", "${correlationId}");
    dictionary.getMappings().put("$..Users[0]", "Christoph");
    dictionary.getMappings().put("$.TestMessage.TimeStamp", "citrus:currentDate()");
    return dictionary;
}
```

Spring XML

```
<citrus:json-path-data-dictionary id="jsonMappingDataDictionary">
  <citrus:mappings>
    <citrus:mapping path="$.TestMessage.MessageId" value="${messageId}"/>
    <citrus:mapping path="$..CorrelationId" value="${correlationId}"/>
    <citrus:mapping path="$..Users[0]" value="Christoph"/>
    <citrus:mapping path="$.TestMessage.TimeStamp" value="citrus:currentDate()"/>
  </citrus:mappings>
</citrus:json-path-data-dictionary>
```

JsonPath mapping expressions are way more powerful and can also handle very complex scenarios. You can apply for all elements named *CorrelationId* in one single entry for instance.

44.3. Dictionary scopes

Now that we have learned how to add data dictionaries to Citrus we need to discuss some advanced topics. Data dictionary scopes do define the boundaries where the dictionary may apply. By default, data dictionaries are global scope dictionaries. This means that the data dictionary applies to all messages sent and received with Citrus. Of course message types are considered so XML data dictionaries do only apply to XML message types. However, global scope dictionaries will be activated throughout all test cases and actions.

You can overwrite the dictionary scope. For instance in order to use an explicit scope. When this is done the dictionary wil not apply automatically but the user has to explicitly set the data dictionary in sending or receiving test action. This way you can activate the dictionary to a very special set of test actions.

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    dictionary.setGlobalScope(false);
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.setMappingFile("classpath:sample-dictionary.properties");
    dictionary.setGlobalScope(false);
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="specialDataDictionary" global-scope="false">
  <citrus:mapping-file path="classpath:sample-dictionary.properties"/>
</citrus:xml-data-dictionary>
```

We set the global scope property to **false** so the dictionary is handled in explicit scope. This means that you have to set the data dictionary explicitly in your test actions:

Java

```
@CitrusTest
public void dictionaryTest() {
    $(send(myEndpoint)
        .message()
        .body("<TestMessage>Hello Citrus</TestMessage>")
        .dictionary("specialDataDictionary"));
}
```

XML

```
<test name="DictionaryTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <send endpoint="myEndpoint">
            <message data-dictionary="specialDataDictionary">
                <body>
                    <data>
                    <![CDATA[
                        <TestMessage>Hello Citrus</TestMessage>
                    ]]>
                    </data>
                </body>
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: DictionaryTest
actions:
  - send:
      endpoint: "myEndpoint"
      message:
        dataDictionary: specialDataDictionary
        body: |
          <TestMessage>Hello Citrus</TestMessage>
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="DictionaryTest">
        <actions>
            <send endpoint="myEndpoint">
                <message data-dictionary="specialDataDictionary">
                    <data>
                    <![CDATA[
                        <TestMessage>Hello Citrus</TestMessage>
                    ]]>
                    </data>
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

The sample above is a sending test action with an explicit data dictionary reference set. Before sending the message the dictionary is asked for translation. So all matching message element values will be set by the dictionary accordingly. Other global data dictionaries do also apply for this message but the explicit dictionary will always overwrite the message element values.

44.4. Path mapping strategies

Another advanced topic about data dictionaries is the path mapping strategy. When using simple path expressions the default strategy is always **EXACT** . This means that the path expression has to evaluate exactly to a message element within the payload data. And only this exact message element is translated.

You can set your own path mapping strategy in order to change this behavior. For instance another mapping strategy would be **STARS_WITH** . All elements are translated that start with a certain path expression. Let us clarify this with an example:

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.Property", "citrus:randomString()");
    dictionry.setPathMappingStrategy(PathMappingStrategy.STARTS_WITH);
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("TestMessage.Property", "citrus:randomString()");
    dictionry.setPathMappingStrategy(PathMappingStrategy.STARTS_WITH);
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="nodeMappingDataDictionary" mapping-
strategy="STARTS_WITH">
  <citrus:mappings>
    <citrus:mapping path="TestMessage.Property" value="citrus:randomString()"/>
  </citrus:mappings>
</citrus:xml-data-dictionary>
```

Now with the path mapping strategy set to **STARS_WITH** all message element path expressions starting with **TestMessage.Property** will find translation in this dictionary. Following sample message payload would be translated accordingly:

```
<TestMessage>
    <Property>XXX</Property>
    <PropertyName>XXX</PropertyName>
    <PropertyValue>XXX</PropertyValue>
</TestMessage>
```

All child elements of **TestMessage** starting with **Property** will be translated with this data dictionary. In the resulting message payload Citrus will use a random string as value for these elements as we used the **citrus:randomString()** function in the dictionary mapping.

The next mapping strategy would be **ENDS_WITH** . No surprises here - this mapping strategy looks for message elements that end with a certain path expression. Again a simple example will clarify this for you.

Citrus Bean

```
@BindToRegistry
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("Id", "citrus:randomNumber()");
    dictionry.setPathMappingStrategy(PathMappingStrategy.ENDS_WITH);
    return dictionary;
}
```

Spring Bean

```
@Bean
public NodeMappingDataDictionary nodeMappingDataDictionary() {
    NodeMappingDataDictionary dictionary = new NodeMappingDataDictionary();
    dictionary.getMappings().put("Id", "citrus:randomNumber()");
    dictionry.setPathMappingStrategy(PathMappingStrategy.ENDS_WITH);
    return dictionary;
}
```

Spring XML

```
<citrus:xml-data-dictionary id="nodeMappingDataDictionary" mapping-
strategy="ENDS_WITH">
  <citrus:mappings>
    <citrus:mapping path="Id" value="citrus:randomNumber()"/>
  </citrus:mappings>
</citrus:xml-data-dictionary>
```

Again let us see some sample message payload for this dictionary usage:

```
<TestMessage>
  <RequestId>XXX</RequestId>
  <Properties>
    <Property>
      <PropertyId>XXX</PropertyId>
      <PropertyValue>XXX</PropertyValue>
    </Property>
    <Property>
      <PropertyId>XXX</PropertyId>
      <PropertyValue>XXX</PropertyValue>
    </Property>
  </Properties>
</TestMessage>
```

In this sample all message elements ending with **Id** would be translated with a random number. No matter where in the message tree the elements are located. This is quite useful but also very powerful. So be careful to use this strategy in global data dictionaries as it may translate message elements that you would not expect in the first place.

Chapter 45. Test actors

The concept of test actors is a way to enable/disable a group of tests actions and message endpoints based on configuration. Test actors help you to use the same tests in different environments where some of the participating services may be available or need to be simulated by Citrus. In particular in end-to-end test scenarios some interface partners may be deployed and ready for participating in the test. In other environments no interface partners are available and Citrus needs to simulate all interfaces accordingly. Test actors allows us to use the very same test definition in both environments by just adjusting the Citrus configuration.

Usually Citrus simulates all interface partners within a test case which is great for continuous integration testing where just the System Under Test is deployed and all surrounding infrastructure and partners get simulated. In end-to-end integration test scenarios some of our interface partners may be deployed and online. Some other interface partners still require Citrus simulation logic.

It would be great if we were able to reuse the Citrus integration tests in all of these described test environments. The Citrus test describes the complete test flow of messages, we only have to remove the simulated send/receive actions for those interface partner applications that are deployed and available in an end-to-end test setup.

With test actors we have the opportunity to link test actions, in particular send/receive message actions, to a named actor. The test actor can be enabled/disabled via configuration and following from that all linked test actions are enabled/disabled, too.

This means the same Citrus test case is runnable with different test actor configurations where some services are deployed as real applications and others may require simulation by Citrus.

45.1. Create test actors

First thing to do is to define one or more test actors in the Citrus configuration. A test actor represents a participating party (e.g. interface partner, backend application, service).

We define the test actors as beans and add them to the Citrus bean registry.

Citrus Bean

```
@BindToRegistry(name = "travel_agency")
public TestActor travelAgencyActor() {
    return new TestActor("travel_agency");
}

@BindToRegistry(name = "royal_airline")
public TestActor royalAirlineActor() {
    return new TestActor("royal_airline");
}

@BindToRegistry(name = "smart_ariline")
public TestActor smartAirlineActor() {
    return new TestActor("smart_ariline");
}
```

Spring Bean

```
@Bean(name = "travel_agency")
public TestActor travelAgencyActor() {
    return new TestActor("travel_agency");
}

@Bean(name = "royal_airline")
public TestActor royalAirlineActor() {
    return new TestActor("royal_airline");
}

@Bean(name = "smart_ariline")
public TestActor smartAirlineActor() {
    return new TestActor("smart_ariline");
}
```

Spring XML

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:citrus="http://www.citrusframework.org/schema/config">
    <citrus:actor id="travel_agency" name="TRAVEL_AGENCY"/>
    <citrus:actor id="royal_airline" name="ROYAL_AIRLINE"/>
    <citrus:actor id="smart_ariline" name="SMART_AIRLINE"/>
</spring:beans>
```

The listing above defines three test actors participating in our test scenario. A `travel_agency` application which is simulated by Citrus as a calling client, the `smart_airline` application and a `royal_airline` application. Now we have the test actors defined we can link those to message sender/receiver instances and/or test actions within our test case.

45.2. Reference test actors

We need to reference the test actor and set it on a message send or receive action in our test case. We can do this in two different ways. First of all we can set a test actor reference on a message endpoint.

Citrus Bean

```
@BindToRegistry
public JmsSyncEndpoint royalAirlineBookingEndpoint() {
    return JmsEndpoints.jms()
                .synchronous()
                .destination("${royal.airline.request.queue}")
                .actor(royalAirlineActor())
                .build();
}
```

Spring Bean

```
@Bean
public JmsSyncEndpoint royalAirlineBookingEndpoint() {
    return JmsEndpoints.jms()
                .synchronous()
                .destination("${royal.airline.request.queue}")
                .actor(royalAirlineActor())
                .build();
}
```

Spring XML

```
<citrus-jms:sync-endpoint id="royalAirlineBookingEndpoint"
        destination-name="${royal.airline.request.queue}"
        actor="royal_airline"/>
```

Now all test actions that are using this message endpoint are linked to the test actor. In addition to that you can also explicitly link test actions to a test actor.

Java

```
@CitrusTest
public void airlineTest() {
    $(receive("royalAirlineBookingEndpoint")
        .message()
        .body("...")
        .actor("royal_airline")
    );

    $(send("royalAirlineBookingEndpoint")
        .message()
        .body("...")
        .actor("royal_airline")
    );
}
```

XML

```
<test name="AirlineTest" xmlns="http://citrusframework.org/schema/xml/testcase">
    <actions>
        <receive endpoint="royalAirlineBookingEndpoint" actor="royal_airline">
            <message>
                <!-- ... -->
            </message>
        </receive>

        <send endpoint="royalAirlineBookingEndpoint" actor="royal_airline">
            <message>
                <!-- ... -->
            </message>
        </send>
    </actions>
</test>
```

YAML

```
name: AirlineTest
actions:
  - receive:
      endpoint: "royalAirlineBookingEndpoint"
      actor: "royal_airline"
      message: {}
  - send:
      endpoint: "royalAirlineBookingEndpoint"
      actor: "royal_airline"
      message: {}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans">
    <testcase name="AirlineTest">
        <actions>
            <receive endpoint="royalAirlineBookingEndpoint" actor="royal_airline">
                <message>
                    <!-- ... -->
                </message>
            </receive>

            <send endpoint="royalAirlineBookingEndpoint" actor="royal_airline">
                <message>
                    <!-- ... -->
                </message>
            </send>
        </actions>
    </testcase>
</spring:beans>
```

This explicitly links the test actor named `royal_ariline` to test actions. The test actor may be enabled/disabled according to the test environment. All linked test actions and message endpoints will be skipped when the referenced test actor is disabled.

45.3. Disable test actors

By default, test actors are enabled. In some environments it may be required to disable the test actor and all its linked test actions and message endpoints though.

In the airline example usually both airline applications are simulated in the integration tests. Assume that there is a test environment where one of the simulated applications (e.g. the `royal_airline` application) is available as a real application instance deployed. In this scenario the Citrus tests should skip all simulated message interactions for the `royal_airline` application and the real application instance in the test environment should consume the messages instead.

This is easy as we have linked all send/receive actions to one of our test actors. So we can disable the `royal_airline` test actor in our configuration:

Citrus Bean

```
@BindToRegistry(name = "travel_agency")
public TestActor travelAgencyActor() {
    return new TestActor("travel_agency");
}

@BindToRegistry(name = "royal_airline")
public TestActor royalAirlineActor() {
    TestActor royalAirlineActor = new TestActor("royal_airline");
    royalAirlineActor.setDisabled(true);
    return royalAirlineActor;
}

@BindToRegistry(name = "smart_ariline")
public TestActor smartAirlineActor() {
    return new TestActor("smart_ariline");
}
```

Spring Bean

```
@Bean(name = "travel_agency")
public TestActor travelAgencyActor() {
    return new TestActor("travel_agency");
}

@Bean(name = "royal_airline")
public TestActor royalAirlineActor() {
    TestActor royalAirlineActor = new TestActor("royal_airline");
    royalAirlineActor.setDisabled(true);
    return royalAirlineActor;
}

@Bean(name = "smart_ariline")
public TestActor smartAirlineActor() {
    return new TestActor("smart_ariline");
}
```

Spring XML

```
<spring:beans xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:citrus="http://www.citrusframework.org/schema/config">
    <citrus:actor id="travel_agency" name="TRAVEL_AGENCY"/>
    <citrus:actor id="royal_airline" name="ROYAL_AIRLINE" disabled="true"/>
    <citrus:actor id="smart_ariline" name="SMART_AIRLINE"/>
</spring:beans>
```

Any test action linked to this test actor `royal_airline` is now skipped in the test execution. The real `royal_airline` application available in our test scenario will handle the requests instead. The

`travel_agency` and the `smart_airline` applications still get simulated by Citrus.

This is a perfect way of reusing integration tests in different test scenarios where you enable and disable simulated participating parties in Citrus.



Server ports may be of special interest when dealing with different test scenarios. You may have to also disable a Citrus embedded Jetty server instance in order to avoid port binding conflicts. You may have to wire endpoint URIs accordingly before executing a test. The real life application may not use the same port and IP address as the Citrus embedded servers for simulation.

45.4. Test actor environment settings

You can enable/disable the test actor also by using System property or environment variable settings. The property and environment variable names follow a specific formatting that includes the test actor name:

- `citrus.test.actor.<name>.enabled=true/false`
- `CITRUS_TEST_ACTOR_<NAME>_ENABLED=true/false`

When the environment setting is available on the test host the test actor with the referenced name is enabled/disabled.

You can set these properties in the `citrus-application.properties` for instance:

citrus-application.properties

```
citrus.test.actor.royal_airline.enabled=false
```

This disables the test actor named `royal_airline` and skips all its linked test actions and message endpoints.

Chapter 46. Test suite actions

A test framework should also provide the functionality to do some work before and after the test run. Typical tasks in the section before or after a test could be preparing data in a database or starting/stopping a service. These tasks fit best into the initialization or cleanup phase of Citrus.

46.1. Before suite

You can influence the behavior of a whole test run in the initialization phase of the test suite. Before any of the tests has started this section is executed once for the whole test run. See the next code example to find out how it works with actions that take place before the first test is executed:

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .actions(
                    echo().message("This is run before suite")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .actions(
                    echo().message("This is run before suite")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                  http://www.springframework.org/schema/beans/spring-beans.xsd
                  http://www.citrusframework.org/schema/config
                  http://www.citrusframework.org/schema/config/citrus-config.xsd
                  http://www.citrusframework.org/schema/testcase
                  http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-suite id="actionsBeforeSuite">
        <citrus:actions>
            <!-- list of actions before suite -->
            <citrus-test:echo>
              <citrus-test:message>This is run before suite</message>
            </citrus-test:echo>
        </citrus:actions>
    </citrus:before-suite>
</spring:beans>
```



It is important to notice that the Citrus Spring bean XML configuration components belong to a separate namespace `citrus:http://www.citrusframework.org/schema/config`. You need to add the namespace declaration to the Spring bean XML root element of configuration file accordingly. Test actions use a different namespace `citrus-test:http://www.citrusframework.org/schema/testcase` so we need to include both namespaces.

The Citrus configuration component holds a list of Citrus test actions that get executed before the test suite. You can add all Citrus test actions here as you would do in a normal test case definition.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                  http://www.springframework.org/schema/beans/spring-beans.xsd
                  http://www.citrusframework.org/schema/config
                  http://www.citrusframework.org/schema/config/citrus-config.xsd
                  http://www.citrusframework.org/schema/testcase
                  http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-suite id="actionsBeforeSuite">
        <citrus:actions>
            <!-- list of actions before suite -->
            <citrus-test:sql datasource="testDataSource">
                <citrus-test:statement>CREATE TABLE PERSON (ID integer, NAME
char(250))</citrus-test:statement>
            </citrus-test:sql>
        </citrus:actions>
    </citrus:before-suite>
</spring:beans>
```

The example above uses an SQL test action to access the database and create a table `PERSON` which is obviously needed in our test cases. You can think of several actions here to prepare the tst infrastructure before the suite.



Citrus offers special startup and shutdown actions that may start and stop server implementations automatically. This might be helpful when dealing with Http servers or other service containers. You can also think of starting/stopping a message broker (e.g. Kafka) before a test run.

Users may have multiple configuration components declared to run in the before suite phase. You

can use different ids to separate the beans in the Citrus of Spring bean registry.

By default, Citrus scans for all available before suite containers and runs them sequentially. You can restrict the before suite action container execution by adding a filter criteria on the suite name, a test group name, environment variables or system properties:

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .onSuite("databaseSuite")
                .onTestGroup("e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .onSuite("databaseSuite")
                .onTestGroup("e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                  http://www.springframework.org/schema/beans/spring-beans.xsd
                  http://www.citrusframework.org/schema/config
                  http://www.citrusframework.org/schema/config/citrus-config.xsd
                  http://www.citrusframework.org/schema/testcase
                  http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-suite id="actionsBeforeSuite" suites="databaseSuite" groups="e2e">
        <citrus:actions>
            <!-- list of actions before suite -->
            <citrus-test:sql datasource="testDataSource">
                <citrus-test:statement>CREATE TABLE PERSON (ID integer, NAME
char(250))</citrus-test:statement>
            </citrus-test:sql>
        </citrus:actions>
    </citrus:before-suite>
</spring:beans>
```

The above before suite container is only executed with the test suite called `databaseSuite`. Also, it adds a filter on the test group that should match the group name `e2e`.



Test groups are only supported when using the TestNG as a testing engine.



You can define multiple suite names and test groups with comma-delimited strings as an attribute value.

Environment variables or system properties are defined as a list of key-value pairs. When such a filter criteria is present the specified variables and properties have to be set in the environment with the respective value. In case the property value is left out in the configuration the System property must simply exist on the in order to enable the before suite container.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .whenEnv("GITHUB_ENV", "")
                .whenSystemProperty("test-stage", "e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeSuite beforeSuite() {
        return new SequenceBeforeSuite.Builder()
                .whenEnv("GITHUB_ENV", "")
                .whenSystemProperty("test-stage", "e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                  http://www.springframework.org/schema/beans/spring-beans.xsd
                  http://www.citrusframework.org/schema/config
                  http://www.citrusframework.org/schema/config/citrus-config.xsd
                  http://www.citrusframework.org/schema/testcase
                  http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-suite id="actionsBeforeSuite">
        <citrus:env>
          <citrus:property name="GITHUB_ENV"/>
        </citrus:env>
        <citrus:system>
          <citrus:property name="test-stage" value="e2e"/>
        </citrus:system>
        <citrus:actions>
            <!-- list of actions before suite -->
            <citrus-test:sql datasource="testDataSource">
                <citrus-test:statement>CREATE TABLE PERSON (ID integer, NAME
char(250))</citrus-test:statement>
            </citrus-test:sql>
        </citrus:actions>
    </citrus:before-suite>
</spring:beans>
```

In the example above the before suite container is restricted to environments with `GITHUB_ENV` property set. Also, the system property `test-stage` must be set to the value `e2e`. Otherwise, the before suite container execution is skipped for this environment.

46.2. After suite

A test may run some actions to clean up the test environment after the test. Just like the before suite is keen to prepare data before the suite is executed, you can add tasks to the test run after the last test in the test suite is finished. This means that the after suite actions are run once after the last test.



It is a good idea to clean up the test environment after the test run. For instance, you can purge all JMS destinations and Kafka topics or clean up the database after the test run. This avoids errors in follow-up test runs where left over test data may influence upcoming tests.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite afterSuite() {
        return new SequenceAfterSuite.Builder()
                .actions(
                    echo().message("This is run after suite")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceAfterSuite afterSuite() {
        return new SequenceAfterSuite.Builder()
                .actions(
                    echo().message("This is run after suite")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                                  http://www.springframework.org/schema/beans/spring-
beans.xsd
                                  http://www.citrusframework.org/schema/config
                                  http://www.citrusframework.org/schema/config/citrus-
config.xsd
                                  http://www.citrusframework.org/schema/testcase

http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:after-suite id="actionsAfterSuite">
        <citrus:actions>
            <!-- list of actions after suite -->
            <citrus-test:echo>
              <citrus-test:message>This is run after suite</message>
            </citrus-test:echo>
        </citrus:actions>
    </citrus:after-suite>
</spring:beans>
```

The after suite configuration component receives a unique id and holds one to many test actions as nested configuration elements.

Users may have multiple configuration components declared to run in the after suite phase. You can use different ids to separate the beans in the Citrus of Spring bean registry.

By default, Citrus scans for all available after suite containers and runs them sequentially. You can restrict the after suite action container execution by adding a filter criteria on the suite name, a test group name, environment variables or system properties. Please refer to the before suite examples in this guide to see how it works.

46.3. Before test

You may have tasks that need to run before each test is executed. Just like you have prepared some data in the actions before the whole test suite you can do the same before each test.



It is reasonable to clean up the test environment also between the tests (e.g. purge all JMS queues or Kafka topics). In case a previous test fails some messages might be left in the message queues. Also, a failing test may leave the database in a dirty state. The next test may be confronted with these invalid messages and data state. You can avoid these follow-up test failures with a good clean up before a test.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeTest() {
        return new SequenceBeforeTest.Builder()
                .actions(
                    echo().message("This is run before test")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeTest beforeTest() {
        return new SequenceBeforeTest.Builder()
                .actions(
                    echo().message("This is run before test")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                                  http://www.springframework.org/schema/beans/spring-
beans.xsd
                                  http://www.citrusframework.org/schema/config
                                  http://www.citrusframework.org/schema/config/citrus-
config.xsd
                                  http://www.citrusframework.org/schema/testcase

http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-test id="actionsBeforeTest">
        <citrus:actions>
            <!-- list of actions before test -->
            <citrus-test:echo>
              <citrus-test:message>This is run before test</message>
            </citrus-test:echo>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```

The before test configuration component receives a unique id and a list of test actions that get executed before a test case is started. The component receives usual test action definitions just like you would write them in a normal test case definition.



It is important to notice that the Citrus Spring bean XML configuration components belong to a separate namespace `citrus:http://www.citrusframework.org/schema/config`. You need to add the namespace declaration to the Spring bean XML root element of configuration file accordingly. Test actions use a different namespace `citrus-test:http://www.citrusframework.org/schema/testcase` so we need to include both namespaces.

The echo test action in the example above is now executed before each test in our test suite run. Also notice that we can restrict the before test container execution on certain filter criteria. We can restrict the execution based on the test name, package, test groups and environment or system properties.

See following example how this works:

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeSuite beforeTest() {
        return new SequenceBeforeTest.Builder()
                .onTests("*_Ok_Test")
                .onPackage("org.citrusframework.longrunning.*")
                .actions(
                    echo().message("This is run before test")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeTest beforeTest() {
        return new SequenceBeforeTest.Builder()
                .onTests("*_Ok_Test")
                .onPackage("org.citrusframework.longrunning.*")
                .actions(
                    echo().message("This is run before test")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                                  http://www.springframework.org/schema/beans/spring-
beans.xsd
                                  http://www.citrusframework.org/schema/config
                                  http://www.citrusframework.org/schema/config/citrus-
config.xsd
                                  http://www.citrusframework.org/schema/testcase

http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-test id="actionsBeforeTest" test="*_Ok_Test"
package="org.citrusframework.longrunning.*">
        <citrus:actions>
            <!-- list of actions before test -->
            <citrus-test:echo>
              <citrus-test:message>This is run before test</message>
            </citrus-test:echo>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```

As an example the above before test component is only executed for test cases that match the name pattern `\*_Ok_Test` and that match the package `org.citrusframework.longrunning.*`.

You can also filter based on environment variables or system properties. When specified the properties have to be present in the test environment with the respective value.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceBeforeTest beforeTest() {
        return new SequenceBeforeTest.Builder()
                .whenEnv("GITHUB_ENV", "")
                .whenSystemProperty("test-stage", "e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceBeforeTest beforeTest() {
        return new SequenceBeforeTest.Builder()
                .whenEnv("GITHUB_ENV", "")
                .whenSystemProperty("test-stage", "e2e")
                .actions(
                    sql().dataSource(testDataSource)
                        .statement("CREATE TABLE PERSON (ID integer, NAME char(250))")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                                  http://www.springframework.org/schema/beans/spring-
beans.xsd
                                  http://www.citrusframework.org/schema/config
                                  http://www.citrusframework.org/schema/config/citrus-
config.xsd
                                  http://www.citrusframework.org/schema/testcase

http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:before-test id="actionsBeforeTest">
        <citrus:env>
          <citrus:property name="GITHUB_ENV"/>
        </citrus:env>
        <citrus:system>
          <citrus:property name="test-stage" value="e2e"/>
        </citrus:system>
        <citrus:actions>
            <!-- list of actions before suite -->
            <citrus-test:sql datasource="testDataSource">
                <citrus-test:statement>CREATE TABLE PERSON (ID integer, NAME
char(250))</citrus-test:statement>
            </citrus-test:sql>
        </citrus:actions>
    </citrus:before-test>
</spring:beans>
```

In the example above the before test container is restricted to environments with `GITHUB_ENV` property set. Also, the system property `test-stage` must be set to the value `e2e`. Otherwise, the before test container execution is skipped for this environment.

46.4. After test

The same logic that applies to the `before-test` configuration component can be done after each test. The `after-test` configuration component defines test actions executed after each test.

Citrus Bean

```
public class BeanConfiguration {

    @BindToRegistry
    public SequenceAfterSuite afterTest() {
        return new SequenceAfterTest.Builder()
                .actions(
                    echo().message("This is run after test")
                );
    }
}
```

Spring Bean

```
@Configuration
public class BeanConfiguration {
    @Bean
    public SequenceAfterTest afterTest() {
        return new SequenceAfterTest.Builder()
                .actions(
                    echo().message("This is run after test")
                );
    }
}
```

Spring XML

```
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xmlns:citrus="http://www.citrusframework.org/schema/config"
              xmlns:citrus-test="http://www.citrusframework.org/schema/testcase"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                                  http://www.springframework.org/schema/beans/spring-
beans.xsd
                                  http://www.citrusframework.org/schema/config
                                  http://www.citrusframework.org/schema/config/citrus-
config.xsd
                                  http://www.citrusframework.org/schema/testcase

http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">

    <citrus:after-test id="actionsBeforeTest">
        <citrus:actions>
            <!-- list of actions after test -->
            <citrus-test:echo>
              <citrus-test:message>This is run after test</message>
            </citrus-test:echo>
        </citrus:actions>
    </citrus:after-test>
</spring:beans>
```

The after test configuration component receives a unique id and a list of test actions that get executed after a test case is finished.



Notice that the after test actions are executed no matter what result (success or failure) the test case has produced.

The after suite container component receives usual test action definitions just like you would write them in a normal test case definition. Of course, you can also restrict the after suite container execution based on filter criteria on the test name, test groups, environment variables and system properties.

Please see the explanations in the previous section about before test actions to see how it works.

Chapter 47. Customize meta information

Test cases in Citrus are usually provided with some meta information like the author's name or the date of creation. In Citrus you are able to extend this test case meta information with your own very specific criteria.

By default a test case comes shipped with meta information that looks like this:

```
<testcase name="PwdChange_OK_1_Test">
    <meta-info>
        <author>Christoph</author>
        <creationdate>2010-01-18</creationdate>
        <status>FINAL</status>
        <last-updated-by>Christoph</last-updated-by>
        <last-updated-on>2010-01-18T15:00:00</last-updated-on>
    </meta-info>

    [...]
</testcase>
```

You can quite easily add data to this section in order to meet your individual testing strategy. Let us have a simple example to show how it is done.

First of all we define a custom XSD schema describing the new elements:

```
<?xml version="1.0" encoding="UTF-8"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema"
        xmlns:tns="http://www.citrusframework.org/samples/my-testcase-info"
        targetNamespace="http://www.citrusframework.org/samples/my-testcase-info"
        elementFormDefault="qualified">

    <element name="requirement" type="string"/>
    <element name="pre-condition" type="string"/>
    <element name="result" type="string"/>
    <element name="classification" type="string"/>
</schema>
```

We have four simple elements (**requirement**, **pre-condition**, **result** and **classification**) all typed as string. These new elements later go into the test case meta information section.

After we added the new XML schema file to the classpath of our project we need to announce the schema to Spring. As you might know already a Citrus test case is nothing else but a simple Spring configuration file with customized XML schema support. If we add new elements to a test case Spring needs to know the XML schema for parsing the test case configuration file. See the **spring.schemas** file usually placed in the META-INF/spring.schemas in your project.

The file content for our example will look like follows:

```
http://www.citrusframework.org/samples/my-testcase-info/my-testcase-
info.xsd=org/citrusframework/schemas/my-testcase-info.xsd
```

So now we are finally ready to use the new meta-info elements inside the test case:

```
<?xml version="1.0" encoding="UTF-8"?>
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
    xmlns:spring="http://www.springframework.org/schema/beans"
    xmlns:custom="http://www.citrusframework.org/samples/my-testcase-info"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans.xsd
          http://www.citrusframework.org/schema/testcase
          http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd
          http://www.citrusframework.org/samples/my-testcase-info
          http://www.citrusframework.org/samples/my-testcase-info/my-testcase-
info.xsd">

    <testcase name="PwdChange_OK_1_Test">
        <meta-info>
            <author>Christoph</author>
            <creationdate>2010-01-18</creationdate>
            <status>FINAL</status>
            <last-updated-by>Christoph</last-updated-by>
            <last-updated-on>2010-01-18T15:00:00</last-updated-on>
            <custom:requirement>REQ10001</custom:requirement>
            <custom:pre-condition>Existing user, sufficient rights</custom:pre-
condition>
            <custom:result>Password reset in database</custom:result>
            <custom:classification>PasswordChange</custom:classification>
        </meta-info>

        [...]
    </testcase>
</spring:beans>
```



We use a separate namespace declaration with a custom namespace prefix “custom” in order to declare the new XML schema to our test case. Of course you can pick a namespace url and prefix that fits best for your project.As you see it is quite easy to add custom meta information to your Citrus test case. The customized elements may be precious for automatic reporting. XSL transformations for instance are able to read those meta information elements in order to generate automatic test reports and documentation.

You can also declare our new XML schema in the Eclipse preferences section as user specific XML catalog entry. Then even the schema code completion in your Eclipse XML editor will be available for our customized meta-info elements.

Chapter 48. Reporting and Test Results

The framework generates different reports and results after a test run for you. These report and result pages will help you to get an overview of the test cases that were executed and which one were failing. They're being generated into `target/citrus-reports` by default, but that can be changed by setting the `citrus.report.directory` property to the desired location. E.g. it could be set to `citrus.report.directory=build/citrus-reports` for Gradle builds.

48.1. General report settings

The following settings apply to all reporters and control general reporting behavior.

Table 15. System properties

System property	Description
citrus.report.directory	Target directory for report files (default="target/citrus-reports")
citrus.report.auto.clear	Automatically clear all test results after finishing the test suite (default="true")
citrus.report.ignore.errors	Ignore errors during report generation (default="true")

Table 16. Environment variables

Environment variable	Description
CITRUS_REPORT_DIRECTORY	Target directory for report files (default="target/citrus-reports")
CITRUS_REPORT_AUTO_CLEAR	Automatically clear all test results after finishing the test suite (default="true")
CITRUS_REPORT_IGNORE_ERRORS	Ignore errors during report generation (default="true")

In the following chapters, we'll provide an overview of the three standard loggers available in Citrus. It's important to note that in Spring, you have the flexibility to replace these standard loggers with your own implementations. This can be achieved by registering custom implementations, as demonstrated in the following code snippet:

```
@Configuration
public static class OverrideReporterConfiguration {

    @Bean
    public LoggingReporter citrusLoggingReporter() {
        return new OverrideLoggingReporter();
    }

    @Bean
    public JUnitReporter citrusJunitReporter() {
        return new OverrideJUnitReporter();
    }

    @Bean
    public HtmlReporter citrusHtmlReporter() {
        return new OverrideHtmlReporter();
    }
}
```

Note that you need to allow spring bean overriding to load this configuration as such. It's worth noting that bean overriding is not recommended by Spring. Thus the following properties are provided to deactivate standard reporters as needed.

```
citrus.default.logging.reporter.enabled=false
citrus.default.junit.reporter.enabled=false
citrus.default.html.reporter.enabled=false
```

48.2. Console logging

During the test run the framework will provide a huge amount of information that is printed out to the console. This includes current test progress, validation results and error information. This enables the user to quickly supervise the test run progress. Failures in tests will be printed to the console just the time the error occurred. The detailed stack trace information and the detailed error messages are helpful to get the idea what went wrong.

As the console output might be limited to a defined buffer limit, the user may not be able to follow the output to the very beginning of the test run. Therefore the framework additionally prints all information to a log file according to the logging configuration.

The logging mechanism uses the SLF4J logging framework. SLF4J is independent of logging framework implementations on the market. So in case you use Log4J logging framework the specified log file path as well as logging levels can be freely configured in the respective log4j.xml file in your project. At the end of a test run the combined test results get printed to both console and log file. The overall test results look like following example:

```
CITRUS TEST RESULTS

  [...]
  HelloService_Ok_1             : SUCCESS
  HelloService_Ok_2             : SUCCESS
  EchoService_Ok_1              : SUCCESS
  EchoService_Ok_2              : SUCCESS
  EchoService_TempError_1       : SUCCESS
  EchoService_AutomaticRetry_1 : SUCCESS
  [...]

  Found 175 test cases to execute
  Skipped 0 test cases (0.0%)
  Executed 175 test cases
  Tests failed:         0 (0.0%)
  Tests successfully: 175 (100.0%)
```

Failed tests will be marked as failed in the result list. The framework will give a short description of the error cause while the detailed stack trace information can be found in the log messages that were made during the test run.

```
HelloService_Ok_3 : failed - Exception is Action timed out
```

48.3. JUnit reports

As tests are executed as TestNG test cases, the framework will also generate JUnit compliant XML and HTML reports. JUnit test reports are very popular and find support in many build management and development tools. In general the Citrus test reports give you an overall picture of all tests and tell you which of them were failing.

Build management tools like Jenkins can easily import and display the generated JUnit XML results. Please have a look at the TestNG and JUnit documentation for more information about this topic as well as the build management tools (e.g. Jenkins) to find out how to integrate the tests results.

48.4. HTML reports

Citrus creates HTML reports after each test run. The report provides detailed information on the test run with a summary of all test results. You can find the report after a test run in the directory ***${project.build.directory}/citrus-reports*** .

The report consists of two parts. The test summary on top shows the total number executed tests. The main part lists all test cases with detailed information. With this report you immediately identify all tests that were failing. Each test case is marked in color according to its result outcome.

The failed tests give detailed error information with error messages and Java StackTrace information. In addition to that the report tries to find the test action inside the XML test part that

failed in execution. With the failing code snippet you can see where the test stopped.



JavaScript should be active in your web browser. This is to enable the detailed information which comes to you in form of tooltips like test author or description. If you want to access the tooltips JavaScript should be enabled in your browser.

The HTML reports are customizable by system properties and environment variables:

Table 17. System properties

System property	Description
citrus.html.report.enabled	Enables/disables HTML report generation (default="true")
citrus.html.report.directory	Output directory path (default=uses `citrus.report.directory`)
citrus.html.report.file	File name for the report file (default="citrus-test-results.html")
citrus.html.report.template	Template HTML file with placeholders for report results
citrus.html.report.detail.template	Template file for detailed test results
citrus.html.report.logo	File resource path pointing to an image that is added to top of HTML report

Table 18. Environment variables

Environment variable	Description
CITRUS_HTML_REPORT_ENABLED	Enables/disables HTML report generation (default="true")
CITRUS_HTML_REPORT_DIRECTORY	Output directory path (default=uses `CITRUS_REPORT_DIRECTORY`)
CITRUS_HTML_REPORT_FILE	File name for the report file (default="citrus-test-results.html")
CITRUS_HTML_REPORT_TEMPLATE	Template HTML file with placeholders for report results
CITRUS_HTML_REPORT_DETAIL_TEMPLATE	Template file for detailed test results
CITRUS_HTML_REPORT_LOGO	File resource path pointing to an image that is added to top of HTML report

The HTML report is based on a template file that is customizable to your special needs.

48.4.1. JUnit report settings

The JUnit XML report settings control the output of JUnit-compatible test result files.

Table 19. System properties

System property	Description
citrus.junit.report.enabled	Enables/disables JUnit report generation (default="true")
citrus.junit.report.directory	Output directory for JUnit report files, relative to the general report directory (default="junitreports")
citrus.junit.report.file.pattern	File name pattern for JUnit report files, using `%s` as test name placeholder (default="TEST-%s.xml")
citrus.junit.report.suite.name	Test suite name used in the JUnit report (default="TestSuite")
citrus.junit.report.template	Template file for the JUnit report
citrus.junit.report.success.template	Template file for successful test results
citrus.junit.report.failed.template	Template file for failed test results

Table 20. Environment variables

Environment variable	Description
CITRUS_JUNIT_REPORT_ENABLED	Enables/disables JUnit report generation (default="true")
CITRUS_JUNIT_REPORT_DIRECTORY	Output directory for JUnit report files (default="junitreports")
CITRUS_JUNIT_REPORT_FILE_PATTERN	File name pattern for JUnit report files (default="TEST-%s.xml")
CITRUS_JUNIT_REPORT_SUITE_NAME	Test suite name used in the JUnit report (default="TestSuite")
CITRUS_JUNIT_REPORT_TEMPLATE	Template file for the JUnit report
CITRUS_JUNIT_REPORT_SUCCESS_TEMPLATE	Template file for successful test results
CITRUS_JUNIT_REPORT_FAILED_TEMPLATE	Template file for failed test results

48.4.2. Summary report settings

The summary report provides a condensed overview of all test results.

Table 21. System properties

System property	Description
citrus.summary.report.enabled	Enables/disables summary report generation (default="true")
citrus.summary.report.file	Output file name for the summary report (default="citrus-summary.xml")
citrus.summary.report.template	Template file for the summary report

Table 22. Environment variables

Environment variable	Description
CITRUS_SUMMARY_REPORT_ENABLED	Enables/disables summary report generation (default="true")
CITRUS_SUMMARY_REPORT_FILE	Output file name for the summary report (default="citrus-summary.xml")
CITRUS_SUMMARY_REPORT_TEMPLATE	Template file for the summary report

48.4.3. Test flow report settings

The test flow reporter outputs a detailed trace of the test execution flow including all test actions and their results.

Table 23. System properties

System property	Description
citrus.test.flow.report.enabled	Enables/disables test flow report generation (default="true")
citrus.test.flow.report.output	Output format for the test flow report: `json`, `yaml`, or comma-separated combination (default="json")
citrus.test.flow.report.history	Keep flow reports of previous test runs by adding a timestamp suffix to the report file names (default="true")

Table 24. Environment variables

Environment variable	Description
CITRUS_TEST_FLOW_REPORT_ENABLED	Enables/disables test flow report generation (default="true")
CITRUS_TEST_FLOW_REPORT_OUTPUT	Output format for the test flow report: `json`, `yaml`, or comma-separated combination (default="json")
CITRUS_TEST_FLOW_REPORT_HISTORY	Keep flow reports of previous test runs by adding a timestamp suffix to the report file names (default="true")

With history preservation enabled, each report generation writes its flow report files with a timestamp suffix (for example `MyTest-flow-1729266600.json`) directly into the report directory, so reports of previous test runs are preserved. Set `citrus.test.flow.report.history` to `false` to write report files without the timestamp suffix like before.

Chapter 49. Configuration options

You have several options to customize your Citrus project. Citrus uses default settings that can be overwritten to some extent. As a framework Citrus internally works with components organized in a central context (e.g. the Spring application context). Citrus registers components in the context in order to share those with the test execution runtime. You can customize the behavior of these components over environment variables and system properties.

49.1. Environment settings

Citrus as an application reads general settings from system properties and environment variables. The Citrus framework settings initialize on the startup and evaluate environment settings in favor of using default values.

The environment settings are well suited for both usual Java runtime environment and containerized runtime environments such as Docker or Kubernetes. The following settings do support this kind of environment configuration.

Table 25. System properties

System properties	Description
citrus.application.properties	File location for application property file that holds other settings. These properties get loaded as system properties on startup. (default="classpath:citrus-application.properties")
citrus.java.config	Class name for custom Java configuration (default=null)
citrus.file.encoding	Default file encoding used in Citrus when reading and writing file content (default=Charset.defaultCharset())
citrus.default.message.type	Default message type for validating payloads (default="XML")
citrus.test.name.variable	Default test name variable that is automatically created for each test (default="citrus.test.name")
citrus.test.package.variable	Default test package variable that is automatically created for each test (default="citrus.test.package")
citrus.default.src.directory	Default test source directory (default="src/test/")
citrus.xml.file.name.pattern	File name patterns used for XML test file package scan (default="/**/*Test.xml,/**/*IT.xml")

System properties	Description
citrus.java.file.name.pattern	File name patterns used for Java test sources package scan (default="/**/*Test.java,/**/*IT.java")
citrus.groovy.file.name.pattern	File name patterns used for Groovy test file package scan (default=" **.citrus.groovy,**.citrus.test.groovy,...")
citrus.yaml.file.name.pattern	File name patterns used for YAML test file package scan (default=" **.citrus.yaml,**.citrus.test.yaml,...")
citrus.message.trace.directory	Directory path for message trace output files (default="target/citrus-logs/trace/messages")
citrus.type.converter	Default type converter to use (default="default")
citrus.perform.default.validation	Enable/disable fallback to default text equals validation when no other validator matches (default="false")
citrus.custom.validator.strategy	Strategy for custom validators: EXCLUSIVE or COMPLEMENTARY (default="EXCLUSIVE")
citrus.message.cache.input.stream	Enable/disable caching of message input streams (default="true")
citrus.message.pretty.print	Enable/disable pretty print formatting for message payloads in logs (default="true")
citrus.file.path.charset.parameter	Separator used for charset parameter in file paths (default="; charset=")
citrus.resources.workdir	Working directory for loading file resources (default="")
citrus.groovy.static.imports	Comma-separated list of static imports for Groovy test action scripts (default="print,sleep")
citrus.default.logging.reporter.print-stack-traces	Enable/disable stack trace output in the default logging reporter (default="false")
citrus.dynamic.endpoints.auto.close	Enable/disable automatic closing of dynamic endpoints after the test (default="false")
citrus.dynamic.endpoints.auto.remove	Enable/disable automatic closing and removal of dynamic endpoints after the test (default="false")
citrus.print.banner	Enable/disable the Citrus banner output at the beginning of the test suite (default="true")
citrus.allow.function.override	Allow overriding of default functions with custom implementations using the same name (default="true")

System properties	Description
citrus.allow.validation.matcher.override	Allow overriding of default validation matchers with custom implementations using the same name (default="true")
citrus.validation.outbound.schema.enabled	Enable/disable outbound schema validation (default="true")
citrus.validation.outbound.json.schema.enabled	Enable/disable outbound JSON schema validation (default="true")
citrus.validation.outbound.xml.schema.enabled	Enable/disable outbound XML schema validation (default="true")
citrus.http.message.builder.force.citrus.header.update.enabled	Enable/disable forced creation of unique Citrus message IDs in the HTTP message builder (default="true")

Same properties are settable via environment variables.

Table 26. Environment variables

Environment variable	Description
CITRUS_APPLICATION_PROPERTIES	File location for application property file that holds other settings. These properties get loaded as system properties on startup. (default="classpath:citrus-application.properties")
CITRUS_JAVA_CONFIG	Class name for custom Java configuration (default=null)
CITRUS_FILE_ENCODING	Default file encoding used in Citrus when reading and writing file content (default=Charset.defaultCharset())
CITRUS_DEFAULT_MESSAGE_TYPE	Default message type for validating payloads (default="XML")
CITRUS_TEST_NAME_VARIABLE	Default test name variable that is automatically created for each test (default="citrus.test.name")
CITRUS_TEST_PACKAGE_VARIABLE	Default test package variable that is automatically created for each test (default="citrus.test.package")
CITRUS_DEFAULT_SRC_DIRECTORY	Default test source directory (default="src/test/")
CITRUS_XML_FILE_NAME_PATTERN	File name patterns used for XML test file package scan (default="/**/*Test.xml,/**/*IT.xml")
CITRUS_JAVA_FILE_NAME_PATTERN	File name patterns used for Java test sources package scan (default="/**/*Test.java,/**/*IT.java")

Environment variable	Description
CITRUS_GROOVY_FILE_NAME_PATTERN	File name patterns used for Groovy test file package scan (default="**.citrus.groovy,**.citrus.test.groovy,...")
CITRUS_YAML_FILE_NAME_PATTERN	File name patterns used for YAML test file package scan (default="**.citrus.yaml,**.citrus.test.yaml,...")
CITRUS_MESSAGE_TRACE_DIRECTORY	Directory path for message trace output files (default="target/citrus-logs/trace/messages")
CITRUS_TYPE_CONVERTER	Default type converter to use (default="default")
CITRUS_PERFORM_DEFAULT_VALIDATION	Enable/disable fallback to default text equals validation when no other validator matches (default="false")
CITRUS_CUSTOM_VALIDATOR_STRATEGY	Strategy for custom validators: EXCLUSIVE or COMPLEMENTARY (default="EXCLUSIVE")
CITRUS_MESSAGE_CACHE_INPUT_STREAM	Enable/disable caching of message input streams (default="true")
CITRUS_MESSAGE_PRETTY_PRINT	Enable/disable pretty print formatting for message payloads in logs (default="true")
CITRUS_FILE_PATH_CHARSET_PARAMETER	Separator used for charset parameter in file paths (default="; charset=")
CITRUS_RESOURCES_WORKDIR	Working directory for loading file resources (default="")
CITRUS_GROOVY_STATIC_IMPORTS	Comma-separated list of static imports for Groovy test action scripts (default="print,sleep")
CITRUS_DEFAULT_LOGGING_REPORTER_PRINT_STACK_TRACES	Enable/disable stack trace output in the default logging reporter (default="false")
CITRUS_DYNAMIC_ENDPOINTS_AUTO_CLOSE	Enable/disable automatic closing of dynamic endpoints after the test (default="false")
CITRUS_DYNAMIC_ENDPOINTS_AUTO_REMOVE	Enable/disable automatic closing and removal of dynamic endpoints after the test (default="false")
CITRUS_PRINT_BANNER	Enable/disable the Citrus banner output at the beginning of the test suite (default="true")
CITRUS_ALLOW_FUNCTION_OVERRIDE	Allow overriding of default functions with custom implementations using the same name (default="true")
CITRUS_ALLOW_VALIDATION_MATCHER_OVERRIDE	Allow overriding of default validation matchers with custom implementations using the same name (default="true")

Environment variable	Description
CITRUS_VALIDATION_OUTBOUND_SCHEMA_ENABLED	Enable/disable outbound schema validation (default="true")
CITRUS_VALIDATION_OUTBOUND_JSON_SCHEMA_ENABLED	Enable/disable outbound JSON schema validation (default="true")
CITRUS_VALIDATION_OUTBOUND_XML_SCHEMA_ENABLED	Enable/disable outbound XML schema validation (default="true")
CITRUS_HTTP_MESSAGE_BUILDER_FORCE_CITRUS_HEADER_UPDATE_ENABLED	Enable/disable forced creation of unique Citrus message IDs in the HTTP message builder (default="true")

49.2. Log settings

Citrus provides settings to control message content logging and sensitive data masking. The log modifier automatically masks sensitive values (such as passwords and secrets) in message payloads before they are written to the log output. These settings can be configured via system properties or environment variables.

Table 27. System properties

System property	Description
citrus.logger.modifier	Enable/disable the log modifier that masks sensitive data in message payloads (default="true")
citrus.logger.mask.keywords	Comma-separated list of keywords that identify sensitive fields to mask (default="password,secret,secretKey")
citrus.logger.mask.value	The mask replacement value used when sensitive data is detected (default="*****")
citrus.logger.mask.xml	Enable/disable log masking for XML message payloads (default="true")
citrus.logger.mask.json	Enable/disable log masking for JSON message payloads (default="true")
citrus.logger.mask.yaml	Enable/disable log masking for YAML message payloads (default="true")
citrus.logger.mask.key.value	Enable/disable log masking for key-value pair payloads (default="true")
citrus.logger.mask.form.url.encoded	Enable/disable log masking for form URL-encoded payloads (default="true")
citrus.logger.print.message.content	Enable/disable printing of message content (headers and body) in log output (default="false")

System property	Description
citrus.logger.print.inbound.message.content	Enable/disable printing of inbound message content in log output. Inherits from `citrus.logger.print.message.content` when not set.
citrus.logger.print.outbound.message.content	Enable/disable printing of outbound message content in log output. Inherits from `citrus.logger.print.message.content` when not set.
citrus.logger.print.message.layout	Layout mode for message content in log output: `verbose`, `summary`, `compact`, `body` (default="verbose")
citrus.logger.message.payload.max.length	Maximum length of message body in log output before truncation (default="2048")
citrus.logger.color	ANSI color mode for log output: `auto`, `always`, `never` (default="auto")

Table 28. Environment variables

Environment variable	Description
CITRUS_LOG_MODIFIER	Enable/disable the log modifier that masks sensitive data in message payloads (default="true")
CITRUS_LOG_MASK_KEYWORDS	Comma-separated list of keywords that identify sensitive fields to mask (default="password,secret,secretKey")
CITRUS_LOG_MASK_VALUE	The mask replacement value used when sensitive data is detected (default="*****")
CITRUS_LOG_MASK_XML	Enable/disable log masking for XML message payloads (default="true")
CITRUS_LOG_MASK_JSON	Enable/disable log masking for JSON message payloads (default="true")
CITRUS_LOG_MASK_YAML	Enable/disable log masking for YAML message payloads (default="true")
CITRUS_LOG_MASK_KEY_VALUE	Enable/disable log masking for key-value pair payloads (default="true")
CITRUS_LOG_MASK_FORM_URL_ENCODED	Enable/disable log masking for form URL-encoded payloads (default="true")
CITRUS_LOG_PRINT_MESSAGE_CONTENT	Enable/disable printing of message content (headers and body) in log output (default="false")

Environment variable	Description
CITRUS_LOG_PRINT_INBOUND_MESSAGE_CONTENT	Enable/disable printing of inbound message content in log output. Inherits from `CITRUS_LOG_PRINT_MESSAGE_CONTENT` when not set.
CITRUS_LOG_PRINT_OUTBOUND_MESSAGE_CONTENT	Enable/disable printing of outbound message content in log output. Inherits from `CITRUS_LOG_PRINT_MESSAGE_CONTENT` when not set.
CITRUS_LOG_PRINT_MESSAGE_LAYOUT	Layout mode for message content in log output: `verbose`, `summary`, `compact`, `body` (default="verbose")
CITRUS_LOG_MESSAGE_PAYLOAD_MAX_LENGTH	Maximum length of message body in log output before truncation (default="2048")
CITRUS_LOG_COLOR	ANSI color mode for log output: `auto`, `always`, `never` (default="auto")

49.3. Property binding support

The Citrus project defines components and endpoints in the form of beans in the Citrus registry, which is either the Java configuration or a Spring bean configuration. The beans in the registry provide properties that you can overwrite with environment variables and system properties.

Each bean that is added to the Citrus registry is being post processed with the Citrus property binding support. Users can overwrite these properties on components, endpoints and endpoint configuration with environment specific settings.

The Citrus property binding support follows a naming convention that looks like follows:

Environment variables

```
CITRUS_COMPONENT_<COMPONENT_NAME>_<PROPERTY_NAME>=<VALUE>
```

Also, specific to endpoints the naming convention looks like this:

Environment variables

```
CITRUS_ENDPOINT_<ENDPOINT_NAME>_<PROPERTY_NAME>=<VALUE>
CITRUS_ENDPOINT_CONFIG_<ENDPOINT_NAME>_<PROPERTY_NAME>=<VALUE>
```

According to that users can also use system property settings that follow the same rules:

System properties

```
citrus.component.<component-name>.<property-name>=<value>
citrus.endpoint.<endpoint-name>.<property-name>=<value>
citrus.endpoint.config.<endpoint-name>.<property-name>=<value>
```

The difference of using `CITRUS_ENDPOINT` compared to using `CITRUS_ENDPOINT_CONFIG` is that the latter targets the specific endpoint configuration of the bean. The former will just set properties on the endpoint itself.

The component or endpoint name references the bean name that is used to store the object into the Citrus registry. The property name is the name of the setter method available on the object (without the `set` prefix)

Given an endpoint named `foo` with a `setPort(int port)` method the environment setting to overwrite the property would be like this:

Environment variable

```
CITRUS_ENDPOINT_FOO_PORT=12345
```

System property

```
citrus.endpoint.foo.port=12345
```

When the bean name or the property name uses camel case style you have multiple ways to represent this in the environment settings.

As an example the given endpoint named `myBean` exposes the method `setRequestUrl(String url)`. You have multiple ways to set this property:

Environment variable

```
CITRUS_ENDPOINT_MYBEAN_REQUESTURL=http://whatever

CITRUS_ENDPOINT_MY_BEAN_REQUEST_URL=http://whatever
```

System properties

```
citrus.endpoint.myBean.requestUrl=http://whatever

citrus.endpoint.myBean.request-url=http://whatever
```

The property bindings may reference other beans in the Citrus registry. Given a bean named `myLog` of type `org.citrusframework.LogModifier`, that is stored to the Citrus registry, you can be reference the bean with the `#bean:` prefix like this:

```
CITRUS_ENDPOINT_FOO_LOG_MODIFIER=#bean:myLog
```

This will resolve the bean named `myLog` from the Citrus registry and set this as a reference on the endpoint `foo` with the `setLogModifier()` method.

The property binding support is enabled by default for components, endpoints and endpoint

configuration. You can disable the bindings with the environment setting:

Table 29. System properties

System properties	Description
citrus.env.var.property.binding.enabled	Set this to `false` to disable the property binding completely.
citrus.component.property.binding.enabled	Set this to `false` to disable the property binding for components.
citrus.endpoint.property.binding.enabled	Set this to `false` to disable the property binding for endpoints.

Table 30. Environment variables

Environment variable	Description
CITRUS_ENV_VAR_PROPERTY_BINDING_ENABLED	Set this to `false` to disable the property binding completely.
CITRUS_COMPONENT_PROPERTY_BINDING_ENABLED	Set this to `false` to disable the property binding for components.
CITRUS_ENDPOINT_PROPERTY_BINDING_ENABLED	Set this to `false` to disable the property binding for endpoints.

49.4. Spring configuration settings

When spring framework is enabled in Citrus you can set specific settings regarding the Spring application context.

Table 31. System properties

System properties	Description
citrus.spring.application.context	File location for Spring XML configurations (default="classpath*:citrus-context.xml")
citrus.spring.java.config	Class name for Spring Java config (default=null)

Table 32. Environment variables

Environment variable	Description
CITRUS_SPRING_APPLICATION_CONTEXT	File location for Spring XML configurations (default="classpath*:citrus-context.xml")
CITRUS_SPRING_JAVA_CONFIG	Class name for Spring Java config (default=null)

49.5. Property file settings

As mentioned in the previous section Citrus as a framework references some basic settings from system environment properties or variables. You can overwrite these settings in a central property file which is loaded at the very beginning of the Citrus runtime.

The properties in that file are automatically loaded as Java system properties. Just add a property file named **citrus-application.properties** to your project classpath. This property file contains customized settings such as:

citrus-application.properties

```
citrus.spring.application.context=classpath*:citrus-custom-context.xml
citrus.spring.java.config=org.citrusframework.config.MyCustomConfig
citrus.file.encoding=UTF-8
citrus.default.message.type=XML
citrus.xml.file.name.pattern=/**/*Test.xml,/**/*IT.xml
```

Citrus automatically loads these application properties at the startup. All properties are also settable with Java system properties directly. The location of the `citrus-application.properties` file is customizable with the system property `citrus.application.properties` or environment variable `CITRUS_APPLICATION_PROPERTIES`. .Custom property file location.

```
CITRUS_APPLICATION_PROPERTIES=file:/custom/path/to/citrus-application.properties
```



You can use **classpath:** and **file:** path prefix in order to give locate a classpath or file-system resource.

Chapter 50. Spring support

The Spring framework provides an awesome set of projects, libraries and tools and is a wide spread and well appreciated framework for Java. The dependency injection and IoC concepts introduced with Spring are awesome.

Citrus is able to work with Spring in terms of loading central components as Spring beans as part of a Spring application context. In the following you can use Spring autowiring and configuration in your tests.

Read the following chapters to know more about how to use Citrus together with Spring.

50.1. Spring XML application context

Citrus supports the Spring framework as IoC container in order to load all components as Spring beans in a central application context. By default, Citrus loads basic components as Spring beans in a Spring Java config class.

With Spring in place it is very easy to change/add custom bean components in the Spring application context. Citrus searches for custom Spring application context files in your project and adds these bean definitions to the Spring application context.

By default, Citrus looks for custom XML Spring application context files in this location: `classpath*:citrus-context.xml`. So you can just add this file named **citrus-context.xml** to your project classpath and Citrus will load all Spring beans automatically.

The location of this file can be customized by setting a System property `citrus.spring.application.context` or the environment variable `CITRUS_SPRING_APPLICATION_CONTEXT`.

Custom Spring XML bean definition context.

```
CITRUS_SPRING_APPLICATION_CONTEXT=file:/custom/path/to/custom-beans.xml
```

custom-beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.citrusframework.org/schema/config
            http://www.citrusframework.org/schema/config/citrus-config.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context.xsd">

  <!-- Add your bean definitions here -->

</beans>
```

The file `custom-beans.xml` should provide a normal Spring bean XML configuration. You can add Spring beans as usual and you can use the Citrus XML components provided by the schemas like `xmlns:citrus="http://www.citrusframework.org/schema/config"`.

Citrus provides several schemas for custom Spring XML components. These are described in more detail in the respective chapters and sections in this reference guide.



You can also use import statements in this Spring application context in order to load other configuration files. So you are free to modularize your configuration in several files that get loaded by Citrus.

50.2. Spring Java config

You can also use pure Java code to load Spring beans as a configuration. Citrus is able to load the Spring beans from a configuration class. Please define the configuration class with the System property `citrus.spring.java.config` or with the environment variable `CITRUS_SPRING_JAVA_CONFIG`.

Custom Spring Java configuration class.

```
CITRUS_SPRING_JAVA_CONFIG=custom.package.to.MyCustomConfig
```

Citrus loads the given Spring bean configuration class in **MyCustomConfig.class** and adds all defined Spring beans to the application context. See the following example for custom Spring Java configuration:

MyCustomConfig.java

```
import org.citrusframework.TestCase;
import org.citrusframework.report.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MyCustomConfig {

    @Bean(name = "plusMinusTestReporter")
    public TestReporter plusMinusTestReporter() {
        return new PlusMinusTestReporter();
    }

    /**
     * Sample test reporter.
     */
    private static class PlusMinusTestReporter extends AbstractTestReporter {

        @Override
        public void generate(TestResults testResults) {
            StringBuilder testReport = new StringBuilder();

            testResults.doWithResults(result -> {
                if (result.isSuccess()) {
                    testReport.append("+");
                } else if (result.isFailed()) {
                    testReport.append("-");
                } else {
                    testReport.append("o");
                }
            });


LoggerFactory.getLogger(PlusMinusTestReporter.class).info(testReport.toString());
        }
    }
}
```



You can mix XML and Java based Spring bean configuration. Citrus loads both sources and adds beans to the Spring bean application context during start.

Chapter 51. Service Provider Interface (SPI)

Citrus uses a Service Provider Interface (SPI) mechanism to discover and load component implementations at runtime. This allows Citrus modules to be decoupled from each other and enables users to add custom implementations simply by placing them on the classpath.

51.1. Resource path lookup

The primary SPI mechanism in Citrus is the **resource path lookup**. Each extensible component type defines a resource path under `META-INF/citrus/` where implementations are registered. When Citrus starts, it scans these paths on the classpath to discover available implementations.

A resource file is a simple Java properties file that contains at minimum a `type` property pointing to the fully qualified class name of the implementation:

```
type=org.example.MyCustomValidator
```

Some resource files also include a `name` property:

```
name=myCustomValidator
type=org.example.MyCustomValidator
```

The following table lists the component types that support resource path lookup:

Type	Resource Path
`CitrusContextProvider`	`META-INF/citrus/context/provider`
`TestActionBuilder`	`META-INF/citrus/action/builder`
`TestLoader`	`META-INF/citrus/test/loader`
`TestEngine`	`META-INF/citrus/engine`
`EndpointComponent`	`META-INF/citrus/endpoint/component`
`EndpointBuilder`	`META-INF/citrus/endpoint/builder`
`AnnotationConfigParser`	`META-INF/citrus/endpoint/parser`
`MessageValidator`	`META-INF/citrus/message/validator`
`SchemaValidator`	`META-INF/citrus/message/schemaValidator`
`HeaderValidator`	`META-INF/citrus/header/validator`
`ValueMatcher`	`META-INF/citrus/value/matcher`
`ValidationMatcher`	`META-INF/citrus/validation/matcher`
`ValidationContext`	`META-INF/citrus/validation/builder`
`MessageProcessor`	`META-INF/citrus/message/processor`
`MessageSelector`	`META-INF/citrus/message/selector`
`MessagePayloadBuilder`	`META-INF/citrus/message/builder`
`ScriptPayloadBuilder`	`META-INF/citrus/script/message/builder`

Type	Resource Path
VariableExtractor	META-INF/citrus/variable/extractor
TypeConverter	META-INF/citrus/type/converter
TypeAliasResolver	META-INF/citrus/context/resolver
Function	META-INF/citrus/function
SqlResultSetScriptValidator	META-INF/citrus/sql/result-set/validator
HamcrestMatcherProvider	META-INF/citrus/hamcrest/matcher/provider

51.1.1. Adding a custom implementation via resource path

To register a custom implementation, create a properties file at the appropriate resource path on your classpath. For example, to add a custom message validator named `myValidator`:

1. Create the file `META-INF/citrus/message/validator/myValidator` in your `src/main/resources` directory.
2. Add the following content:

```
name=myValidator
type=com.example.MyCustomMessageValidator
```

3. Ensure your implementation class is available on the classpath.

Citrus will automatically discover and load the implementation when it starts.

51.2. Java ServiceLoader fallback

In addition to the resource path lookup, Citrus supports standard Java `ServiceLoader` as a fallback mechanism. This allows implementations to be registered using the standard `META-INF/services/` convention defined by the Java platform.

The `ServiceLoader` is used as a **fallback** — the resource path lookup always takes precedence. If a component cannot be found via resource path lookup, Citrus will attempt to discover it via `ServiceLoader`. When both mechanisms find implementations, resource path results take priority and `ServiceLoader` results fill in any gaps.

51.2.1. The @NamedService annotation

Implementations discovered via `ServiceLoader` must be annotated with `@NamedService` to provide a name for lookup. The `name` attribute serves the same purpose as the resource file name in the resource path lookup — it identifies the implementation so it can be looked up by name.

```
import org.citrusframework.spi.NamedService;
import org.citrusframework.validation.MessageValidator;

@NamedService(name = "myCustomValidator")
public class MyCustomMessageValidator implements MessageValidator<ValidationContext> {
    // implementation
}
```

51.2.2. Registering via ServiceLoader

To register a component using the standard Java `ServiceLoader` mechanism:

1. Annotate your implementation with `@NamedService`:

```
import org.citrusframework.spi.NamedService;

@NamedService(name = "custom")
public class CustomEndpointComponent implements EndpointComponent {
    // implementation
}
```

2. Create the service provider file `META-INF/services/org.citrusframework.endpoint.EndpointComponent` in your `src/main/resources` directory:

```
com.example.CustomEndpointComponent
```

Citrus will discover the implementation via `ServiceLoader` when the resource path lookup does not find it.

51.2.3. Lookup precedence

When both resource path files and `ServiceLoader` implementations exist for the same component type, Citrus follows this precedence:

1. **Single lookup** (`lookup(name)`): The resource path is tried first. Only if it fails to find the named implementation does Citrus fall back to `ServiceLoader`.
2. **Bulk lookup** (`lookup()`): Results from both mechanisms are merged. Resource path results take precedence — a `ServiceLoader` implementation is only added if no resource path result exists with the same name.

This ensures full backward compatibility while allowing new implementations to be registered using either mechanism.

51.2.4. Choosing the right mechanism

Both mechanisms are fully supported. Choose based on your needs:

Criteria	Resource path lookup	Java ServiceLoader
Standard Java convention	No	Yes
Requires `@NamedService` annotation	No	Yes
Requires service provider file	No (uses custom properties file)	Yes (`META-INF/services/`)
Constructor arguments support	Yes	No
IDE/tooling support	Limited	Better (standard SPI)

For most use cases, the **resource path lookup** remains the recommended approach as it supports constructor arguments and is the established pattern in Citrus. The **ServiceLoader** fallback is useful when you prefer standard Java conventions or when integrating with frameworks that already use `ServiceLoader`.

Chapter 52. Tools

In the following sections we have a closer into various tooling available for Citrus.

Chapter 53. Cucumber Steps

Citrus is able to use the Cucumber runtime to execute BDD feature files as tests. Read more about the section about Cucumber integration with Citrus.

Cucumber feature files follow the Gherkin Given-When-Then syntax where each line in a feature file is backed by a step implementation. The step implementation represents the actual runtime logic executed with the feature step.

Citrus provides a set of default ready-to-use step implementations that you can just add to your feature file. The predefined steps leverage Citrus endpoints and test actions to perform integration testing with proper message exchange and validation.



If you want to use the predefined Citrus step implementations, please make sure to include the extra glue package as follows:

MyFeatureIT.java

```
@RunWith(Cucumber.class)
@CucumberOptions(
    extraGlue = { "org.citrusframework.cucumber" },
    plugin = { "pretty", "org.citrusframework.cucumber.CitrusReporter" } )
public class MyFeatureIT {

}
```

As an alternative you may declare the glue package in `cucumber.properties` in your project.

cucumber.properties

```
cucumber.object-factory=org.citrusframework.cucumber.backend.CitrusObjectFactory
cucumber.glue=org.citrusframework.cucumber
cucumber.plugin=pretty,org.citrusframework.cucumber.CitrusReporter
```



The Citrus step implementations require a special Cucumber object factory `org.citrusframework.cucumber.backend.CitrusObjectFactory`.

See the following step implementations that enable you to cover various areas of messaging and integration testing.

53.1. Standard steps

The standard steps in Citrus provide a lot of basic functionality that you can just use in your feature files. The functionality is shipped as predefined steps that you add to a feature as you write your test.

Most of the standard steps do leverage capabilities of the underlying test framework Citrus such as

creating test variables or printing messages to the log output.

53.1.1. Create variables

Test variables represent the fundamental concept to own test data throughout your test. Once a variable has been created you can reference its value in many places in Citrus. You can add a new identifier as a variable and reference its value in many places such as message headers, body content, SQL statements and many more.

@Given("^variable {name} is \"{value}\"$")

```
Given variable orderId is "1001"
```

This will create the variable `orderId` in the current test context. All subsequent steps and operations may reference the variable with the expression `${orderId}`. Citrus makes sure to replace the variable placeholder with its actual value before sending out messages and before validating incoming messages. As already mentioned you can use the variable placeholder expression in many places such as message headers and body content:

Variable placeholder in a Json payload

```
{
  "id": "${orderId}",
  "name": "Watermelon",
  "amount": 10
}
```

You can create multiple variables in one single step using:

@Given("^variables$")

```
Given variables
  | orderId  | 1001      |
  | name     | Pineapple |
```

You can also load variables from an external property file resource.

@Given("^load variables {file}$")

```
Given load variables {file}
```

The given `{file}` should be a property file holding one to many test variables with key and value.

variable.properties

```
greeting=Hola
name=Christoph
text=Citrus rocks!
```

Sometimes variables values are too big to write them directly into the feature file (e.g. large request/response body data). In this case you may want to load the variable value from an external file.

@Given("^load variable {name} from {file}$")

```
Given load variable {name} from {file}
```

The step loads the file content as a String value and references it as a new test variable with the given name.

Load test variable from file

```
Given load variable body from request.json
Then log 'Sending request body: ${body}'
```

53.1.2. Log steps

Logging a message to the output can be helpful in terms of debugging and/or to give information about the context of an operation.

Citrus provides following steps to add log output:

@Then("^(?:log|print) '{text}'$")

```
Then print 'Citrus provides Cloud native BDD testing!'
And log 'Citrus rocks!'
```

The steps are printing log messages to the output using INFO level. The text that is printed supports test variables and functions. All placeholders will be replaced before logging.

You can also use multiline log messages as shown in the next example.

@Then("^(?:log|print)$")

```
Given print
"""
Hello users!

Citrus provides Cloud Native BDD testing on Kubernetes!
"""
```

53.1.3. Sleep

The `sleep` step lets the test run wait for a given amount of time (in milliseconds). During the sleep no action will be performed and the subsequent steps are postponed respectively.

@Then("^sleep$")

```
Then sleep
```

The above step performs a sleep with the default time of 5000 milliseconds.

You can also provide the duration to sleep in milliseconds or ISO 8601 format.

@Then("^sleep {time} ms$")

```
Then sleep 2500 ms
```

The step receives a numeric parameter that represents the amount of time (in milliseconds) to wait.

@Then("^sleep {hours}h {minutes}min {seconds}sec {milliseconds}ms$")

```
Then sleep 1h 25min 15sec 500ms
```

53.2. Apache Camel steps

Apache Camel is a very popular enterprise integration library that provides a huge set of ready to use components and endpoints for you to connect with different messaging transports. Also, many data formats are supported in Camel so you will be able to incorporate with almost any software interface exchanging data over the wire.

Citrus adds steps to use Apache Camel as part of a test. You are able to send and receive messages with Camel components and make use of the enterprise integration patterns and data formats implemented in Apache Camel.

53.2.1. Create Camel context

The Camel context is a central place to add routes and manage Camel capabilities and services. You can start a new default (empty) Camel context using the following step.

@Given("^(?:Default|New) Camel context$")

```
Given Default Camel context
```

This will setup and start a new Camel context as part og the current test scenario. You can now create and add routes to this context. In case you have special configuration and/or some default routes that you need to initialize as part of the context you can provide a Camel Spring bean configuration in the Camel context step.

@Given("^New Spring Camel context$")

```
Given New Spring Camel context
"""
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
                          http://camel.apache.org/schema/spring
http://camel.apache.org/schema/spring/camel-spring.xsd">
  <camelContext id="helloContext" xmlns="http://camel.apache.org/schema/spring">
    <route id="helloRoute">
      <from uri="direct:hello"/>
      <to uri="log:org.citrusframework.camel?level=INFO"/>
      <split>
        <tokenize token=" "/>
        <to uri="seda:tokens"/>
      </split>
    </route>
  </camelContext>
</beans>
"""
```

53.2.2. Create Camel routes

In Apache Camel you need to create routes in order to start producing/consuming data from endpoints. The routes can be defined in XML or Groovy.

@Given("^Camel route {name}.xml")

```
Given Camel route hello.xml
"""
<route>
  <from uri="direct:hello"/>
  <to uri="log:org.citrusframework.camel?level=INFO"/>
  <split>
    <tokenize token=" "/>
    <to uri="seda:tokens"/>
  </split>
</route>
"""
```

In addition to XML route definitions Citrus also supports the Groovy DSL.

@Given("^Camel route {name}.groovy")

```
Given Camel route hello.groovy
"""
from("direct:hello")
 .to("log:org.citrusframework.camel?level=${logLevel}")
 .split(body().tokenize(" "))
   .to("seda:tokens")
 .end()
"""
```

The above steps create the Camel routes and automatically starts them in the current context. The given routes start to consume messages from the endpoint `direct:hello`.

You can also load the Camel route from a file resource.

@Given("^load Camel route path/to/{name}.{language}")

```
Given load Camel route hello.groovy
```

Currently, the languages `xml` and `groovy` are supported.

53.2.3. Start/stop Camel routes

We are able to explicitly start and stop routes in the current context.

@Then("^start Camel route {name}$")

```
Then start Camel route {name}
```

The given name references a route in the current Camel context. This starts the route and consumes messages from the enpoint URI.

@Then("^stop Camel route {name}$")

```
Then stop Camel route {name}
```

After stopping a route the route will not consume any messages on the given endpoint URI.

In case a Camel route is not needed anymore you can also remove it from the current Camel context.

@Then("^remove Camel route {name}$")

```
Then remove Camel route {name}
```

53.2.4. Send messages via Camel

We can send exchanges using any Camel endpoint URI. The endpoint URI can point to an external component or to a route in the current Camel context and trigger its processing logic. The exchange body is given as single or multiline body content.

@When("^send Camel exchange to\|(\"{endpoint_uri}\"\|) with body: {body}$")

```
When send Camel exchange to("direct:hello") with body: Hello Camel!
```

The step sends an exchange with the body `Hello Camel!`. You can also use multiline body content with the following step:

@When("^send Camel exchange to\|(\"{endpoint_uri}\"\|) with body$")

```
When send Camel exchange to("direct:hello") with body
"""
Hello Camel!

This is a multiline content!
"""
```

In addition to a body content the Camel exchange also defines a set of message headers. You can use a data table to specify message headers when sending a message.

@When("^send Camel exchange to\|(\"{endpoint_uri}\"\|) with body and headers: {body}$")

```
When send Camel exchange to("direct:hello") with body and headers Hello Camel!
    | id        | 1234     |
    | operation | sayHello |
```

53.2.5. Receive messages via Camel

The Citrus test is able to receive messages from a Camel endpoint URI in order to verify the message content (header and body) with an expected control message.

Once the message is received Citrus makes use of the powerful message validation capabilities of Citrus to make sure that the content is as expected.

@When("^receive Camel exchange from\|(\"{endpoint_uri}\"\|) with body: {body}$")

```
When receive Camel exchange from("seda:tokens") with body: Hello
```

The step receives an exchange from the endpoint URI `seda:tokens` and verifies the body to be equal to `Hello`. See the next example on how to validate a multiline message body content.

@When("^receive Camel exchange from\|(\"{endpoint_uri}\"\|) with body$")

```
When receive Camel exchange from("seda:tokens") with body
"""
{
  "message": "Hello Camel!"
}
"""
```

We can also verify a set of message headers that must be present on the received exchange. Once again we use a data table to define the message headers. This time we provide expected message header values.

@When("^receive Camel exchange from\|(\"{endpoint_uri}\"\|) with body and headers: {body}$")

```
When receive Camel exchange from("seda:tokens") with body and headers: Hello
    | id        | 1234     |
    | operation | sayHello |
```

53.2.6. Define Camel exchanges

In the previous steps we have seen how to send and receive messages to anf from Camel endpoint URIs. We have used the exchange body and header in a single step so far.

In some cases it might be a better option to use multiple steps for defining the complete exchange data upfront. The actual send/receive operation then takes place in a separate step.

The following examples should clarify the usage.

@Given("^Camel exchange message header {name}=\"{value}\"$")

```
Camel exchange message header {name}="{value}"
```

This sets a message header on the exchange. We can also use a data table to set multiple headers in one single step:

@Given("^Camel exchange message headers$")

```
Camel exchange message headers
    | id        | 1234     |
    | operation | sayHello |
```

Then we can also set the body in another step.

@Given("^Camel exchange body$")

```
Camel exchange body: Hello Camel!
```

Multiline body content is also supported.

@Given("^Camel exchange body$")

```
Camel exchange body
"""
{
  "message": "Hello Camel!"
}
"""
```

When the body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load Camel exchange body {file}$")

```
Given load Camel exchange body {file}
```

This step loads the body content from the given file resource.

Now that we have specified the exchange headers and body content we can send or receive that specific echange in a separate step.

@When("^send Camel exchange to\|(\"{endpoint_uri}\"\|)$")

```
send Camel exchange to("{endpoint_uri}")
```

@When("^receive Camel exchange from\|(\"{endpoint_uri}\"\|)$")

```
receive Camel exchange from("{endpoint_uri}")
```

In the previous section we have covered a 2nd approach to send and receive messages with Apache Camel. You specify the exchange in multiple steps first and then send/receive the exchange to/from and endpoint URI in a separate step.

53.2.7. Basic Camel settings

@Given("^Camel consumer timeout is {time}(?: ms\| milliseconds)$")

```
Given Camel consumer timeout is {time} milliseconds
```

Sets the default timeout for all Camel components that consume data from messaging transports. After that time the test will fail with a timeout exception when no message has been received.

53.2.8. Manage Camel resources

The Apache Camel steps are able to create resources such as routes. By default these resources get removed automatically after the test scenario.

The auto removal of Camel resources can be turned off with the following step.

@Given("^Disable auto removal of Camel resources$")

```
Given Disable auto removal of Camel resources
```

Usually this step is a `Background` step for all scenarios in a feature file. This way multiple scenarios can work on the very same Camel resources and share integrations.

There is also a separate step to explicitly enable the auto removal.

@Given("^Enable auto removal of Camel resources$")

```
Given Enable auto removal of Camel resources
```

By default, all Camel resources are automatically removed after each scenario.

53.3. Groovy steps

The Groovy support in Citrus adds ways to configure the framework with bean configurations and test actions via Groovy script snippets. In particular, you can add customized endpoints that send/receive data over various messaging transports.

The Groovy script support is there to cover specific use cases where you hit limitations with the predefined conventional BDD steps. Of course the syntax and handling of the Groovy scripts are less human-readable and more developer-coherent.

53.3.1. Bean configuration

Many steps use Citrus components behind the scenes. The Citrus components are configurable through a Groovy domain specific language. You can add endpoints and other components as Citrus framework configuration like follows:

@Given("^(?:create|new) configuration$")

```
Given create configuration
"""
<<Groovy DSL>>
"""
```

In the next example the step uses a Groovy domain specific language to define a new Http server endpoint.

Http server endpoint configuration

```
Scenario: Endpoint script config
  Given URL: http://localhost:18080
  Given create configuration
  """
  citrus {
      endpoints {
          http {
              server('helloServer') {
                  port = 18080
                  autoStart = true
              }
          }
      }
  }
  """
  When send GET /hello
  Then receive HTTP 200 OK
```

The configuration step creates a new Citrus endpoint named `helloServer` with given properties (`port`, `autoStart`) in form of a Groovy configuration script. The endpoint is a Http server Citrus component that is automatically started listening on the given port. In the following the scenario can send messages to that server endpoint.

The Groovy configuration script adds Citrus components to the test context and supports following elements:

- `endpoints`: Configure Citrus endpoint components that can be used to exchange data over various messaging transports
- `queues`: In memory queues to handle message forwarding for incoming messages
- `beans`: Custom beans configuration (e.g. data source, SSL context, request factory) that can be used in Citrus endpoint components

Let's quickly have a look at a bean configuration where a new JDBC data source is added to the test suite.

JDBC data source configuration

```
Scenario: Bean configuration
  Given create configuration
  """
  citrus {
      beans {
          dataSource(org.apache.commons.dbcp2.BasicDataSource) {
              driverClassName = "org.h2.Driver"
              url = "jdbc:h2:mem:camel"
              username = "sa"
              password = ""
          }
      }
  }
  """
```

The data source will be added as a bean named `dataSource` and can be referenced in all Citrus SQL test actions.

All Groovy configuration scripts that we have seen so far can also be loaded from external file resources, too.

@Given("^load configuration {file_path}\|.groovy$")

```
Given load configuration {file_path}.groovy
```

The file content is loaded as a Groovy configuration DSL. The next code sample shows such a configuration script.

citrus.configuration.groovy

```
citrus {
    queues {
        queue('say-hello')
    }

    endpoints {
        direct {
            asynchronous {
                name = 'hello'
                queue = 'say-hello'
            }
        }
    }
}
```

53.3.2. Endpoint configuration

Endpoints describe an essential part in terms of messaging integration during a test. There are multiple ways to add custom endpoints to a test. Endpoint Groovy scripts is one comfortable way to add custom endpoint configurations in a test scenario. You can do so with the following step.

@Given("^(?:create|new) endpoint {name}\|\.groovy$")

```
Given("^(?:create|new) endpoint {name}.groovy
"""
<<Groovy DSL>>
"""
```

The step receives a unique name for the endpoint and a Groovy DSL that sepcifies the endpoint component with all its properties. In the following sample a new Http server endpoint component will be created.

Create new Http server endpoint

```
Scenario: Create Http endpoint
  Given URL: http://localhost:18081
  Given create endpoint helloServer.groovy
  """
  http()
    .server()
    .port(18081)
    .autoStart(true)
  """
  When send GET /hello
  Then receive HTTP 200 OK
```

The scenario creates a new Http server endpoint named `helloServer`. This server component can be used directly in the scenario to receive and verify messages sent to that endpoint.

You can also load the endpoint configuration from an external file resources.

@Given("^load endpoint {file_path}\|\.groovy$")

```
Given("^load endpoint {file_path}.groovy$")
```

The referenced file should contain the endpoint Groovy DSL.

Create endpoint from file resource

```
Scenario: Load endpoint
  Given URL: http://localhost:18088
  Given load endpoint fooServer.groovy
  When send GET /hello
  Then receive HTTP 200 OK
```

fooServer.groovy

```
http()
    .server()
    .port(18088)
    .autoStart(true)
```

53.3.3. Test actions

Citrus provides a huge set of predefined test actions that users can add to the Gherkin feature files out of the box. However, there might be situations where you want to run a customized test action code as a step in your feature scenario.

With the Groovy script support in Citrus you can add such customized test actions via script snippets:

@Given("^(?:create|new) actions {name}||.groovy$")

```
Given create actions {name}.groovy$")
"""
<<Groovy DSL>>
"""
```

The Groovy test action DSL script receives a unique `{name}`. You can reference this name later in the test in order to apply the defined actions. When applied to the test the defined actions are executed. A sample will show how it is done.

Create test actions with a script

```
Scenario: Custom test actions
  Given create actions basic.groovy
  """
  $actions {
    $(echo('Hello from Groovy script'))
    $(delay().seconds(1))

    $(createVariables()
        .variable('foo', 'bar'))

    $(echo('Variable foo=${foo}'))
  }
  """
  Then apply basic.groovy
```

The example above defines the test actions with the Groovy DSL under the name `basic.groovy`. Later in the test the actions are executed with the `apply` step.

@Then("^(?:apply|verify) actions {name}\|.groovy$")

```
Then apply actions {name}.groovy
```

Users familiar with Citrus will notice immediately that the action script is using the Citrus actions DSL to describe what should be done when running the Groovy script as part of the test.

The Citrus action DSL is quite powerful and allows you to perform complex actions such as iterations, conditionals and send/receive operations as shown in the next sample.

```
Scenario: Messaging actions
  Given create actions messaging.groovy
  """
  $actions {
    $(send('direct:myQueue')
      .payload('Hello from Groovy script!'))

    $(receive('direct:myQueue')
      .payload('Hello from Groovy script!'))
  }
  """
  Then apply actions messaging.groovy
```

As an alternative to write the Groovy DSL directly into the test feature file you can also laod the test action script from external file resources.

@Given("^load actions {file_name}\|.groovy$")

```
Given load actions {file_name}.groovy$")
```

The file name is the name of the action script. So you can use the file name to apply the script in the test for execution.

Apply Groovy script

```
Then apply actions {file_name}.groovy
```

You can also use a shortcut syntax to directly call a test action.

@Then("^${{action_code})$")

```
Then $(echo('Hello from Groovy script!'))
```

This will add a new `echo` test action and run the action. The action code uses a Groovy script that defines the test action by using the common Citrus test action domain specific language.

You can apply multiline scripts directly, too.

@Given("^apply script$")

```
Given apply script
    """
    $actions {
        $(delay().seconds(1))

        $(echo('Hello from Groovy script!'))
    }
    """
```

53.3.4. Finally actions

Sometimes it is mandatory to cleanup test data after a scenario. It would be good to have a set of test actions that get executed in a guaranteed way - even in case the test scenario failed with errors before.

The Citrus framework provides a concept of `finally block` actions. These actions will be run after the test in all circumstances (success and failure).

Finally block actions

```
Given apply script
    """
    $finally {
        echo('${greeting} in finally!')
    }
    """
```

As an alternative syntax you can add a 'doFinally()' test action to your script.

Finally test action

```
Given apply script
    """
    $actions {
        $(doFinally().actions(
            echo('${greeting} in finally!')
        ))
    }
    """
```

This is how you can define test actions in Groovy that get executed after the test.

53.4. Http steps

The Http protocol is a widely used communication protocol when it comes to exchanging data between systems. REST Http services are very prominent and producing/consuming those services

is a common task in software development these days. Citrus provides ready to use steps that are able to exchange request/response messages via Http as a client and server during the test.

The sample below shows how to use Http communication in a test:

Http communication sample

```
Feature: Http client

  Background:
    Given URL: https://hello-service

  Scenario: Health check
    Given path /health is healthy

  Scenario: GET request
    When send GET /todo
    Then verify HTTP response body: {"id": "@ignore@", "task": "Sample task",
"completed": 0}
    And receive HTTP 200 OK
```

The example above sets a base request URL to `https://hello-service` and performs a health check on path `/health`. After that we can send a Http `GET` request to the endpoint and verify the response status code.

All steps shown are part of the Citrus framework so you can use them out of the box. The next sections explore the Http capabilities in more detail.

53.4.1. Http client steps

As a client you can specify the server URL and send requests to it.

@Given("^(?:URL|url): {url}$")

```
Given URL: {url}
```

The given URL points to a server endpoint. All further client Http steps send the requests to that endpoint.

As an alternative you can reference a Http client component that previously has been added to the framework configuration.

@Given("^HTTP client \"{name}\"$")

```
Given HTTP client "{name}"
```

This step loads a Http client component by its name and uses that for further requests.

Once you have configured the Http endpoint URL or the Http client you can start sending request

messages.

53.4.2. Send Http requests

Sending requests via Http is as easy as choosing the Http method (GET, POST, PUT, DELETE, ...) to use.

@When("^send (GET|HEAD|POST|PUT|PATCH|DELETE|OPTIONS|TRACE) {path}$")

```
When send {method} {path}
```

The given path resides to a valid resource on the server endpoint. The resource path is added to the base URL and identifies the resource on the server.

Send Http GET request

```
When send GET /todo
```

You can choose the Http method that should be used to send the request (e.g. `GET`). Of course the request can have headers and a message body. You need to set these before sending the request in separate steps.

Request headers

@Given("^HTTP request header {name}=\"{value}\"$")

```
Given HTTP request header {name}="{value}"
```

The step above adds a Http header to the request. The header is defined with a name and receives a value. You can set multiple headers in a single step, too:

@Given("^HTTP request headers$")

```
Given HTTP request headers
  | {name} | {value} |
```

The step uses a data table to define multiple message headers with name and value.

Set request headers

```
Given HTTP request headers
  | Accept          | application/json |
  | Accept-Encoding | gzip |
```

Request body

The Http request can have a body content which is sent as part of the request.

@Given("^HTTP request body: {body}$")

```
Given HTTP request body: {body}
```

The step above specifies the Http request body in a single line. When you need to use multiline body content please use the next step:

@Given("^HTTP request body$")

```
Given HTTP request body
"""
<<content>>
"""
```

When the request body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load HTTP request body {file}$")

```
Given load HTTP request body {file}
```

This step loads the body content from the given file resource.

Request parameters

The Http request is able to use parameters that get added to the request URL. You can set those parameters in a separate step.

@Given("^HTTP request query parameter {name}=\"{value}\"$")

```
Given HTTP request query parameter {name}="{value}"
```

Request timeouts

In some cases the client waits a long time for the server to respond. As Http is a synchronous communication protocol by its nature the client will synchronously wait for the response before doing any other step. You can specify the time to wait for the server to respond.

@Given("^HTTP request timeout is {time} milliseconds$")

```
Given HTTP request timeout is {time} milliseconds
```

This sets the client timeout to thee given time in milliseconds.

Request fork mode

As seen in the previous section Http is synchronous by default. This can be a problem when the test needs to do multiple things in parallel. By default the Http client step will always block any other

step until the server response has been received. You can change this behavior to an asynchronous behavior so the next steps will not be blocked by the Http client step.

@Given("^HTTP request fork mode is (enabled|disabled)$")

```
Given HTTP request fork mode is enabled
```

This will enable the fork mode so all client request will be non-blocking. By default the fork mode is disabled.

53.4.3. Send raw Http request data

In the previous section several steps have defined the Http request (header, parameter, body) before sending the message in a separate step. As an alternative to this approach you can also specify the complete Http request data in a single step.

@Given("^send HTTP request$")

```
Given send HTTP request
"""
<<request_data>>
"""
```

The next example shows the complete Http request data step:

Send raw Http request data

```
Given send HTTP request
"""
GET https://hello-service
Accept-Charset:utf-8
Accept:application/json, application/*+json, */*
Host:localhost:8080
Content-Type:text/plain;charset=UTF-8
"""
```

53.4.4. Verify Http responses

The time you send out a Http request you will be provided with a response from the server. Citrus is able to verify the Http response content in order to make sure that the server has processed the request as expected.

@Then("^receive HTTP {status_code}(?: {reason_phrase})?$")

```
Then receive HTTP {status_code} {reason_phrase}
```

The most critical part of the Http response is thee status code (e.g. 200, 404, 500). The status code should refer to the success or error of the request. The server can use a wide range of Http status

codes that are categorized as follows, also see W3C.

- *1xx* informational response – the request was received, continuing process
- *2xx* successful – the request was successfully received, understood, and accepted
- *3xx* redirection – further action needs to be taken in order to complete the request
- *4xx* client error – the request contains bad syntax or cannot be fulfilled
- *5xx* server error – the server failed to fulfil an apparently valid request

Verify Http status code

```
Then receive HTTP 200 OK
```

The reason phrase `OK` is optional and is also not part of the verification mechanism for the response. It just gives human readers a better understanding of the status code.

Of course the Http response can also have headers and a message body. Citrus is able to verify those response data, too. Please define the expected headers and body content before verifying the status code.

Response headers

@Then("^expect HTTP response header {name}=\"{value}\"$")

```
Then expect HTTP response header {name}="{value}"
```

The step above adds a Http header to the response. The header is defined with a name and receives a value. You can set multiple headers in a single step, too:

@Then("^expect HTTP response headers$")

```
Then expect HTTP response headers
  | {name} | {value} |
```

The step uses a data table to define multiple message headers with name and value.

Verify response headers

```
Then expect HTTP response headers
  | Encoding     | gzip |
  | Content-Type | application/json |
```

Header name ignore case

Header names are not case-sensitive per Http specification. This means servers and clients may choose the header name case individually. In order to avoid verification errors when expecting headers in Http the header validation mechanism in Citrus is able to ignore the header name case.

@Given("^HTTP header name ignore case is (enabled|disabled)$")

```
Given HTTP header name ignore case is enabled
```

This will enable the mode where header name verification is case-insensitive.

You may also use an environment setting `CITRUS_HTTP_HEADER_NAME_IGNORE_CASE=true/false` to enable/disable the setting for the whole test suite. By default, the setting is disabled so header names are verified with case-sensitive mode.

Response body

@Then("^expect HTTP response body: {body}$")

```
Then expect HTTP response body: {body}
```

The step above specifies the Http response body in a single line. When you need to use multiline body content please use the next step:

@Then("^expect HTTP response body$")

```
Then expect HTTP response body
"""
<<content>>
"""
```

When the response body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^expect HTTP response body loaded from {file}$")

```
Given expect HTTP response body loaded from {file}
```

This step loads the body content from the given file resource.

53.4.5. Verify raw Http response data

In the previous section several steps have defined the Http response (header, parameter, body) before verifying the message received. As an alternative to this approach you can also specify the complete expected Http response data in a single step.

@Then("^receive HTTP response$")

```
Then receive HTTP response
"""
<<response_data>>
"""
```

The next example shows the complete Http response data step:

Verify raw Http response data

```
Then receive HTTP response
"""
HTTP/1.1 200 OK
Content-Type:application/json
X-TodoId:@isNumber()@
Date: @ignore@

{"id": "@ignore@", "task": "Sample task", "completed": 0}
"""
```

53.4.6. Verify response using JsonPath

When verifying Http client responses sent by the server you can use JsonPath expressions to validate the response message body content.

@Then("^(?:expect|verify) HTTP response expression: {expression}=\"{value}\"$")

```
Then expect HTTP response expression: {expression}="{value}"
```

The step defines a JsonPath expression (e.g. `$.person.age`) and an expected value. The expression is evaluated against the received response message body and the value is compared to the expected value. This way you can explicitly verify elements in the Json body.

The very same mechanism also applies to XML message body content. Just use a XPath expression instead of JsonPath.

53.4.7. Http server steps

On the server side Citrus needs to start a Http server instance on a given port and listen for incoming requests. The test is able to verify incoming requests and then provide a simulated response message with response headers and body content.

Http communication sample

```
Feature: Http server

  Background:
    Given HTTP server listening on port 8080
    And start HTTP server

  Scenario: Expect GET request
    When receive GET /todo
    Then HTTP response body:  {"id": 1000, "task": "Sample task", "completed": 0}
    And send HTTP 200 OK

  Scenario: Expect POST request
    Given expect HTTP request body: {"id": "@isNumber()@", "task": "New task",
"completed": "@matches(0|1)@"}
    When receive POST /todo
    Then send HTTP 201 CREATED
```

In the HTTP server sample above we create a new server instance listening on port `8080`. Then we expect a `GET` request on path `/todo`. The server responds with a Http `200 OK` response message and given Json body as payload.

The second scenario expects a POST request with a given body as Json payload. The expected request payload is verified with the powerful Citrus JSON message validator being able to compare JSON tree structures in combination with validation matchers such as `isNumber()` or `matches(0|1)`.

After the request verification has passed the server responds with a simple Http `201 CREATED`.

The next sections guide you through the Http server capabilities in Citrus.

53.4.8. Http server configuration

When the test run starts Citrus will initialize the Http server instance and listen on a port for incoming requests.

Http server port

By default this server uses the port `8080`, but you can adjust the port with following step.

@Given("^HTTP server listening on port {port}$")

```
Given HTTP server listening on port {port}
```

Http server timeout

The test waits for incoming requests but the test may hit request timeouts when no request has been received. By default the server waits for five seconds each time a request is expected. You can adjust the server timeout.

@Given("^HTTP server timeout is {time} milliseconds$")

```
Given HTTP server timeout is {time} milliseconds
```

This sets the server timeout to the given time in milliseconds.

Http server component

You can use the default Http server instance that is automatically created or reference server component in the project configuration (e.g. Spring bean, component configuration).

@Given("^HTTP server \"{name}\"$")

```
Given HTTP server "{name}"
```

This step loads a Http server component by its name and uses that for server side testing.

Create Http server

When dealing with multiple server components at the same time or when a fresh server instance is required you can create a new Http server with:

@Given("^(?:create|new) HTTP server \"{name}\"$")

```
Given new HTTP server "{name}"
```

This creates a fresh Http server instance. Please mind that a port can only be bound once so you may need to stop other server instances or choose another server port.

@Given("^(?:create|new) HTTP server \"{name}\" with configuration$")

```
Given new HTTP server "{name}" with configuration
| port    | 8081 |
| timeout | 1000 |
```

53.4.9. Receive Http requests

You can define expected incoming Http requests as part of the test.

@When("^receive (GET|HEAD|POST|PUT|PATCH|DELETE|OPTIONS|TRACE) {path}$")

```
When receive {method} {path}
```

The incoming request must match the given '{method}` and `{path}`.

Receive Http GET request

```
When receive GET /todo
```

Of course, you can also verify headers and the request message body. You need to specify the expected request before receiving the request with the `receive` steps.

Request headers

@Given("^(?:expect|verify) HTTP request header {name}=\"{value}\"$")

```
Given expect HTTP request header {name}="{value}"
```

The step above adds the Http header to the request validation. The header must be present in the incoming request and must match the expected value. You can verify multiple headers in a single step, too:

@Given("^(?:expect|verify) HTTP request headers$")

```
Given expect HTTP request headers
  | {name} | {value} |
```

The step uses a data table to define the message headers with name and value.

Expect request headers

```
Given expect HTTP request headers
  | Accept          | application/json |
  | Accept-Encoding | gzip |
```

Request body

Each incoming Http request can have a body and you are able to verify the body content in multiple ways.

@Given("^(?:expect|verify) HTTP request body: {body}$")

```
Given expect HTTP request body: {body}
```

The step above specifies the expected Http request body in a single line. Multiline body content must use the next step:

@Given("^(?:expect|verify) HTTP request body$")

```
Given expect HTTP request body
"""
<<content>>
"""
```

When the request body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^expect HTTP request body loaded from {file}$")

```
Given expect HTTP request body loaded from {file}
```

This step loads the body content from the given file resource.

Request parameters

The Http request can have parameters on the request URL. You can verify those parameters in a separate step.

@Given("^(?:expect|verify) HTTP request query parameter {name}=\"{value}\"$")

```
Given expect HTTP request query parameter {name}="{value}"
```

53.4.10. Receive raw Http request data

In the previous section several steps have defined the expected Http request (header, parameter, body). As an alternative to this approach you can also specify the complete Http request data in a single step.

@Given("^receive HTTP request$")

```
Given receive HTTP request
"""
<<request_data>>
"""
```

The next example shows the complete Http request data step:

Receive raw Http request data

```
Given receive HTTP request
"""
GET https://hello-service
Accept-Charset:utf-8
Accept:application/json, application/*+json, */*
Host:localhost:8080
Content-Type:text/plain;charset=UTF-8
"""
```

53.4.11. Verify requests using JsonPath

When verifying Http client requests you can use JsonPath expressions to validate the request message body content.

@When("^(?:expect|verify) HTTP request expression: {expression}=|"{value}|"$")

```
When expect HTTP request expression: {expression}="{value}"
```

The step defines a JsonPath expression (e.g. `$.person.age`) and an expected value. The expression is evaluated against the received request message body and the value is compared to the expected value. This way you can explicitly verify elements in the Json body.

The very same mechanism also applies to XML message body content. Just use a XPath expression instead of JsonPath.

53.4.12. Send Http responses

The time you have verified a Http request as a server you need to provided a proper response to the calling client. Citrus is able to simulate the Http response content.

@Then("^send HTTP {status_code}(?: {reason_phrase})?$")

```
Then send HTTP {status_code} {reason_phrase}
```

The step defines the Http response status code (e.g. 200, 404, 500) to return.

Return Http status code

```
Then send HTTP 200 OK
```

The reason phrase `OK` is optional. It just gives human readers a better understanding of the status code returned.

Of course the Http response can also have headers and a message body. Citrus is able to simulate this response data, too.

Response headers

@Given("^HTTP response header {name}=\"{value}\"$")

```
Given HTTP response header {name}="{value}"
```

The step above adds a Http header to the response. The header is defined with a name and value. You can set multiple headers in a single step, too:

@Given("^HTTP response headers$")

```
Given HTTP response headers
  | {name} | {value} |
```

The step uses a data table to define multiple message headers with name and value.

Return response headers

```
Given HTTP response headers
  | Encoding     | gzip |
  | Content-Type | application/json |
```

Response body

@Given("^HTTP response body: {body}$")

```
Given HTTP response body: {body}
```

The step above specifies the Http response body in a single line. When you need to use multiline body content please use the next step:

@Given("^HTTP response body$")

```
Given HTTP response body
"""
<<content>>
"""
```

When the response body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load HTTP response body {file}$")

```
Given load HTTP response body {file}
```

This step loads the body content from the given file resource.

53.4.13. Send raw Http response data

In the previous section several steps have defined the Http response (header, parameter, body). As an alternative to this approach you can also specify the complete Http response data in a single step.

@Then("^send HTTP response$")

```
Then send HTTP response
"""
<<response_data>>
"""
```

The next example shows the complete Http response data step:

Return raw Http response data

```
Then send HTTP response
"""
HTTP/1.1 200 OK
Content-Type:application/json
X-TodoId:@isNumber()@
Date: @ignore@

{"id": "@ignore@", "task": "Sample task", "completed": 0}
"""
```

53.4.14. Http health checks

Often Http server provide a health endpoint so clients can check the status of the server to be up and running. The health check is supported with the following steps.

@Given("^{URL} is healthy$")

```
Given {URL} is healthy
```

The step performs a health check on the given `{URL}` by sending a request to the endpoint and checking for a response status code marking success (200 OK).

Health check

```
Given https://some-service-url/health is healthy
```

Instead of specifying the complete health check URL you can make use of the base URL given in the central Http step.

@Given("^URL {path} is healthy$")

```
Given path {path} is healthy
```

The given path is added to the base URL and should resolve to the health check resource on the server (e.g. `/health`).

Health path check

```
Given URL: https://hello-service
Given path /health is healthy
```

The steps above perform the health check only a single time. Based on the provided Http server response status the step passes or fails. In some cases can not make sure that the server has been started yet and the health check might fail occasionally. In these cases it is a good iodea to use the wait health check step.

@Given("^wait for URL {url}$")

```
Given wait for URL {url}
```

The step will wait for given URL to return a `200 OK` response. The step is actively waiting while polling the URL multiple times when the response is not positive. By default this step uses a `HEAD` request. You can explicitly choose another Http method, too.

@Given("^wait for (GET|HEAD|POST|PUT|PATCH|DELETE|OPTIONS|TRACE) on URL {url}$")

```
Given wait for GET on URL {url}
```

The sample above uses a `GET` request for the health checks.

Also you can explicitly specify the expected return code that must match in order to pass the wait health check.

@Given("^wait for URL {url} to return {status_code}(?: {reason_phrase}?$")

```
Given wait for URL {url} to return {status_code} {reason_phrase}
```

Once again the `{reason_phrase}` is optional and only for better readability reasons.

Wait for specific status code

```
Given wait for URL https://hello-service/health to return 200 OK
```

Last not least you can specify the request method on the wait operation, too.

@Given("^wait for (GET|HEAD|POST|PUT|PATCH|DELETE|OPTIONS|TRACE) on URL {url} to return {status_code}(?: {reason_phrase}?$")

```
Given wait for {method} on URL {url} to return {status_code} {reason_phrase}
```

This completes the health check capabilities in the Http steps.

53.4.15. Https support

Citrus steps support secure Http connections on both client and server.

On the client side enabling secure Http connection is as simple as defining a request URL with `https` scheme.

@Given("^(?:URL|url): {url}$")

```
Given URL: https://some.endpoint/path
```

Citrus is going to initialize the client with a SSL request factory and use secure Http connections.

On the server side you have to enable secure Http with the following step:

@Given("^enable secure HTTP server$")

```
Given enable secure HTTP server
```

Please use this step before starting the server with:

@Given("^start HTTP server$")

```
Given start HTTP server
```

The secure SSL connector uses the port `8443` by default. You can adjust this secure port with:

@Given("^HTTP server secure port {port}$")

```
Given HTTP server secure port {port}
```

The Http server uses a default SSL keystore with a self signed certificate. Users are able to customize the server certificate with a custom SSL keystore.

@Given("^HTTP server SSL keystore path {file}$")

```
Given HTTP server SSL keystore path {file}
```

@Given("^HTTP server SSL keystore password {password}$")

```
Given HTTP server SSL keystore password {password}
```

The steps set a custom keystore (e.g. server.jks) file and a custom keystore password.

The keystore settings are also accessible via environment variables.

SSL keystore environment variables

```
CITRUS_HTTP_SECURE_KEYSTORE_PATH={file}
CITRUS_HTTP_SECURE_KEYSTORE_PASSWIRD={password}
```

The Http server components also provide a convenient way to add server properties when creating new instances:

@Given("^(?:create|new) HTTP server \"{name}\" with configuration$")

```
Given new HTTP server "{name}" with configuration
| secure              | true       |
| securePort          | 8443       |
| timeout             | 1000       |
| sslKeystorePath     | server.jks |
| sslKeystorePassword | secret     |
```

53.5. JDBC steps

Citrus provides steps that allow executing SQL actions on relational databases. This includes updates and queries. In case of a database query you are able to verify the result set with expected values.

53.5.1. Connection configuration

Before running any SQL statement you need to configure a datasource that allows connecting to the database.

@Given("^(?:D|d)atabase connection$")

```
Given Database connection
| {property} | {value} |
```

The step configures a new database connection and uses a data table to define connection properties such as connection URL, username and password.

Specify connection properties

```
Given Database connection
| driver    | org.postgresql.Driver |
| url       | jdbc:postgresql://localhost:5432/testdb |
| username  | test |
| password  | secret |
```

This defines the connection parameters so the test is able to connect to the database.

In addition to that you can also reference an existing datasource that has been added to the framework configuration.

@Given("^(?:D|d)ata source: {name}$")

```
Given Data source: {name}
```

The name of the datasource should reference a configured component in the test project. You can add components as Spring beans for instance.

53.5.2. SQL update

The test is able to run SQL updates (UPDATE, INSERT, DELETE) on the database.

@When("^(?:execute |perform)?SQL update: {statement}$")

```
When execute SQL update: {statement}
```

The step executes the given SQL statement using the configured database connection. For multiline statements please use:

@When("^(?:execute |perform)?SQL update$")

```
When execute SQL update
"""
{statement}
"""
```

You can also run multiple statements in a single step by using a data table.

@When("^(?:execute |perform)?SQL updates$")

```
When execute SQL updates
| {statement_1} |
| {statement_2} |
...
| {statement_x} |
```

53.5.3. SQL query

The SQL query obtains data from the database in form of result sets. The Citrus test is able to verify the result sets with an expected set of rows and column values returned.

@Given("^SQL query: {statement}$")

```
Given SQL query: {statement}
```

This step defines the query to execute. Multiline SQL query statements are supported, too.

@Given("^SQL query$")

```
Given SQL query
"""
{statement}
"""
```

You can also run multiple queries in one step. As usual the step uses a data table.

@Given("^SQL query statements$")

```
When SQL query statements
| {statement_1} |
| {statement_2} |
...
| {statement_x} |
```

In a next step you can provide the expected outcome in form of column name and value.

Verify SQL result set

@Then("^verify column {name}={value}$")

```
Then verify column {name}={value}
```

This step executes the query and verifies the column with given name to match the expected value.

You can use multiple verifications on several columns with a data table.

@Then("^verify columns$")

```
Then verify columns
| {column_1} | {value_1_1} | {value_1_2} |
| {column_2} | {value_2_1} | {value_2_2} |
...
| {column_x} | {value_x_x} | {value_x_x} |
```

The data table is able to verify a matrix of rows and columns. Each column can have multiple row

values.

Validate multi row result sets

```
Given SQL query: SELECT ID, TASK, COMPLETED FROM todo ORDER BY id
Then verify columns
| ID        | 1                  | 2             | 3          | 4            |
@ignore@               |
| TASK      | Learn some Camel K! | Get some milk | Do laundry | Wash the dog | Test
Camel with Citrus! |
| COMPLETED | 0                  | 0             | 0          |            0 | 0
|
```

53.5.4. Result set verification script

For more complex result set validation you can use a Groovy result set verification script.

@Then("^verify result set$")

```
Then verify result set
"""
<<Groovy>>
"""
```

The Groovy script can work with the complete result set and is quite powerful.

```
Given SQL query: SELECT TASK FROM todo
Then verify result set
"""
assert rows.size == 1
assert rows[0].TASK == 'Learn some Camel K!'
"""
```

53.6. JMS steps

JMS is well-known as transport for point-to-point and publish/subscribe messaging. Users can produce and consume messages on queues and topics on a message broker.

Citrus has support for JMS related messaging on both producer and consumer.

53.6.1. Connection factory

The JMS standard requires clients to open connections over a connection factory. The connection factory is a vendor specific implementation and receives a set of properties such as connection URL, username and password.

@Given("^(?:JMS|jms) connection factory$")

```
Given JMS connection factory
| {property} | {value} |
```

The configuration step receives a data table that defines the connection settings.

Connection factory settings

```
Given JMS connection factory
| type       | org.apache.activemq.artemis.jms.client.ActiveMQConnectionFactory |
| brokerUrl  | tcp://localhost:61616 |
| username   | ${activemq.user}      |
| password  | ${activemq.password}  |
```

The connection factory type is vendor specific and depends on what kind of message broker you are using in your environment. Please make sure to add the respective client library as a project dependency in the Citrus configuration.

Sensitive values such as `username` and `password` can be set with a test variable placeholder. The variable value can be set by a secret in Kubernetes/Openshift. This ensures to not share sensitive data in the public.

As an alternative to defining the connection factory as part of the test steps you can load a predefined connection factory component from the configuration.

@Given("^(?:JMS|jms) connection factory {name}$")

```
Given JMS connection factory {name}
```

The step references a connection factory component that has been added to the framework configuration (e.g. as Spring bean). This way you can share the connection factory in multiple tests.

53.6.2. Destination and endpoint configuration

In addition to the connection factory the test needs to specify the JMS destination (queue or topic) to use.

@Given("^(?:JMS|jms) destination: {name}$")

```
Given JMS destination: {name}
```

This sets the destination name for the next steps. As an alternative to that you can also reference a predefined endpoint component from the configuration.

@Given("^(?:JMS|jms) endpoint \"{name}\"$")

```
Given JMS endpoint {name}
```

The step tries to resolve the JMS endpoint with given `{name}` in the available configuration. The endpoint has a destination set and references a connection factory on its own.

So now the test is ready to produce and consume messages from JMS destinations.

53.6.3. Send JMS messages

A test can publish messages on a JMS destination. The message consists of message headers and a body content. Before sending a message the tests needs to specify the message content.

Message headers

The message headers are key value pairs that are sent as part of the message. You can add a new header with the following step:

@Given("^(?:JMS|jms) message header {name}(?:=| is)\"{value}\"$")

```
Given JMS message header {name}="{value}"
```

When using a data table you can set multiple headers in one step.

@Given("^(?:JMS|jms) message headers$")

```
Given JMS message headers
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

Message body

@Given("^(?:JMS|jms) message body: {body}$")

```
Given JMS message body: {body}
```

This step can set a single line body content. Of course you can also work with multiline body content.

@Given("^(?:JMS|jms) message body$")

```
Given JMS message body
"""
{body}
"""
```

When the body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load (?:JMS|jms) message body {file}$")

```
Given load JMS message body {file}
```

This step loads the body content from the given file resource.

Now another step can send the message as it has been specified in the previous steps.

@When("^send (?:JMS|jms) message$")

```
When send JMS message
```

This sends the message to the previously configured JMS destination. You can overwrite this destination in the send step.

@When("^send (?:JMS|jms) message to destination {destination}$")

```
When send JMS message to destination {destination}
```

The approach described clearly separates message specification and send operation as all of it is done in separate steps. As an alternative you can also specify the message content in one step.

@When("^send (?:JMS|jms) message with body: {body}$")

```
When send JMS message with body: {body}
```

You can also add some message headers to this step.

@When("^send (?:JMS|jms) message with body and headers: {body}$")

```
When send JMS message with body and headers: {body}
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

The step combines message header and body specification with the actual send operation.

53.6.4. Receive JMS messages

Similar to sending messages to a JMS destination the test can also consume messages from a queue or topic. When the message has been received a validation mechanism makes sure that the message content received matches the expectations.

Users are able to provide expected message headers and body content in order to verify the received message.

Message headers

The expected message headers need to be set before receiving the message from the destination.

@Given("^(?:JMS|jms) message header {name}(?:=| is)\"{value}\"$")

```
Given JMS message header {name}="{value}"
```

When using a data table you can expect multiple headers in one step.

@Given("^(?:JMS|jms) message headers$")

```
Given JMS message headers
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

Message body

In addition to verify message headers you can also verify the body content. Once again the user specifies the expected message body before the message is received.

@Given("^(?:JMS|jms) message body: {body}$")

```
Given JMS message body: {body}
```

This step can expect a single line body content. Of course you can also work with multiline body content.

@Given("^(?:JMS|jms) message body$")

```
Given JMS message body
"""
{body}
"""
```

When the body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load (?:JMS|jms) message body {file}$")

```
Given load JMS message body {file}
```

This step loads the body content from the given file resource.

With the steps above the test has specified the expected message content. With that in place another step can receive the message and perform the validation.

@Then("^receive (?:JMS|jms) message$")

```
Then receive JMS message
```

The step uses the previously defined JMS destination to consume messages from it. You can use another destination in the step, too.

@Then("^receive (?:JMS|jms) message from destination {destination}$")

```
Then receive JMS message from destination {destination}
```

With this approach you have a clean separation of the expected message content specification and the actual receive operation. Of course you can also combine everything in one single step.

@Then(?:receive|expect|verify) (?:JMS|jms) message with body: {body}$")

```
Then receive JMS message with body: {body}
```

You can also add some message headers to this step.

@Then(?:receive|expect|verify) (?:JMS|jms) message with body and headers: {body}$")

```
Then receive JMS message with body and headers: {body}
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

The step combines message header and body specification with the actual receive operation.

Consumer timeout

The receive operation takes the first message available on the destination and performs the validation. In case there is no message available the consumer will wait for a given amount of time before a timeout will fail the test. You can adjust the timeout on the JMS consumer.

@Given("^(?:JMS|jms) consumer timeout is {time}(?: ms| milliseconds)$")

```
Given JMS consumer timeout is {time} milliseconds
```

Message selectors

The JMS standard provides a concept of message selectors so consumers can specify which message they want to consume from a destination. The consumer usually evaluates the selector expression on the message headers.

@Given("^(?:JMS|jms) selector: {expression}$")

```
Given JMS selector: {expression}
```

The selector expression defines a key and value that the message must match. The first message to match the selector on the destination it received by the consumer.

Use message selector

```
Given JMS selector: key='value'
```

53.7. Kafka steps

Apache Kafka is a powerful and widely used event streaming platform. Users are able to publish events and subscribe to event streams.

The following sections describe the support for Kafka related event streaming in Citrus.

53.7.1. Connection

First of all the test needs to connect to Kafka bootstrap servers. The connection provides several parameters that you can set with the following step.

@Given("^(?:Kafka|kafka) connection")

```
Given Kafka connection
| {property} | {value} |
```

The configuration step receives a data table that defines the connection settings.

Connection settings

```
Given Kafka connection
| url           | localhost:9092 |
| consumerGroup | citrus_group   |
| topic         | citrus_test    |
```

The most important part of the connection settings is the `url` that points to one or more Kafka bootstrap servers.

In addition to the connection settings there is a set of producer and consumer properties that you can set in order to configure the behavior of producers and consumers connecting with Kafka.

@Given("^(?:Kafka|kafka) producer configuration$")

```
Given Kafka producer configuration
| {property} | {value} |
```

@Given("^(?:Kafka|kafka) consumer configuration$")

```
Given Kafka consumer configuration
| {property} | {value} |
```

The available properties to set here are described in the Apache Kafka documentation. See the following example how to set producer and consumer properties.

```
Given Kafka producer configuration
| client.id          | citrus_producer |
| request.timeout.ms | 5000 |

Given Kafka consumer configuration
| client.id          | citrus_consumer |
| max.poll.records   | 1 |
```

53.7.2. Topic and endpoint configuration

In addition to the connection the test needs to specify the Kafka topic to use.

@Given("^(?:Kafka|kafka) topic: {name}$")

```
Given Kafka topic: {name}
```

This sets the topic name for the next steps. As an alternative to that you can also reference a predefined endpoint component from the configuration.

@Given("^(?:Kafka|kafka) endpoint \"{name}\"$")

```
Given Kafka endpoint {name}
```

The step tries to resolve the Kafka endpoint with given `{name}` in the available configuration. The endpoint can reference a topic set and connection settings on its own.

So now the test is ready to produce and consume events from Kafka topics.

53.7.3. Send Kafka events

A test can publish events on a Kafka topic. The event consists of message headers and a body content. Before sending an event the tests needs to specify the message content.

Message key

Each event on a Kafka event stream has a message key set. You can set this key in a separate step before sending the event.

@Given("^(?:Kafka|kafka) message key: {key}$")

```
Given Kafka message key: {key}$")
```

This specifies the message key for the next event that is published.

Message headers

The message headers are key value pairs that are sent as part of the message. You can add a new header with the following step:

@Given("^(?:Kafka|kafka) message header {name}(?:=| is)\"{value}\"$")

```
Given Kafka message header {name}="{value}"
```

When using a data table you can set multiple headers in one step.

@Given("^(?:Kafka|kafka) message headers$")

```
Given Kafka message headers
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

Message body

@Given("^(?:Kafka|kafka) message body: {body}$")

```
Given Kafka message body: {body}
```

This step can set a single line body content. Of course you can also work with multiline body content.

@Given("^(?:Kafka|kafka) message body$")

```
Given Kafka message body
"""
{body}
"""
```

When the body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load (?:Kafka|kafka) message body {file}$")

```
Given load Kafka message body {file}
```

This step loads the body content from the given file resource.

Now another step can publish the message as it has been specified in the previous steps.

@When("^send (?:Kafka|kafka) message$")

```
When send Kafka message
```

This publishes the message to the previously configured Kafka topic. You can overwrite this topic in the publish step.

@When("^send (?:Kafka|kafka) message to topic {topic}$")

```
When send Kafka message to topic {topic}
```

The approach described clearly separates message specification and send operation as all of it is done in separate steps. As an alternative you can also specify the message content in one step.

@When("^send (?:Kafka|kafka) message with body: {body}$")

```
When send Kafka message with body: {body}
```

You can also add some message headers to this step.

@When("^send (?:Kafka|kafka) message with body and headers: {body}$")

```
When send Kafka message with body and headers: {body}
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

The step combines message header and body specification with the actual send operation.

53.7.4. Receive Kafka events

Similar to publishing events to a Kafka topic the test can also consume events from an event stream. When the event has been received a validation mechanism makes sure that the message content received matches the expectations.

Users are able to provide expected message headers and body content in order to verify the received event.

Message headers

The expected message headers need to be set before receiving the event from the topic.

@Given("^(?:Kafka|kafka) message header {name}(?:=| is)\"{value}\"$")

```
Given Kafka message header {name}="{value}"
```

When using a data table you can expect multiple headers in one step.

@Given("^(?:Kafka|kafka) message headers$")

```
Given Kafka message headers
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

Message body

In addition to verify message headers you can also verify the body content. Once again the user specifies the expected message body before the message is received.

@Given("^(?:Kafka|kafka) message body: {body}$")

```
Given Kafka message body: {body}
```

This step can expect a single line body content. Of course, you can also work with multiline body content.

@Given("^(?:Kafka|kafka) message body$")

```
Given Kafka message body
"""
{body}
"""
```

When the body is getting too big it may be a better idea to load the content from an external file resource:

@Given("^load (?:Kafka|kafka) message body {file}$")

```
Given load Kafka message body {file}
```

This step loads the body content from the given file resource.

With the steps above the test has specified the expected message content. With that in place another step can receive the message and perform the validation.

@Then("^receive (?:Kafka|kafka) message$")

```
Then receive Kafka message
```

The step uses the previously defined Kafka topic to consume events from it. You can use another topic in the step, too.

@Then("^receive (?:Kafka|kafka) message from topic {topic}$")

```
Then receive Kafka message from topic {topic}
```

With this approach you have a clean separation of the expected message content specification and the actual receive operation. Of course you can also combine everything in one single step.

@Then(?:receive|expect|verify) (?:Kafka|kafka) message with body: {body}$")

```
Then receive Kafka message with body: {body}
```

You can also add some message headers to this step.

@Then(?:receive|expect|verify) (?:Kafka|kafka) message with body and headers: {body}$")

```
Then receive Kafka message with body and headers: {body}
| {header_1} | {value_1} |
| {header_2} | {value_2} |
...
| {header_x} | {value_x} |
```

The step combines message header and body specification with the actual receive operation.

Consumer timeout

The `receive` operation takes the first event available on the topic and performs the validation. In case there is no event available the consumer will wait for a given amount of time before a timeout will fail the test. You can adjust the timeout on the Kafka consumer.

@Given("^(?:Kafka|kafka) consumer timeout is {time}(?: ms| milliseconds)$")

```
Given Kafka consumer timeout is {time} milliseconds
```

53.7.5. Special configuration

The Kafka standard provides a set of special configuration that you can set as part of the test.

@Given("^(?:Kafka|kafka) topic partition: {partition}$")

```
Given Kafka topic partition: {partition}
```

This set the topic partition for all further steps publishing or consuming events from that topic.

53.8. Kubernetes steps

Kubernetes is a famous container management platform that allows automation of deployment, scaling and management of containerized applications.

Citrus uses the Kubernetes client API and is able to create Kubernetes resources (e.g. secrets, services, deployments and so on) as part of the test.

53.8.1. API version

The default Kubernetes API version used to create and manage resources is `v1`. You can overwrite this version with an environment variable set on the Citrus configuration.

Overwrite Kubernetes API version

```
CITRUS_KUBERNETES_API_VERSION=v1
```

This sets the Kubernetes API version for all operations.

53.8.2. Client configuration

@Given("^Kubernetes timeout is {time}(?: ms| milliseconds)$")

```
Given("^Kubernetes timeout is {time} milliseconds
```

This sets the timeout for all Kubernetes client operations.

53.8.3. Set namespace

Kubernetes uses the concept of namespaces to separate workloads on the cluster. You can connect to a specific namespace with the follwing step.

@Given("^Kubernetes namespace {name}$")

```
Given Kubernetes namespace {name}
```

53.8.4. Verify pod state

A Kubernetes pod has a state and is in a phase (e.g. running, stopped). You can verify the state with an expectation.

@Given("^Kubernetes pod {name} is running/stopped$")

```
Given Kubernetes pod {name} is running
```

Checks that the Kubernetes pod with given `{name}` is in state running and that the number of replicas is > 0. The step polls the state of the pod for a given amount of attempts with a given delay

between attempts. You can adjust the polling settings with:

@Given Kubernetes resource polling configuration

```
Given Kubernetes resource polling configuration
    | maxAttempts          | 10   |
    | delayBetweenAttempts | 1000 |
```

Instead of identifying the pod by its name you can also filter the pod with a label expression. The expression is a label key and value that identifies the pod in the current namespace.

@Given("^Kubernetes pod labeled with {label}={value} is running/stopped$")

```
Given Kubernetes pod labeled with {label}={value} is running
```

53.8.5. Watch Kubernetes pod logs

@Given("^Kubernetes pod {name} should print (.)$")*

```
Given Kubernetes pod {name} should print {log-message}
```

Watches the log output of a Kubernetes pod and waits for given `{log-message}` to be present in the logs. The step polls the logs for a given amount of time. You can adjust the polling configuration with:

@Given Kubernetes resource polling configuration

```
Given Kubernetes resource polling configuration
    | maxAttempts          | 10   |
    | delayBetweenAttempts | 1000 |
```

You can also wait for a log message to **not** be present in the output. Just use this step:

@Given("^Kubernetes pod {name} should not print (.)$")*

```
Given Kubernetes pod {name} should not print {log-message}
```

53.8.6. Kubernetes services

One of the most important features in the Citrus Kubernetes support is the management of services and in particular the automatic deployment of simulated services in Kubernetes.

The user is able to start a local Http server instance and create a service in Kubernetes out of it. This way the test is able to simulate services in Kubernetes and receive and verify incoming requests as part of the test.

First of all we define a new service within the test.

@Given("^Kubernetes service \"{name}\"$")

```
Given Kubernetes service "{name}"
```

This initializes a new Http server that will be used as Kubernetes service. The server is listening on a default port `8080`. You can use another port.

@Given("^Kubernetes service port {port}$")

```
Given Kubernetes service port {port}
```

In the following the test is able to create a new Kubernetes service with that Http server.

@Given("^create Kubernetes service {name}$")

```
Given create Kubernetes service {name}
```

The step creates the service in the Kubernetes namespace and exposes the given service port as target port. Clients are now able to connect to that new service. Each requests on the service will reside in a request in the test pod. The test is able to receive the request and verify its content as usual.

This way we can easily simulate Kubernetes services in the current namespace.

In case you need to use another target port you can adjust the port as follows.

@Given("^create Kubernetes service {name} with target port {port}$")

```
Given create Kubernetes service {name} with target port {port}
```

This exposes the service with the given target port.

In case you do not need the service anymore you can delete it with this step:

@Given("^delete Kubernetes service {name}$")

```
Given delete Kubernetes service {name}
```

53.8.7. Secrets

Secrets are resources that hold sensitive data. Other resources on the cluster can mount the secret content and access it.

You can create secrets in the current namespace in multiple ways.

@Given("^create Kubernetes secret {name}$")

```
Given create Kubernetes secret {name}
| {property} | {value} |
```

The step receives a secret name and a data table holding the property keys and values. These properties build the content of the secret.

Instead of listing all properties in the test itself you can load the secret from an external property file.

@Given("^load Kubernetes secret from file {file}.properties$")

```
Given load Kubernetes secret from file {file}.properties
```

The step loads the property file and creates the secret from the file content. The file name is used as the name fo the secret.

In case you want to clean up the secret you can delete it with:

@Given("^delete Kubernetes secret {name}$")

```
Given delete Kubernetes secret {name}
```

Sometimes secrets need to have labels and/or annotations that mark the secret with important metadata. You can add labels/annotations on the Kubernetes resource with these steps:

@Then("^create label {label}={value} on Kubernetes secret {name}$")

```
Then create label app=citrus on Kubernetes secret my-secret
```

@Then("^create annotation {annotation}={value} on Kubernetes secret {name}$")

```
Then create annotation app=citrus on Kubernetes secret my-secret
```

Or use multiple labels/annotations in one single step:

@Then("^create labels on Kubernetes secret {name}$")

```
Then create labels on Kubernetes secret my-secret
    | app      | citrus |
    | owned-by | odo  |
```

@Then("^create annotations on Kubernetes secret {name}$")

```
Then create annotations on Kubernetes secret my-secret
    | app      | citrus |
    | owned-by | odo  |
```

53.8.8. Pods, deployments and other resources

In the previous sections the test has creates services and secrets as Kubernetes resources. In addition to that the test is able to apply any resource as a YAML file on the Kubernetes cluster.

@Given("^create Kubernetes resource$")

```
Given create Kubernetes resource
"""
<<YAML>>
"""
```

With this step you can apply any Kubernetes resource as a YAML file.

Apply Kubernetes resource

```
Given create Kubernetes resource
"""
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
"""
```

The step above creates a new pod resource with the given specification. Instead of adding the resource specification in the test itself you can also load an external YAML file.

@Given("^load Kubernetes resource {file_path}$")

```
Given load Kubernetes resource {file_path}
```

Load pod.yaml

```
Given load Kubernetes resource pod.yaml
```

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

In case you need to delete a resource you can do so by specifying the minimal resource as a YAML specification.

@Given("^delete Kubernetes resource$")

```
Given delete Kubernetes resource
"""
<<YAML>>
"""
```

Delete resource

```
Given delete Kubernetes resource
"""
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
"""
```

You can also provide the external YAML file when deleting a resource. The step will automatically extract the resource kind and name from the file content.

@Given("^delete Kubernetes resource {file_path}$")

```
Given delete Kubernetes resource {file_path}
```

Delete resource from file

```
Given delete Kubernetes resource pod.yaml
```

53.8.9. Custom resources

In the previous sections the test has created Kubernetes resources (pods, services, secrets, deployments, ...). The user can also define custom resources in order to extend Kubernetes. Citrus is also able to manage these custom resources.

@Given("^create Kubernetes custom resource in {crd}$")

```
Given create Kubernetes custom resource in {crd}
"""
<<YAML>>
"""
```

The user has to provide a YAML specification of the custom resource.

Create custom resource

```
Given create Kubernetes custom resource in brokers.eventing.knative.dev
"""
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
  name: my-broker
"""
```

The step needs to know the `{crd}` (Custom Resource Definition) of the custom resource. In the example above the test creates a new resource of kind `Broker` in the custom resource definition `brokers.eventing.knative.dev`.

Of course, you can also load the custom resource from external file resource.

@Given("^load Kubernetes custom resource {file_path} in {crd}$")

```
Given load Kubernetes custom resource {file_path} in {crd}
```

Load custom resource from file

```
Given load Kubernetes custom resource broker.yaml in brokers.eventing.knative.dev
```

Once again the step needs to have the CRD type and the YAML specification as a file resource.



You need to make sure that the Citrus runtime and the user that connects to the Kubernetes cluster has proper permissions to manage the custom resource. The proper roles and role bindings need to apply to the user as well as a service account.

Prior to using the custom resource in a Citrus test you need to grant role permissions to the Citrus runtime. Otherwise, the test is not allowed to create the custom resource due to security constraints

on the cluster.

The Citrus runtime uses a service account `citrus-viewer` to run the test. The service account needs to have proper roles and permissions for managing the custom resource.

Assume that there is a CRD `foos.citrus.dev` and you want to manage the resources in your test:

crd-foo.yaml

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  labels:
    app: "citrus"
  creationTimestamp: null
  name: foos.citrus.dev
spec:
  group: citrus.dev
  names:
    kind: Foo
    listKind: FooList
    plural: foos
    singular: foo
  scope: Namespaced
  versions:
  - name: v1alpha1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        description: Foo resource schema
        properties:
          apiVersion:
            description: 'APIVersion defines the versioned schema of this
representation of an object. Servers should convert recognized schemas to the latest
internal value, and may reject unrecognized values. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#resources'
            type: string
          kind:
            description: 'Kind is a string value representing the REST resource this
object represents. Servers may infer this from the endpoint the client submits
requests to. Cannot be updated. In CamelCase. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#types-kinds'
            type: string
          metadata:
            type: object
          spec:
            description: Spec defines the desired state of Test
            properties:
```

```
              message:
                type: string
            required:
              - message
            type: object
          status:
            description: Status defines the observed state of Foo
            properties:
              conditions:
                items:
                  description: Condition describes the state of a resource at a
certain point.
                  properties:
                    message:
                      description: A human readable message indicating details about
the transition.
                      type: string
                    reason:
                      description: The reason for the condition's last transition.
                      type: string
                    status:
                      description: Status of the condition, one of True, False,
Unknown.
                      type: string
                    type:
                      description: Type of condition.
                      type: string
                  required:
                    - status
                    - type
                  type: object
                type: array
              version:
                type: string
            type: object
        type: object
    subresources:
      status: {}
```

The role to manage the new CRD `foos.citrus.dev` would be:

role-foo.yaml

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: foo-operator
  labels:
    app: "citrus"
    citrus.citrusframework.org/append-to-viewer: "true"
rules:
- apiGroups:
    - citrus.dev
  resources:
    - foos
  verbs:
    - create
    - delete
    - get
    - list
    - update
```

The role `foo-operator` is granted to create/delete/get/list/update custom resources of type `foos.citrus.dev`.

You also need a role binding to the `citrus-viewer` service account:

role-pipe-foo.yaml

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: citrus-foo-permission
  labels:
    app: "citrus"
subjects:
- kind: ServiceAccount
  name: citrus-viewer
roleRef:
  kind: Role
  name: foo-operator
  apiGroup: rbac.authorization.k8s.io
```



You can use the usual Kubernetes tools to create the role and role bindings for the user and a service account.



This role setup must be done by a cluster administrator.

In case you need to delete a custom resource from Kubernetes the user has to provide a minimal YAML specification that identifies the resource.

@Then("^delete Kubernetes custom resource in {crd}$")

```
Then delete Kubernetes custom resource in {crd}
"""
<<YAML>>
"""
```

Delete custom resource

```
Then delete Kubernetes custom resource in {crd}
"""
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
  name: my-broker
"""
```

As an alternative to that you can use an external file resource that holds the minimal YAML specification.

@Then("^delete Kubernetes custom resource {file_path} in {crd}$")

```
Then delete Kubernetes custom resource {file_path} in {crd}
```

Delete custom resource from file

```
Then delete Kubernetes custom resource broker.yaml in brokers.eventing.knative.dev
```

53.8.10. Verify custom resource conditions

Custom resources often define a status and describe multiple conditions (e.g. ready, available, completed). You can verify the condition on a custom resource.

@Given("^wait for condition={condition} on Kubernetes custom resource {name} in {type}$")

```
Given wait for condition={condition} on Kubernetes custom resource {name} in {type}
```

The step verifies that the given Kubernetes resource with name `{name}` describes a status condition `{condition}`. The step polls the state of the resource for a given amount of attempts with a given delay between attempts. You can adjust the polling settings with:

@Given Kubernetes resource polling configuration

```
Given Kubernetes resource polling configuration
    | maxAttempts          | 10   |
    | delayBetweenAttempts | 1000 |
```

Assume you have a custom resource like this:

foo-crd.yaml

```
apiVersion: citrus.dev/v1
kind: Foo
metadata:
  name: my-foo-resource
spec:
  message: Hello
status:
  conditions:
  - type: Initialized
    status: true
  - type: Ready
    status: true
```

The custom resource defines a status with multiple conditions. You can wait for a condition in the test.

Wait for condition=Ready

```
Given wait for condition=Ready on Kubernetes custom resource foo/my-foo-resource in
foos.citrus.dev/v1
```

The expected condition is `Ready` for the resource named `my-foo-resource`. The resource name can use a prefix that represents the kind information `foo/`. The kind helps to identify the custom resource. Also, the step defines the resource type `foos.citrus.dev` and version `v1`. This is required to identify the custom resource on the cluster.

Instead of identifying the resource by its name you can also filter the resources with a label expression. The expression is a label key and value that identifies the resource in the current namespace.

@Given("^wait for condition={condition} on Kubernetes custom resource in {type} labeled with {key}={value}$")

```
Given wait for condition={condition} on Kubernetes custom resource in {type} labeled
with {key}={value}
```

This will get all resources of type `{type}` and filter by given label `{key}={value}`. Then the given condition is verified on the resource.

53.8.11. Cleanup Kubernetes resources

The described steps are able to create Kubernetes resources on the current Kubernetes namespace. By default these resources get removed automatically after the test scenario.

The auto removal of Kubernetes resources can be turned off with the following step.

@Given("^Disable auto removal of Kubernetes resources$")

```
Given Disable auto removal of Kubernetes resources
```

Usually this step is a `Background` step for all scenarios in a feature file. This way multiple scenarios can work on the very same Kubernetes resources.

There is also a separate step to explicitly enable the auto removal.

@Given("^Enable auto removal of Kubernetes resources$")

```
Given Enable auto removal of Kubernetes resources
```

By default, all Kubernetes resources are automatically removed after each scenario.

53.9. Knative steps

Knative represents the Kubernetes-based platform to manage serverless workloads. You are able to leverage Knative eventing features such as producing and consuming events.

The following sections guide you through the Knative eventing capabilities in Citrus.

53.9.1. API version

The default Knative API version used to create and manage resources is `v1beta1`. You can overwrite this version with a environment variable set on the Citrus configuration.

Overwrite Knative API version

```
CITRUS_KNATIVE_API_VERSION=v1
```

This sets the Knative API version for all operations.

53.9.2. Client configuration

@Given("^Knative timeout is {time}(?: ms| milliseconds)$")

```
Given("^Knative timeout is {time} milliseconds
```

This sets the timeout for all Knative client operations.

53.9.3. Set namespace

Knative uses the concept of namespaces to separate workloads on the cluster. You can connect to a specific namespace with the follwing step.

@Given("^Knative namespace {name}$")

```
Given Knative namespace {name}
```

53.9.4. Knative broker

Eventing deals with publish/subscribe delivery of events in Knative. Knative eventing uses a broker that manages channels, subscriptions and events.

@Given("^Knative broker {name}$")

```
Given Knative broker {name}
```

This sets the broker name to use in all further steps that publish and consume events. The broker should already be present on the Kubernetes namespace. In case there is no broker yet you can create one.

@Given("^create Knative broker {name}$")

```
Given create Knative broker {name}
```

The step creates a new broker with the given `{name}`. The broker uses the default settings given in the Knative platform.

You can verify that the broker is up and running with the following step:

@Given("^Knative broker {name} is running$")

```
Given Knative broker {name} is running
```

53.9.5. Create event consumer service

The Knative broker delivers events to sinks. In order to start consuming events in a test you should create a event consumer service which acts as a sink.

@Given("^create Knative event consumer service {service}$")

```
Given create Knative event consumer service {service}
```

This step creates a new event consumer service. In particular this step creates a new Kubernetes service. The service instantiates a new local Http server and creates a new Kubernetes service from it. The service exposes a port which is `8080` by default.

You can adjust the service port as follows:

@Given("^Knative service port {port}$")

```
Given Knative service port {port}
```

By default the Kubernetes service uses the service port as a target port when exposing the service. You can choose another target port, too.

@Given("^create Knative event consumer service {service} with target port {port}$")

```
Given create Knative event consumer service {service} with target port {port}
```

53.9.6. Manage triggers

Triggers are used to deliver events to services and channels. Users can create a trigger as part of the test in order to start consuming events.

Triggers on services

@Given("^create Knative trigger {trigger} on service {service}$")

```
Given create Knative trigger {trigger} on service {service}
```

The step creates a new trigger on the given Knative broker. The trigger watches for events on the broker and forwards these events to the given service.

The service name either references an existing Kubernetes service or a new service that is created as part of the test as described in this guide.

Triggers can use filters on event attributes. This narrows the amount of events handled by the trigger.

@Given("^create Knative trigger {trigger} on service {service} with filter on attributes$")

```
Given create Knative trigger {trigger} on service {service} with filter on attributes
| {attribute} | {value} |
```

You need to add one or many attributes with respective value that should be added to the filter. As a result the trigger will only handle events matching the given filters.

Triggers on channels

Triggers can also forward events to channels. Subscribers are able to start subscriptions on these channels in order to receive the events.

@Given("^create Knative trigger {trigger} on channel {channel}$")

```
Given create Knative trigger {trigger} on channel {channel}
```

Of course, you can also add filters on attributes that narrow the amount of events handled by the trigger.

@Given("^create Knative trigger {trigger} on channel {channel} with filter on attributes$")

```
Given create Knative trigger {trigger} on channel {channel} with filter on attributes
| {attribute} | {value} |
```

The step uses a data table with attributes and values that should be added to the filter. As a result the trigger will only handle events matching the given filters.

Delete Knative trigger

You can delete a trigger by giving its name in the current namespace.

@Given("^delete Knative trigger {trigger}$")

```
Given delete Knative trigger {trigger}
```

This removes the Knative trigger from the current namespace.

53.9.7. Create channels

Channels represent a central concept of Knative eventing. Channels are able to deliver events to multiple subscribers. A test in Citrus is able to create new channels.

@Given("^create Knative channel {channel}$")

```
Given create Knative channel {channel}
```

Once the channel is available you can subscribe a service to the channel.

@Given subscribe service {service} to Knative channel {channel}$")

```
Given subscribe service {service} to Knative channel {channel}
```

53.9.8. Publish events

The test is able to publish events on the Knative broker. Citrus uses the Knative Http client API to publish events on the broker.

Because of that the test needs to specify a proper broker URL before publishing any events.

Knative broker URL

@Given("^Knative broker (?:URL|url): {url}$")

```
Given Knative broker URL: {url}
```

The URL points to a Knative broker and uses Http as transport. The test is able to publish events using this broker endpoint.

Knative client

As an alternative to that you can also specify a Http client component which connects to the broker.

@Given("^Knative client \"{name}\"$")

```
Given Knative client "{name}"
```

The client references a component in the configuration (e.g. Spring bean).

Now the test is ready to publish the event.

Create cloud events

@When("^(?:create|send) Knative event$")

```
When send Knative event
| {property} | {value} |
```

The step uses a data table in order to specify the cloud event properties that should be published. The cloud event data structure defines following properties:

- specversion
- type
- source
- subject
- id
- datacontenttype
- data

Following these properties you can specify the cloud event in the send operation.

Send cloud event

```
When send Knative event
| specversion     | 1.0 |
| type            | greeting |
| source          | https://github.com/citrusframework |
| subject         | hello |
| id              | say-hello |
| datacontenttype | application/json |
| data            | {"msg": "Hello Knative!"} |
```

The `data` property defines the cloud event payload which is a Json payload in the example above.

This can be any payload and depends on what you want to send as part of the event.

As we are using the Http cloud event model we can also use Http property equivalents as property keys.

Send cloud event via Http properties

```
When send Knative event
| ce-specversion     | 1.0 |
| ce-type            | greeting |
| ce-source          | https://github.com/citrusframework |
| ce-subject         | hello |
| ce-id              | say-hello-${id} |
| Content-Type       | application/json;charset=UTF-8 |
| data               | {"msg": "Hello Knative!"} |
```

Instead of using a `data` property in the data table you can also specify the event payload in a separate step.

@Given("^Knative event data: {data}$")

```
Given Knative event data: {data}
```

The step sets a single line event data that is going to represent the payload of the cloud event.

The following step supports multiline event data.

@Given("^Knative event data$")

```
Given Knative event data
"""
<<data>>
"""
```

With these steps the cloud event data table must not specify the `data` property anymore.

Send cloud event

```
Given Knative event data
"""
{
  "msg": "Hello Knative!"
}
"""
Then send Knative event
| specversion     | 1.0 |
| type            | greeting |
| source          | https://github.com/citrusframework |
| subject         | hello |
| id              | say-hello |
| datacontenttype | application/json |
```

Create cloud events via Json

The cloud events model supports Json so you can also specify the event with a single step in Json.

@When("^(?:create|send) Knative event as json$")

```
When send Knative event as json
"""
<<json>>
"""
```

Send cloud event via Json

```
When send Knative event as json
"""
{
  "specversion" : "1.0",
  "type" : "greeting",
  "source" : "https://github.com/citrusframework",
  "subject" : "hello",
  "id" : "say-hello",
  "datacontenttype" : "application/json",
  "data" : "{\"msg\": \"Hello Knative!\"}"
}
"""
```

Producer timeouts

The producer connects to the Knative broker in order to publish events. In case the broker is not available a timeout will fail the test. You can adjust the producer timeout.

@Given("^Knative event producer timeout is {time}(?: ms| milliseconds)$")

```
Given Knative event producer timeout is {time} milliseconds
```

53.9.9. Receive events

In order to receive events from Knative you should setup a service or channel in combination with a trigger. The trigger watches for events on the broker and forwards these to the service or channel.

The test is able to receive events and verify its content.

Receive cloud events

@Then("^(?:receive|verify) Knative event$")

```
Then receive Knative event
| {property} | {value} |
```

The step uses a data table in order to specify the cloud event properties as expected content. The cloud event data structure defines following properties:

- specversion
- type
- source
- subject
- id
- datacontenttype
- data

Following these properties you can specify the cloud event in the receive operation.

Receive cloud event

```
Then receive Knative event
| specversion     | 1.0 |
| type            | greeting |
| source          | https://github.com/citrusframework |
| subject         | hello |
| id              | say-hello |
| datacontenttype | application/json |
| data            | {"msg": "Hello Knative!"} |
```

The `data` property defines the cloud event payload which is a Json payload in the example above. This can be any payload and depends on what you want to receive as part of the event.

As we are using the Http cloud event model we can also use Http property equivalents as property

keys.

Receive cloud event via Http properties

```
Then receive Knative event
| ce-specversion     | 1.0 |
| ce-type            | greeting |
| ce-source          | https://github.com/citrusframework |
| ce-subject         | hello |
| ce-id              | say-hello-${id} |
| Content-Type       | application/json;charset=UTF-8 |
| data               | {"msg": "Hello Knative!"} |
```

Instead of using a `data` property in the data table you can also specify the event payload in a separate step.

@Then("^(?:expect|verify) Knative event data: {data}$")

```
Then expect Knative event data: {data}
```

The step sets a single line event data that is going to represent the payload of the cloud event.

The following step supports multiline event data.

@Then("^(?:expect|verify) Knative event data$")

```
Then expect Knative event data
"""
<<data>>
"""
```

With these steps the cloud event data table must not specify the `data` property anymore.

Receive cloud event

```
Given expect Knative event data
"""
{
  "msg": "Hello Knative!"
}
"""
Then receive Knative event
| specversion     | 1.0 |
| type            | greeting |
| source          | https://github.com/citrusframework |
| subject         | hello |
| id              | say-hello |
| datacontenttype | application/json |
```

Receive cloud events via Json

The cloud events model supports Json so you can also specify the event with a single step in Json.

@When("^(?:receive|verify) Knative event as json$")

```
Then receive Knative event as json
"""
<<json>>
"""
```

Receive cloud event via Json

```
Then receive Knative event as json
"""
{
  "specversion" : "1.0",
  "type" : "greeting",
  "source" : "https://github.com/citrusframework",
  "subject" : "hello",
  "id" : "say-hello",
  "datacontenttype" : "application/json",
  "data" : "{\"msg\": \"Hello Knative!\"}"
}
"""
```

Consumer timeouts

The consumer connects to the Knative broker in order to consume events. The consumer will wait for events and in case no event arrives in time a timeout will fail the test. You can adjust this event consumer timeout.

@Given("^Knative event consumer timeout is {time}(?: ms| milliseconds)$")

```
Given Knative event consumer timeout is {time} milliseconds
```

53.9.10. Manage Knative resources

The described steps are able to create Knative resources on the current Kubernetes namespace. By default these resources get removed automatically after the test scenario.

The auto removal of Knative resources can be turned off with the following step.

@Given("^Disable auto removal of Knative resources$")

```
Given Disable auto removal of Knative resources
```

Usually this step is a `Background` step for all scenarios in a feature file. This way multiple scenarios can work on the very same Knative resources and share integrations.

There is also a separate step to explicitly enable the auto removal.

@Given("^Enable auto removal of Knative resources$")

```
Given Enable auto removal of Knative resources
```

By default, all Knative resources are automatically removed after each scenario.

53.10. OpenAPI steps

OpenAPI documents specify RESTful Http services in a standardized, language-agnostic way. The specifications describe resources, path items, operations, security schemes and many more components. All these components specified are part of a REST Http service.

Citrus as a framework is able to use this information in an OpenAPI document in order to generate proper request and response data for your test.

```
Feature: Petstore API V3

  Background:
    Given OpenAPI specification: http://localhost:8080/petstore/v3/openapi.json

  Scenario: getPet
    When invoke operation: getPetById
    Then verify operation result: 200 OK

  Scenario: petNotFound
    Given variable petId is "0"
    When invoke operation: getPetById
    Then verify operation result: 404 NOT_FOUND

  Scenario: addPet
    When invoke operation: addPet
    Then verify operation result: 201 CREATED

  Scenario: updatePet
    When invoke operation: updatePet
    Then verify operation result: 200 OK

  Scenario: deletePet
    When invoke operation: deletePet
    Then verify operation result: 204 NO_CONTENT
```

53.10.1. Load OpenAPI specifications

The test is able to load OpenAPI specifications via Http URL or the local file system. When loaded into the test steps can make use of all available operations in the specification.

@Given("^OpenAPI (?:specification|resource): {url}$")

```
Given OpenAPI specification {url}
```

The given url can point to a local file on the file system or to a Http endpoint. The step loads the OpenAPI specification so all operations are ready to be used.

53.10.2. Invoke operations

You can invoke operations as a client by referencing the operation name given in the specification. Citrus loads the operation from the specification and automatically generates proper request/response data for you.

The rules in the OpenAPI specification define how to generate proper test data with randomized values.

@When("^(?:I|i)nvoke operation: {id}$")

```
When invoke operation: {id}
```

The step obtains the operation with the given `{id}` and invokes it on the Http URL that is given in the specification. In case the server URL is missing in the specification the step uses the base URL of the OpenAPI endpoint where the document has been loaded from.

The step uses the specification rules for the operation to generate a proper request for you. Request parameters, headers and body content are automatically generated. In case the operation defines a request body the step will also generate it with randomized values.

Generated body example with randomized values

```
{
  "id": 26866048,
  "name": "mGNTgkfxgg",
  "photoUrls": [
    "XHAGIyFcyh"
  ],
  "category": {
    "name": "konwOUYwMo",
    "id": 18676332
  },
  "tags": [
    {
      "name": "KDnoWCfUBn",
      "id": 31444049
    }
  ],
  "status": "sold"
}
```

The generated request should be valid according to the rules in the OpenAPI specification. You can overwrite the randomized values with test variables and inbound/outbound data dictionaries in order to have more human-readable test data.

Now that the test has sent the request you can verify the operation result in a next step.

53.10.3. Verify operation result

The test is able to verify the response status code returned by the server.

@Then("^(?:V|v)erify operation result: {status_code}(?: {reason_phrase})?$")

```
Then verify operation result: {status_code} {reason_phrase}
```

The step expects a `{status_code}` (e.g. 200, 404, 500) and optionally gives the `{reason_phrase}` (e.g. OK, NOT_FOUND, INTERNAL_SERVER_ERROR). Thee reason phrase is optional and is only for better readability reasons.

The operation defines a set of responses in the OpenAPI specification. The step tries to find the response with the given `{status_code}` and reads the given rules such as response body, headers etc. Based on the response definition in the OpenAPI specification the step automatically verifies the server reponse and makes sure that the response matches the given rules.

In particular the step generates an expected response body (if any is specified) and compares the actual response with the generated one.

Generated response body example with validations

```
{
  "id": "@isNumber()@",
  "name": "@notEmpty()@",
  "photoUrls": "@ignore@",
  "category": {
    "id": "@isNumber()@",
    "name": "@notEmpty()@"
  },
  "tags": "@ignore@",
  "status": "@matches(available|pending|sold)@"
}
```

The generated response makes use of Citrus validation matchers based on the rules in the specification. Id values are validated with `@isNumber()@`, String values should not be empty `@notEmpty()@` and enumeration values are checked with `@matches(value_1|value_2|…|value_x)@`.

The received response must match all these validation matchers. In addition to that a Json schema validation is performed on the response.

53.10.4. Verify operation requests

On the server side your test is able to verify incoming requests based on the rules in the specification. The test references a given operation by its name in the specification and generates proper verification steps on the request content. Based on the specification the incoming request must follow the rules and schemas attached to the OpenAPI operation.

@Then("^(?:receive|expect|verify) operation: {id}$")

```
Then expect operation {id}
```

The step expects a request matching the operation with the given `{id}`. The step loads the operation from the specification and autoamtically verifies that the incoming request matches the specified request.

In fact the step generates a request body (if any is specified on the operation) with validation expressions and compares the incoming request with the generated template. In case the incoming request does not match the generated validation rules the test will fail accordingly.

Generated request body example with validations

```
{
  "id": "@isNumber()@",
  "name": "@notEmpty()@",
  "photoUrls": "@ignore@",
  "category": {
    "id": "@isNumber()@",
    "name": "@notEmpty()@"
  },
  "tags": "@ignore@",
  "status": "@matches(available|pending|sold)@"
}
```

The generated request uses Citrus validation matchers based on the rules in the OpenAPI specification. Identifiers are validated with `@isNumber()@`, String values should not be empty `@notEmpty()@` and enumeration values are checked with `@matches(value_1|value_2|⋯|value_x)@`.

This way you can make sure that the incoming request matches the rules defined in the OpenAPI specification. In addition to the message request body verification the step will automatically verify `Content-Type` headers, path parameters, query parameters and the general request path used.

53.10.5. Send operation response

When the OpenAPI step has received and verified a request it is time to respond with proper message content. The given operation in the OpenAPI specification is able to define multiple response messages that are valid. In the test the user picks one of these response messages and the step generates the message content based on the specification.

The step will generate proper test data with randomized values as response body.

@Then("^send operation result: {status} {reason_phrase}$")

```
Then send operation result: 201 CREATED
```

The step obtains the operation response with the status `201` and generates the response data. The response is able to define `Content-Type` headers and response body content.

Generated response body example with randomized values

```
{
  "id": 26866048,
  "name": "mGNTgkfxgg",
  "photoUrls": [
    "XHAGIyFcyh"
  ],
  "category": {
    "name": "konwOUYwMo",
    "id": 18676332
  },
  "tags": [
    {
      "name": "KDnoWCfUBn",
      "id": 31444049
    }
  ],
  "status": "sold"
}
```

The generated response should be valid according to the rules in the OpenAPI specification. You can overwrite the randomized values with test variables and inbound/outbound data dictionaries in order to have more human-readable test data.

53.10.6. Generate test data

The Citrus OpenAPI steps use the information in the specification to generate proper message content for requests and responses. The generated test data follows the schemas attached to the operations and response definitions. By default, the steps include optional fields when generating and validating message contents.

You can disable the optional fields in generation and validation accordingly:

@Given("^Disable OpenAPI generate optional fields$")

```
Given Disable OpenAPI generate optional fields
```

@Given("^Disable OpenAPI validate optional fields$")

```
Given Disable OpenAPI validate optional fields
```

With this setting the OpenAPI steps will exclude optional fields from both test data generation and message content validation.

53.10.7. Inbound/outbound data dictionaries

Data dictionaries are a good way to make generated randomized values more human-readable. By default, Citrus generates random values based on the specifications in the OpenAPI document. You can overwrite the basic generation rules by specifying rules in a data dictionary.

Outbound dictionary

Outbound dictionaries are used to customize generated client requests.

@Given("^OpenAPI outbound dictionary$")

```
Given OpenAPI outbound dictionary
| {expression} | {value} |
```

The outbound dictionary holds a list of expressions that overwrite values in the generated request body.

Based on the body data format (e.g. Json or XML) you can use JsonPath or XPath expressions in the dictionary. Citrus evaluates the given expressions on the generated request body before the request is sent to the server.

Outbound dictionary sample

```
Given OpenAPI outbound dictionary
  | $.name          | citrus:randomEnumValue('hasso','cutie','fluffy') |
  | $.category.name | citrus:randomEnumValue('dog', 'cat', 'fish') |
```

You can also load the dictionary rules from an external file resource.

@Given("^load OpenAPI outbound dictionary {file_path}$")

```
Given load OpenAPI outbound dictionary {file_path}
```

With this outbound data dictionary in place a generated request can look like follows:

Generated request with outbound dictionary

```
{
  "id": 12337393,
  "name": "hasso",
  "photoUrls": [
    "aaKoEDhLYc"
  ],
  "category": {
    "name": "cat",
    "id": 23927231
  },
  "tags": [
    {
      "name": "FQxvuCbcqT",
      "id": 58291150
    }
  ],
  "status": "pending"
}
```

You see that the request now uses more human readable values for `$.name` and `$.category.name`.

The same mechanism applies for inbound messages that are verified by Citrus. The framework will generate an expected response data structure coming from the OpenAPI specification.

Inbound dictionary

Inbound dictionaries adjust the generated expected responses which verify incoming messages with expected validation statements.

@Given("^OpenAPI inbound dictionary$")

```
Given OpenAPI inbound dictionary
| {expression} | {value} |
```

You can also load the dictionary rules from an external file resource.

@Given("^load OpenAPI inbound dictionary {file_path}$")

```
Given load OpenAPI inbound dictionary {file_path}
```

The inbound dictionary holds a list of expressions that overwrite values in the generated response body.

Based on the body data format (e.g. Json or XML) you can use JsonPath or XPath expressions in the dictionary. Citrus evaluates the given expressions on the generated response body. This way you can overwrite given values in the body structure before the validation takes place.

Inbound dictionary sample

```
Given OpenAPI inbound dictionary
  | $.name          | @assertThat(anyOf(is(hasso),is(cutie),is(fluffy)))@ |
  | $.category.name | @assertThat(anyOf(is(dog),is(cat),is(fish)))@ |
```

Below is a sample Json payload that has been generated with the inbound data dictionary.

Generated response with inbound dictionary

```
{
  "id": "@isNumber()@",
  "name": "@assertThat(anyOf(is(hasso),is(cutie),is(fluffy)))@",
  "photoUrls": "@ignore@",
  "category": {
    "name": "@assertThat(anyOf(is(dog),is(cat),is(fish)))@",
    "id": "@isNumber()@"
  },
  "tags": "@ignore@",
  "status": "@matches(available|pending|sold)@"
}
```

The generated response ensures that the rules defined in the OpenAPI specification do match and in addition that the received data meets our expectations in the dictionary.

In case you need to have a more specific response validation where each field gets validated with an expected value please consider using the Http steps in Citrus. Here you can provide a complete expected Http response with body and headers.

53.10.8. Request fork mode

When the OpenAPI steps fire requests to the server the step synchronously waits for the response. All other steps are in the feature are blocked by the synchronous communication. In some cases this is a problem because you might want to run some steps in parallel to the synchronous communication.

In these cases you can make use of the form mode when sending Http client requests.

@Given("^OpenAPI request fork mode is (enabled|disabled)$")

```
Given OpenAPI request fork mode is enabled
```

With this in place the step will not block other steps while waiting for the synchronous response from the server. The feature will continue with the next steps when fork mode is enabled. At a later point in time you may verify the response as usual with the separate verification step.

53.11. Playwright steps

Playwright is a modern browser automation toolkit that drives Chromium, Firefox and WebKit through a single API. Citrus provides ready-to-use Cucumber steps that leverage the `connectors/citrus-playwright` browser endpoint with Cloud-native BDD, mirroring the Selenium steps phrasing where it makes sense.

Add the module to your project:

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-cucumber-playwright</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

The glue package `org.citrusframework.cucumber.steps.playwright` is discovered automatically when the module is on the test classpath.

53.11.1. Browser selection and lifecycle

Citrus creates a fallback browser from the settings below when no `PlaywrightBrowser` bean is configured. You can also declare the endpoint yourself and reference it by name.

@Given("^(?:Playwright|playwright) browser \"([^\"]+)\"$")

```
Given Playwright browser "shopBrowser"
```

This loads the `PlaywrightBrowser` component with the given name from the configuration (e.g. Spring application context). Unknown ids fail with `CitrusRuntimeException: Unable to find playwright browser for id: …`.



lifecycle and selection phrases carry a `playwright` qualifier on purpose: the Selenium steps module defines `Browser "…"`, `start browser` and `stop browser` with different meaning, and Cucumber-JVM rejects identical expressions when both glues share a classpath (e.g. via `citrus-cucumber-all`).

With several named browsers configured and none matching the default name, pick one explicitly with this step — no default browser is auto-created while any bean exists, and any other step fails fast naming the available ids.

Start and stop

```
When start playwright browser
When stop playwright browser
```

53.11.2. Settings

Property	Env	Default
citrus.playwright.browser.name	CITRUS_PLAYWRIGHT_BROWSER_NAME	playwrightBrowser
citrus.playwright.browser.type	CITRUS_PLAYWRIGHT_BROWSER_TYPE	chromium
citrus.playwright.headless	CITRUS_PLAYWRIGHT_HEADLESS	true
citrus.playwright.base.url	CITRUS_PLAYWRIGHT_BASE_URL	(unset)
citrus.playwright.default.timeout.ms	CITRUS_PLAYWRIGHT_DEFAULT_TIMEOUT_MS	30000

53.11.3. Navigation

```
When user opens "https://example.com/login"
```

Every value supports Citrus variables (`${baseUrl}/login`).

53.11.4. Element interaction

Locators are CSS selectors by default. Every element step also accepts an explicit locator type (`css`, `xpath`, `text`, `role` or `test-id`):

```
When user clicks "#login"
When user clicks xpath "//button[@id='go']"
When user clicks role "button" named "Login"
When user fills "alice" to "#username"
When user fills "alice" to text "Username"
When user enters text "${who}" to "#username"
When user clears "#username"
When user clears xpath "//input[@id='name']"
When user presses key "Enter" on "#username"
When user checks "#remember"
When user unchecks test-id "remember"
When user selects option "blue" on "#color"
When user hovers over "#menu"
When user hovers over role "menu"
```

Accessible-name selection (`role "…" named "…"`) is available on the click, fill and verification steps; all other steps accept a bare `role "…"`.

53.11.5. Verification

```
Then browser should display element "#welcome"
Then browser should display element test-id "welcome"
Then element "#welcome" should have text "Hello"
Then element xpath "//h1" should have text "Hello"
Then page title should be "Shop"
Then page url should be "https://example.com/shop"
```

A mismatch raises a validation error naming the check, locator and expected/actual values.

53.11.6. Page objects

Page-object classes are instantiated by name and bridged to the Playwright `pageObject()` action. Methods may take `String` arguments (resolved through Citrus variables) and can declare `Page`, `BrowserContext`, `PlaywrightBrowser` or `TestContext` parameters, which Citrus injects automatically.

```
When playwright page com.example.LoginPage performs submit
When playwright page com.example.LoginPage performs setUserName with arguments
  | alice |
Then playwright page com.example.LoginPage should validate with
com.example.LoginValidator
```

Page-object phrases carry the same `playwright` qualifier: the Selenium module defines `browser page … performs …` for pre-registered page ids, while these steps load a class by name.

The validator must implement `PlaywrightPageValidator` and provide a no-argument constructor; the page type should implement `PlaywrightPage` (validators receive it as such). `performs` steps accept any class whose methods take `String` arguments and/or injected `Page`, `BrowserContext`, `PlaywrightBrowser` or `TestContext` parameters. Unknown class names fail with a `CitrusRuntimeException` naming the type.

53.11.7. Dialogs and screenshots

```
When user accepts dialog
When user dismisses dialog
When take screenshot "target/shot.png"
```



dialog steps register a one-shot handler for the next dialog. Place the step before the action that opens the dialog in the same scenario.

53.12. Selenium steps

Selenium is a very popular UI testing tool that enables you to simulate user interactions on a web frontend. Selenium supports a wide range of browsers and is able to run with a grid of nodes in a Kubernetes environment.

Citrus provides ready-to-use steps that leverage Selenium UI testing with Cloud-native BDD.

The following sections guide you through the Selenium capabilities in Citrus.

53.12.1. Browser type

Selenium supports many browsers. When running a Citrus test with Selenium you need to choose which browser type to use. By default, Citrus uses the `htmlunit` browser type. You can overwrite this type with the system property `citrus.selenium.browser.type` or environment variable `CITRUS_SELENIUM_BROWSER_TYPE` set on the Citrus configuration.

Set Selenium browser type

```
CITRUS_SELENIUM_BROWSER_TYPE=chrome
```

These are the supported browser types:

- firefox
- safari
- MicrosoftEdge
- safariproxy
- chrome
- htmlunit
- internet explorer
- phantomjs

This sets the Selenium browser type for all operations.

53.12.2. Selenium broker

Citrus creates a default browser component using the given browser type. You can also create a browser instance by yourself and reference this in your test by its name.

@Given("^(?:Browser|browser) "{name}"$")

```
Given browser "{name}"
```

This loads the browser component with given name from the configuration (e.g. Spring application context).

You can also use the system property `citrus.selenium.browser.name` environment variable `CITRUS_SELENIUM_BROWSER_NAME`

53.12.3. Remote web driver

As soon as the Citrus test is run on a Kubernetes environment you must use a remote web driver.

This is because the default Citrus test runtime pod is not able to run UI tests because no web browser infrastructure is installed in the image.

You can automatically start a Selenium sidecar container as part of the test runtime pod. The Selenium images provide the required web browser infrastructure for UI testing. Citrus adds the Selenium container as a sidecar to the test runtime. Now your test is able to connect with the remote browser using a remote web driver connection.

Add the following configuration to the `citrus-config.yaml` in your test.

citrus-config.yaml

```
config:
  runtime:
    selenium:
      image: selenium/standalone-chrome:latest
    env:
      - name: CITRUS_SELENIUM_BROWSER_TYPE
        value: chrome
      - name: CITRUS_SELENIUM_BROWSER_REMOTE_SERVER_URL
        value: http://localhost:4444/wd/hub
```

This sets the Selenium container image (`selenium/standalone-chrome:latest`), the browser type and the remote web driver URL. Citrus automatically starts the given Selenium container image and connects the browser to the remote web driver.

This way you can run the UI tests in a Kubernetes environment.

53.12.4. Start/stop browser

Before you can run some user interactions with Selenium you need to start the browser.

@Given("^start browser$")

```
Given start browser
```

@Given("^stop browser$")

```
Given stop browser
```

53.12.5. Navigate to URL

See the following step that navigates to a given URL in the browser.

@When("^(?:User|user) navigates to "{url}"$")

```
When User navigates to "{url}"
```

53.12.6. Verify elements on page

Selenium is able to verify web elements on the current page. You can select the element by its attributes (e.g. id, name, style) and verify its properties (e.g. text value, styles).

@When("^(?:Browser|browser) page should display {element-type} with {attribute}="{value}"$")

```
When Browser page should display {element-type} with {attribute}="{value}"
```

This step verifies that the given element of type `{element-type}` is present on the current web page. The element is selected by the given `{attribute}` and `{value}`.

Verify heading with text

```
When Browser page should display heading with text="Welcome!"
```

Possible element types are:

- element
- button
- link
- input
- textfield
- form
- heading

Possible attributes to select the element are:

- id
- name
- class-name
- link-text
- css-selector
- tag-name
- xpath

You can add additional attribute validations in a data table

Verify element with attributes

```
When And browser page should display element with id="hello-text" having
    | text   | Hello!         |
    | styles | background-color=rgba(0, 0, 0, 0) |
```

53.12.7. Click elements

You can click on elements such as buttons or links. The element must be identified by an attribute (e.g. id, name, style) with a given value.

@When("^(?:User|user) clicks (?:element|button|link) with {attribute}="{value}"$")

```
When User clicks (element|button|link) with {attribute}="{value}"
```

Click button by id

```
When User clicks button with id="submit"
```

53.12.8. Form controls

Filling out a user form on a web page is a very common use case in UI testing. Citrus is able to enter text into input fields, select items from a drop down list and check/uncheck checkboxes.

Input fields

@When("^(?:User|user) enters text "{input}" to (?:element|input|textfield) with {attribute}="{value}"$")

```
When User enters text "{input}" to (element|input|textfield) with
{attribute}="{value}"
```

Enter text in input field

```
When User enters text "Christoph" to input with id="name"
```

Checkboxes

@When("^(?:User|user) (checks|unchecks) checkbox with {attribute}="{value}"$")

```
When User (checks|unchecks) checkbox with {attribute}="{value}"
```

Check checkbox

```
When User checks checkbox with id="show-details"
```

Dropdowns

@When("^(?:User|user) selects option "{option}" on (?:element|dropdown) with {attribute}="{value}"$")

```
When User selects option "{option}" with {attribute}="{value}"
```

Check checkbox

```
When User selects option "21-30" on dropdown with id="age"
```

53.12.9. Alert dialogs

Web pages can open alert dialogs that need to be accepted or dismissed.

@When("^(?:User|user) (accepts|dismisses) alert$")

```
When User (accepts|dismisses) alert
```

You can also verify the alert text displayed to the user.

@When("^(?:Browser|browser) page should display alert with text "{text}"$")

```
When Browser page should display alert with text "{text}"
```

Verify alert with text

```
When Browser page should display alert with text "WARNING!"
```



The alert text verification implicitly accepts the alert dialog after validation.

53.12.10. Page objects

Selenium provides a good way to encapsulate web page capabilities in form of page objects. These object usually defines elements on a web page and perform predefined operations on that page.

Page object

```
public class UserFormPage implements WebPage {

    @FindBy(id = "userForm")
    private WebElement form;

    @FindBy(id = "username")
    private WebElement userName;

    /**
     * Sets the user name.
     * @param value
     */
    public void setUserName(String value) {
        userName.clear();
        userName.sendKeys(value);
    }

    /**
     * Submits the form.
     */
    public void submit() {
        form.submit();
    }
}
```

The page object above defines a `form` element as well as a `username` input text field. The page identifies the elements with `@FindBy` annotations. In addition, the page defines operations such as `setUserName` and `submit`.

Citrus is able to load the page objects by its name in the current configuration (e.g. Spring application context).

@Given("^(?:Browser|browser) page "{name}"$")

```
Given Browser page "{name}"
```

The step loads the page object that has been added to the configuration with the given name.

You can also instantiate new page objects by its types as follows:

@Given("^(?:Browser|browser) page "{name}" of type {type}$")

```
Given Browser page "{name}" of type {type}
```

Instantiate UserForm page

```
Given Browser page "userForm" of type org.sample.UserFormPage
```

This loads a new page object of type `org.sample.UserFormPage`. Please make sure that the given class is available on the test classpath and that the class provides a default constructor.

You can instantiate many web page objects in a single step.

Instantiate many page objects

```
Given Browser page types
  | indexPage | org.sample.IndexPage     |
  | userForm  | org.sample.UserFormPage  |
  | orderForm | org.sample.OrderFormPage |
```

Once the page objects are loaded you can perform operations.

@Given("^(?:Browser|browser) page {name} performs {operation}$")

```
Given Browser page {name} performs {operation}
```

Call submit operation on userForm

```
Given Browser page userForm performs submit
```

The step uses the given page object `userForm` and performs the `submit` operation. This simply calls the `submit()` method on the page object.

Page object

```
public class UserFormPage implements WebPage {

    @FindBy(id = "userForm")
    private WebElement form;

    /**
     * Submits the form.
     */
    public void submit() {
        form.submit();
    }
}
```

In case the operation requires parameters you can set those on the operation call.

Call setUserName operation with arguments

```
Given Browser page userForm performs setUserName with arguments
  | Christoph |
```

The `setUserName` operation on the page object requires the username value as a parameter. This

value is set as `Christoph` in the step above.

Page object

```
public class UserFormPage implements WebPage {

    @FindBy(id = "username")
    private WebElement userName;

    /**
     * Sets the user name.
     * @param value
     */
    public void setUserName(String value) {
        userName.clear();
        userName.sendKeys(value);
    }
}
```

Each page operation can use the current `TestContext` as argument, too. This context will be automatically injected by Citrus when the operation is called.

Use test context in page objects

```
public class UserFormPage implements WebPage {

    @FindBy(id = "username")
    private WebElement userName;

    /**
     * Sets the user name.
     * @param value
     * @param context
     */
    public void setUserName(String value, TestContext context) {
        userName.clear();
        userName.sendKeys(value);

        context.setVariable("username", value);
    }
}
```

The page operation `setUserName` uses the `TestContext` as additional method argument and is able to set a new test variable. All subsequent steps in the test are able to access this new variable with `${username}` then.

53.12.11. Page validator

The previous section has introduced the concept of page objects and how to perform operations on the given page. You can also use page objects to verify the page contents.

Page validator

```
public class UserFormPageValidator implements PageValidator<UserFormPage> {

    @Override
    public void validate(UserFormPage webPage, SeleniumBrowser browser, TestContext
context) {
        Assert.assertNotNull(webPage.userName);

Assert.assertTrue(StringUtils.hasText(webPage.userName.getAttribute("value")));
        Assert.assertNotNull(webPage.form);
    }
}
```

The page validator implements the `PageValidator<>` interface and implements a `validate` method. The validation is provided with the actual page object, the browser instance and the current test context.

The validator should verify that the current state on the page is as expected.

Citrus is able to load the page validator by its name in the current configuration (e.g. Spring application context).

@Given("^(?:Browser|browser) page validator "{name}"$")

```
Given Browser page validator "{name}"
```

The step loads the page validator that has been added to the configuration with the given name.

You can also instantiate new page validator objects by its types as follows:

@Given("^(?:Browser|browser) page validator "{name}" of type {type}$")

```
Given Browser page validator "{name}" of type {type}
```

Create page validator

```
Given Browser page validator "userFormValidator" of type
org.sample.UserFormPageValidator
```

This loads a new page validator of type `org.sample.UserFormPageValidator`. Please make sure that the given class is available on the test classpath and that the class provides a default constructor.

You can instantiate many web page validator objects in a single step.

Instantiate many page validator objects

```
Given Browser page validator types
  | indexPageValidator | org.sample.IndexPageValidator     |
  | userFormValidator  | org.sample.UserFormPageValidator  |
  | orderFormValidator | org.sample.OrderFormPageValidator |
```

Once the page validator objects are loaded you can perform its validations.

@Given("^(?:Browser|browser) page {name} should validate with {validator}$")

```
Given Browser page {name} should validate with {validator}
```

Validate userForm page with userFormValidator

```
Given Browser page userForm should validate with userFormValidator
```

The step calls the `validate` method on the page validator `userFormValidator` and passes the `userForm` page object as argument.

Page validator

```
public class UserFormPageValidator implements PageValidator<UserFormPage> {

    @Override
    public void validate(UserFormPage webPage, SeleniumBrowser browser, TestContext
context) {
        Assert.assertNotNull(webPage.userName);

Assert.assertTrue(StringUtils.hasText(webPage.userName.getAttribute("value")));
        Assert.assertNotNull(webPage.form);
    }
}
```

The validator accesses the elements and operations provided in the page object and makes sure the state is as expected.



The page object itself can also implement the page validator interface. This way you can combine the concept of page objects and validator in a single class. The step to verify the page is then able to just use the page object name.

Validate userForm page with implicit validator

```
Given Browser page userForm should validate
```

Page object implementing validator

```
public class UserFormPage implements WebPage, PageValidator<UserFormPage> {

    @FindBy(id = "userForm")
    private WebElement form;

    @FindBy(id = "username")
    private WebElement userName;

    [...]

    @Override
    public void validate(UserFormPage webPage, SeleniumBrowser browser, TestContext
context) {
        Assert.assertNotNull(userName);
        Assert.assertTrue(StringUtils.hasText(userName.getAttribute("value")));
        Assert.assertNotNull(form);
    }
}
```

53.13. Testcontainers steps

Citrus provides steps that allow to leverage the power of Testcontainers library as part of Cloud native tests. You can configure and start Testcontainers modules as pod in the current namespace. Testcontainers provides a great set of ready-to-use containers (e.g. database containers, AWS cloud stack and many more).

53.13.1. Setup required privileges

Running Testcontainers in Kubernetes may require privileges on the Citrus service account. This depends on the restrictions and RBAC settings in your cluster.

Be sure to add `privileged` security context constraint to the `citrus-viewer` service account in the namespace where tests are run.

Add privileged security context constraint

```
oc adm policy add-scc-to-user privileged system:serviceaccount:{namespace}:citrus-
viewer
```

This adds the `privileged` security constraint to the `citrus-viewer` service account. This is required for the test to start and manage a Testcontainer pod as part of the test.

53.13.2. Database containers

You can choose from a wide range of database modules provided in Testcontainers. Citrus is able to run these database containers as part of the test.

53.13.3. PostgreSQL container

Configure and manage a PostgreSQL database container.

@Given("^PostgreSQL version (^||s+)$")

```
Given PostgreSQL version 9.6.12
```

@Given("^start PostgreSQL container$")

```
Given start PostgreSQL container
```

53.13.4. MongoDB container

Configure and manage a MongoDB database container.

@Given("^MongoDB version (^||s+)$")

```
Given MongoDB version 4.10.0
```

@Given("^start MongoDB container$")

```
Given start MongoDB container
```

Chapter 54. Kaoto Visual Designer

Kaoto is an open-source visual designer for building integration flows. Starting with Kaoto 2.11 the designer includes full Citrus support, allowing you to create, edit, and run Citrus integration tests in a visual, low-code environment — right inside VS Code or in a browser.



54.1. Overview

The Kaoto editor renders Citrus YAML tests as visual flows on a canvas. Each test action appears as a step with a dedicated icon, making it easy to distinguish action types at a glance. You can browse the complete Citrus component catalog, configure endpoints and actions through guided property forms, and run tests directly from the editor.

Kaoto is aware of the complete Citrus component catalog: every test action, every endpoint type, every function and validation matcher is available for selection in the designer.

Key capabilities:

- Visual canvas with dedicated Citrus icons for each action type
- Full Citrus component catalog with 35+ endpoint types
- Schema-based property forms with input types, dropdowns, and contextual help
- Direct test execution from within the editor
- Tests are saved as standard Citrus YAML — fully portable and compatible with the broader Citrus ecosystem



Kaoto currently supports the Citrus YAML DSL. XML DSL support is planned for a future release.

54.2. Getting Started

There are two ways to use Kaoto with Citrus: the VS Code extension or the online playground.

54.2.1. VS Code Extension

Install the Kaoto extension from the VS Code marketplace:

1. Open VS Code

2. Go to the Extensions view (`Ctrl+Shift+X` / `Cmd+Shift+X`)
3. Search for **Kaoto** (publisher: Red Hat, extension ID `redhat.vscode-kaoto`)
4. Click **Install**

Alternatively, install from the command line:

```
code --install-extension redhat.vscode-kaoto
```

Once installed, create a new file with the `.citrus.yaml` extension and open it with the Kaoto editor. VS Code will automatically use Kaoto to render the file as a visual test flow.

54.2.2. Online Playground

If you want to try Kaoto without installing anything, use the online playground at https://kaotoio.github.io/kaoto/.

The playground runs entirely in your browser and supports the same Citrus catalog and visual editing features as the VS Code extension.

54.3. Creating Tests

To create a new Citrus test in Kaoto:

1. Create a file with the `.citrus.yaml` extension (e.g., `my-test.citrus.yaml`)
2. Open the file — the Kaoto editor shows an empty canvas
3. Add test actions from the catalog by clicking the + button on the canvas
4. Configure each action using the property form panel

The resulting YAML file is a standard Citrus test definition that can be executed with the Citrus YAML test runtime, JBang, or any other Citrus-compatible runner.

54.4. Component Catalog

The Kaoto editor integrates the full Citrus component catalog. When adding a new step to the canvas, you can browse and search for components across the following categories:

54.4.1. Test Actions

All core Citrus test actions are available, including:

- **send** / **receive** — send and receive messages on endpoints
- **echo** / **print** — log messages to the console
- **sleep** / **delay** — pause test execution
- **create-variables** — define test variables

- **assert** / **verify** — assert conditions
- **transform** — transform message payloads

54.4.2. Containers

Test action containers for controlling execution flow:

- **sequential** — run actions in order
- **parallel** — run actions concurrently
- **repeat** / **iterate** — loop over actions
- **conditional** — conditional execution
- **async** — asynchronous execution
- **timer** — time-based execution

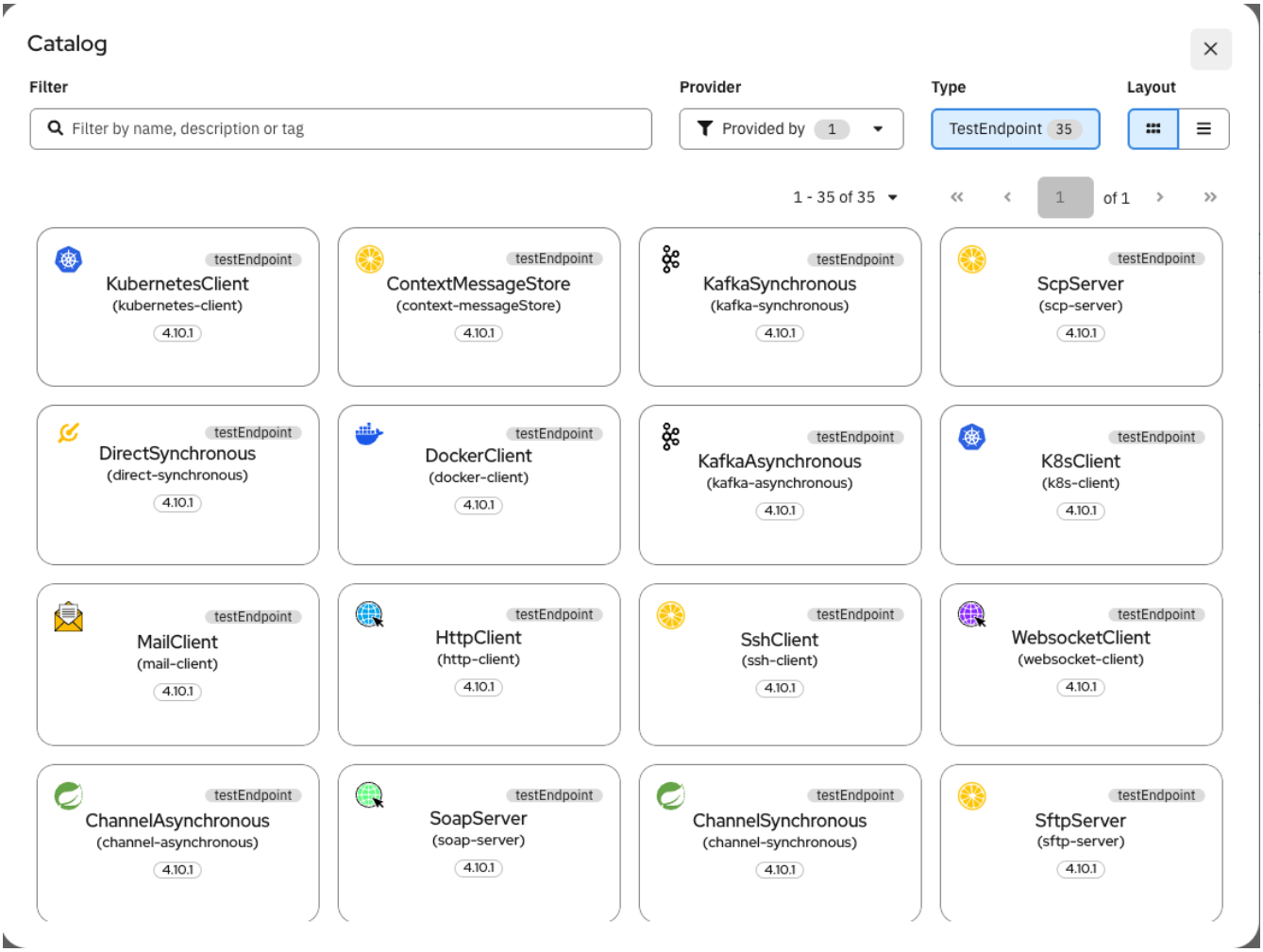
54.4.3. Test Endpoints

Kaoto includes 35+ Citrus endpoint types covering a wide range of protocols and transports:

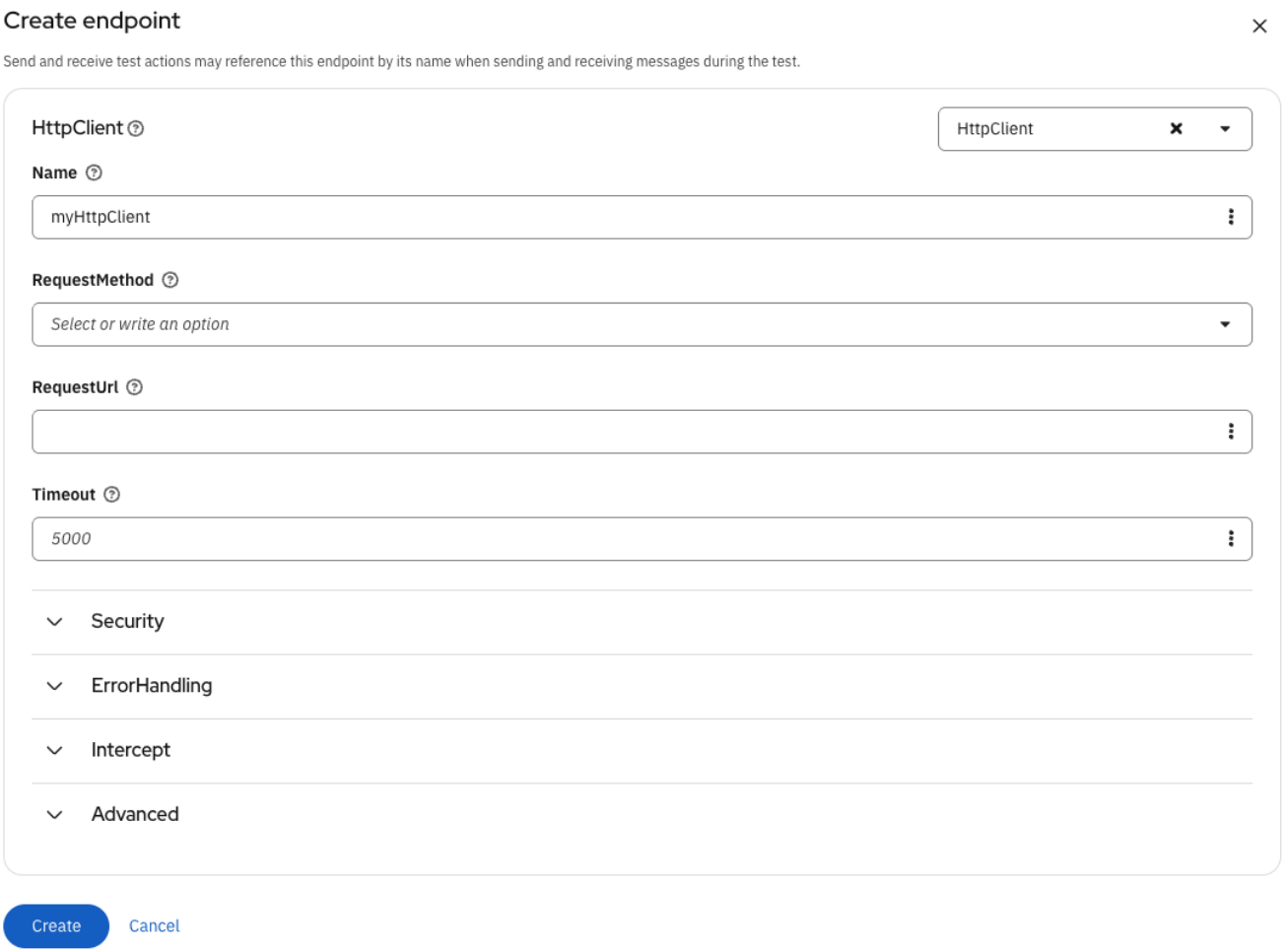
- **HTTP** — HTTP client and server endpoints
- **Kafka** — Apache Kafka messaging
- **JMS** — Java Message Service
- **SOAP/WS** — SOAP web services
- **FTP/SFTP** — file transfer
- **Mail** — email (SMTP/IMAP)
- **SSH** — secure shell
- **Kubernetes** — Kubernetes API
- **Docker** — Docker container management
- **SQL/JDBC** — database operations
- **Apache Camel** — Camel endpoint integration

And many more — each with its own guided configuration form.

You can pick the Citrus endpoint from the catalog in Kaoto.



With the endpoint schema that provides detailed information to all available properties and options Kaoto is able to render the property forms so you can easily configure the endpoint.



54.4.4. Functions and Validation Matchers

The catalog also includes Citrus functions (for generating dynamic test data) and validation matchers (for flexible message validation).

54.5. Schema-Based Property Forms

When you select a component on the canvas, Kaoto renders a tailored configuration form based on the component's JSON schema from the Citrus catalog. The forms provide:

- Proper input types for each property (text fields, numbers, booleans, dropdowns)
- Required field indicators
- Default values where applicable
- Contextual help text describing what each property does

This guided approach eliminates the need to scan documentation for property names or value formats — the editor tells you exactly what each component expects.

54.6. Running Tests

You can run Citrus tests directly from the Kaoto interface during development, keeping the feedback loop short. The test output is displayed in the editor, so you can immediately see whether

your test passes or fails.

Tests built in Kaoto are standard Citrus YAML files, so they can also be executed outside of Kaoto using any Citrus-compatible runner:

```
# Run with JBang Citrus app
citrus run my-test.citrus.yaml

# Run with Maven (if the project is set up with citrus-yaml runtime)
./mvnw verify
```

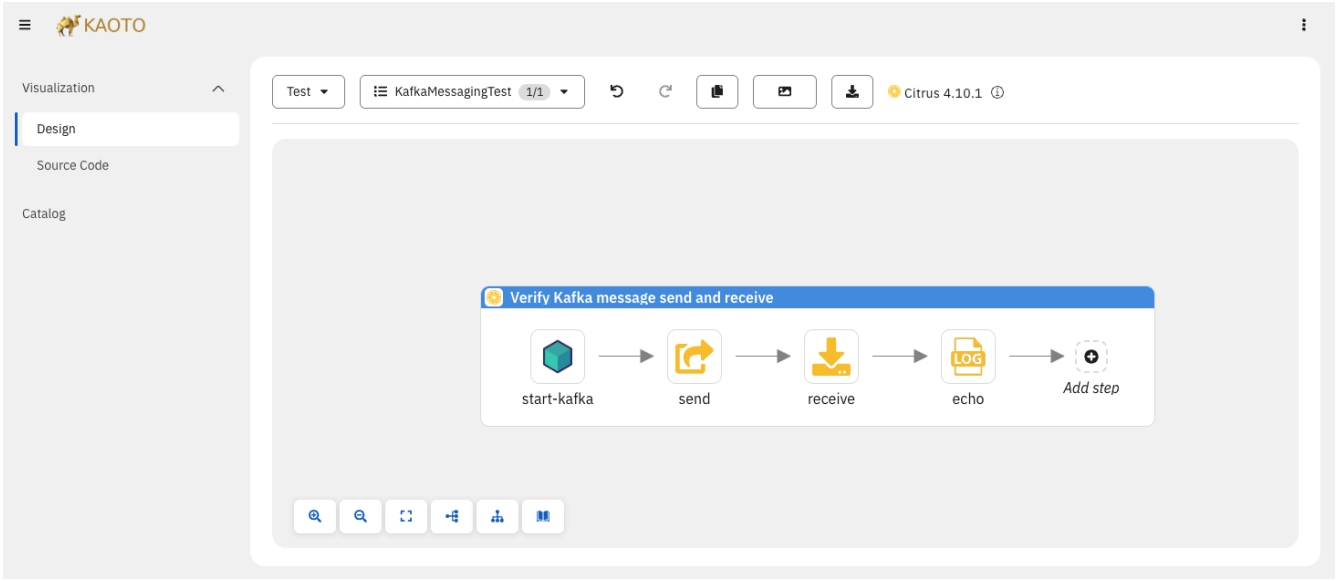
See JBang runtime and YAML runtime for more details on running YAML tests.

54.7. Kaoto Example

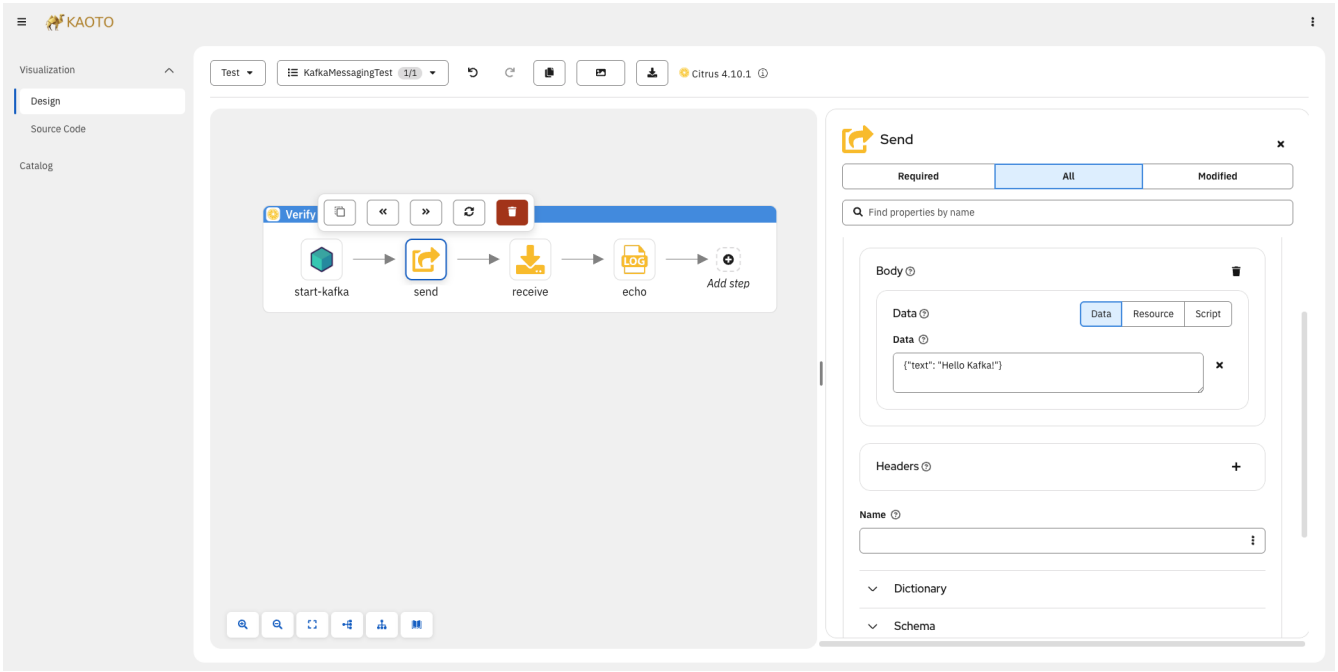
The following example shows a simple Citrus YAML test that can be visualized and edited in Kaoto. It starts a Kafka Testcontainers instance, sends a message to a Kafka topic, and verifies the received message.

```
name: KafkaMessagingTest
description: Verify Kafka message send and receive
actions:
  - testcontainers:
      start:
        kafka: {}
  - send:
      endpoint: "kafka:test-topic?server=${kafka.bootstrapServers}"
      message:
        body:
          data: |
            {"text": "Hello Kafka!"}
  - receive:
      endpoint: "kafka:test-topic?server=${kafka.bootstrapServers}"
      message:
        body:
          data: |
            {"text": "Hello Kafka!"}
  - echo:
      message: "Kafka messaging test completed!"
```

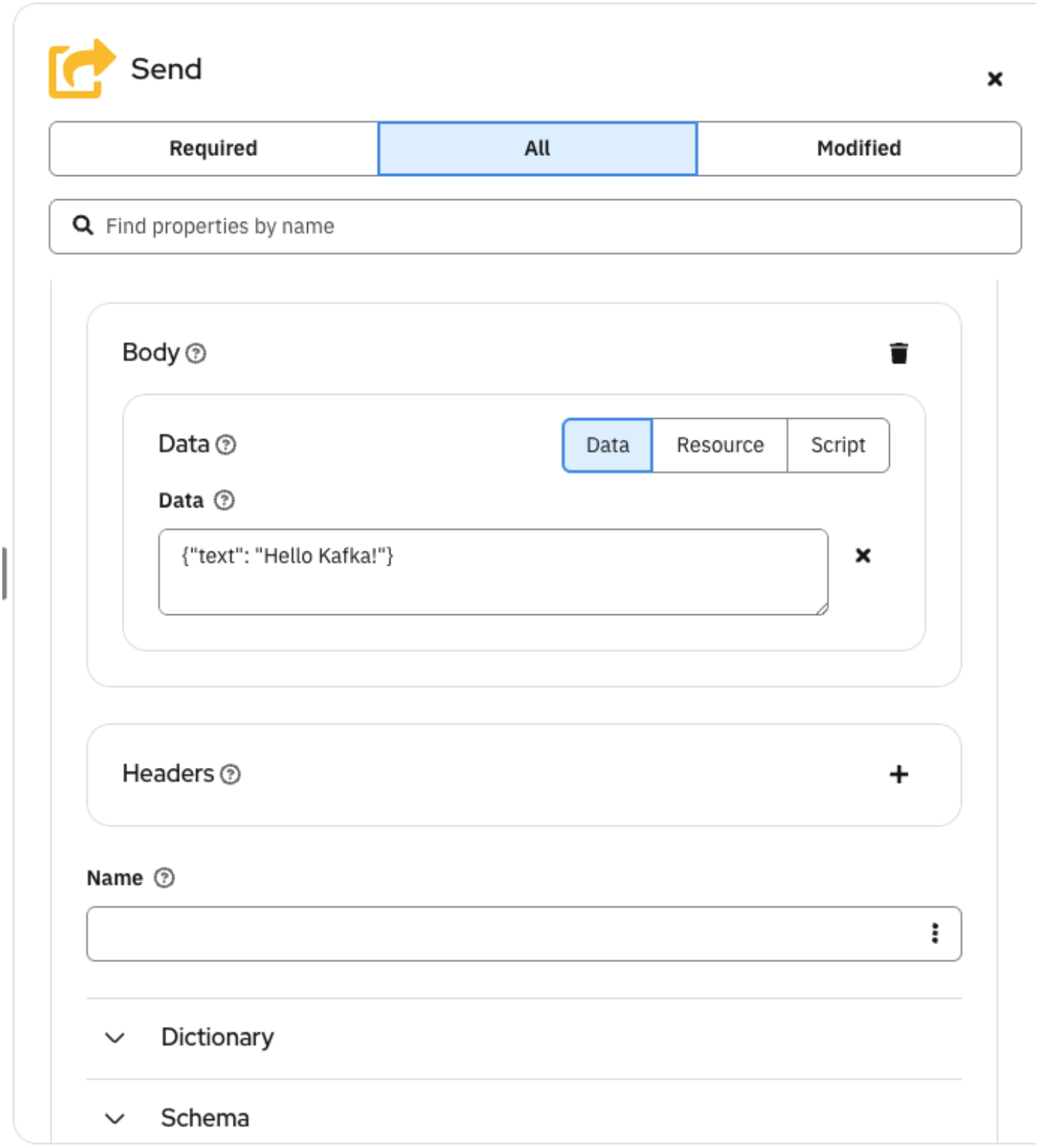
Open this file in Kaoto and each action renders as a visual step on the canvas. You can modify properties, add new actions, or rearrange the flow — all changes are reflected in the YAML file.



Select one of the test action items and see the individual property forms being generated from the test action schema.



You can adjust all available test action properties and Kaoto makes sure to render the proper YAML code for you.



54.8. Resources

- Kaoto project: https://kaoto.io/
- Kaoto VS Code extension: https://marketplace.visualstudio.com/items?itemName=redhat.vscode-kaoto
- Kaoto online playground: https://kaotoio.github.io/kaoto/
- Blog post: https://citrusframework.org/news/2026/06/19/kaoto-meets-citrus/ = Citrus MCP Server

The Citrus MCP Server is a Model Context Protocol (MCP) server implementation that enables Large Language Models (LLMs) and AI assistants to interact with Citrus framework metadata,

documentation, and schemas. This tool helps AI-powered development environments understand Citrus components and generate accurate test code.

54.9. Overview

The MCP server exposes Citrus framework capabilities through standardized tools and resources that can be consumed by AI assistants like Claude, GitHub Copilot, and other MCP-compatible clients. It provides programmatic access to:

- Catalog tools
  - **Catalog Information**: Query available components such as test actions, endpoints, functions, and validation matchers
  - **Component Information**: Query component details with module information and property specifications
- Schema resources
  - **Component Schemas**: Retrieve JSON schemas for Citrus components such as test actions, endpoints, functions with detailed property specifications
  - **DSL Schemas**: Get YAML and XML DSL schemas for writing tests
- Documentation tools
  - **Documentation index**: Query all available documentation files with filter criteria
  - **Documentation**: Access AsciiDoc documentation for specific Citrus components
- Migration resources
  - **Migration Guides**: Retrieve structured version migration guides with package renames, Maven dependency changes, API changes, and SPI changes
- Settings resources
  - **Settings Catalog**: Browse all available Citrus system property and environment variable settings by group or module

This enables AI assistants to provide context-aware suggestions, validate component usage, and generate syntactically correct Citrus tests.

54.10. What is MCP?

The Model Context Protocol (MCP) is an open protocol that enables AI applications to securely connect to external data sources and tools. MCP provides a standardized way for:

- **Tools**: Functions that the AI can call to perform actions or retrieve data
- **Resources**: Static or dynamic content that can be read by the AI
- **Prompts**: Reusable prompt templates for common tasks

The Citrus MCP Server implements this protocol using the Quarkus MCP Server framework, providing both STDIO and HTTP/SSE transport mechanisms.

For more information about MCP:

- MCP Specification: https://modelcontextprotocol.io/
- Quarkus MCP Server: https://docs.quarkiverse.io/quarkus-mcp-server/

54.11. Installation

The MCP server is packaged as an uber-JAR that can be run standalone or integrated with AI development tools.

54.11.1. Maven Dependency

If you need to add the MCP server module to your project:

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-mcp-server</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

54.11.2. Integrate with AI assistance tools

Install as a plugin

Claude Code can install the Citrus MCP server from the marketplace defined in the `citrusframework/citrus` repository, so you do not have to edit any configuration file by hand. JBang must still be installed and available on your PATH.

```
claude plugin marketplace add citrusframework/citrus
claude plugin install citrus-mcp@citrus-marketplace
```

The first command registers the marketplace; the second installs the `citrus-mcp` plugin, which ships the MCP server configuration. The server is launched on demand over STDIO via JBang.

Manual configuration

Alternatively, you can integrate the Citrus MCP server into AI code assistance tools using an MCP configuration file. The exact file name and location depends on the AI tool you are using (see Configuration file locations below).

The MCP server configuration looks like this:

```
{
  "mcpServers": {
    "citrus": {
      "command": "jbang",
        "args": [
          "--quiet",
          "org.citrusframework:citrus-mcp-server:${citrus.version}:runner"
        ]
      }
  }
}
```

Running the MCP server with JBang is the most convenient way to do this. The `--quiet` flag prevents JBang's output from interfering with the MCP protocol.



JBang supports the `LATEST` version placeholder. Use `org.citrusframework:citrus-mcp-server:LATEST:runner` to always fetch the latest available release from Maven Central without having to specify a concrete version number.



You can also run the server directly without using JBang - then it would be something like `java -jar <FULL PATH>/citrus-mcp-server-${citrus.version}-runner.jar`. JBang might be the simpler approach though because it automatically deals with downloading the JAR file from Maven central.

```
{
  "mcpServers": {
    "citrus": {
      "command": "java",
      "args": [
        "-jar",
        "/path/to/citrus-mcp-server-${citrus.version}-runner.jar"
      ]
    }
  }
}
```

Configuration file locations

Each AI tool has its own conventions for where the MCP configuration file lives. The table below shows both project and global locations for each tool. When adding the configuration to the project root directory the configuration applies only to sessions started in that project. Global configuration applies to all sessions for that AI tool (`~/` refers to your individual user home directory).

AI Tool	Project	Global
Claude Code	`.mcp.json` in project root directory	`~/.claude/mcp.json`

AI Tool	Project	Global
Claude Desktop	*n/a*	`~/Library/Application Support/Claude/claude_desktop_config.json` (macOS) `%APPDATA%\Claude\claude_desktop_config.json` (Windows)
IBM Bob	`.bob/mcp.json` in project root directory	`~/.bob/settings/mcp.json` or `~/.bob/settings/mcp_settings.json` (Bob 1.x)
VS Code / GitHub Copilot	`.vscode/mcp.json` in project root directory	VS Code user `settings.json` under the `"mcp"` key
JetBrains IDEs (2025.1+)	`.junie/mcp.json` in project root directory or in `Settings > Tools > AI Assistant > MCP Servers`	`~/.junie/mcp.json`



VS Code uses `"servers"` as the top-level key instead of `"mcpServers"`. See the example below.

VS Code .vscode/mcp.json example

```
{
  "servers": {
    "citrus": {
      "command": "jbang",
      "args": [
        "--quiet",
        "org.citrusframework:citrus-mcp-server:LATEST:runner"
      ]
    }
  }
}
```

54.11.3. Building from Source

You can build the Citrus MCP server uber-JAR locally with:

```
./mvnw clean package -pl tools/mcp-server
```

The resulting JAR will be located at `tools/mcp-server/target/citrus-mcp-server-${citrus.version}-runner.jar`.

54.12. Running the MCP Server

Any MCP-compatible client can connect to the Citrus MCP Server using either:

- **STDIO transport**: Execute the JAR as a subprocess and communicate via stdin/stdout

- **HTTP/SSE transport**: Connect to the HTTP endpoint at `/mcp`

54.12.1. STDIO Mode (Recommended for AI Assistants)

The server runs in STDIO mode by default, which is the standard transport for MCP clients like Claude Desktop:

Run with JBang

```
jbang --quiet org.citrusframework:citrus-mcp-server:${citrus.version}:runner
```

Java JAR runner

```
java -jar citrus-mcp-server-${citrus.version}-runner.jar
```

54.12.2. HTTP/SSE Mode

For web-based integrations, the server can also run with HTTP transport:

Run with JBang

```
jbang --quiet \
      -Dquarkus.http.host-enabled=true \
      -Dquarkus.http.port=8080 \
 org.citrusframework:citrus-mcp-server:${citrus.version}:runner
```

Java JAR runner

```
java -Dquarkus.http.host-enabled=true \
     -Dquarkus.http.port=8080 \
     -jar citrus-mcp-server-${citrus.version}-runner.jar
```

The MCP endpoint will be available at `http://localhost:8080/mcp`.

54.13. Tools

The Citrus MCP server provides several tools that AI assistants can call to query Citrus metadata.

54.13.1. Catalog Tools

Test actions catalog

Lists available Citrus test actions and containers from the catalog.

Parameters: * `filter` (optional): Filter actions by name, title, or description (case-insensitive substring match) * `group` (optional): Filter by group (e.g., camel, kubernetes, selenium) * `version` (optional): Citrus version to query (defaults to the server version)

Returns: * Version information * Count of matching components * List of components with name, title, description, group, and module

Example Response:

```
{
  "version": "5.2.0-SNAPSHOT",
  "count": 3,
  "components": [
    {
      "name": "print",
      "fullName": "print",
      "version": "5.2.0-SNAPSHOT",
      "group": "core",
      "module": "citrus-base",
      "title": "Print",
      "description": "Prints messages to the console",
      "properties": []
    }
  ]
}
```

Test action details

Gets detailed information for a specific test action, including all properties.

Parameters: * `name` (required): Test action name (e.g., send, receive, print) * `version` (optional): Citrus version to query

Returns: Detailed component information including: * Component metadata (name, version, group, module) * Property list with name, description, type, required flag, and default value

Test action schema

Retrieves the complete JSON schema for a test action with all property definitions.

Parameters: * `name` (required): Test action name * `version` (optional): Citrus version to query

Returns: JSON schema string with complete property specifications

Endpoint catalog

Lists available Citrus endpoints.

Parameters: * `filter` (optional): Filter endpoints by name (case-insensitive substring match) * `group` (optional): Filter by group (e.g., http, kafka, jms) * `version` (optional): Citrus version to query

Returns: List of endpoint components with metadata and properties

Endpoint details

Gets detailed information for a specific endpoint.

Parameters: * `name` (required): Endpoint name (e.g., kafka, http, file, timer) * `version` (optional): Citrus version to query

Returns: Detailed endpoint information with all properties

Endpoint schema

Retrieves the complete JSON schema for an endpoint.

Parameters: * `name` (required): Endpoint name (e.g., kafka, http, direct) * `version` (optional): Citrus version to query

Returns: JSON schema string for the endpoint

54.13.2. Documentation Tools

Documentation Index

Lists all available Citrus documentation files.

Parameters: * `filter` (optional): Filter documentation by file name (case-insensitive substring match) * `version` (optional): Citrus version to query

Returns: List of documentation file names

Example Response:

```
[
  "actions-send.adoc",
  "actions-receive.adoc",
  "endpoints-http.adoc",
  "endpoints-kafka.adoc"
]
```

Documentation Page

Retrieves documentation content for a specific component.

Parameters: * `kind` (required): Component type (e.g., action, endpoint, function, validationMatcher) * `searchKey` (required): Search key (e.g., name of a test action or endpoint) * `version` (optional): Citrus version to query

Returns: * `fileName`: Name of the documentation file * `version`: Citrus version * `adoc`: AsciiDoc content

Example:

```
{
  "fileName": "actions-send.adoc",
  "version": "5.2.0-SNAPSHOT",
  "adoc": "[[actions-send]]\n= Send Action\n\nThe send action..."
}
```


54.14. Resources

Resources provide read-only access to schemas, specifications, and documentation.

54.14.1. YAML DSL Schema

URI: `citrus://schema/dsl/yaml`

Provides the JSON schema for the Citrus YAML DSL, enabling schema-aware test authoring.

MIME Type: `application/json`

54.14.2. XML DSL Schema

URI: `citrus://schema/dsl/xml`

Provides the XSD schema for the Citrus XML DSL.

MIME Type: `application/xml`

54.14.3. Test Action Schemas

URI Template: `citrus://schema/action/{name}`

Retrieves the JSON schema for a specific test action.

Parameters: * `name`: Test action name (e.g., print, send, receive)

MIME Type: `application/json`

Example URI: `citrus://schema/action/send`

54.14.4. Endpoint Schemas

URI Template: `citrus://schema/endpoint/{name}`

Retrieves the JSON schema for a specific endpoint.

Parameters: * `name`: Endpoint name (e.g., kafka, http, direct)

MIME Type: `application/json`

Example URI: `citrus://schema/endpoint/kafka`

54.14.5. Migration Guide

URI Template: `citrus://docs/migration-guide/5.1.0`

Retrieves the Citrus version migration guide for a given target version in a structured JSON format. The guide covers package renames, Maven artifact renames, dependency upgrades, API changes, and SPI changes to assist with automated migration from the previous major version.

Parameters: * `version`: Target migration version (e.g., 5.0)

MIME Type: `application/json`

Example URI: `citrus://docs/migration-guide/5.0`

Response structure:

```
{
  "targetVersion": "5.0",
  "title": "Citrus 4.x to 5.x",
  "description": "...",
  "artifactRenames": [
    { "groupId": "org.citrusframework", "oldArtifactId": "citrus-junit5",
"newArtifactId": "citrus-junit-jupiter" }
  ],
  "dependencyUpgrades": [
    { "dependency": "Spring Framework", "oldVersion": "6.x", "newVersion": "7.x" }
  ],
  "packageRenames": [
    { "oldPackage": "org.citrusframework.TestActionSupport", "newPackage":
"org.citrusframework.dsl.TestActionSupport", "module": "citrus-base" }
  ],
  "apiChanges": [
    { "type": "interface-extraction", "summary": "CitrusContext is now an interface",
"details": "..." }
  ],
  "spiChanges": [
    { "serviceFile": "META-INF/services/org.citrusframework.CitrusContext",
"implementation": "...", "changeType": "new" }
  ]
}
```

54.14.6. Settings Resources

The settings resources provide a catalog of all Citrus system property and environment variable settings. Each setting entry includes the system property key, environment variable name, default value, value type, and a description.

All Settings

URI: `citrus://settings/all`

Retrieves all available Citrus settings grouped by category.

MIME Type: `application/json`

Response structure:

```
{
  "core": {
    "name": "core",
    "title": "Core Settings",
    "description": "Core Citrus framework settings from CitrusSettings",
    "module": "citrus-api",
    "settings": [
      {
        "property": "citrus.default.message.type",
        "envVariable": "CITRUS_DEFAULT_MESSAGE_TYPE",
        "defaultValue": "XML",
        "type": "string",
        "description": "Default message type for validation"
      }
    ]
  },
  "logging": { ... },
  "camel": { ... }
}
```

Settings by Group

URI Template: `citrus://settings/group/{name}`

Retrieves settings for a specific group (e.g., core, logging, camel, kubernetes, testcontainers).

Parameters:

- `name`: Group name

MIME Type: `application/json`

Example URI: `citrus://settings/group/kubernetes`

Settings by Module

URI Template: `citrus://settings/module/{module}`

Retrieves settings for a specific Citrus module (e.g., citrus-api, citrus-camel, citrus-kubernetes, citrus-testcontainers). A module may contain multiple setting groups. For example, `citrus-camel` contains the groups `camel`, `camel-cli`, and `camel-infra`.

Parameters:

- `module`: Citrus module name

MIME Type: `application/json`

Example URI: `citrus://settings/module/citrus-camel`

54.15. Server Configuration

The Citrus MCP server uses Quarkus MCP so the server can be configured using Quarkus application properties.

54.15.1. Default Configuration

The default configuration (`application.properties`) includes:

```
quarkus.application.name=citrus-mcp-server
quarkus.application.version=${citrus.version}

# Disable banner for cleaner stdio output
quarkus.banner.enabled=false

# Logging configuration
quarkus.log.console.stderr=true
quarkus.log.level=WARN
quarkus.log.category."org.citrusframework".level=INFO
quarkus.log.category."io.quarkiverse.mcp".level=INFO

# Disable HTTP server for STDIO mode
quarkus.http.host-enabled=false
```

54.15.2. Custom Configuration

Override properties using system properties or environment variables:

```
java -Dquarkus.log.level=DEBUG \
     -Dquarkus.http.host-enabled=true \
     -Dquarkus.http.port=9090 \
     -jar citrus-mcp-server-${citrus.version}-runner.jar
```

54.16. Use Cases

54.16.1. AI-Assisted Test Generation

AI assistants can use the MCP server to:

1. **Discover available components**: Query catalog tools to find relevant test actions and endpoints
2. **Validate syntax**: Retrieve schemas to ensure correct property usage

3. **Access documentation**: Get detailed documentation for specific components
4. **Generate code**: Create syntactically correct Citrus tests based on user requirements

54.16.2. Schema Validation

Development environments can use the schema resources to:

- Provide autocomplete for YAML/XML test definitions
- Validate test files against official schemas
- Show inline documentation for properties

54.16.3. Documentation Lookup

Tools can query documentation resources to:

- Display contextual help for components
- Search documentation by component type and name

54.16.4. Version Migration

AI assistants can use the migration guide resource to:

- Read structured package rename mappings and apply them to import statements
- Identify Maven artifact renames and dependency upgrades required for a version upgrade
- Detect API changes (interface extractions, deleted classes) that need manual attention
- Update SPI registrations referencing moved classes
- Provide code examples from documentation = Spring Restdocs support

Spring Restdocs project helps to easily generate API documentation for RESTful services. While messages are exchanged the Restdocs library generates request/response snippets and API documentation. You can add the Spring Restdocs documentation to the Citrus client components for Http **and** SOAP endpoints.



The Spring Restdocs support components in Citrus are kept in a separate Maven module. If not already done, you have to include the module as Maven dependency to your project

```
<dependency>
  <groupId>org.citrusframework</groupId>
  <artifactId>citrus-restdocs</artifactId>
  <version>${citrus.version}</version>
</dependency>
```

54.17. Spring Restdocs using Http

First of all we concentrate on adding the Spring Restdocs feature to Http client communication. The next sample configuration uses the new Spring Restdocs components in Citrus:

Citrus Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@BindToRegistry
public HttpClient httpClient() {
    return new HttpClientBuilder()
                .requestUrl("http://localhost:11080/hello")
                .requestMethod(RequestMethod.POST)
                .contentType(MediaType.APPLICATION_XML_VALUE)
                .interceptors(Arrays.asList(restDocumentationConfigurer,
restDocumentationInterceptor))
                .build();
}

@BindToRegistry
public CitrusRestDocConfigurer restDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@BindToRegistry
public RestDocClientInterceptor restDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("rest-docs/{method-name}");
}
```

Spring Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
                .requestUrl("http://localhost:11080/hello")
                .requestMethod(RequestMethod.POST)
                .contentType(MediaType.APPLICATION_XML_VALUE)
                .interceptors(Arrays.asList(restDocumentationConfigurer,
restDocumentationInterceptor))
                .build();
}

@Bean
public CitrusRestDocConfigurer restDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@Bean
public RestDocClientInterceptor restDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("rest-docs/{method-name}");
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-restdocs="http://www.citrusframework.org/schema/restdocs/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/restdocs/config
       http://www.citrusframework.org/schema/restdocs/config/citrus-restdocs-
config.xsd">

    <citrus-restdocs:documentation id="restDocumentation"
                                output-directory="target/citrus-docs/generated-snippets"
                                identifier="rest-docs/{method-name}"/>

    <citrus-http:client id="httpClient"
          request-url="http://localhost:8080/test"
          request-method="POST"
          interceptors="restDocumentation"/>
</beans>
```



For easy configuration Citrus has created a separate namespace and schema definition for Spring XML Restdocs related documentation. Include this namespace into your Spring configuration in order to use the Citrus Restdocs configuration elements. The namespace URI and schema location are added to the Spring configuration XML file as seen in the example above.



The RestDocs configurer must be added to the list of Citrus test listeners. This is a mandatory step in order to enable the configurer for documentation preparations before each test. Otherwise, we would not be able to generate proper documentation. Usually, when binding the bean to the Citrus registry or the Spring bean application context or when using the Spring bean XML configuration this is done automatically for you.

The above component adds a new documentation configuration. Behind the scenes the component creates a new restdocs configurer and a client interceptor. We can reference the new restdocs component in **citrus-http** client components like see in the example above.

The Spring Restdocs documentation component acts as a client interceptor. Every time the client component is used to send and receive a message the restdocs interceptor will automatically create its API documentation. The configuration **identifier** attribute describes the output format `rest-docs/{method-name}` which results in a folder layout like this:

```
target/citrus-docs
  |- rest-docs
    |- test-a
      |- curl-request.adoc
      |- http-request.adoc
      |- http-response.adoc
    |- test-b
      |- curl-request.adoc
      |- http-request.adoc
      |- http-response.adoc
    |- test-c
      |- curl-request.adoc
      |- http-request.adoc
      |- http-response.adoc
```

The example above is the result of three test cases each of them performing a client Http request/response communication. Each test message exchange is documented with separate files.

Let's use the HttpClient with the documentation enabled in a test like this:

Java DSL

```
public class RestDocConfigurationIT extends TestNGCitrusSpringSupport implements
TestActionSupport {

    @Test
    @CitrusTest
    public void testRestDocs() {
        when(http().client(httpClient)
                .send()
                .post()
                .message()
                .body("""
                <testRequestMessage>
                    <text>Hello HttpServer>/text>
                </testRequestMessage>
                """)
                .header("Operation", "sayHello"));

        then(http().client(httpClient)
                .receive()
                .response(HttpStatus.OK)
                .message()
                .body("""
                <testResponseMessage>
                    <text>Hello Citrus!>/text>
                </testResponseMessage>
                """)
                .header("Operation", "sayHello"));
    }
}
```

The test produces the request.

http-request.adoc

```
POST /test HTTP/1.1
Accept: application/xml
CustomHeaderId: 123456789
Content-Type: application/xml;charset=UTF-8
Content-Length: 118
Accept-Charset: utf-8
Host: localhost
Operation: sayHello

<testRequestMessage>
    <text>Hello HttpServer>/text>
</testRequestMessage>
```

The **http-request.adoc** file represents the send message data for the client request. The respective

http-response.adoc represents the response that was sent to the client.

http-response.adoc

```
HTTP/1.1 200 OK
Date: Tue, 07 Jun 2016 12:10:46 GMT
Content-Type: application/xml;charset=UTF-8
Accept-Charset: utf-8
Content-Length: 122
Server: Jetty(9.2.15.v20160210)
Operation: sayHello

<testResponseMessage>
    <text>Hello Citrus!>/text>
</testResponseMessage>
```

Nice work! We have automatically created snippets for the RESTful API by just adding the interceptor to the Citrus client component. Spring Restdocs components can be combined manually. See the next configuration that uses this approach.

Citrus Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@BindToRegistry
public HttpClient httpClient() {
    return new HttpClientBuilder()
                .requestUrl("http://localhost:11080/hello")
                .requestMethod(RequestMethod.POST)
                .contentType(MediaType.APPLICATION_XML_VALUE)
                .interceptors(Arrays.asList(restDocumentationConfigurer,
restDocumentationInterceptor))
                .build();
}

@BindToRegistry
public CitrusRestDocConfigurer restDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@BindToRegistry
public RestDocClientInterceptor restDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("rest-docs/{method-name}");
}
```

Spring Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@Bean
public HttpClient httpClient() {
    return new HttpClientBuilder()
                .requestUrl("http://localhost:11080/hello")
                .requestMethod(RequestMethod.POST)
                .contentType(MediaType.APPLICATION_XML_VALUE)
                .interceptors(Arrays.asList(restDocumentationConfigurer,
restDocumentationInterceptor))
                .build();
}

@Bean
public CitrusRestDocConfigurer restDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@Bean
public RestDocClientInterceptor restDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("rest-docs/{method-name}");
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-restdocs="http://www.citrusframework.org/schema/restdocs/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/restdocs/config
       http://www.citrusframework.org/schema/restdocs/config/citrus-restdocs-
config.xsd">

    <citrus-restdocs:configurer id="restDocConfigurer" output-
directory="target/citrus-docs/generated-snippets"/>
    <citrus-restdocs:client-interceptor id="restDocClientInterceptor"
identifier="rest-docs/{method-name}"/>

    <util:list id="restDocInterceptors">
        <ref bean="restDocConfigurer"/>
        <ref bean="restDocClientInterceptor"/>
    </util:list>

    <citrus-http:client id="httpClient"
          request-url="http://localhost:8080/test"
          request-method="POST"
          interceptors="restDocInterceptors"/>
</beans>
```

What exactly is the difference to the **citrus-restdocs:documentation** that we have used before? In general there is no difference. Both configurations are identical in its outcome. Why should someone use the second approach then? It is more verbose as we need to also define a list of interceptors. The answer is easy. If you want to combine the restdocs interceptors with other client interceptors in a list then you should use the manual combination approach. We can add basic authentication interceptors for instance to the list of interceptors then. The more comfortable **citrus-restdocs:documentation** component only supports exclusive restdocs interceptors.

54.18. Spring Restdocs using SOAP

You can use the Spring Restdocs features also for SOAP clients in Citrus. This is a controversy idea as SOAP endpoints are different to RESTful concepts. But at the end SOAP Http communication is Http communication with request and response messages. Why should we miss out the fantastic documentation feature here just because of ideology reasons.

The concept of adding the Spring Restdocs documentation as interceptor to the client is still the same.

Citrus Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
                .defaultUri("http://localhost:8080/test")
                .interceptors(Arrays.asList(soapDocumentationConfigurer,
soapDocumentationInterceptor))
                .build();
}

@BindToRegistry
public CitrusRestDocConfigurer soapDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@BindToRegistry
public RestDocClientInterceptor soapDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("soap-docs/{method-name}");
}
```

Spring Bean

```
import org.citrusframework.restdocs.http.CitrusRestDocsSupport;

@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
                .defaultUri("http://localhost:8080/test")
                .interceptors(Arrays.asList(soapDocumentationConfigurer,
soapDocumentationInterceptor))
                .build();
}

@Bean
public CitrusRestDocConfigurer soapDocumentationConfigurer() {
    return CitrusRestDocsSupport.restDocsConfigurer(new
ManualRestDocumentation("target/citrus-docs/generated-snippets"));
}

@Bean
public RestDocClientInterceptor soapDocumentationInterceptor() {
    return CitrusRestDocsSupport.restDocsInterceptor("soap-docs/{method-name}");
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus-restdocs="http://www.citrusframework.org/schema/restdocs/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/restdocs/config
       http://www.citrusframework.org/schema/restdocs/config/citrus-restdocs-
config.xsd">

    <citrus-restdocs:documentation id="soapDocumentation"
                                type="soap"
                                output-directory="target/citrus-docs/generated-snippets"
                                identifier="soap-docs/{method-name}"/>

    <citrus-ws:client id="soapClient"
                  request-url="http://localhost:8080/test"
                  interceptors="soapDocumentation"/>
</beans>
```

We have added a **type** setting with value **soap** . And that is basically all we need to do. Now Citrus knows that we would like to add documentation for a SOAP client:

Following from that the **soapClient** is enabled to generate Spring Restdocs documentation for each request/response. The generated snippets then do represent the SOAP request and response messages.

http-request.adoc

```
POST /test HTTP/1.1
SOAPAction: "test"
Accept: application/xml
CustomHeaderId: 123456789
Content-Type: application/xml;charset=UTF-8
Content-Length: 529
Accept-Charset: utf-8
Host: localhost

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header>
    <Operation xmlns="http://citrusframework.org/test">sayHello>/Operation>
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <testRequestMessage>
      <text>Hello HttpServer>/text>
    </testRequestMessage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

http-response.adoc

```
HTTP/1.1 200 OK
Date: Tue, 07 Jun 2016 12:10:46 GMT
Content-Type: application/xml;charset=UTF-8
Accept-Charset: utf-8
Content-Length: 612
Server: Jetty(9.2.15.v20160210)

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header>
    >Operation xmlns="http://citrusframework.org/test">sayHello>/Operation>
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <testResponseMessage>
      <text>Hello Citrus!>/text>
    </testResponseMessage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The file names are still using **http-request** and **http-response** but the content is clearly the SOAP request/response message data.

Chapter 55. Tracing messages

As we deal with message based interfaces Citrus will send and receive a lot of messages during a test run. Now we may want to see these messages in chronological order as they were processed by Citrus. We can enable message tracing in Citrus in order to save messages to the file system for further investigations.

Citrus offers an easy way to debug all received messages to the file system. You need to enable some specific loggers and interceptors in the Spring application context.

```
<bean class="org.citrusframework.report.MessageTracingTestListener"/>
```

Just add this bean to the Spring configuration and Citrus will listen for sent and received messages for saving those to the file system. You will find files like these in the default test output folder after the test run:

For example:

```
logs/trace/messages/MyTest.msgs
logs/trace/messages/FooTest.msgs
logs/trace/messages/SomeTest.msgs
```

Each Citrus test writes a **.msgs** file containing all messages that went over the wire during the test. By default the debug directory is set to ***logs/trace/messages/*** relative to the project test output directory. But you can set your own output directory in the configuration

```
<bean class="org.citrusframework.report.MessageTracingTestListener">
  <property name="outputDirectory" value="file:/path/to/folder"/>
</bean>
```



As the file names do not change with each test run message tracing files may be overwritten. So you eventually need to save the generated message debug files before running another group of test cases.

Let's see some sample output for a test case with message communication over SOAP Http:

```
Sending SOAP request:
<?xml version="1.0" encoding="UTF-8"?><SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Header>
<Operation xmlns="http://citrusframework.org/test">sayHello</Operation>
</SOAP-ENV:Header>
<SOAP-ENV:Body>
<ns0:HelloRequest xmlns:ns0="http://citrusframework.org/schemas/samples/sayHello.xsd">
    <ns0:MessageId>0857041782</ns0:MessageId>
    <ns0:CorrelationId>6915071793</ns0:CorrelationId>
    <ns0:User>Christoph</ns0:User>
    <ns0:Text>Hello WebServer</ns0:Text>
</ns0:HelloRequest>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

=======================================================================

Received SOAP response:
<?xml version="1.0" encoding="UTF-8"?><SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Header/>
<SOAP-ENV:Body>
<ns0:HelloResponse
xmlns:ns0="http://citrusframework.org/schemas/samples/sayHello.xsd">
    <ns0:MessageId>0857041782</ns0:MessageId>
    <ns0:CorrelationId>6915071793</ns0:CorrelationId>
    <ns0:User>WebServer</ns0:User>
    <ns0:Text>Hello Christoph</ns0:Text>
</ns0:HelloResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

For this message tracing to work we need to add logging listeners to our sender and receiver components accordingly.

Citrus Bean

```
@BindToRegistry
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/services/ws/todolist")
            .interceptors(clientInterceptors())
            .build();
}

public List<ClientInterceptor> clientInterceptors() {
    return Arrays.asList(new LoggingClientInterceptor());
}
```

Spring Bean

```
@Bean
public WebServiceClient soapClient() {
    return new WebServiceClientBuilder()
            .defaultUri("http://localhost:8080/services/ws/todolist")
            .interceptors(clientInterceptors())
            .build();
}

public List<ClientInterceptor> clientInterceptors() {
    return Arrays.asList(new LoggingClientInterceptor());
}
```

Spring XML

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:citrus="http://www.citrusframework.org/schema/config"
       xmlns:citrus-ws="http://www.citrusframework.org/schema/ws/config"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.citrusframework.org/schema/config
       http://www.citrusframework.org/schema/config/citrus-config.xsd
       http://www.citrusframework.org/schema/ws/config
       http://www.citrusframework.org/schema/ws/config/citrus-ws-config.xsd">


    <citrus-ws:client id="soapClient"
                      request-
url="http://localhost:8080/services/ws/todolist[http://localhost:8071]"
                      interceptors="clientInterceptors"/>

    <util:list id="clientInterceptors">
        <bean class="org.citrusframework.ws.interceptor.LoggingClientInterceptor"/>
    </util:list>
</beans>
```



Be aware of adding the Spring **util** XML namespace to the application context when using the **util:list** construct.

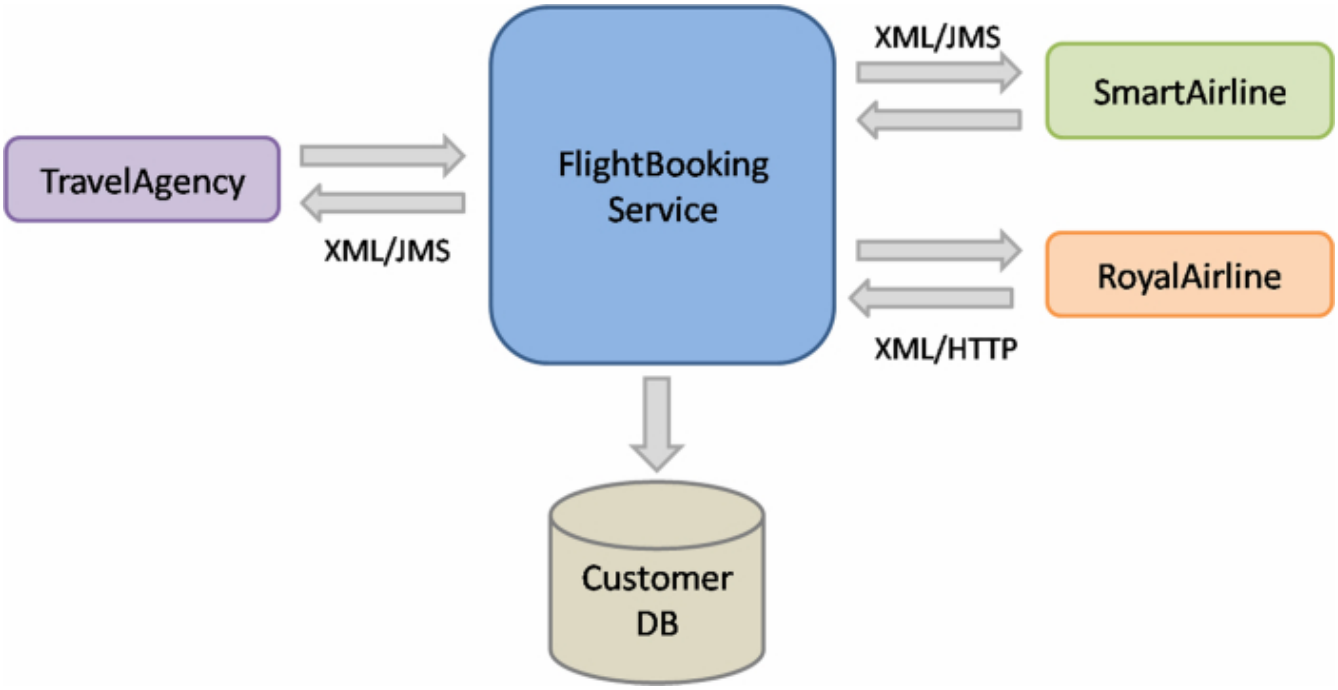
Chapter 56. Samples

This chapter gives some samples where you can see Citrus in action.

56.1. The FlightBooking sample

A simple project example should give you the idea how Citrus works. The system under test is a flight booking service that handles travel requests from a travel agency. A travel request consists of a complete travel route including several flights. The FlightBookingService application will split the complete travel booking into separate flight bookings that are sent to the respective airlines in charge. The booking and customer data is persisted in a database.

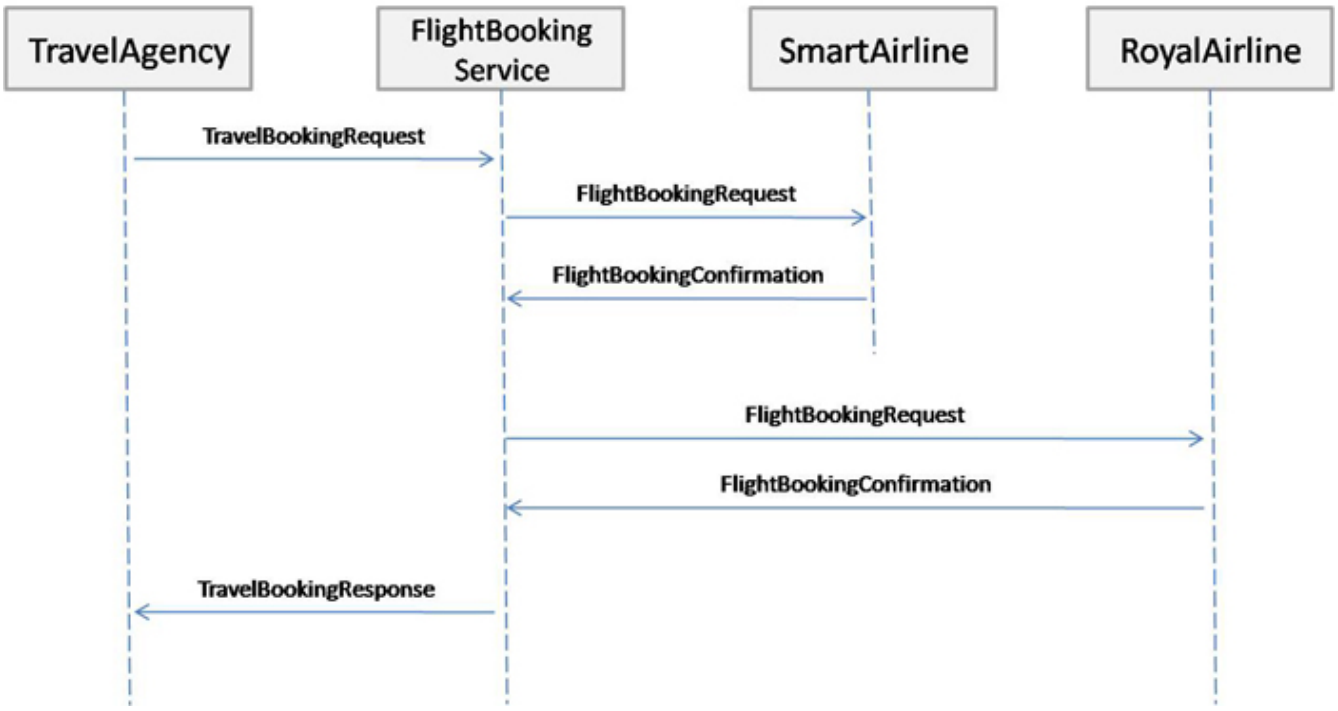
The airlines will confirm or deny the flight bookings. The FlightBookingService application consolidates all incoming flight confirmations and combines them to a complete travel confirmation or denial that is sent back to the travel agency. Next picture tries to put the architecture into graphics:



In our example two different airlines are connected to the FlightBookingService application: the SmartAriline over JMS and the RoyalAirline over Http.

56.1.1. The use case

The use case that we would like to test is quite simple. The test should handle a simple travel booking and expect a positive processing to the end. The test case neither simulates business errors nor technical problems. Next picture shows the use case as a sequence diagram.



The travel agency puts a travel booking request towards the system. The travel booking contains two separate flights. The flight requests are published to the airlines (SmartAirline and RoyalAirline). Both airlines confirm the flight bookings with a positive answer. The consolidated travel booking response is then sent back to the travel agency.

56.1.2. Configure the simulated systems

Citrus simulates all surrounding applications in their behavior during the test. The simulated applications are: TravelAgency, SmartAirline and RoyalAirline. The simulated systems have to be configured in the Citrus configuration first. The configuration is done in Spring XML configuration files, as Citrus uses Spring to glue all its services together.

First of all we have a look at the TravelAgency configuration. The TravelAgency is using JMS to connect to our tested system, so we need to configure this JMS connection in Citrus.

```
<bean id="connectionFactory"
class="org.apache.activemq.artemis.jms.client.ActiveMQConnectionFactory">
  <property name="brokerURL" value="tcp://localhost:61616"/>
</bean>

<citrus-jms:endpoint id="travelAgencyBookingRequestEndpoint"
                     destination-name="${travel.agency.request.queue}"/>

<citrus-jms:endpoint id="travelAgencyBookingResponseEndpoint"
                     destination-name="${travel.agency.response.queue}"/>
```

This is all Citrus needs to send and receive messages over JMS in order to simulate the TravelAgency. By default all JMS message senders and receivers need a connection factory. Therefore Citrus is searching for a bean named "connectionFactory". In the example we connect to a ActiveMQ message broker. A connection to other JMS brokers like TIBCO EMS or Apache

ActiveMQ is possible too by simply changing the connection factory implementation.

The identifiers of the message senders and receivers are very important. We should think of suitable ids that give the reader a first hint what the sender/receiver is used for. As we want to simulate the TravelAgency in combination with sending booking requests our id is "travelAgencyBookingRequestEndpoint" for example.

The sender and receivers do also need a JMS destination. Here the destination names are provided by property expressions. The Spring IoC container resolves the properties for us. All we need to do is publish the property file to the Spring container like this.

```
<bean name="propertyLoader"
   class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="locations">
        <list>
            <value>citrus.properties</value>
        </list>
    </property>
    <property name="ignoreUnresolvablePlaceholders" value="true"/>
</bean>
```

The citrus.properties file is located in our project's resources folder and defines the actual queue names besides other properties of course:

```
#JMS queues
travel.agency.request.queue=Travel.Agency.Request.Queue
travel.agency.response.queue=Travel.Agency.Response.Queue
smart.airline.request.queue=Smart.Airline.Request.Queue
smart.airline.response.queue=Smart.Airline.Response.Queue
royal.airline.request.queue=Royal.Airline.Request.Queue
```

What else do we need in our Spring configuration? There are some basic beans that are commonly defined in a Citrus application but I do not want to bore you with these details. So if you want to have a look at the Spring application context file in the resources folder and see how things are defined there.

We continue with the first airline to be configured the SmartAirline. The SmartAirline is also using JMS to communicate with the FlightBookingService. So there is nothing new for us, we simply define additional JMS message senders and receivers.

```
<citrus-jms:endpoint id="smartAirlineBookingRequestEndpoint"
                     destination-name="${smart.airline.request.queue}"/>

<citrus-jms:endpoint id="smartAirlineBookingResponseEndpoint"
                     destination-name="${smart.airline.response.queue}"/>
```

We do not define a new JMS connection factory because TravelAgency and SmartAirline are using

the same message broker instance. In case you need to handle multiple connection factories simply define the connection factory with the attribute "connection-factory".

```
<citrus-jms:endpoint id="smartAirlineBookingRequestEndpoint"
                             destination-name="${smart.airline.request.queue}"
                             connection-factory="smartAirlineConnectionFactory"/>

<citrus-jms:endpoint id="smartAirlineBookingResponseEndpoint"
                            destination-name="${smart.airline.response.queue}"
                            connection-factory="smartAirlineConnectionFactory"/>
```

56.1.3. Configure the Http adapter

The RoyalAirline is connected to our system using Http request/response communication. This means we have to simulate a Http server in the test that accepts client requests and provides proper responses. Citrus offers a Http server implementation that will listen on a port for client requests. The adapter forwards incoming request to the test engine over JMS and receives a proper response that is forwarded as a Http response to the client. The next picture shows this mechanism in detail.



The RoyalAirline adapter receives client requests over Http and sends them over JMS to a message receiver as we already know it. The test engine validates the received request and provides a proper response back to the adapter. The adapter will transform the response to Http again and publishes it to the calling client. Citrus offers these kind of adapters for Http and SOAP communication. By writing your own adapters like this you will be able to extend Citrus so it works with protocols that are not supported yet.

Let us define the Http adapter in the Spring configuration:

```
<citrus-http:server id="royalAirlineHttpServer"
                       port="8091"
                       uri="/flightbooking"
                       endpoint-adapter="jmsEndpointAdapter"/>

<citrus-jms:endpoint-adapter id="jmsEndpointAdapter
      destination-name="${royal.airline.request.queue}"/>
      connection-factory="connectionFactory" />
      timeout="2000"/>

<citrus-jms:sync-endpoint id="royalAirlineBookingEndpoint"
                             destination-name="${royal.airline.request.queue}"/>
```

We need to configure a Http server instance with a port, a request URI and the endpoint adapter. We define the JMS endpoint adapter to handle request as described. In Addition to the endpoint adapter we also need synchronous JMS message sender and receiver instances. That's it! We are able to receive Http request in order to simulate the RoyalAirline application. What is missing now? The test case definition itself.

56.1.4. The test case

The test case definition is also a Spring configuration file. Citrus offers a customized XML syntax to define a test case. This XML test defining language is supposed to be easy to understand and more specific to the domain we are dealing with. Next listing shows the whole test case definition. Keep in mind that a test case defines every step in the use case. So we define sending and receiving actions of the use case as described in the sequence diagram we saw earlier.

```
<?xml version="1.0" encoding="UTF-8"?>
<spring:beans xmlns="http://www.citrusframework.org/schema/testcase"
              xmlns:spring="http://www.springframework.org/schema/beans"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:schemaLocation="http://www.springframework.org/schema/beans
                  http://www.springframework.org/schema/beans/spring-beans.xsd
                  http://www.citrusframework.org/schema/testcase
                  http://www.citrusframework.org/schema/testcase/citrus-testcase.xsd">
    <testcase name="FlightBookingTest">
        <meta-info>
            <author>Christoph Deppisch</author>
            <creationdate>2009-04-15</creationdate>
            <status>FINAL</status>
            <last-updated-by>Christoph Deppisch</last-updated-by>
            <last-updated-on>2009-04-15T00:00:00</last-updated-on>
        </meta-info>
        <description>
            Test flight booking service.
        </description>
        <variables>
            <variable name="correlationId"
                value="citrus:concat('Lx1x', 'citrus:randomNumber(10)')"/>
            <variable name="customerId"
                value="citrus:concat('Mx1x', citrus:randomNumber(10))"/>
        </variables>
        <actions>
            <send endpoint="travelAgencyBookingRequestEndpoint">
                <message>
                    <data>
                      <![CDATA[
                        <TravelBookingRequestMessage
                          xmlns="http://citrusframework.org/schemas/TravelAgency">
                          <correlationId>${correlationId}</correlationId>
                          <customer>
                            <id>${customerId}</id>
                            <firstname>John</firstname>
```

```
                    <lastname>Doe</lastname>
                  </customer>
                  <flights>
                    <flight>
                      <flightId>SM 1269</flightId>
                      <airline>SmartAirline</airline>
                      <fromAirport>MUC</fromAirport>
                      <toAirport>FRA</toAirport>
                      <date>2009-04-15</date>
                      <scheduledDeparture>11:55:00</scheduledDeparture>
                      <scheduledArrival>13:00:00</scheduledArrival>
                    </flight>
                    <flight>
                      <flightId>RA 1780</flightId>
                      <airline>RoyalAirline</airline>
                      <fromAirport>FRA</fromAirport>
                      <toAirport>HAM</toAirport>
                      <date>2009-04-15</date>
                      <scheduledDeparture>16:00:00</scheduledDeparture>
                      <scheduledArrival>17:10:00</scheduledArrival>
                    </flight>
                  </flights>
                </TravelBookingRequestMessage>
              ]]>
            </data>
        </message>
        <header>
            <element name="correlationId" value="${correlationId}"/>
        </header>
    </send>

    <receive endpoint="smartAirlineBookingRequestEndpoint">
        <message>
            <data>
              <![CDATA[
                <FlightBookingRequestMessage
                  xmlns="http://citrusframework.org/schemas/AirlineSchema">
                  <correlationId>${correlationId}</correlationId>
                  <bookingId>???</bookingId>
                  <customer>
                    <id>${customerId}</id>
                    <firstname>John</firstname>
                    <lastname>Doe</lastname>
                  </customer>
                  <flight>
                    <flightId>SM 1269</flightId>
                    <airline>SmartAirline</airline>
                    <fromAirport>MUC</fromAirport>
                    <toAirport>FRA</toAirport>
                    <date>2009-04-15</date>
                    <scheduledDeparture>11:55:00</scheduledDeparture>
```

```
                        <scheduledArrival>13:00:00</scheduledArrival>
                      </flight>
                    </FlightBookingRequestMessage>
                  ]]>
                </data>
                <ignore path="//:FlightBookingRequestMessage/:bookingId"/>
            </message>
            <header>
                <element name="correlationId" value="${correlationId}"/>
            </header>
            <extract>
                <message path="//:FlightBookingRequestMessage/:bookingId"
                            variable="${smartAirlineBookingId}"/>
            </extract>
        </receive>

        <send endpoint="smartAirlineBookingResponseEndpoint">
            <message>
                <data>
                  <![CDATA[
                    <FlightBookingConfirmationMessage
                      xmlns="http://citrusframework.org/schemas/AirlineSchema">
                      <correlationId>${correlationId}</correlationId>
                      <bookingId>${smartAirlineBookingId}</bookingId>
                      <success>true</success>
                      <flight>
                        <flightId>SM 1269</flightId>
                        <airline>SmartAirline</airline>
                        <fromAirport>MUC</fromAirport>
                        <toAirport>FRA</toAirport>
                        <date>2009-04-15</date>
                        <scheduledDeparture>11:55:00</scheduledDeparture>
                        <scheduledArrival>13:00:00</scheduledArrival>
                      </flight>
                    </FlightBookingConfirmationMessage>
                  ]]>
                </data>
            </message>
            <header>
                <element name="correlationId" value="${correlationId}"/>
            </header>
        </send>

        <receive endpoint="royalAirlineBookingEndpoint">
            <message>
                <data>
                  <![CDATA[
                    <FlightBookingRequestMessage

xmlns="http://citrusframework.org/schemas/FlightBooking/AirlineSchema">
                      <correlationId>${correlationId}</correlationId>
```

```
                    <bookingId>???</bookingId>
                    <customer>
                        <id>${customerId}</id>
                        <firstname>John</firstname>
                        <lastname>Doe</lastname>
                    </customer>
                    <flight>
                      <flightId>RA 1780</flightId>
                      <airline>RoyalAirline</airline>
                      <fromAirport>FRA</fromAirport>
                      <toAirport>HAM</toAirport>
                      <date>2009-04-15</date>
                      <scheduledDeparture>16:00:00</scheduledDeparture>
                      <scheduledArrival>17:10:00</scheduledArrival>
                    </flight>
                  </FlightBookingRequestMessage>
                ]]>
            </data>
            <ignore path="//:FlightBookingRequestMessage/:bookingId"/>
        </message>
        <header>
            <element name="correlationId" value="${correlationId}"/>
        </header>
        <extract>
            <message path="//:FlightBookingRequestMessage/:bookingId"
                        variable="${royalAirlineBookingId}"/>
        </extract>
    </receive>

    <send endpoint="royalAirlineBookingEndpoint">
        <message>
            <data>
              <![CDATA[
                <FlightBookingConfirmationMessage
                  xmlns="http://citrusframework.org/schemas/AirlineSchema">
                  <correlationId>${correlationId}</correlationId>
                  <bookingId>${royalAirlineBookingId}</bookingId>
                  <success>true</success>
                  <flight>
                    <flightId>RA 1780</flightId>
                    <airline>RoyalAirline</airline>
                    <fromAirport>FRA</fromAirport>
                    <toAirport>HAM</toAirport>
                    <date>2009-04-15</date>
                    <scheduledDeparture>16:00:00</scheduledDeparture>
                    <scheduledArrival>17:10:00</scheduledArrival>
                  </flight>
                </FlightBookingConfirmationMessage>
              ]]>
            </data>
        </message>
```

```
                <header>
                    <element name="correlationid" value="${correlationId}"/>
                </header>
            </send>

            <receive endpoint="travelAgencyBookingResponseEndpoint">
                <message>
                    <data>
                      <![CDATA[
                        <TravelBookingResponseMessage
                          xmlns="http://citrusframework.org/schemas/TravelAgency">
                          <correlationId>${correlationId}</correlationId>
                          <success>true</success>
                          <flights>
                            <flight>
                              <flightId>SM 1269</flightId>
                              <airline>SmartAirline</airline>
                              <fromAirport>MUC</fromAirport>
                              <toAirport>FRA</toAirport>
                              <date>2009-04-15</date>
                              <scheduledDeparture>11:55:00</scheduledDeparture>
                              <scheduledArrival>13:00:00</scheduledArrival>
                            </flight>
                            <flight>
                              <flightId>RA 1780</flightId>
                              <airline>RoyalAirline</airline>
                              <fromAirport>FRA</fromAirport>
                              <toAirport>HAM</toAirport>
                              <date>2009-04-15</date>
                              <scheduledDeparture>16:00:00</scheduledDeparture>
                              <scheduledArrival>17:10:00</scheduledArrival>
                            </flight>
                          </flights>
                        </TravelBookingResponseMessage>
                      ]]>
                    </data>
                </message>
                <header>
                    <element name="correlationId" value="${correlationId}"/>
                </header>
            </receive>

        </actions>
    </testcase>
</spring:beans>
```

Similar to a sequence diagram the test case describes every step of the use case. At the very beginning the test case gets name and its meta information. Following with the variable values that are used all over the test. Here it is the correlationId and the customerId that are used as test variables. Inside message templates header values the variables are referenced several times in the

test

```
<correlationId>${correlationId}</correlationId>
<id>${customerId}</id>
```

The sending/receiving actions use a previously defined message sender/receiver. This is the link between test case and basic Spring configuration we have done before.

```
send endpoint="travelAgencyBookingRequestEndpoint"
```

The sending action chooses a message sender to actually send the message using a message transport (JMS, Http, SOAP, etc.). After sending this first "TravelBookingRequestMessage" request the test case expects the first "FlightBookingRequestMessage" message on the SmartAirline JMS destination. In case this message is not arriving in time the test will fail with errors. In positive case our FlightBookingService works well and the message arrives in time. The received message is validated against a defined expected message template. Only in case all content validation steps are successful the test continues with the action chain. And so the test case proceeds and works through the use case until every message is sent respectively received and validated. The use case is done automatically without human interaction. Citrus simulates all surrounding applications and provides detailed validation possibilities of messages.

Chapter 57. Appendix

Maven archetype

If you start from scratch or in case you would like to have Citrus operating in a separate Maven module you can use the Citrus Maven archetype to create a new Maven project. The archetype will setup a basic Citrus project structure with basic settings and files.

```
mvn archetype:generate -Dfilter=org.citrusframework.mvn:citrus

1: remote -> org.citrusframework.mvn:citrus-quickstart (Citrus quickstart project)
2: remote -> org.citrusframework.mvn:citrus-quickstart-jms (Citrus quickstart project
with JMS consumer and producer)
3: remote -> org.citrusframework.mvn:citrus-quickstart-soap (Citrus quickstart project
with SOAP client and producer)
Choose a number: 1

Define value for groupId: org.citrusframework.samples
Define value for artifactId: citrus-sample
Define value for version: 1.0-SNAPSHOT
Define value for package: org.citrusframework.samples
```

In the sample above we used the Citrus archetype available in Maven central repository. As the list of default archetypes available in Maven central is very long, it has been filtered for official Citrus archetypes.

After choosing the Citrus quickstart archetype you have to define several values for your project: the groupId, the artifactId, the package and the project version. After that we are done! Maven created a Citrus project structure for us which is ready for testing. You should see the following basic project folder structure.

```
citrus-sample
  |   + src
  |   |   + main
  |   |    |   + java
  |   |    |   + resources
  |   |   + test
  |   |    |   + java
  |   |    |   + resources
  pom.xml
```

The Citrus project is absolutely ready for testing. With Maven we can build, package, install and test our project right away without any adjustments. Try to execute the following commands:

```
mvn clean verify
mvn clean verify -Dit.test=MyFirstCitrusTest
```



If you need additional assistance in setting up a Citrus Maven project please visit our Maven setup tutorial on https://citrusframework.org/tutorials.html.